BLADUS (formerly DARNA)

Backend Discovery & Blueprint

Django · PostgreSQL + PostGIS · Algeria

Analysis and documentation only. Build date 2026-11-09.
Legal, tax and financial points are research, not advice: VERIFY WITH A PROFESSIONAL.

Contents

Start here

1. 00. Start here: BLADUS backend blueprint
2. Explain it like I'm new
3. Glossary

Phase 1 and 2: discovery and research

1. 01. System inventory: what BLADUS does today
2. 02. Research: what others do, which Django tools to use, and what Algeria requires

Phase 3: requirements and rules

1. 03. Requirements: what the backend must do
2. 04. Business rules

Phase 4: diagrams

1. Diagrams
2. Diagram gallery

Phase 5: database

1. 05. Database schema
2. 05b. Data dictionary

Phase 6: API

1. 06. API design: how the apps talk to the backend
2. API reference

Phase 7: architecture and operations

1. 07. Architecture: how the Django backend is organised
2. 08. Security and privacy
3. 09. DevOps and hosting: where BLADUS runs and how we operate it
4. 10. Testing and quality

Phase 8: plan and decisions

1. 11. Roadmap: how we build the BLADUS backend
2. 12. Risks: what could go wrong, and what we do about it
3. 13. Decisions needed: who must decide what, and by when
4. 14. Open questions (running list)
5. Traceability matrix: requirement to rule, table, API, test and milestone
6. 15. Readiness verdict and first build prompt

Appendices (raw working notes)

1. BLADUS frontend inventory (raw analysis notes for the backend blueprint)
2. BLADUS admin back-office: raw inventory for the backend blueprint
3. BLADUS: spec vs code (raw analysis for the backend blueprint)
4. Click-through observations (website, dev server, 2026-11-04)
5. BLADUS backend research: comparable platforms and Django ecosystem
6. BLADUS backend research: Algeria context, payments, compliance, infra, security
7. Backend blueprint changelog

00. Start here: BLADUS backend blueprint

> **In plain words**
> BLADUS has a finished-looking website and admin, but no real backend: data lives in memory and the two apps do not share it.
> This folder is the complete plan for that backend, built with Django and PostgreSQL, designed for Algeria.
> It was written before any backend code: what exists, what to build, the rules, the database, the API, security, hosting, tests and the plan.
> Verdict: **ready to start the first milestone now**; real customer data waits for two legal answers on hosting.
> New to these topics? Read explain-like-im-new.md first. Everything in one file: `dist/BLADUS-backend-blueprint.pdf`.

Terms used here (all in glossary.md): **backend** (the server program that stores data and applies rules), **API** (the list of addresses the apps call), **schema** (the design of the database tables), **milestone** (a 3 to 5 week block of work).

Prepared 2026-11-04 to 2026-11-09. Documentation only: no backend code exists yet.

---

1. The plan in ten lines

1. **Stack**: Django 6.1, Django REST Framework, PostgreSQL with PostGIS, Redis and Celery, django-allauth (email and phone codes, two-factor login). The two Next.js apps stay and call the API through their server (02-research.md B, 07-architecture.md).
2. **Money (DEC-001)**: guests pay hosts directly; hosts keep a prepaid deposit; BLADUS deducts its commission (12% individual, 15% agency, 18% hotel, 10% hostel) on each confirmed booking. Guests pay no BLADUS fee. Everything is recorded in a double-entry ledger.
3. **No double booking**: the database itself refuses overlapping stays and over-selling of rooms, beds and seats (05-database-schema.md).
4. **Who publishes what (DEC-014)**: solo hosts publish homes and vehicles, agencies publish experiences and trips, hotels and hostels publish their property.
5. **Types (DEC-015)**: 8 experience types and 8 trip types, one shared list.
6. **Moderation (DEC-003)**: new listings and trips are reviewed by staff before going live.
7. **Algeria**: hosting in Algeria and a `.com.dz` domain are likely required; personal data should not leave the country without authorisation (VERIFY WITH A PROFESSIONAL).
8. **Size**: 147 requirements (126 in the first release), 77 tables, 266 API operations.
9. **Plan**: 8 milestones (M0 to M7, about 29 weeks plus a 2-week buffer) for a team of 2 backend developers and 1 part-time frontend developer, then R2 (11-roadmap.md).
10. **Open**: 48 decisions remain, most of them for the lawyer and accountant (13-decisions-needed.md).

2. Reading order

#	Document	Read it to learn	Time
0	explain-like-im-new.md	The whole system as a simple story	10 min
1	01-system-inventory.md	What the website and admin do today, and what is broken or missing	25 min
2	02-research.md	Other platforms, Django tool choices, Algerian payments, law, geography, hosting, security	30 min
3	03-requirements.md + requirements.csv	Every requirement, the 15 product decisions	30 min
4	04-business-rules.md	Exact rules: prices, refunds, statuses, money postings	30 min
5	diagrams/README.md	29 diagrams with captions and legends	15 min
6	05-database-schema.md + 05b-data-dictionary.md + schema.dbml / schema.sql	The database, design decisions, constraints, query plan	40 min
7	06-api-design.md + openapi.yaml	API conventions and every endpoint	40 min
8	07-architecture.md	Django apps, jobs, transactions, how the apps switch from demo data	30 min
9	08-security-privacy.md	Threats, login, permissions, privacy under Algerian law	30 min
10	09-devops-hosting.md	Environments, hosting in Algeria, CI/CD, backups, monitoring, costs	25 min
11	10-testing-quality.md	Test strategy and quality gates	20 min
12	11-roadmap.md	Milestones, dates, gates	20 min
13	12-risks.md	32 risks with mitigations	15 min
14	13-decisions-needed.md + 14-open-questions.md	What is still to decide, by whom, by when	20 min
15	traceability.md + traceability.csv	Each requirement linked to rules, tables, endpoints, tests, milestone	reference
16	15-readiness-and-first-build-prompt.md	The verdict and the prompt to start building	10 min
	glossary.md	Every term explained	reference

3. Deliverables checklist

Asked for	Delivered
Phase 1: inventory (features per actor, data, `lib/data` bridge, rules, gaps, click-through)	01-system-inventory.md with appendices in `_work/`
Phase 2: research A to D with URLs, dates, decisions, verify flags	02-research.md (full source lists in `_work/research-*.md`)
Phase 3: requirements, CSV, business rules	03-requirements.md, requirements.csv, 04-business-rules.md
Phase 4: rendered diagrams with captions and legends	`diagrams/*.mmd`, `diagrams/rendered/` (SVG and PNG), diagrams/README.md

Asked for	Delivered
Phase 5: schema, data dictionary, DBML, ADRs, seed plan, mapping, query plan	05-database-schema.md, 05b-data-dictionary.md, schema.dbml, schema.sql
Phase 6: API design and OpenAPI	06-api-design.md, openapi.yaml (lint: 0 errors, 0 warnings)
Phase 7: architecture, security and privacy, DevOps and hosting, testing	07, 08, 09, 10
Phase 8: roadmap, risks, decisions, open questions	11, 12, 13, 14
Plain-language layer	this page, explain-like-im-new.md, glossary.md, README.md, an "In plain words" box and "Check yourself" in every document
Traceability matrix	traceability.md, traceability.csv
Rendered HTML package and one PDF with everything	`dist/site/index.html`, `dist/BLADUS-backend-blueprint.pdf`
Change log and question list	CHANGELOG.md, 14-open-questions.md
Readiness verdict and first build prompt	15-readiness-and-first-build-prompt.md

4. Verdict

GO for milestone M0 (foundations) now, once five small technical and product choices are confirmed in the first week (T2, T3, T4, T9, Q22). **Not yet GO for real customer data**: the lawyer must answer L1 and L2 (hosting in Algeria, data transfer) before production hosting is signed at the end of M5. Collecting guests' money stays out of the first release until a payment licence question (L3) is settled. Details and the first build prompt: 15-readiness-and-first-build-prompt.md.

Check yourself

- [ ] In one sentence, what is missing from BLADUS today?
- [ ] Who pays BLADUS, and how?
- [ ] Which document do you open to find the endpoint for creating a booking?
- [ ] Which milestone can start now, and which two legal answers block real customer data?
- [ ] Where is the single PDF with everything?

Explain it like I'm new

> **In plain words**
> This page explains the whole BLADUS backend plan with no jargon, as a story.
> Read it first if words like "API", "database" or "ledger" feel unclear.
> It follows one guest, one host and one staff member through the system.
> Every other document goes deeper into one part of this story.
> Words in **bold** are explained in glossary.md.

---

1. What we have today, and what is missing

BLADUS already has two websites:

- the **website** where travellers search and book, and where hosts manage their places (the `frontend` folder);
- the **admin** where BLADUS staff check hosts, listings, bookings and money (the `admin` folder).

Both look finished, but behind the screens there is no real storage. Every booking, every host, every message lives in the computer's memory and disappears when the server restarts. The two websites also do not talk to each other: a booking made on the website never appears in the admin.

The missing piece is the **backend**: one program, running on a server, that keeps all the data safely in a **database** and is the single source of truth for both websites. We will build it with **Django** (a Python framework the owner already knows).

2. The main idea in one picture



- The websites stay as they are. Only the inside of their "data" functions changes: instead of reading memory, they ask the API.
- The **API** is a list of addresses like `POST /api/v1/bookings` ("create a booking"). The full list is in 06-api-design.md and openapi.yaml.
- The **database** is PostgreSQL. It has 77 tables (like 77 spreadsheet tabs) described in 05-database-schema.md.

3. A guest books a villa

Yacine wants 3 nights in a villa in Tipaza.

1. **Search.** The website asks the API for villas in Tipaza for his dates. The API looks in the database and only returns villas whose nights are free.
2. **Price.** He opens the villa. The API calculates the price itself (3 nights x 18,500 DZD + 4,000 cleaning = 59,500 DZD). Guests pay **no BLADUS fee**.
3. **Book.** He clicks Reserve. The API saves the booking. The database has a built-in rule that refuses two bookings on the same night for the same villa, so even if two people click at the same second, only one wins.
4. **Pay.** At launch Yacine pays the host directly (cash or transfer). BLADUS does not touch the guest's money. (This is decision **DEC-001**; the reason is Algerian payment law, see 02-research.md section C.1.)
5. **Messages.** He and the host chat in the app. Phone numbers are hidden until the booking is confirmed, to stop scams.

4. How BLADUS earns money

BLADUS takes a **commission** from the host, not from the guest:

Host type	Commission
Individual	12%
Agency	15%
Hotel	18%
Hostel	10%

Each host keeps a **deposit** with BLADUS (like a prepaid phone card). When a booking is confirmed, the commission is taken from that deposit. For Yacine's villa: 12% of 55,500 = 6,660 DZD.

Every money movement is written in a **ledger**: a notebook where lines are only added, never erased. If there is a mistake, a correcting line is added. This makes it easy to prove where every dinar went.

5. A host joins

Amina wants to rent her home.

1. She signs up and verifies her phone with a code by SMS.
2. She fills the "become a host" steps. Her answers are saved on the server, so she can continue on another device.
3. She uploads her ID and proof of ownership. These files go to private storage that only staff can open, and every opening is recorded.

4. A moderator checks the documents and approves her.
5. She creates a listing. It is checked by staff before it appears in search (**DEC-003**).

What each host type can publish is fixed (**DEC-014**): individuals publish homes and vehicles, agencies publish experiences and trips, hotels and hostels publish their property with its rooms.

---

6. Staff work in the admin

Nadia, a moderator, opens the admin. She sees queues: hosts to verify, listings to review, reported reviews. Each item shows how long it has been waiting. When she approves or refuses, the reason is saved and the host is notified. Every action she takes is written in an **audit log** that nobody can edit.

Finance staff record host deposits (for example a transfer to BLADUS's CCP account). Above 500,000 DZD a second person must approve, to prevent mistakes and fraud.

---

7. Things that happen by themselves

Some work runs in the background on a schedule (**jobs**):

- a request that the host does not answer within 24 hours expires and both sides are told;
- after check-out, the booking becomes "completed" and the review window opens;
- SMS and emails are sent after a change is saved, and retried if they fail.

---

8. Why Algeria matters in the design

- **Hosting**: Algerian law likely requires an e-commerce site hosted in Algeria with a `.com.dz` name, and personal data should not leave the country without permission. So the plan puts the servers in Algeria (VERIFY WITH A PROFESSIONAL).
- **Payments**: collecting guests' money and paying hosts may need a payment licence, which is why we start with the deposit model.
- **Geography**: 69 wilayas and 1,541 communes, in Arabic, French and English.
- **Formats**: weekend is Friday and Saturday, Latin digits even in Arabic, phone numbers start with +213 5, 6 or 7.

---

9. Where to read next

If you want to know...	Read
What exists today and what is broken	01-system-inventory.md
What other platforms do and which tools we chose	02-research.md
The full list of what the backend must do	03-requirements.md
The exact rules (prices, refunds, statuses)	04-business-rules.md
The database	05-database-schema.md
The API	06-api-design.md
How it is built, secured, hosted and tested	07, 08, 09, 10
The plan, risks and open decisions	11, 12, 13, 14

Check yourself

- [ ] Why does a booking made today on the website not appear in the admin?
- [ ] Who pays BLADUS, the guest or the host? How much for an agency?
- [ ] What stops two people from booking the same night?
- [ ] Why does BLADUS not collect guests' money at launch?
- [ ] What is a ledger, and why are its lines never erased?
- [ ] Name two things that happen by themselves without anyone clicking.

Glossary

> **In plain words**
> Every technical or business word used in the blueprint, explained in one or two sentences.
> If a document uses a word you do not know, look here first. The list grows with each phase.

Term	Meaning
`ActionResult`	What a Next.js server action returns today: success with data, or an error code.
`lib/data` seam	The folder of data functions both apps call; each becomes an API call.
ADR	Architecture Decision Record: a short note saying what was decided, why, and what else was considered.
Allauth (headless)	A Django library for sign-up, login, phone codes and two-factor login, used through an API.
ANPDP	Algeria's personal data protection authority (Autorité nationale de protection des données à caractère personnel).
API	The set of web addresses (endpoints) the apps call to read and change data.
Audit log	A list of who did what and when; never edited, only added to.
BFF	Backend for frontend: the Next.js server keeps the login cookie and calls Django on the browser's behalf.
Booking / Reservation	One guest's purchase of a unit for a time. The admin calls it Booking, the website Reservation.
Celery	A Python tool that runs background jobs (expire requests, send SMS) outside the web request.
Centimes	Hundredths of a dinar. We store money as whole centimes to avoid rounding errors.
CI/CD	Continuous integration and delivery: every code change is automatically checked (tests, lint) and, if green, deployed to staging.
CIB / Edahabia	Algeria's bank card and postal card, accepted online through SATIM.
Commission	The percentage BLADUS earns on a booking, paid by the host. Guests pay no BLADUS fee.
Cursor (pagination)	A bookmark returned by the API (for example the last message id) so the next request asks only for what came after it.
Double-entry ledger	Money records where each movement has lines that add up to zero, so money is never created or lost by mistake.
DPIA	Data protection impact assessment: a written analysis of the risks of a sensitive processing (here: host ID documents).
DRF	Django REST Framework, the library that turns Django into an API.
Endpoint	One API address with one method, for example `POST /api/v1/bookings` (create a booking).
ETag / If-Match	A version tag returned with a record; sending it back with a change makes the API refuse the change if someone else edited the record meanwhile.
Exclusion constraint	A PostgreSQL rule that refuses overlapping date ranges for the same unit.
Feature flag	A setting that turns a feature on or off without changing code (for example the dormant payment endpoints).
Gantt chart	A timeline chart showing each milestone as a bar between its start and end dates.
Gate	A decision or legal answer that must exist before a milestone can finish.
Half-open range	A date range that includes the first day and excludes the last: `[check-in, check-out)`. Check-out day is free for the next guest.
HMAC hash	A one-way fingerprint made with a secret key; used to compare ID or phone numbers without storing them in clear.
Hold	A short reservation of inventory while a guest pays; it expires automatically.
Host type	individual, agency, hotel or hostel.
Idempotency key	A unique request ID; repeating the request with the same key does not repeat its effect.
import-linter	A tool that fails the build if one Django app imports another it is not allowed to use.
k6	A load-testing tool that simulates many users to check response times.
KYC	Know your customer: checking identity and business documents.
MFA / TOTP / passkey	Two-step login. TOTP is a 6-digit code from an authenticator app; a passkey uses the phone or computer's fingerprint or PIN.
Milestone	A block of work of 3 to 5 weeks with clear done criteria (M0 to M7, then R2).
Mock store	Data kept in server memory for demos; lost on restart.
Modular monolith	One application split internally into separate parts with clear rules about who talks to whom.
OpenAPI	A standard file (`openapi.yaml`) describing every endpoint, from which typed client code is generated.
Organisation	An agency, hotel or hostel business with team members.
Outbox	A table where the backend writes 'send this email' or 'refresh search' in the same save as the change; workers send them afterwards, so nothing is lost.
OWASP API Top 10	The ten most common security mistakes in APIs, published by the OWASP foundation; used as a checklist.
PITR	Point-in-time recovery: restoring the database to any minute in the past from continuous backups.
PostGIS	The map and geography extension for PostgreSQL.
Presigned URL	A temporary, signed web address that lets a browser upload or download one file directly from storage.
Property-based test	A test that checks a rule on thousands of random inputs (for example: lines of a price always add up to the total).
PSP	Payment service provider: a licensed company that holds and moves other people's money.

Term	Meaning
RBAC	Role-based access control: rights come from a role (owner, manager, moderator...).
REST	A common style of API where each address names a thing (`/bookings`) and the method says what to do (GET read, POST create, PATCH change, DELETE remove).
Risk score	Likelihood x impact (each 1 to 5); used to sort the risk register.
Room type	A kind of room or bed in a hotel or hostel, with a count of identical units.
RPO / RTO	Recovery point objective (how much data we may lose, here 15 minutes) and recovery time objective (how long until service is back, here 4 hours).
SATIM	The interbank company that runs card payments in Algeria.
Schemathesis	A tool that reads the OpenAPI file and sends many generated requests to check the API keeps its contract.
Server action	A Next.js function that runs on the server when a form is submitted.
Snapshot	A copy of a value (price, rate, policy) saved on a booking so later changes do not affect it.
Soft delete	Marking a row as deleted instead of removing it, so it can be restored.
State machine	The allowed list of statuses and the allowed moves between them.
STRIDE	A checklist of six threat kinds: spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege.
Taxe de séjour	Tourist tax per person per night, depending on the hotel's star rating.
Traceability matrix	A table linking each requirement to its rules, tables, endpoints, tests and milestone, so nothing is forgotten.
Vertical	A product family: homes, hotels, experiences, vehicles, trips.
Webhook	A call that another service (for example a payment gateway) makes to our API to tell us something happened.
Wilaya / commune	Algerian province / municipality. Today 69 wilayas and 1,541 communes.
Workspace (host)	The host account a user is acting for right now: themselves as a solo host, or one of their organisations.

Check yourself

- [ ] What is the difference between an endpoint and an API?
- [ ] What do RPO and RTO mean for BLADUS (15 minutes, 4 hours)?
- [ ] Why does the outbox exist?
- [ ] What is a gate in the roadmap?
- [ ] What is a half-open date range, and why does it allow back-to-back stays?

01. System inventory: what BLADUS does today

> **In plain words**
> BLADUS has two web apps and no real backend yet. Everything is kept in memory and resets when the server restarts.
> This document lists what each kind of user can do, what data exists, and which functions the future Django API must replace.
> It also lists the rules hidden in the code and every place where the two apps, or the original spec, disagree.
> The disagreements matter most: a few of them (money model, statuses, who is a host) must be decided before any table is designed.
> Raw, line-by-line notes are kept in the appendices so nothing is lost.

Date: 2026-11-04. Scope: `frontend/` (guest and host website) and `admin/` (back office). Method: reading the code, comparing it with the recovered spec (`_sources/darna_spec.md`), and clicking through the main flows on a local dev server.

Terms used in this document (all also in glossary.md):

- **Mock store**: data kept in a JavaScript object in server memory (`globalThis`). It disappears on restart and is not shared between servers.
- `lib/data` **seam**: the folder of functions every page calls to read or write data. It is the bridge: each function there becomes one API call later.
- **Server action**: a Next.js function that runs on the server when a form or button is used. It returns `ActionResult` (success, or an error code).
- **Host type**: `individual` , `agency` , `hotel` or `hostel` .
- **Vertical**: a product family: homes, hotels, experiences, vehicles, trips.
- **Ledger**: a list of money movements that is only ever added to; a balance is the sum of its lines.

---

1. The system at a glance

Part	What it is	Users	Session
Website (`frontend/`)	Next.js 14 App Router, next-intl (en, fr, ar with right-to-left), Tailwind	Visitors, guests, hosts (individual and organisation teams)	Signed cookie `darna_session` (HMAC)
Admin (`admin/`)	Same stack, separate app	Staff with 5 roles: super_admin, moderator, support, finance, analyst	JWT cookie `darna_admin_session` , 8 hours, HS256
Backend	**None.** Each app has its own in-memory mock store with its own seed data		

The two apps do **not** share data. A booking made on the website never appears in the admin, and an approval in the admin never reaches the website. The Django backend will become the one shared source of truth.

---

2. Features per actor

Status: **built** (works end to end on mock data), **partial** (some parts real, some hardcoded), **mocked** (screen only, nothing saved), **missing**.

2.1 Visitor (not signed in)

Feature	Status	Backend need
Home feed: homes, experiences, trips, hotels and hostels, vehicles, featured, wilaya tiles	built	Public listing and trip queries, featured flag
Search bar (where, when, who) and results page with left filter sidebar, facet counts, map (Leaflet + OpenFreeMap), vertical tabs	built	Search endpoint with filters, facets, map bounds
Listing detail (home, vehicle, experience): gallery, amenities, rules, policies, reviews, map, booking card	built	Listing detail, availability, price quote
Hotel and hostel detail with room-type cards (per room or per bed)	built	Room types and per-night inventory
Trip detail (itinerary, agency card, per-person price)	built	Trips and departures
Organisation profile pages (agency, hotel, hostel) and individual host profile	built	Public organisation and host projections
Sign up, sign in	built (email + password only)	Accounts, phone OTP, verification

2.2 Guest (signed-in traveller)

Feature	Status	Backend need
Book instantly	built, **no payment step**	Payment before confirmation
Request to book (24 h host reply window)	built, expiry applied only when data is read	Scheduled expiry job
My reservations ("Trips" page), cancel with policy refund	built, shows fake "Total paid by CIB/Edahabia"	Payments, refunds
Favourites	built	Favourites table
Messages with hosts (threads, read state, unread count)	built	Conversations, messages
Notifications	built (seeded only, nothing produces new ones)	Notification producers
Write a review	**missing** (reviews are read-only seeds)	Review write path tied to a completed stay
Profile, change password	built	Account endpoints

2.3 Host, individual

Feature	Status	Backend need
Become a host: 12-step guided flow with draft and preview	built, draft in browser only	Persisted onboarding drafts
"My listings" (2-page solo interface) and edit modal	partial	Listing CRUD
Bookings list with Accept / Decline, mini calendar	built	Respond to requests
Calendar: block dates, set nightly prices	built (not hidden from solo hosts)	Availability overrides
Prepaid commission wallet widget	built	Ledger
Settings: default approval mode	partial: does not affect existing or new listings	Approval-mode resolution rule

2.4 Host organisation (agency, hotel, hostel team member)

Feature	Status	Backend need
Onboarding: agency 5 steps, hotel 7, hostel 7	built (agency never asks the approval mode)	Organisation creation, verification
Today page (command centre), bookings, calendar per room type, listings with room-type manager	built / partial	Org-scoped queries
Trips (agency only): 5-step creation, list, pause, delete	built	Trips and departures
Team: invite, change role, transfer ownership, remove	partial: **invites can never be accepted**	Invitations with tokens
Organisation settings, legal fields	built	Re-verification when legal fields change
Switch between own organisations	**missing** (active org is fixed per session)	Organisation switcher

2.5 Staff (admin app)

Area	Status	Roles that can act
Sign in / out, login rate limit	built (no MFA, no password reset, invites cannot sign in)	all
Dashboard KPIs and "needs attention"	partial (trends and charts hardcoded)	all view
Approvals inbox: hosts, listings, documents, reviews, disputes	partial (document approve approves the whole host; undo is a toast)	SA, MO
Provider (host) validation	partial ("request changes" not saved)	SA, MO
Hosts directory, commission override, wallet recharge	built	SA, MO, FI
Users (guests) suspend, ban, delete, restore	built (reset password mocked)	SA, MO, SU
Listings across verticals: approve, request changes, feature, suspend, delete, restore, bulk approve, CSV	built	SA, MO
Listing create wizards, change requests, featured placements	**mocked**	
Bookings: cancel, refund, resolve dispute, CSV	built	SA, SU, FI (refund)
Finance: wallets, ledger, payouts approve/reject, default commission rates	built	SA, FI
Reviews: hide, restore, dismiss flag, delete	built	SA, MO
Coupons create, toggle, delete	built (no checkout uses them)	SA, MO
Messages, notifications, templates, broadcast	**mocked**, and no permission check at all	anyone signed in
Stats	partial (mostly hardcoded)	SA, MO, FI, AN
Settings, team and roles, audit log, onboarding configurator	built / partial	SA (audit: all view)

Roles: SA super_admin (34 permissions), MO moderator (17), SU support (10), FI finance (10), AN analyst (8). The full permission matrix is in Appendix B section 1.1.

---

3. Data inventory

3.1 Entities that exist today

Entity	Website	Admin	Notes for the backend
User	`User` with `role guest/host/admin`, one `hostType`, `organizationId`	`GuestUser` and `Host` are **separate, unlinked tables**	One `users` table plus host profile and memberships
Organisation	`Organization` with `businessCategory`, verification `pending/verified/rejected`, hotel and hostel policy fields	`Organization` type never seeded; flattened into `Host` + metadata blobs	Real table; verification as a state machine
Membership	owner, admin, manager, guide, agent (6 seeds)	type only	Role per organisation
Listing	`Listing` with `ownerType`, `latlng`, `commune`, `cleaningFee`, `cancellationPolicy`, `minNights/maxNights`, vehicle and experience fields	`Listing` with `wilayaCode`, `coordinates`, `propertyType` free text, 8 statuses	Admin cannot see cleaning fee, policy or stay rules today
RoomType	`bookingUnit room/bed`, `totalInventory`, price, capacity	same idea	One inventory column; availability computed
Reservation / Booking	`Reservation` (`res_*`), statuses incl. `declined`, `expired`	`Booking` (`BLD-2026-NNNN`), statuses incl. `disputed`, `refunded`	Union state machine needed
Trip	Separate `Trip` with itinerary and `departureDates: string[]`	A `Listing` with `category: trip`	Decide: own tables with departures

Entity	Website	Admin	Notes for the backend
Conversation, Message	built	mocked	
Notification	built (4 seeds)	mocked	
Review	5 sub-ratings, no status, read-only module constant	single rating, status, flags, host reply	Merge both shapes
Calendar override	blocked days and price per day per listing	none	Per listing and per room type
Wallet, Transaction	**prepaid commission** wallet per user or org	wallet per host mixing guest money and payouts	Fundamental decision (D-16)
Payout request	none	built	
Coupon	`Coupon`	`PromotionCoupon` (different field names)	One shape
Category, Amenity	i18n keys	`{en,fr,ar}` names; amenities stored as free strings	Reference tables
Onboarding config	4 flows in code plus browser `localStorage`	in-memory versioned config, different schema	One versioned schema served by the API
Audit entry	own `AuditLogEntry` type	append-only, 53 logged actions	Append-only table
Platform settings	commission rates store	`PlatformSettings` (field-name mismatches)	One settings row
Payment, Refund, Invoice, Payout to bank	**none**	refund is a number on the booking	Must be designed from scratch

3.2 Enums that disagree

Concept	Website	Admin
Booking status	pending, confirmed, cancelled, completed, declined, expired	pending, confirmed, cancelled, completed, disputed, refunded
Listing status	active, pending, suspended, draft	draft, pending_review, approved, changes_requested, rejected, suspended, archived, deleted (+ 3 legacy aliases)
Payment status	paid, pending, refunded, partially_refunded, voided	paid, partially_refunded, refunded, pending, failed
Transaction type	recharge, commission_deduction, payout, adjustment	wallet_recharge, booking_payment, commission_deduction, host_payout, booking_refund, adjustment
Onboarding field type	text, number, textarea, select, multi-select, image-upload, toggle	text, textarea, number, select, multiselect, radio, file, location, amenities, boolean
Wilayas	58, numeric codes	68, string codes "01".."68" (UI text says 69)
Verticals	5 (hotel created as `home` + business category)	5 with `hotel` a real vertical

Algeria today has **69 wilayas and 1,541 communes** (see 02-research.md section C.4). Neither app is correct yet.

Full field lists: website in Appendix A section 2, admin in Appendix B section 2.

---

4. The `lib/data` function inventory (the bridge to the API)

Every page calls these functions. When the backend exists, each function body becomes one HTTP call and the page code stays the same. This is the most useful part of this inventory for the build: it is the list of endpoints the apps already need.

4.1 Website (`frontend/lib/data/*` and `app/actions/*`)

Module	Functions	Proposed endpoint family
`auth.ts`	`verifyCredentials`, `registerUser`, `changePassword`, `getUserById`, `updateUserProfile`, `updateUserHostStatus`, `setUserFavorite`	`/api/v1/auth/*`, `/api/v1/me`, `/api/v1/me/favorites/{id}`
`listings.ts`	`getListings` (search + availability overlap), `getPublicListingById`, `getFeaturedListings`, `getHostListings`, `createListing`, `updateListing`, `deleteListing`	`GET /listings`, `GET /listings/{id}`, `/host/listings` CRUD
`reservations.ts`	`getReservations`, `getReservationById`, `getListingAvailability`, `createListingReservation`, `createTripReservation`, `cancelReservation`, `respondToRequest`, `seatsSoldByDeparture`	`POST /reservations`, `/reservations/{id}/cancel`, `/host/reservations/{id}/accept`
`calendar.ts`	`getAvailabilityOverrides`, `setDatesBlocked`, `setDatePrice`	`/host/listings/{id}/calendar/*`
`conversations.ts`	`getConversations`, `getMessages`, `sendMessage`, `markConversationAsRead`, `findOrCreateConversation`	`/conversations/*`
`notifications.ts`, `favorites.ts`, `reviews.ts`, `experiences.ts`, `vehicles.ts`	list and mark-read; favourites list; reviews by listing or user; vertical lists	`/me/notifications/*`, `/me/favorites`, `/listings/{id}/reviews`
`organizations.ts`	`createOrganization`, `updateOrganization`, `deleteOrganization`, `getOrganizationMembers`, `addOrganizationMember`, `updateOrganizationMemberRole`, `transferOrganizationOwnership`, `removeOrganizationMember`, `updateOrganizationVerification`	`/organizations`, `/host/organization/*`, admin verification
`trips.ts`	`getTrips`, `getPublicTripById`, `createTrip`, `updateTrip`, `deleteTrip`	`/trips`, `/host/trips`
`wallet.ts`	`getWalletByOwner`, `rechargeWallet`, `deductBookingCommission`, `reverseBookingCommission`, `getCommissionRates`, `saveCommissionRates`	`/host/wallet`, admin wallets; deduction as an internal service
Server actions (29)	thin wrappers over the above, each with session and ownership checks	same endpoints

4.2 Admin (`admin/lib/data/*` and `lib/actions/*`)

Module	Main functions	Proposed endpoint family
`hosts.ts`	list, get, approve, reject, suspend, unsuspend, ban, delete, restore, overrideCommission, export	`/api/v1/admin/hosts/*`
`listings.ts`	list with facets, get, approve, reject, requestChanges, updateStatus, feature, delete, restore, export	`/api/v1/admin/listings/*`
`bookings.ts`	list, cancel, refund, resolveDispute, export	`/api/v1/admin/bookings/*`
`finance.ts`, `ledger.ts`	summary, wallets, transactions, recharge, payouts approve/reject, refunds, commission rates, exports	`/api/v1/admin/finance/*`
`users.ts`	list, suspend, unsuspend, ban, delete, restore, updateStatus	`/api/v1/admin/users/*`
`reviews.ts`	list, hide, restore, dismissFlag, delete	`/api/v1/admin/reviews/*`
`approvals.ts`	inbox of 5 queues with waiting time and priority	`/api/v1/admin/moderation/queues`
`categories.ts`, `promotions.ts`, `onboarding.ts`, `settings.ts`, `team.ts`, `audit.ts`, `analytics.ts`, `search.ts`	CRUD, publish, settings, staff, audit list/export, KPIs, global search	`/api/v1/admin/...`

Seven admin actions have no caller in the UI, and many features are missing entirely (document approve/reject, amenity CRUD, coupon edit, payout "processed", staff password reset and MFA). The complete table with file and line for every function is in Appendix A section 3 and Appendix B section 3.

---

5. Business rules found in the code

These rules are real today, even though no one wrote them down. The backend must keep them or change them on purpose.

5.1 Money

1. All amounts are **whole Algerian dinars (DZD)** in the apps. The backend will store centimes (see 02-research.md B.10).
2. **Guests pay no platform fee.** Both apps set the guest service fee to 0. BLADUS earns only the host commission. (`admin/docs/DATA-MODEL.md` still says 7%: stale.)
3. Default commission by host type: **individual 12%, agency 15%, hotel 18%, hostel 10%**. The admin also allows a per-host override.
4. Commission base differs: website uses the **accommodation subtotal** (no cleaning fee, no taxes); admin uses the **base price**. Spec says "booking total".
5. The commission rate is copied onto the booking when it is created (snapshot), so later rate changes do not reprice old bookings.
6. Website price = nights x nightly price (with per-day overrides) + cleaning fee + taxes; weekly 10% and monthly discounts exist for display.
7. Website wallet: the host **pre-pays** a commission deposit; BLADUS deducts the commission on every confirmed booking and gives it back on decline or cancel. The balance may go negative; the host then sees a warning banner.
8. Admin wallet recharge needs a receipt reference and an amount between 1 and 10,000,000 DZD. The same receipt can be credited twice (no duplicate check).
9. Payout approval debits the wallet and refuses when the balance is too low. There is no "money actually sent" step.
10. Refunds: partial refunds add up; a refund takes back the host share proportionally; the platform keeps its commission on refunds.

5.2 Bookings and availability

11. Date ranges are **half-open**: check-in day is booked, check-out day is free. Back-to-back stays are allowed (confirmed in the click-through, O5).
12. Overlapping stays on the same single unit are refused (O4). For hotels and hostels, the room type's inventory is counted per night; a hostel bed counts one per guest.
13. Search keeps a listing when its availability **overlaps** the searched dates and shows "Available X to Y within your dates" for partial matches.
14. Stay rules: minimum and maximum nights per listing; guest count up to capacity.
15. Approval mode per listing: `instant` confirms at once; `request` creates a pending booking that expires after **24 hours**.
16. Website booking statuses move: pending to confirmed, declined, expired or cancelled; confirmed to completed or cancelled. Completion and expiry are applied **when the data is read**, not by a timer.
17. Cancellation policies (flexible, moderate, strict) decide the refund by days before check-in.

5.3 Moderation and accounts

18. Host lifecycle (admin): applied, under review, approved, rejected, suspended, banned, deleted (with restore). Suspending a host does **not** pause its listings or bookings.
19. Listing lifecycle (admin): 8 statuses; a reason of at least 3 characters is required to reject, suspend or request changes. On the website, new listings and trips go **live at once** without review.
20. Moderation waiting time and priority: hosts high after 3 days, listings after 4, documents after 3; flagged reviews and disputes always high.
21. Organisation roles: owner, admin, manager, guide, agent. Any active member may accept or decline a booking request.
22. Changing an organisation's legal fields should trigger re-verification.
23. Admin login: 5 failures in 15 minutes lock for 60 seconds, per email and per IP; same error for every failure; scrypt password hashing.
24. Admin audit log is append-only and records 53 kinds of action with actor, IP, before and after.
25. CSV exports protect against spreadsheet formula injection and are themselves audited.

The full numbered rule lists with file and line are in Appendix A section 4 and Appendix B section 4. They are rewritten as testable rules in `04-business-rules.md` (Phase 3).

---

6. What we saw when clicking through (website, 2026-11-04)

#	Flow	Observation	What the backend must do
O1	Guest sign-in	Email and password only, no verification	Add phone OTP and email verification
O2	Instant booking, 3 nights	Confirmed at once: 55,500 + 4,000 cleaning = 59,500 DZD, "No BLADUS fees"	Take payment (or a decision on payment) before confirming
O3	Link with `guestCount=2`	Booking card showed 1 guest	One parameter name in the API contract
O4	Overlapping dates	Correctly blocked	Move the guarantee into the database

#	Flow	Observation	What the backend must do
O5	Back-to-back dates	Allowed	Half-open ranges confirmed
O6	Request to book	Pending, "reply within 24 hours"	Real expiry job
O7	Trips page	Every card says "Total paid by CIB / Edahabia", even unpaid requests	Payment records and honest payment status
O8	Messages	Threads, read receipts, unread count	Conversations API
O9	Become a host	4 host types with guided flows	Persisted drafts, published flow config
O10	Host mode as an agency owner who also owns 2 hotels	Rating 4.96 from 159 reviews next to "No guest reviews yet"; prepaid wallet 145,000 DZD at 15%	Ratings derived from reviews; active-organisation context
O11	Host earnings	0 DZD while a confirmed booking exists	Define when earnings are recognised

Source: Appendix D.

---

7. Gaps and inconsistencies

7.1 The decisions that block the data model

These come from comparing the spec, the website and the admin (Appendix C, items D-1 to D-22). The most important:

ID	Topic	The conflict	Why it blocks
D-16	**Money model**	Spec section 9 and the website: host pre-pays commission, guest pays the host directly. Admin seed: BLADUS collects guest money and pays hosts out. Spec section 3 says "the moment a guest completes payment".	Decides payment integration, ledger accounts, payouts and legal licensing
D-1	Who is a host	Website: user plus organisations, one person can be both. Admin: one flat `Host` row	Decides the keys for wallets, listings, commission, moderation
D-3, D-5	Commission base and override	subtotal vs base vs total; override only in admin	Ledger posting rules
D-9	Booking statuses	two different sets of six	State machine and payments
D-10	Publish requires approval?	website: live at once; admin: review queue	Listing status flow
D-11	Trips	own table vs listing subtype; departures as strings	Booking target model
D-12	Is "hotel" a vertical?	yes in admin, no in spec	Search filters and analytics
D-14, D-15	Onboarding	12 steps vs admin cap of 2; schemas differ; config in browser storage	Config tables
D-6, D-7, D-8	Approval mode and request expiry	stored in 3 places; any member may respond; expiry lazy	Settings and scheduler

7.2 Mock shortcuts that will not survive production

1. Safety against double booking and double debit relies on JavaScript running one thing at a time. A real server needs database transactions, row locks and constraints.
2. No idempotency keys (a protection that makes a retried request harmless) on recharge, refund, payout or booking.
3. IDs from `Date.now()` and `Math.random()`; collisions are possible.
4. Rate limits live in process memory.
5. Admin logout only deletes the cookie; the token stays valid until it expires.
6. Staff are recorded by **name**, not ID, in the ledger, payouts, onboarding and disputes.
7. Names, ratings and counters are copied into many rows and will drift.
8. Expiry and completion happen only when someone reads the data.
9. All images are two placeholder files; there is no upload, storage or virus scan.
10. The demo password ships in the website's code.
11. No payments, refunds as records, invoices, tax lines or payouts to banks.
12. No review writing, no notification producers, no invitation acceptance, no organisation switcher.
13. Three amenity vocabularies, free-text property types and cities where references are needed.
14. Messages and broadcast in the admin need no permission.

7.3 Things that are fine and should be kept

Server-side permission checks in both apps, signed sessions, half-open date logic, commission snapshot on booking, append-only audit with before/after, CSV formula protection, generic login errors with constant-time comparison, soft delete with restore in the admin.

Appendices (raw notes, kept unchanged)

	File	Lines	Content
A	_work/frontend-inventory.md	913	Website: features, every enum and entity, every function with proposed endpoint, rules 4.1 to 4.18, gaps, 34 routes
B	_work/admin-inventory.md	830	Admin: permission matrix, 35 features, types, functions, rules, onboarding contract, gaps, 80 routes
C	_work/spec-vs-code.md	235	Requirements R1 to R69 with status, disagreements D-1 to D-22, usability findings U1 to U16

	File	Lines	Content
D	_work/click-through.md	17	Observations O1 to O11
E	_sources/		Recovered original spec and usability reports

Check yourself

- [ ] Can you name the two apps and say why they do not share data today?
- [ ] Which four host types exist, and what is each one's default commission?
- [ ] Does a guest pay any BLADUS fee? Who pays BLADUS?
- [ ] What does "half-open date range" mean for a check-out day?
- [ ] Why is D-16 (money model) the first decision to make?
- [ ] Name three mock shortcuts that would cause real bugs with two servers running.
- [ ] Where would you look to find the endpoint that replaces `createListingReservation`?

02. Research: what others do, which Django tools to use, and what Algeria requires

> **In plain words**
> Before designing tables we looked at how Airbnb, Booking.com, Expedia and GetYourGuide solve the same problems, and which Django tools are best today.
> We also checked Algerian realities: how online payment works, SMS costs, the laws on e-commerce and personal data, maps and hosting.
> Big finding: Algerian law likely requires the site to be hosted in Algeria, and collecting guest money to pay hosts may need a payment licence.
> So the safest first version is "BLADUS takes only its commission", which matches what the website already does.
> Every legal, tax or money point here is marked **VERIFY WITH A PROFESSIONAL**. This is engineering research, not legal advice.

Research date: all sources accessed on **2026-11-04**. Labels: **[verified]** read on an official or primary page; **[secondary]** press, vendor or blog; **[unverified]** could not confirm. Full source lists with every URL: Appendix R1 and Appendix R2.

Terms used here (also in glossary.md):

- **Exclusion constraint**: a database rule that refuses two rows whose date ranges overlap for the same unit. It makes double booking impossible even if the app has a bug.
- **Idempotency key**: a unique ID the client sends with a payment request; if the request is retried, the server returns the first result instead of charging twice.
- **Double-entry ledger**: every money movement is written as two or more lines that add up to zero (money leaves one account and enters another), so errors are visible.
- **PSP**: payment service provider, a company licensed to hold and move other people's money.
- **KYC**: "know your customer", checking a host's identity and documents.
- **BFF**: "backend for frontend", here the Next.js server holding the login cookie and calling Django for the browser.

A. Comparable platforms

A.1 What they publish

Platform	Lesson for BLADUS	Source
Airbnb	Started as one big app, split into services only when hundreds of engineers needed it	https://medium.com/airbnb-engineering/building-services-at-airbnb-part-1-c4c1d8fa811b
Airbnb payments	Idempotency keys with a row lock, always on the main database, to avoid double payments	https://medium.com/airbnb-engineering/avoiding-double-payments-in-a-distributed-payments-system-2981f6b070bb
Airbnb finance	Money modelled as accounting events in a double-entry ledger	https://medium.com/airbnb-engineering/tracking-the-money-scaling-financial-reporting-at-airbnb-6d742b80f040
Airbnb messaging	Clients fetch only what changed since their last sync	https://medium.com/airbnb-engineering/messaging-sync-scaling-mobile-messaging-at-airbnb-659142036f06
Airbnb trust	Extra checks only for risky users ("targeted friction")	https://medium.com/airbnb-engineering/fighting-financial-fraud-with-targeted-friction-82d950d8900e
Airbnb reviews	14 days after checkout, both sides blind until both wrote or time ran out	https://www.airbnb.com/help/article/13
Booking.com	Hotel inventory = room type x rate plan x date; "rooms to sell" per room type per date	https://developers.booking.com/connectivity/docs/ari.md
Booking.com reviews	Only real bookers review; moderated; visible 36 months	https://partner.booking.com/en-us/help/guest-reviews/general/guest-review-process-and-conditions
Expedia	Cancellation policy is structured data (penalty windows), shown before booking	https://developers.expediagroup.com/rapid/lodging/shopping/constructing-cancellation-policies
GetYourGuide	Activities sold as time slots with seats; flow is check, hold, confirm	https://www.getyourguide.careers/posts/scaling-the-inventory-parts-of-our-system-using-kafka-streams
pretix (open source, Django)	Best Django example of quota inventory with time-limited holds	https://docs.pretix.eu/dev/development/concepts.html
django-hordak	Double-entry accounting in Django with database constraints	https://github.com/adamcharnock/django-hordak

Not found: Airbnb's calendar architecture, Vrbo developer docs, Airbnb's current cancellation tiers from a primary page [unverified].

A.2 One idea that covers all six products

Every bookable thing is **a unit, a time shape and a capacity**:

Vertical	What is sold	Time shape	Capacity per slot
Home	the whole place	nights `[check-in, check-out)`	1
Hotel	a room type (pool of N rooms)	nights	N per night
Hostel	a bed type or dorm (pool of beds)	nights	beds per night
Experience	an option at a start time	time slot	seats per slot
Vehicle	a vehicle	pickup to return	1
Agency trip	a departure on fixed dates	fixed departure	seats per departure

A.3 Hard problems and our decisions

Problem	Decision
Double booking	Capacity 1: date range column plus **exclusion constraint** in PostgreSQL. Pools (rooms, beds, seats): one **inventory row per unit per day or slot** with a rule `held + booked <= total`, updated atomically inside a transaction. Holds expire by a scheduled job. Search results are re-checked inside the booking transaction.
Search at Algeria scale	PostgreSQL only at first (map box, filters, availability), with a small search table per listing. An external engine only when measured slow.
Pricing	Nightly prices stored per unit and date, built from rules (base, weekend, season, long stay). The server gives a **quote** with an ID and expiry, re-checked at booking.
Money	Idempotency key on every money endpoint; append-only double-entry ledger; balances computed from entries; gateway webhooks stored raw, verified and processed once.
Cancellation	Named policy templates stored as windows (hours before start, refund %); copied onto the booking at confirmation.
Fake listings and fraud	Listings go through review; host KYC before any payout; rule-based flags (new account + high value, duplicate photos, phone numbers in messages) that add friction, not blanket bans.
Reviews	Only after a completed booking, one per direction, 14-day window, double-blind, host can reply once.
Architecture	**Modular monolith**: one Django project split into clear apps. Services later only if needed.

B. Django ecosystem (two or more options per decision)

Versions read from PyPI and npm on 2026-11-04.

#	Decision	Option 1	Option 2	Choice	Why
B.0	Django version	6.1.3 (supported to Dec 2027)	5.2 LTS (to Apr 2028)	**Django 6.1 on Python 3.13**, move to 6.2 LTS in 2027	6.x has built-in tasks API and CSP; 6.1 overlaps 6.2 LTS
B.1	Database and maps	PostgreSQL + PostGIS (GeoDjango)	Plain lat/lng numbers	**PostGIS**	Real distance and map-box queries, wilaya polygons, range types and exclusion constraints
B.2	API framework	Django REST Framework 3.18.3	Django Ninja 1.7.3	**DRF** (HackSoft style)	Mature permissions, throttling, OpenAPI; most Algerian Django devs know it
B.3	Accounts and login	django-allauth 65.21 headless	dj-rest-auth + SimpleJWT	**allauth headless**	Phone OTP, MFA (TOTP, passkeys), rate limits built in; session tokens, not long-lived JWTs
B.4	Permissions	django-rules (predicates)	django-guardian (per-object rows)	**Own `Membership` roles + django-rules** + always filtering lists by the caller's organisations	Rights come from membership, not per-object grants
B.5	Background jobs	Celery 5.6 + Redis	Django tasks (6.0+), Dramatiq, Huey	**Celery + Redis**	Retries, schedules, monitoring; tasks kept thin so moving to Django tasks later is easy
B.6	Live messaging	Polling a "changes since" endpoint	Centrifugo 6.10, Django Channels	**Polling first, Centrifugo later**	Simplest to run; Airbnb's sync idea
B.7	Search	PostgreSQL full-text + pg_trgm + PostGIS	Meilisearch 1.56, Typesense, OpenSearch	**PostgreSQL first**	One store, consistent with availability; Meilisearch only if Arabic typo search is needed
B.8	Files and images	django-storages + Cloudflare R2 (S3 API)	Cloudinary	**S3-compatible storage**, presigned uploads, Pillow renditions, ClamAV for documents	Vendor-neutral. **Location must respect Algerian law, see C.3**
B.9	Translations	django-modeltranslation 0.20 (columns per language)	django-parler 2.4 (translation tables)	**modeltranslation** for platform lists (amenities, wilayas); hosts write in one language with `source_language`	Fixed 3 languages, fast reads
B.10	Back office	Existing Next.js admin	Django admin + Unfold 0.110	**Both**: Next.js admin is the product; Django admin is a staff-only, MFA, mostly read-only console	Fast internal tools without bypassing rules
B.11	History / audit	django-pghistory 3.9 (database triggers)	django-simple-history, django-auditlog (signals)	**pghistory + a domain `AuditEvent` table**	Triggers also catch bulk updates
B.12	Limits, cache, idempotency	DRF throttles + Redis cache + in-house idempotency	django-ratelimit (inactive since 2024), django-idempotency-key (inactive since 2023), django-cachalot	**First option**	Libraries in option 2 are unmaintained or unsuited to write-heavy tables
B.13	API contract	drf-spectacular 0.30 + openapi-typescript/openapi-fetch	orval 8.46, hey-api	**spectacular + openapi-typescript**	Small typed client that works in Next.js server components
B.14	Tests	pytest-django + factory_boy + hypothesis + schemathesis	model_bakery	**First option**	Readable factories; property tests for money and dates; contract fuzzing
B.15	Structure	Modular monolith (HackSoft services/selectors), ideas from cookiecutter-django	Many small services	**Modular monolith**	Small team, one database, clear boundaries
B.16	Ops tooling	pydantic-settings, ruff, mypy + django-stubs, GitHub Actions, Sentry, structlog, Prometheus	django-environ, black/isort	**First option**	Typed settings that fail fast; one fast linter
B.17	Money storage	Whole numbers in centimes (`BigIntegerField`) + currency	Decimal / django-money (repo archived, moving to Codeberg)	**Centimes + currency code** + small in-house `Money` type	Exact sums; DZD has minor unit 2 in ISO 4217 even though prices are whole dinars

C. Algeria

C.1 Payments

Rail or provider	What it is	Facts	Status
CIB card	Bank debit card through SATIM	Merchant needs commerce register (RC), CNRC e-supplier filing, GIE Monétique authorisation and a bank contract	[verified] cibweb.dz
Edahabia card	Algérie Poste card	Accepted through the same SATIM gateway	[verified]
BaridiMob / CCP transfer	Postal account transfers	Limits and fees conflict across sources	[secondary]
BaridiPay QR	Merchant QR since June 2025	No public API found	[unverified]
Cash on arrival	Very common	Manual reconciliation	[secondary]
Foreign cards	Through BDL bank	5% per transaction + 65,000 DA/month + 70,000 DA setup, companies only	[verified] bdl.dz
Chargily Pay V2	Aggregator with API and webhooks	Fees 0% (starter cap), 1.5%, 3%, 4.5% by plan; refunds 2%; payout holds 24 to 72 h; CIB, Edahabia	[verified] dev.chargily.com
SlickPay	Aggregator	1.4% to 2% by payout schedule	[verified] slick-pay.com
Guiddini	Certified SATIM module, not an aggregator	Merchant still needs its own bank contract	[verified] cibweb.dz

Key realities (VERIFY WITH A PROFESSIONAL):

1. **No split payment product** (no "Stripe Connect" for Algeria) was found [unverified: absence of evidence].
2. Collecting guest money and paying it to hosts looks like a **payment service**. Banque d'Algérie Règlement 25-02 (April 2025) sets PSP licensing with **160 million DA** minimum capital [secondary].
3. Law 18-05 art. 27: electronic payment must go through platforms run by approved banks and Algérie Poste [verified, text read].

Payment models, from lowest to highest legal risk:

#	Model	Fits current website?
M1	Guest pays host directly (cash or transfer); host pre-pays commission to BLADUS; BLADUS invoices commission	**Yes**: this is the prepaid wallet the website already has
M2	Guest pays a deposit equal to BLADUS's commission through Chargily; rest paid to host on arrival	Close
M3	Professional hosts use their own gateway accounts; BLADUS creates checkouts with their keys	For hotels and agencies only
M4	BLADUS collects everything and pays hosts out (Airbnb model)	Needs a licensed PSP or bank partner: phase 2

Recommendation for the blueprint: design the ledger so all four models fit, launch with **M1 (plus optional M2)**, and keep M4 behind a decision gate. This resolves disagreement D-16 in favour of the spec and the website, and makes the admin's "booking payment / host payout" flow a later phase. **Decision needed from you** (see `13-decisions-needed.md`).

C.2 Messaging and verification

Channel	Cost / fact	Status
Twilio SMS to Algeria	about USD 0.27 per SMS, sender ID registration required	[verified] twilio.com
BulkGate	EUR 0.08 to 0.24 by carrier	[secondary]
Local aggregator (e.g. Sobersys)	prices not published	[secondary]
WhatsApp Business	paid per template message since July 2025; Algeria rate not found	[secondary]
Email (Resend, SES, Brevo)	free tiers 300/day to 3,000/month	[secondary]

Decision: one `SmsProvider` interface, a local aggregator first with an international fallback; email and WhatsApp OTP where possible; strict limits per phone, IP and device against SMS pumping fraud.

C.3 Legal (all points: VERIFY WITH A PROFESSIONAL, not legal advice)

Law	What it says that matters	Status	Consequence
Law 18-05 (e-commerce), art. 8	E-commerce requires RC registration and **a site hosted in Algeria with a `.com.dz` domain**	[verified] JO n° 28, 2018	Plan production hosting in Algeria
Law 18-05 art. 10 to 13, 25, 31 to 33	Mandatory offer content; order in 3 steps (review, confirm, contract); keep transaction records for CNRC; opt-in for marketing	[verified]	Booking flow and consent records
Law 18-07 on personal data, amended by Law 25-11 (July 2025)	Prior declaration or authorisation to ANPDP; **transfer abroad needs ANPDP authorisation**; breach notice within **5 days**	[secondary]	Keep personal data in Algeria; register of processing; breach runbook
Circular of 16 June 2012 ("chez l'habitant")	Max 5 rooms, 15 guests, insurance, declaration at the APC	[secondary]	Individual host documents
Law 08-11 art. 29	Foreign guests must be declared to security services	[secondary]	Guest nationality field and host reminder
Law 99-06, Décret 17-161	Travel agencies need a Ministry of Tourism licence (A or B)	[verified exists] mta.gov.dz	Agency KYC
Décret 19-158	Hotel classification	[secondary]	Star rating drives tourist tax
Vehicle rental rules	No decree found	[unverified]	Ask a lawyer
Whole-home short-term rental	No specific framework found	[unverified]	Ask a lawyer

C.4 Tax

Item	Value	Status
VAT standard / reduced	19% / 9%	[verified] PwC
VAT for tourism, hotels, travel, tourist vehicle rental	9% until 31 Dec 2027	[secondary]
Taxe de séjour (tourist tax) per person per night	5* 600 DA, 4* 500, 3* 300, 2* 200, 1* 100; 50 DA proposed for unclassified (draft)	[secondary]
Invoice content (Décret 05-468)	RC, NIF, NIS, article d'imposition, sequential number, HT, VAT, TTC	[secondary]
Accounting retention	10 years commonly cited	[secondary]
Auto-entrepreneur	Turnover cap 5M DA/year, IFU 0.5%	[secondary]

C.5 Geography

- **69 wilayas and 1,541 communes** since Loi 26-06 (JO n° 25, 5 April 2026) [secondary; JO not opened]. Codes 59 to 69 differ between sources.
- Seed data: **el-amin-dev/algeria-wilayas-communes** (CC0, names in ar/fr/en) + coordinates from **ihahachi/Algeria-Cities** (MIT). Keep `source` and `valid_from` so the next reform is a data migration; keep the old 58-wilaya codes too.
- Maps: OpenFreeMap tiles are free with attribution [verified]. Public **Nominatim forbids autocomplete** and allows 1 request per second [verified]: use it only server-side and cached. Addresses are often informal: commune + pin on the map.

C.6 Hosting

Option	Price	Fit
ICOSNET (Algiers, Oran)	from 2,800 DZD/month per VPS	**Recommended for production data** [secondary]
Djezzy Cloud (Béjaïa)	quote	Alternative [secondary]
Hetzner (EU)	raised prices in 2026; CX23 about EUR 5.49	Dev and non-personal assets only [secondary]
Cloudflare	PoPs in Algiers, Oran, Constantine, Annaba, status varies	Static assets [conflicting]

Backups: encrypted, second Algerian data centre, restore test monthly. Offsite backups abroad only after ANPDP analysis.

C.7 Culture and formats

Topic	Rule
Weekend	Friday and Saturday; week starts Sunday
Digits	Latin digits even in Arabic (`ar-DZ` with `latn`)
Phone	`+213` then 5, 6 or 7 and 8 digits: `^\+213[5-7]\d{8}$`
Time zone	Africa/Algiers, UTC+1, no daylight saving
Names	Store Arabic and Latin forms separately
ID	Biometric ID card; NIN has 18 digits (different from card number)

D. Security

D.1 OWASP API Top 10 (2023) applied to BLADUS

Risk	BLADUS example	Control
API1 object access	`GET /bookings/{id}` of someone else	Check ownership on every object; non-guessable IDs (UUIDv7)
API2 authentication	OTP brute force, SMS pumping	6 digits, 5 tries, 10 min, limits per phone/IP/device; MFA for staff and hosts with payouts
API3 property access	Guest sees host phone before booking; mass-assigning `status` or `commission_rate`	Output fields per role; allowlist of writable fields
API4 resource use	Scraping, big uploads, SMS cost	Page size caps, upload limits, SMS budget alarms
API5 function access	Host calling a staff refund endpoint	Separate admin API, deny by default
API6 business flows	Holding calendars without paying, coupon abuse, fake reviews	Hold expiry, velocity limits
API7 SSRF	Calendar import URL, image by URL	Block private IPs, egress proxy
API8 misconfiguration	Open Redis, debug on, CORS `*`	Hardening checklist
API9 inventory	Old endpoints, prod data in staging	OpenAPI inventory, no prod data in staging
API10 unsafe APIs	Trusting payment redirect instead of webhook	Verify webhook signature, re-check status server-side

Source: https://owasp.org/API-Security/editions/2023/en/0x11-t10/ [verified].

D.2 Fraud and abuse

- Off-platform payment pushes (BaridiMob/CCP numbers, WhatsApp links in messages before confirmation): detect and warn.
- Fake listings with stolen photos: perceptual hash, proof of property for the first listing, delay on first payout.

- Duplicate accounts after a ban: phone, device and hashed NIN.
- Account takeover via SIM swap: re-verify before payout detail changes, hold payouts 48 to 72 h after such a change.
- Reviews only from completed bookings; watch for bursts and rings.

D.3 Data rights, backups, audit

- Access, correction, deletion: deletion anonymises the person but keeps booking and invoice records for legal retention.
- Append-only audit log with actor, action, object, before/after hash, IP, request ID; hash-chained for tamper evidence. KYC file views always logged.
- Breach runbook with a 5-day ANPDP clock.

E. Decisions taken in this phase, and what needs a professional

#	Decision	Confidence	Needs
R-1	Django 6.1 + DRF + PostgreSQL/PostGIS + Celery/Redis + allauth headless	High	none
R-2	Exclusion constraints for single units, daily inventory rows for pools	High	none
R-3	Centimes + currency, double-entry ledger, idempotency keys	High	none
R-4	Launch money model M1 (prepaid commission), ledger ready for M2 to M4	Medium	**Your decision** + **lawyer / accountant**
R-5	Host production data in Algeria (ICOSNET or Djezzy Cloud)	Medium	**Lawyer** (Law 18-05 art. 8, Law 18-07/25-11)
R-6	`.com.dz` domain for the public site	Medium	**Lawyer**
R-7	Chargily as first gateway, SlickPay as second	Medium	**Accountant / bank**: contract terms, amount units in API
R-8	KYC document list per host type (see R2 C3.5)	Low	**Lawyer**
R-9	VAT 9% and taxe de séjour handling per booking	Low	**Accountant**
R-10	69 wilayas from el-amin + ihahachi	Medium	Check official JO and ONS codes
R-11	Polling for messages, PostgreSQL search	High	none

Check yourself

- [ ] Why does an exclusion constraint matter more than a check in Python code?
- [ ] What are the four payment models, and which one does the website already use?
- [ ] Which Algerian law article points to hosting in Algeria, and why is it flagged "verify"?
- [ ] Why do we store money in centimes even though prices are whole dinars?
- [ ] Why polling before WebSockets for messages?
- [ ] Name two fraud patterns that are specific to Algeria's payment habits.
- [ ] How many wilayas and communes does Algeria have today, and which seed datasets will we use?

03. Requirements: what the backend must do

> **In plain words**
> This is the list of everything the Django backend must do, one line per need, each with an ID like `BKG-03`.
> Each line says who needs it, how important it is, in which release, and how we will know it works ("accepted when").
> It also says what the apps do today, so you can see the size of the gap.
> The same list is in requirements.csv for spreadsheets. The rules behind the lines are in 04-business-rules.md.
> The product decisions you approved on 2026-11-04 are written first, because many lines depend on them.

Terms used here (see glossary.md):

- **MoSCoW priority**: **Must** (no launch without it), **Should** (important, can slip a little), **Could** (nice to have).
- **Release**: **MVP** (first public launch), **R2** (the next release after launch), **Later** (needs a legal or business decision first).
- **Acceptance criterion** ("accepted when"): a sentence a tester can check with yes or no.
- **Non-functional requirement (NFR)**: a quality the system must have (speed, safety, backups) rather than a feature.

---

1. How to read the IDs

Prefix in "Source"	Meaning	Where to find it
R1 to R69	A requirement from the original spec	_work/spec-vs-code.md section 1
D-1 to D-22	A disagreement between spec, website and admin	same file, section 2
U1 to U16	A usability finding	same file, section 4
O1 to O11	Something seen while clicking through	01-system-inventory.md section 6
INV-F / INV-A	Website / admin inventory section	Appendices A and B
RES	Research section	02-research.md
DEC-001 to DEC-013	A product decision (section 2 below)	here
BR-xxx	A business rule	04-business-rules.md

---

2. Product decisions (approved 2026-11-04)

The product owner accepted the proposed defaults for all open product questions (DEC-001 to DEC-013) and later asked for two more rules, already built in the apps (DEC-014, DEC-015).

ID	Question	Decision	Consequence for the design
DEC-001	Money model	**M1: prepaid commission.** Guests pay the host directly (cash, transfer, or the host's own means). The host keeps a commission deposit with BLADUS, which deducts its commission on each confirmed booking. The ledger is designed so models M2 to M4 can be switched on later.	No guest payment or payout in MVP; double-entry ledger with accounts ready for them
DEC-002	Commission base	**Accommodation subtotal**: nights x nightly price (after length-of-stay discount), excluding cleaning fee and taxes. For trips and experiences: seats x price per person.	One base, stored on each booking
DEC-003	Review before publishing	**Yes.** New listings and trips go to `pending_review`; edits of key fields return them to review.	Moderation queue drives publication
DEC-004	One person, many host roles	**Yes**, with an active-organisation switcher.	Host account = user or organisation; memberships
DEC-005	Hotel vertical	**Own vertical in search**, rolled into the home family in analytics.	`vertical` enum includes `hotel`
DEC-006	Trips	**Own tables with departures** (dates, seats, price, status).	`trip`, `trip_departure` tables
DEC-007	Individual onboarding length	**Keep 12 steps**; drop the admin's 2-step cap.	Config schema without that rule
DEC-008	Request reply window	**24 hours**, as a platform setting.	Scheduled expiry job
DEC-009	Who may answer requests in an organisation	**Owner, admin, manager.**	Permission `bookings.respond`
DEC-010	When earnings count	**At completion** (after check-out).	Reporting rule
DEC-011	Cancellation policies	**Flexible, moderate, strict**, stored as time windows.	`cancellation_policy` table
DEC-012	Guest phone verification	**Required before the first booking.**	Phone OTP in MVP
DEC-014	What each host type may publish (2026-11-04)	**Individual: homes and vehicles. Agency: experiences and trips. Hotel and hostel: their property.**	`HST-07`, `BR-HST-05`; demo data moved accordingly
DEC-015	Experience and trip types (2026-11-04)	**8 experience types and 8 trip types, one shared list** used everywhere	`LST-12`, `TRP-07`, `BR-LST-05`; enums in the schema

ID	Question	Decision	Consequence for the design
DEC-013	Wilaya count shown	**69** (codes 59 to 69 to be confirmed against the official journal).	Geography seed

Legal and accounting questions (L1 to L7, F1 to F4 in 14-open-questions.md) are still open. Every requirement that depends on them carries "VERIFY" in its source.

---

3. Actors

Actor	Who	App
Visitor	Anyone not signed in	Website
Guest	Signed-in traveller	Website
Host	Individual host, or a member of an organisation acting for it	Website (host mode)
Org owner / admin / manager / agent / guide	Roles inside an agency, hotel or hostel	Website (host mode)
Moderator, Support, Finance, Analyst, Super admin	BLADUS staff roles	Admin
System	Scheduled jobs and automatic checks	Backend

---

4. Summary and full list

The tables below are generated from _tools/requirements_data.py by `python _tools/build_requirements.py`, which also writes requirements.csv. Edit the data file, not the tables.

Priority	MVP	R2	Later	Total
Must	109	0	0	109
Should	17	12	0	29
Could	0	7	2	9
Total	126	19	2	**147**

ACC: Accounts and sign-in (9)

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
ACC-01	Sign up with email or Algerian phone number	Visitor	Must	MVP	A visitor can create an account with email+password or phone (+213[5-7]XXXXXXXX) and must verify it by code before booking	Email+password only, no verification	INV-F 1.1, RES B.3	BR-ACC-01
ACC-02	Sign in with password or one-time phone code	Guest, Host	Must	MVP	Login works with email+password or phone OTP (6 digits, 10 min, 5 tries); same generic error for every failure	Email+password only	RES B.3, O1	BR-ACC-02
ACC-03	Session for both Next.js apps	All	Must	MVP	Website and admin hold an httpOnly cookie on the Next.js server; Django session tokens can be revoked server-side; logout revokes the token	Signed cookie; admin JWT cannot be revoked	RES B.3	BR-ACC-03
ACC-04	Password reset and change	Guest, Host, Staff	Must	MVP	Reset by email or phone code; change requires current password; all other sessions revoked after change	Change only (website); none for staff	INV-A 4.15	
ACC-05	Two-factor login	Staff, Host with payouts	Must	MVP	Staff must enrol TOTP or passkey before first use; hosts must enrol before editing payout details	Flag only, never enforced	RES B.3, D.1	BR-ACC-04
ACC-06	Profile management	Guest, Host	Must	MVP	User edits name (Arabic and Latin), avatar, language, phone; changing phone or email requires re-verification	Built (no re-verification)	INV-F 3.2	
ACC-07	Login rate limiting shared across servers	All	Must	MVP	5 failures in 15 min lock 60 s per account and per IP, stored in Redis	In-process memory	INV-A 4.15, gap 6.1-4	BR-ACC-05
ACC-08	One user, many roles	Guest, Host	Must	MVP	One user may be a guest, an individual host and a member of several organisations at once	Single hostType per user	DEC-004, D-1	BR-ORG-01
ACC-09	Favourites	Guest	Should	MVP	Guest adds or removes a listing or trip from favourites; list endpoint paginated	Built	INV-F 3.2	

ORG: Organisations and teams (7)

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
ORG-01	Create an organisation (agency, hotel, hostel)	Host	Must	MVP	Creating an organisation makes the creator its owner; status starts pending_verification	Built (frontend)	R3, R7-R9	BR-ORG-02
ORG-02	Active-organisation switcher	Host	Must	MVP	A user with several memberships picks the active one; every host request carries it and is checked	Built in website 2026-11-04 (host menu, membership checked server-side)	DEC-004, INV-F gaps	BR-ORG-01
ORG-03	Invite a team member	Org owner/admin	Must	MVP	Invite by email or phone with a role; invite token expires in 7 days; invitee accepts after signing in	Invites cannot be accepted	INV-F 3.8, gap	BR-ORG-03
ORG-04	Change role, remove member, transfer ownership	Org owner/admin	Must	MVP	Only owner transfers ownership; an organisation always has exactly one owner; audited	Built	INV-F 3.8	BR-ORG-04

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
ORG-05	Per-role permissions inside an organisation	Org members	Must	MVP	Permissions per role (owner, admin, manager, agent, guide) incl. bookings.respond for owner/admin/manager only	Any member can do anything operational	DEC-009, D-7	BR-ORG-05
ORG-06	Re-verification on legal field change	Org owner	Should	MVP	Changing legal name, RC, NIF or licence number moves verification back to pending and notifies staff	Field list exists	INV-F 3.8	BR-ORG-06
ORG-07	Public organisation profile	Visitor	Must	MVP	Public page shows name, logo, verified badge, listings, trips, rating; never legal numbers or staff phones	Built	INV-F 1.1	

HST: Becoming a host and verification (7)

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
HST-01	Guided onboarding per host type	Host	Must	MVP	Individual 12 steps, agency 5, hotel 7, hostel 7, all served from the published config; every flow asks the booking approval mode	Hardcoded; agency misses approval step	R6-R9, R17, D-14	BR-HST-01
HST-02	Save onboarding drafts on the server	Host	Must	MVP	Draft saved per user and host type with the config version it started from; resumable on any device	Browser localStorage	O9, R49	BR-HST-02
HST-03	Upload verification documents	Host	Must	MVP	Required document list depends on host type; files go to a private bucket; each document has its own review status	Placeholder images only	RES C3.5	BR-HST-03
HST-04	Host verification lifecycle	Host, Moderator	Must	MVP	Statuses applied, under_review, approved, changes_requested, rejected, suspended, banned; moves only along the allowed transitions with reasons	Admin only, not linked to website	INV-A 4.6	BR-HST-04
HST-05	Payout or refund account details	Host	Should	MVP	Host stores CCP (RIP) or bank RIB in own name; changing it needs 2FA and holds payouts 72 h	Missing	RES C1, D4	BR-PAY-09
HST-07	What each host type may publish	Host	Must	MVP	Individual: homes and vehicles; agency: experiences and trips; hotel and hostel: their property with room types; enforced by API and database, create-listing flow offers only allowed choices	Built in website 2026-11-04 (UI + server actions)	DEC-014	BR-HST-05
HST-06	Default approval mode per host or organisation	Host	Must	MVP	Owner default applies to new listings; a listing may override; platform switch can force request mode	Stored in 3 places, not applied	R20, D-6	BR-BKG-02

LST: Listings and catalogue (12)

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
LST-01	Create and edit a listing (home, hotel, hostel, experience, vehicle)	Host	Must	MVP	Host creates a draft, fills vertical-specific fields, submits; server validates every field	Built (no review)	INV-F 3.3	BR-LST-01
LST-02	Listing review before publication	Host, Moderator	Must	MVP	Submitted listing goes to pending_review; published only after staff approval; edits of key fields go back to review	Live at once	DEC-003, D-10	BR-LST-02
LST-03	Listing status lifecycle	Host, Moderator	Must	MVP	Statuses draft, pending_review, changes_requested, published, paused, rejected, suspended, archived, deleted (soft); allowed moves enforced	Two different sets	INV-A 4.7	BR-LST-03
LST-04	Room types for hotels and hostels	Host	Must	MVP	A property has room types with price, capacity, unit (room or bed), inventory count, amenities, photos	Built	R4, R5, R14	BR-AVL-03
LST-05	Amenities and categories from a shared list	Host, Staff	Must	MVP	Amenities and property types are reference rows with en/fr/ar names; listings link to them	Three vocabularies, free text	INV-A 6.3	BR-LST-04
LST-06	Location: wilaya, commune, exact and public point	Host	Must	MVP	Listing references a commune; exact point hidden until booking confirmed; public point fuzzed by about 500 m	Free text city, exact coordinates public	RES C.5, B.1	BR-SRC-03
LST-07	House rules, stay rules and cancellation policy	Host	Must	MVP	Min/max nights, check-in/out times, rules list, policy chosen from templates	Built (website only)	INV-F 4.5, 4.8	BR-CAN-01
LST-08	Pause and soft delete	Host	Must	MVP	Host can pause or delete; delete is soft and keeps bookings; staff can restore	Hard delete on website	U7	BR-LST-03
LST-09	Featured listings	Staff	Could	R2	Staff feature a published listing with optional start and end dates and position	Boolean only	INV-A 4.7	
LST-10	Vehicle listing specifics	Host	Should	MVP	Make, model, year, seats, gearbox, fuel, pickup points, driver age and licence rules	Built (website)	INV-F 2.2	
LST-12	Canonical experience types	Agency, Guest	Must	MVP	Every experience has one of 8 types (excursion, guided tour, nature and adventure, desert, sport, workshops and culture, gastronomy, wellness); search filters, cards, creation forms and admin use the same list	Built in both apps 2026-11-04 (frontend/lib/catalog-types.ts)	DEC-015	BR-LST-05
LST-11	Experience listing specifics	Host	Should	MVP	Duration, languages, meeting point, time slots with seats	Built partly	INV-F 2.2	

TRP: Agency trips and departures (7)

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
TRP-01	Agency-only trips	Agency	Must	MVP	Only verified agencies create trips; hotels and hostels never see the option	Built	R29, R30, DEC-006	BR-TRP-01
TRP-02	Trip itinerary and inclusions	Agency	Must	MVP	Day-by-day itinerary, inclusions, exclusions, meeting point, group limits, cancellation policy step	5 steps, no policy step	R29, R31	
TRP-03	Departures with seats and price	Agency	Must	MVP	Each departure has dates, seats total, price per person, minimum group, status (scheduled, guaranteed, cancelled, completed)	Array of dates	R36, DEC-006	BR-TRP-02

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
TRP-04	Trips go through review	Agency, Moderator	Must	MVP	Same review flow as listings	Live at once	DEC-003	BR-LST-02
TRP-05	Search trips by date	Guest	Should	MVP	Trip search filters departures overlapping the searched dates	Missing	R43	
TRP-07	Canonical trip types	Agency, Guest	Must	MVP	Every trip has one of 8 types (group tour, private trip, adventure, Saharan expedition, cultural, hiking, relaxation, religious); used by trip creation, search filters, cards and admin	Built in both apps 2026-11-04	DEC-015	BR-LST-05
TRP-06	Cancel a departure under minimum group	Agency	Should	R2	Agency cancels a departure; all its bookings are cancelled with full refund and notifications	Missing	RES A.4	BR-TRP-03

AVL: Availability and calendar (6)

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
AVL-01	No double booking for single units	System	Must	MVP	Database refuses two active bookings on the same unit with overlapping half-open ranges, even under concurrent requests	App-level check only	O4, RES A.5	BR-AVL-01
AVL-02	Pooled inventory for rooms, beds, seats	System	Must	MVP	Per day or slot: held + booked never exceeds total, enforced by a database check	In-process count	R4, RES A.5	BR-AVL-02
AVL-03	Host calendar: block days and set prices	Host	Must	MVP	Block or unblock days and set per-day price, per listing or per room type	Per listing only	INV-F 3.5, R15	BR-AVL-04
AVL-04	Availability window and partial matches	Guest	Must	MVP	Search shows listings overlapping the dates with a 'available X to Y' hint; booking only inside free nights	Built	R38-R40	BR-AVL-05
AVL-05	Temporary holds during payment	System	Should	MVP	A hold blocks inventory for 15 min while a guest pays; expires automatically	Missing	RES A.5	BR-AVL-06
AVL-06	External calendar sync (iCal)	Host	Could	Later	Import and export iCal feeds with SSRF protection	Missing	RES D.1 API7	

PRC: Pricing and quotes (5)

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
PRC-01	Server-side price quote	Guest	Must	MVP	Quote lists nights x per-day price, cleaning fee, taxes, discounts, total; has an ID and expires in 30 min	Client and server compute separately	RES A.5	BR-PRC-01
PRC-02	No guest service fee	Guest	Must	MVP	Guests never pay a BLADUS fee; quote shows 'No BLADUS fees'	Built	Policy 2026-10	BR-PRC-02
PRC-03	Length-of-stay discounts	Host	Should	MVP	Weekly and monthly discount percentages applied in the quote	Display only	INV-A 4.1	BR-PRC-03
PRC-04	Tourist tax and VAT lines	System	Should	R2	Quote shows taxe de séjour by star rating and VAT where applicable; rates from settings	Free 'taxes' number	RES C.4	BR-PRC-04
PRC-05	Whole dinars to guests, centimes inside	System	Must	MVP	Prices shown in whole DZD; stored in centimes; rounding rule documented and tested	Integer DZD	RES B.17	BR-PAY-01

BKG: Bookings (11)

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
BKG-01	Instant booking	Guest	Must	MVP	Instant mode: booking confirmed once the commission deposit check passes (model M1)	Confirmed at once	R18, O2	BR-BKG-01
BKG-02	Request to book	Guest, Host	Must	MVP	Request mode creates pending; host accepts or declines within 24 h (setting)	Built	R22, O6	BR-BKG-03
BKG-03	Automatic expiry of requests	System	Must	MVP	A scheduled job expires pending requests at the deadline and notifies both sides	Lazy on read	D-8	BR-BKG-04
BKG-04	Automatic completion after check-out	System	Must	MVP	Job marks confirmed bookings completed after check-out; opens the review window	Lazy on read	INV-F 4.7	BR-BKG-05
BKG-05	Unified booking state machine	System	Must	MVP	Statuses: pending, confirmed, declined, expired, cancelled, completed, disputed; payment status is separate	Two sets	D-9	BR-BKG-06
BKG-06	Booking snapshot	System	Must	MVP	At confirmation store price lines, commission rate and amount, cancellation policy, listing title and address	Partly	INV-A 4.1	BR-BKG-07
BKG-07	Guest and host booking lists	Guest, Host	Must	MVP	Paginated lists filtered by status and dates; host list scoped to active organisation	Built	INV-F 3.4	
BKG-08	Booking reference	All	Must	MVP	Human reference BLD-YYYY-XXXXXX unique, shown everywhere	Admin only	INV-A 2.1	
BKG-09	Trip departure booking	Guest	Must	MVP	Books seats on a departure; per-person price; respects agency approval mode	Built (string dates)	R35	BR-TRP-02
BKG-10	Idempotent booking creation	System	Must	MVP	POST with the same Idempotency-Key returns the same booking	Missing	RES B.12	BR-SYS-01
BKG-11	Disputes	Guest, Host, Support	Should	R2	Either party opens a dispute within 14 days after check-out with statement and files; support resolves	Admin only	U8, D-9	BR-CAN-05

CAN: Cancellations and refunds (4)

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
CAN-01	Guest cancellation by policy	Guest	Must	MVP	Refund percentage computed from the snapshot policy and time before check-in	Built	INV-F 4.8	BR-CAN-02
CAN-02	Host cancellation	Host	Must	MVP	Host cancels with reason; guest gets full refund of anything paid; host commission returned; host cancellation counted	Built partly	D-4	BR-CAN-03
CAN-03	Staff cancellation and refund	Support, Finance	Must	MVP	Staff cancel or refund with reason; partial refunds add up and never exceed amount paid	Built (admin)	INV-A 4.5	BR-CAN-04
CAN-04	Commission reversal rules	System	Must	MVP	Decline and host cancel: full reversal; guest cancel: commission on the non-refunded part kept	Built (website)	D-4	BR-PAY-05

PAY: Payments, wallet and ledger (12)

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
PAY-01	Double-entry ledger	System	Must	MVP	Every money movement is a transaction whose entries sum to zero; entries never updated or deleted	Single-sided lists	RES A.5, D-16	BR-PAY-02
PAY-02	Prepaid commission wallet (model M1)	Host	Must	MVP	Each host account (user or organisation) has a commission wallet; balance derived from the ledger	Built (website)	DEC-001	BR-PAY-03
PAY-03	Commission deduction on confirmation	System	Must	MVP	On confirm: commission = rate x accommodation subtotal (excl. cleaning, taxes), posted once per booking	Built (website)	R63, DEC-002	BR-PAY-04
PAY-04	Commission rate resolution	System	Must	MVP	Host override if set, else host-type default from settings; rate snapshot on booking	Two sources	D-5	BR-PAY-06
PAY-05	Manual wallet top-up by staff	Finance	Must	MVP	Amount 1 to 10,000,000 DZD, method, receipt reference unique per method, optional four-eyes above a threshold	No duplicate check	R60, D-17	BR-PAY-07
PAY-06	Online wallet top-up via gateway	Host	Should	R2	Host tops up with CIB/Edahabia through Chargily; credited only after a verified webhook	Missing	RES C.1	BR-PAY-08
PAY-07	Negative balance handling	Host, Finance	Must	MVP	Negative balance shows host banner, admin flag and dashboard item; below a threshold new instant bookings switch to request	Banner only	R65	BR-PAY-10
PAY-08	Wallet history for hosts	Host	Must	MVP	Host sees balance and paginated entries with booking links	Built	R64	
PAY-09	Gateway webhooks	System	Should	R2	Webhook payloads stored raw, signature verified, deduplicated by event id, processed once	Missing	RES D.1 API10	BR-SYS-02
PAY-10	Payouts and guest payments (model M4)	Finance	Could	Later	Schema supports guest payments, escrow and host payouts; disabled until legal approval	Admin mock	DEC-001	BR-PAY-11
PAY-11	Commission invoices to hosts	Finance	Should	R2	Monthly invoice per host with sequential number, NIF/NIS of both sides, HT, VAT, TTC; PDF kept 10 years	Missing	RES C.4	BR-PAY-12
PAY-12	Money as centimes with currency	System	Must	MVP	All amounts BIGINT centimes + currency code DZD	Integer DZD	RES B.17	BR-PAY-01

REV: Reviews (5)

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
REV-01	Write a review after a stay	Guest	Must	MVP	Only for a completed booking, within 14 days, one per booking; 5 sub-ratings + comment	Missing (read only)	INV-F gaps	BR-REV-01
REV-02	Host reviews guest	Host	Should	R2	Host reviews guest under the same window; double-blind reveal	Missing	RES A.1	BR-REV-02
REV-03	Host reply	Host	Should	MVP	One public reply per review, editable for 48 h	Display only	INV-A 4.8	
REV-04	Report and moderate reviews	Guest, Host, Moderator	Must	MVP	Anyone reports with a reason; moderator hides, restores or deletes with reason; allowed moves enforced	No state machine	INV-A 4.8	BR-REV-03
REV-05	Ratings derived from reviews	System	Must	MVP	Listing, host and organisation ratings and counts recomputed from published reviews	Stored numbers	O10	BR-REV-04

MSG: Messaging (6)

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
MSG-01	Conversations between guest and host	Guest, Host	Must	MVP	Thread per listing inquiry or booking; org members with access see org threads	Built	O8	
MSG-02	Incremental sync	Guest, Host	Must	MVP	Endpoint returns messages changed since a cursor; clients poll every 5 to 15 s while open	Full fetch	RES B.6	
MSG-03	Read state and unread counts	Guest, Host	Must	MVP	Per participant last-read pointer; unread counts per thread and total	Built	O8	
MSG-04	Contact detail masking before confirmation	System	Should	MVP	Phone numbers, emails, RIP/CCP numbers and WhatsApp links masked until a booking is confirmed	Missing	RES D.2	BR-MSG-01

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
MSG-05	Attachments	Guest, Host	Could	R2	Images and PDFs up to 10 MB, scanned	Missing		
MSG-06	Staff messages and broadcast	Support	Should	R2	Staff message a user or segment with permission and audit	Mocked, no permission	INV-A 6.2	

NTF: Notifications (4)

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
NTF-01	In-app notifications	All	Must	MVP	Events (request, accept, decline, expiry, cancel, review, verification) create notifications; mark read, read all	Seeds only	INV-F 3.7	BR-NTF-01
NTF-02	Email and SMS notifications	All	Must	MVP	Critical events also by email or SMS in the user's language; provider behind an interface	Missing	RES C.2	BR-NTF-02
NTF-03	Notification preferences	All	Should	R2	User opts in or out per channel for non-critical events; marketing needs explicit opt-in	Missing	INV-A 6.2	BR-PRV-03
NTF-04	Outbox pattern	System	Must	MVP	Events written in the same transaction as the change, then sent by workers with retries	Missing	U5	BR-SYS-03

SRC: Search and geography (6)

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
SRC-01	Search with filters and facets	Visitor	Must	MVP	Filters: vertical, wilaya, commune, dates, guests, price, property type, amenities, rating, instant; facet counts returned	Built (in memory)	INV-F 1.1, U4	BR-SRC-01
SRC-02	Map search	Visitor	Must	MVP	Bounding box filter and up to 500 pins; public points only	Built	INV-F 1.1	
SRC-03	Text search in three languages	Visitor	Should	MVP	Title and place search tolerant to accents and transliteration (trigram)	Static list of 10 cities	RES B.7	
SRC-04	69 wilayas and 1,541 communes	System	Must	MVP	Reference tables seeded from el-amin (CC0) + ihahachi (MIT) with source and valid_from	58 or 68	RES C.5	BR-SRC-02
SRC-05	Sorting and pagination	Visitor	Must	MVP	Sort by relevance, price, rating, newest; cursor or page pagination capped at 50	Built partly		
SRC-06	Hotels as their own vertical	Visitor	Must	MVP	Vertical filter includes hotel; analytics roll hotel into the home family	Inconsistent	DEC-005	BR-SRC-04

MED: Photos and documents (3)

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
MED-01	Photo upload with direct-to-storage	Host	Must	MVP	Presigned upload to quarantine, validation, re-encode, EXIF stripped, renditions, perceptual hash	Placeholders	RES B.8	BR-MED-01
MED-02	Private documents	Host, Moderator	Must	MVP	KYC files in a private bucket, virus scanned, short-lived signed URLs, every view logged	Placeholders	RES C3.5	BR-MED-02
MED-03	Photo ordering and cover	Host	Must	MVP	Host orders photos and picks a cover; minimum 5 photos for homes	Built partly		

MOD: Moderation and trust (8)

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
MOD-01	Moderation queues with waiting time	Moderator	Must	MVP	Queues for hosts, listings, trips, documents, reviews, disputes with count, oldest submitted_at, priority	Partly hardcoded	U1, INV-A 4.10	BR-MOD-01
MOD-02	Change requests to hosts	Moderator, Host	Must	MVP	Moderator sends checklist items and a note; host sees them and resubmits	Admin only, mocked	U6	BR-MOD-02
MOD-03	Reason catalogue	Moderator	Should	MVP	Preset reasons per action with localized host-facing text; free note allowed	Free text	U5	
MOD-04	Bulk actions with per-item results	Moderator	Should	R2	Bulk approve returns success or error per item; one audit entry per item	Client loop	U3	
MOD-05	Undo window	Moderator	Could	R2	Approve and hide can be reversed within 8 s via the inverse transition, audited	Toast only	U2	
MOD-06	Cascade on host suspension	System	Must	MVP	Suspending or banning a host pauses its listings and trips; future bookings flagged for support	No cascade	INV-A 4.6	BR-MOD-03
MOD-07	Risk flags	System	Should	R2	Rules flag new account + high value, duplicate photos, contact info in messages; flags create review cases	Missing	RES A.5, D.2	BR-MOD-04
MOD-08	Document-level review	Moderator	Must	MVP	Approve or reject each document separately; host approval requires all required documents approved	Approves whole host	INV-A 6.4-7	BR-HST-03

STF: Staff, roles and audit (6)

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
STF-01	Staff roles and permissions in the database	Super admin	Must	MVP	5 roles, 34+ permissions stored and enforced by the API, deny by default	Code constants	U16, INV-A 1.1	BR-STF-01
STF-02	Staff invitation and activation	Super admin	Must	MVP	Invite sends a one-time link; staff sets password and MFA; deactivation revokes sessions immediately	No acceptance	INV-A 4.15	
STF-03	Append-only audit log	System	Must	MVP	Every staff mutation, every host money or team action, logins and KYC views logged with actor id, IP, before/after; no update or delete	Admin memory only	U13	BR-STF-02
STF-04	Actors by id, not name	System	Must	MVP	Ledger, payouts, disputes, config publishes reference staff user ids	Names	INV-A 6.1-6	BR-STF-03
STF-05	Separate admin API surface	System	Must	MVP	/api/v1/admin/* only for staff tokens; never reachable with a guest or host session	n/a	RES D.1 API5	
STF-06	Audited CSV exports	Analyst, Finance	Must	MVP	Exports cover full filtered set, formula-injection safe, audited with filters and row count; large exports run as jobs	Built (admin)	INV-A 4.16	BR-STF-04

CFG: Platform settings and onboarding config (5)

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
CFG-01	Platform settings	Super admin	Must	MVP	One settings record: commission defaults, request TTL, instant-book switch, maintenance mode, session timeout, locales	Field mismatches	D-2, INV-A 6.4-1	BR-CFG-01
CFG-02	Versioned onboarding config	Super admin	Must	MVP	Draft and published versions per host type; publish creates an immutable version; rollback = republish older	Global counter, no history	D-13, D-15	BR-CFG-02
CFG-03	Onboarding field editor	Super admin	Should	R2	Add, edit, reorder fields with canonical types; core steps and fields cannot be removed	Steps only	R48	BR-CFG-03
CFG-04	Serve published config to the website	Host	Must	MVP	GET returns the published flow; answers to custom fields stored as JSON and validated against that version	Not read	R49	BR-HST-02
CFG-05	Taxonomy management	Moderator	Must	MVP	Categories, amenities, property types with en/fr/ar, order, active flag; soft delete only when unused	Hard delete	INV-A 4.12	BR-LST-04

PRM: Promotions (2)

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
PRM-01	Coupons	Moderator, Guest	Could	R2	Code, type, value, dates, usage limits, scope; redemption at quote time with per-user limit	Not consumed	INV-A 4.11	BR-PRM-01
PRM-02	Who pays the discount	Finance	Could	R2	A coupon is funded by BLADUS (reduces commission) or by the host; recorded in the ledger	n/a	Policy	BR-PRM-02

RPT: Reports and analytics (3)

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
RPT-01	KPIs by period, host type, vertical, wilaya	Analyst	Should	MVP	Bookings, gross booking value, commission, active listings, hosts; period over period from real dates	Hardcoded	R52-R55, U12	BR-RPT-01
RPT-02	Host-type by vertical cross-tab	Analyst	Could	R2	Impossible cells shown as not applicable, not zero	Missing	R54	
RPT-03	Host earnings dashboard	Host	Should	MVP	Earnings recognised at completion; shows upcoming, completed, commission paid	Shows 0	O11, DEC-010	BR-RPT-02

PRV: Privacy and legal (6)

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
PRV-01	Consent records	Visitor	Must	MVP	Terms version, privacy notice version, marketing opt-in stored with timestamp	Missing	RES C.3 18-05 art 26, 31	BR-PRV-01
PRV-02	Data subject requests	User, Staff	Must	MVP	Access export (JSON), correction, deletion by anonymisation keeping legal records; tracked with deadlines	Missing	RES D.3	BR-PRV-02
PRV-03	Data residency	System	Must	MVP	Personal data and backups stored in Algeria unless an ANPDP authorisation exists	n/a	RES C.3 (VERIFY)	BR-PRV-04
PRV-04	Retention schedule	System	Should	MVP	Jobs delete or redact KYC images after the retention window; accounting records kept 10 years	Missing	RES C3.5 (VERIFY)	BR-PRV-05
PRV-05	Breach response	Staff	Must	MVP	Runbook with a 5-day ANPDP notification clock and incident register	Missing	RES C3.1	
PRV-06	Foreign guest declaration support	Host	Could	R2	Guest nationality and ID type captured when required; host reminded of the declaration duty	Missing	RES C3.3 (VERIFY)	

NFR: Quality (non-functional) (13)

ID	Requirement	Actor	Pri.	Release	Accepted when	Today in code	Source	Rules
NFR-01	API response time	System	Must	MVP	p95 under 300 ms for reads and 600 ms for booking create at 50 requests/s on one app node	n/a	RES A.5	
NFR-02	Search response time	System	Must	MVP	p95 under 500 ms with 20,000 listings and dates	n/a		
NFR-03	Availability	System	Should	MVP	99.5% monthly for the public API; planned maintenance announced	n/a		
NFR-04	Backups and restore	System	Must	MVP	Point-in-time recovery 7 days, nightly encrypted dump in a second Algerian location, monthly restore drill; RPO 15 min, RTO 4 h	n/a	RES D.3	
NFR-05	Security baseline	System	Must	MVP	OWASP API Top 10 controls, TLS everywhere, secrets outside code, dependency scanning in CI	Partly	RES D.1	
NFR-06	Three languages and right-to-left	All	Must	MVP	Every API error and notification has en/fr/ar text; Latin digits in Arabic	Partly	R68	BR-SYS-04
NFR-07	Time and dates	System	Must	MVP	Timestamps stored in UTC; business dates in Africa/Algiers; week starts Sunday; weekend Friday-Saturday	Partly	U15	BR-SYS-05
NFR-08	API contract	System	Must	MVP	OpenAPI generated in CI; typed client shared by both apps; breaking changes need a new version	n/a	RES B.13	
NFR-09	Observability	System	Must	MVP	JSON logs with request, user and org ids; error tracking; metrics and health checks	n/a	RES B.16	
NFR-10	Test coverage	System	Must	MVP	Services 90% line coverage; property tests for money and dates; concurrency tests on real PostgreSQL	n/a	RES B.14	
NFR-11	Rate limits	System	Must	MVP	Per user and per IP throttles per endpoint group; SMS budget alarms	Login only (admin)	RES B.12	
NFR-12	Unique identifiers	System	Must	MVP	UUIDv7 primary keys plus human references; no time or random based ids	Date.now/Math.random	INV-A 6.1-3	
NFR-13	Accessibility of messages	All	Should	MVP	Errors have stable codes the apps map to localized text	Built (ActionResult codes)		

5. What is deliberately out of the MVP

Item	Why	When
BLADUS collecting guest payments and paying hosts (models M2 to M4)	Needs legal approval (payment licence)	Later, behind a decision gate
Online wallet top-up by card	Needs a gateway contract	R2
Host reviews of guests, disputes, coupons, iCal sync	Not needed to launch	R2 or Later
Realtime messaging (WebSockets)	Polling is enough at launch	R2
Commission invoices as PDFs	Needs an accountant's answer on VAT (F1)	R2

Check yourself

- [ ] What is the difference between "Must / MVP" and "Should / R2"?
- [ ] Which decision explains why there is no guest payment in the MVP?
- [ ] Find the requirement that prevents double booking. What is its acceptance criterion?
- [ ] Which requirements change what the website does today (live listings, invites, ratings)?
- [ ] Where do you edit a requirement so the CSV and this page stay identical?

04. Business rules

> **In plain words**
> A business rule is a sentence like "a guest cannot book nights that are already booked" or "BLADUS takes 12% from an individual host".
> Each rule has an ID (`BR-PAY-04`), says **where** it is enforced (the database, the backend code, or a scheduled job) and how to test it.
> Rules enforced by the database are the strongest: even a bug in the code cannot break them.
> The statuses each object can have, and the allowed moves between them, are written as tables (state machines).
> Money examples are worked out in full so anyone can check the arithmetic.

Terms used here (see glossary.md):

- **Enforced by DB**: a constraint inside PostgreSQL (unique, check, exclusion, foreign key). Cannot be bypassed.
- **Enforced by service**: a check in the Django service function that performs the action, inside a database transaction.
- **Enforced by job**: a scheduled background task (Celery) that applies the rule at a given time.
- **Basis points (bp)**: hundredths of a percent. 12% = 1,200 bp. Used to store rates without decimals.
- **Half-open range** `[a, b)` : includes `a` , excludes `b` .

Sources: website code (Appendix A section 4), admin code (Appendix B section 4), decisions DEC-001 to DEC-013 (03-requirements.md section 2). "Today" says what the apps do now.

---

1. Accounts (ACC)

ID	Rule	Enforced by	Today	Test
BR-ACC-01	Email is unique case-insensitively; phone is unique in E.164 form and must match `^\+213[5-7]\d{8}$` for Algerian mobiles (foreign numbers allowed for diaspora, flagged). A guest must have a verified phone before the first booking (DEC-012).	DB unique on `lower(email)` and `phone_e164` ; service	email only	sign up twice with `A@x.dz` / `a@x.dz` fails
BR-ACC-02	One-time codes: 6 digits, valid 10 minutes, 5 wrong attempts invalidate the code, max 3 sends per phone per hour and 10 per IP per hour. Every failure returns the same error.	service + Redis	none	6th attempt with right code fails
BR-ACC-03	Sessions: website session 30 days sliding; staff session 8 hours absolute and 30 minutes idle (setting). Logout, password change and deactivation revoke server-side immediately.	service	admin token not revocable	token rejected right after logout
BR-ACC-04	Staff must have two-factor enabled before any admin endpoint works. Hosts must have it before changing payout or refund account details.	service (permission class)	flag only	staff without MFA gets 403 `mfa_required`
BR-ACC-05	Login lock: 5 failures in 15 minutes lock 60 seconds, counted per account and per IP, shared by all servers.	Redis	in-process	two servers share the counter

2. Organisations (ORG)

ID	Rule	Enforced by	Today	Test
BR-ORG-01	A **host account** is either a user (individual host) or an organisation. One user can be an individual host and a member of several organisations (DEC-004). Each host request names the active host account and the user must hold it.	service + permission	single hostType	request with someone else's org id gets 404
BR-ORG-02	Organisation `business_category` is `agency` , `hotel` or `hostel` and never changes after creation.	DB check + service	same	update category refused
BR-ORG-03	An invitation is for one email or phone, one role, expires after 7 days, can be accepted once by a signed-in user whose verified email or phone matches.	DB unique (org, invitee, pending) + service	not acceptable	accept twice fails
BR-ORG-04	Exactly one active `owner` per organisation. Only the owner can transfer ownership, which demotes them to `admin` . The owner cannot be removed.	DB partial unique index `(org) WHERE role='owner' AND active` + service	built	two owners impossible
BR-ORG-05	Organisation role permissions: see table 2a. Agents and guides never see money.	service (django-rules)	any member does anything	agent accept request gets 403
BR-ORG-06	Changing legal name, RC, NIF, NIS or tourism licence number sets verification back to `pending` and creates a moderation case. Listings stay published meanwhile.	service	field list exists	change NIF, status becomes pending

Table 2a: organisation permissions

Permission	owner	admin	manager	agent	guide
org.edit, org.delete, members.manage	X	X (no delete, no ownership)			
listings.manage, trips.manage, calendar.manage	X	X	X		
bookings.view	X	X	X	X	X (own departures)
bookings.respond (accept, decline) DEC-009	X	X	X		
bookings.cancel_as_host	X	X			
messages.reply	X	X	X	X	X
wallet.view, invoices.view	X	X			
payout_details.edit	X				

3. Becoming a host (HST)

ID	Rule	Enforced by	Today	Test
BR-HST-01	Onboarding steps come from the published config for the host type: individual 12, agency 5, hotel 7, hostel 7 (DEC-007). Every flow contains the approval-mode step.	service (config)	hardcoded; agency missing step	agency draft without approval mode cannot submit
BR-HST-02	A draft records the config version it started from. Submitted answers are validated against that version; custom answers are stored as JSON keyed by field id.	service	browser storage	answer with unknown field id rejected
BR-HST-03	Required documents per host type (table 3a). A host is approved only when every required document is `approved`. Each document is reviewed on its own.	service	whole host approved	approve with a pending document fails
BR-HST-05	What each host account may publish (DEC-014): individual: home, vehicle; agency: experience, trip; hotel and hostel: their property (hotel or hostel vertical with room types). Checked when creating or changing the vertical of a listing and when creating a trip.	service + DB trigger	built in website	individual creating an experience gets 403 `host_offer_not_allowed`
BR-HST-04	Host verification state machine: table 3b. Reasons of at least 10 characters for reject, request changes, suspend, ban.	service	admin only	approved to applied refused

Table 3a: required documents (proposal, VERIFY WITH A PROFESSIONAL)

Host type	Required	Optional
Individual	National ID or passport, proof of ownership or lease with owner's permission	APC declaration ("chez l'habitant"), insurance, auto-entrepreneur card
Agency	Representative ID, RC, NIF, NIS, Ministry of Tourism licence	
Hotel	Manager ID, RC, NIF, NIS, classification decision or operating authorisation	
Hostel	Manager ID, RC or association status, operating authorisation	
Vehicle rental (agency)	Representative ID, RC with rental activity, NIF, NIS; per vehicle: carte grise, insurance, contrôle technique	

Table 3b: host (user or organisation) verification states

From	Allowed next	Who
applied	under_review	host submits
under_review	approved, changes_requested, rejected	staff
changes_requested	under_review (resubmit) or rejected	host / staff
approved	under_review (legal change, BR-ORG-06), suspended	system / staff
rejected	applied (after 30 days)	host
suspended	approved (unsuspend)	staff
any of the above	banned	staff with `hosts.ban`
banned	(final)	

Soft delete is separate: any state can be soft-deleted by a super admin and restored to the previous state.

4. Listings and trips (LST, TRP)

ID	Rule	Enforced by	Today	Test
BR-LST-01	A listing belongs to exactly one host account. It has one vertical: `home`, `hotel`, `hostel`, `experience`, `vehicle` (trips are separate, DEC-006). Hotels and hostels must have at least one room type to be submitted.	DB FK + service	built	hotel without room type cannot submit
BR-LST-02	Publishing needs staff approval (DEC-003). Editing title, description, photos, location, vertical, capacity or room types of a published listing creates a revision that goes to review while the published version stays visible. Price, calendar and rules changes do not need review.	service	live at once	edit title, public page unchanged until approved
BR-LST-03	Listing state machine: table 4a. Only published listings appear in search or accept bookings. Paused, suspended and archived listings keep existing bookings.	service + DB check on enum	two sets	booking on paused listing refused
BR-LST-04	Amenities, property types and categories are reference rows. A reference row in use cannot be hard-deleted; it is deactivated.	DB FK `ON DELETE RESTRICT`	free text	delete used amenity fails
BR-LST-05	Experience and trip types come from one fixed list each (DEC-015): 8 experience types and 8 trip types with en/fr/ar labels. An experience listing must have exactly one experience type; a trip exactly one trip type. Changing the lists is a product change made in one place (backend enum, served to both apps).	DB enum + check + service	built in both apps (shared list file)	experience without type refused
BR-TRP-01	Only an organisation with `business_category = agency` and verification `approved` can create trips.	service	built	hotel creates trip, 403
BR-TRP-02	A departure has `seats_total`; held + booked seats never exceed it. Price is per person and snapshot on the booking.	DB check on departure counters	per date by maxGroupSize	concurrent last-seat bookings: one wins
BR-TRP-03	Cancelling a departure cancels every booking on it with a full refund obligation and commission reversal, and notifies guests.	service	missing	all bookings cancelled

Table 4a: listing states (same for trips)

From	Allowed next	Who
draft	pending_review, deleted	host
pending_review	published, changes_requested, rejected (staff); draft (host withdraws)	

From	Allowed next	Who
changes_requested	pending_review	host
published	paused, archived (host); suspended, changes_requested (staff)	
paused	published (host, no review unless key fields changed)	
rejected	draft	host
suspended	published, rejected	staff
archived	draft	host
any except deleted	deleted (soft); staff restore to previous state	

5. Availability (AVL)

ID	Rule	Enforced by	Today	Test
BR-AVL-01	For a single unit (home, vehicle), two bookings in a blocking state (`pending` , `confirmed` , `completed` , `disputed`) cannot have overlapping half-open ranges. Back-to-back is allowed (check-out day = next check-in).	**DB exclusion constraint** `(unit WITH =, stay WITH &&) WHERE status IN (...)`	app check	50 parallel requests for the same nights: exactly 1 succeeds
BR-AVL-02	For pooled units (room types, hostel beds, experience slots, trip departures), each night or slot has `total` , `held` , `booked` , with `held + booked <= total` and all values `>= 0` . Reserving is one atomic `UPDATE ... WHERE held + booked + n <= total` .	**DB check** + service	in process	inventory never negative
BR-AVL-03	A hostel bed room type books one unit per guest; a hotel room type books one unit per room requested. Capacity per unit limits guests per unit.	service	built	3 guests in a dorm use 3 units
BR-AVL-04	A night is unavailable if blocked by the host, booked, outside the listing's availability window, inside the advance notice period, or the listing is not published. Blocks may be per listing or per room type.	service + DB	per listing only	blocked night not bookable
BR-AVL-05	Search keeps a listing whose availability overlaps the searched dates and returns the largest free sub-range; booking is allowed only on fully free nights with stay length between min and max nights (defaults 1 and 90).	service	built	partial match shows sub-range
BR-AVL-06	A hold lasts 15 minutes. Expired holds are released by a job every minute and ignored on read.	job + service	none	hold released after 15 min

6. Pricing (PRC)

ID	Rule	Enforced by	Today	Test
BR-PRC-01	The server computes every price. A quote lists per-night prices (base or day override), length-of-stay discount, subtotal, cleaning fee, taxes, total. A quote expires after 30 minutes and is re-validated at booking; if anything changed the guest sees the new price.	service	client and server both	tampered price ignored
BR-PRC-02	Guests pay no BLADUS fee. Guest total = subtotal + cleaning fee + taxes.	service	built	quote has no fee line
BR-PRC-03	Discount: weekly (7+ nights) and monthly (28+ nights) percentages set by the host, 0 to 90%; only the larger applies; applied to the subtotal before commission.	service + DB check	display only	30 nights uses monthly only
BR-PRC-04	Taxes (R2): taxe de séjour per person per night by star rating from settings; VAT shown only if the host is VAT-registered. Rates live in settings with validity dates. VERIFY WITH A PROFESSIONAL.	service	free number	rate change does not alter old quotes

7. Bookings (BKG)

ID	Rule	Enforced by	Today	Test
BR-BKG-01	Instant booking confirms immediately if the listing resolves to instant mode, the guest has a verified phone, the dates are free, and the host's wallet is above its credit floor (BR-PAY-10). Otherwise it becomes a request.	service	confirmed at once	host below floor: becomes pending
BR-BKG-02	Approval mode resolution: if the platform switch forces request mode, request; else the listing's own mode if set; else the host account default; else `request` .	service	listing only	org default applied to new listing
BR-BKG-03	A request blocks the dates (BR-AVL-01) until accepted, declined, cancelled or expired. Only users with `bookings.respond` may accept or decline. Decline needs a reason code.	service + DB	any member	agent cannot accept
BR-BKG-04	A pending request expires `request_ttl_hours` (default 24, DEC-008) after creation, or at check-in time if sooner. A job runs every 5 minutes.	job	lazy	expired after 24 h without any read
BR-BKG-05	A confirmed booking becomes completed at check-out time + 2 hours (Africa/Algiers) by a job; this opens the review window and counts host earnings (DEC-010).	job	lazy	completion without page view
BR-BKG-06	Booking state machine: table 7a. Payment state is a separate field (M1: `not_collected`).	DB enum + service	two sets	confirmed to pending refused
BR-BKG-07	At creation the booking stores a snapshot: price lines, nights, guests, cancellation policy windows, commission rate (bp) and base, listing title, host account, address (revealed to the guest only once confirmed). Later edits to the listing never change it.	service	partial	listing price change keeps booking total

Table 7a: booking states

From	To	Trigger
(new)	pending	request mode

From	To	Trigger
(new)	confirmed	instant mode (BR-BKG-01)
pending	confirmed	host accepts
pending	declined	host declines
pending	expired	job (BR-BKG-04)
pending	cancelled	guest withdraws
confirmed	cancelled	guest, host or staff cancels (section 8)
confirmed	completed	job (BR-BKG-05)
confirmed, completed	disputed	guest or host opens a dispute (R2), until 14 days after check-out
disputed	completed or cancelled	support resolves

Final states: declined, expired, cancelled, completed (once the dispute window has passed).

8. Cancellations and refunds (CAN)

Under DEC-001 the guest paid the host directly, so a "refund" is an amount the **host owes the guest**. BLADUS records it, shows it to both sides, and adjusts its commission. When BLADUS collects payments later (M2 to M4), the same amounts drive real refunds.

ID	Rule	Enforced by	Today	Test
BR-CAN-01	Policy templates (DEC-011); "days before" = calendar days in Algeria between today and check-in: **flexible** 100% if 1+ days before, 50% on arrival day; **moderate** 100% if 5+ days, 50% from 4 to 1 day, 0% on arrival day; **strict** 100% if 14+ days, 50% from 13 to 7, 0% within 6 days. Percentages apply to the accommodation subtotal; cleaning fee and its taxes are always refunded before check-in. Guests cannot cancel after check-in time on arrival day (support can).	service (pure function, property tests)	built (same values)	table-driven tests per day
BR-CAN-02	Guest cancellation: refund = policy % x subtotal + cleaning fee + matching taxes.	service	built	example 8a
BR-CAN-03	Host cancellation of a confirmed booking: full refund owed to the guest, full commission reversal, host cancellation counter +1; 3 in 12 months open a moderation case.	service	partly	example 8a
BR-CAN-04	Staff refunds: amount between 1 centime and (total minus already refunded); partial refunds add up; reason required; audited.	DB check `refunded <= total` + service	admin built	over-refund refused
BR-CAN-05	Disputes (R2): opened until 14 days after check-out by guest or host; resolution `refund_full`, `refund_partial` or `declined`; resolution note required.	service	admin only	dispute after 15 days refused

Example 8a (Tipaza villa from the click-through, individual host, 12%):

- 3 nights x 18,500 = **55,500 DZD** subtotal; cleaning 4,000; total **59,500 DZD**.
- Commission at confirmation: 12% x 55,500 = **6,660 DZD** deducted from the host's wallet.
- Guest cancels 3 days before, moderate policy: 50% of 55,500 = 27,750 + cleaning 4,000 = **31,750 DZD** owed back to the guest.
- BLADUS keeps commission only on the part the host keeps: 12% x 27,750 = 3,330, so **3,330 DZD** goes back to the wallet.
- If the host cancels instead: the guest is owed 59,500 and all 6,660 goes back to the wallet.

9. Money, wallet and ledger (PAY)

ID	Rule	Enforced by	Today	Test
BR-PAY-01	Amounts are stored as whole centimes (`BIGINT`) with a currency code (`DZD`). Prices shown to people are whole dinars. Rates are basis points. Rounding: half up at each line; a total is always the sum of its rounded lines.	DB types + Money type	integer DZD	property test: sum of lines == total
BR-PAY-02	Ledger: every posting is one `ledger_transaction` with 2+ entries; entries of a transaction sum to 0 per currency; entries are never updated or deleted (corrections are new reversing transactions).	DB deferred constraint trigger + no UPDATE/DELETE grant	single-sided	unbalanced insert fails at commit
BR-PAY-03	Each host account has one commission wallet account in the ledger. Balance = sum of its entries (cached in a balance row updated in the same transaction).	DB + service	built (website)	cache equals sum
BR-PAY-04	Commission = round_half_up(commission_base x rate_bp / 10,000); base per DEC-002 (subtotal after discount; trips and experiences: seats x price). Posted once per booking at confirmation (unique key booking + kind).	DB unique + service	built (website)	confirm twice posts once
BR-PAY-05	Commission reversal: decline and expiry (nothing was posted); host cancel and staff cancel without guest fault: full; guest cancel: reverse the commission on the refunded subtotal share.	service	built (website)	example 8a
BR-PAY-06	Rate resolution: host account override (if set) else host-type default from settings (individual 1,200 bp, agency 1,500, hotel 1,800, hostel 1,000). Changing defaults affects new bookings only. Overrides can be cleared.	service	two sources	clearing an override returns to the default
BR-PAY-07	Manual top-up: amount 1 to 10,000,000 DZD; method in `ccp_transfer`, `bank_transfer`, `baridimob`, `cash_office`; receipt reference unique per method; top-ups over 500,000 DZD need a second finance user to approve (four-eyes).	DB unique + service	duplicates allowed	same receipt twice fails
BR-PAY-08	Online top-up (R2): credited only from a verified gateway webhook, once per gateway event id.	DB unique `(provider, event_id)`	none	replayed webhook ignored
BR-PAY-09	Changing payout or refund account details requires 2FA, notifies the old email and phone, and blocks payouts for 72 hours.	service	none	payout during hold refused
BR-PAY-10	Credit floor: a host wallet may go below zero down to `-credit_limit` (setting per host type, default 20,000 DZD). Below 0: banner and admin flag. Below the floor: instant bookings become requests and the host cannot accept new requests until topped up.	service	banner only	accept refused below floor

ID	Rule	Enforced by	Today	Test
BR-PAY-11	Models M2 to M4 (guest payments, escrow, payouts) are switched off by a setting until legal approval; their ledger accounts exist from day one.	settings	admin mock	endpoints return 404 when off
BR-PAY-12	Commission invoices (R2): monthly per host account, sequential numbers without gaps per year, never deleted (cancelled by a credit note). VERIFY WITH A PROFESSIONAL.	DB sequence + service	none	numbers continuous

Ledger accounts

Account	Type	Used by
`host_wallet:{host_account}`	liability (BLADUS holds the host's deposit)	M1
`platform_cash:{method}`	asset (money received by BLADUS)	top-ups
`platform_commission_revenue`	revenue	commission
`vat_payable`	liability	R2
`guest_receivable`, `escrow`, `host_payable`, `payout_clearing`	various	M2 to M4 (switched off)

Example 9a: postings

Event	Debit	Credit	Amount (DZD)
Finance records a CCP top-up	platform_cash:ccp	host_wallet:H	50,000
Booking confirmed	host_wallet:H	platform_commission_revenue	6,660
Guest cancels (example 8a)	platform_commission_revenue	host_wallet:H	3,330

Wallet balance after: 50,000 - 6,660 + 3,330 = **46,670 DZD**.

10. Reviews (REV)

ID	Rule	Enforced by	Today	Test
BR-REV-01	A guest review needs a `completed` booking of that guest, within 14 days after check-out, one per booking. Ratings 1 to 5: overall plus cleanliness, accuracy, communication, location, value.	DB unique `(booking, author_role)` + check + service	read only	second review refused
BR-REV-02	Double-blind (R2): both reviews published when both are submitted or when the 14 days end. No edits after publication.	job + service	none	hidden until both
BR-REV-03	Review states: `submitted` (waiting), `published`, `flagged`, `hidden`, `removed`. Moderator moves need a reason; `removed` only by super admin.	service	no state machine	restoring a removed review refused
BR-REV-04	Ratings and counts on listing, host account and organisation are computed from published reviews by a job after each change; never typed in.	job	stored numbers	rating equals average

11. Messaging and notifications (MSG, NTF)

ID	Rule	Enforced by	Today	Test
BR-MSG-01	Before a booking is confirmed, phone numbers, emails, CCP/RIP numbers and messaging-app links in messages are masked for the other side; repeated attempts flag the thread for trust review.	service	none	`0555 12 34 56` shown masked
BR-NTF-01	Every booking state change, verification decision, review publication and wallet going below 0 creates an in-app notification for the people concerned.	service via outbox	seeds	decline notifies guest
BR-NTF-02	Critical events (booking confirmed, declined, expired, cancelled; one-time codes; verification result) also go by email or SMS in the user's language. Non-critical events respect preferences.	job	none	SMS sent once per event

12. Search and geography (SRC)

ID	Rule	Enforced by	Today	Test
BR-SRC-01	Search returns only published listings of approved, non-suspended host accounts. Facet counts are computed on the filtered set without the facet's own filter.	service	in memory	suspended host hidden
BR-SRC-02	Geography: 69 wilayas, 1,541 communes (DEC-013). Every listing references a commune; the wilaya derives from it. Legacy 58-wilaya codes kept for mapping.	DB FK	58 / 68	listing without commune refused
BR-SRC-03	The exact point is shown only to the guest of a confirmed booking and to the host; everyone else gets the public point (random offset up to 500 m, fixed per listing).	service	exact point public	anonymous response has no exact point
BR-SRC-04	`hotel` is a search vertical; analytics group `home`, `hotel`, `hostel` as "stays" (DEC-005).	service	inconsistent	counts add up

13. Files (MED)

ID	Rule	Enforced by	Today	Test
BR-MED-01	Photos: JPEG, PNG, WebP, HEIC; max 15 MB; re-encoded; metadata (incl. GPS) stripped; min 5 photos for homes and 3 per room type; duplicates detected by perceptual hash.	job	placeholders	GPS removed
BR-MED-02	Documents: PDF, JPEG, PNG; max 10 MB; virus scanned; private; signed URL valid 5 minutes; each view audited.	job + service	placeholders	URL fails after 5 min

14. Moderation (MOD)

ID	Rule	Enforced by	Today	Test
BR-MOD-01	Queue priority: hosts high after 3 days waiting, listings and trips after 2 days, documents after 3 days; flagged reviews and disputes always high. Waiting time counts from the last submission, not creation.	service	creation time used	resubmission resets the clock
BR-MOD-02	A change request lists items (from a checklist) and a note; the host sees exactly those; the resubmission links to the request.	service	mocked	items visible to host
BR-MOD-03	Suspending or banning a host account pauses all its listings and trips, blocks new bookings, and opens a support case for future confirmed bookings. Unsuspending does not republish automatically.	service	no cascade	listings paused
BR-MOD-04	Risk flags (R2) open a case; they never silently block a user.	service	none	flag creates case

15. Staff and audit (STF)

ID	Rule	Enforced by	Today	Test
BR-STF-01	Staff permissions come from the role (5 roles, matrix in Appendix B section 1.1) stored in the database; checked on every admin endpoint; deny by default.	service	code constants	analyst refund gets 403
BR-STF-02	Audit log is insert-only (no UPDATE or DELETE grant for the app's database role); each row stores actor id, actor role, action, object type and id, before/after (personal data minimised), IP, request id, and a hash of the previous row.	DB grants + trigger	memory	update fails
BR-STF-03	Every record of who did something stores a user id (foreign key), never a display name.	DB FK	names	
BR-STF-04	Exports: values starting with `= + - @ TAB CR` are prefixed with `'`; need `reports.export` plus the view permission of the dataset; audited with filters and row count.	service	built	formula neutralised

16. Settings and config (CFG)

ID	Rule	Enforced by	Today	Test
BR-CFG-01	Settings are one versioned record; each change creates a new version with author and time; only whitelisted keys with validated values.	service	shallow merge	unknown key refused
BR-CFG-02	Onboarding config: one draft and many immutable published versions per host type; publishing copies the draft into the next version number for that host type.	DB unique `(host_type, version)`	global counter	old version still readable
BR-CFG-03	Field types are canonical: `text, textarea, number, select, multi_select, image_upload, file_upload, toggle, location, amenities`. Core steps and fields cannot be removed or have their type changed.	service	two schemas	removing a core step refused

17. Promotions (PRM), reports (RPT), privacy (PRV), system (SYS)

ID	Rule	Enforced by	Today	Test
BR-PRM-01	Coupon (R2): unique code (case-insensitive), valid dates, total and per-user limits; redemption counted when the booking is confirmed, released if cancelled before check-in.	DB unique + service	not used	limit reached refused
BR-PRM-02	Coupon funding: BLADUS-funded reduces the commission (never below 0); host-funded reduces the subtotal. Recorded in the ledger.	service	n/a	
BR-RPT-01	KPIs use real date windows in Africa/Algiers; gross booking value = sum of totals of confirmed and completed bookings by check-in date; revenue = net commission.	service	hardcoded	
BR-RPT-02	Host earnings = subtotal + cleaning fee - commission - refunds owed, counted when completed (DEC-010).	service	shows 0	
BR-PRV-01	Store the accepted terms and privacy versions with time and IP; marketing only with explicit opt-in, withdrawable any time.	DB	none	
BR-PRV-02	Deletion request: anonymise the profile (name, contacts, photos, ID files) within 30 days; keep bookings, ledger and invoices with a pseudonymous id for the legal period. VERIFY WITH A PROFESSIONAL.	job	none	

ID	Rule	Enforced by	Today	Test
BR-PRV-03	Notification preferences cannot switch off critical messages (BR-NTF-02).	service	none	
BR-PRV-04	Personal data and its backups stay in Algerian data centres unless an ANPDP authorisation is recorded. VERIFY WITH A PROFESSIONAL.	operations	n/a	
BR-PRV-05	Retention: ID images deleted 90 days after the verification decision (keep the decision, document type and a hash of the number); messages 3 years after last activity; accounting 10 years. VERIFY WITH A PROFESSIONAL.	job	none	
BR-SYS-01	Every create or money endpoint accepts an `Idempotency-Key`; same key and same body returns the stored response; same key with a different body returns 422; a concurrent request with the same key returns 409. Keys kept 24 h.	DB unique + service	none	retry returns the same booking
BR-SYS-02	Incoming webhooks are stored raw before processing, signature checked, processed once.	DB unique	none	
BR-SYS-03	Side effects (notifications, emails, search refresh) are written to an outbox in the same transaction and sent after commit with retries and backoff.	DB + job	none	a rolled-back change sends nothing
BR-SYS-04	API errors have a stable code, a field (if any) and a message in the requested language (en, fr, ar).	service	ActionResult codes	
BR-SYS-05	Timestamps in UTC (`timestamptz`); stay dates as `date` in Africa/Algiers; week starts Sunday; Friday and Saturday are the weekend for pricing rules.	DB types	mixed	

Check yourself

- [ ] Which rules are guaranteed by the database itself, and why does that matter for BR-AVL-01?
- [ ] Recompute example 8a for a strict-policy guest who cancels 10 days before check-in.
- [ ] What happens to a host's instant bookings when the wallet is below the credit floor?
- [ ] Which organisation roles can accept a booking request?
- [ ] Why is the exact address hidden until a booking is confirmed?
- [ ] Why does a correction in the ledger add a new transaction instead of editing the old one?

Diagrams

> **In plain words**
> Pictures of the planned system: who uses it, what runs where, how statuses change, how a booking flows step by step, and how the tables link.
> Each picture has a short caption and a legend explaining its symbols.
> The `.mmd` files are the editable sources (Mermaid text). The `rendered/` folder holds SVG (sharp at any zoom) and PNG versions.
> The table diagrams (`erd-*`) are generated from ../schema.dbml, so they always match the schema.

How to re-render: `python ../_tools/build_schema_docs.py` (ERD sources), then `python ../_tools/render_diagrams.py` (needs `@mermaid-js/mermaid-cli` installed in a tools folder outside the repo and a local Chrome; see the script header).

Legends

Diagram type	How to read it
Flowchart (01, 02, 03, 30, 40)	Boxes are systems or parts; rounded boxes are people; cylinders are data stores. A solid arrow is a call or a flow; a dotted arrow is optional, later (R2) or asynchronous. Grey frames group things that run together.
State diagram (10 to 13)	Each box is a status. An arrow is an allowed move, labelled with who or what causes it. The black dot is the start; the circled dot is the end. A move not drawn is forbidden.
Sequence diagram (20 to 24)	Columns are actors (people) and participants (systems). Time goes down. Solid arrows are requests, dashed arrows are replies, open arrows are messages sent later by a worker. Numbered steps match the text. `alt` boxes show alternative paths.
ERD (erd-*)	Each box is a table with its columns (type, name, and PK = primary key, FK = link, UK = unique). A line links a table to the one it points to; the label is the linking column. `

Architecture

File	Caption
01-system-context	Who uses BLADUS and which outside services it talks to. Guests and hosts use the website, staff use the admin app; both call one Django API. The payment gateway is dotted because online payment comes in release R2.
02-containers	What runs where. Everything holding personal data runs in Algerian data centres (research C.3, to be confirmed by a lawyer). The Next.js servers keep the login cookie and call Django on the browser's behalf (BFF). Workers send messages and run timed jobs.
03-django-apps	The backend is one application split into 17 apps plus a small `core`, arranged in layers. An app may only use apps in lower layers (checked automatically by import-linter); for example bookings may use pricing and the ledger, never the reverse. Events (emails, search refresh) go through the outbox instead of direct calls. Details in 07-architecture.md.
30-money-flow	Where money goes. At launch (M1) the guest pays the host, the host keeps a deposit at BLADUS, and BLADUS takes its commission from that deposit. M4 (BLADUS collects and pays out) is drawn for later and needs a licence.
40-document-map	How the blueprint documents feed each other, from inventory to tests.

Lifecycles (state machines)

File	Caption
10-booking-states	Every status a booking can have. A request waits up to 24 hours for the host; a confirmed booking becomes completed automatically after check-out. `awaiting_payment` stays unused until online payment exists. Rules BR-BKG-01 to 07.
11-listing-states	A listing or trip is checked by staff before it is published (DEC-003). Hosts can pause; staff can suspend or ask for changes. Deleting is reversible by staff.
12-host-verification-states	A host (person or organisation) is approved only when all required documents are approved. Changing legal details sends it back to review.
13-review-states	A review is written after a completed stay, published when both sides wrote or after 14 days, and can be hidden after a report.

Key flows (sequences)

File	Caption
20-seq-instant-booking	An instant booking from price quote to confirmation in one database transaction; the database itself refuses overlapping dates; the commission is posted to the host's wallet; messages go out after the save.
21-seq-request-to-book	A request: the host accepts, declines, or a timed job expires it after 24 hours. Dates stay blocked while it waits.
22-seq-wallet-topup	A host tops up the commission deposit by transfer; finance records it with the receipt reference; large amounts need a second person.
23-seq-host-onboarding	Becoming a host: the steps come from the published configuration, the draft is saved on the server, documents go to private storage, staff verify.
24-seq-message-sync	Messages are fetched with a cursor so polling is cheap; phone numbers are masked until a booking is confirmed (anti-fraud).

Database (ERDs, generated)

File	Caption
erd-00-overview	The 26 central tables and their main links. Start here. `host_account` sits between people and organisations on one side and listings, trips and wallets on the other.
erd-01-accounts	People, staff roles and permissions, consents, favourites.
erd-02-geography	69 wilayas, dairas and 1,541 communes.
erd-03-host-accounts-and-organisations	Host accounts, organisations, team members, invitations, verification documents, payout accounts.
erd-04-taxonomy	Shared lists: categories, property types, amenities, cancellation policies.
erd-05-catalogue-listings-and-room-types	Listings, edits under review, vehicle and experience details, room types, photos, featured places.
erd-06-inventory-and-calendar	What is free per day (homes, vehicles), per night and room type (hotels, hostels) and per time slot (experiences).
erd-07-trips-agency-only	Agency trips, day-by-day itinerary, dated departures with seats.
erd-08-pricing	Price quotes and tax rates.
erd-09-bookings	Bookings, their status history, refunds owed, disputes.
erd-10-money-ledger-wallet-dormant-payments	The ledger, balances, deposit top-ups, gateway webhooks, and the tables kept unused until the money model changes.
erd-11-reviews	Reviews, replies, reports.
erd-12-messaging-and-notifications	Conversations, messages, notifications, preferences, the outbox and delivery log.
erd-13-media	Uploaded files (public photos, private documents).
erd-14-moderation-config-audit-privacy-system	Staff queues and decisions, settings and onboarding versions, coupons, audit log, exports, privacy requests, idempotency, search table.

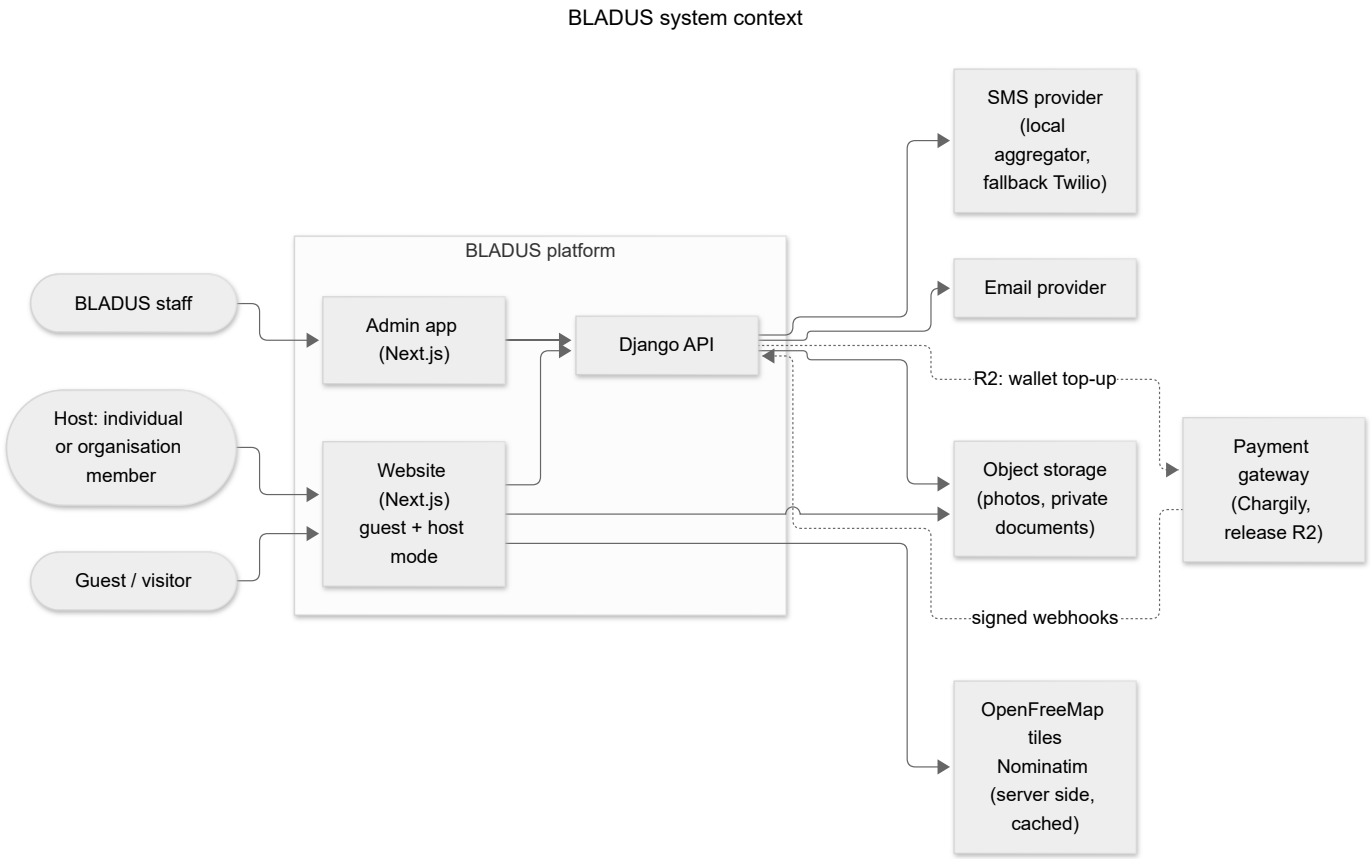
Check yourself

- [ ] In the booking state diagram, which statuses are final?
- [ ] In the instant booking sequence, at which step could a double booking be refused, and by what?
- [ ] In an ERD, what does a box with only `id` mean?
- [ ] Which diagram shows that personal data stays in Algeria?

Diagram gallery

In plain words
Every diagram of the blueprint, embedded as a picture with its caption.
How to read each kind of diagram is explained in the legend on the previous page.

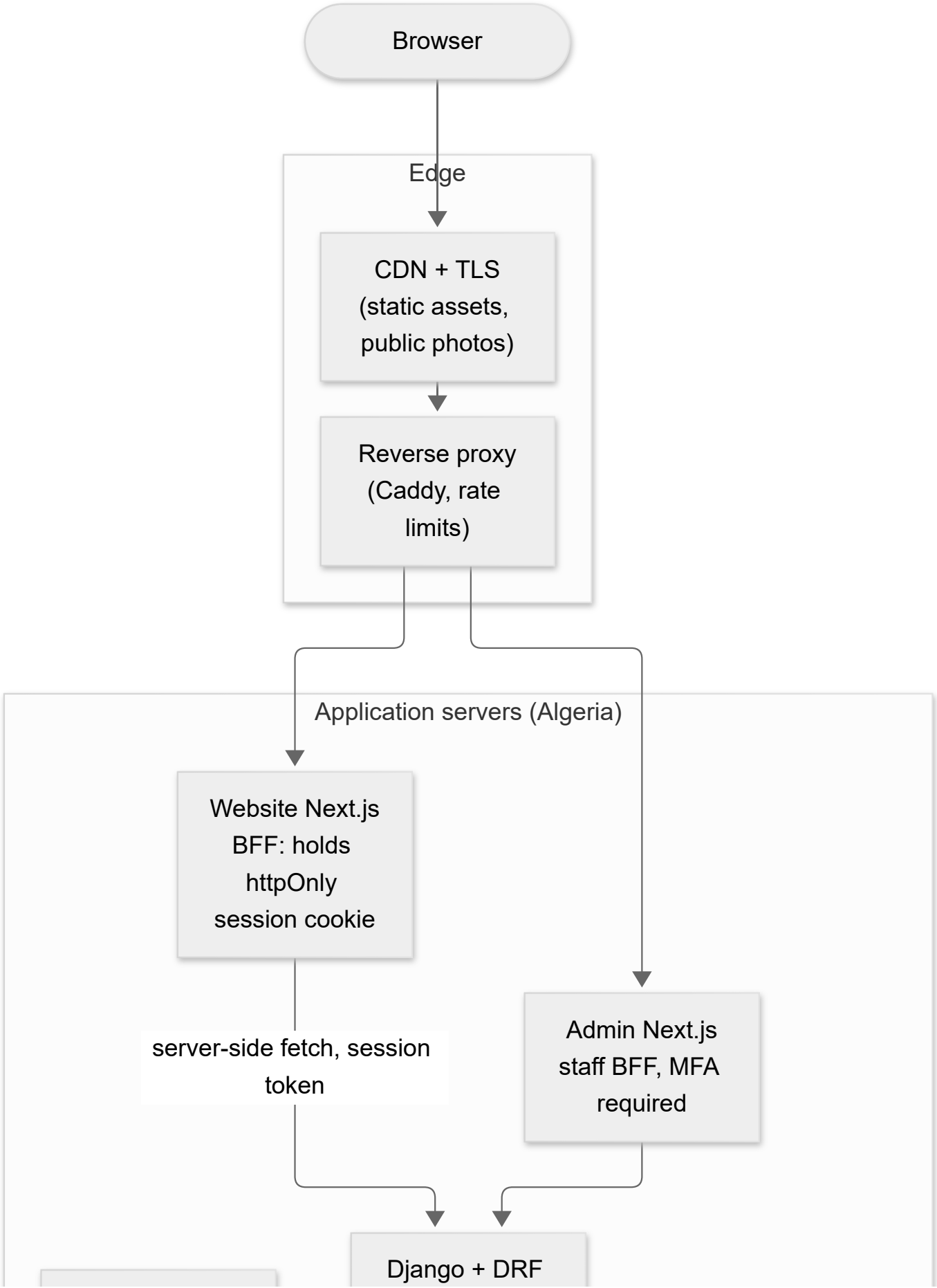
01-system-context

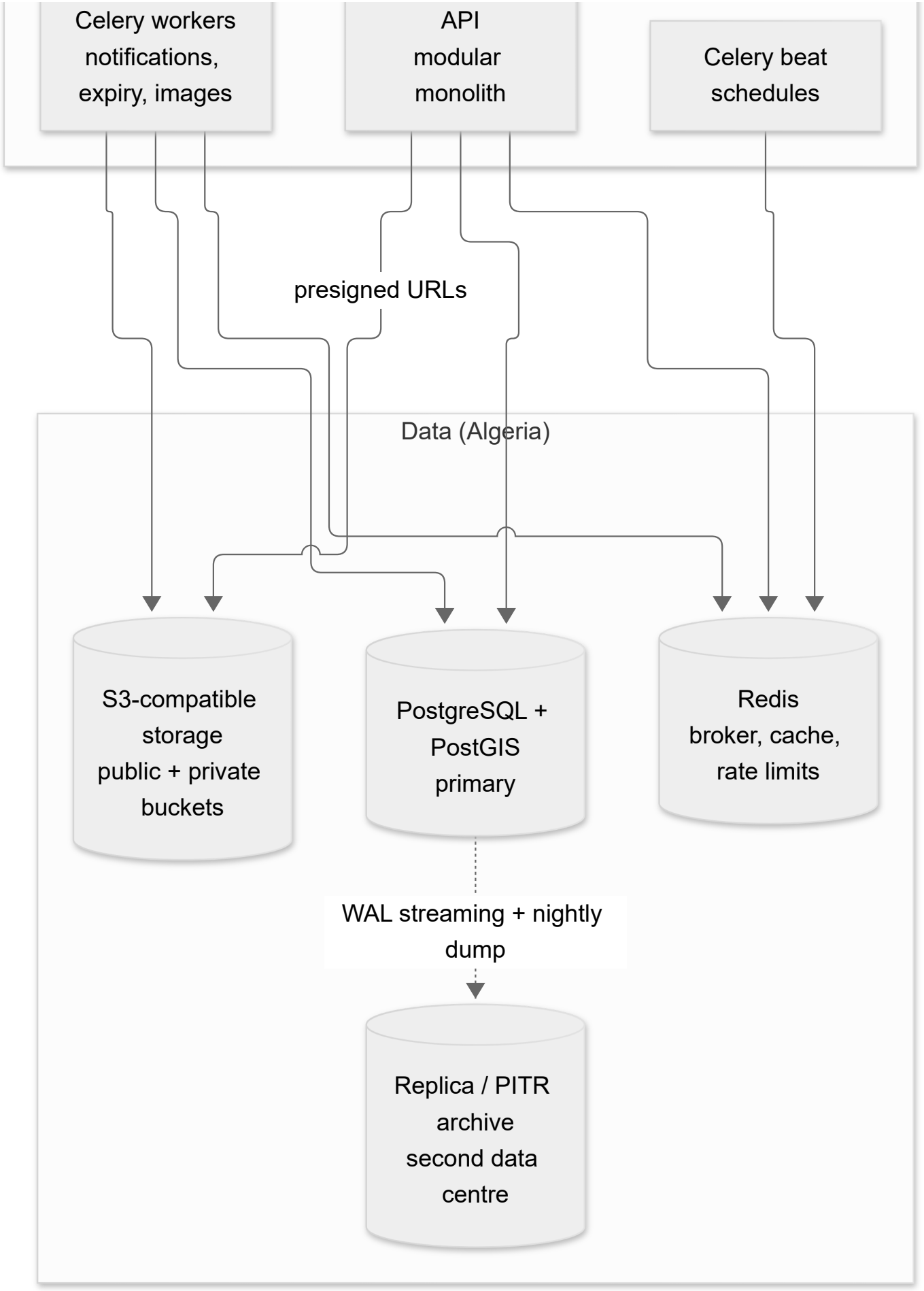


Who uses BLADUS and which outside services it talks to. Guests and hosts use the website, staff use the admin app; both call one Django API. The payment gateway is dotted because online payment comes in release R2.

02-containers

BLADUS containers (what runs where)

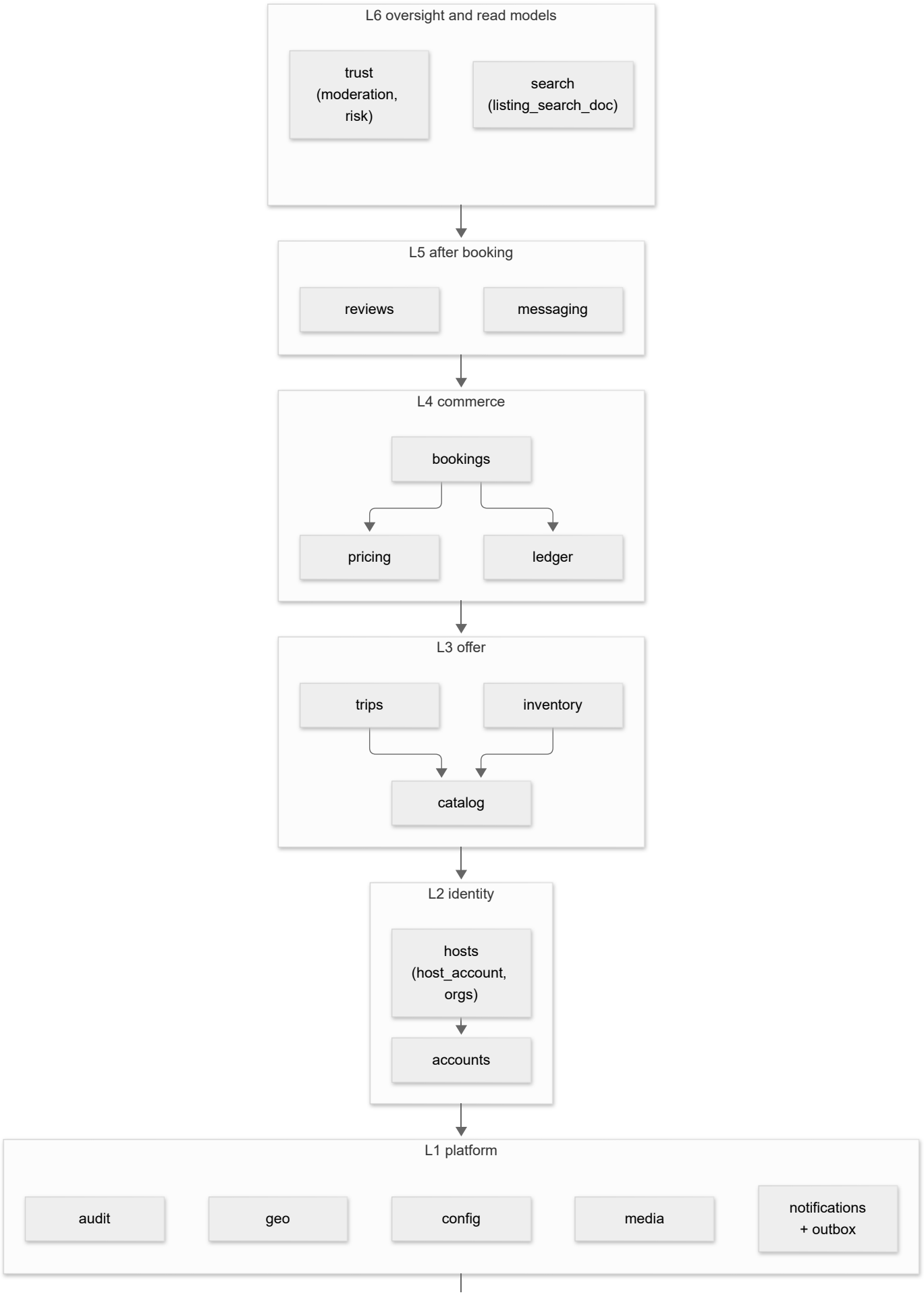


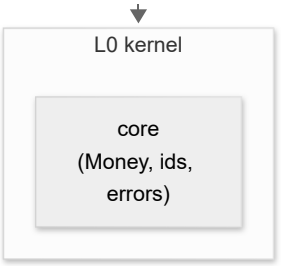


What runs where. Everything holding personal data runs in Algerian data centres (research C.3, to be confirmed by a lawyer). The Next.js servers keep the login cookie and call Django on the browser's behalf (BFF). Workers send messages and run timed jobs.

03-django-apps

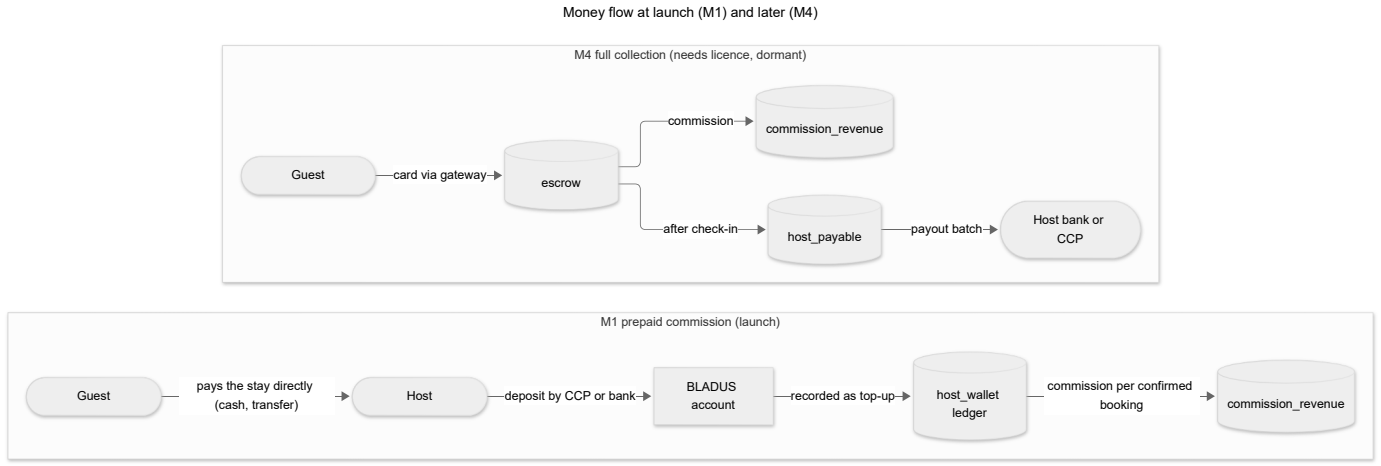
Django apps in layers (arrow = may use; only downward)





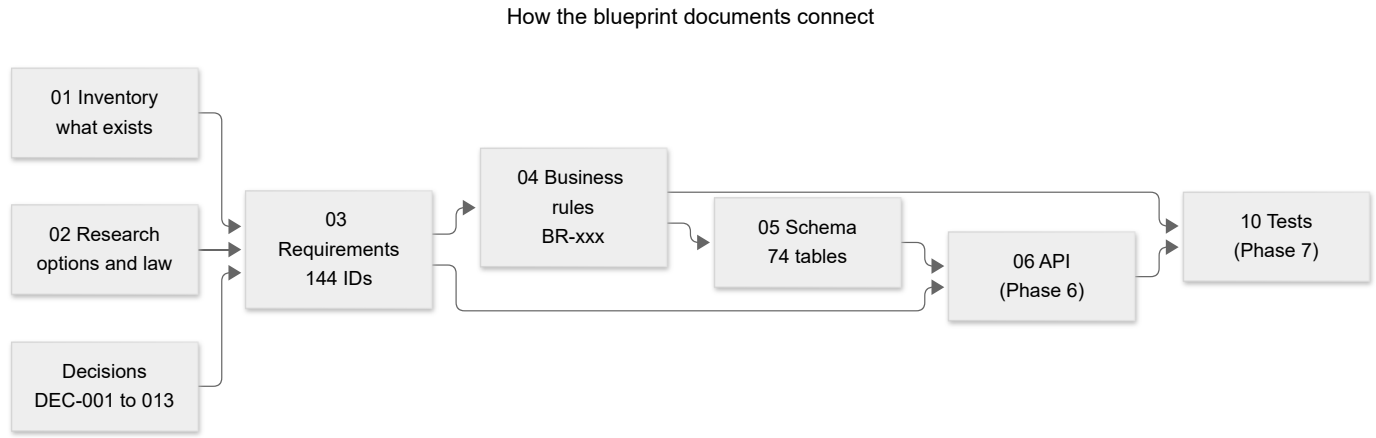
The backend is one application split into 17 apps plus a small `core`, arranged in layers. An app may only use apps in lower layers (checked automatically by import-linter); for example bookings may use pricing and the ledger, never the reverse. Events (emails, search refresh) go through the outbox instead of direct calls. Details in 07-architecture.md.

30-money-flow



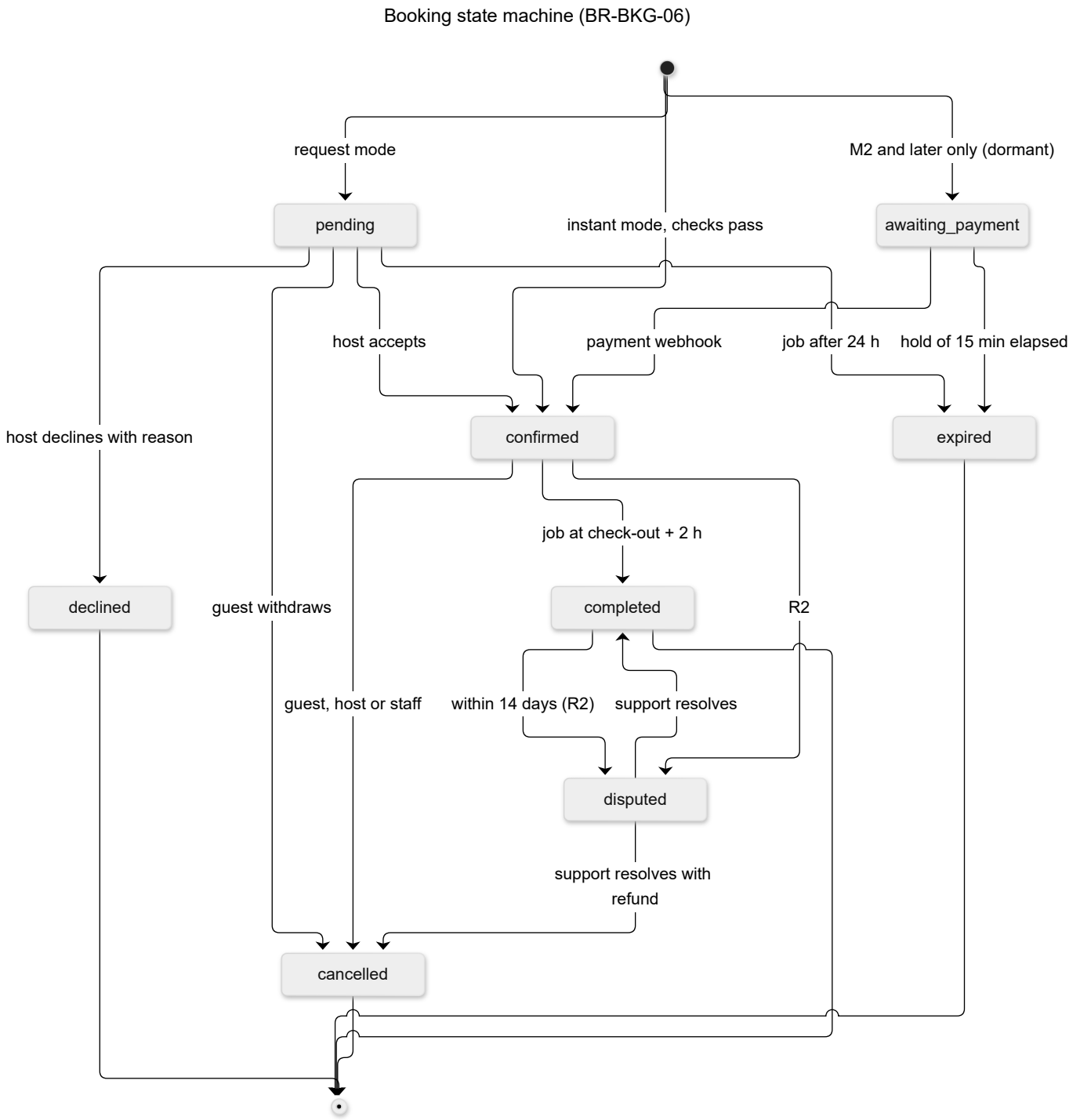
Where money goes. At launch (M1) the guest pays the host, the host keeps a deposit at BLADUS, and BLADUS takes its commission from that deposit. M4 (BLADUS collects and pays out) is drawn for later and needs a licence.

40-document-map



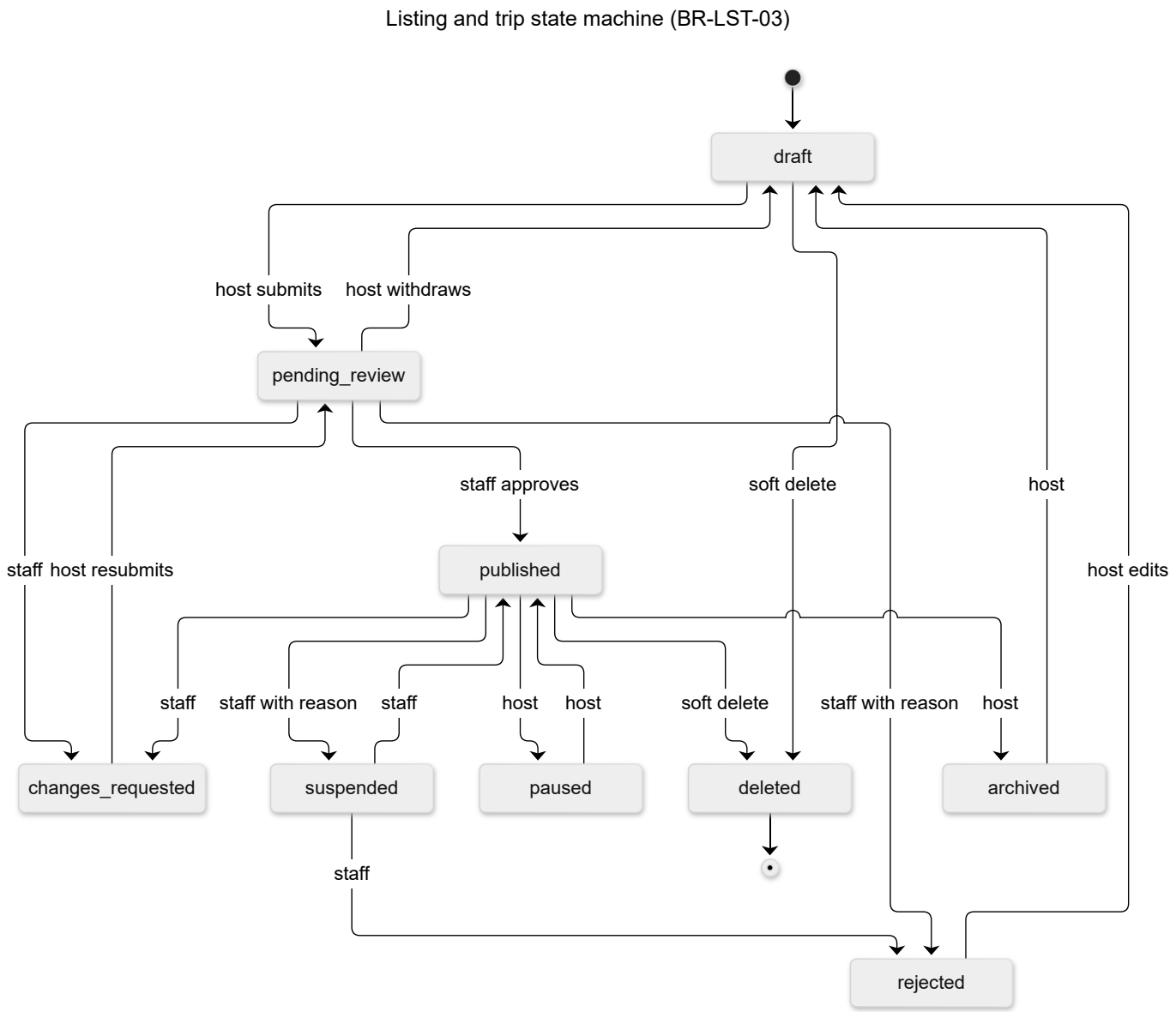
How the blueprint documents feed each other, from inventory to tests.

10-booking-states



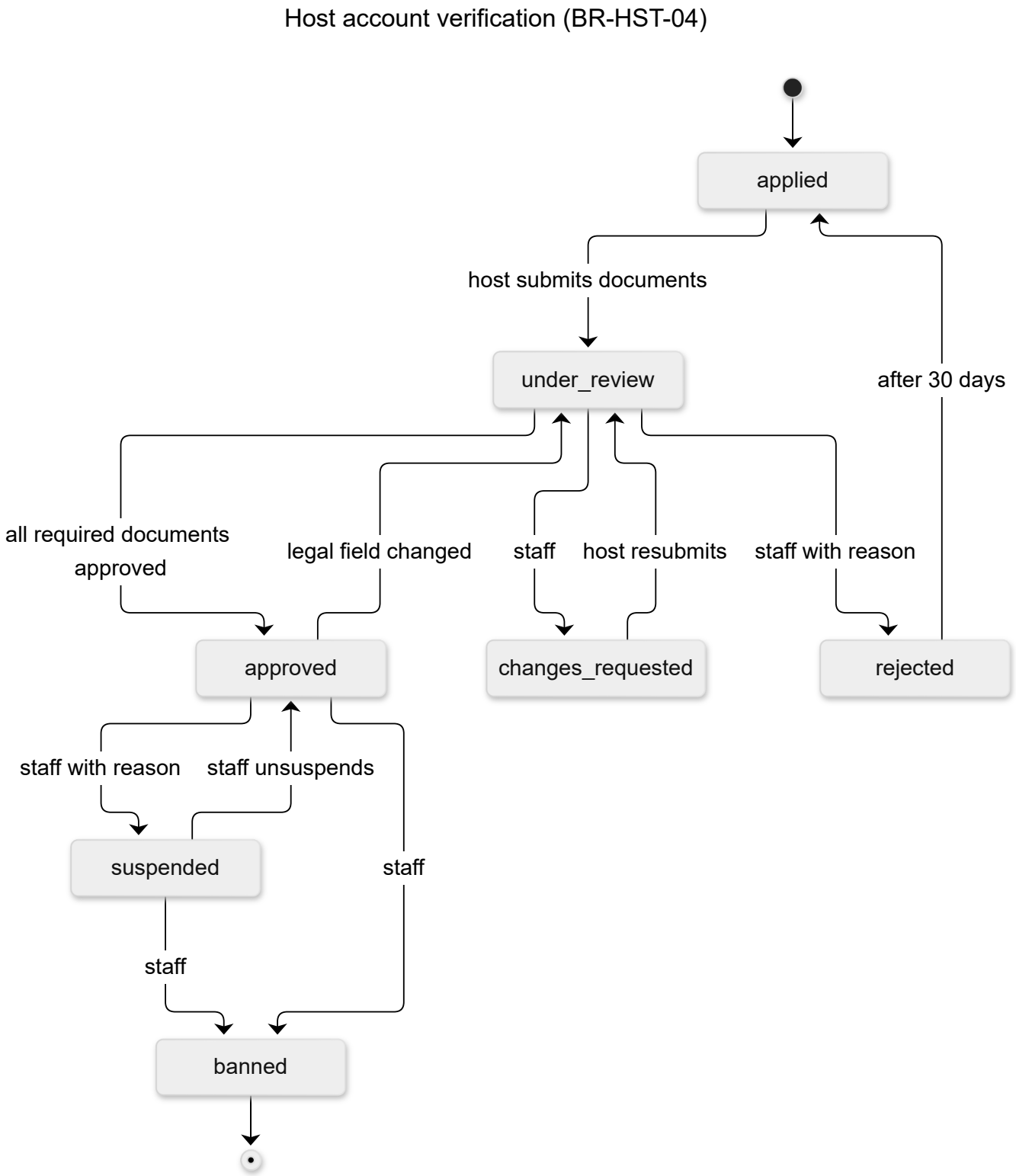
Every status a booking can have. A request waits up to 24 hours for the host; a confirmed booking becomes completed automatically after check-out. `awaiting_payment` stays unused until online payment exists. Rules BR-BKG-01 to 07.

11-listing-states



A listing or trip is checked by staff before it is published (DEC-003). Hosts can pause; staff can suspend or ask for changes. Deleting is reversible by staff.

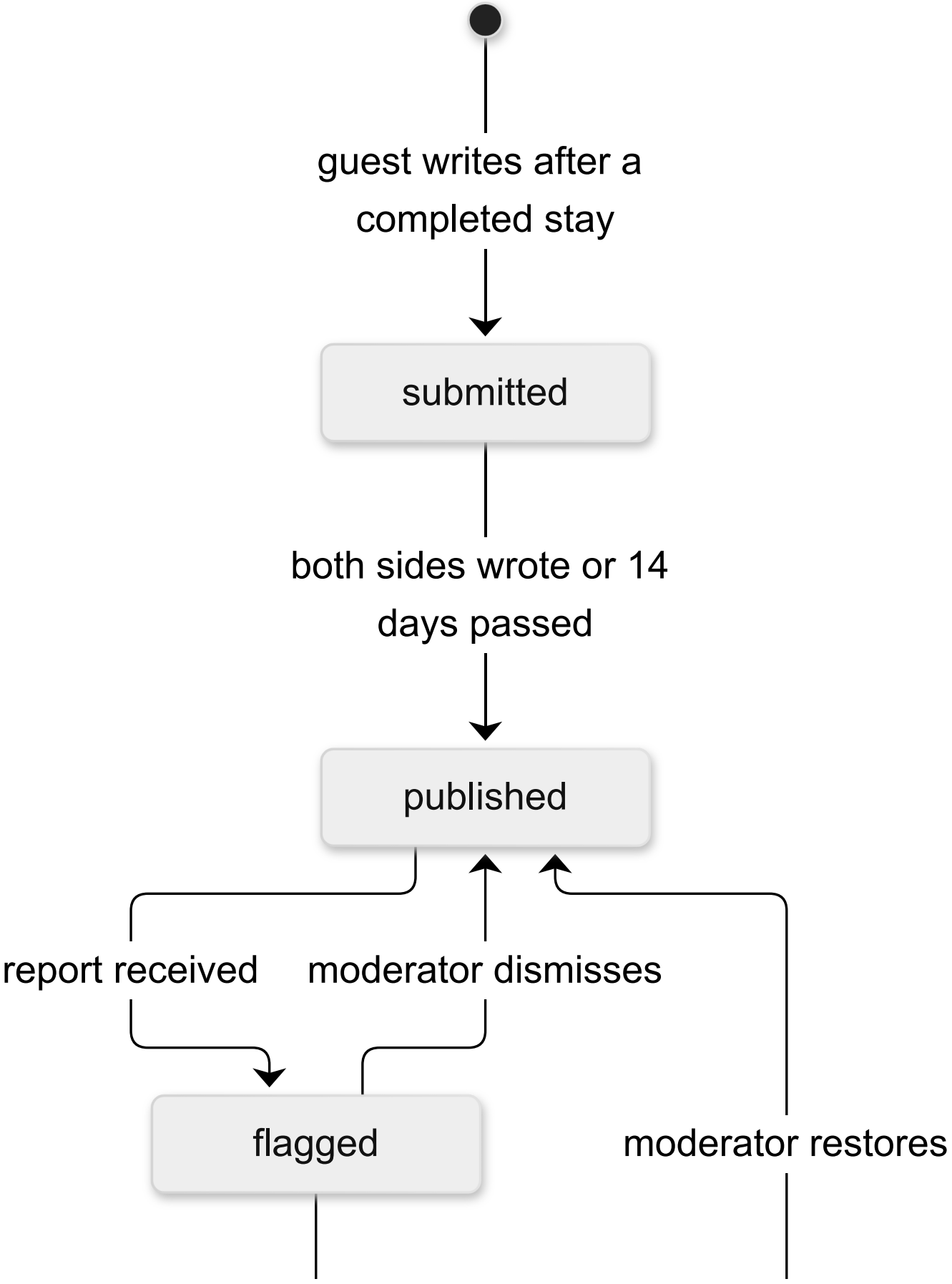
12-host-verification-states

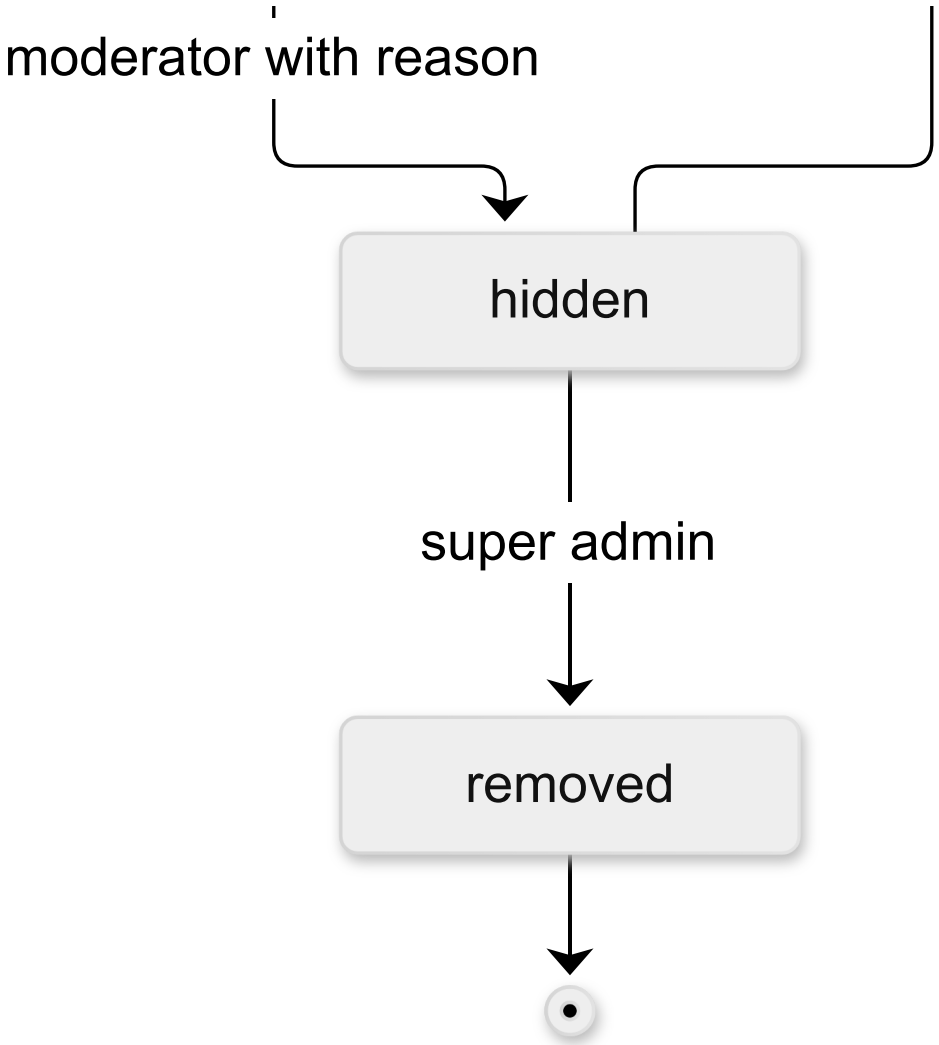


A host (person or organisation) is approved only when all required documents are approved. Changing legal details sends it back to review.

13-review-states

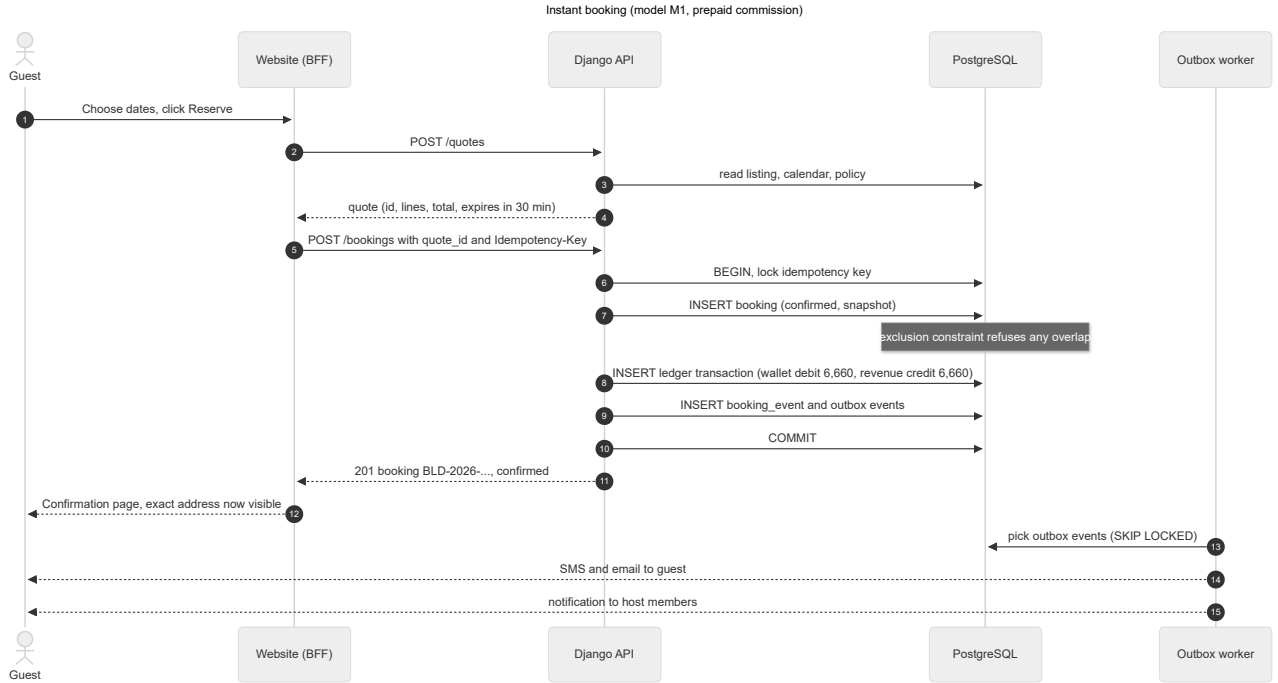
Review state machine (BR-REV-03)





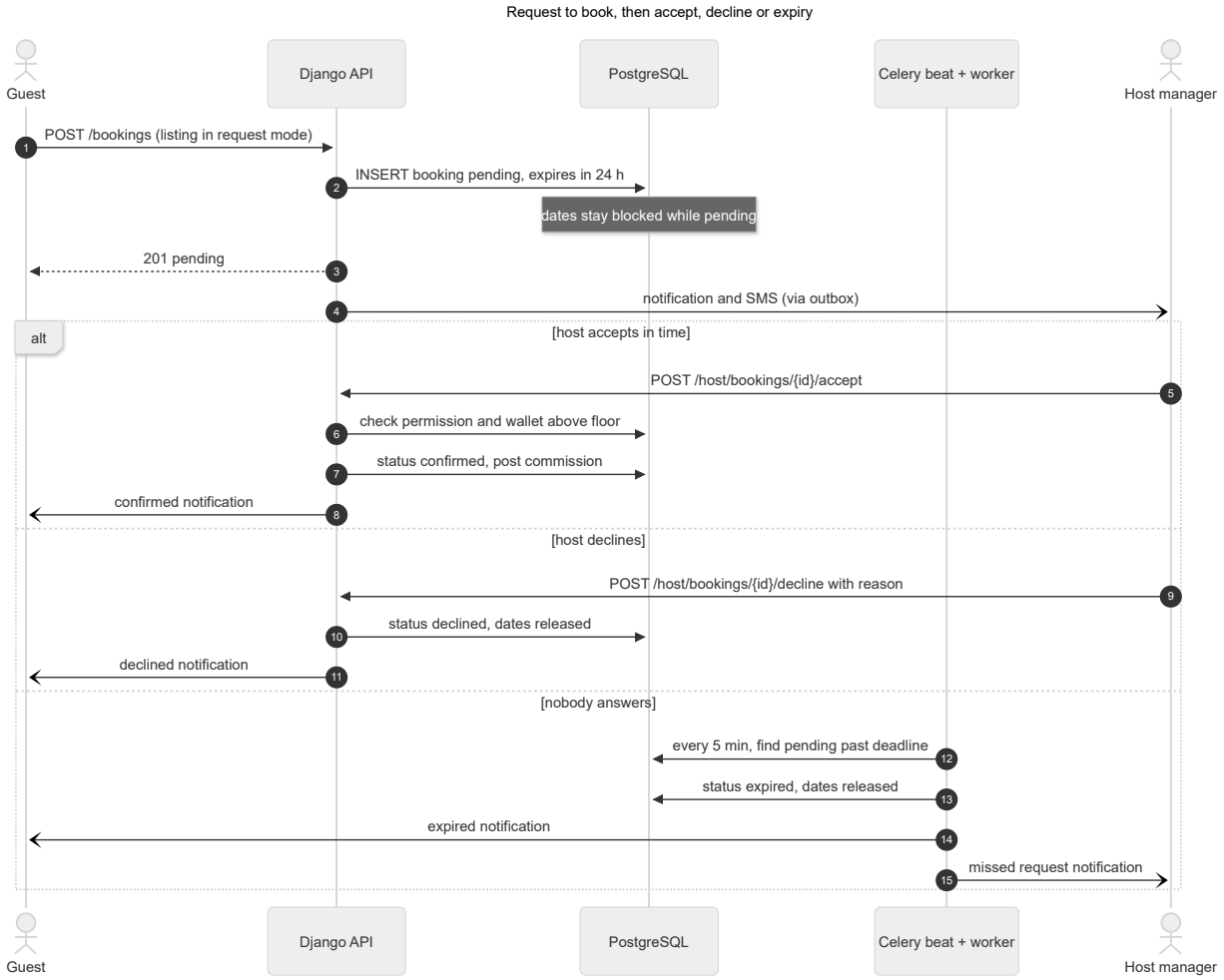
A review is written after a completed stay, published when both sides wrote or after 14 days, and can be hidden after a report.

20-seq-instant-booking



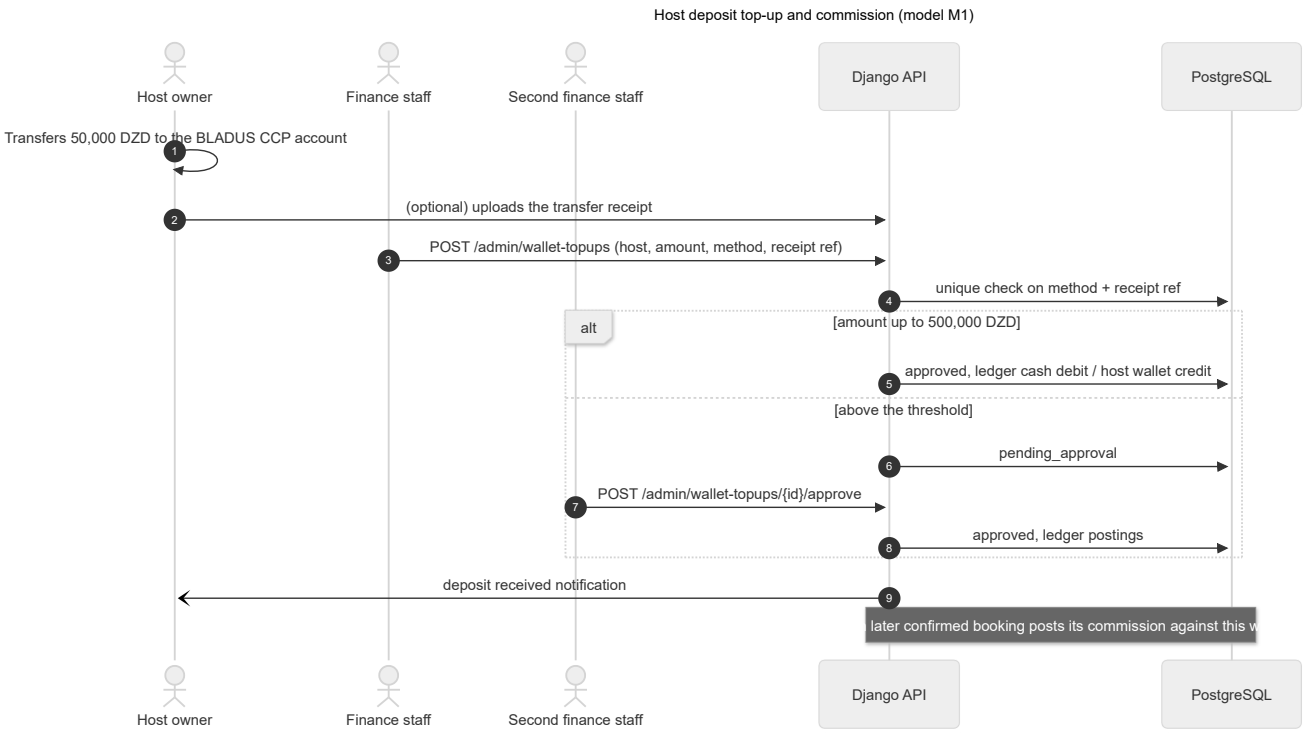
An instant booking from price quote to confirmation in one database transaction; the database itself refuses overlapping dates; the commission is posted to the host's wallet; messages go out after the save.

21-seq-request-to-book



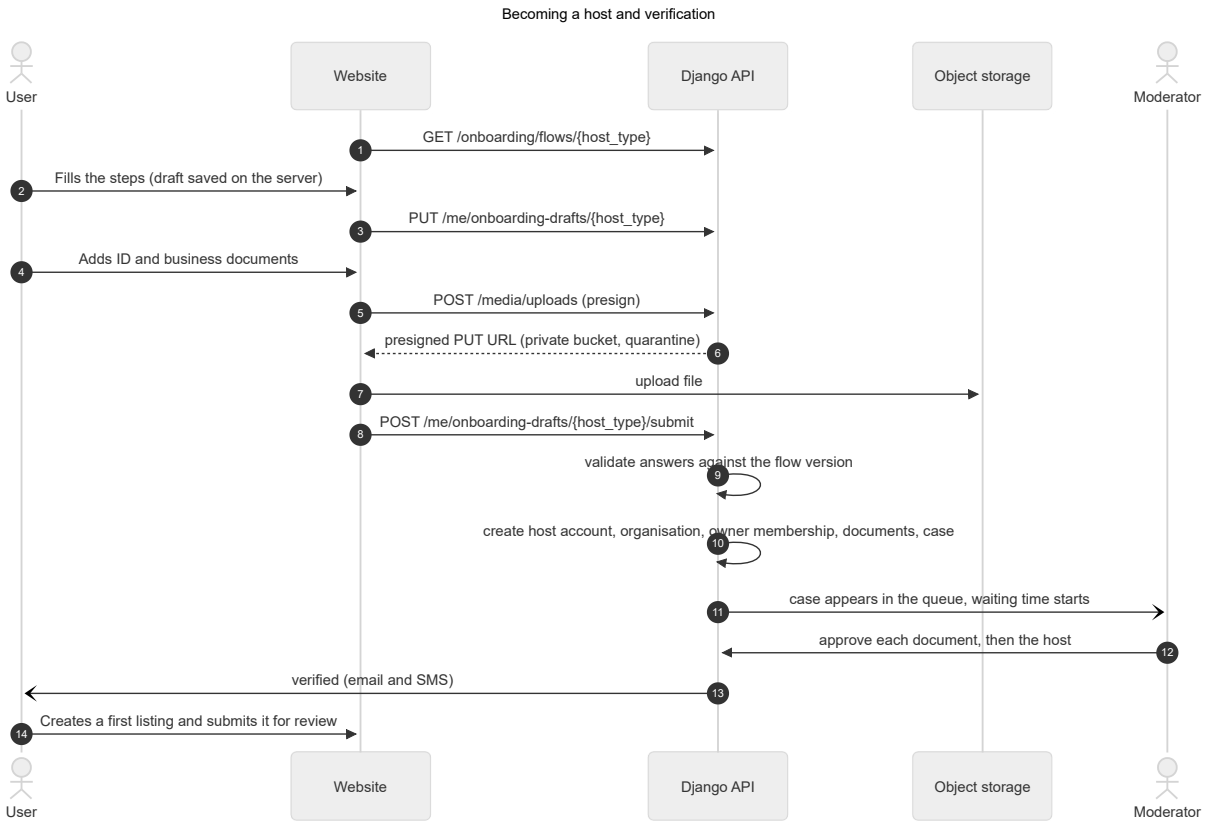
A request: the host accepts, declines, or a timed job expires it after 24 hours. Dates stay blocked while it waits.

22-seq-wallet-topup



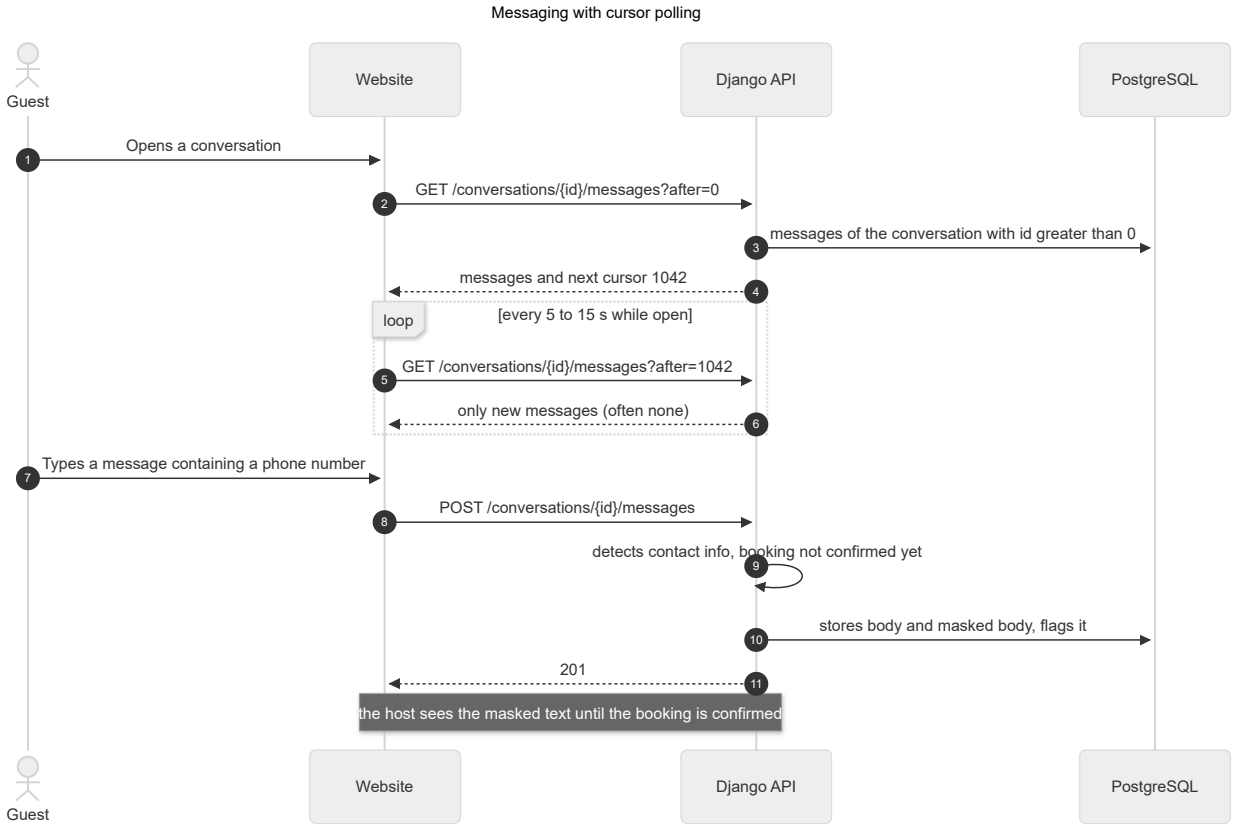
A host tops up the commission deposit by transfer; finance records it with the receipt reference; large amounts need a second person.

23-seq-host-onboarding



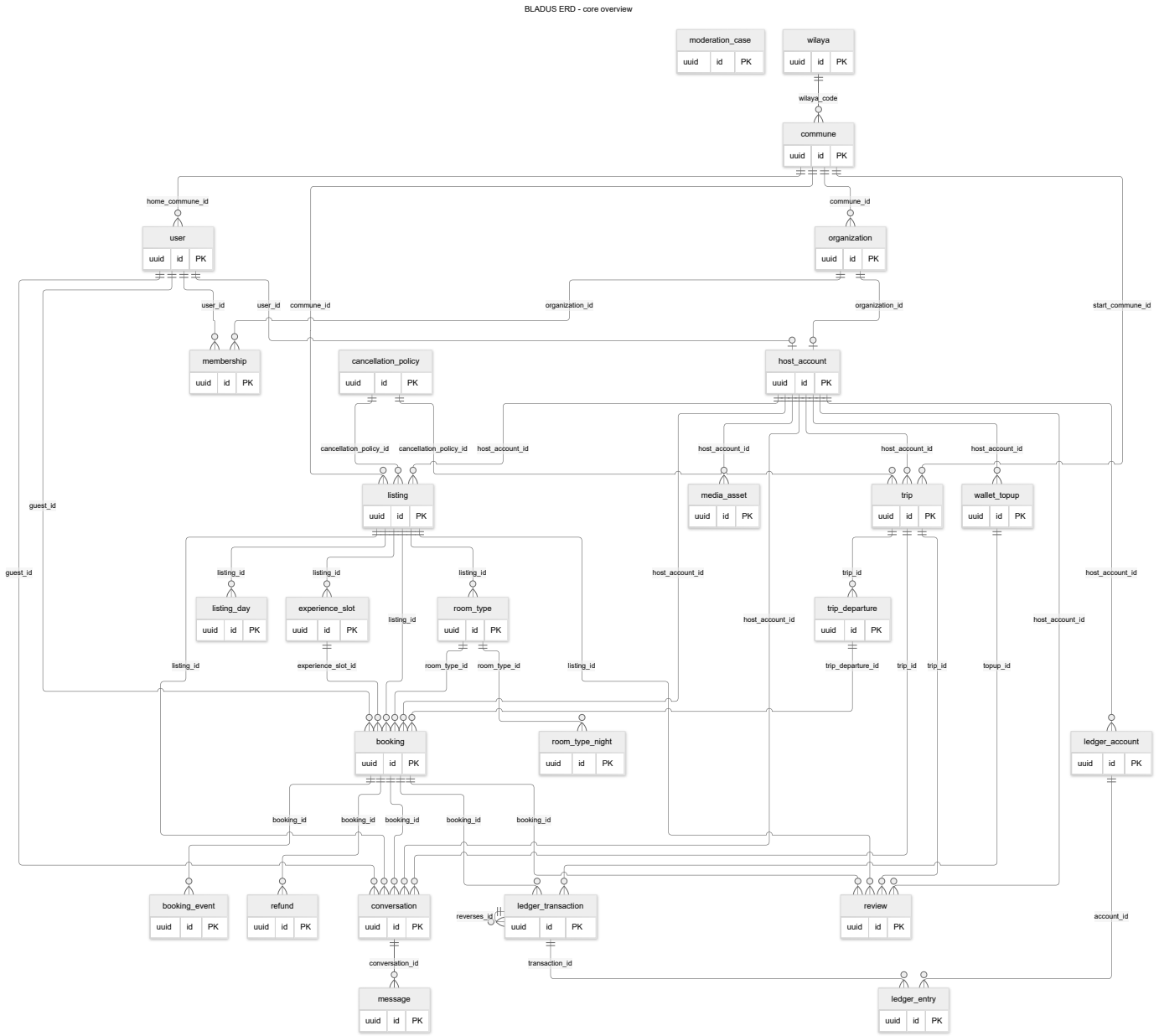
Becoming a host: the steps come from the published configuration, the draft is saved on the server, documents go to private storage, staff verify.

24-seq-message-sync



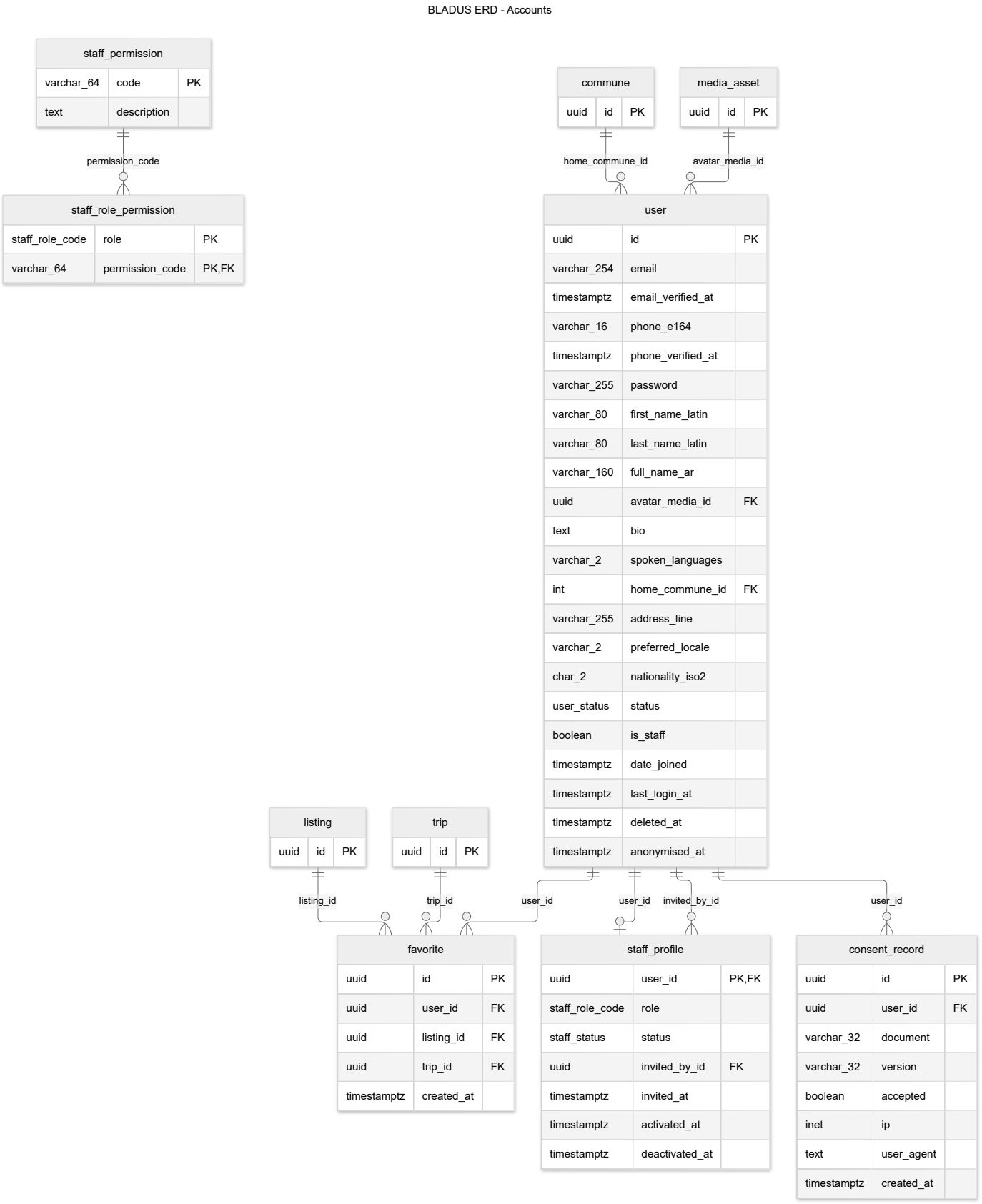
Messages are fetched with a cursor so polling is cheap; phone numbers are masked until a booking is confirmed (anti-fraud).

erd-00-overview



The 26 central tables and their main links. Start here. `host_account` sits between people and organisations on one side and listings, trips and wallets on the other.

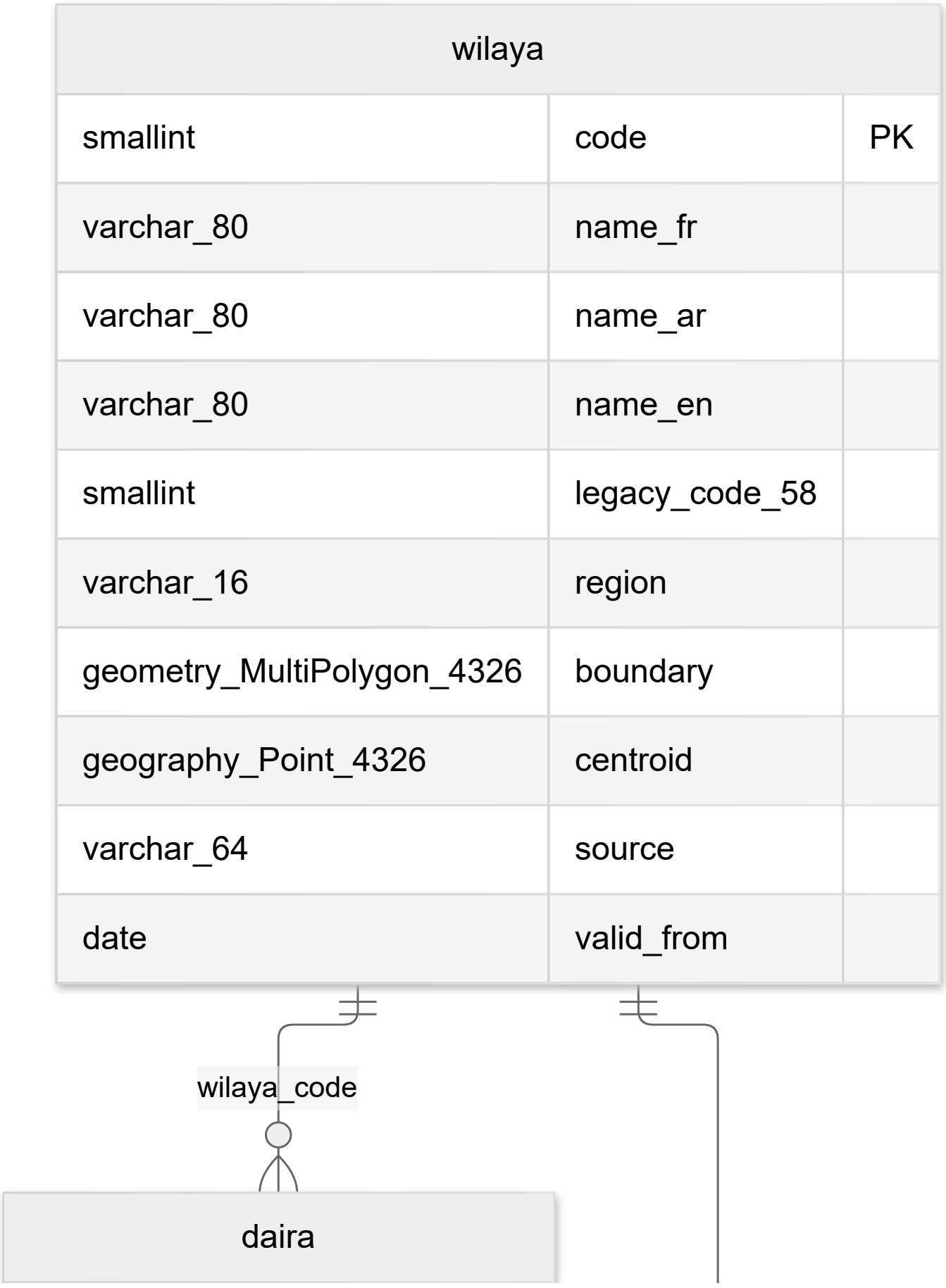
erd-01-accounts

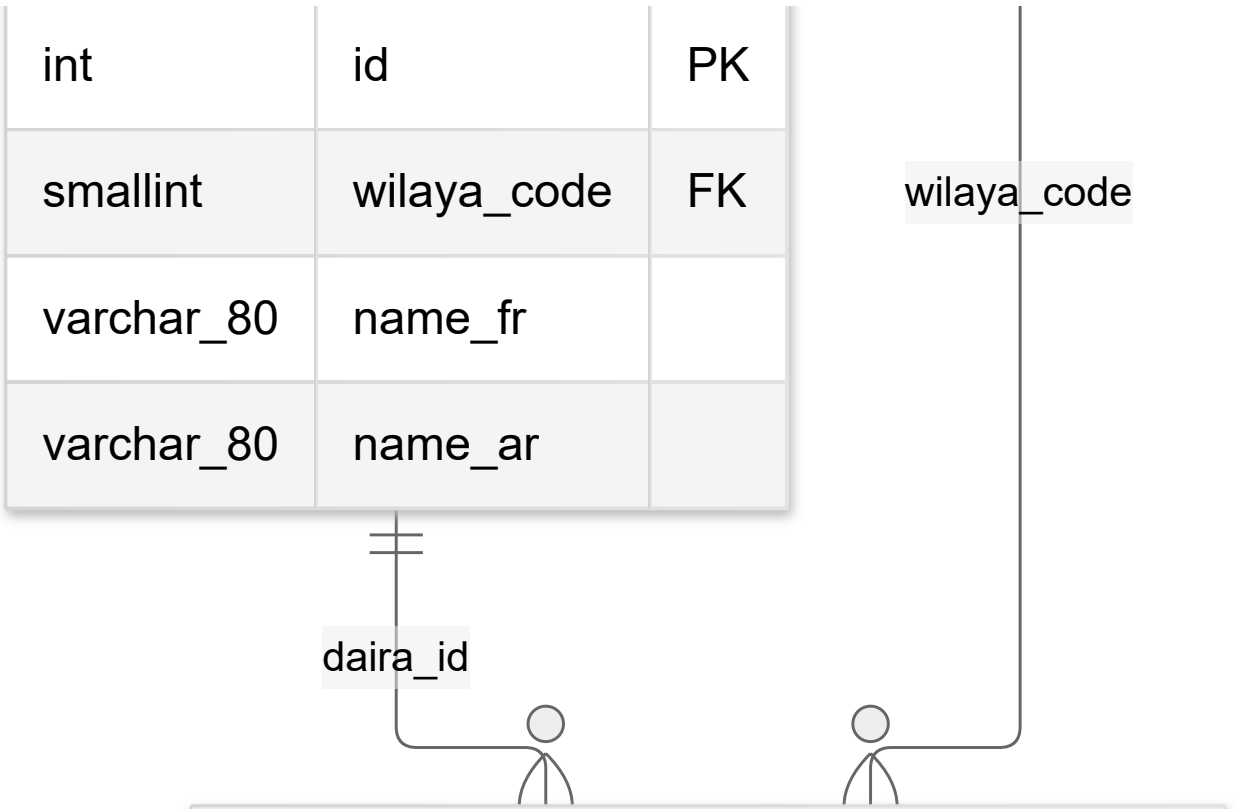


People, staff roles and permissions, consents, favourites.

erd-02-geography

BLADUS ERD - Geography



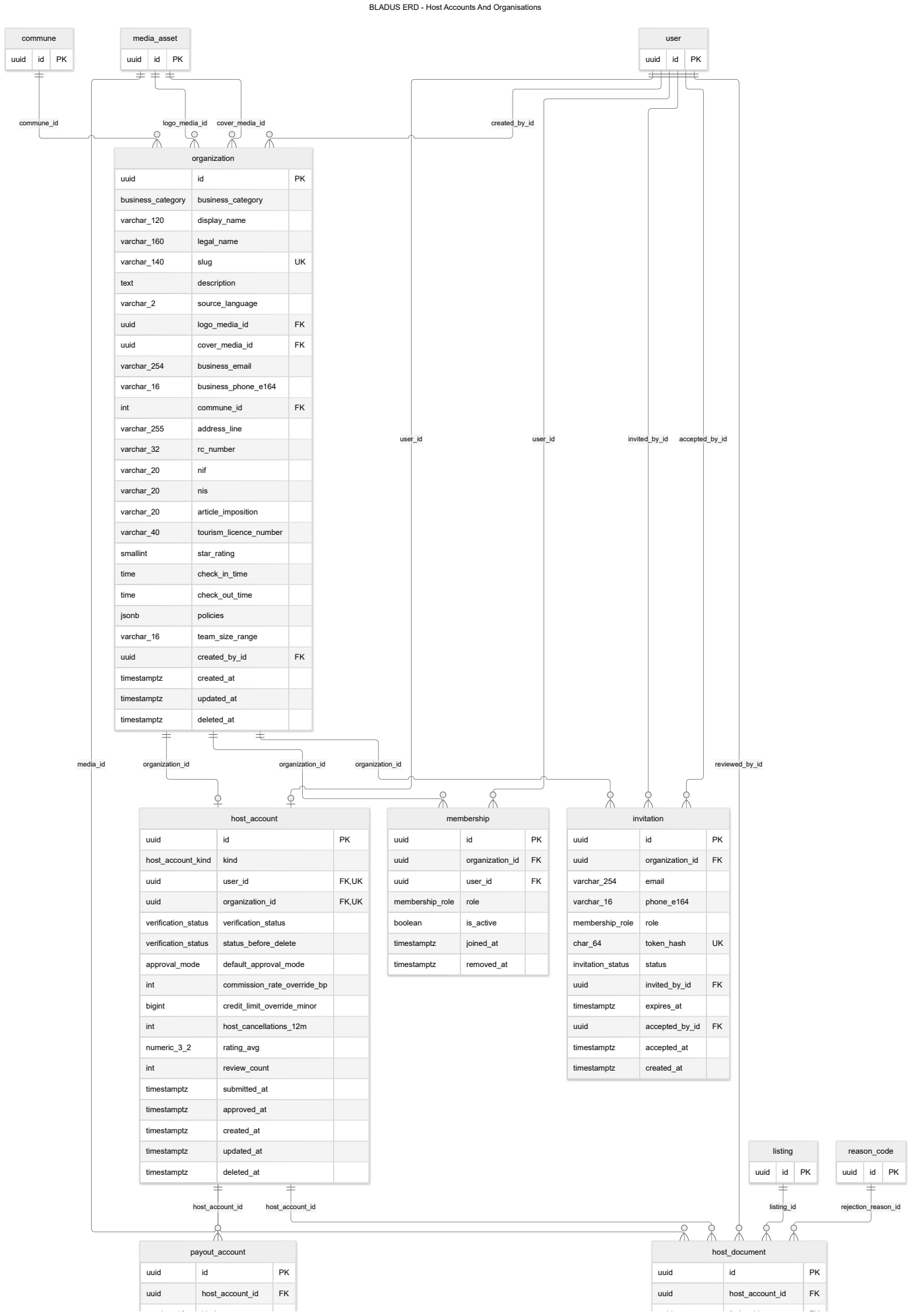


commune		
int	id	PK
smallint	wilaya_code	FK
int	daira_id	FK
varchar_100	name_fr	
varchar_100	name_ar	
varchar_100	name_en	
varchar_5	postal_code	
geography_Point_4326	centroid	

varcnar_64	source	
date	valid_from	

69 wilayas, dairas and 1,541 communes.

erd-03-host-accounts-and-organisations


BLADUS ERD - Host Accounts And Organisations
commune
uuid id PK
media_asset
uuid id PK
user
uuid id PK
commune_id
logo_media_id
cover_media_id
created_by_id
organization
uuid id PK
business_category business_category
varchar_120 display_name
varchar_160 legal_name
varchar_140 slug UK
text description
varchar_2 source_language
uuid logo_media_id FK
uuid cover_media_id FK
varchar_254 business_email
varchar_16 business_phone_e164
int commune_id FK
varchar_255 address_line
varchar_32 rc_number
varchar_20 nif
varchar_20 nis
varchar_20 article_imposition
varchar_40 tourism_licence_number
smallint star_rating
time check_in_time
time check_out_time
jsonb policies
varchar_16 team_size_range
uuid created_by_id FK
timestamptz created_at
timestamptz updated_at
timestamptz deleted_at
user_id
user_id
invited_by_id
accepted_by_id
media_id
organization_id
organization_id
organization_id
reviewed_by_id
host_account
uuid id PK
host_account_kind kind
uuid user_id FK,UK
uuid organization_id FK,UK
verification_status verification_status
verification_status status_before_delete
approval_mode default_approval_mode
int commission_rate_override_bp
bigint credit_limit_override_minor
int host_cancellations_12m
numeric_3_2 rating_avg
int review_count
timestamptz submitted_at
timestamptz approved_at
timestamptz created_at
timestamptz updated_at
timestamptz deleted_at
membership
uuid id PK
uuid organization_id FK
uuid user_id FK
membership_role role
boolean is_active
timestamptz joined_at
timestamptz removed_at
invitation
uuid id PK
uuid organization_id FK
varchar_254 email
varchar_16 phone_e164
membership_role role
char_64 token_hash UK
invitation_status status
uuid invited_by_id FK
timestamptz expires_at
uuid accepted_by_id FK
timestamptz accepted_at
timestamptz created_at
listing
uuid id PK
reason_code
uuid id PK
host_account_id
host_account_id
listing_id
rejection_reason_id
payout_account
uuid id PK
uuid host_account_id FK
host_document
uuid id PK
uuid host_account_id FK

varchar_16	kind	
varchar_160	holder_name	
bytea	number_encrypted	
char_4	number_last4	
boolean	is_default	
timestamptz	hold_until	
timestamptz	created_at	
timestamptz	deleted_at	

uuid	listing_id	FK
document_type	doc_type	
uuid	media_id	FK
document_status	status	
char_64	number_hash	
date	expires_on	
uuid	reviewed_by_id	FK
timestamptz	reviewed_at	
uuid	rejection_reason_id	FK
timestamptz	created_at	
timestamptz	deleted_at	

Host accounts, organisations, team members, invitations, verification documents, payout accounts.

erd-04-taxonomy

BLADUS ERD - Taxonomy

category		
uuid	id	PK
vertical	vertical	
varchar_60	slug	
varchar_80	name_en	
varchar_80	name_fr	
varchar_80	name_ar	
varchar_40	icon	
int	sort_order	
boolean	is_active	

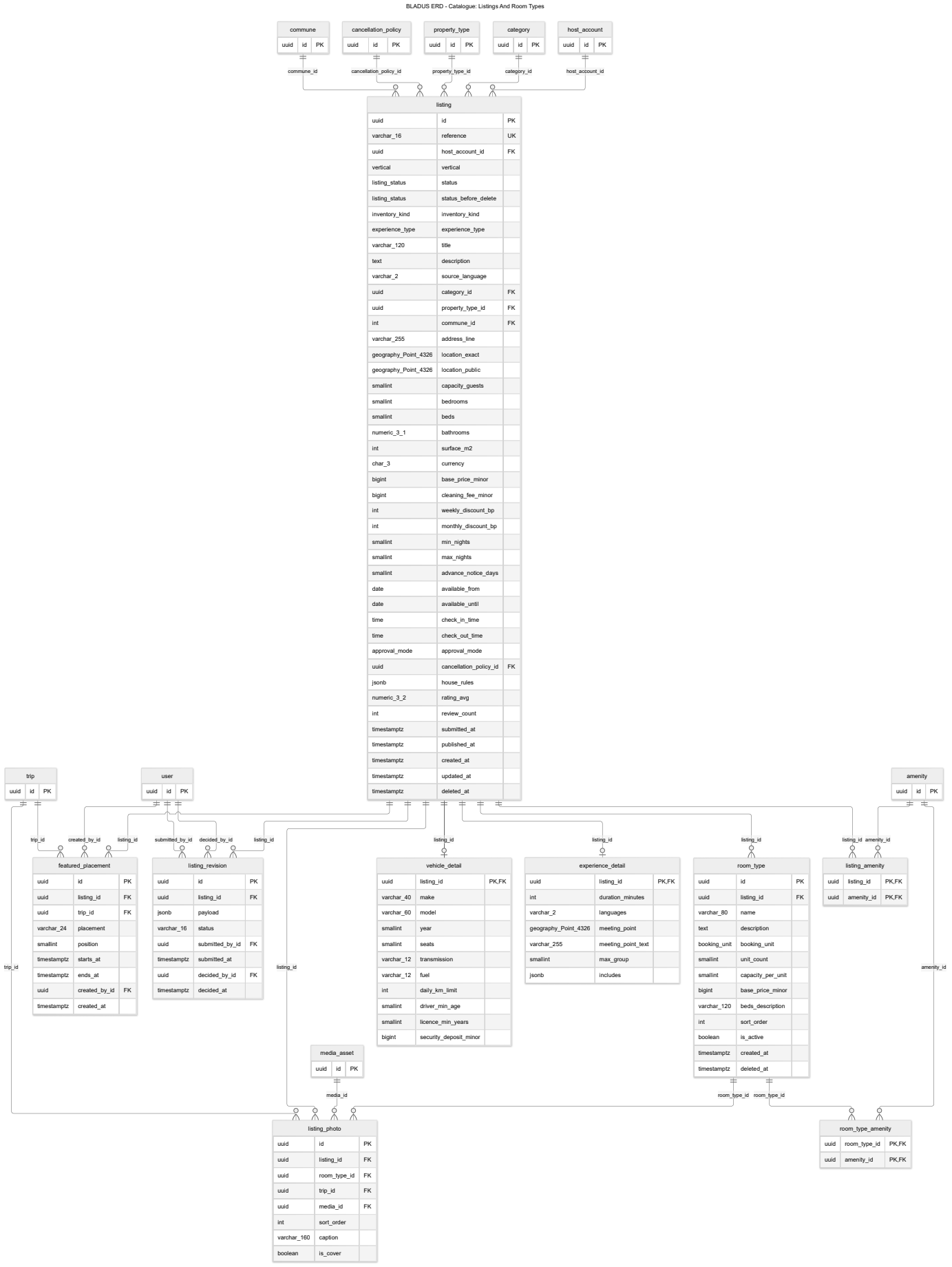
property_type		
uuid	id	PK
vertical	vertical	
varchar_60	slug	
varchar_80	name_en	
varchar_80	name_fr	
varchar_80	name_ar	
boolean	is_active	

amenity		
uuid	id	PK
varchar_60	slug	UK
varchar_20	group_code	
varchar_80	name_en	
varchar_80	name_fr	
varchar_80	name_ar	
varchar_40	icon	
vertical	applies_to	
boolean	is_active	

cancellation_policy		
uuid	id	PK
varchar_20	code	
int	version	
varchar_60	name_en	
varchar_60	name_fr	
varchar_60	name_ar	
jsonb	windows	
boolean	is_current	
timestamptz	created_at	

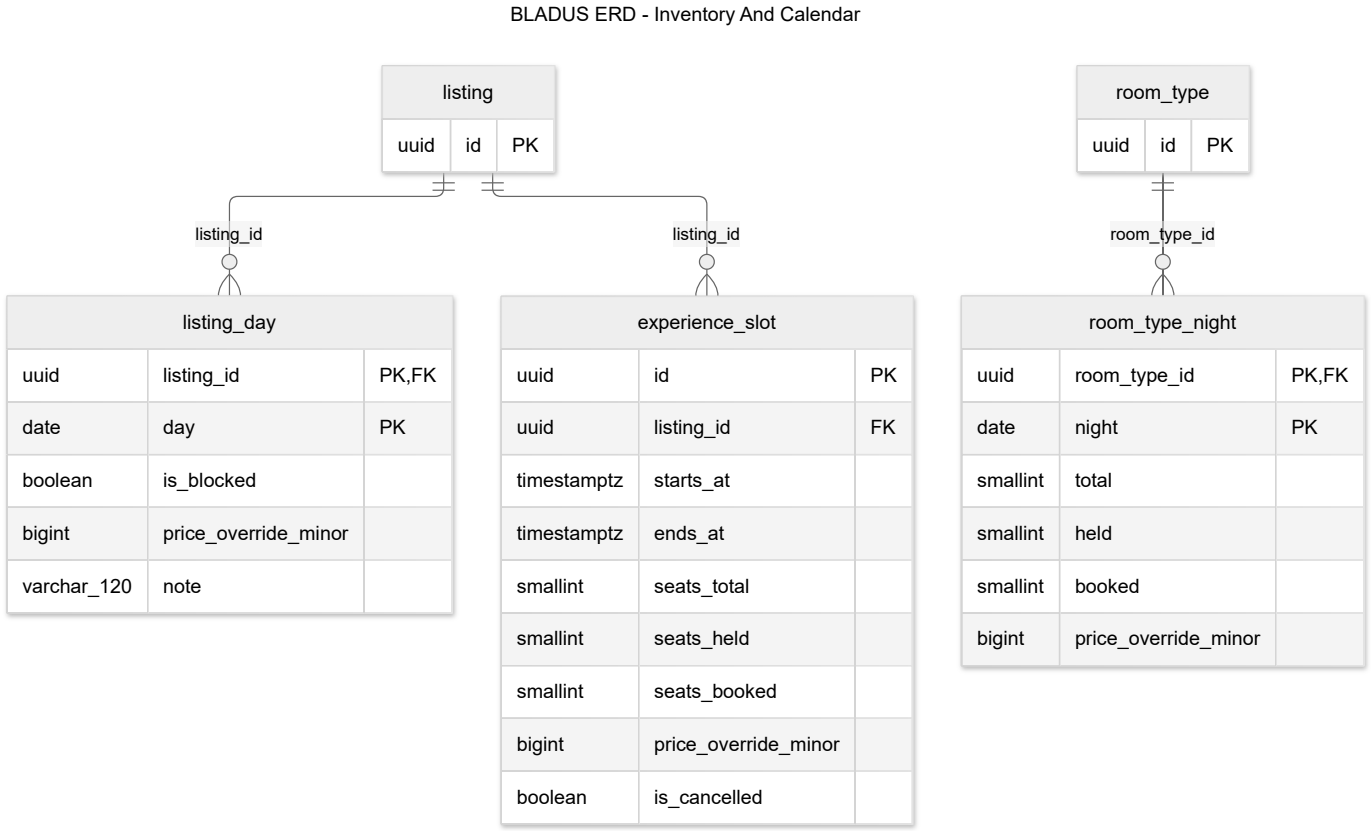
Shared lists: categories, property types, amenities, cancellation policies.

erd-05-catalogue-listings-and-room-types



Listings, edits under review, vehicle and experience details, room types, photos, featured places.

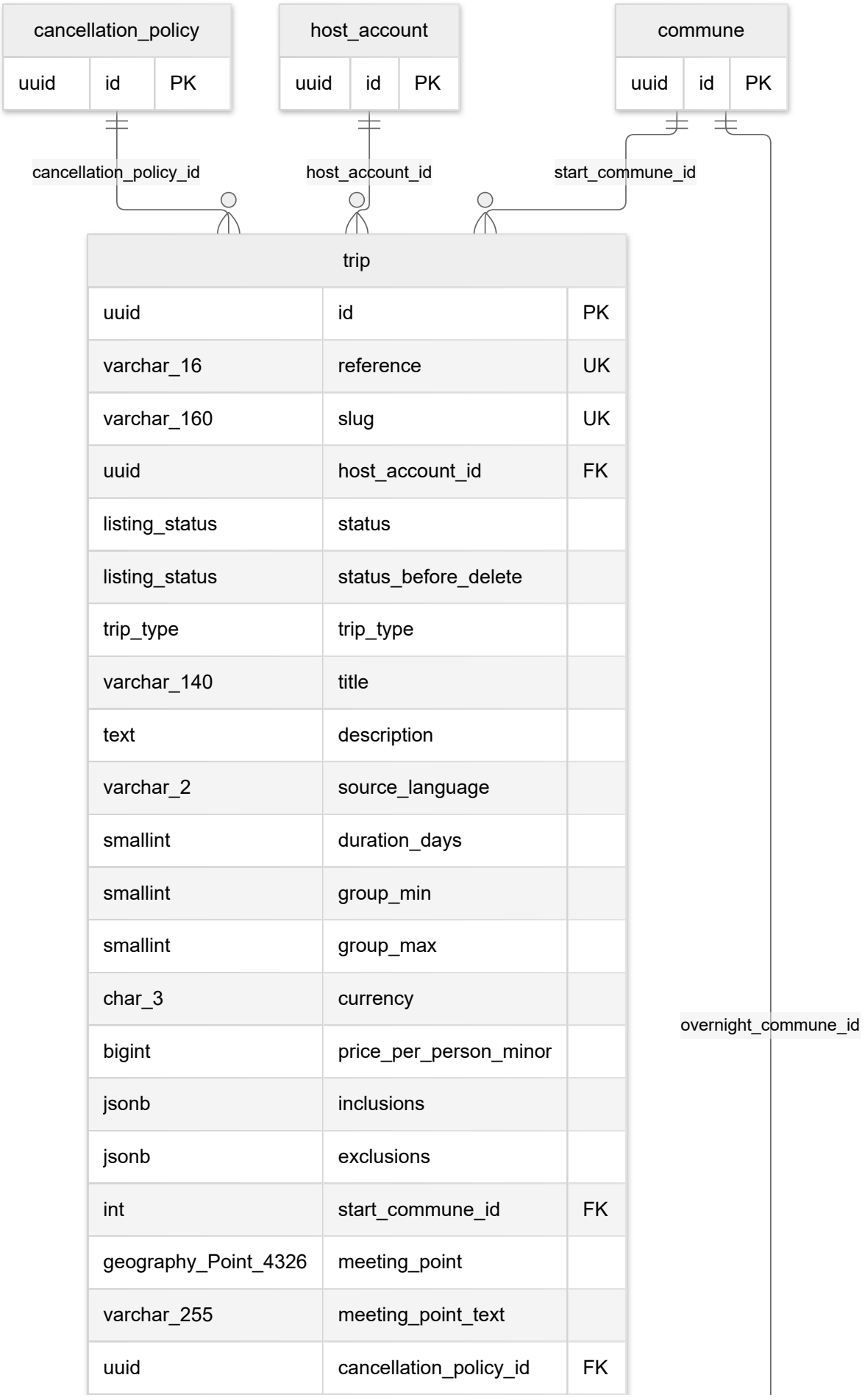
erd-06-inventory-and-calendar

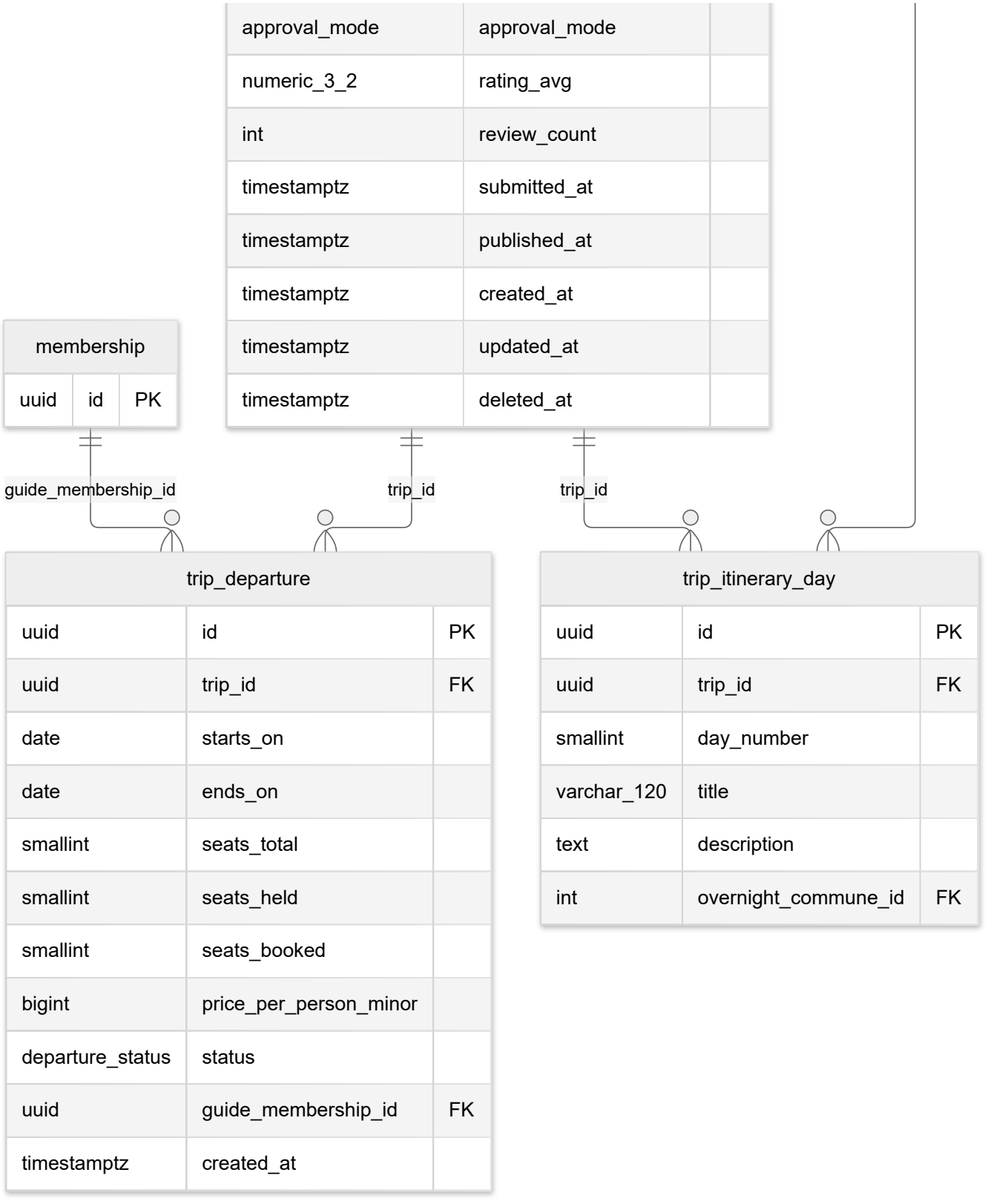


What is free per day (homes, vehicles), per night and room type (hotels, hostels) and per time slot (experiences).

erd-07-trips-agency-only

BLADUS ERD - Trips (Agency Only)





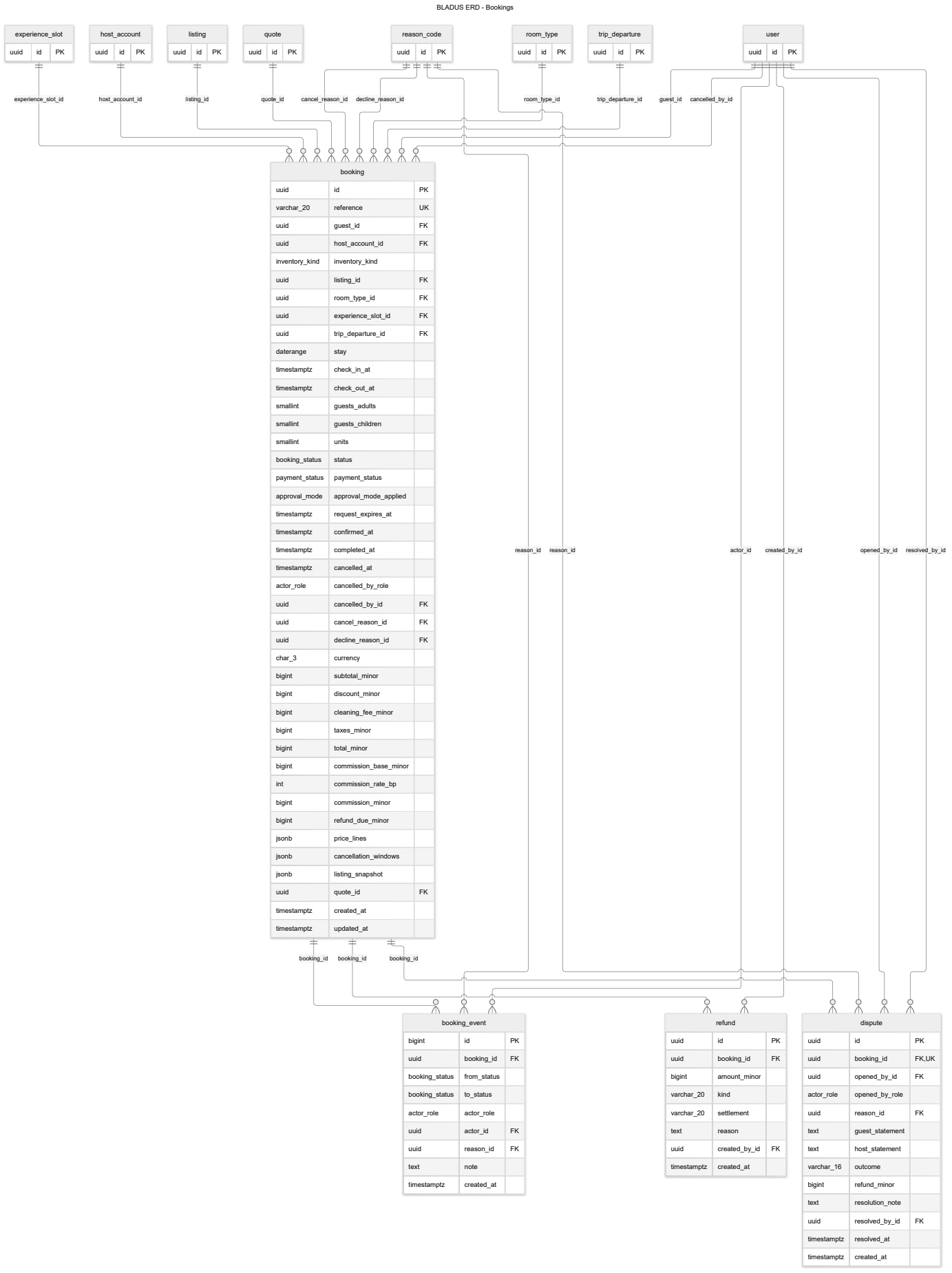
Agency trips, day-by-day itinerary, dated departures with seats.

erd-08-pricing



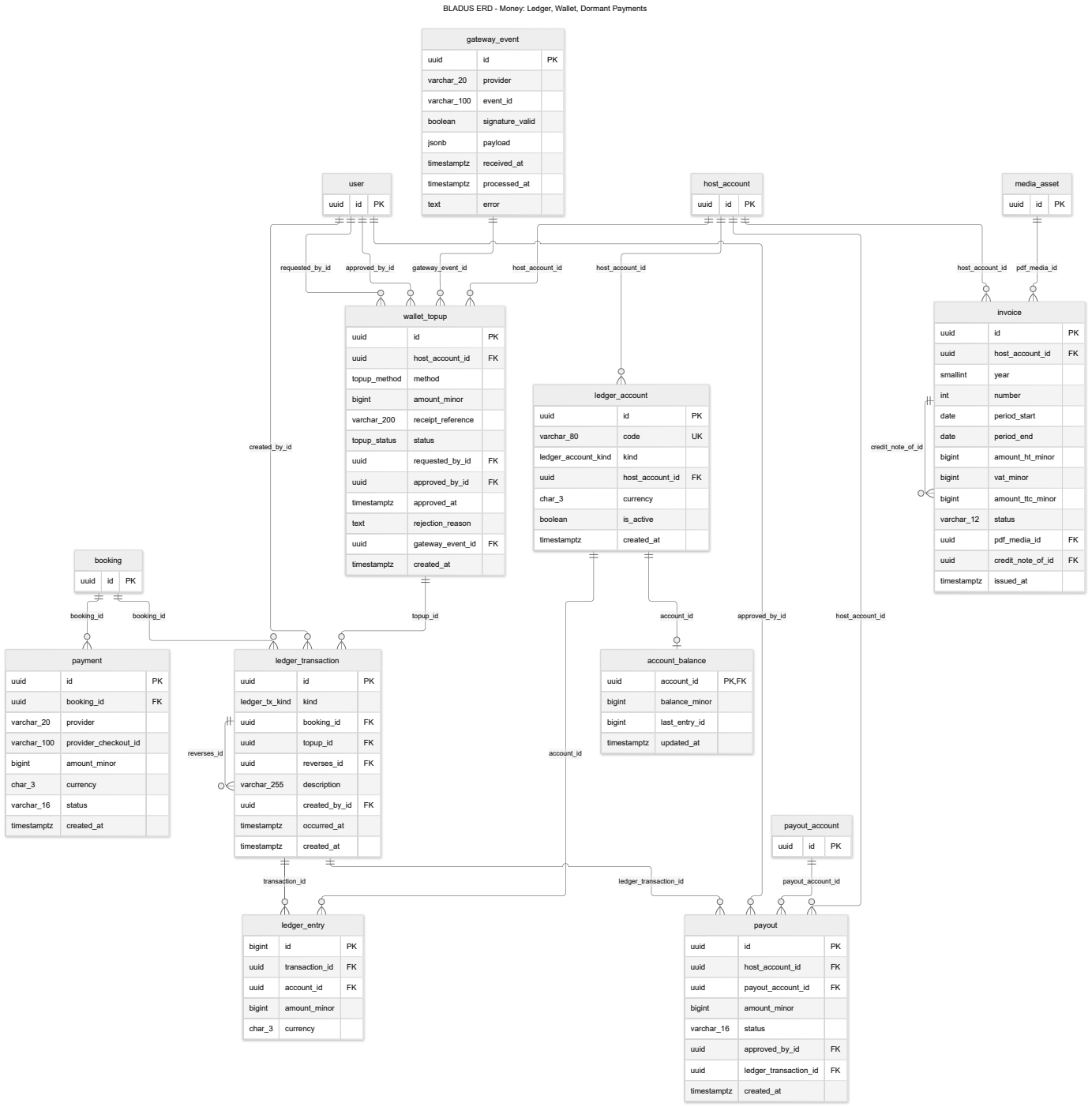
Price quotes and tax rates.

erd-09-bookings



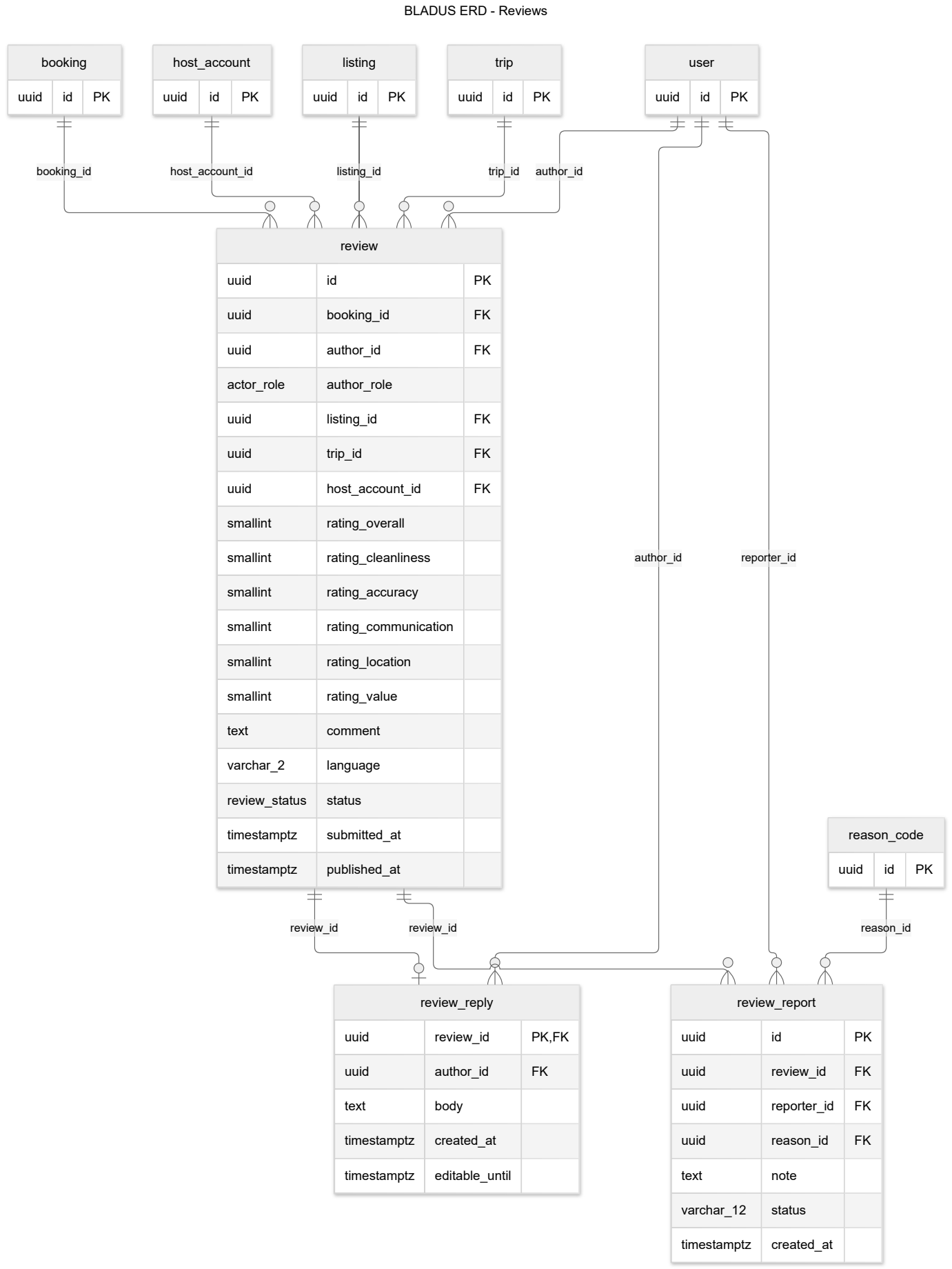
Bookings, their status history, refunds owed, disputes.

erd-10-money-ledger-wallet-dormant-payments



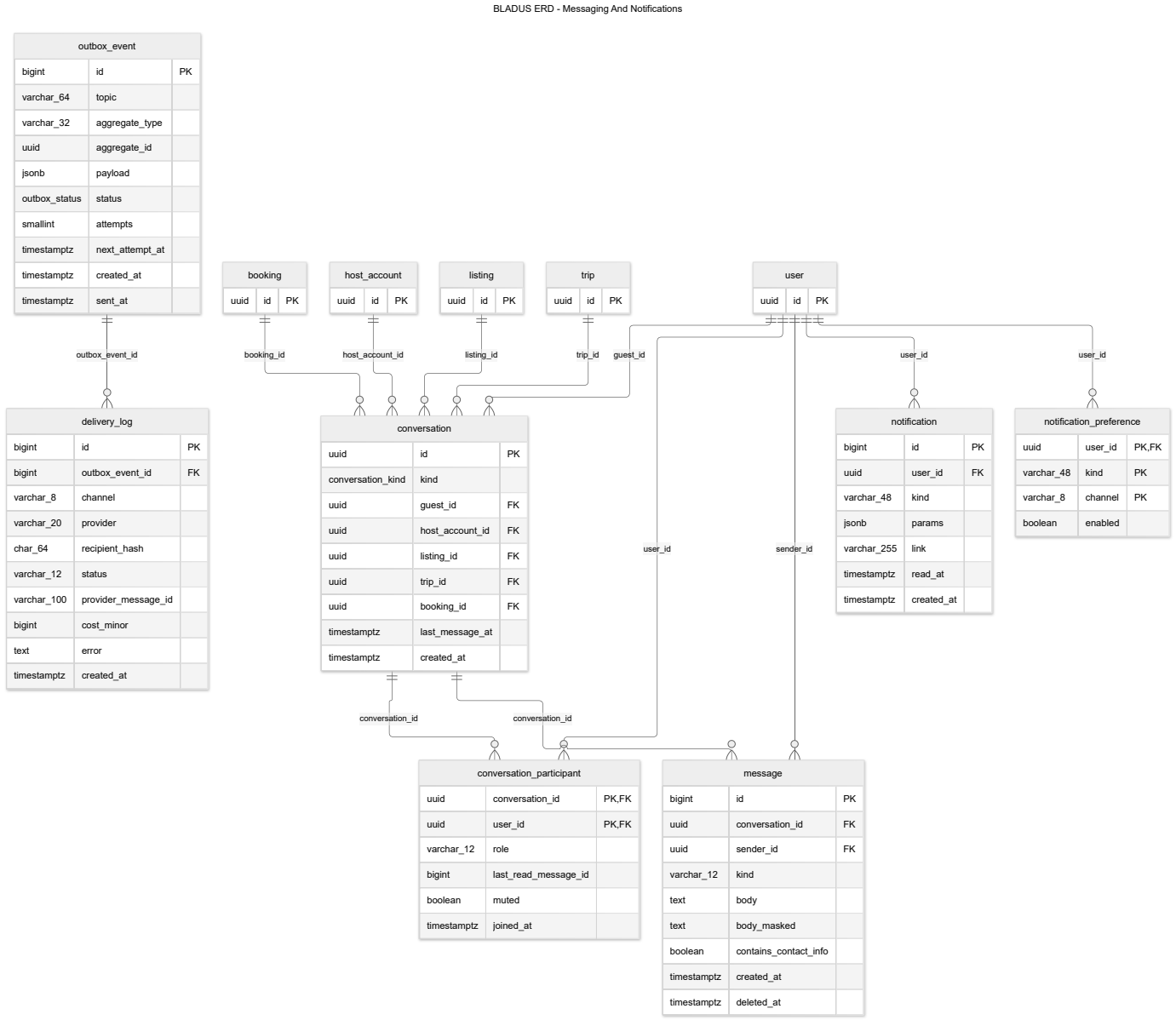
The ledger, balances, deposit top-ups, gateway webhooks, and the tables kept unused until the money model changes.

erd-11-reviews



Reviews, replies, reports.

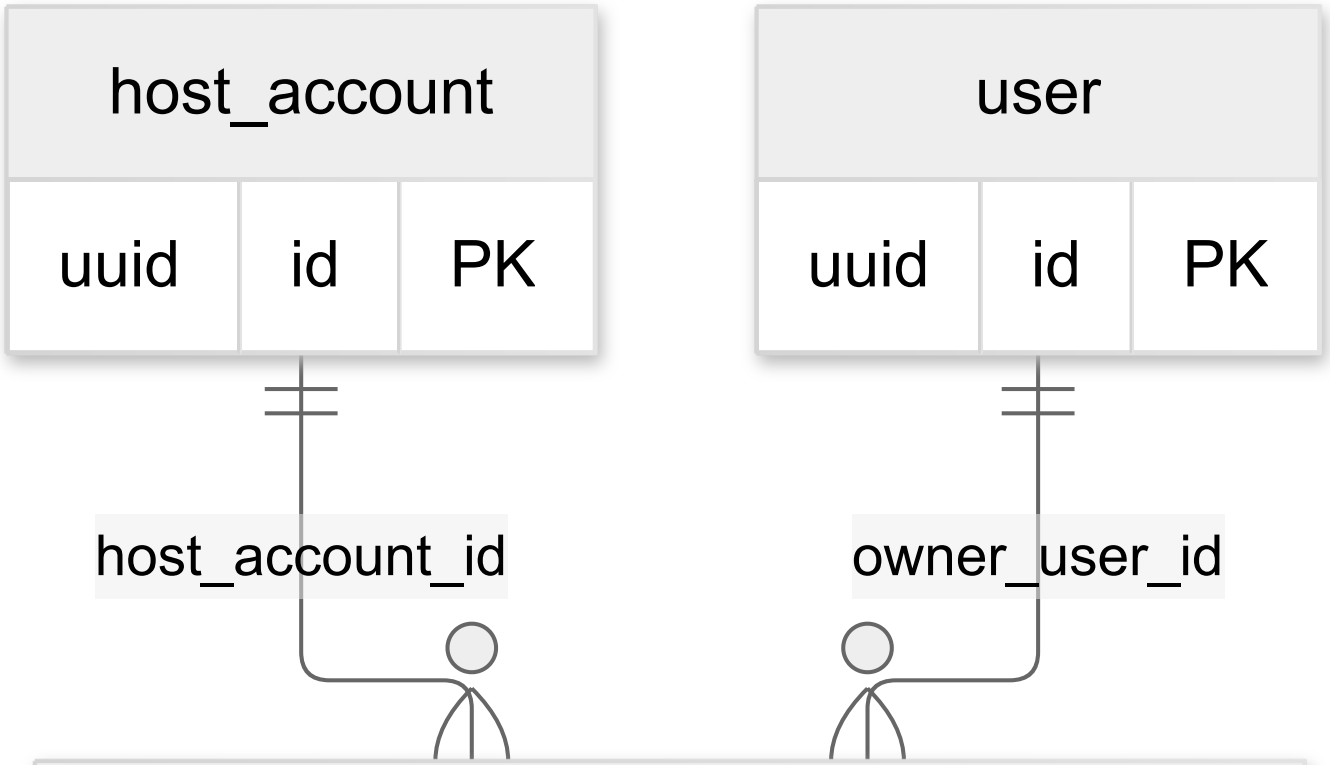
erd-12-messaging-and-notifications



Conversations, messages, notifications, preferences, the outbox and delivery log.

erd-13-media

BLADUS ERD - Media

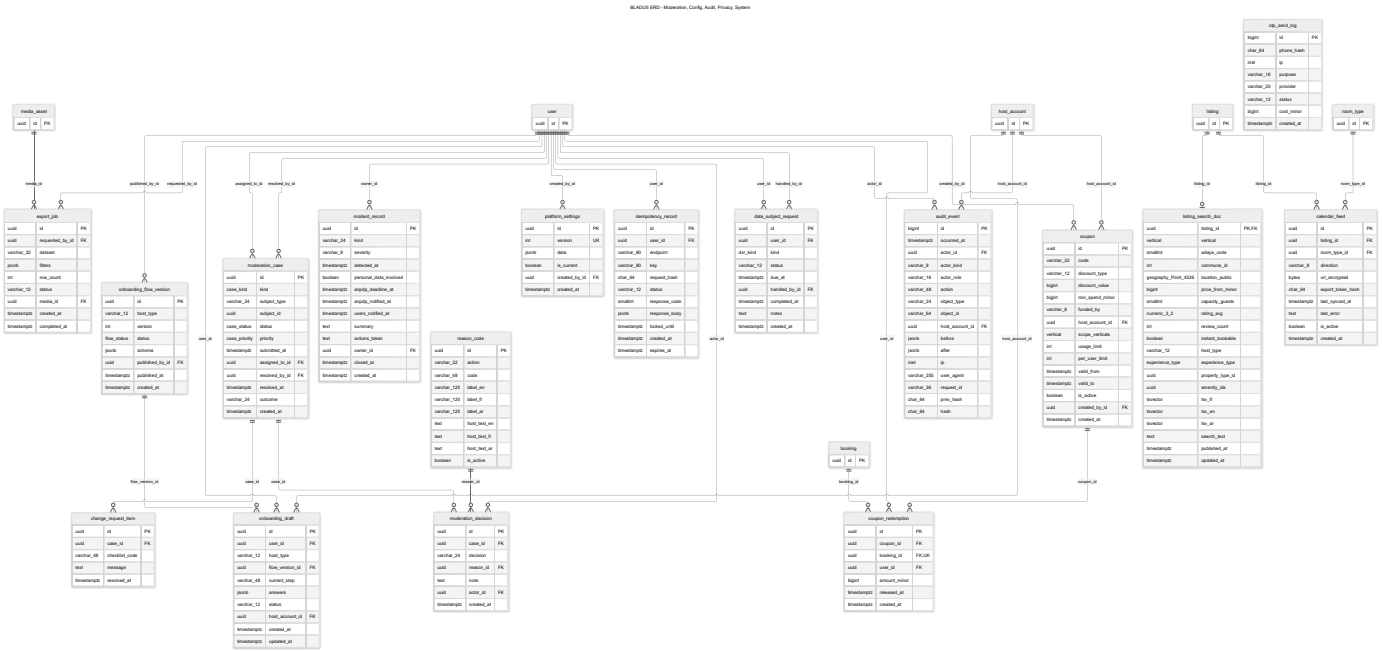


media_asset		
uuid	id	PK
uuid	owner_user_id	FK
uuid	host_account_id	FK
varchar_20	purpose	
media_bucket	bucket	

varchar_255	storage_key	UK
varchar_64	mime_type	
bigint	size_bytes	
int	width	
int	height	
char_64	sha256	
bigint	phash	
media_status	status	
varchar_32	scan_result	
timestamptz	created_at	
timestamptz	deleted_at	

Uploaded files (public photos, private documents).

erd-14-moderation-config-audit-privacy-system



Staff queues and decisions, settings and onboarding versions, coupons, audit log, exports, privacy requests, idempotency, search table.

05. Database schema

> **In plain words**
> This is the plan for the database: which tables exist, how they link, and which rules the database itself enforces.
> It has 77 tables in 14 groups. The full column list is in 05b-data-dictionary.md and the machine-readable version in schema.dbml.
> The most important ideas: one "host account" for every kind of host; the database refuses double bookings; money is kept in a ledger that is never edited.
> Each big choice is written as a short decision record (ADR) with the options we rejected.
> At the end: how today's demo data maps onto the new tables, what to seed first, and how the busiest queries will run.

Terms used here (see glossary.md):

- **Table / row / column**: a table is like a spreadsheet tab; a row is one record; a column is one field.
- **Primary key (PK)**: the column that identifies a row. **Foreign key (FK)**: a column pointing to a row in another table.
- **Constraint**: a rule the database checks on every write (unique, check, exclusion, foreign key).
- **Index**: a lookup structure that makes searches fast, like the index at the back of a book.
- **Partial index**: an index (often unique) that applies only to rows matching a condition, e.g. "only active owners".
- **GiST / GIN**: PostgreSQL index types for ranges, map points, arrays and text search.
- **Migration**: a versioned script that creates or changes tables. Not written yet (no backend code in this phase).

Inputs: 01-system-inventory.md, 03-requirements.md, 04-business-rules.md, research 02-research.md section B.

---

1. Shape of the database

Group (Django app)	Tables	Purpose
accounts	user, staff_profile, staff_permission, staff_role_permission, consent_record, favorite	People, staff roles, consents
geo	wilaya, daira, commune	69 wilayas, 1,541 communes
hosts	host_account, organization, membership, invitation, host_document, payout_account	Who hosts, teams, verification
catalog	category, property_type, amenity, cancellation_policy, listing, listing_revision, vehicle_detail, experience_detail, room_type, listing_amenity, room_type_amenity, listing_photo, featured_placement	What is offered
inventory	listing_day, room_type_night, experience_slot	What is free, and at what price, per day or slot
trips	trip, trip_itinerary_day, trip_departure	Agency multi-day products
pricing	quote, tax_rate	Server-computed prices
bookings	booking, booking_event, refund, dispute	Reservations and their history
ledger	ledger_account, ledger_transaction, ledger_entry, account_balance, wallet_topup, gateway_event, payment*, payout*, invoice	Money (* dormant until the money model changes)
reviews	review, review_reply, review_report	Reviews after stays
messaging, notifications	conversation, conversation_participant, message, notification, notification_preference, outbox_event, delivery_log	Messages, alerts, reliable sending
media	media_asset	Photos and private documents
trust, config, audit, privacy, core	incident_record, otp_send_log, calendar_feed (inventory), reason_code, moderation_case, moderation_decision, change_request_item, platform_settings, onboarding_flow_version, onboarding_draft, coupon, coupon_redemption, audit_event, export_job, data_subject_request, idempotency_record, listing_search_doc	Staff work, settings, history, legal requests, system safety

Diagrams: diagrams/erd-00-overview shows the 26 core tables; one diagram per group follows (Phase 4).

---

2. Conventions

Topic	Convention	Why
IDs	UUIDv7 (time-ordered) made by the app; human references for people (`BLD-2026-7K2M9Q` , `LST-...`)	Not guessable (BR security API1), sortable, no `Date.now()` collisions
Money	`BIGINT` centimes in columns ending `_minor` , plus `currency char(3)`	Exact sums (ADR-006)
Rates	integer basis points `_bp` (1,200 = 12%)	No float rounding
Time	`timestamptz` in UTC; stay nights as `date` / `daterange` in Africa/Algiers	BR-SYS-05
Names in 3 languages	`name_en` , `name_fr` , `name_ar` columns on platform lists (django-modeltranslation)	Research B.9
User text	original text + `source_language`	Hosts write once
Soft delete	`deleted_at` (+ `status_before_delete` where restore exists)	Admin restore (U7)
Who did it	always an FK to `user.id` , never a name	BR-STF-03
Cached numbers	ratings, counts, balances are caches recomputed by jobs; never edited by hand	BR-REV-04, BR-PAY-03
Table names	Django will prefix with the app (`bookings_booking`); this document uses short names	Readability

---

3. Architecture decision records (ADRs)

Each ADR: the context, the decision, the options rejected, the consequences.

ADR-001: One `host_account` for every kind of host

- **Context**: the website has users with organisations; the admin has a flat `Host` row; one person may host alone and also run an agency (DEC-004, disagreement D-1).
- **Decision**: `host_account` is either one user (`kind = individual`) or one organisation (`kind = organization`). Listings, trips, wallets, commission overrides, verification and reviews point to the host account.
- **Rejected**: (a) host fields on `user` (cannot represent organisations); (b) only organisations, with a "solo organisation" per individual (confusing for individuals, more rows, forces team concepts on solo hosts).
- **Consequences**: one wallet per host account; Karim's individual hosting and his agency have separate wallets and commission rates; the API carries an "active host account" header.

ADR-002: Database-enforced availability, two mechanisms

- **Context**: double booking must be impossible (AVL-01, AVL-02); today it relies on single-threaded JavaScript.
- **Decision**: single units (homes, vehicles) use a `daterange` on `booking` with an **exclusion constraint**; pooled units (hotel room types, hostel beds, experience slots, trip departures) use **counter rows** (`room_type_night`, `experience_slot`, `trip_departure`) with `CHECK (held + booked <= total)` and an atomic conditional `UPDATE`.
- **Rejected**: (a) application locks only (lost under multiple servers); (b) one row per physical room for hotels (heavier; hosts do not assign rooms at booking time); (c) Redis counters (not transactional with the booking).
- **Consequences**: concurrent requests for the last unit: exactly one succeeds, the other gets `409 dates_unavailable`. Counter rows are created 18 months ahead by a job.

ADR-003: One booking table with typed targets

- **Context**: bookings target a listing, a room type, an experience slot or a trip departure.
- **Decision**: `booking` has `inventory_kind` and four nullable target columns; a check constraint requires exactly the right ones. MVP: one room type per booking (multi-room bookings = several bookings in one checkout later).
- **Rejected**: generic foreign keys (no database integrity); one table per kind (four copies of the money and status columns).
- **Consequences**: every booking report reads one table.

ADR-004: Trips are their own tables (DEC-006)

- **Decision**: `trip`, `trip_itinerary_day`, `trip_departure`. Trips share the `listing_status` enum and moderation flow but not the `listing` table.
- **Rejected**: listing subtype (admin today): itinerary and departures do not fit listing columns.
- **Consequences**: search runs on listings and trips separately; the website's guest "Trips" page should be renamed "My bookings" in the API (`/me/bookings`) to avoid the name clash (D-11).

ADR-005: Verticals

- **Decision**: `vertical = home, hotel, hostel, experience, vehicle` (DEC-005 plus hostel split out because its unit is a bed). The search tab "Hotels" filters `hotel, hostel`; analytics group `home, hotel, hostel` as "stays".
- **Rejected**: hotel as a business category of home (spec): breaks search filters and analytics (D-12).

ADR-006: Money in centimes with a double-entry ledger

- **Context**: DEC-001 (prepaid commission now, maybe collected payments later); research B.17 and A.5.
- **Decision**: `ledger_account`, `ledger_transaction`, `ledger_entry` (insert-only, sum to zero per transaction, enforced by a deferred trigger) and `account_balance` cache. Amounts in centimes.
- **Rejected**: single-sided wallet rows (both apps today: cannot prove that money balances); `DECIMAL` with django-money (repository archived 2026).
- **Consequences**: M2 to M4 only add accounts and transaction kinds; no schema rewrite. Corrections are reversing transactions.

ADR-007: Snapshots on the booking

- **Decision**: prices, commission base, rate and amount, cancellation windows, listing title and address are copied onto the booking at creation (BR-BKG-07).
- **Rejected**: read live values (old bookings would change when hosts edit).
- **Consequences**: some duplication by design; editing a listing never alters a booking.

ADR-008: Statuses as PostgreSQL enums with transition checks in services

- **Decision**: enum types for statuses; allowed transitions in service code with a `booking_event` row per change; the database rejects unknown values.
- **Rejected**: transition triggers in SQL (hard to test and translate errors); free text statuses (today's three conflicting sets).
- **Consequences**: adding a status is a migration (rare, deliberate).

ADR-009: Listing revisions for edits under review

- **Decision**: edits to reviewed fields of a published listing go to `listing_revision` (JSON payload) while the live listing stays visible (BR-LST-02).
- **Rejected**: unpublishing on every edit (hosts lose bookings); no review of edits (fraud path: approved listing turned into a fake one).

ADR-010: Versioned config as JSON validated by the app

- **Decision**: `platform_settings` and `onboarding_flow_version` store a validated JSON document per version; old versions stay readable.
- **Rejected**: one column per setting (migration per setting); key-value rows (no validation, no atomic versions).

ADR-011: Search table, PostgreSQL only

- **Decision**: `listing_search_doc` holds one denormalised row per published listing, refreshed through the outbox; queries combine it with live availability (research B.7).
- **Rejected**: external search engine at launch.

ADR-012: Audit as a hash-chained, insert-only table plus automatic row history

- **Decision**: `audit_event` (human "who did what") with `prev_hash` / `hash`, monthly partitions, no UPDATE/DELETE grant; django-pghistory tables for automatic row history of bookings, listings, memberships, settings (research B.11).
- **Rejected**: signal-based history only (misses bulk updates).

ADR-013: Geography as reference tables with provenance

- **Decision**: `wilaya` (code PK 1 to 69), `daira`, `commune` with `source` and `valid_from`; listings store `commune_id`; the wilaya is derived. `legacy_code_58` keeps old data mappable.
- **Rejected**: free text city (today).

ADR-014: Personal data minimisation in the schema

- **Decision**: document numbers stored only as HMAC hashes; payout account numbers encrypted with a key outside the database; exact listing location separate from public location; message `body_masked`.
- **Consequences**: support cannot read raw ID numbers from the database; a key management step exists in operations (08-security-privacy.md, Phase 7).

---

4. The key tables in plain words

Table	One sentence
`user`	A person; may be a guest, a host, an organisation member and staff at the same time.
`host_account`	"The one who hosts": either one person or one organisation. Owns listings, trips and a wallet.
`organization` / `membership`	An agency, hotel or hostel and who works in it with which role.
`listing`	Something to book: a home, hotel, hostel, experience or vehicle.
`room_type`	A kind of room or bed in a hotel or hostel, with a count of identical units.
`room_type_night`	For one room type and one night: how many units exist, are held, are booked.
`listing_day`	For a home or vehicle: a day the host blocked or priced differently.
`trip` / `trip_departure`	An agency trip and its dated departures with seats.
`booking`	One reservation with its frozen prices, commission and policy.
`booking_event`	Every status change of a booking, in order.
`ledger_*`	Money movements written as balanced entries; balances are sums.
`wallet_topup`	A host's deposit recorded by finance staff, with its receipt reference.
`review`	A rating and comment after a completed booking.
`conversation` / `message`	Message threads between guest and host.
`moderation_case`	One item in a staff queue (a host to verify, a listing to review, a report).
`audit_event`	A tamper-evident record of who did what.
`outbox_event`	A message to send after a change is saved (notification, email, search refresh).

---

5. Booking lifecycle in the schema

1. Guest asks for a price: a `quote` row (30 minutes).
2. Guest books with an `Idempotency-Key`: `idempotency_record` locks the key.
3. In one transaction: insert `booking` (status `pending` or `confirmed`), reserve inventory (exclusion constraint or counter `UPDATE`), insert `booking_event`, and if confirmed post the commission `ledger_transaction` (2 entries) and update `account_balance`; insert `outbox_event` rows (notify host and guest, refresh search).
4. Commit. Workers send notifications from the outbox.
5. Jobs later: expire pending (`expired`, release counters), complete after check-out (`completed`), open review window.

---

6. Constraints the database enforces (SQL sketch)

These are written as SQL for clarity; they will become Django `Meta.constraints` or migration operations in the build phase.

```
-- Extensions
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS btree_gist;   -- equality inside exclusion constraints
CREATE EXTENSION IF NOT EXISTS pg_trgm;      -- fuzzy text search
CREATE EXTENSION IF NOT EXISTS unaccent;

-- BR-AVL-01: no overlapping stays on a single unit
```

```
ALTER TABLE booking ADD CONSTRAINT booking_no_overlap
  EXCLUDE USING gist (listing_id WITH =, stay WITH &&)
  WHERE (inventory_kind = 'single'
         AND status IN ('awaiting_payment','pending','confirmed','completed','disputed'));

-- ADR-003: targets match the inventory kind
ALTER TABLE booking ADD CONSTRAINT booking_target_matches_kind CHECK (
  (inventory_kind = 'single'    AND listing_id IS NOT NULL AND room_type_id IS NULL AND experience_slot_id IS NULL AND trip_departure_id IS NULL) OR
  (inventory_kind = 'room_pool' AND listing_id IS NOT NULL AND room_type_id IS NOT NULL AND experience_slot_id IS NULL AND trip_departure_id IS NULL) OR
  (inventory_kind = 'slot'      AND listing_id IS NOT NULL AND experience_slot_id IS NOT NULL AND room_type_id IS NULL AND trip_departure_id IS NULL) OR
  (inventory_kind = 'departure' AND trip_departure_id IS NOT NULL AND listing_id IS NULL AND room_type_id IS NULL AND experience_slot_id IS NULL));

ALTER TABLE booking ADD CONSTRAINT booking_money_ok CHECK (
  subtotal_minor >= 0 AND discount_minor >= 0 AND cleaning_fee_minor >= 0 AND taxes_minor >= 0
  AND total_minor = subtotal_minor - discount_minor + cleaning_fee_minor + taxes_minor
  AND refund_due_minor BETWEEN 0 AND total_minor
  AND commission_rate_bp BETWEEN 0 AND 10000 AND commission_minor >= 0
  AND NOT isempty(stay) AND lower_inc(stay) AND NOT upper_inc(stay));

-- BR-AVL-02: pooled inventory never oversold
ALTER TABLE room_type_night ADD CONSTRAINT rtn_capacity
  CHECK (held >= 0 AND booked >= 0 AND total >= 0 AND held + booked <= total);
ALTER TABLE trip_departure ADD CONSTRAINT dep_capacity
  CHECK (seats_held >= 0 AND seats_booked >= 0 AND seats_held + seats_booked <= seats_total AND ends_on >= starts_on);
ALTER TABLE experience_slot ADD CONSTRAINT slot_capacity
  CHECK (seats_held >= 0 AND seats_booked >= 0 AND seats_held + seats_booked <= seats_total AND ends_at > starts_at);

-- Reserving 2 rooms for nights [2026-12-10, 2026-12-13): one statement, all nights or nothing
UPDATE room_type_night SET booked = booked + 2
 WHERE room_type_id = :rt AND night >= '2026-12-10' AND night < '2026-12-13'
   AND held + booked + 2 <= total;
-- the service checks that the updated row count = number of nights, else rolls back (409)

-- ADR-001: host account is exactly one of user or organisation
ALTER TABLE host_account ADD CONSTRAINT host_account_one_owner CHECK (
  (kind = 'individual'   AND user_id IS NOT NULL AND organization_id IS NULL) OR
  (kind = 'organization' AND organization_id IS NOT NULL AND user_id IS NULL));

-- BR-ORG-04: exactly one active owner
CREATE UNIQUE INDEX membership_one_owner ON membership (organization_id)
  WHERE role = 'owner' AND is_active;
CREATE UNIQUE INDEX membership_active_unique ON membership (organization_id, user_id) WHERE is_active;

-- BR-ACC-01
CREATE UNIQUE INDEX user_email_ci ON "user" (lower(email)) WHERE email IS NOT NULL;
CREATE UNIQUE INDEX user_phone ON "user" (phone_e164) WHERE phone_e164 IS NOT NULL;

-- BR-PAY-02: balanced, insert-only ledger
CREATE FUNCTION ledger_tx_balanced() RETURNS trigger AS $$
BEGIN
  IF EXISTS (SELECT 1 FROM ledger_entry WHERE transaction_id = NEW.transaction_id
             GROUP BY currency HAVING sum(amount_minor) <> 0) THEN
    RAISE EXCEPTION 'ledger transaction % is not balanced', NEW.transaction_id;
  END IF;
  RETURN NULL;
END $$ LANGUAGE plpgsql;
CREATE CONSTRAINT TRIGGER ledger_entry_balanced AFTER INSERT ON ledger_entry
  DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION ledger_tx_balanced();
ALTER TABLE ledger_entry ADD CONSTRAINT ledger_entry_nonzero CHECK (amount_minor <> 0);
REVOKE UPDATE, DELETE ON ledger_entry, ledger_transaction, audit_event, booking_event FROM bladus_app;

-- BR-PAY-04: commission posted once per booking
CREATE UNIQUE INDEX ledger_one_commission ON ledger_transaction (booking_id) WHERE kind = 'commission';

-- BR-LST-02, BR-CFG-02, BR-MOD: one open item at a time
CREATE UNIQUE INDEX listing_one_pending_revision ON listing_revision (listing_id) WHERE status = 'pending_review';
CREATE UNIQUE INDEX flow_one_draft ON onboarding_flow_version (host_type) WHERE status = 'draft';
CREATE UNIQUE INDEX flow_one_published ON onboarding_flow_version (host_type) WHERE status = 'published';
CREATE UNIQUE INDEX case_one_open ON moderation_case (kind, subject_id) WHERE status <> 'resolved';
CREATE UNIQUE INDEX settings_one_current ON platform_settings (is_current) WHERE is_current;
```

Other checks (listed in each table's note in the dictionary): ratings 1 to 5, discounts 0 to 9,000 bp, `min_nights <= max_nights`, `listing.inventory_kind` consistent with `vertical`, favourites and photos pointing to exactly one parent.

---

7. Indexes and query plan for the busiest queries

Targets from NFR-01 and NFR-02. "Plan" is the expected PostgreSQL strategy; it will be confirmed with `EXPLAIN ANALYZE` on seeded data (20,000 listings, 1 million booking rows) in the build phase.

#	Query	Frequency	Indexes	Expected plan	Target p95
Q1	Search: vertical + wilaya + guests + price + amenities + map box, page of 24	very high	`listing_search_doc`: B-tree `(vertical, wilaya_code, price_from_minor)`, GiST `location_public`, GIN `amenity_ids`	Bitmap AND of B-tree and GIN, GiST for map box; then availability anti-join (Q2) on at most a few hundred candidates	500 ms
Q2	Availability for candidates and dates	very high	`booking` GiST from the exclusion constraint `(listing_id, stay)`; `room_type_night` PK `(room_type_id, night)`; `listing_day` PK	`NOT EXISTS` probe on the exclusion index per candidate; range scan on night PK for pools	inside Q1
Q3	Listing detail + calendar 90 days	high	PKs; `room_type (listing_id)`; `listing_photo (listing_id, sort_order)`	index scans	150 ms
Q4	Create booking (transaction)	medium	constraint indexes, `idempotency_record` unique	row locks on counters only	600 ms
Q5	Host bookings list by host account + status + dates	high	`booking (host_account_id, status, check_in_at)`	index range scan	200 ms
Q6	Guest bookings	high	`booking (guest_id, created_at DESC)`	index scan	150 ms
Q7	Message sync since cursor	very high (polling)	`message (conversation_id, id)`; `conversation_participant (user_id)`	index range `id > cursor`	100 ms

#	Query	Frequency	Indexes	Expected plan	Target p95
Q8	Unread notifications	very high	`notification (user_id, read_at, id)`	index only	50 ms
Q9	Moderation queue by kind, oldest first	medium	`moderation_case (status, kind, submitted_at)`	index scan	200 ms
Q10	Wallet balance and last entries	medium	`account_balance` PK; `ledger_entry (account_id, id DESC)`	PK + index scan	100 ms
Q11	Expire pending requests (job, every 5 min)	job	partial `booking (request_expires_at) WHERE status = 'pending'`	index range	1 s
Q12	Complete stays (job)	job	partial `booking (check_out_at) WHERE status = 'confirmed'`	index range	1 s
Q13	Outbox dispatch	constant	`outbox_event (status, next_attempt_at)` with `FOR UPDATE SKIP LOCKED`	index range	50 ms
Q14	Admin lists with facets (listings, hosts, bookings)	medium	B-tree on filter columns; counts via `GROUP BY` on the filtered set	bitmap scans	400 ms
Q15	Commune autocomplete	high	GIN trigram on `unaccent(name_fr)`, `name_ar`	GIN	50 ms
Q16	Audit by object	low	`audit_event (object_type, object_id, occurred_at)` on monthly partitions	partition pruning	300 ms

Growth estimate for capacity planning (first two years, assumption): 20,000 listings, 500,000 bookings, 5 million `room_type_night` rows (18 months x 10,000 room types x ~30 nights per month), 10 million messages, 20 million ledger entries. All fit comfortably on one PostgreSQL primary with 8 vCPU / 32 GB; partition `audit_event` and `message` by month when they pass 50 million rows.

---

8. Mapping today's data to the new schema

8.1 Entities

Website (frontend)	Admin	New table(s)	Notes
`User` (role, hostType, organizationId)	`GuestUser`, `Host` (separate)	`user` + `host_account(kind=individual)` + `membership`	admin `Host` of type individual becomes a host account of its owner user
`Organization` (businessCategory)	`Host` with hostType agency/hotel/hostel + metadata	`organization` + `host_account(kind=organization)`	admin `agencyMetadata`, `hotelMetadata`, `hostelMetadata` become columns or `policies`
`Membership`	`OrganizationMember` (unused)	`membership`	
`Listing` (+ vehicle, experience fields)	`Listing`	`listing` + `vehicle_detail` / `experience_detail`	`instantBook` and `bookingApprovalMode` merge into `approval_mode` (D-6)
`RoomType`	`RoomType`	`room_type`	`totalInventory` becomes `unit_count`; `availableInventory` dropped (computed)
`Trip` (departureDates[])	`Listing` with category trip	`trip`, `trip_itinerary_day`, `trip_departure`	each date string becomes a departure with `seats_total = maxGroupSize`
`Reservation`	`Booking`	`booking` + `booking_event`	ids re-generated; old ids kept in a migration map
`DateAvailabilityOverride`	none	`listing_day` or `room_type_night`	
`Wallet`, `Transaction` (prepaid)	`HostWallet`, `WalletTransaction`	`ledger_*`, `account_balance`, `wallet_topup`	each old transaction becomes a balanced transaction against `platform_cash` or `commission_revenue`
none	`PayoutRequest`	`payout` (dormant)	M1 has no payouts
`Review` (5 sub-ratings)	`ReviewItem` (status, flags, reply)	`review`, `review_reply`, `review_report`	
`Conversation`, `Message`	mocked	`conversation`, `conversation_participant`, `message`	
`Notification`	mocked	`notification`	
`Coupon`	`PromotionCoupon`	`coupon`	
amenities (keys), categories (keys)	names {en,fr,ar}, free strings	`amenity`, `category`, `property_type`	free strings mapped by a lookup table
onboarding config in code / localStorage	`OnboardingConfig`	`onboarding_flow_version` (v1 per host type from the website's 12/5/7/7 steps)	
none	`AuditEntry`	`audit_event`	
none	`PlatformSettings`	`platform_settings`	

8.2 Enum mapping

Concept	Website value	Admin value	New value
Listing status	active	approved, published (legacy)	published
	pending	pending_review, pending (legacy)	pending_review
	draft	draft	draft
	suspended	suspended	suspended (staff) or paused (host)
	n/a	changes_requested, rejected, refused (legacy), archived, deleted	same names (refused becomes rejected)
Booking status	pending, confirmed, cancelled, completed, declined, expired	pending, confirmed, cancelled, completed, disputed	same names

Concept	Website value	Admin value	New value
	n/a	refunded	booking stays in its status; `payment_status` and `refund_due_minor` carry the refund
Payment status	paid, pending, refunded, partially_refunded, voided	paid, partially_refunded, refunded, pending, failed	M1: `not_collected` for all demo bookings (the "paid" labels were never true, O7)
Transaction type	recharge	wallet_recharge	`wallet_topup`
	commission_deduction	commission_deduction	`commission`
	adjustment (reversal)	booking_refund (host share)	`commission_reversal`
	payout	host_payout	`payout` (dormant)
	n/a	booking_payment	`guest_payment` (dormant)
Onboarding field type	multi-select, image-upload, toggle	multiselect, file, boolean, radio	multi_select, image_upload / file_upload, toggle, select (radio becomes select with a display hint)
Wilaya code	number 1 to 58	string "01" to "68"	smallint 1 to 69 via `legacy_code_58` and a name match

8.3 Data that is not migrated

The apps run on demo data only. Nothing real is migrated: the demo seeds are re-created by the seed plan below. The mapping above is still needed so the two apps' screens can switch to the API without surprises.

---

9. Seed plan

Order	Data	Source	Size
1	Extensions, enums	migrations	
2	`wilaya`, `daira`, `commune`	el-amin-dev/algeria-wilayas-communes (CC0) names + ihahachi/Algeria-Cities (MIT) coordinates; keep `source`	69 / ~591 / 1,541
3	`staff_permission`, `staff_role_permission`	admin `lib/permissions.ts` matrix	34 permissions x 5 roles
4	`amenity`, `category`, `property_type`	merge of the three vocabularies, translated	~80 / ~20 / ~25
5	`cancellation_policy` v1	BR-CAN-01	3
6	`reason_code`	admin reasons + new catalogue	~60
7	`platform_settings` v1	defaults: 1,200 / 1,500 / 1,800 / 1,000 bp; request TTL 24; credit limit 20,000 DZD	1
8	`onboarding_flow_version` v1 per host type	website `lib/onboarding-config.ts` converted to canonical types	4
9	Demo fixtures (dev and staging only)	website demo users (Yacine, Amina, Karim...), admin staff (Nadia...), 18 listings, 4 trips, reservations, wallets	factory_boy factories; never in production
10	`room_type_night` rows	job generating 18 months ahead	computed

Production seeds are only steps 1 to 8. Demo passwords exist only in dev fixtures, never in the website bundle (fixes the current leak).

Additions on 2026-11-09

- `experience_type` and `trip_type` enums with `listing.experience_type` and `trip.trip_type` (DEC-015, BR-LST-05).
- Host offer rule (DEC-014, BR-HST-05) documented on `listing`: a trigger checks the vertical against the host account kind and organisation category.
- `incident_record` (breach register and 5-day clock, PRV-05), `calendar_feed` (iCal, AVL-06), `otp_send_log` (one-time code sends, ACC-02). These closed decision T5.
- schema.sql: the PostgreSQL rendering of `schema.dbml` produced by the official DBML tool.

Check yourself

- [ ] What does `host_account` replace, and which disagreement does it solve?
- [ ] Two guests click "Reserve" on the last hotel room at the same second. Which constraint decides, and what does the loser see?
- [ ] Why does the ledger refuse `UPDATE`? How is a mistake corrected?
- [ ] Which tables are dormant until the money model changes?
- [ ] Which index makes message polling cheap?
- [ ] Map the admin status `refunded` to the new model.

05b. Data dictionary

> **In plain words**
> Every table and every column of the planned database, with what it means.
> It is generated from schema.dbml, so it is always identical to the schema.
> "Key" says if a column is the row's identity (PK), a link to another table (FK) or must be unique (UQ).
> Money columns end in `_minor` (centimes). Rates end in `_bp` (basis points: 1,200 = 12%).
> Read 05-database-schema.md first for the design and the reasons behind it.

Generated by `python _tools/build_schema_docs.py`. 77 tables, 808 columns, 35 enumerations, 156 foreign keys.

Contents

- **Accounts**: user, staff_profile, staff_permission, staff_role_permission, consent_record, favorite
- **Geography**: wilaya, daira, commune
- **Host Accounts And Organisations**: host_account, organization, membership, invitation, host_document, payout_account
- **Taxonomy**: category, property_type, amenity, cancellation_policy
- **Catalogue: Listings And Room Types**: listing, listing_revision, vehicle_detail, experience_detail, room_type, listing_amenity, room_type_amenity, listing_photo, featured_placement
- **Inventory And Calendar**: listing_day, room_type_night, experience_slot
- **Trips (Agency Only)**: trip, trip_itinerary_day, trip_departure
- **Pricing**: quote, tax_rate
- **Bookings**: booking, booking_event, refund, dispute
- **Money: Ledger, Wallet, Dormant Payments**: ledger_account, ledger_transaction, ledger_entry, account_balance, wallet_topup, gateway_event, payment, payout, invoice
- **Reviews**: review, review_reply, review_report
- **Messaging And Notifications**: conversation, conversation_participant, message, notification, notification_preference, outbox_event, delivery_log
- **Media**: media_asset
- **Moderation, Config, Audit, Privacy, System**: reason_code, moderation_case, moderation_decision, change_request_item, platform_settings, onboarding_flow_version, onboarding_draft, coupon, coupon_redemption, audit_event, export_job, data_subject_request, idempotency_record, incident_record, calendar_feed, otp_send_log, listing_search_doc
- Enumerations

Accounts

user

Every person: guest, host, organisation member, staff. Constraints: unique lower(email), unique phone_e164, check (email is not null or phone_e164 is not null). Auth tables (emails, MFA authenticators, sessions) come from django-allauth and are not repeated here.

Column	Type	Required	Key	Default	Meaning
`id`	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
`email`	varchar(254)	no			Login email. Unique on lower(email) when not null.
`email_verified_at`	timestamptz	no			Time of the event (UTC).
`phone_e164`	varchar(16)	no			E.164 phone, Algerian mobiles match ^+213[5-7][0-9]{8}$. Unique when not null.
`phone_verified_at`	timestamptz	no			Required before the first booking (DEC-012).
`password`	varchar(255)	no			Django password hash (argon2). Empty for phone-only accounts.
`first_name_latin`	varchar(80)	no			
`last_name_latin`	varchar(80)	no			
`full_name_ar`	varchar(160)	no			Name in Arabic script, stored separately from the Latin form.
`avatar_media_id`	uuid	no	FK media_asset		Link to the media_asset row (many to one).
`bio`	text	no			Short public introduction (max 500 characters).
`spoken_languages`	varchar(2)[]	no			Languages the person speaks, ISO 639-1 codes, shown on host profiles.
`home_commune_id`	int	no	FK commune		Where the person lives (optional, private).
`address_line`	varchar(255)	no			Private postal address (optional); never shown publicly.
`preferred_locale`	varchar(2)	yes		fr	en, fr or ar.
`nationality_iso2`	char(2)	no			Only when legally needed (foreign guest declaration, VERIFY).
`status`	user_status	yes		active	One of the user_status values.
`is_staff`	boolean	yes		false	Yes/no flag: is staff.
`date_joined`	timestamptz	yes			
`last_login_at`	timestamptz	no			Time of the event (UTC).
`deleted_at`	timestamptz	no			Soft delete.
`anonymised_at`	timestamptz	no			Set when personal data was erased after a deletion request (BR-PRV-02).

staff_profile

Staff-only data. Permission checks read staff_role_permission for role. MFA must be enrolled before status can be active (BR-ACC-04).

Column	Type	Required	Key	Default	Meaning
user_id	uuid	yes	PK FK user		Link to the user row (one to one).
role	staff_role_code	yes			One of the staff_role_code values.
status	staff_status	yes		invited	One of the staff_status values.
invited_by_id	uuid	no	FK user		Link to the user row (many to one).
invited_at	timestamptz	no			Time of the event (UTC).
activated_at	timestamptz	no			Time of the event (UTC).
deactivated_at	timestamptz	no			Time of the event (UTC).

staff_permission

Column	Type	Required	Key	Default	Meaning
code	varchar(64)	yes	PK		e.g. hosts.approve, bookings.refund (34 codes today).
description	text	no			

staff_role_permission

Role to permission matrix (Appendix B section 1.1), editable data rather than code.

Column	Type	Required	Key	Default	Meaning
role	staff_role_code	yes	PK		One of the staff_role_code values.
permission_code	varchar(64)	yes	PK FK staff_permission		Link to the staff_permission row (many to one).

consent_record

Append-only history of consents (BR-PRV-01). Latest row per (user, document) is current.

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
user_id	uuid	yes	FK user		Link to the user row (many to one).
document	varchar(32)	yes			terms, privacy, marketing.
version	varchar(32)	yes			
accepted	boolean	yes			Yes/no flag: accepted.
ip	inet	no			
user_agent	text	no			
created_at	timestamptz	yes			When the row was created (UTC).

favorite

Check exactly one of listing_id, trip_id. Unique (user_id, listing_id) and (user_id, trip_id).

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
user_id	uuid	yes	FK user		Link to the user row (many to one).
listing_id	uuid	no	FK listing		Link to the listing row (many to one).
trip_id	uuid	no	FK trip		Link to the trip row (many to one).
created_at	timestamptz	yes			When the row was created (UTC).

Geography

wilaya

Column	Type	Required	Key	Default	Meaning
code	smallint	yes	PK		Official code 1 to 69 (codes 59 to 69 to confirm with the JO and ONS).

Column	Type	Required	Key	Default	Meaning
name_fr	varchar(80)	yes			French text.
name_ar	varchar(80)	yes			Arabic text.
name_en	varchar(80)	yes			English text.
legacy_code_58	smallint	no			Code of the parent wilaya in the 58-wilaya system, for old data.
region	varchar(16)	no			north, high_plateaus, south.
boundary	geometry(MultiPolygon,4326)	no			
centroid	geography(Point,4326)	no			
source	varchar(64)	yes			Dataset name and commit, e.g. el-amin-dev@sha.
valid_from	date	yes			2026-04-05 for Loi 26-06 wilayas.

daira

Column	Type	Required	Key	Default	Meaning
id	int	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
wilaya_code	smallint	yes	FK wilaya		Link to the wilaya row (many to one).
name_fr	varchar(80)	yes			French text.
name_ar	varchar(80)	yes			Arabic text.

commune

1,541 rows. GIN trigram index on unaccent(name_fr) and name_ar for autocomplete.

Column	Type	Required	Key	Default	Meaning
id	int	yes	PK		Official commune code.
wilaya_code	smallint	yes	FK wilaya		Link to the wilaya row (many to one).
daira_id	int	no	FK daira		Link to the daira row (many to one).
name_fr	varchar(100)	yes			French text.
name_ar	varchar(100)	yes			Arabic text.
name_en	varchar(100)	yes			English text.
postal_code	varchar(5)	no			
centroid	geography(Point,4326)	no			
source	varchar(64)	yes			
valid_from	date	yes			

Host Accounts And Organisations

host_account

The single thing that hosts: either one user (individual) or one organisation. Owns listings, trips, wallet and commission (resolves disagreement D-1). The active host account of a request comes from the session and is re-checked against membership on every call (workspace switcher, ORG-02). Check: (kind = individual and user_id not null and organization_id is null) or (kind = organization and organization_id not null and user_id is null).

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
kind	host_account_kind	yes			One of the host_account_kind values.
user_id	uuid	no	FK user UQ		Set when kind = individual.
organization_id	uuid	no	FK organization UQ		Set when kind = organization.
verification_status	verification_status	yes		applied	One of the verification_status values.
status_before_delete	verification_status	no			One of the verification_status values.
default_approval_mode	approval_mode	yes		request	Default for new listings (BR-BKG-02).
commission_rate_override_bp	int	no			Null = use host-type default from settings. 0 to 10000.
credit_limit_override_minor	bigint	no			Null = settings default for the host type (BR-PAY-10).
host_cancellations_12m	int	yes		0	Cache recomputed by job.
rating_avg	numeric(3,2)	no			Cache from published reviews (BR-REV-04).

Column	Type	Required	Key	Default	Meaning
review_count	int	yes		0	
submitted_at	timestamptz	no			Last submission for review; drives queue waiting time.
approved_at	timestamptz	no			Time of the event (UTC).
created_at	timestamptz	yes			When the row was created (UTC).
updated_at	timestamptz	yes			When the row was last changed (UTC).
deleted_at	timestamptz	no			Soft delete time; null = not deleted.

organization

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
business_category	business_category	yes			Never changes after creation (BR-ORG-02).
display_name	varchar(120)	yes			
legal_name	varchar(160)	no			
slug	varchar(140)	yes	UQ		
description	text	no			
source_language	varchar(2)	yes		fr	
logo_media_id	uuid	no	FK media_asset		Link to the media_asset row (many to one).
cover_media_id	uuid	no	FK media_asset		Link to the media_asset row (many to one).
business_email	varchar(254)	no			
business_phone_e164	varchar(16)	no			
commune_id	int	no	FK commune		Link to the commune row (many to one).
address_line	varchar(255)	no			
rc_number	varchar(32)	no			Registre du commerce. Legal field (BR-ORG-06).
nif	varchar(20)	no			Tax id. Legal field.
nis	varchar(20)	no			Statistical id. Legal field.
article_imposition	varchar(20)	no			
tourism_licence_number	varchar(40)	no			Agencies. Legal field.
star_rating	smallint	no			Hotels 1 to 5; drives tourist tax.
check_in_time	time	no			
check_out_time	time	no			
policies	jsonb	no			Hostel age limits, curfew, other house policies.
team_size_range	varchar(16)	no			
created_by_id	uuid	yes	FK user		Link to the user row (many to one).
created_at	timestamptz	yes			When the row was created (UTC).
updated_at	timestamptz	yes			When the row was last changed (UTC).
deleted_at	timestamptz	no			Soft delete time; null = not deleted.

membership

Unique (organization_id, user_id) where is_active. Partial unique (organization_id) where role = owner and is_active: exactly one owner (BR-ORG-04).

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
organization_id	uuid	yes	FK organization		Link to the organization row (many to one).
user_id	uuid	yes	FK user		Link to the user row (many to one).
role	membership_role	yes			One of the membership_role values.
is_active	boolean	yes		true	Yes/no flag: is active.
joined_at	timestamptz	yes			Time of the event (UTC).
removed_at	timestamptz	no			Time of the event (UTC).

invitation

Check email or phone present. Partial unique (organization_id, lower(email)) where status = pending.

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
organization_id	uuid	yes	FK organization		Link to the organization row (many to one).
email	varchar(254)	no			
phone_e164	varchar(16)	no			
role	membership_role	yes			Cannot be owner.
token_hash	char(64)	yes	UQ		SHA-256 of the one-time token; the token itself is only sent.
status	invitation_status	yes		pending	One of the invitation_status values.
invited_by_id	uuid	yes	FK user		Link to the user row (many to one).
expires_at	timestamptz	yes			Created + 7 days.
accepted_by_id	uuid	no	FK user		Link to the user row (many to one).
accepted_at	timestamptz	no			Time of the event (UTC).
created_at	timestamptz	yes			When the row was created (UTC).

host_document

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
host_account_id	uuid	yes	FK host_account		Link to the host_account row (many to one).
listing_id	uuid	no	FK listing		For per-vehicle or per-property documents.
doc_type	document_type	yes			One of the document_type values.
media_id	uuid	yes	FK media_asset		Private bucket only.
status	document_status	yes		pending	One of the document_status values.
number_hash	char(64)	no			HMAC of the document number, kept after the image is deleted (BR-PRV-05).
expires_on	date	no			
reviewed_by_id	uuid	no	FK user		Link to the user row (many to one).
reviewed_at	timestamptz	no			Time of the event (UTC).
rejection_reason_id	uuid	no	FK reason_code		Link to the reason_code row (many to one).
created_at	timestamptz	yes			When the row was created (UTC).
deleted_at	timestamptz	no			Soft delete time; null = not deleted.

payout_account

Used for refunds of deposit and, later, payouts (M4).

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
host_account_id	uuid	yes	FK host_account		Link to the host_account row (many to one).
kind	varchar(16)	yes			ccp_rip, bank_rib, baridimob.
holder_name	varchar(160)	yes			
number_encrypted	bytea	yes			Encrypted with a key outside the database.
number_last4	char(4)	yes			
is_default	boolean	yes		false	Yes/no flag: is default.
hold_until	timestamptz	no			Changes block payouts for 72 h (BR-PAY-09).
created_at	timestamptz	yes			When the row was created (UTC).
deleted_at	timestamptz	no			Soft delete time; null = not deleted.

Taxonomy

category

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
vertical	vertical	yes			One of the vertical values.

Column	Type	Required	Key	Default	Meaning
slug	varchar(60)	yes			
name_en	varchar(80)	yes			English text.
name_fr	varchar(80)	yes			French text.
name_ar	varchar(80)	yes			Arabic text.
icon	varchar(40)	no			
sort_order	int	yes		0	Display order, smallest first.
is_active	boolean	yes		true	Yes/no flag: is active.

Indexes: `(vertical, slug) [unique]`

property_type

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
vertical	vertical	yes			One of the vertical values.
slug	varchar(60)	yes			
name_en	varchar(80)	yes			English text.
name_fr	varchar(80)	yes			French text.
name_ar	varchar(80)	yes			Arabic text.
is_active	boolean	yes		true	Yes/no flag: is active.

Indexes: `(vertical, slug) [unique]`

amenity

One vocabulary replacing the three in the apps today.

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
slug	varchar(60)	yes	UQ		
group_code	varchar(20)	yes			essentials, features, location, safety.
name_en	varchar(80)	yes			English text.
name_fr	varchar(80)	yes			French text.
name_ar	varchar(80)	yes			Arabic text.
icon	varchar(40)	no			
applies_to	vertical[]	yes			Which verticals may use it.
is_active	boolean	yes		true	Yes/no flag: is active.

cancellation_policy

Partial unique (code) where is_current. Bookings copy the windows (snapshot).

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
code	varchar(20)	yes			flexible, moderate, strict (DEC-011).
version	int	yes			
name_en	varchar(60)	yes			English text.
name_fr	varchar(60)	yes			French text.
name_ar	varchar(60)	yes			Arabic text.
windows	jsonb	yes			Ordered [{min_days_before, refund_bp}], see BR-CAN-01.
is_current	boolean	yes			Yes/no flag: is current.
created_at	timestamptz	yes			When the row was created (UTC).

Indexes: `(code, version) [unique]`

Catalogue: Listings And Room Types

listing

Checks: capacity_guests > 0; base_price_minor > 0; min_nights <= max_nights; discount bp between 0 and 9000; inventory_kind matches vertical (home/vehicle single, hotel/hostel room_pool, experience slot); (vertical = experience) = (experience_type is not null). Host offer rule (DEC-014, BR-HST-05): individual host accounts publish home and vehicle, agencies experience, hotels and hostels hotel/hostel; enforced by service and a trigger that reads host_account.kind and organization.business_category.

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
reference	varchar(16)	yes	UQ		Human id, e.g. LST-7K2M9Q.
host_account_id	uuid	yes	FK host_account		Link to the host_account row (many to one).
vertical	vertical	yes			One of the vertical values.
status	listing_status	yes		draft	One of the listing_status values.
status_before_delete	listing_status	no			One of the listing_status values.
inventory_kind	inventory_kind	yes			single (home, vehicle), room_pool (hotel, hostel), slot (experience).
experience_type	experience_type	no			Required when vertical = experience, null otherwise (DEC-015, BR-LST-05).
title	varchar(120)	yes			
description	text	yes			
source_language	varchar(2)	yes			
category_id	uuid	no	FK category		Link to the category row (many to one).
property_type_id	uuid	no	FK property_type		Link to the property_type row (many to one).
commune_id	int	yes	FK commune		Link to the commune row (many to one).
address_line	varchar(255)	no			Private until a booking is confirmed (BR-SRC-03).
location_exact	geography(Point,4326)	yes			
location_public	geography(Point,4326)	yes			Offset up to 500 m, fixed.
capacity_guests	smallint	yes			
bedrooms	smallint	no			
beds	smallint	no			
bathrooms	numeric(3,1)	no			
surface_m2	int	no			
currency	char(3)	yes		DZD	ISO 4217 code, DZD today.
base_price_minor	bigint	yes			Per night (stays), per day (vehicles), per person (experiences).
cleaning_fee_minor	bigint	yes		0	Amount in centimes.
weekly_discount_bp	int	yes		0	
monthly_discount_bp	int	yes		0	
min_nights	smallint	yes		1	
max_nights	smallint	yes		90	
advance_notice_days	smallint	yes		0	
available_from	date	no			
available_until	date	no			
check_in_time	time	no			
check_out_time	time	no			
approval_mode	approval_mode	no			Null = inherit from host account (BR-BKG-02).
cancellation_policy_id	uuid	yes	FK cancellation_policy		Link to the cancellation_policy row (many to one).
house_rules	jsonb	no			List of rule codes plus free text.
rating_avg	numeric(3,2)	no			Rating 1 to 5 (avg).
review_count	int	yes		0	
submitted_at	timestamptz	no			Time of the event (UTC).
published_at	timestamptz	no			Time of the event (UTC).
created_at	timestamptz	yes			When the row was created (UTC).
updated_at	timestamptz	yes			When the row was last changed (UTC).
deleted_at	timestamptz	no			Soft delete time; null = not deleted.

listing_revision

Partial unique (listing_id) where status = pending_review.

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
listing_id	uuid	yes	FK listing		Link to the listing row (many to one).
payload	jsonb	yes			Proposed values of reviewed fields (BR-LST-02).
status	varchar(16)	yes			pending_review, approved, rejected, superseded.
submitted_by_id	uuid	yes	FK user		Link to the user row (many to one).
submitted_at	timestamptz	yes			Time of the event (UTC).
decided_by_id	uuid	no	FK user		Link to the user row (many to one).
decided_at	timestamptz	no			Time of the event (UTC).

vehicle_detail

Column	Type	Required	Key	Default	Meaning
listing_id	uuid	yes	PK FK listing		Link to the listing row (one to one).
make	varchar(40)	yes			
model	varchar(60)	yes			
year	smallint	yes			
seats	smallint	yes			
transmission	varchar(12)	yes			manual, automatic.
fuel	varchar(12)	yes			
daily_km_limit	int	no			
driver_min_age	smallint	yes		25	
licence_min_years	smallint	yes		2	
security_deposit_minor	bigint	no			Amount in centimes.

experience_detail

Column	Type	Required	Key	Default	Meaning
listing_id	uuid	yes	PK FK listing		Link to the listing row (one to one).
duration_minutes	int	yes			
languages	varchar(2)[]	yes			
meeting_point	geography(Point,4326)	no			
meeting_point_text	varchar(255)	no			
max_group	smallint	yes			
includes	jsonb	no			

room_type

Checks: unit_count > 0, capacity_per_unit > 0, base_price_minor > 0.

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
listing_id	uuid	yes	FK listing		Link to the listing row (many to one).
name	varchar(80)	yes			
description	text	no			
booking_unit	booking_unit	yes			room or bed (hostel dorms).
unit_count	smallint	yes			Physical rooms or beds of this type (inventory total).
capacity_per_unit	smallint	yes			
base_price_minor	bigint	yes			Amount in centimes.
beds_description	varchar(120)	no			
sort_order	int	yes		0	Display order, smallest first.
is_active	boolean	yes		true	Yes/no flag: is active.

Column	Type	Required	Key	Default	Meaning
created_at	timestamptz	yes			When the row was created (UTC).
deleted_at	timestamptz	no			Soft delete time; null = not deleted.

listing_amenity

Column	Type	Required	Key	Default	Meaning
listing_id	uuid	yes	PK FK listing		Link to the listing row (many to one).
amenity_id	uuid	yes	PK FK amenity		Link to the amenity row (many to one).

room_type_amenity

Column	Type	Required	Key	Default	Meaning
room_type_id	uuid	yes	PK FK room_type		Link to the room_type row (many to one).
amenity_id	uuid	yes	PK FK amenity		Link to the amenity row (many to one).

listing_photo

Check exactly one of listing_id, trip_id (room photos also carry listing_id). Partial unique cover per listing and per trip.

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
listing_id	uuid	no	FK listing		Link to the listing row (many to one).
room_type_id	uuid	no	FK room_type		Link to the room_type row (many to one).
trip_id	uuid	no	FK trip		Link to the trip row (many to one).
media_id	uuid	yes	FK media_asset		Link to the media_asset row (many to one).
sort_order	int	yes			Display order, smallest first.
caption	varchar(160)	no			
is_cover	boolean	yes		false	Yes/no flag: is cover.

featured_placement

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
listing_id	uuid	no	FK listing		Link to the listing row (many to one).
trip_id	uuid	no	FK trip		Link to the trip row (many to one).
placement	varchar(24)	yes			home_weekend, home_hero, search_top.
position	smallint	yes		0	Display order, smallest first.
starts_at	timestamptz	yes			Time of the event (UTC).
ends_at	timestamptz	no			Time of the event (UTC).
created_by_id	uuid	yes	FK user		Link to the user row (many to one).
created_at	timestamptz	yes			When the row was created (UTC).

Inventory And Calendar

listing_day

Host calendar for single-unit listings (homes, vehicles). Only rows that differ from defaults are stored.

Column	Type	Required	Key	Default	Meaning
listing_id	uuid	yes	PK FK listing		Link to the listing row (many to one).
day	date	yes	PK		
is_blocked	boolean	yes		false	Yes/no flag: is blocked.
price_override_minor	bigint	no			Amount in centimes.
note	varchar(120)	no			

room_type_night

Pooled inventory (BR-AVL-02). Check: held >= 0 and booked >= 0 and held + booked <= total. Rows created 18 months ahead by job.

Column	Type	Required	Key	Default	Meaning
room_type_id	uuid	yes	PK FK room_type		Link to the room_type row (many to one).
night	date	yes	PK		
total	smallint	yes			Units sellable that night (unit_count minus host blocks).
held	smallint	yes		0	
booked	smallint	yes		0	
price_override_minor	bigint	no			Amount in centimes.

experience_slot

Check: seats_held + seats_booked <= seats_total, all >= 0.

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
listing_id	uuid	yes	FK listing		Link to the listing row (many to one).
starts_at	timestamptz	yes			Time of the event (UTC).
ends_at	timestamptz	yes			Time of the event (UTC).
seats_total	smallint	yes			Seat counter (total).
seats_held	smallint	yes		0	Seat counter (held).
seats_booked	smallint	yes		0	Seat counter (booked).
price_override_minor	bigint	no			Amount in centimes.
is_cancelled	boolean	yes		false	Yes/no flag: is cancelled.

Indexes: `(listing_id, starts_at) [unique]`

Trips (Agency Only)

trip

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
reference	varchar(16)	yes	UQ		
slug	varchar(160)	yes	UQ		Public URL name, e.g. 5-day-grand-tassili (the website links trips by slug).
host_account_id	uuid	yes	FK host_account		Must be an approved agency (BR-TRP-01).
status	listing_status	yes		draft	One of the listing_status values.
status_before_delete	listing_status	no			One of the listing_status values.
trip_type	trip_type	yes			DEC-015.
title	varchar(140)	yes			
description	text	yes			
source_language	varchar(2)	yes			
duration_days	smallint	yes			
group_min	smallint	yes		1	
group_max	smallint	yes			
currency	char(3)	yes		DZD	ISO 4217 code, DZD today.
price_per_person_minor	bigint	yes			Amount in centimes.
inclusions	jsonb	yes			
exclusions	jsonb	yes			
start_commune_id	int	yes	FK commune		Link to the commune row (many to one).
meeting_point	geography(Point,4326)	no			
meeting_point_text	varchar(255)	no			
cancellation_policy_id	uuid	yes	FK cancellation_policy		Link to the cancellation_policy row (many to one).
approval_mode	approval_mode	no			One of the approval_mode values.

Column	Type	Required	Key	Default	Meaning
rating_avg	numeric(3,2)	no			Rating 1 to 5 (avg).
review_count	int	yes		0	
submitted_at	timestamptz	no			Time of the event (UTC).
published_at	timestamptz	no			Time of the event (UTC).
created_at	timestamptz	yes			When the row was created (UTC).
updated_at	timestamptz	yes			When the row was last changed (UTC).
deleted_at	timestamptz	no			Soft delete time; null = not deleted.

trip_itinerary_day

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
trip_id	uuid	yes	FK trip		Link to the trip row (many to one).
day_number	smallint	yes			
title	varchar(120)	yes			
description	text	no			
overnight_commune_id	int	no	FK commune		Link to the commune row (many to one).

Indexes: `(trip_id, day_number) [unique]`

trip_departure

Check: ends_on >= starts_on; seats_held + seats_booked <= seats_total (BR-TRP-02).

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
trip_id	uuid	yes	FK trip		Link to the trip row (many to one).
starts_on	date	yes			
ends_on	date	yes			
seats_total	smallint	yes			Seat counter (total).
seats_held	smallint	yes		0	Seat counter (held).
seats_booked	smallint	yes		0	Seat counter (booked).
price_per_person_minor	bigint	no			Null = trip price.
status	departure_status	yes		scheduled	One of the departure_status values.
guide_membership_id	uuid	no	FK membership		Link to the membership row (many to one).
created_at	timestamptz	yes			When the row was created (UTC).

Indexes: `(trip_id, starts_on) [unique]`

Pricing

quote

Short-lived; purged after 7 days by job.

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
guest_id	uuid	no	FK user		Link to the user row (many to one).
listing_id	uuid	no	FK listing		Link to the listing row (many to one).
room_type_id	uuid	no	FK room_type		Link to the room_type row (many to one).
experience_slot_id	uuid	no	FK experience_slot		Link to the experience_slot row (many to one).
trip_departure_id	uuid	no	FK trip_departure		Link to the trip_departure row (many to one).
stay	daterange	no			[check-in, check-out).
guests	smallint	yes			
units	smallint	yes		1	

Column	Type	Required	Key	Default	Meaning
currency	char(3)	yes			ISO 4217 code, DZD today.
lines	jsonb	yes			Per-night prices, discount, cleaning, taxes.
subtotal_minor	bigint	yes			Amount in centimes.
discount_minor	bigint	yes		0	Amount in centimes.
cleaning_fee_minor	bigint	yes		0	Amount in centimes.
taxes_minor	bigint	yes		0	Amount in centimes.
total_minor	bigint	yes			Amount in centimes.
expires_at	timestamptz	yes			Created + 30 min (BR-PRC-01).
created_at	timestamptz	yes			When the row was created (UTC).

tax_rate

R2. Values VERIFY WITH A PROFESSIONAL.

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
kind	varchar(16)	yes			taxe_sejour, vat.
star_rating	smallint	no			Taxe de sejour by hotel stars; null = unclassified.
rate_bp	int	no			VAT.
amount_per_person_night_minor	bigint	no			Taxe de sejour.
valid_from	date	yes			
valid_to	date	no			

Bookings

booking

Checks: exactly the target columns matching inventory_kind are set; total_minor = subtotal - discount + cleaning + taxes; 0 <= refund_due_minor <= total_minor; guests_adults >= 1. Exclusion (BR-AVL-01): EXCLUDE USING gist (listing_id WITH =, stay WITH &&) WHERE (inventory_kind = 'single' AND status IN ('awaiting_payment','pending','confirmed','completed','disputed')).

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
reference	varchar(20)	yes	UQ		BLD-YYYY-XXXXXX.
guest_id	uuid	yes	FK user		Link to the user row (many to one).
host_account_id	uuid	yes	FK host_account		Link to the host_account row (many to one).
inventory_kind	inventory_kind	yes			One of the inventory_kind values.
listing_id	uuid	no	FK listing		Link to the listing row (many to one).
room_type_id	uuid	no	FK room_type		Link to the room_type row (many to one).
experience_slot_id	uuid	no	FK experience_slot		Link to the experience_slot row (many to one).
trip_departure_id	uuid	no	FK trip_departure		Link to the trip_departure row (many to one).
stay	daterange	yes			Half-open [check-in, check-out). For slots and departures: their dates.
check_in_at	timestamptz	yes			Time of the event (UTC).
check_out_at	timestamptz	yes			Time of the event (UTC).
guests_adults	smallint	yes			
guests_children	smallint	yes		0	
units	smallint	yes		1	Rooms, beds or seats.
status	booking_status	yes			One of the booking_status values.
payment_status	payment_status	yes		not_collected	One of the payment_status values.
approval_mode_applied	approval_mode	yes			One of the approval_mode values.
request_expires_at	timestamptz	no			Set for pending (BR-BKG-04).
confirmed_at	timestamptz	no			Time of the event (UTC).
completed_at	timestamptz	no			Time of the event (UTC).
cancelled_at	timestamptz	no			Time of the event (UTC).

Column	Type	Required	Key	Default	Meaning
cancelled_by_role	actor_role	no			One of the actor_role values.
cancelled_by_id	uuid	no	FK user		Link to the user row (many to one).
cancel_reason_id	uuid	no	FK reason_code		Link to the reason_code row (many to one).
decline_reason_id	uuid	no	FK reason_code		Link to the reason_code row (many to one).
currency	char(3)	yes			ISO 4217 code, DZD today.
subtotal_minor	bigint	yes			Amount in centimes.
discount_minor	bigint	yes		0	Amount in centimes.
cleaning_fee_minor	bigint	yes		0	Amount in centimes.
taxes_minor	bigint	yes		0	Amount in centimes.
total_minor	bigint	yes			Amount in centimes.
commission_base_minor	bigint	yes			DEC-002.
commission_rate_bp	int	yes			Snapshot at creation.
commission_minor	bigint	yes			Amount in centimes.
refund_due_minor	bigint	yes		0	Cumulative amount owed back to the guest.
price_lines	jsonb	yes			
cancellation_windows	jsonb	yes			Snapshot of policy windows.
listing_snapshot	jsonb	yes			Title, photo, address at booking time.
quote_id	uuid	no	FK quote		Link to the quote row (many to one).
created_at	timestamptz	yes			When the row was created (UTC).
updated_at	timestamptz	yes			When the row was last changed (UTC).

Indexes: `(host_account_id, status, check_in_at) [note: 'Q5 host lists']; (guest_id, created_at) [note: 'Q6 guest lists']; request_expires_at [note: 'Q11, partial WHERE status = pending']; check_out_at [note: 'Q12, partial WHERE status = confirmed']`

booking_event

Append-only history of every status change.

Column	Type	Required	Key	Default	Meaning
id	bigint	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
booking_id	uuid	yes	FK booking		Link to the booking row (many to one).
from_status	booking_status	no			One of the booking_status values.
to_status	booking_status	yes			One of the booking_status values.
actor_role	actor_role	yes			One of the actor_role values.
actor_id	uuid	no	FK user		Link to the user row (many to one).
reason_id	uuid	no	FK reason_code		Link to the reason_code row (many to one).
note	text	no			
created_at	timestamptz	yes			When the row was created (UTC).

refund

Check amount_minor > 0. Sum per booking equals booking.refund_due_minor.

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
booking_id	uuid	yes	FK booking		Link to the booking row (many to one).
amount_minor	bigint	yes			Amount in centimes.
kind	varchar(20)	yes			policy, host_cancel, staff, dispute.
settlement	varchar(20)	yes		owed_by_host	M1: owed_by_host, settled_confirmed. M4: via gateway.
reason	text	no			
created_by_id	uuid	no	FK user		Link to the user row (many to one).
created_at	timestamptz	yes			When the row was created (UTC).

dispute

R2.

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
booking_id	uuid	yes	FK booking UQ		Link to the booking row (many to one).
opened_by_id	uuid	yes	FK user		Link to the user row (many to one).
opened_by_role	actor_role	yes			One of the actor_role values.
reason_id	uuid	no	FK reason_code		Link to the reason_code row (many to one).
guest_statement	text	no			
host_statement	text	no			
outcome	varchar(16)	no			refund_full, refund_partial, declined.
refund_minor	bigint	no			Amount in centimes.
resolution_note	text	no			
resolved_by_id	uuid	no	FK user		Link to the user row (many to one).
resolved_at	timestamptz	no			Time of the event (UTC).
created_at	timestamptz	yes			When the row was created (UTC).

Money: Ledger, Wallet, Dormant Payments

ledger_account

Partial unique (host_account_id, kind) where host_account_id is not null.

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
code	varchar(80)	yes	UQ		e.g. host_wallet:<host_account_id>, platform_cash:ccp_transfer.
kind	ledger_account_kind	yes			One of the ledger_account_kind values.
host_account_id	uuid	no	FK host_account		Link to the host_account row (many to one).
currency	char(3)	yes			ISO 4217 code, DZD today.
is_active	boolean	yes		true	Yes/no flag: is active.
created_at	timestamptz	yes			When the row was created (UTC).

ledger_transaction

Insert-only. Partial unique (booking_id) where kind = commission (BR-PAY-04).

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
kind	ledger_tx_kind	yes			One of the ledger_tx_kind values.
booking_id	uuid	no	FK booking		Link to the booking row (many to one).
topup_id	uuid	no	FK wallet_topup		Link to the wallet_topup row (many to one).
reverses_id	uuid	no	FK ledger_transaction		For reversals and corrections.
description	varchar(255)	yes			
created_by_id	uuid	no	FK user		Null = system.
occurred_at	timestamptz	yes			Time of the event (UTC).
created_at	timestamptz	yes			When the row was created (UTC).

ledger_entry

Insert-only. Deferred constraint trigger: per transaction and currency, sum(amount_minor) = 0 (BR-PAY-02).

Column	Type	Required	Key	Default	Meaning
id	bigint	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
transaction_id	uuid	yes	FK ledger_transaction		Link to the ledger_transaction row (many to one).
account_id	uuid	yes	FK ledger_account		Link to the ledger_account row (many to one).
amount_minor	bigint	yes			Positive = debit, negative = credit. Never 0.
currency	char(3)	yes			ISO 4217 code, DZD today.

account_balance

Cache updated in the same transaction as entries (row lock). Nightly job checks it equals the sum.

Column	Type	Required	Key	Default	Meaning
account_id	uuid	yes	PK FK ledger_account		Link to the ledger_account row (one to one).
balance_minor	bigint	yes			Amount in centimes.
last_entry_id	bigint	yes			
updated_at	timestamptz	yes			When the row was last changed (UTC).

wallet_topup

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
host_account_id	uuid	yes	FK host_account		Link to the host_account row (many to one).
method	topup_method	yes			One of the topup_method values.
amount_minor	bigint	yes			1 to 10,000,000 DZD in centimes.
receipt_reference	varchar(200)	yes			
status	topup_status	yes			One of the topup_status values.
requested_by_id	uuid	yes	FK user		Link to the user row (many to one).
approved_by_id	uuid	no	FK user		Second finance user above 500,000 DZD (BR-PAY-07).
approved_at	timestamptz	no			Time of the event (UTC).
rejection_reason	text	no			
gateway_event_id	uuid	no	FK gateway_event		Link to the gateway_event row (many to one).
created_at	timestamptz	yes			When the row was created (UTC).

Indexes: `(method, receipt_reference) [unique]`

gateway_event

R2. Raw webhooks (BR-SYS-02).

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
provider	varchar(20)	yes			chargily, slickpay.
event_id	varchar(100)	yes			
signature_valid	boolean	yes			Yes/no flag: signature valid.
payload	jsonb	yes			
received_at	timestamptz	yes			Time of the event (UTC).
processed_at	timestamptz	no			Time of the event (UTC).
error	text	no			

Indexes: `(provider, event_id) [unique]`

payment

DORMANT until money model M2 to M4 is approved (BR-PAY-11).

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
booking_id	uuid	no	FK booking		Link to the booking row (many to one).
provider	varchar(20)	yes			
provider_checkout_id	varchar(100)	yes			
amount_minor	bigint	yes			Amount in centimes.
currency	char(3)	yes			ISO 4217 code, DZD today.
status	varchar(16)	yes			
created_at	timestamptz	yes			When the row was created (UTC).

payout

DORMANT (M4). Also used for deposit withdrawals in M1 if the product allows them.

Column	Type	Required	Key	Default	Meaning
`id`	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
`host_account_id`	uuid	yes	FK host_account		Link to the host_account row (many to one).
`payout_account_id`	uuid	yes	FK payout_account		Link to the payout_account row (many to one).
`amount_minor`	bigint	yes			Amount in centimes.
`status`	varchar(16)	yes			requested, approved, sent, confirmed, rejected.
`approved_by_id`	uuid	no	FK user		Link to the user row (many to one).
`ledger_transaction_id`	uuid	no	FK ledger_transaction		Link to the ledger_transaction row (many to one).
`created_at`	timestamptz	yes			When the row was created (UTC).

invoice

R2. Commission invoices (BR-PAY-12). VERIFY WITH A PROFESSIONAL.

Column	Type	Required	Key	Default	Meaning
`id`	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
`host_account_id`	uuid	yes	FK host_account		Link to the host_account row (many to one).
`year`	smallint	yes			
`number`	int	yes			
`period_start`	date	yes			
`period_end`	date	yes			
`amount_ht_minor`	bigint	yes			Amount in centimes.
`vat_minor`	bigint	yes			Amount in centimes.
`amount_ttc_minor`	bigint	yes			Amount in centimes.
`status`	varchar(12)	yes			issued, cancelled.
`pdf_media_id`	uuid	no	FK media_asset		Link to the media_asset row (many to one).
`credit_note_of_id`	uuid	no	FK invoice		Link to the invoice row (many to one).
`issued_at`	timestamptz	yes			Time of the event (UTC).

Indexes: `(year, number) [unique]`

Reviews

review

Ratings between 1 and 5. Window: 14 days after check-out (BR-REV-01).

Column	Type	Required	Key	Default	Meaning
`id`	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
`booking_id`	uuid	yes	FK booking		Link to the booking row (many to one).
`author_id`	uuid	yes	FK user		Link to the user row (many to one).
`author_role`	actor_role	yes			guest or host.
`listing_id`	uuid	no	FK listing		Link to the listing row (many to one).
`trip_id`	uuid	no	FK trip		Link to the trip row (many to one).
`host_account_id`	uuid	yes	FK host_account		Link to the host_account row (many to one).
`rating_overall`	smallint	yes			Rating 1 to 5 (overall).
`rating_cleanliness`	smallint	no			Rating 1 to 5 (cleanliness).
`rating_accuracy`	smallint	no			Rating 1 to 5 (accuracy).
`rating_communication`	smallint	no			Rating 1 to 5 (communication).
`rating_location`	smallint	no			Rating 1 to 5 (location).
`rating_value`	smallint	no			Rating 1 to 5 (value).
`comment`	text	yes			
`language`	varchar(2)	yes			
`status`	review_status	yes		submitted	One of the review_status values.

Column	Type	Required	Key	Default	Meaning
submitted_at	timestamptz	yes			Time of the event (UTC).
published_at	timestamptz	no			Time of the event (UTC).

Indexes: (booking_id, author_role) [unique]

review_reply

Column	Type	Required	Key	Default	Meaning
review_id	uuid	yes	PK FK review		Link to the review row (one to one).
author_id	uuid	yes	FK user		Link to the user row (many to one).
body	text	yes			
created_at	timestamptz	yes			When the row was created (UTC).
editable_until	timestamptz	yes			

review_report

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
review_id	uuid	yes	FK review		Link to the review row (many to one).
reporter_id	uuid	yes	FK user		Link to the user row (many to one).
reason_id	uuid	yes	FK reason_code		Link to the reason_code row (many to one).
note	text	no			
status	varchar(12)	yes			open, dismissed, actioned.
created_at	timestamptz	yes			When the row was created (UTC).

Indexes: (review_id, reporter_id) [unique]

Messaging And Notifications

conversation

Partial unique (booking_id) where not null; partial unique (guest_id, listing_id) where kind = inquiry.

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
kind	conversation_kind	yes			One of the conversation_kind values.
guest_id	uuid	no	FK user		Link to the user row (many to one).
host_account_id	uuid	no	FK host_account		Link to the host_account row (many to one).
listing_id	uuid	no	FK listing		Link to the listing row (many to one).
trip_id	uuid	no	FK trip		Link to the trip row (many to one).
booking_id	uuid	no	FK booking		Link to the booking row (many to one).
last_message_at	timestamptz	no			Time of the event (UTC).
created_at	timestamptz	yes			When the row was created (UTC).

conversation_participant

Column	Type	Required	Key	Default	Meaning
conversation_id	uuid	yes	PK FK conversation		Link to the conversation row (many to one).
user_id	uuid	yes	PK FK user		Link to the user row (many to one).
role	varchar(12)	yes			guest, host_member, staff.
last_read_message_id	bigint	no			
muted	boolean	yes		false	Yes/no flag: muted.
joined_at	timestamptz	yes			Time of the event (UTC).

message

Column	Type	Required	Key	Default	Meaning
id	bigint	yes	PK		Ordered id, used as sync cursor (MSG-02).
conversation_id	uuid	yes	FK conversation		Link to the conversation row (many to one).
sender_id	uuid	no	FK user		Null for system messages.
kind	varchar(12)	yes			text, system.
body	text	yes			
body_masked	text	no			Shown to the other side before confirmation (BR-MSG-01).
contains_contact_info	boolean	yes		false	Yes/no flag: contains contact info.
created_at	timestamptz	yes			When the row was created (UTC).
deleted_at	timestamptz	no			Soft delete time; null = not deleted.

Indexes: (conversation_id, id)

notification

Column	Type	Required	Key	Default	Meaning
id	bigint	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
user_id	uuid	yes	FK user		Link to the user row (many to one).
kind	varchar(48)	yes			
params	jsonb	yes			Values for the localized template.
link	varchar(255)	no			
read_at	timestamptz	no			Time of the event (UTC).
created_at	timestamptz	yes			When the row was created (UTC).

Indexes: (user_id, read_at, id)

notification_preference

Column	Type	Required	Key	Default	Meaning
user_id	uuid	yes	PK FK user		Link to the user row (many to one).
kind	varchar(48)	yes	PK		
channel	varchar(8)	yes	PK		email, sms, push.
enabled	boolean	yes			Yes/no flag: enabled.

outbox_event

Written in the same transaction as the change (BR-SYS-03).

Column	Type	Required	Key	Default	Meaning
id	bigint	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
topic	varchar(64)	yes			
aggregate_type	varchar(32)	yes			
aggregate_id	uuid	yes			
payload	jsonb	yes			
status	outbox_status	yes		pending	One of the outbox_status values.
attempts	smallint	yes		0	
next_attempt_at	timestamptz	yes			Time of the event (UTC).
created_at	timestamptz	yes			When the row was created (UTC).
sent_at	timestamptz	no			Time of the event (UTC).

Indexes: (status, next_attempt_at)

delivery_log

Column	Type	Required	Key	Default	Meaning
id	bigint	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
outbox_event_id	bigint	no	FK outbox_event		Link to the outbox_event row (many to one).
channel	varchar(8)	yes			

Column	Type	Required	Key	Default	Meaning
provider	varchar(20)	yes			
recipient_hash	char(64)	yes			
status	varchar(12)	yes			
provider_message_id	varchar(100)	no			
cost_minor	bigint	no			Amount in centimes.
error	text	no			
created_at	timestamptz	yes			When the row was created (UTC).

Media

media_asset

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
owner_user_id	uuid	no	FK user		Link to the user row (many to one).
host_account_id	uuid	no	FK host_account		Link to the host_account row (many to one).
purpose	varchar(20)	yes			listing_photo, document, avatar, logo, invoice, export.
bucket	media_bucket	yes			One of the media_bucket values.
storage_key	varchar(255)	yes	UQ		
mime_type	varchar(64)	yes			
size_bytes	bigint	yes			
width	int	no			
height	int	no			
sha256	char(64)	yes			
phash	bigint	no			Perceptual hash for duplicate photos (BR-MED-01).
status	media_status	yes		quarantined	One of the media_status values.
scan_result	varchar(32)	no			
created_at	timestamptz	yes			When the row was created (UTC).
deleted_at	timestamptz	no			Soft delete time; null = not deleted.

Indexes: phash

Moderation, Config, Audit, Privacy, System

reason_code

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
action	varchar(32)	yes			reject_listing, decline_booking, cancel_booking, report_review ...
code	varchar(48)	yes			
label_en	varchar(120)	yes			English text.
label_fr	varchar(120)	yes			French text.
label_ar	varchar(120)	yes			Arabic text.
host_text_en	text	no			English text.
host_text_fr	text	no			French text.
host_text_ar	text	no			Arabic text.
is_active	boolean	yes		true	Yes/no flag: is active.

Indexes: (action, code) [unique]

moderation_case

Partial unique (kind, subject_id) where status <> resolved.

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
kind	case_kind	yes			One of the case_kind values.
subject_type	varchar(24)	yes			
subject_id	uuid	yes			
status	case_status	yes		open	One of the case_status values.
priority	case_priority	yes		normal	One of the case_priority values.
submitted_at	timestamptz	yes			Waiting time counts from here (BR-MOD-01).
assigned_to_id	uuid	no	FK user		Link to the user row (many to one).
resolved_by_id	uuid	no	FK user		Link to the user row (many to one).
resolved_at	timestamptz	no			Time of the event (UTC).
outcome	varchar(24)	no			
created_at	timestamptz	yes			When the row was created (UTC).

Indexes: (status, kind, submitted_at)

moderation_decision

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
case_id	uuid	yes	FK moderation_case		Link to the moderation_case row (many to one).
decision	varchar(24)	yes			
reason_id	uuid	no	FK reason_code		Link to the reason_code row (many to one).
note	text	no			
actor_id	uuid	yes	FK user		Link to the user row (many to one).
created_at	timestamptz	yes			When the row was created (UTC).

change_request_item

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
case_id	uuid	yes	FK moderation_case		Link to the moderation_case row (many to one).
checklist_code	varchar(48)	yes			
message	text	no			
resolved_at	timestamptz	no			Time of the event (UTC).

platform_settings

Partial unique (is_current) where is_current. New row per change (BR-CFG-01).

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
version	int	yes	UQ		
data	jsonb	yes			Validated whitelist: commission defaults (bp), credit limits, request_ttl_hours, instant_booking_enabled, maintenance_mode, staff session timeouts, locales, feature flags.
is_current	boolean	yes			Yes/no flag: is current.
created_by_id	uuid	yes	FK user		Link to the user row (many to one).
created_at	timestamptz	yes			When the row was created (UTC).

onboarding_flow_version

Partial unique (host_type) where status = draft; partial unique (host_type) where status = published.

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
host_type	varchar(12)	yes			individual, agency, hotel, hostel.
version	int	yes			

Column	Type	Required	Key	Default	Meaning
status	flow_status	yes			One of the flow_status values.
schema	jsonb	yes			Steps and fields with canonical types (BR-CFG-03).
published_by_id	uuid	no	FK user		Link to the user row (many to one).
published_at	timestamptz	no			Time of the event (UTC).
created_at	timestamptz	yes			When the row was created (UTC).

Indexes: (host_type, version) [unique]

onboarding_draft

Partial unique (user_id, host_type) where status = in_progress.

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
user_id	uuid	yes	FK user		Link to the user row (many to one).
host_type	varchar(12)	yes			
flow_version_id	uuid	yes	FK onboarding_flow_version		Link to the onboarding_flow_version row (many to one).
current_step	varchar(48)	yes			
answers	jsonb	yes			
status	varchar(12)	yes			in_progress, submitted, abandoned.
host_account_id	uuid	no	FK host_account		Link to the host_account row (many to one).
created_at	timestamptz	yes			When the row was created (UTC).
updated_at	timestamptz	yes			When the row was last changed (UTC).

coupon

R2. Unique lower(code).

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
code	varchar(32)	yes			
discount_type	varchar(12)	yes			percentage, fixed.
discount_value	bigint	yes			bp or centimes.
min_spend_minor	bigint	yes		0	Amount in centimes.
funded_by	varchar(8)	yes			platform, host (BR-PRM-02).
host_account_id	uuid	no	FK host_account		Link to the host_account row (many to one).
scope_verticals	vertical[]	no			
usage_limit	int	no			
per_user_limit	int	yes		1	
valid_from	timestamptz	yes			
valid_to	timestamptz	no			
is_active	boolean	yes		true	Yes/no flag: is active.
created_by_id	uuid	yes	FK user		Link to the user row (many to one).
created_at	timestamptz	yes			When the row was created (UTC).

coupon_redemption

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
coupon_id	uuid	yes	FK coupon		Link to the coupon row (many to one).
booking_id	uuid	yes	FK booking UQ		Link to the booking row (many to one).
user_id	uuid	yes	FK user		Link to the user row (many to one).
amount_minor	bigint	yes			Amount in centimes.
released_at	timestamptz	no			Time of the event (UTC).
created_at	timestamptz	yes			When the row was created (UTC).

audit_event

Insert-only for the app role; hash chain (BR-STF-02). Partitioned by month.

Column	Type	Required	Key	Default	Meaning
`id`	bigint	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
`occurred_at`	timestamptz	yes			Time of the event (UTC).
`actor_id`	uuid	no	FK user		Link to the user row (many to one).
`actor_kind`	varchar(8)	yes			staff, user, system.
`actor_role`	varchar(16)	no			
`action`	varchar(48)	yes			e.g. LISTING_APPROVE, WALLET_TOPUP (53 kinds today).
`object_type`	varchar(24)	yes			
`object_id`	varchar(64)	yes			
`host_account_id`	uuid	no	FK host_account		Link to the host_account row (many to one).
`before`	jsonb	no			
`after`	jsonb	no			
`ip`	inet	no			
`user_agent`	varchar(255)	no			
`request_id`	varchar(36)	no			
`prev_hash`	char(64)	yes			
`hash`	char(64)	yes			

Indexes: `(object_type, object_id, occurred_at)`; `(actor_id, occurred_at)`

export_job

Column	Type	Required	Key	Default	Meaning
`id`	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
`requested_by_id`	uuid	yes	FK user		Link to the user row (many to one).
`dataset`	varchar(32)	yes			
`filters`	jsonb	yes			
`row_count`	int	no			
`status`	varchar(12)	yes			
`media_id`	uuid	no	FK media_asset		Link to the media_asset row (many to one).
`created_at`	timestamptz	yes			When the row was created (UTC).
`completed_at`	timestamptz	no			Time of the event (UTC).

data_subject_request

Column	Type	Required	Key	Default	Meaning
`id`	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
`user_id`	uuid	yes	FK user		Link to the user row (many to one).
`kind`	dsr_kind	yes			One of the dsr_kind values.
`status`	varchar(12)	yes			
`due_at`	timestamptz	yes			Time of the event (UTC).
`handled_by_id`	uuid	no	FK user		Link to the user row (many to one).
`completed_at`	timestamptz	no			Time of the event (UTC).
`notes`	text	no			
`created_at`	timestamptz	yes			When the row was created (UTC).

idempotency_record

Column	Type	Required	Key	Default	Meaning
`id`	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
`user_id`	uuid	yes	FK user		Link to the user row (many to one).
`endpoint`	varchar(80)	yes			

Column	Type	Required	Key	Default	Meaning
key	varchar(80)	yes			
request_hash	char(64)	yes			
status	varchar(12)	yes			in_progress, completed.
response_code	smallint	no			
response_body	jsonb	no			
locked_until	timestamptz	no			
created_at	timestamptz	yes			When the row was created (UTC).
expires_at	timestamptz	yes			Created + 24 h (BR-SYS-01).

Indexes: `(user_id, endpoint, key) [unique]`

incident_record

Incident register and breach clock (PRV-05). Staff-only, audited.

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
kind	varchar(24)	yes			security_breach, data_leak, outage, fraud.
severity	varchar(8)	yes			low, medium, high, critical.
detected_at	timestamptz	yes			Time of the event (UTC).
personal_data_involved	boolean	yes			Yes/no flag: personal data involved.
anpdp_deadline_at	timestamptz	no			detected_at + 5 days when personal data is involved (Law 18-07 as amended, VERIFY WITH A PROFESSIONAL).
anpdp_notified_at	timestamptz	no			Time of the event (UTC).
users_notified_at	timestamptz	no			Time of the event (UTC).
summary	text	yes			
actions_taken	text	no			
owner_id	uuid	yes	FK user		Link to the user row (many to one).
closed_at	timestamptz	no			Time of the event (UTC).
created_at	timestamptz	yes			When the row was created (UTC).

calendar_feed

Later release (AVL-06): iCal import and export.

Column	Type	Required	Key	Default	Meaning
id	uuid	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
listing_id	uuid	yes	FK listing		Link to the listing row (many to one).
room_type_id	uuid	no	FK room_type		Link to the room_type row (many to one).
direction	varchar(8)	yes			import or export.
url_encrypted	bytea	no			Import URL (other platform), encrypted; fetched through the egress proxy with private-IP blocking (SSRF control).
export_token_hash	char(64)	no			Hash of the secret token in the export URL.
last_synced_at	timestamptz	no			Time of the event (UTC).
last_error	text	no			
is_active	boolean	yes		true	Yes/no flag: is active.
created_at	timestamptz	yes			When the row was created (UTC).

otp_send_log

Record of every one-time code sent (codes themselves stay in Redis, never stored). Feeds SMS budget alarms and pumping-fraud detection (ACC-02, BR-ACC-02). Kept 90 days.

Column	Type	Required	Key	Default	Meaning
id	bigint	yes	PK		Primary key (UUIDv7 made by the app, ordered by time).
phone_hash	char(64)	yes			HMAC of the phone number.
ip	inet	no			
purpose	varchar(16)	yes			signup, login, phone_change, payout_change.
provider	varchar(20)	yes			

Column	Type	Required	Key	Default	Meaning
status	varchar(12)	yes			sent, failed, blocked_rate_limit.
cost_minor	bigint	no			Amount in centimes.
created_at	timestamptz	yes			When the row was created (UTC).

Indexes: (phone_hash, created_at) ; (ip, created_at)

listing_search_doc

Denormalised row per published listing, refreshed by outbox job. GiST on location_public, GIN on amenity_ids, tsv_* and search_text (gin_trgm_ops).

Column	Type	Required	Key	Default	Meaning
listing_id	uuid	yes	PK FK listing		Link to the listing row (one to one).
vertical	vertical	yes			One of the vertical values.
wilaya_code	smallint	yes			
commune_id	int	yes			
location_public	geography(Point,4326)	yes			
price_from_minor	bigint	yes			Amount in centimes.
capacity_guests	smallint	yes			
rating_avg	numeric(3,2)	no			Rating 1 to 5 (avg).
review_count	int	yes			
instant_bookable	boolean	yes			Yes/no flag: instant bookable.
host_type	varchar(12)	yes			
experience_type	experience_type	no			For the experience type filter and facet counts.
property_type_id	uuid	no			
amenity_ids	uuid[]	yes			
tsv_fr	tsvector	no			French text.
tsv_en	tsvector	no			English text.
tsv_ar	tsvector	no			Arabic text.
search_text	text	no			Title and place names, unaccented, for trigram search.
published_at	timestamptz	yes			Time of the event (UTC).
updated_at	timestamptz	yes			When the row was last changed (UTC).

Enumerations

user_status: active , suspended , banned , deleted

staff_role_code: super_admin , moderator , support , finance , analyst

staff_status: invited , active , deactivated

host_account_kind: individual , organization

business_category: agency , hotel , hostel

verification_status: applied , under_review , changes_requested , approved , rejected , suspended , banned

membership_role: owner , admin , manager , agent , guide

invitation_status: pending , accepted , revoked , expired

document_type: national_id , passport , property_proof , lease_authorisation , apc_declaration , insurance , auto_entrepreneur_card , commercial_register , tax_card_nif , statistics_nis , tourism_licence , hotel_classification , operating_authorisation , vehicle_registration , vehicle_insurance , technical_inspection , driving_licence

document_status: pending , approved , rejected , expired

approval_mode: instant , request

vertical: home , hotel , hostel , experience , vehicle

experience_type: excursion , guided_tour , nature_adventure , desert , sport , culture_workshop , gastronomy , wellness

trip_type: group_tour , private_trip , adventure , desert_expedition , cultural , hiking , relaxation , religious

listing_status: draft , pending_review , changes_requested , published , paused , rejected , suspended , archived , deleted

booking_unit: room , bed

inventory_kind: single , room_pool , slot , departure

booking_status: `awaiting_payment`, `pending`, `confirmed`, `declined`, `expired`, `cancelled`, `completed`, `disputed`

payment_status: `not_collected`, `awaiting`, `paid`, `partially_refunded`, `refunded`, `failed`

actor_role: `guest`, `host`, `staff`, `system`

departure_status: `scheduled`, `guaranteed`, `cancelled`, `completed`

ledger_account_kind: `host_wallet`, `platform_cash`, `commission_revenue`, `vat_payable`, `guest_receivable`, `escrow`, `host_payable`, `payout_clearing`, `adjustments`

ledger_tx_kind: `wallet_topup`, `commission`, `commission_reversal`, `adjustment`, `guest_payment`, `refund`, `payout`

topup_method: `ccp_transfer`, `bank_transfer`, `baridimob`, `cash_office`, `gateway`

topup_status: `pending_approval`, `approved`, `rejected`

review_status: `submitted`, `published`, `flagged`, `hidden`, `removed`

conversation_kind: `inquiry`, `booking`, `support`

media_status: `quarantined`, `ready`, `rejected`, `deleted`

media_bucket: `public`, `private`

case_kind: `host_verification`, `listing_review`, `listing_revision`, `trip_review`, `document`, `review_report`, `dispute`, `risk_flag`, `legal_change`, `host_cancellations`

case_status: `open`, `in_progress`, `resolved`

case_priority: `low`, `normal`, `high`

flow_status: `draft`, `published`, `archived`

outbox_status: `pending`, `sent`, `failed`

dsr_kind: `access`, `rectification`, `deletion`, `opposition`

Check yourself

- [ ] Which table links a host to either a user or an organisation?
- [ ] In which column is the commission rate frozen on a booking, and in which unit?
- [ ] Which tables are marked DORMANT, and why?
- [ ] Where is the cursor used by message sync?

06. API design: how the apps talk to the backend

> **In plain words**
> This page explains the contract between the two Next.js apps (website and admin) and the Django backend.
> The exact contract is the file openapi.yaml: 266 operations, each one linked to the requirements and rules it serves.
> The browser never calls Django. The Next.js server calls Django for it, with a session token the browser never sees.
> Every function the apps use today in `lib/data` and their server actions gets an endpoint (table in section 8), or the table says why it goes away.
> Money is always in centimes, errors always have a stable code and a message in English, French or Arabic.
> Guest payments and payouts are written in the contract but switched off ("dormant") until the money model changes.

Terms used here (see also glossary.md):

- **API**: the list of requests one program can send to another and the answers it gets back.
- **REST**: a common style of API where each thing has a web address (a URL) and you act on it with HTTP verbs: GET (read), POST (create or do an action), PUT (replace), PATCH (change part), DELETE (remove).
- **Endpoint** (or operation): one verb on one URL, for example `POST /api/v1/me/bookings`.
- **OpenAPI**: a standard file format that describes every endpoint, its inputs and outputs. Tools read it to check the API, draw documentation and generate code.
- **BFF (backend for frontend)**: the server part of each Next.js app. It receives the browser's requests and calls Django on its behalf.
- **Session token**: a long random string that proves who you are to Django. django-allauth creates it at login; it can be cancelled at any time.
- **Idempotency key**: a random value the app sends with a "create" request. If the same request arrives twice (double click, network retry), Django does the work once and returns the same answer.
- **Cursor**: a bookmark returned with a page of results; you send it back to get the next page. Used for feeds that grow (messages, notifications).
- **ETag**: a version label of a response. The app sends it back to ask "has this changed?" or "save only if nobody changed it meanwhile".
- **Webhook**: a request that another company's server (a payment gateway) sends to us when something happens.
- **Dormant endpoint**: present in the contract, but returns 404 until a feature flag is switched on (`x-dormant: true`).

---

1. The contract file

Item	Value
File	openapi.yaml, OpenAPI 3.1.0
Size	266 operations on 220 paths, 38 tags in 7 groups
Groups	auth 15, public 22, me (guest) 26, shared (messaging 7, media 4), host 72, admin 119, webhooks 1
Dormant (money models M2 to M4)	8: `createGuestPayment`, `listHostPayouts`, `requestHostPayout`, `receiveGatewayWebhook`, `adminListPayouts`, `adminApprovePayout`, `adminRejectPayout`, `adminMarkPayoutSent`
Lint	`npx @redocly/cli lint openapi.yaml` (recommended rules): valid, 0 errors, 0 warnings
Enums	Copied from schema.dbml by name and value (each carries `x-db-enum`)

Every operation has: `operationId`, `summary`, `tags`, `security`, error responses, and these extensions:

Extension	Meaning
`x-requirements`	Requirement IDs from 03-requirements.md
`x-rules`	Rule IDs from 04-business-rules.md
`x-release`	MVP, R2 or Later
`x-dormant`	`true` for the switched-off money endpoints
`x-host-permission`	Organisation permission needed (host endpoints, table in section 4.4)
`x-staff-permission`	Staff permission needed (admin endpoints, section 4.5)

When Django is built, drf-spectacular generates the real file from the code. CI then compares it with this one (see section 10): this file is the target, not decoration.

Requirements not cited by any operation are the ones with no endpoint of their own: jobs and quality targets (AVL-05 holds, BKG-04 completion job, NTF-02 email/SMS, NTF-04 outbox, NFR-01 to NFR-10 and NFR-13, PRV-03 to PRV-05), plus two features not designed yet: MSG-06 staff broadcast (R2) and PRV-06 foreign guest declaration (R2). They are covered by the conventions below or left for a later phase.

---

2. Big picture

```
Browser ──(cookie)──> Next.js website server (BFF) ──(X-Session-Token)──> Django /api/v1/*
Browser ──(cookie)──> Next.js admin server (BFF)   ──(X-Session-Token)──> Django /api/v1/admin/*
Payment gateway ──────────────(signature header)──────────────────────────> Django /api/v1/webhooks/*
```

- Django is never exposed to browsers for app traffic. The public listing pages are rendered by Next.js, which calls the public endpoints.
- Images and documents are the exception: the browser uploads them straight to object storage with a presigned URL (section 3.10), so big files never go through Next.js or Django.

---

3. Conventions

3.1 URLs and naming

- Base path `/api/v1`. Production host `https://api.bladus.dz`.
- Nouns in plural, lowercase, words separated by `-` (`/room-types`). Path ids in `{snake_case}`.
- Groups by who is acting: `/api/v1/...` public, `/api/v1/me/...` the signed-in person, `/api/v1/host/...` the active host account, `/api/v1/admin/...` staff.
- Actions that are not plain edits use a verb sub-path with POST: `/host/bookings/{id}/accept`, `/admin/hosts/{id}/suspend`. One action, one permission, one audit row.
- Status changes with many targets use `POST .../status` with the target status in the body.

3.2 JSON

- Field names in **snake_case**, both ways.
- Missing optional values are `null`, not left out, in responses.
- Text written by users comes back in its original language with `source_language`. Platform labels (amenities, types, wilayas) come in the language asked by `Accept-Language`; `labels` gives all three when needed.

3.3 Dates and times

- Timestamps: ISO 8601 in UTC with `Z` (`2026-11-09T09:30:00Z`).
- Stay dates: plain dates (`2026-12-18`) meaning the day in Africa/Algiers. Ranges are half-open: `check_out` is the day you leave, it is not a night of the stay.
- The week starts on Sunday; Friday and Saturday are the weekend (BR-SYS-05). Digits are always Latin (0 to 9), also in Arabic.

3.4 Money

- Always the `Money` object: `{"amount_minor": 1850000, "currency": "DZD"}` = 18,500 DZD. `amount_minor` is an integer number of centimes (1 DZD = 100 centimes). Never a float.
- Inputs use `MoneyInput` (positive amount). Rates are basis points (`1200` = 12%).
- Query filters on prices say their unit in the name: `min_price_minor`.
- Totals are sums of rounded lines (half up), so the apps never recompute prices: they show what the quote says (BR-PAY-01, BR-PRC-01).

3.5 Identifiers

- Every id is a UUIDv7 (time-ordered, not guessable).
- People also see short references: bookings `BLD-2026-7K2M9Q`, listings `LST-7K2M9Q`, trips `TRP-...`. Endpoints that show one item accept either the UUID or the reference in the path (`booking_id`, `listing_id`, `trip_id`).
- Messages and notifications use growing integer ids, because they are also the sync cursor.

3.6 Pagination

Kind	Used by	Query	Response
Cursor	feeds and website lists: search, reviews, notifications, favourites, my bookings, host lists, wallet entries	`cursor`, `limit` (1 to 50, default 20)	`results`, `next_cursor` (null at the end), `has_more`
Page numbers	admin tables (need "page 4 of 12")	`page`, `page_size` (1 to 100, default 20)	`results`, `meta {page, page_size, total_count, total_pages}`, often `facets`

Cursors are opaque strings: the apps must not build or read them.

3.7 Errors

All errors use `application/problem+json`, in the RFC 9457 "Problem Details" style:

```
{
  "type": "https://api.bladus.dz/errors/dates_unavailable",
  "title": "Dates unavailable",
  "status": 409,
  "code": "dates_unavailable",
  "detail": "Another booking took 2026-12-19 while you were booking.",
  "message": "Ces dates viennent d'être réservées. Choisissez d'autres dates.",
  "instance": "/api/v1/me/bookings",
  "request_id": "01933c10-77aa-7d00-9a51-5b2b0b6f3a10",
  "errors": []
}
```

- `code` never changes once published. The apps map it to their own text when they want (the website's `ActionResult` codes become these codes, NFR-13).
- `message` is ready to show, in the language of `Accept-Language` (en, fr, ar; default fr), BR-SYS-04.
- `errors` lists field problems on 422: `{"field": "room_types.0.unit_count", "code": "out_of_range", "message": "...", "params": {"min": 1}}`.
- Some codes add members: `price_changed` adds the new `quote`; `permission_denied` adds `required_permission`; `night_booked` adds `dates`.

Status codes used

Status	When	Example codes
400	Malformed request; also all login failures (same answer every time)	`bad_request`, `invalid_credentials`, `invalid_code`, `invalid_current_password`
401	No valid session	`not_authenticated`

Status	When	Example codes
403	Signed in but not allowed	`permission_denied`, `mfa_required`, `phone_verification_required`, `host_offer_not_allowed`, `agency_not_verified`, `cannot_book_own_listing`, `four_eyes_same_user`, `invitation_contact_mismatch`, `account_suspended`
404	Not found, not yours, or a dormant feature switched off	`not_found`, `feature_disabled`, `not_an_organization`
409	Conflict with the current state	`dates_unavailable`, `not_enough_units`, `seats_unavailable`, `price_changed`, `quote_expired`, `booking_not_pending`, `request_expired`, `booking_not_cancellable`, `refund_changed`, `wallet_below_credit_floor`, `invalid_transition`, `receipt_already_used`, `topup_not_pending`, `idempotency_in_progress`, `email_taken`, `phone_taken`, `already_member`, `invitation_pending`, `invitation_used`, `cannot_remove_owner`, `review_exists`, `already_reported`, `reply_locked`, `inventory_in_use`, `night_booked`, `slug_taken`, `in_use`, `has_future_bookings`, `case_resolved`, `last_super_admin`
412	`If-Match` is stale	`stale_version`
422	Validation failed	`validation_failed` (with `errors`), `idempotency_key_reused`, `listing_incomplete`, `documents_missing`, `documents_not_approved`, `reason_required`, `refund_exceeds_total`, `review_window_closed`, `dispute_window_closed`, `invitation_expired`, `unknown_field`, `unknown_setting`, `core_step_removed`, `preference_locked`, `min_nights`, `max_nights`, `advance_notice`, `capacity_exceeded`, `file_type_not_allowed`, `file_too_large`
429	Rate limit	`rate_limited`, `login_locked` (with `Retry-After`)

A host who asks for another host account's data gets **404, not 403**, so ids cannot be probed (BR-ORG-01).

3.8 Filtering and sorting

- Filters are plain query parameters named like the field (`status`, `vertical`, `wilaya`). Lists of values are comma-separated: `?vertical=home,hotel`.
- Free text is always `q`.
- Sorting is `ordering`, with `-` for descending: `?ordering=-created`. Only the values listed in the contract are accepted.
- Facets: the admin list endpoints return `facets` with counts per value; public search has its own `GET /listings/facets`. A facet is counted without its own filter, so the user sees what they would get by changing it (BR-SRC-01).

3.9 Rate limits (per group)

Counters live in Redis, shared by all servers (ACC-07, NFR-11). Because all website traffic comes from the BFF, the BFF sends the real client IP in `X-Forwarded-For`; Django trusts that header **only** from the BFF's private network address.

Group	Limit (proposal, tune after launch)	Key
Login failures	5 in 15 min, then 60 s lock	account and IP
One-time codes	3 sends per phone per hour, 10 per IP per hour; 5 tries per code	phone, IP
Signup, password reset	10 per IP per hour	IP
Public reads (search, pages)	120 per minute; search and map 60 per minute	IP
Quotes	60 per minute	user or IP
Booking create	10 per minute, 50 per day	user
Messages	30 per minute; new conversations 20 per day	user
Uploads	120 per hour	user
Other user and host writes	120 per minute	user
Admin	600 per minute; exports 10 per hour	staff user

Responses over the limit are 429 with `Retry-After`. SMS sends also have a daily budget alarm.

3.10 Uploads

1. `POST /media/uploads` with purpose, type, size, hash: returns `media_id` and a presigned URL (valid 10 minutes).
2. The browser PUTs the file straight to storage (quarantine area).
3. `POST /media/{media_id}/complete`: checks start (type sniffing, virus scan for documents, metadata removal, renditions, duplicate detection).
4. The app polls `GET /media/{media_id}` until `ready` (or `rejected` with a code), then uses the `media_id` in a listing, room type, document or avatar. Private documents are only read through `GET .../url`, which returns a 5-minute link and writes an audit row (BR-MED-02).

3.11 Caching and ETags

Endpoint	Caching
`GET /reference`, `/geo/*`, `/onboarding-flows/{host_type}`, `/reference/reasons`	`Cache-Control: public, max-age=3600`, `ETag`; `If-None-Match` gives 304. Next.js caches them and revalidates by tag.
`GET /listings/{id}`, `/trips/{id}`, `/organizations/{slug}`	`ETag`; public when anonymous, `private` when the session adds personal fields
Search, facets, map	Short public cache (60 s) only for anonymous requests without dates
Everything under `/me`, `/host`, `/admin`	`Cache-Control: no-store`

Optimistic locking: edits where two people may work at once (listing, trip, organisation, settings, onboarding flow draft, commission rates) need `If-Match` with the ETag of the last read. If someone saved in between, the answer is 412 and the app reloads.

3.12 Other headers

Header	Direction	Use
`X-Session-Token`	BFF to Django	Who is calling
`X-Host-Account`	BFF to Django	Active host account (section 4.3)
`Accept-Language`	BFF to Django	en, fr, ar
`Idempotency-Key`	BFF to Django	Section 5
`X-Request-Id`	both	Correlation id; created by the BFF, echoed in logs and errors (NFR-09)
`Idempotency-Replayed: true`	Django to BFF	The response is a stored replay
`Deprecation`, `Sunset`	Django to BFF	Section 9

---

4. Authentication and permissions

4.1 Website users (BFF cookie plus allauth session token)

1. The login form posts to a Next.js route handler (today `app/api/auth/login/route.ts`).
2. The handler calls `POST /api/v1/auth/login` (or the phone code flow `POST /auth/code/request` then `POST /auth/code/confirm`).
3. Django (django-allauth headless, "app" client) answers with a **session token**. The handler stores it in an encrypted, `httpOnly`, `Secure`, `SameSite=Lax` cookie. JavaScript in the browser cannot read it.
4. On every server render or server action, the BFF decrypts the cookie and sends `X-Session-Token` to Django.
5. Logout calls `DELETE /api/v1/auth/session` (the token dies at once on the server, BR-ACC-03) and deletes the cookie.

If allauth answers with a pending step (`verify_phone`, `mfa_authenticate`), the BFF shows that step. Sessions last 30 days, sliding. A password change revokes all other sessions. The website keeps its CSRF guard on its own route handlers and server actions (same-origin check); Django itself only receives server-to-server calls with a header token, so it needs no CSRF cookie.

4.2 Staff (admin BFF, mandatory second factor)

- The admin app uses a separate, staff-only allauth mount at `/api/v1/admin/auth/*`. Non-staff accounts get the same `invalid_credentials` as a bad password.
- Login is always two steps: password, then TOTP or passkey (`POST /admin/auth/2fa/authenticate`). A staff member without a second factor can only enrol it; every other admin endpoint answers 403 `mfa_required` (BR-ACC-04).
- Sessions: 8 hours absolute, 30 minutes idle (settings). Cookie `SameSite=Strict`.
- `/api/v1/admin/*` refuses website tokens, and host and guest endpoints refuse staff tokens (STF-05). A staff member who is also a host uses two separate sign-ins.
- Deactivating a staff member or changing their role takes effect on the next request: permissions are read from the database every time.

4.3 The active host account

A **host account** is either one user (individual host) or one organisation (agency, hotel, hostel). One person can hold several (DEC-004).

1. `GET /me/workspaces` lists the host accounts the user can act for, with role, permissions, allowed verticals.
2. The workspace switcher calls `PUT /me/workspaces/active {host_account_id}`. Django stores it in the session. This replaces `switchWorkspaceAction`.
3. Every `/host/*` request acts for the account in the `X-Host-Account` header if present, else the one in the session.
4. On every request Django checks that the user owns that individual account or has an **active membership** in that organisation. If not: 404. Then it checks the role permission for the endpoint (403 `permission_denied` if missing).
5. All lists are filtered by that host account in the query itself, never after loading (BR-ORG-01).

The header lets two browser tabs work on two organisations at once without fighting over the session value.

4.4 Host permissions (organisation roles, BR-ORG-05, DEC-009)

Permission	owner	admin	manager	agent	guide
org.edit, members.manage	X	X			
org.delete, transfer ownership, payout_details.edit	X				
listings.manage, trips.manage, calendar.manage	X	X	X		
bookings.view	X	X	X	X	X (own departures)
bookings.respond (accept, decline)	X	X	X		
bookings.cancel_as_host	X	X			
messages.reply	X	X	X	X	X
wallet.view, invoices.view	X	X			

An individual host has every permission on their own host account. Agents and guides get `commission: null` in bookings (they never see money). The offer rule (DEC-014) comes on top of permissions: individual = home, vehicle; agency = experience and trips; hotel = hotel; hostel = hostel. A forbidden vertical gives 403 `host_offer_not_allowed`.

4.5 Staff permissions

The 34 permissions and 5 roles of the admin app (matrix in _work/admin-inventory.md section 1.1) move into the `staff_role_permission` table and are checked on every admin endpoint, deny by default (BR-STF-01). Each admin operation names its permission in `x-staff-permission`. This contract adds three permissions that the admin app does not have yet:

New permission	Why	Proposed roles
`reports.view`	Dashboard KPIs and charts were open to any staff (no check)	all 5 roles
`ledger.adjust`	Ledger corrections did not exist	super_admin, finance
`privacy.handle_requests`	Data protection requests (PRV-02)	super_admin, support

---

5. Idempotency and concurrency

5.1 Idempotency keys (BR-SYS-01)

Required on: booking create, onboarding submit, listing and trip create, staff refunds, manual top-ups, ledger adjustments, departure cancel, online top-up and the dormant payment endpoints. Optional (recommended) on accept, decline, cancel, top-up approval and moderation decisions.

Situation	Answer
First call with key K	Normal processing; response stored 24 h with the hash of the body
Same K, same body, first call finished	Stored response, same status, header `Idempotency-Replayed: true`
Same K while the first call still runs	409 `idempotency_in_progress` (retry after a moment)
Same K, different body	422 `idempotency_key_reused`

The BFF creates the key when the user opens the confirm step (not when they click), so a double click or a network retry sends the same key. Keys are per user and per endpoint (`idempotency_record` table).

5.2 The booking race

Two guests press "Book" for the same nights at the same second:

1. Both requests open a database transaction and re-price their quote.
2. Single units (homes, vehicles): both try to insert a booking. The **exclusion constraint** on `(listing, stay)` lets exactly one succeed; the other gets an error from PostgreSQL that Django turns into **409** `dates_unavailable` (BR-AVL-01).
3. Pooled units (rooms, beds, slots, seats): each request runs one `UPDATE ... SET held/booked = booked + n WHERE booked + held + n <= total` per night. The loser updates 0 rows and gets **409** `not_enough_units` or `seats_unavailable` (BR-AVL-02, BR-TRP-02).
4. The winner gets 201. The loser's app shows the message and reloads availability.

No application lock is needed; the database is the referee.

5.3 Other 409 cases

- Status moves that are not allowed by the state machines: `invalid_transition` (listings, trips, hosts, reviews, top-ups).
- Host accepts a request that expired a second ago: `request_expired` . Host accepts while the wallet is below the credit floor: `wallet_below_credit_floor` (BR-PAY-10).
- Quote no longer valid at booking: `price_changed` (with the new quote) or `quote_expired` .
- Duplicates: `receipt_already_used` (same receipt and method), `email_taken` , `review_exists` , `slug_taken` .
- Shrinking inventory under existing bookings: `inventory_in_use` , `night_booked` .

---

6. Webhooks (dormant)

`POST /api/v1/webhooks/gateway/{provider}` receives gateway callbacks (Chargily first). It is **dormant**: it returns 404 until online top-up (R2, PAY-06) or guest payments (M2 to M4) are switched on.

When active (BR-SYS-02, BR-PAY-08):

1. Store the raw body and headers in `gateway_event` first.
2. Check the signature (HMAC-SHA256 of the raw body with the provider secret, header `signature`), mark `signature_valid` .
3. Deduplicate by `(provider, event_id)` (database unique key). A replay answers 200 `{"received": true, "duplicate": true}` and does nothing.
4. Answer 200 within a second; a Celery worker processes the event once (credit the wallet, mark the top-up).
5. Never trust the amount in the browser return URL; only the verified webhook credits money.

BLADUS does not send outgoing webhooks to partners in this version.

---

7. How the Next.js apps consume the API

> Note: both apps run a Next.js version with breaking changes. Before writing the client code, read the guides in `node_modules/next/dist/docs/` of each app, as `frontend/AGENTS.md` asks.

1. **Types**: `openapi-typescript openapi.yaml -o lib/api/schema.d.ts` in each app (a script in package.json, run in CI). No hand-written API types.
2. **Client**: one server-only module `lib/api/client.ts` per app with `openapi-fetch` 's `createClient<paths>()` . A middleware adds `X-Session-Token` (from the cookie), `Accept-Language` , `X-Request-Id` , `X-Host-Account` (website host pages) and turns `problem+json` into the existing `ActionResult` error codes.
3. **`lib/data` keeps its function names**: each body becomes one typed call, for example `getListings(params)` becomes `client.GET("/api/v1/listings", { params: { query } })` . Pages do not change (this was the point of the `lib/data` layer).

4. **Server actions stay** as thin wrappers: session check, call, `revalidatePath` / `revalidateTag`. Business checks move to Django; the action no longer decides anything about money or availability.
5. **Route handlers** stay only where the browser must call the BFF directly: login, logout, register, and a small proxy for message sync polling (`/api/messages/sync`) and upload tickets. They forward to Django and never expose the token.
6. **Mock store**: `lib/data/_store.ts` and the seeds stay behind a flag for demos and tests; a `BLADUS_API_MODE=mock|live` switch picks the implementation per module, so modules can move one at a time.
7. **Contract tests**: schemathesis runs against Django with this file; the apps' unit tests use MSW handlers generated from the examples.

---

8. Every app function mapped to an endpoint

Paths below omit the `/api/v1` prefix. "Removed" means the function has no endpoint, with the reason. "Internal" means it becomes backend code that the apps no longer call.

8.1 Website: `lib/data` infrastructure (`_store.ts`, `_tables.ts`, `_sanitize.ts`)

Function	Endpoint
`globalStore`, `removeWhere`, `clone`, `delay`, table constants	Removed: replaced by the Django ORM and PostgreSQL (kept only in mock mode)
`ListingInputError`	Replaced by 422 Problem with field `errors`
`isAllowedImageUrl`	Removed: images are only `media_id`s from `POST /media/uploads`
`OWNER_SETTABLE_LISTING_STATUSES`	Becomes the `HostStatusChange.status` enum of `POST /host/listings/{listing_id}/status`
`sanitizeListingInput`	Becomes server validation of `ListingWrite`

8.2 Website: accounts (`auth.ts`, `account.ts`, `favorites.ts`, route handlers)

Function or action	Endpoint
`verifyCredentials`, route `POST /api/auth/login`	`POST /auth/login` (`authLogin`); phone OTP: `POST /auth/code/request`, `POST /auth/code/confirm`
`registerUser`, route `POST /api/auth/register`	`POST /auth/signup`, then `POST /auth/phone/verify` or `/auth/email/verify`
route `POST /api/auth/logout`	`DELETE /auth/session`
`getUserById`	Own: `GET /me`; others: `GET /users/{user_id}` (public projection)
`getUsersByIds`	Removed: no caller; people are embedded as `UserRef` in responses
`getUserByEmail`	Removed: internal (invitation matching happens in Django)
`changePassword`, `changePasswordAction`	`POST /auth/password/change`
`updateUserProfile`, `updateProfileAction`	`PATCH /me`; phone change: `PUT /me/phone` + `POST /auth/phone/verify`
`updateUserHostStatus`	Removed: being a host is now holding a host account, created by `POST /host/onboarding/drafts/{host_type}/submit`; approval mode moved to `PATCH /host/account`
`setUserFavorite`, `setFavoriteAction`	`PUT` / `DELETE /me/favorites/{target_type}/{target_id}`
`getFavoriteListings`	`GET /me/favorites`
`DEMO_PASSWORD`	Removed: seed fixtures only
`lib/server` session helpers (`getSession`, `getSessionUser`, `startSession`, `refreshSession`, `endSession`)	Stay in the BFF; they call `GET /auth/session` and the login/logout endpoints
`isTrustedJsonRequest`	Stays in the BFF (CSRF guard of its own routes)
`toPublicUser`, `toPublicOrganization`, `toPublicMembers`, `toCardListing`, `toCardTrip`	Removed: Django serializers return the public shapes (`PublicUser`, `PublicOrganization`, `ListingCard`, `TripCard`)

8.3 Website: catalogue (`listings.ts`, `experiences.ts`, `vehicles.ts`, `reviews.ts`)

Function or action	Endpoint
`getListings`	`GET /listings` (`searchListings`); facets `GET /listings/facets`; map `GET /listings/map`
`isPublicListing`	Internal: the search query only returns published listings of approved hosts
`getPublicListingById`	`GET /listings/{listing_id}`
`getFeaturedListings`	`GET /listings?featured=true&vertical=`
`getExperiences`	`GET /listings?vertical=experience`
`getVehicles`	`GET /listings?vertical=vehicle`
`getListingById`	Owner: `GET /host/listings/{listing_id}`; other uses were internal checks, now in Django
`getHostListings`	`GET /host/listings`; public organisation and host pages: `GET /listings?host_account=`
`createListing`, `createMyListingAction`	`POST /host/listings` (offer rule, starts as draft)
`updateListing`, `updateMyListingAction`	`PATCH /host/listings/{listing_id}`; status part: `POST /host/listings/{listing_id}/status`; photos: `PUT .../photos`; room types: `.../room-types`
`deleteListing`, `deleteMyListingAction`	`DELETE /host/listings/{listing_id}` (soft delete now)

Function or action	Endpoint
`mockReviews`	Removed: replaced by the `review` table
`getReviewsByListingId`	`GET /listings/{listing_id}/reviews` (and `GET /trips/{trip_id}/reviews`)
`getReviewsByUserId`	`GET /users/{user_id}/reviews?role=author

8.4 Website: bookings and availability (`reservations.ts`, `bookings.ts`)

Function or action	Endpoint
`getReservations` (guest, `userId`)	`GET /me/bookings`
`getReservations` (host, `hostId` / `organizationId`)	`GET /host/bookings`; calendar page: `GET /host/calendar`
`getReservationById`	`GET /me/bookings/{booking_id}` or `GET /host/bookings/{booking_id}`
`getReservationOwner`	Removed: permission check inside Django
`ReservationConflictError`	409 Problem (`dates_unavailable`, `not_enough_units`, `seats_unavailable`)
`getListingAvailability`	`GET /listings/{listing_id}/availability`
`unavailableNights`, `isListingFreeForRange`	Internal availability service (feeds search and availability)
`seatsSoldByDeparture`	`GET /trips/{trip_id}/departures` (`seats_left`)
`createListingReservation`, `createTripReservation`, `createBookingAction`	`POST /quotes`, then `POST /me/bookings` with `Idempotency-Key`
`cancelReservation` (guest), `cancelMyReservationAction`	`GET /me/bookings/{booking_id}/cancellation-preview`, `POST /me/bookings/{booking_id}/cancel`
`cancelReservation` (host), `respondToReservationAction` decision `cancel`	`POST /host/bookings/{booking_id}/cancel`
`respondToRequest`, `respondToReservationAction` decision `confirm` / `decline`	`POST /host/bookings/{booking_id}/accept`, `POST /host/bookings/{booking_id}/decline`
Internal helpers (`guestForHost`, `tripFor`, `settle`, `populate`, `ownedInventory`, `activeListingReservations`, `reservationRoomTypeId`, `usageByNight`, `capacityOf`, `buildReservation`, `commissionOwner`, `commissionBase`, `chargeCommission`)	Internal: Django booking, availability and ledger services. `settle` (lazy expiry and completion) becomes scheduled jobs (BR-BKG-04, BR-BKG-05)

8.5 Website: calendar (`calendar.ts`, `calendar` actions)

Function or action	Endpoint
`getAvailabilityOverrides`	`GET /host/listings/{listing_id}/calendar`
`getOverridesForListings`	`GET /host/calendar?listing_ids=&from=&to=`
`blockedDateKeys`, `priceOverridesFor`	Internal (availability and pricing services)
`setDatesBlocked`, `setListingDatesBlockedAction`	`PUT /host/listings/{listing_id}/calendar/blocks`
`setDatePrice`, `setListingDatePriceAction`	`PUT /host/listings/{listing_id}/calendar/prices` (now also per room type)

8.6 Website: messages and notifications

Function or action	Endpoint
`getConversations`	`GET /conversations?as=guest
`getConversationForParticipant`	`GET /conversations/{conversation_id}`
`isConversationParticipant`	Removed: permission check in Django
`getMessages`, `getConversationMessagesAction`	`GET /conversations/{conversation_id}/messages?after=` (and `before=` for history); live updates: `GET /conversations/sync?after=`
`sendMessage`, `sendMessageAction`	`POST /conversations/{conversation_id}/messages`
`markConversationAsRead`, `markConversationReadAction`	`POST /conversations/{conversation_id}/read`
`findOrCreateConversation`, `startConversationAction`	`POST /conversations`
`getNotifications`, `getMyNotificationsAction`	`GET /me/notifications`
`markNotificationAsRead`, `markNotificationReadAction`	`POST /me/notifications/{notification_id}/read`
`markAllNotificationsAsRead`, `markAllNotificationsReadAction`	`POST /me/notifications/read-all`

8.7 Website: organisations, team and workspaces (`organizations.ts`)

Function or action	Endpoint
`ORGANIZATION_LEGAL_FIELDS`, `ORGANIZATION_EDITABLE_FIELDS`	Become the `OrganizationUpdate` schema; legal fields trigger re-verification (BR-ORG-06)
`getOrganizationById`	Public: `GET /organizations/{org_slug}`; private: `GET /host/organization`
`getOrganizations`	`GET /organizations`
`getOrganizationMembers`	`GET /host/organization/members`; public team inside `PublicOrganization.team`

Function or action	Endpoint
`getMembershipsForUser`, `resolveActiveOrganization`, `getHostContext`, `requireHostContext`, `hostOwnerKey`	`GET /me/workspaces` and `GET /host/account`; the BFF keeps a thin `getHostContext` built on them
`switchWorkspaceAction`	`PUT /me/workspaces/active`
`getMembership`, `getMemberById`, `getOrgContext`, `hostOwnsListing`, `hostOwnsTrip`, `getHostContextForReservation`	Removed: permission checks in Django (section 4.3)
`createOrganization`	`POST /host/onboarding/drafts/{host_type}/submit` (creates organisation, owner, invitations)
`updateOrganization`, `updateOrganizationAction`	`PATCH /host/organization` (with `If-Match`)
`deleteOrganization`, `deleteOrganizationAction`	`DELETE /host/organization?confirm_name=` (soft)
`addOrganizationMember`, `inviteMemberAction`	`POST /host/organization/invitations`; the invitee accepts with `POST /me/invitations/accept`. Direct add is removed: a member always accepts first (ORG-03)
(new) list and revoke invitations	`GET /host/organization/invitations`, `DELETE /host/organization/invitations/{invitation_id}`
`updateOrganizationMemberRole`, `changeMemberRoleAction`	`PATCH /host/organization/members/{membership_id}`
`transferOrganizationOwnership`, `transferOwnershipAction`	`POST /host/organization/transfer-ownership`
`removeOrganizationMember`, `removeMemberAction`	`DELETE /host/organization/members/{membership_id}` (soft)
`updateOrganizationVerification`	Staff only: `POST /admin/hosts/{host_account_id}/approve` (and reject, request-changes) or `POST /admin/moderation/cases/{case_id}/decisions`

8.8 Website: becoming a host (`host.ts`)

Function or action	Endpoint
`completeHostOnboardingAction`	`GET /onboarding-flows/{host_type}`, `PUT /host/onboarding/drafts/{host_type}` (save as you go), `POST /host/onboarding/drafts/{host_type}/submit`; then documents (`POST /host/documents`) and `POST /host/verification/submit`
`setBookingApprovalModeAction`	`PATCH /host/account` (`default_approval_mode`)
`localStorage` drafts in `BecomeHostFlow`	Replaced by server drafts (`GET /host/onboarding/drafts`)

8.9 Website: trips (`trips.ts`)

Function or action	Endpoint
`getTrips` (public)	`GET /trips`
`getTrips` (agency, `status: all`)	`GET /host/trips`
`getTripById`	Owner: `GET /host/trips/{trip_id}`; other uses were internal checks
`getPublicTripById`	`GET /trips/{trip_id}` (UUID or reference; old slugs redirect)
`createTrip`, `createTripAction`	`POST /host/trips` (+ `POST /host/trips/{trip_id}/departures` for each date, replacing the array of dates)
`updateTrip`	`PATCH /host/trips/{trip_id}`
`setTripStatusAction`	`POST /host/trips/{trip_id}/status`
`deleteTrip`, `deleteTripAction`	`DELETE /host/trips/{trip_id}` (soft)

8.10 Website: wallet (`wallet.ts`)

Function or action	Endpoint
`getWalletByOwner`, `getMyWalletAction`	`GET /host/wallet`; history `GET /host/wallet/entries`
`getRateForHostType`, `DEFAULT_COMMISSION_RATES`	Internal; the effective rate is in `GET /host/wallet` (`commission_rate_bp`)
`getCommissionRates`, `saveCommissionRates`	Staff only: `GET / PUT /admin/commission-rates`
`getWallets`, `getWalletById`	Staff only: `GET /admin/wallets`, `GET /admin/wallets/{host_account_id}`
`rechargeWallet`	Staff only: `POST /admin/wallets/{host_account_id}/topups`
`deductBookingCommission`, `reverseBookingCommission`, `getBookingCommission`, `computeBalance`, `findOrCreateWallet`, `appendTransaction`, `netCommissionFor`	Internal ledger service, called inside the booking transaction; results visible in `HostBooking.commission` and wallet entries

8.11 Admin: dashboard, moderation, audit, search

Function (data and action)	Endpoint
`getDashboardKPIs`	`GET /admin/analytics/kpis`
`getDashboardCharts`	`GET /admin/analytics/charts`
`getAttentionItems`	`GET /admin/dashboard/attention`
`exportKpiReport`	`POST /admin/exports {dataset: "kpi"}`

Function (data and action)	Endpoint
`getApprovalsInbox`	`GET /admin/moderation/cases?kind=`, counts `GET /admin/moderation/counts`; decisions `POST /admin/moderation/cases/{case_id}/decisions`
`logAudit`, `logExport`	Removed as calls: Django writes audit rows in the same transaction as each change
`listAuditLogs`	`GET /admin/audit-events`
`exportAuditLogs`	`POST /admin/exports {dataset: "audit"}`
`searchGlobalCommandPalette`	`GET /admin/search?q=`

8.12 Admin: sign-in and staff (`auth.ts` , `credentials.ts` , `team.ts`)

Function	Endpoint
`loginAction`	`POST /admin/auth/login`, then `POST /admin/auth/2fa/authenticate`
`logoutAction`	`DELETE /admin/auth/session`
`getCurrentAdmin`, `requireAdmin`, `requirePermission`	`GET /admin/auth/session` (user + permissions); checks done by Django on every call
`verifyAdminCredentials`, `getAdminPasswordHash`	Removed: Django authentication
`getDemoCredentials`, `listDevDemoAccounts`, `getDevSeedPassword`	Removed: development only, replaced by seed fixtures
`listTeamMembers`	`GET /admin/staff` (roles matrix: `GET /admin/staff/roles`)
`getTeamMemberById`, `findTeamMemberByEmail`, `recordTeamMemberLogin`	Removed: internal to Django authentication
`updateMemberRole`	`PATCH /admin/staff/{staff_id}`
`toggleMemberActive`	`POST /admin/staff/{staff_id}/deactivate` or `/activate`
`inviteMember`	`POST /admin/staff/invitations`; acceptance `POST /admin/auth/invitations/accept` then `POST /admin/auth/authenticators/totp`
(missing today) staff password change, MFA reset	`POST /admin/auth/password/change`, `POST /admin/staff/{staff_id}/reset-mfa`

8.13 Admin: hosts (`hosts.ts`)

Function	Endpoint
`listHosts`	`GET /admin/hosts`
`getHostById`	`GET /admin/hosts/{host_account_id}`
`exportHosts`	`POST /admin/exports {dataset: "hosts"}`
`getProviderCategory`	Removed: `host_type` is a field of every host
`transitionHost`	Internal state machine service
`approveHost`, `rejectHost`, `suspendHost`, `unsuspendHost`, `banHost`, `restoreHost`	`POST /admin/hosts/{host_account_id}/approve`, `/reject`, `/suspend`, `/unsuspend`, `/ban`, `/restore` (+ new `/request-changes`)
`deleteHost`	`DELETE /admin/hosts/{host_account_id}`
`overrideCommission`	`PUT / DELETE /admin/hosts/{host_account_id}/commission-override`
(missing today) document review, credit limit	`GET /admin/documents/{document_id}/url`, `POST .../approve`, `POST .../reject`; `PUT /admin/hosts/{host_account_id}/credit-limit-override`

8.14 Admin: listings, trips and network (`listings.ts` , `network.ts`)

Function	Endpoint
`listListings`	`GET /admin/listings`; trips (now separate tables): `GET /admin/trips`
`getListingById`	`GET /admin/listings/{listing_id}`; trips `GET /admin/trips/{trip_id}`
`exportListings`	`POST /admin/exports {dataset: "listings"}` (or `"trips"`)
`matchListingStatus`	Removed: one status set, no aliases
`transitionListing`	Internal state machine service
`approveListing`, `rejectListing`, `requestChangesListing`, `suspendListing`, `restoreListing`	`POST /admin/listings/{listing_id}/approve`, `/reject`, `/request-changes`, `/suspend`, `/restore`
`deleteListing`	`DELETE /admin/listings/{listing_id}`
`featureListing`	`PUT /admin/listings/{listing_id}/featured`
`updateListingStatus`	`POST /admin/listings/{listing_id}/status`; trips `POST /admin/trips/{trip_id}/status`
`searchNetwork`	`GET /admin/listings` (same filters, every status)

8.15 Admin: bookings, users, reviews

Function	Endpoint
`listBookings`	`GET /admin/bookings`

Function	Endpoint
`getBookingById`	`GET /admin/bookings/{booking_id}`
`exportBookings`	`POST /admin/exports {dataset: "bookings"}`
`cancelBooking`	`POST /admin/bookings/{booking_id}/cancel`
`refundBooking` (and private `applyRefund`)	`POST /admin/bookings/{booking_id}/refunds` with `Idempotency-Key`
`resolveDispute`	`POST /admin/bookings/{booking_id}/dispute/resolve` (R2)
`listUsers`	`GET /admin/users`
`getUserById`	`GET /admin/users/{user_id}`
`suspendUser`, `unsuspendUser`, `banUser`, `restoreUser`	`POST /admin/users/{user_id}/suspend`, `/unsuspend`, `/ban`, `/restore`
`deleteUser`	`DELETE /admin/users/{user_id}` (anonymisation follows, BR-PRV-02)
`updateUserStatus`	Removed: replaced by the explicit transitions above (the schema has no "pending" user status)
`listReviews`	`GET /admin/reviews`
`hideReview`, `restoreReview`, `dismissFlag`	`POST /admin/reviews/{review_id}/hide`, `/restore`, `/dismiss-reports`
`deleteReview`	`DELETE /admin/reviews/{review_id}` (status `removed`)
(new) data protection requests	`GET /admin/data-requests`, `POST /admin/data-requests/{data_request_id}/complete`

8.16 Admin: finance and ledger (`finance.ts`, `ledger.ts`)

Function	Endpoint
`getFinanceSummary`	`GET /admin/finance/summary`
`listWallets`	`GET /admin/wallets`
`exportWallets`	`POST /admin/exports {dataset: "wallets"}`
`listTransactions`	`GET /admin/ledger/transactions`
`exportTransactions`	`POST /admin/exports {dataset: "ledger"}`
`rechargeWallet`	`POST /admin/wallets/{host_account_id}/topups`; four-eyes: `GET /admin/topups?status=pending_approval`, `POST /admin/topups/{topup_id}/approve`, `/reject`
`getCommissionRates`, `updateCommissionRates`	`GET` / `PUT /admin/commission-rates`
`listPayouts`, `approvePayout`, `rejectPayout`	Dormant: `GET /admin/payouts`, `POST /admin/payouts/{payout_id}/approve`, `/reject` (+ `/mark-sent`)
`listRefunds`	`GET /admin/refunds`
`computeWalletAggregates`, `getHostBalance`, `appendLedgerEntry`	Internal ledger service (balances from `account_balance`)
(missing today) ledger correction	`POST /admin/ledger/adjustments`

8.17 Admin: configuration (`settings.ts`, `onboarding.ts`, `categories.ts`, `promotions.ts`)

Function	Endpoint
`getPlatformSettings`	`GET /admin/settings` (history `GET /admin/settings/versions`)
`updatePlatformSettings`	`PATCH /admin/settings` with `If-Match`
`getOnboardingConfig`	`GET /admin/onboarding-flows`, `GET /admin/onboarding-flows/{host_type}/versions[/{version}]`
`updateOnboardingFlow`	`PUT /admin/onboarding-flows/{host_type}/draft`, then `POST /admin/onboarding-flows/{host_type}/publish`; rollback `POST .../versions/{version}/republish`
`listCategories`, `createCategory`, `toggleCategoryStatus`, `deleteCategory`	`GET` / `POST /admin/categories`, `PATCH /admin/categories/{category_id}` (`is_active`), `DELETE` (only when unused)
(missing today) amenities and property types	`/admin/amenities`, `/admin/property-types` (same four operations)
`listPromotions`, `createPromotion`, `togglePromotionStatus`, `deletePromotion`	`GET` / `POST /admin/coupons`, `PATCH /admin/coupons/{coupon_id}`, `DELETE` (R2)
`unwrap` (`lib/actions/result.ts`), `DomainError`, `ValidationError` (`lib/data/errors.ts`)	Stay in the admin app; they now wrap Problem responses

8.18 New endpoints with no caller today

These serve requirements the apps do not implement yet: second-factor enrolment, email and phone verification, password reset, consents, data requests, notification preferences (R2), media uploads, host documents and verification submission, change requests, payout accounts, room-type and slot management, departures, review writing, replies and reports, disputes (R2), host earnings, invoices (R2), online top-up (R2), reference lists and reason codes, moderation cases and bulk decisions (R2), staff MFA reset. Each is in `openapi.yaml` with its requirement IDs.

8.19 Gaps found while mapping

- The website profile edits `bio`, `address` and `languages` (`updateProfileAction`), but `schema.dbml` has no such columns on `user`. `PATCH /me` leaves them out. Decide: add columns or drop the fields (proposed question for 14-open-questions.md).
- Trips are addressed by slug in the website today; the schema has only a reference. Old slugs need a redirect table or are dropped.
- `addOrganizationMember` added members directly; the new flow always goes through an invitation the person accepts.

---

9. Versioning and deprecation

- The major version is in the path (`/api/v1`). The `info.version` of the file follows semantic versioning (`1.0.0-draft.1` today).
- **Allowed without a new major version**: new endpoints, new optional request fields, new response fields, new enum values in responses that the apps already treat as "unknown" (the apps must not crash on an unknown value).
- **Breaking** (needs `/api/v2` for the affected endpoints): removing or renaming a field or endpoint, changing a type or unit, making an optional input required, changing an error `code` , changing pagination style.
- A deprecated endpoint answers with `Deprecation: true` and `Sunset: <date>` headers (RFC 9745 and RFC 8594) and is marked `deprecated: true` in the file. Both apps are ours, so the notice is at least one release (minimum 30 days) before removal; any future partner API needs 6 months.
- CI runs `oasdiff breaking` between the main-branch file and the pull request; a breaking change fails the build unless the version changes.

---

10. Worked examples

All examples use the Tipaza villa of example 8a in 04-business-rules.md: 3 nights x 18,500 DZD, cleaning 4,000 DZD, individual host at 12%. Money is in centimes. The same examples are embedded in `openapi.yaml` .

10.1 Search

```
GET /api/v1/listings?vertical=home&wilaya=42&check_in=2026-12-18&check_out=2026-12-21&guests=5&amenities=wifi,pool&ordering=price&limit=20
Accept-Language: fr
X-Session-Token: (from the BFF cookie, optional here)
```

```
{
  "results": [{
    "id": "01933b80-1111-7aaa-8bbb-000000000001",
    "reference": "LST-7K2M9Q",
    "vertical": "home",
    "experience_type": null,
    "title": "Villa with sea view in Tipaza",
    "property_type": "villa",
    "commune": {"id": 4201, "name": "Tipaza", "wilaya_code": 42, "wilaya_name": "Tipaza"},
    "location_public": {"lat": 36.5921, "lng": 2.4433},
    "price_from": {"amount_minor": 1850000, "currency": "DZD"},
    "price_unit": "night",
    "rating_avg": 4.8, "review_count": 23,
    "instant_bookable": false, "host_type": "individual",
    "matched_availability": {"full_match": true, "from": "2026-12-18", "to": "2026-12-21"},
    "quoted_total": {"amount_minor": 5950000, "currency": "DZD"},
    "is_favorite": false
  }],
  "next_cursor": "eyJyIjowLjkxLCJpZCI6IjAxOTMzYjgwIn0",
  "has_more": true,
  "total_estimate": 37
}
```

The filter panel calls `GET /api/v1/listings/facets` with the same query to show counts. `location_public` is the fuzzed point (BR-SRC-03).

10.2 Quote, then booking

```
POST /api/v1/quotes
Content-Type: application/json

{"listing_id": "01933b80-1111-7aaa-8bbb-000000000001", "check_in": "2026-12-18", "check_out": "2026-12-21",
 "guests_adults": 4, "guests_children": 1, "units": 1}
```

Response `201` (shortened: three `night` lines of 1,850,000 and one `cleaning_fee` line of 400,000):

```
{
  "id": "01933c0f-aaaa-7bbb-8ccc-00000000a001",
  "expires_at": "2026-11-09T10:00:00Z",
  "nights": 3,
  "subtotal": {"amount_minor": 5550000, "currency": "DZD"},
  "discount": {"amount_minor": 0, "currency": "DZD"},
  "cleaning_fee": {"amount_minor": 400000, "currency": "DZD"},
  "taxes": {"amount_minor": 0, "currency": "DZD"},
  "total": {"amount_minor": 5950000, "currency": "DZD"},
  "platform_fee": {"amount_minor": 0, "currency": "DZD"},
  "approval_mode": "request",
  "cancellation_policy": {"code": "moderate", "version": 1, "label": "Modérée", "windows": [
    {"min_days_before": 5, "refund_bp": 10000}, {"min_days_before": 1, "refund_bp": 5000}, {"min_days_before": 0, "refund_bp": 0}]}
}
```

The guest confirms (phone already verified, DEC-012):

```
POST /api/v1/me/bookings
Idempotency-Key: 6f1d2c0e-5a7b-4c1e-9d3a-2b8f0e7c4a11
Content-Type: application/json

{"quote_id": "01933c0f-aaaa-7bbb-8ccc-00000000a001", "guest_message": "We arrive around 16:00 with a baby.", "terms_version": "2026-10"}
```

Response `201` : a `GuestBooking` with `reference: "BLD-2026-7K2M9Q"` , `status: "pending"` (request mode), `request_expires_at` 24 hours later (DEC-008), `total` 5,950,000, `address_line: null` (hidden until confirmed), `payment_status: "not_collected"` (model M1). If the guest clicks twice, the second call returns the same body with `Idempotency-Replayed: true` . If someone else took the nights first: `409 dates_unavailable` .

10.3 Host accepts the request

```
POST /api/v1/host/bookings/BLD-2026-7K2M9Q/accept
X-Host-Account: 01933b7e-8a10-7c3e-9f00-1a2b3c4d5e01
Idempotency-Key: 0b9e4f7a-3c2d-4e11-8a6b-7d5c9e1f2a33
Content-Type: application/json

{"message_to_guest": "Welcome! The key box code will be sent the day before."}
```

Response `200` (extract):

```
{
  "reference": "BLD-2026-7K2M9Q",
  "status": "confirmed",
  "confirmed_at": "2026-11-09T11:02:00Z",
  "guest": {"id": "01933a00-0000-7000-8000-0000000a0001", "display_name": "Yacine M.", "avatar_url": null},
  "guest_phone_e164": "+213555123456",
  "commission": {"rate_bp": 1200,
                 "base": {"amount_minor": 5550000, "currency": "DZD"},
                 "amount": {"amount_minor": 666000, "currency": "DZD"},
                 "reversed": {"amount_minor": 0, "currency": "DZD"}},
  "allowed_actions": ["cancel", "message"]
}
```

In the same transaction Django posts the ledger transaction "debit host_wallet 6,660 DZD, credit commission revenue 6,660 DZD" (example 9a), writes the outbox events (notification, SMS to the guest) and the message. An agent or guide would get `403 permission_denied` with `required_permission: "bookings.respond"`. A host whose wallet is below the credit floor gets `409 wallet_below_credit_floor`.

10.4 Wallet top-up with four-eyes approval

Finance user Samira records a CCP transfer of 600,000 DZD for Atlas Escapes (above the 500,000 DZD threshold):

```
POST /api/v1/admin/wallets/01933b7e-8a10-7c3e-9f00-1a2b3c4d5e02/topups
Idempotency-Key: 9a7c1e22-4b6f-4d0a-8e3c-1f2d3b4c5d66

{"amount": {"amount_minor": 60000000, "currency": "DZD"}, "method": "ccp_transfer",
 "receipt_reference": "CCP-0012345-2026-11-08", "note": "Transfer seen on CCP statement 08/11"}
```

Response `201`: `status: "pending_approval"`, `needs_second_approval: true`, nothing posted to the ledger yet. Using the same receipt again gives `409 receipt_already_used`.

A second finance user, Nadir, approves:

```
POST /api/v1/admin/topups/01933d00-0000-7000-8000-0000000e0001/approve
Idempotency-Key: 2c4e6a80-1b3d-4f5a-9c7e-0d2f4a6b8c99
```

```
{
  "id": "01933d00-0000-7000-8000-0000000e0001",
  "status": "approved",
  "amount": {"amount_minor": 60000000, "currency": "DZD"},
  "requested_by": {"id": "01933a00-0000-7000-8000-0000000d0001", "display_name": "Samira (finance)"},
  "approved_by": {"id": "01933a00-0000-7000-8000-0000000d0002", "display_name": "Nadir (finance)"},
  "ledger_transaction_id": "01933d00-0000-7000-8000-0000000f0001",
  "needs_second_approval": true,
  "wallet_balance_after": {"amount_minor": 61240000, "currency": "DZD"}
}
```

If Samira tried to approve her own top-up: `403 four_eyes_same_user`. Both actions are in the audit log with ids, not names (BR-STF-03).

10.5 Message sync (polling)

The guest's messages page first calls `GET /api/v1/conversations/sync` (no cursor) and gets the current cursor. Then, every `poll_after_ms`:

```
GET /api/v1/conversations/sync?after=m90398&as=guest
```

```
{
  "conversations": [{"id": "01933c10-0000-7000-8000-00000000c001", "kind": "booking", "my_role": "guest",
                     "unread_count": 1, "last_read_message_id": 90398, "muted": false,
                     "subject": {"booking_reference": "BLD-2026-7K2M9Q", "title": "Villa with sea view in Tipaza"},
                     "other_party": {"id": "01933a00-0000-7000-8000-0000000a0009", "display_name": "Karim B.", "avatar_url": null}}],
  "messages": [{"id": 90412, "conversation_id": "01933c10-0000-7000-8000-00000000c001", "sender_role": "host_member",
                "kind": "text", "body": "Welcome! The key box code will be sent the day before.",
                "is_masked": false, "created_at": "2026-11-09T11:02:05Z"}],
  "unread_total": 1,
  "next_cursor": "m90412",
  "has_more": false,
  "poll_after_ms": 10000
}
```

When nothing changed, `conversations` and `messages` are empty and the cursor stays the same. When the guest opens the thread, the app calls `POST /conversations/{id}/read {"last_read_message_id": 90412}`. Before a booking is confirmed, a phone number in a message arrives as `"0555 ** ** **"` with `is_masked: true` (BR-MSG-01). Later, Centrifugo can push the same events without changing this endpoint.

Check yourself

- [ ] Why does the browser never hold the Django session token, and where is it kept instead?
- [ ] A user is a member of two organisations. How does Django know which one a host request is for, and what happens if the id is not theirs?
- [ ] Write the `Money` object for 12,500 DZD.
- [ ] Two guests book the last room at the same moment. Which part of the system decides who wins, and what does the loser receive?
- [ ] What happens when the same `Idempotency-Key` is sent twice with the same body? And with a different body?
- [ ] Which website function has no endpoint any more because being a host is now a host account, and which endpoint replaces it?
- [ ] Why is the gateway webhook marked `x-dormant`, and what does it answer today?
- [ ] Which change to the API would force a `/api/v2` : adding a response field, or renaming one?

API reference (generated from openapi.yaml)

> **In plain words**
> Every API address the backend will offer, grouped by area, with the requirements each one serves.
> It is generated from openapi.yaml, the machine-readable contract. 266 operations.

Title: BLADUS API, version 1.0.0-draft.1.

auth

Method	Path	What it does	Requirements
POST	`/api/v1/auth/signup`	Sign up with email or Algerian phone	ACC-01, PRV-01
POST	`/api/v1/auth/login`	Sign in with email or phone and password	ACC-02, ACC-03, ACC-07, NFR-11
POST	`/api/v1/auth/code/request`	Send a one-time login code by SMS (phone OTP)	ACC-02, ACC-07
POST	`/api/v1/auth/code/confirm`	Confirm the one-time login code	ACC-02
GET	`/api/v1/auth/session`	Read the current session	ACC-03
DELETE	`/api/v1/auth/session`	Sign out (revoke the session token)	ACC-03
POST	`/api/v1/auth/phone/verify`	Verify a phone number with the SMS code	ACC-01, ACC-06
POST	`/api/v1/auth/email/verify`	Verify an email address with the emailed key	ACC-01, ACC-06
POST	`/api/v1/auth/password/request`	Ask for a password reset (email link or SMS code)	ACC-04
POST	`/api/v1/auth/password/reset`	Set a new password with the reset key	ACC-04
POST	`/api/v1/auth/password/change`	Change password (signed in)	ACC-04
POST	`/api/v1/auth/2fa/authenticate`	Second step of login: TOTP or recovery code	ACC-05
GET	`/api/v1/auth/authenticators`	List my second factors	ACC-05
GET	`/api/v1/auth/authenticators/totp`	Get a TOTP secret to enrol	ACC-05
POST	`/api/v1/auth/authenticators/totp`	Enrol a TOTP authenticator	ACC-05

public.geo

Method	Path	What it does	Requirements
GET	`/api/v1/geo/wilayas`	List the 69 wilayas	SRC-04
GET	`/api/v1/geo/wilayas/{wilaya_code}/communes`	List the communes of one wilaya	SRC-04
GET	`/api/v1/geo/communes`	Autocomplete communes by name (fr, ar, en)	SRC-03, SRC-04

public.reference

Method	Path	What it does	Requirements
GET	`/api/v1/reference`	All shared lists (types, amenities, policies, offer rules)	LST-05, LST-12, TRP-07, CFG-05, HST-07
GET	`/api/v1/reference/reasons`	Reason catalogue for an action	MOD-03, BKG-02, CAN-02
GET	`/api/v1/onboarding-flows/{host_type}`	Get the published onboarding flow for a host type	CFG-04, HST-01

public.catalog

Method	Path	What it does	Requirements
GET	`/api/v1/listings`	Search listings	SRC-01, SRC-02, SRC-03, SRC-05, SRC-06, AVL-04, LST-12, NFR-02
GET	`/api/v1/listings/facets`	Facet counts for the current search	SRC-01, LST-12
GET	`/api/v1/listings/map`	Map pins for the current search (max 500)	SRC-02
GET	`/api/v1/listings/{listing_id}`	Public listing page	LST-06, LST-10, LST-11, SRC-06
GET	`/api/v1/listings/{listing_id}/availability`	Free nights, units or slots for a period	AVL-04, AVL-02, LST-11
GET	`/api/v1/listings/{listing_id}/reviews`	Published reviews of a listing	REV-05, REV-03
POST	`/api/v1/quotes`	Get a server price quote	PRC-01, PRC-02, PRC-03, PRC-04, PRC-05, PAY-12, AVL-04, BKG-09

Method	Path	What it does	Requirements
GET	`/api/v1/quotes/{quote_id}`	Read a quote again	PRC-01

public.trips

Method	Path	What it does	Requirements
GET	`/api/v1/trips`	Search agency trips	TRP-05, TRP-07, SRC-05
GET	`/api/v1/trips/{trip_id}`	Public trip page (path accepts the trip id or its slug)	TRP-02, TRP-03
GET	`/api/v1/trips/{trip_id}/departures`	Upcoming departures with seats left	TRP-03, BKG-09
GET	`/api/v1/trips/{trip_id}/reviews`	Published reviews of a trip	REV-05

public.profiles

Method	Path	What it does	Requirements
GET	`/api/v1/organizations`	List verified organisations (sitemap, directory)	ORG-07
GET	`/api/v1/organizations/{org_slug}`	Public organisation profile	ORG-07
GET	`/api/v1/users/{user_id}`	Public user profile	ACC-06
GET	`/api/v1/users/{user_id}/reviews`	Reviews written by a user or received as a host	REV-05

me.profile

Method	Path	What it does	Requirements
GET	`/api/v1/me`	My profile	ACC-06, ACC-08
PATCH	`/api/v1/me`	Update my profile	ACC-06
PUT	`/api/v1/me/phone`	Start a phone change (sends an SMS code)	ACC-06, ACC-01
GET	`/api/v1/me/consents`	My current consents	PRV-01
POST	`/api/v1/me/consents`	Accept or withdraw a consent (terms, privacy, marketing)	PRV-01, NTF-03
GET	`/api/v1/me/data-requests`	My data protection requests	PRV-02
POST	`/api/v1/me/data-requests`	Ask for my data, a correction or deletion	PRV-02

me.favorites

Method	Path	What it does	Requirements
GET	`/api/v1/me/favorites`	My favourites	ACC-09
PUT	`/api/v1/me/favorites/{target_type}/{target_id}`	Add a listing or trip to favourites	ACC-09
DELETE	`/api/v1/me/favorites/{target_type}/{target_id}`	Remove from favourites	ACC-09

me.notifications

Method	Path	What it does	Requirements
GET	`/api/v1/me/notifications`	My notifications, newest first	NTF-01
POST	`/api/v1/me/notifications/{notification_id}/read`	Mark one notification as read	NTF-01
POST	`/api/v1/me/notifications/read-all`	Mark all my notifications as read	NTF-01
GET	`/api/v1/me/notification-preferences`	My notification preferences	NTF-03
PUT	`/api/v1/me/notification-preferences`	Change my notification preferences	NTF-03

me.bookings

Method	Path	What it does	Requirements
POST	`/api/v1/me/bookings`	Book from a quote (instant or request)	BKG-01, BKG-02, BKG-05, BKG-06, BKG-09, BKG-10, AVL-01, AVL-02, PAY-03, ACC-01
GET	`/api/v1/me/bookings`	My bookings (trips page)	BKG-07
GET	`/api/v1/me/bookings/{booking_id}`	One of my bookings	BKG-07, BKG-08, NFR-12
GET	`/api/v1/me/bookings/{booking_id}/cancellation-preview`	What I would get back if I cancel now	CAN-01
POST	`/api/v1/me/bookings/{booking_id}/cancel`	Cancel my booking or withdraw my request	CAN-01, CAN-04, BKG-05
POST	`/api/v1/me/bookings/{booking_id}/review`	Review a completed stay	REV-01
POST	`/api/v1/me/bookings/{booking_id}/dispute`	Open a dispute about a booking	BKG-11
POST	`/api/v1/me/bookings/{booking_id}/payments`	Pay a booking online (guest payment)	PAY-10 (dormant)
GET	`/api/v1/me/reviews`	Reviews I wrote	REV-01
POST	`/api/v1/reviews/{review_id}/reports`	Report a review	REV-04
POST	`/api/v1/me/invitations/accept`	Accept an organisation invitation	ORG-03, ACC-08

messaging

Method	Path	What it does	Requirements
GET	`/api/v1/conversations`	My conversations	MSG-01, MSG-03
POST	`/api/v1/conversations`	Start (or find) a conversation about a listing, trip or booking	MSG-01
GET	`/api/v1/conversations/sync`	Changes since a cursor (polling)	MSG-02, MSG-03
GET	`/api/v1/conversations/{conversation_id}`	One conversation	MSG-01
GET	`/api/v1/conversations/{conversation_id}/messages`	Messages of a conversation	MSG-01, MSG-02, MSG-04
POST	`/api/v1/conversations/{conversation_id}/messages`	Send a message	MSG-01, MSG-04
POST	`/api/v1/conversations/{conversation_id}/read`	Move my read pointer	MSG-03

media

Method	Path	What it does	Requirements
POST	`/api/v1/media/uploads`	Get a presigned upload URL	MED-01, MED-02, MSG-05
POST	`/api/v1/media/{media_id}/complete`	Tell the server the upload finished	MED-01, MED-02
GET	`/api/v1/media/{media_id}`	Upload status and renditions	MED-01
GET	`/api/v1/media/{media_id}/url`	Short-lived URL for a private file	MED-02

host.workspace

Method	Path	What it does	Requirements
GET	`/api/v1/me/workspaces`	Host accounts I can act for (workspace switcher)	ORG-02, ACC-08
PUT	`/api/v1/me/workspaces/active`	Switch the active host account	ORG-02
GET	`/api/v1/host/account`	The active host account	ORG-02, HST-04, HST-06, HST-07
PATCH	`/api/v1/host/account`	Change the default approval mode	HST-06

host.onboarding

Method	Path	What it does	Requirements
GET	`/api/v1/host/onboarding/drafts`	My onboarding drafts	HST-02
GET	`/api/v1/host/onboarding/drafts/{host_type}`	My draft for one host type	HST-02
PUT	`/api/v1/host/onboarding/drafts/{host_type}`	Save my draft (create or replace)	HST-02, CFG-04
DELETE	`/api/v1/host/onboarding/drafts/{host_type}`	Abandon my draft	HST-02
POST	`/api/v1/host/onboarding/drafts/{host_type}/submit`	Finish onboarding: create the host account	HST-01, HST-02, ORG-01, HST-07, LST-01, CFG-04

host.verification

Method	Path	What it does	Requirements
GET	`/api/v1/host/documents`	Verification documents of the active host account	HST-03, MOD-08
POST	`/api/v1/host/documents`	Attach an uploaded document	HST-03, MED-02
DELETE	`/api/v1/host/documents/{document_id}`	Remove a pending or rejected document	HST-03
POST	`/api/v1/host/verification/submit`	Send the host account for review (or resubmit)	HST-04, MOD-02
GET	`/api/v1/host/change-requests`	What moderators asked me to fix	MOD-02
GET	`/api/v1/host/payout-accounts`	Refund and payout account details	HST-05
POST	`/api/v1/host/payout-accounts`	Add a CCP, bank or BaridiMob account	HST-05, ACC-05
DELETE	`/api/v1/host/payout-accounts/{payout_account_id}`	Remove an account	HST-05

host.organization

Method	Path	What it does	Requirements
GET	`/api/v1/host/organization`	The active organisation (private view)	ORG-01, ORG-06
PATCH	`/api/v1/host/organization`	Edit the organisation	ORG-06, ORG-01
DELETE	`/api/v1/host/organization`	Delete the organisation (soft)	ORG-04, LST-08
GET	`/api/v1/host/organization/members`	Team members	ORG-04, ORG-05
PATCH	`/api/v1/host/organization/members/{membership_id}`	Change a member's role	ORG-04, ORG-05
DELETE	`/api/v1/host/organization/members/{membership_id}`	Remove a member	ORG-04
POST	`/api/v1/host/organization/transfer-ownership`	Make another member the owner	ORG-04
GET	`/api/v1/host/organization/invitations`	Pending and past invitations	ORG-03
POST	`/api/v1/host/organization/invitations`	Invite someone by email or phone	ORG-03
DELETE	`/api/v1/host/organization/invitations/{invitation_id}`	Revoke an invitation	ORG-03

host.listings

Method	Path	What it does	Requirements
GET	`/api/v1/host/listings`	Listings of the active host account (all statuses)	LST-01, LST-03, BKG-07
POST	`/api/v1/host/listings`	Create a listing (draft)	LST-01, HST-07, LST-04, LST-05, LST-06, LST-07, LST-10, LST-11, LST-12
GET	`/api/v1/host/listings/{listing_id}`	One listing with private fields	LST-01
PATCH	`/api/v1/host/listings/{listing_id}`	Edit a listing	LST-01, LST-02, LST-07, PRC-03, HST-07, LST-12
DELETE	`/api/v1/host/listings/{listing_id}`	Delete a listing (soft)	LST-08
POST	`/api/v1/host/listings/{listing_id}/status`	Submit, withdraw, pause, resume or archive	LST-02, LST-03, LST-08, BKG-02
PUT	`/api/v1/host/listings/{listing_id}/photos`	Order photos and choose the cover	MED-03, MED-01
GET	`/api/v1/host/listings/{listing_id}/room-types`	Room types of a hotel or hostel	LST-04
POST	`/api/v1/host/listings/{listing_id}/room-types`	Add a room type	LST-04
PATCH	`/api/v1/host/listings/{listing_id}/room-types/{room_type_id}`	Edit a room type	LST-04
DELETE	`/api/v1/host/listings/{listing_id}/room-types/{room_type_id}`	Remove a room type (soft)	LST-04
GET	`/api/v1/host/listings/{listing_id}/slots`	Time slots of an experience	LST-11
POST	`/api/v1/host/listings/{listing_id}/slots`	Add a time slot	LST-11
PATCH	`/api/v1/host/listings/{listing_id}/slots/{slot_id}`	Edit or cancel a slot	LST-11

host.calendar

Method	Path	What it does	Requirements
GET	`/api/v1/host/calendar`	Calendar of several listings	AVL-03
GET	`/api/v1/host/listings/{listing_id}/calendar`	Calendar of one listing or room type	AVL-03
PUT	`/api/v1/host/listings/{listing_id}/calendar/blocks`	Block or unblock days	AVL-03

Method	Path	What it does	Requirements
PUT	`/api/v1/host/listings/{listing_id}/calendar/prices`	Set or clear per-day prices	AVL-03, PRC-01

host.trips

Method	Path	What it does	Requirements
GET	`/api/v1/host/trips`	Trips of the active agency (all statuses)	TRP-01
POST	`/api/v1/host/trips`	Create a trip (draft)	TRP-01, TRP-02, TRP-07, HST-07
GET	`/api/v1/host/trips/{trip_id}`	One trip with private fields	TRP-02
PATCH	`/api/v1/host/trips/{trip_id}`	Edit a trip	TRP-02, TRP-04, TRP-07
DELETE	`/api/v1/host/trips/{trip_id}`	Delete a trip (soft)	TRP-01, LST-08
POST	`/api/v1/host/trips/{trip_id}/status`	Submit, withdraw, pause, resume or archive a trip	TRP-04, LST-03
GET	`/api/v1/host/trips/{trip_id}/departures`	Departures of a trip	TRP-03
POST	`/api/v1/host/trips/{trip_id}/departures`	Add a departure	TRP-03
PATCH	`/api/v1/host/trips/{trip_id}/departures/{departure_id}`	Edit a departure	TRP-03
POST	`/api/v1/host/trips/{trip_id}/departures/{departure_id}/cancel`	Cancel a departure and all its bookings	TRP-06

host.bookings

Method	Path	What it does	Requirements
GET	`/api/v1/host/bookings`	Bookings of the active host account	BKG-07
GET	`/api/v1/host/bookings/{booking_id}`	One booking of the active host account	BKG-07, BKG-08
POST	`/api/v1/host/bookings/{booking_id}/accept`	Accept a pending request	BKG-02, PAY-03, PAY-04, PAY-07, ORG-05
POST	`/api/v1/host/bookings/{booking_id}/decline`	Decline a pending request	BKG-02, CAN-04, ORG-05
GET	`/api/v1/host/bookings/{booking_id}/cancellation-preview`	What a host cancellation means	CAN-02
POST	`/api/v1/host/bookings/{booking_id}/cancel`	Cancel a confirmed booking as host	CAN-02, CAN-04
POST	`/api/v1/host/bookings/{booking_id}/guest-review`	Review the guest	REV-02
POST	`/api/v1/host/bookings/{booking_id}/dispute`	Open a dispute as host	BKG-11

host.reviews

Method	Path	What it does	Requirements
GET	`/api/v1/host/reviews`	Reviews received by the active host account	REV-03, REV-05
PUT	`/api/v1/host/reviews/{review_id}/reply`	Write or edit the public reply	REV-03

host.wallet

Method	Path	What it does	Requirements
GET	`/api/v1/host/wallet`	Commission wallet of the active host account	PAY-02, PAY-07, PAY-08, PAY-04
GET	`/api/v1/host/wallet/entries`	Wallet history	PAY-08, PAY-01
POST	`/api/v1/host/wallet/topups`	Top up the wallet online (CIB, Edahabia)	PAY-06
GET	`/api/v1/host/earnings`	Earnings dashboard	RPT-03
GET	`/api/v1/host/invoices`	Commission invoices	PAY-11
GET	`/api/v1/host/payouts`	Payouts to the host	PAY-10 (dormant)
POST	`/api/v1/host/payouts`	Ask for a payout	PAY-10, HST-05 (dormant)

webhooks

Method	Path	What it does	Requirements
POST	`/api/v1/webhooks/gateway/{provider}`	Payment gateway callback	PAY-09, PAY-06, PAY-10 (dormant)

admin.auth

Method	Path	What it does	Requirements
POST	`/api/v1/admin/auth/login`	Staff sign in (step 1: password)	STF-05, ACC-07, ACC-03
POST	`/api/v1/admin/auth/2fa/authenticate`	Staff sign in (step 2: second factor)	ACC-05, STF-05
GET	`/api/v1/admin/auth/session`	Current staff session and permissions	STF-01, ACC-03
DELETE	`/api/v1/admin/auth/session`	Staff sign out	ACC-03
POST	`/api/v1/admin/auth/invitations/accept`	Accept a staff invitation and set a password	STF-02
POST	`/api/v1/admin/auth/authenticators/totp`	Enrol TOTP (required before any admin endpoint)	ACC-05, STF-02
POST	`/api/v1/admin/auth/password/change`	Change my staff password	ACC-04

admin.moderation

Method	Path	What it does	Requirements
GET	`/api/v1/admin/moderation/counts`	Queue sizes, oldest waiting, high priority	MOD-01
GET	`/api/v1/admin/dashboard/attention`	Dashboard: what needs attention now	MOD-01, PAY-07
GET	`/api/v1/admin/moderation/cases`	Moderation queue	MOD-01, MOD-08
GET	`/api/v1/admin/moderation/cases/{case_id}`	One case with its history	MOD-01, MOD-02
POST	`/api/v1/admin/moderation/cases/{case_id}/assign`	Take or assign a case	MOD-01
POST	`/api/v1/admin/moderation/cases/{case_id}/decisions`	Decide a case (approve, reject, request changes…)	MOD-02, MOD-03, MOD-05, MOD-08, LST-02, TRP-04, HST-04, REV-04
POST	`/api/v1/admin/moderation/bulk-decisions`	Same decision on many cases	MOD-04
GET	`/api/v1/admin/reason-codes`	Reason catalogue (all actions)	MOD-03

admin.hosts

Method	Path	What it does	Requirements
GET	`/api/v1/admin/hosts`	Host accounts (individuals and organisations)	HST-04, MOD-01, PAY-07
GET	`/api/v1/admin/hosts/{host_account_id}`	One host account with documents, team and cases	HST-04
DELETE	`/api/v1/admin/hosts/{host_account_id}`	Soft-delete a host account	HST-04
POST	`/api/v1/admin/hosts/{host_account_id}/approve`	Approve a host account	HST-04, MOD-08
POST	`/api/v1/admin/hosts/{host_account_id}/request-changes`	Ask the host for changes	HST-04, MOD-02
POST	`/api/v1/admin/hosts/{host_account_id}/reject`	Reject a host account	HST-04
POST	`/api/v1/admin/hosts/{host_account_id}/suspend`	Suspend a host account (cascade)	HST-04, MOD-06
POST	`/api/v1/admin/hosts/{host_account_id}/unsuspend`	Lift a suspension	HST-04, MOD-06
POST	`/api/v1/admin/hosts/{host_account_id}/ban`	Ban a host account (final)	HST-04, MOD-06
POST	`/api/v1/admin/hosts/{host_account_id}/restore`	Restore a soft-deleted host account	HST-04
PUT	`/api/v1/admin/hosts/{host_account_id}/commission-override`	Set a commission rate for this host	PAY-04
DELETE	`/api/v1/admin/hosts/{host_account_id}/commission-override`	Go back to the host-type default rate	PAY-04
PUT	`/api/v1/admin/hosts/{host_account_id}/credit-limit-override`	Set the credit floor for this host	PAY-07
GET	`/api/v1/admin/documents/{document_id}/url`	View a KYC document (signed URL, audited)	MED-02, MOD-08, STF-03
POST	`/api/v1/admin/documents/{document_id}/approve`	Approve one document	MOD-08
POST	`/api/v1/admin/documents/{document_id}/reject`	Reject one document	MOD-08

admin.catalog

Method	Path	What it does	Requirements
GET	`/api/v1/admin/listings`	All listings, every status, with facets	LST-03, MOD-01, LST-12
GET	`/api/v1/admin/listings/{listing_id}`	One listing (with pending revision and cases)	LST-02
DELETE	`/api/v1/admin/listings/{listing_id}`	Soft-delete a listing	LST-08
POST	`/api/v1/admin/listings/{listing_id}/approve`	Approve and publish (or approve the pending revision)	LST-02, LST-03
POST	`/api/v1/admin/listings/{listing_id}/reject`	Reject a listing	LST-02, LST-03
POST	`/api/v1/admin/listings/{listing_id}/request-changes`	Ask the host for changes	LST-02, LST-03, MOD-02
POST	`/api/v1/admin/listings/{listing_id}/suspend`	Suspend a published listing	LST-02, LST-03
POST	`/api/v1/admin/listings/{listing_id}/restore`	Restore a soft-deleted listing to its previous status	LST-02, LST-03
POST	`/api/v1/admin/listings/{listing_id}/status`	Any other allowed status change	LST-03
PUT	`/api/v1/admin/listings/{listing_id}/featured`	Feature or unfeature a listing	LST-09
GET	`/api/v1/admin/trips`	All trips, every status	TRP-04, TRP-07
GET	`/api/v1/admin/trips/{trip_id}`	One trip	TRP-04
POST	`/api/v1/admin/trips/{trip_id}/status`	Approve, reject, suspend, restore a trip	TRP-04

admin.bookings

Method	Path	What it does	Requirements
GET	`/api/v1/admin/bookings`	All bookings with facets	BKG-05, BKG-07, BKG-08
GET	`/api/v1/admin/bookings/{booking_id}`	One booking with its event history	BKG-05, BKG-06
POST	`/api/v1/admin/bookings/{booking_id}/cancel`	Cancel a booking as staff	CAN-03, CAN-04
POST	`/api/v1/admin/bookings/{booking_id}/refunds`	Record a full or partial refund	CAN-03, CAN-04, PAY-01
POST	`/api/v1/admin/bookings/{booking_id}/dispute/resolve`	Resolve a dispute	BKG-11
GET	`/api/v1/admin/refunds`	All refunds	CAN-03

admin.users

Method	Path	What it does	Requirements
GET	`/api/v1/admin/users`	Users (guests and hosts as people)	ACC-08, MOD-07
GET	`/api/v1/admin/users/{user_id}`	One user	ACC-08
DELETE	`/api/v1/admin/users/{user_id}`	Soft-delete a user (anonymisation follows)	PRV-02
POST	`/api/v1/admin/users/{user_id}/suspend`	Suspend a user	ACC-03, STF-03
POST	`/api/v1/admin/users/{user_id}/unsuspend`	Lift a user suspension	ACC-03, STF-03
POST	`/api/v1/admin/users/{user_id}/ban`	Ban a user	ACC-03, STF-03
POST	`/api/v1/admin/users/{user_id}/restore`	Restore a soft-deleted user	ACC-03, STF-03
GET	`/api/v1/admin/data-requests`	Data protection requests with deadlines	PRV-02
POST	`/api/v1/admin/data-requests/{data_request_id}/complete`	Close a data protection request	PRV-02

admin.reviews

Method	Path	What it does	Requirements
GET	`/api/v1/admin/reviews`	Reviews with reports	REV-04
POST	`/api/v1/admin/reviews/{review_id}/hide`	Hide a review	REV-04, REV-05
POST	`/api/v1/admin/reviews/{review_id}/restore`	Publish a hidden or flagged review again	REV-04, REV-05
POST	`/api/v1/admin/reviews/{review_id}/dismiss-reports`	Dismiss open reports and keep the review	REV-04, REV-05
DELETE	`/api/v1/admin/reviews/{review_id}`	Remove a review (status removed)	REV-04

admin.finance

Method	Path	What it does	Requirements
GET	`/api/v1/admin/finance/summary`	Finance overview	PAY-02, PAY-07, RPT-01
GET	`/api/v1/admin/wallets`	Host wallets	PAY-02, PAY-07
GET	`/api/v1/admin/wallets/{host_account_id}`	One host wallet	PAY-02
GET	`/api/v1/admin/ledger/transactions`	Ledger transactions with their entries	PAY-01, PAY-08
POST	`/api/v1/admin/ledger/adjustments`	Correct a wallet with an adjustment transaction	PAY-01
POST	`/api/v1/admin/wallets/{host_account_id}/topups`	Record a manual wallet top-up	PAY-05, PAY-02
GET	`/api/v1/admin/topups`	Top-ups (e.g. waiting for a second approval)	PAY-05
POST	`/api/v1/admin/topups/{topup_id}/approve`	Second approval of a large top-up (four-eyes)	PAY-05
POST	`/api/v1/admin/topups/{topup_id}/reject`	Reject a pending top-up	PAY-05
GET	`/api/v1/admin/commission-rates`	Default commission rates per host type	PAY-04, CFG-01
PUT	`/api/v1/admin/commission-rates`	Change default commission rates	PAY-04, CFG-01
GET	`/api/v1/admin/invoices`	Commission invoices	PAY-11
GET	`/api/v1/admin/payouts`	Payout requests	PAY-10 (dormant)
POST	`/api/v1/admin/payouts/{payout_id}/approve`	Approve a payout	PAY-10 (dormant)
POST	`/api/v1/admin/payouts/{payout_id}/reject`	Reject a payout	PAY-10 (dormant)
POST	`/api/v1/admin/payouts/{payout_id}/mark-sent`	Mark a payout as sent	PAY-10 (dormant)

admin.settings

Method	Path	What it does	Requirements
GET	`/api/v1/admin/settings`	Current platform settings	CFG-01
PATCH	`/api/v1/admin/settings`	Change settings (creates a new version)	CFG-01, BKG-03
GET	`/api/v1/admin/settings/versions`	Settings history	CFG-01

admin.onboarding

Method	Path	What it does	Requirements
GET	`/api/v1/admin/onboarding-flows`	Onboarding flows per host type	CFG-02
GET	`/api/v1/admin/onboarding-flows/{host_type}/versions`	All versions of one flow	CFG-02
GET	`/api/v1/admin/onboarding-flows/{host_type}/versions/{version}`	One version (published versions never change)	CFG-02
PUT	`/api/v1/admin/onboarding-flows/{host_type}/draft`	Save the draft flow	CFG-02, CFG-03
POST	`/api/v1/admin/onboarding-flows/{host_type}/publish`	Publish the draft as the next version	CFG-02, CFG-04
POST	`/api/v1/admin/onboarding-flows/{host_type}/versions/{version}/republish`	Roll back: republish an older version	CFG-02

admin.taxonomy

Method	Path	What it does	Requirements
GET	`/api/v1/admin/categories`	List categories	CFG-05, LST-05
POST	`/api/v1/admin/categories`	Create one of the categories	CFG-05, LST-05
PATCH	`/api/v1/admin/categories/{category_id}`	Edit, reorder, activate or deactivate one of the categories	CFG-05, LST-05
DELETE	`/api/v1/admin/categories/{category_id}`	Delete one of the categories if never used	CFG-05, LST-05
GET	`/api/v1/admin/amenities`	List amenities	CFG-05, LST-05
POST	`/api/v1/admin/amenities`	Create one of the amenities	CFG-05, LST-05
PATCH	`/api/v1/admin/amenities/{amenity_id}`	Edit, reorder, activate or deactivate one of the amenities	CFG-05, LST-05
DELETE	`/api/v1/admin/amenities/{amenity_id}`	Delete one of the amenities if never used	CFG-05, LST-05
GET	`/api/v1/admin/property-types`	List property types	CFG-05, LST-05
POST	`/api/v1/admin/property-types`	Create one of the property types	CFG-05, LST-05

Method	Path	What it does	Requirements
PATCH	`/api/v1/admin/property-types/{property_type_id}`	Edit, reorder, activate or deactivate one of the property types	CFG-05, LST-05
DELETE	`/api/v1/admin/property-types/{property_type_id}`	Delete one of the property types if never used	CFG-05, LST-05

admin.promotions

Method	Path	What it does	Requirements
GET	`/api/v1/admin/coupons`	Coupons	PRM-01
POST	`/api/v1/admin/coupons`	Create a coupon	PRM-01, PRM-02
PATCH	`/api/v1/admin/coupons/{coupon_id}`	Edit, enable or disable a coupon	PRM-01
DELETE	`/api/v1/admin/coupons/{coupon_id}`	Delete an unused coupon	PRM-01

admin.staff

Method	Path	What it does	Requirements
GET	`/api/v1/admin/staff`	Staff members	STF-01, STF-02
GET	`/api/v1/admin/staff/roles`	Roles and their permissions	STF-01
POST	`/api/v1/admin/staff/invitations`	Invite a staff member	STF-02
PATCH	`/api/v1/admin/staff/{staff_id}`	Change a staff role	STF-01
POST	`/api/v1/admin/staff/{staff_id}/deactivate`	Deactivate (revokes sessions at once)	STF-02
POST	`/api/v1/admin/staff/{staff_id}/activate`	Reactivate a staff member	STF-02
POST	`/api/v1/admin/staff/{staff_id}/reset-mfa`	Reset a lost second factor	ACC-05, STF-02

admin.audit

Method	Path	What it does	Requirements
GET	`/api/v1/admin/audit-events`	Audit log	STF-03, STF-04
POST	`/api/v1/admin/exports`	Export a filtered dataset as CSV	STF-06
GET	`/api/v1/admin/exports`	My recent exports	STF-06
GET	`/api/v1/admin/exports/{export_id}`	Export status and download link	STF-06

admin.analytics

Method	Path	What it does	Requirements
GET	`/api/v1/admin/analytics/kpis`	KPIs with the previous period	RPT-01
GET	`/api/v1/admin/analytics/charts`	Chart series	RPT-01, RPT-02
GET	`/api/v1/admin/search`	Command palette search	STF-05

07. Architecture: how the Django backend is organised

> **In plain words**
> The backend is **one Django application** split into 17 small parts ("apps"), each owning its own tables and rules.
> Parts may only call parts below them, so a change in bookings can never break accounts by accident.
> Every write goes through a "service" function inside a database transaction; side effects (SMS, emails, search refresh) are queued in an "outbox" and sent after the save.
> The two Next.js apps never talk to Django from the browser: their own server keeps the login cookie and calls the API (BFF).
> We will move the apps from demo data to the real API **one module at a time**, each behind a switch, so we can always go back.
> This page also lists every background job, the caching and search plan, the photo upload pipeline, and the future repository layout.

Terms used here (see also glossary.md):

- **App (Django app)**: a folder inside the Django project with its own models (tables), logic and API views. Here one app = one business area ("bounded context").
- **Modular monolith**: one program, one database, deployed as one unit, but split inside into apps with strict rules about who may call whom.
- **Service**: a Python function that changes data (create a booking, approve a host). It runs the checks and the transaction.
- **Selector**: a Python function that only reads data (list my bookings). It never changes anything.
- **Transaction**: a group of database changes that all succeed together or are all cancelled together.
- **Row lock** (`SELECT ... FOR UPDATE`): while one transaction works on a row, others that want to change the same row wait.
- **Deadlock**: two transactions each wait for a row the other has locked. PostgreSQL stops one of them. A fixed lock order prevents it.
- **Outbox**: a table where a service writes "things to do after saving" in the same transaction; a worker reads it and does them.
- **Celery worker / Celery beat**: a worker runs background jobs; beat is the clock that starts scheduled jobs.
- **Queue**: a waiting line of jobs inside Redis. Different queues let slow jobs (images) not delay urgent ones (codes by SMS).
- **Cache**: a fast copy of data kept in Redis memory to avoid asking the database again.
- **Presigned URL**: a web address, valid a few minutes, that lets the browser upload or download one file directly to storage without seeing our keys.
- **BFF (backend for frontend)**: the Next.js server keeps the login cookie and calls Django for the browser.
- **Typed client**: TypeScript code generated from `openapi.yaml`, so the apps cannot call an endpoint with the wrong fields without a compile error.
- **Feature flag**: a setting that turns one part on or off without a new deployment.
- **import-linter**: a tool that fails the build if one app imports code it is not allowed to import.

Inputs: 02-research.md (section B), 03-requirements.md, 04-business-rules.md, 05-database-schema.md (ADR-001 to ADR-014), schema.dbml, diagrams 01-system-context, 02-containers, 03-django-apps.

---

1. The big picture

Layer	What runs	Where it is described
People	Guests, hosts, organisation members, BLADUS staff	01-system-context
Edge	Reverse proxy (Caddy) with TLS and coarse rate limits; CDN only for non-personal static files	09-devops-hosting.md
Frontends	Website (Next.js, guests and hosts) and Admin (Next.js, staff). Each has its own server (BFF)	section 10
API	Django 6.1 + DRF, one process type, modular monolith, `/api/v1/...`	sections 2 to 6
Background	Celery workers (3 queues) and one Celery beat	section 6
Data	PostgreSQL 17 + PostGIS, Redis, S3-compatible object storage, all in Algeria	02-containers

Decisions behind this shape are summarised as ADRs in section 12. The main ones: one Django project, not microservices (ADR-015); PostgreSQL for search at launch (ADR-020); polling for messages (ADR-021).

---

2. The Django apps

Seventeen domain apps plus one shared kernel (`core`). Tables are the short names used in schema.dbml; Django will prefix them with the app name (`bookings_booking`). All 77 tables have exactly one owner.

#	App	Responsibility	Owned tables	Main rules
0	`core` (shared kernel)	Base model (UUIDv7 id, timestamps), `Money` type, error codes, idempotency, request context (user, host account, locale, request id), pagination, clock	`idempotency_record`	BR-SYS-01, BR-SYS-04, BR-SYS-05, NFR-12
1	`geo`	Wilayas, dairas, communes; distance and map helpers; commune autocomplete	`wilaya`, `daira`, `commune`	BR-SRC-02
2	`config`	Versioned platform settings; versioned onboarding flows per host type; feature flags (inside settings JSON)	`platform_settings`, `onboarding_flow_version`	BR-CFG-01 to 03
3	`audit`	Hash-chained audit log; audited CSV exports; incident register with the 5-day breach clock	`audit_event`, `export_job`, `incident_record`	BR-STF-02, BR-STF-04, PRV-05
4	`media`	Upload slots, quarantine, scanning, re-encoding, renditions, signed download URLs	`media_asset`	BR-MED-01, BR-MED-02
5	`notifications`	Outbox table and dispatcher, in-app notifications, email and SMS providers, preferences, delivery log	`outbox_event`, `delivery_log`, `notification`, `notification_preference`	BR-SYS-03, BR-NTF-01, BR-NTF-02
6	`accounts`	Users, sign-up and login (allauth headless), phone OTP and its send log, MFA, staff profiles, staff roles and permissions, consents, favourites, data subject requests	`user`, `staff_profile`, `staff_permission`, `staff_role_permission`, `consent_record`, `favorite`, `data_subject_request`, `otp_send_log`	BR-ACC-01 to 05, BR-STF-01, BR-PRV-01, BR-PRV-02, ACC-02

#	App	Responsibility	Owned tables	Main rules
7	`hosts`	Host accounts, organisations, memberships and roles, invitations, onboarding drafts, KYC documents, payout accounts, host verification state machine	`host_account`, `organization`, `membership`, `invitation`, `host_document`, `payout_account`, `onboarding_draft`	BR-ORG-01 to 06, BR-HST-01 to 05, BR-PAY-09
8	`catalog`	Listings of the 5 verticals, revisions under review, vehicle and experience details, room types, photos order, amenities, categories, property types, cancellation policies, featured placements	`listing`, `listing_revision`, `vehicle_detail`, `experience_detail`, `room_type`, `listing_amenity`, `room_type_amenity`, `listing_photo`, `featured_placement`, `category`, `property_type`, `amenity`, `cancellation_policy`	BR-LST-01 to 05, BR-HST-05
9	`inventory`	Calendar per day (homes, vehicles), per night and room type (hotels, hostels), per time slot (experiences); holds; horizon generation; iCal import and export feeds (Later)	`listing_day`, `room_type_night`, `experience_slot`, `calendar_feed`	BR-AVL-01 to 06, AVL-06
10	`trips`	Agency trips, itinerary days, departures with seats	`trip`, `trip_itinerary_day`, `trip_departure`	BR-TRP-01 to 03
11	`pricing`	Quotes, length-of-stay discounts, tax rates, coupons (R2)	`quote`, `tax_rate`, `coupon`, `coupon_redemption`	BR-PRC-01 to 04, BR-PRM-01, 02
12	`ledger`	Double-entry ledger, balances, commission posting and reversal, manual top-ups, gateway webhooks (R2), dormant payments, payouts, invoices	`ledger_account`, `ledger_transaction`, `ledger_entry`, `account_balance`, `wallet_topup`, `gateway_event`, `payment`, `payout`, `invoice`	BR-PAY-01 to 12, BR-SYS-02
13	`bookings`	Booking creation (instant and request), state machine, snapshots, cancellations, refunds owed, disputes (R2)	`booking`, `booking_event`, `refund`, `dispute`	BR-BKG-01 to 07, BR-CAN-01 to 05
14	`reviews`	Reviews, replies, reports, rating caches	`review`, `review_reply`, `review_report`	BR-REV-01 to 04
15	`messaging`	Conversations, participants, messages, contact masking, cursor sync	`conversation`, `conversation_participant`, `message`	BR-MSG-01
16	`trust`	Moderation queues and cases, decisions, change requests, reason catalogue, suspension cascade, risk flags (R2)	`moderation_case`, `moderation_decision`, `change_request_item`, `reason_code`	BR-MOD-01 to 04
17	`search`	Denormalised search table, query building, facets, map pins, trip search	`listing_search_doc`	BR-SRC-01, 03, 04

Count check: 1 + 3 + 2 + 3 + 1 + 4 + 8 + 7 + 13 + 4 + 3 + 4 + 9 + 4 + 3 + 3 + 4 + 1 = 77 tables (74 from Phase 5 plus `incident_record`, `calendar_feed` and `otp_send_log` added on 2026-11-09).

Differences from the Phase 4 diagram 03-django-apps: it shows 16 apps. This page adds `search` (owner of `listing_search_doc`) and the `core` kernel. `onboarding_draft` sits in `hosts` (it becomes a host account on submit), `data_subject_request` in `accounts` (it acts on the user), coupons in `pricing` (they change a quote). The diagram will be updated in the next diagram pass; the table above wins if they differ.

2.1 Allowed dependencies

Apps are placed in layers. **An app may import only the public functions (`services.py`, `selectors.py`) of apps in a lower layer, or of an earlier app in the same layer as listed.** Model foreign keys follow the same direction, with one exception: every app may hold a foreign key to `accounts.User` (the shared identity) and to `hosts.HostAccount`, because "who did it" and "for which host" are everywhere.

Layer	Apps (in order)	May import from
L0 kernel	`core`	nothing inside the project
L1 platform	`geo`, `config`, `audit`, `media`, `notifications`	`core`
L2 identity	`accounts`, then `hosts`	L0, L1; `hosts` may import `accounts`
L3 offer	`catalog`, then `inventory`, then `trips`	L0 to L2; `inventory` may import `catalog`; `trips` may import `catalog` (shared policies, statuses)
L4 commerce	`pricing`, then `ledger`, then `bookings`	L0 to L3; `bookings` may import `pricing` and `ledger`
L5 after booking	`reviews`, `messaging`	L0 to L4
L6 oversight and read models	`trust`, `search`	L0 to L5

How lower apps learn about higher events without importing them: through **outbox topics**. For example `bookings` writes `booking.confirmed`; `notifications` (L1) dispatches it to handlers that higher apps registered in their `apps.py` (`search` refreshes availability, `reviews` schedules the review window). The handler code lives in the higher app; the dispatcher only knows topic names.

The rule is checked in CI by **import-linter** with one "layers" contract and one "independence" contract per layer (for example `reviews` and `messaging` may not import each other). A violation fails the build.

2.2 Inside one app (services and selectors pattern)

Each app has the same files (HackSoft Django styleguide, research B.2 and B.15):

File	Contains	Rules
`models.py`	Tables, `Meta.constraints`, enums	No business logic beyond `clean()` for single-field format checks
`services.py`	Every write: `booking_create(...)`, `host_approve(...)`	Keyword-only arguments; opens the transaction; checks permissions passed in by the API layer; raises `DomainError(code, field, params)`; writes `booking_event` / `audit_event` / `outbox_event`
`selectors.py`	Every read used by APIs and other apps	Always scoped by the caller (`bookings_for_guest(user)`, `bookings_for_host(host_account)`); returns querysets or plain data; no writes
`apis.py`	DRF views, one class per endpoint, with its input and output serializers inside	Thin: parse input, call one service or selector, serialize output. Never `Model.objects.create` here
`permissions.py`	django-rules predicates (`is_member_with(perm)`, `is_staff_with(perm)`)	Deny by default
`events.py`	Outbox topic names this app emits, handlers it registers	Handlers are idempotent
`tasks.py`	Celery tasks	Thin: load ids, call a service. No logic in the task itself, so moving to Django's built-in tasks later is easy (research B.5)
`admin.py`	Django admin console registrations	Read-mostly; any write calls a service
`tests/`	Factories, unit, service, API tests	See 10-testing-quality.md

Other conventions:

- **No Django signals for business logic.** Signals run hidden and cannot be rolled back with the outbox; we use explicit service calls and outbox topics (ADR-017). `pghistory` triggers are the only automatic history.
- **Errors**: services raise `DomainError("dates_unavailable")`; one DRF exception handler turns it into `{"code", "field", "message", "params"}` with HTTP status from a fixed table (BR-SYS-04).
- **Time**: services receive `now` from `core.clock` so tests can freeze it.
- **Money**: only the `core.money.Money` type crosses function boundaries; raw integers only in models.

2.3 API surfaces

Prefix	Who	Authentication	Notes
`/_allauth/app/v1/...`	Everyone	allauth headless (app client, session token)	Sign-up, login, phone code, MFA, password reset
`/api/v1/public/...`	Anyone	none or session	Search, listing detail, trip detail, organisation profile, geography, taxonomy
`/api/v1/me/...`	Signed-in user	session token	Profile, favourites, my bookings (`/me/bookings`, ADR-004), conversations, notifications, privacy requests
`/api/v1/host/...`	Host or organisation member	session token + `X-Host-Account` header	Every request is checked: the user must hold that host account (BR-ORG-01)
`/api/v1/admin/...`	Staff only	staff session token with MFA done	Separate URL module and permission classes; a guest or host token never reaches it (STF-05)
`/api/v1/webhooks/...`	Payment gateways (R2)	signature check	Only path exposed on the public webhook host name
`/internal/healthz`, `/internal/readyz`, `/internal/metrics`	Monitoring	private network only	Not routed by the public proxy

Common headers: `Accept-Language` (`en`, `fr`, `ar`), `Idempotency-Key` (required on create and money endpoints, BR-SYS-01), `X-Request-ID` (created by the BFF if missing, logged everywhere). Pagination: cursor based, page size 24 by default, maximum 50 (SRC-05). Versioning: breaking changes go to `/api/v2/` (NFR-08).

---

3. Transactions and locking rules

General rules:

1. `ATOMIC_REQUESTS` is **off**. Each service that writes decorates itself with `transaction.atomic()`. One service call = one transaction.
2. Isolation level stays **READ COMMITTED** (PostgreSQL default). Correctness comes from constraints, conditional updates and row locks, not from SERIALIZABLE.
3. **No network call inside a transaction** (no SMS, no storage, no gateway). Those go to the outbox or to `transaction.on_commit`.
4. Database time limits per connection: `statement_timeout = 5 s` for the API, `60 s` for workers; `lock_timeout = 2 s`; `idle_in_transaction_session_timeout = 30 s`.
5. A `@retry_on_conflict` decorator retries a whole service up to 3 times on deadlock (`40P01`) or serialization failure (`40001`), with a short random wait. Constraint violations are **not** retried; they become clean `409` errors.
6. **Global lock order** (always take locks in this order, never the reverse, to prevent deadlocks):
    1. `idempotency_record` (by key)
    2. `booking` row (state changes)
    3. inventory rows: `room_type_night` by (room type, night ascending), `experience_slot`, `trip_departure`
    4. `account_balance` row of the host wallet
    5. inserts only: `ledger_transaction`, `ledger_entry`, `booking_event`, `audit_event`, `outbox_event`

3.1 Creating a booking (instant or request)

One transaction, in this order (matches 20-seq-instant-booking):

Step	What	Protects against	Rule
1	Insert `idempotency_record (user, endpoint, key)` with `status = in_progress`. If it exists: same body hash and completed returns the stored response; still in progress returns `409 request_in_progress`; different body returns `422 idempotency_key_reused`	Double click, network retry	BR-SYS-01
2	Load and re-validate the quote (not expired, prices unchanged); recompute if changed and return `409 price_changed` with the new quote	Stale or tampered price	BR-PRC-01
3	Resolve approval mode and check guest phone verified	Wrong flow	BR-BKG-01, 02, BR-ACC-01
4a	**Single unit** (home, vehicle): insert the `booking` row. The exclusion constraint refuses overlap; catch SQLSTATE `23P01` and return `409 dates_unavailable`	Double booking under concurrency	BR-AVL-01
4b	**Pool** (room type, beds, slot, departure): one conditional `UPDATE ... SET booked = booked + n WHERE ... AND held + booked + n <= total`; if updated rows differ from the number of nights (or 1 for slots and departures) the transaction is rolled back with `409 dates_unavailable`	Overselling	BR-AVL-02, BR-TRP-02
5	If instant and confirmed: `SELECT ... FOR UPDATE` on the host wallet `account_balance`; check the credit floor; if below the floor the booking becomes `pending` instead	Two confirmations spending the same deposit	BR-PAY-10
6	Post the commission transaction (2 entries), update the balance row	Missing or double commission (unique `(booking_id) WHERE kind = commission`)	BR-PAY-04
7	Insert `booking_event`, `audit_event` (host money effect), `outbox_event` rows (`booking.created` or `booking.confirmed`)	Lost notifications	BR-SYS-03
8	Mark `idempotency_record` completed with the response body	Retry returns the same booking	BR-SYS-01
9	Commit. The deferred ledger trigger checks the balance of every transaction at this moment	Unbalanced ledger	BR-PAY-02

3.2 Changing a booking status

Accept, decline, cancel, expire, complete: `SELECT booking FOR UPDATE`, check the transition table (BR-BKG-06, table 7a in 04-business-rules.md), check the actor's permission (`bookings.respond` for accept and decline, DEC-009), then release or convert inventory (pool counters `booked - n`; single units are freed by the status change itself because the exclusion constraint only covers blocking statuses), post commission or reversal, insert `booking_event`, outbox. Jobs (expiry, completion) use the same service with `actor = system`, and process rows in small batches with `FOR UPDATE SKIP LOCKED` so two job runs never fight.

3.3 Inventory counters and holds

- Holds (`held + n`) are used only when a payment step exists (R2 and later) or for the 15-minute checkout of pooled inventory (AVL-05). A hold has an expiry; the job `inventory.release_expired_holds` turns `held - n` back, and reads ignore expired holds (BR-AVL-06).
- Host calendar edits (block a night, change a price) lock the same counter rows in the same order, so a host cannot block a night while a guest books it: one of them waits, then sees the result.
- Reducing `total` on a room type below `held + booked` is refused by the check constraint; the host sees `409 inventory_in_use` with the nights in conflict.

3.4 Ledger

- Only `ledger.services` writes ledger tables. Every posting function takes an **idempotency source** (`booking_id + kind`, `wallet_topup.id`, `gateway_event.id`) protected by a unique index, so a retried job never posts twice.
- The balance row is locked (`FOR UPDATE`) before any posting that can reduce it. Balance = cached sum, verified nightly against the real sum (`ledger.reconcile_balances`).
- Corrections are reversing transactions, never updates (BR-PAY-02). The app database role has no `UPDATE` or `DELETE` grant on ledger tables.
- Four-eyes top-ups (BR-PAY-07): the first finance user creates the top-up in `pending_approval`; nothing is posted until a different finance user approves. Approval and posting are one transaction.

---

4. Outbox

```
service (one transaction) ──writes──> business rows + outbox_event(topic, payload, next_attempt_at=now)
                                  |
transaction.on_commit ──nudges──> Celery "outbox.dispatch" (also runs every 10 s as a safety net)
                                  |
dispatcher: SELECT ... FROM outbox_event WHERE status='pending' AND next_attempt_at <= now
            ORDER BY id LIMIT 100 FOR UPDATE SKIP LOCKED
            -> for each registered handler of the topic: run it (idempotent)
            -> success: status='sent'; failure: attempts+1, next_attempt_at = now + backoff
```

- **Backoff**: 30 s, 2 min, 10 min, 30 min, 1 h, then every hour; after 10 attempts the row becomes `failed` and an alert fires.
- **Idempotent handlers**: email and SMS handlers check `delivery_log (outbox_event_id, channel)` before sending; search refresh is naturally idempotent (it rebuilds the row).
- **Payload**: ids plus a small display snapshot (booking reference, listing title in the source language, amounts) so handlers rarely need to read other apps.
- **Retention**: sent rows deleted after 14 days; failed rows kept until a person resolves them.

Main topics: `booking.created`, `booking.confirmed`, `booking.declined`, `booking.expired`, `booking.cancelled`, `booking.completed`, `listing.published`, `listing.changed`, `listing.paused`, `host.verification_changed`, `review.published`, `wallet.below_zero`, `wallet.below_floor`, `wallet.topup_posted`, `message.created`, `media.uploaded`, `otp.requested`, `privacy.request_created`, `export.requested`.

---

5. Background jobs (Celery)

Queues: `critical` (one-time codes and booking notifications, low latency), `default` (most jobs), `media` (image work, CPU heavy, its own worker), `reports` (exports, long). Beat runs as **exactly one** container. All schedules are in Africa/Algiers time. Every task is safe to run twice (`acks_late = True`, idempotent services).

Job	Schedule	What it does	Rule / req
`notifications.dispatch_outbox`	every 10 s + after each commit	Sends pending outbox events to their handlers	BR-SYS-03, NTF-04
`notifications.send_email` / `send_sms`	on event (`critical` queue)	Renders the template in the user's language and sends via the provider interface; writes `delivery_log`	BR-NTF-02
`notifications.sms_budget_check`	every 15 min	Sums SMS spend today and this month from `otp_send_log` and `delivery_log`; compares to budget; alerts at 80%; at 100% switches OTP to email or WhatsApp where possible; flags phone or IP ranges with abnormal OTP volume (pumping)	NFR-11, ACC-02, research C.2
`inventory.release_expired_holds`	every 1 min	Releases holds older than 15 minutes	BR-AVL-06
`inventory.extend_calendar_horizon`	daily 02:00	Creates `room_type_night` and recurring `experience_slot` rows up to 18 months ahead	ADR-002
`bookings.expire_pending_requests`	every 5 min	Pending requests past `request_expires_at` become `expired`; counters released; both sides notified	BR-BKG-04, BKG-03
`bookings.complete_stays`	every 15 min	Confirmed bookings past check-out + 2 h become `completed`; review window opens; earnings counted	BR-BKG-05, BKG-04
`pricing.purge_expired_quotes`	hourly	Deletes quotes expired more than 24 h ago and not linked to a booking	BR-PRC-01
`reviews.recompute_ratings`	on `review.*` event + nightly 03:00 full pass	Recomputes listing, host account and organisation ratings and counts	BR-REV-04, REV-05
`reviews.publish_blind_reviews`	hourly (R2)	Publishes double-blind reviews when both are in or 14 days passed	BR-REV-02
`hosts.expire_invitations`	hourly	Marks invitations past 7 days as expired	BR-ORG-03
`hosts.recount_host_cancellations`	daily 01:00	Recomputes `host_cancellations_12m`; opens a moderation case at 3	BR-CAN-03
`ledger.reconcile_balances`	daily 01:30	Checks every `account_balance` equals the sum of its entries and every transaction sums to zero; alerts on any difference	BR-PAY-02, BR-PAY-03
`ledger.monthly_invoices`	1st of month 03:00 (R2)	Builds commission invoices with gap-free numbers	BR-PAY-12

Job	Schedule	What it does	Rule / req
`ledger.process_gateway_event`	on webhook (R2)	Verifies and applies a stored gateway event once	BR-PAY-08, BR-SYS-02
`media.process_photo`	on `media.uploaded` (`media` queue)	Validates, re-encodes, strips metadata, makes renditions, computes perceptual hash	BR-MED-01
`media.scan_document`	on `media.uploaded` (`media` queue)	ClamAV scan, moves the file to the private bucket or rejects it	BR-MED-02
`media.purge_orphans`	daily 04:00	Deletes quarantined or unattached uploads older than 24 h	BR-MED-01
`trust.refresh_queue_priority`	every 15 min	Raises priority of cases waiting too long	BR-MOD-01
`trust.evaluate_risk_rules`	on events (R2)	Runs fraud rules, opens cases, never blocks silently	BR-MOD-04
`search.refresh_listing_doc`	on `listing.*`, `review.published`, `host.verification_changed`	Rebuilds the listing's search row or removes it	ADR-011, BR-SRC-01
`search.rebuild_all`	weekly, Sunday 04:00	Full rebuild and comparison, alerts on drift	ADR-011
`accounts.purge_expired_sessions`	daily 02:30	Removes expired session tokens and OTP records	BR-ACC-03
`accounts.process_deletion_requests`	daily 05:00	Anonymises users whose deletion request is due; keeps legal records	BR-PRV-02, PRV-02
`accounts.retention_sweep`	daily 03:30	Deletes ID images 90 days after decision, old messages, `otp_send_log` rows older than 90 days, old logs per the retention table	BR-PRV-05, PRV-04
`audit.incident_deadline_watch`	hourly	For open `incident_record` rows with personal data involved and no `anpdp_notified_at`: alerts the owner and super admins at day 2, day 4 and 12 hours before `anpdp_deadline_at`	PRV-05
`inventory.sync_calendar_feeds`	every 3 h (Later)	Fetches active iCal import feeds through the egress proxy (private IPs blocked), applies blocks, records `last_error`	AVL-06
`core.purge_idempotency_records`	hourly	Deletes idempotency records older than 24 h	BR-SYS-01
`audit.verify_hash_chain`	daily 00:30	Recomputes the chain for yesterday, stores the day's last hash as an anchor, alerts on break	BR-STF-02
`audit.create_next_partition`	monthly, 20th	Creates next month's partitions of `audit_event` (and `message` once partitioned)	ADR-012
`audit.run_export`	on request (`reports` queue)	Builds a CSV export, formula-safe, stores it privately, audits it	BR-STF-04, STF-06

---

6. Caching strategy

Redis is used for three separate things, in separate logical databases so one cannot evict the other: **db 0** Celery broker, **db 1** cache, **db 2** rate limits, OTP counters and login locks (with `maxmemory-policy noeviction` on that instance, because losing a lock counter is a security hole).

What	Where	Lifetime	Invalidation
Geography, taxonomy lists, experience and trip type lists	Redis + HTTP `Cache-Control: public, max-age=3600`	24 h	Version key bumped by the service that changes them
Current platform settings and published onboarding flows	Redis	60 s	Bumped on publish (BR-CFG-01, 02)
Public listing detail (without availability)	Redis	5 min	Outbox `listing.changed` deletes the key
Search results	not cached at launch		Measure first (NFR-02). If needed: 60 s cache on identical anonymous queries without dates
Facet counts for the home page	Redis	5 min	Time only
Availability, prices in a booking, balances, permissions, anything personal	**never cached**		Always read from PostgreSQL inside the transaction

HTTP rules: personal responses send `Cache-Control: private, no-store`. Public GET responses send an `ETag`; the Next.js BFF may cache them with tagged revalidation. The CDN caches only static files and public photo renditions (see 09-devops-hosting.md), never API responses.

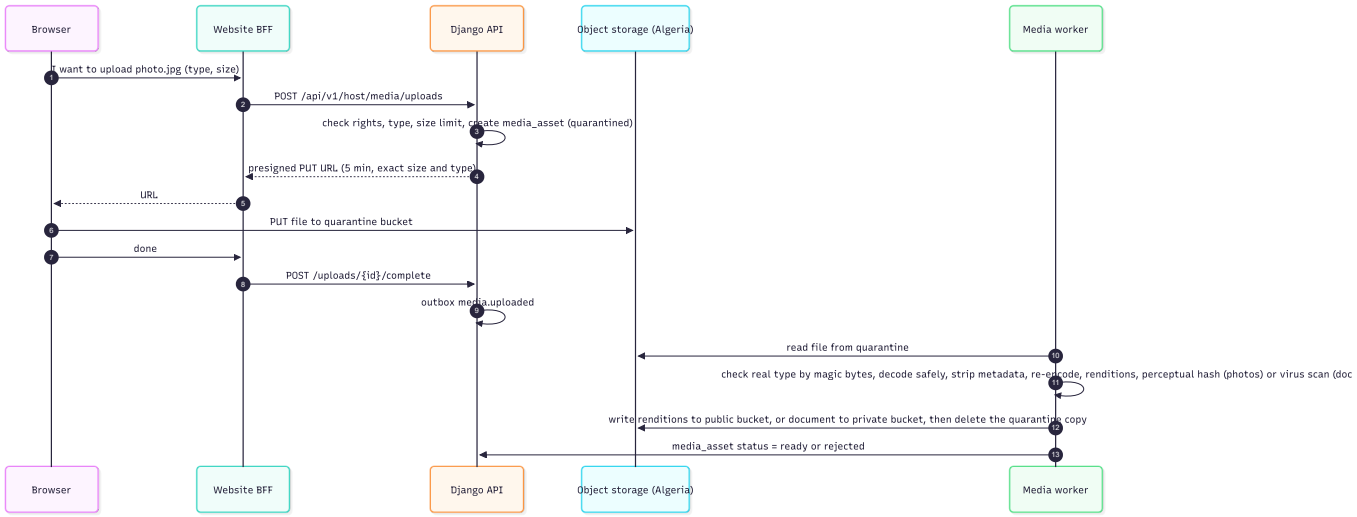
7. Search strategy

PostgreSQL only at launch (research B.7, ADR-011, ADR-020).

1. **Candidate filter** on `listing_search_doc`: vertical, wilaya or commune, guests (`capacity_guests >= n`), price range, property type, experience type, amenities (`amenity_ids @> {...}` with GIN), instant only, rating, map box (`location_public && ST_MakeEnvelope(...)` with GiST). Only published listings of approved, non-suspended host accounts have a row (BR-SRC-01).
2. **Text**: `tsv_fr`, `tsv_en`, `tsv_ar` with the right dictionary for the request language, plus trigram similarity on `search_text` (unaccented title and place names) for typos and transliteration ("Tipaza", "Tipasa", "تيبازة").
3. **Availability** (only when dates are given): for each candidate, a `NOT EXISTS` probe on the booking exclusion index (single units) or a range read on `room_type_night` (pools), plus host blocks from `listing_day`. Partial matches return the largest free sub-range (BR-AVL-05).
4. **Facets**: each facet count is computed on the filtered set without that facet's own filter (BR-SRC-01), with one `GROUP BY` per facet on the candidate set.
5. **Sort and page**: relevance, price, rating, newest; cursor pagination, 24 per page, max 50; map mode returns up to 500 pins with public points only (SRC-02, BR-SRC-03).
6. **Trips** are searched separately on `trip` + `trip_departure` (departures overlapping the dates, seats left) and shown in their own tab (ADR-004, TRP-05).
7. **Place autocomplete** uses `commune` with trigram indexes on French and Arabic names (Q15 in the schema). Free-text address lookup through Nominatim is server-side, cached, and at most 1 request per second (research C.5).

When to add an external engine (Meilisearch first): if search p95 stays above 500 ms with 20,000 listings after index tuning (NFR-02), or if Arabic typo tolerance is not good enough in user tests. The search table and outbox topics make this a new consumer, not a rewrite.

8. File upload pipeline



Step	Photos (listing, avatar, logo)	Private documents (KYC, message attachments R2)
Allowed types	JPEG, PNG, WebP, HEIC	PDF, JPEG, PNG
Max size	15 MB	10 MB
Checks	Magic bytes, Pillow decompression-bomb limit (50 megapixels), dimensions at least 1024 px wide for listing photos	Magic bytes, ClamAV scan, PDF without JavaScript or embedded files
Transform	Re-encode to WebP and JPEG; renditions 320, 640, 1280, 1920 px; all metadata including GPS removed	Images re-encoded and metadata removed; PDFs stored as uploaded after a clean scan
Duplicates	Perceptual hash compared with other hosts' photos; a match opens a trust case (BR-MOD-04)	SHA-256 stored
Final bucket	`public` (served by CDN with random keys)	`private` (no public access, signed GET valid 5 minutes, every view audited)
Rules	BR-MED-01, MED-01, MED-03	BR-MED-02, MED-02

The browser never receives storage keys, only presigned URLs. Storage keys are random UUIDs, never file names.

9. Internationalisation (i18n)

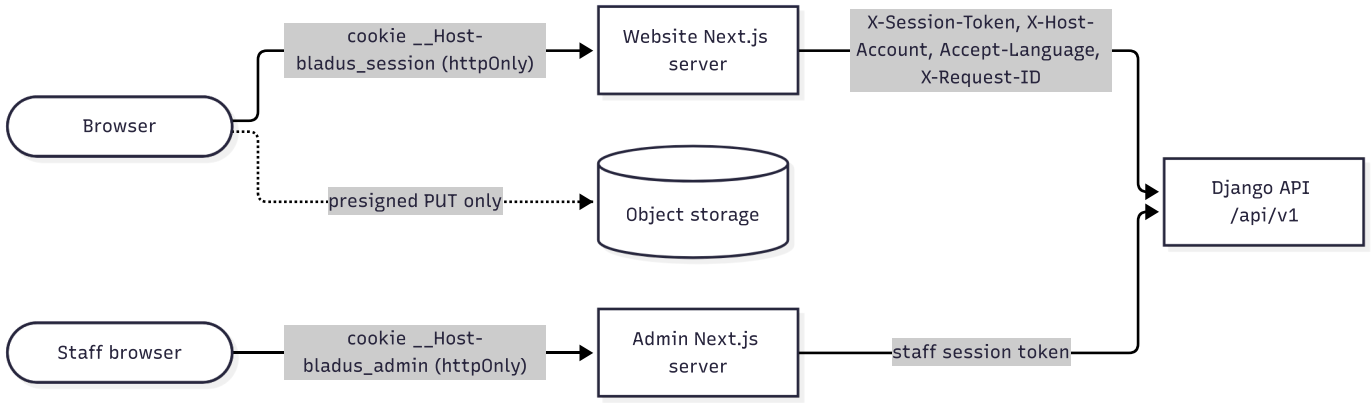
Topic	Approach
Languages	`en`, `fr`, `ar`. Default `fr` (user column `preferred_locale`). Fallback order: requested, then `fr`, then `en`
How the API knows the language	The BFF sends `Accept-Language` from the URL locale (`/ar/...`); Django `LocaleMiddleware` activates it. Background jobs use the recipient's `preferred_locale`
Platform lists (amenities, categories, property types, wilayas, communes, reason codes, experience and trip types)	django-modeltranslation columns `name_en`, `name_fr`, `name_ar` (research B.9). The API returns the active language only, or all three on admin endpoints
Host-written text (titles, descriptions, reviews, messages)	Stored once with `source_language`; shown as written. Machine translation is a later option
API errors	Stable snake_case `code` (for example `dates_unavailable`) plus a translated `message`. The apps map the code to their own text (NFR-13, BR-SYS-04)
Notifications, emails, SMS	Django templates per language; SMS texts kept short (Arabic SMS has 70 characters per segment)
Numbers and dates	Latin digits in all languages (`ar-DZ` with `latn`); dates ISO 8601 in the API; display formatting is done by the apps
Time zone	Stored in UTC; stay dates and business days in Africa/Algiers; week starts Sunday; weekend Friday and Saturday (BR-SYS-05)
Translation files	`.po` files in the repository; CI fails if a message is missing in one language (10-testing-quality.md section 8)

10. How the website and the admin will use the API

10.1 What exists today

Both apps are Next.js 14. All data goes through `lib/data/*` functions that read and write **in-memory mock stores** on the Next.js server (`frontend/lib/data/_store.ts`, `admin/lib/mock-data/store`). Pages and server actions call only these functions; the comments say "TODO: replace with Supabase", which this blueprint replaces with "replace with the Django API". The website signs its own session cookie (`DARNA_SESSION_SECRET`); the admin signs a JWT (`ADMIN_AUTH_SECRET`). The website has a **test mode** (`NEXT_PUBLIC_BLADUS_TEST_MODE`, ON unless set to `0`) that allows one-click demo login, a built-in secret and non-Secure cookies. **Test mode must be OFF (`0`) in every environment with real people's data**; the deploy pipeline refuses a production build where it is not `0` (09-devops-hosting.md section 7).

10.2 Target: BFF



- The browser **never** holds the Django token in JavaScript. Next.js stores the allauth session token inside an encrypted, httpOnly cookie and sends it to Django in the `X-Session-Token` header from server code only (cookie flags in 08-security-privacy.md section 4).
- Django is reachable from the Next.js servers over the private network. The public internet reaches only the Next.js apps and, in R2, the webhook path.
- The BFF passes the real client IP in `X-Forwarded-For`; Django trusts that header only from the BFF and proxy addresses (`TRUSTED_PROXY_IPS`), so rate limits and audit logs see the real IP.
- Client-side components that need fresh data (message polling, notifications badge) call a **Next.js route handler** that forwards to Django; they never call Django directly.

10.3 Typed client

- Django generates `openapi.yaml` with drf-spectacular in CI (NFR-08). It is published as a build artifact and copied into both apps (`lib/api/schema.d.ts`) by `openapi-typescript`.
- A small `server-only` wrapper `lib/api/client.ts` built on `openapi-fetch` adds the headers, timeouts (5 s), one retry for idempotent GETs, and turns API errors into the existing `ActionResult` shape. API `code` values are mapped to the apps' `ERROR_CODES` list (for example `dates_unavailable` to the website's existing date error codes); new codes are added to the `messages/*.json` files in en, fr and ar.
- Both apps run `tsc --noEmit` against the generated types in CI. An API change that breaks a call breaks the app build before deployment.

10.4 Migration from mock data to the API, one module at a time

Each `lib/data/<module>.ts` keeps its exported function names and return types. Inside, it chooses a source:

```
lib/data/listings.ts          (unchanged signatures, used by pages and actions)
  ├─ lib/data/mock/listings.ts (today's code, moved)
  └─ lib/api/listings.ts       (calls Django through the typed client, maps DTOs)
switch: server-only env var BLADUS_API_MODULES="geo,taxonomy,listings"
```

The flag is a **server-only** variable (never `NEXT_PUBLIC_`), read at request time, so a module can be switched back without rebuilding. For each module: (1) build the Django endpoints and contract tests; (2) write the API adapter and a parity test (same fixture through mock and API gives the same DTO shape); (3) in staging, run in "shadow" mode for a few days (serve mock, call the API too, log differences); (4) switch staging, then production; (5) after two weeks without rollback, delete the mock code.

Order	Website module(s)	Admin module(s)	Why this order
1	`geo` (via `lib/algeria.ts`), catalog type lists	`categories`	Read-only reference data, lowest risk; proves the client, headers and i18n
2	`auth` (sessions, OTP, profile)	`auth`, `team` (staff login with MFA)	Every personal module needs real users; removes the signed-cookie and JWT secrets
3	`listings`, `experiences`, `vehicles`, `trips` (read: search, detail)	`listings` (read, moderation)	Public pages; search performance can be measured
4	`organizations` + onboarding, documents	`hosts`, `approvals`, `onboarding`, `settings`	Hosts must exist and be verified before they publish
5	`listings` (write), `calendar`	`listings` (write)	Listing review flow (DEC-003)
6	`reservations`, `wallet`	`bookings`, `finance`, `ledger`	The money and availability core; needs concurrency tests green
7	`favorites`, `notifications`, `conversations`	`users`, `network`	Personal features on top of bookings
8	`reviews`	`reviews`, `audit`, `analytics`, `promotions`, `command-palette`	Need real completed bookings and audit data

Rules during migration: a module switched to the API never writes to the mock store again; mixed states are allowed only between modules that do not share writes (for example `reservations` and `wallet` switch together because a booking posts commission).

---

11. Future Django repository layout

A separate repository (`bladus-api`), or a `backend/` folder in this monorepo; decided at build time. Nothing below is created in this phase.

```
bladus-api/
├── pyproject.toml             # dependencies (uv), ruff, mypy, pytest, import-linter config
├── uv.lock
├── manage.py
├── Dockerfile                 # one image for api, workers and beat
├── compose.yaml               # dev stack (see 09-devops-hosting.md)
├── compose.prod.yaml
├── .github/workflows/         # ci.yml, contract.yml, deploy-staging.yml, deploy-prod.yml
├── openapi/
│   └── openapi.yaml           # generated, committed, diffed in CI
├── config/                    # the Django project
│   ├── settings/
│   │   ├── base.py
│   │   ├── dev.py
```

```
│   │   ├── test.py
│   │   ├── staging.py
│   │   └── production.py
│   ├── env.py                # pydantic-settings: typed, fails fast on missing values
│   ├── urls.py               # /api/v1/public, /me, /host, /admin, /webhooks, /_allauth
│   ├── celery.py             # queues, beat schedule (section 5)
│   ├── asgi.py
│   └── wsgi.py
├── bladus/                   # all apps live in one package
│   ├── core/                 # kernel: base model, money, errors, idempotency, clock, pagination
│   ├── geo/
│   ├── config/               # app named "platform_config" in INSTALLED_APPS to avoid clashing with the project folder
│   ├── audit/
│   ├── media/
│   ├── notifications/
│   │   ├── providers/        # sms_local.py, sms_fallback.py, email.py (one interface)
│   │   └── templates/        # en/, fr/, ar/
│   ├── accounts/
│   ├── hosts/
│   ├── catalog/
│   ├── inventory/
│   ├── trips/
│   ├── pricing/
│   ├── ledger/
│   ├── bookings/
│   │   ├── models.py
│   │   ├── services.py
│   │   ├── selectors.py
│   │   ├── apis.py
│   │   ├── permissions.py
│   │   ├── events.py
│   │   ├── tasks.py
│   │   ├── admin.py
│   │   ├── migrations/
│   │   └── tests/
│   │       ├── factories.py
│   │       ├── test_services.py
│   │       ├── test_apis.py
│   │       ├── test_properties.py      # hypothesis
│   │       └── test_concurrency.py     # real PostgreSQL
│   ├── reviews/
│   ├── messaging/
│   ├── trust/
│   └── search/
├── locale/                   # en, fr, ar .po files
├── seeds/                    # wilayas, communes, permissions, taxonomy, settings v1 (production seeds 1 to 8)
├── tests/
│   ├── contract/             # schemathesis
│   ├── performance/          # k6 scripts
│   └── traceability/         # BR rule to test checker
├── ops/
│   ├── caddy/Caddyfile
│   ├── postgres/             # postgresql.conf, pgbackrest.conf, init SQL (extensions, roles, grants)
│   ├── prometheus/           # scrape config, alert rules
│   └── runbooks/             # copies of the runbooks in 09
└── docs/                     # links back to this blueprint
```

---

12. Architecture decisions (summary)

Numbering continues from 05-database-schema.md (ADR-001 to ADR-014).

ADR	Decision	Rejected options	Why
ADR-015	**Modular monolith**: one Django project, 17 apps + kernel, one database	Microservices; one big app without boundaries	Small team, one transaction for booking + inventory + ledger; Airbnb split only at hundreds of engineers (research A.1)
ADR-016	**Services and selectors** (HackSoft style), thin DRF views	Logic in serializers or models ("fat models"); Django Ninja	Testable functions, one place per rule, known to Algerian Django developers (research B.2)
ADR-017	**Transactional outbox** for all side effects	Django signals; calling Celery directly inside the transaction	A rolled-back change sends nothing; a crash after commit still sends (BR-SYS-03)
ADR-018	**Celery + Redis** with 4 queues and one beat	Django built-in tasks (6.x), Dramatiq, Huey	Retries, schedules, monitoring today; thin tasks keep the door open to Django tasks (research B.5)
ADR-019	**BFF with allauth headless session tokens** in encrypted httpOnly cookies	JWT in browser storage; SimpleJWT; Django cookies shared across domains	Tokens never reach JavaScript; revocable server-side (ACC-03); OTP, MFA and rate limits included (research B.3)
ADR-020	**PostgreSQL search** (`listing_search_doc` + PostGIS + trigram)	Meilisearch, Typesense, OpenSearch at launch	One store, consistent with availability; external engine only when measured slow
ADR-021	**Polling** a cursor endpoint for messages (5 to 15 s)	WebSockets via Django Channels or Centrifugo at launch	Simplest to run; Centrifugo later (research B.6)
ADR-022	**S3-compatible storage hosted in Algeria**, presigned uploads, quarantine then process	Uploads through Django; Cloudinary; Cloudflare R2 for personal files	Data residency (BR-PRV-04), no large files in the API process, vendor-neutral
ADR-023	**OpenAPI + openapi-typescript + openapi-fetch** typed client	Hand-written fetch calls; orval; hey-api	Small, works in server components; breaking changes fail compile (NFR-08)
ADR-024	**Module-by-module migration** behind server-only flags with shadow reads	Big-bang switch; keeping mock and API both writable	Each step reversible without a rebuild; risk spread over time
ADR-025	**Layered app dependencies checked by import-linter**	Convention only	Boundaries stay real as the team grows
ADR-026	**READ COMMITTED + constraints + ordered row locks**	SERIALIZABLE everywhere; Redis locks	Constraints are the final guard (ADR-002); fewer retries; Redis locks are not transactional with the data

---

Check yourself

- [ ] Which app owns `onboarding_draft`, and why is it not in `config`?
- [ ] `notifications` is in layer L1. How can it send a "booking confirmed" email without importing `bookings`?
- [ ] Two guests book the last hotel room at the same second. Which step of section 3.1 stops the second one, and what error does it get?
- [ ] Why is no SMS ever sent from inside a database transaction?
- [ ] Which job releases inventory held for an unpaid checkout, and how often does it run?
- [ ] Which data is never cached, and why?
- [ ] What must be true before the website's `reservations` module is switched to the API, and which module switches with it?
- [ ] Where does a KYC document go after upload, and how long is a download link valid?

08. Security and privacy

> **In plain words**
> This page says what we protect (people's identities, ID documents, money records, the platform's reputation), from whom, and how.
> It maps the ten most common API attacks (OWASP API Top 10) to BLADUS endpoints, and defines login, sessions, roles, secrets, encryption, file safety and rate limits.
> It explains how Algerian personal data law (Law 18-07 as amended by Law 25-11) shapes the design: data stays in Algeria, a register of processing, consents, a 5-day breach notice.
> It ends with a checklist to complete before launch and the questions a lawyer must answer.
> Every legal point is marked **VERIFY WITH A PROFESSIONAL**: this is an engineering plan, not legal advice.

Terms used here (see also glossary.md):

- **Threat model**: a structured list of what could go wrong, who could cause it, and what stops it.
- **STRIDE**: six kinds of threat: **S**poofing (pretending to be someone), **T**ampering (changing data), **R**epudiation (denying you did something), **I**nformation disclosure (leaks), **D**enial of service (making the site unusable), **E**levation of privilege (getting rights you should not have).
- **OWASP API Top 10**: the list of the 10 most common API security risks (2023 edition), published by the OWASP foundation.
- **OTP**: one-time password, here a 6-digit code sent by SMS or email.
- **MFA / 2FA**: multi-factor login: a password plus a second proof (an authenticator app code (TOTP) or a passkey).
- **TOTP**: time-based one-time password, the 6-digit code shown by apps such as Google Authenticator.
- **Passkey (WebAuthn)**: login with the phone or laptop's fingerprint or PIN; cannot be phished.
- **Session token**: a random value that proves "this request comes from a signed-in user". Here created by allauth, kept by the Next.js server.
- **Cookie flags**: `HttpOnly` (JavaScript cannot read it), `Secure` (only sent over HTTPS), `SameSite` (not sent with requests coming from other sites).
- **CSRF**: cross-site request forgery: another site makes your browser send a request to ours.
- **Encryption at rest / in transit**: data is unreadable on disk / on the network without a key.
- **Envelope encryption**: data is encrypted with a data key; the data key is itself encrypted with a master key kept elsewhere.
- **HMAC**: a hash mixed with a secret key; the same input always gives the same output, but nobody without the key can compute or reverse it.
- **Hash chain**: each audit row stores the hash of the previous row, so deleting or changing a row breaks the chain visibly.
- **ClamAV**: a free antivirus engine that scans uploaded files.
- **EXIF**: hidden information inside photos (camera, date, **GPS position**).
- **DPIA**: data protection impact assessment, a written study of the risks of a sensitive processing (here: ID documents).
- **Data subject**: the person the data is about (guest, host, staff).
- **Controller / processor**: the controller (BLADUS) decides why data is processed; a processor (hosting company, SMS provider) processes it on BLADUS's instructions.
- **RPO / RTO**: how much data we may lose (recovery point) / how long we may be down (recovery time) after a disaster.

Inputs: 02-research.md sections C.3 and D, 03-requirements.md (ACC, STF, MED, PRV, NFR-05, NFR-11), 04-business-rules.md, 05-database-schema.md ADR-012 and ADR-014, 07-architecture.md.

---

1. What we protect (assets)

Asset	Where it lives	Sensitivity	Why it matters
ID documents and their numbers (KYC)	`host_document`, private bucket	**Very high**	Identity theft; law on personal data; trust of hosts
Payout and refund account numbers (CCP RIP, bank RIB)	`payout_account.number_encrypted`	**Very high**	Redirecting money (account takeover)
Login secrets: passwords, OTPs, session tokens, MFA secrets	`user`, allauth tables, Redis	**Very high**	Full account takeover
Ledger, wallet balances, top-ups, invoices	`ledger_*`, `wallet_topup`, `invoice`	**High**	Money; legal record keeping (10 years, VERIFY)
Exact addresses of homes	`listing` exact point, booking snapshot	**High**	Physical safety of hosts and guests
Contact details (phone, email), names in Arabic and Latin	`user`, `organization`	High	Spam, fraud, harassment
Messages between guests and hosts	`message`	High	Private conversations, contact details
Bookings (who stays where, when)	`booking`	High	Reveals movements and home absence
Audit log	`audit_event`	High	Our proof of who did what
Staff accounts and permissions	`staff_profile`, role tables	High	One staff takeover reaches everyone's data
Platform secrets (keys, database passwords)	secret store, servers	**Very high**	Everything above
Public listings and photos	`listing`, public bucket	Low (public)	Integrity: fake or changed listings harm guests
Availability and prices	inventory tables	Medium (business)	Scraping by competitors, calendar blocking
Reputation and service availability	whole platform	High	Outages and fraud destroy trust

2. Who might attack, and who must be trusted

Actor	Motivation	Typical attempts
Anonymous visitor / scraper	Data, competition	Bulk listing scraping, enumeration of ids, map scraping for exact addresses
Fraudster posing as host	Money	Fake listing with stolen photos, asks for a CCP transfer outside BLADUS, disappears
Fraudster posing as guest	Money, harm	Stolen identity, fake payment receipts sent to hosts, harassment
Account takeover attacker	Money, data	Password reuse, OTP brute force, SIM swap, phishing of staff
SMS pumping fraud	Money (premium SMS revenue share)	Mass OTP requests to number ranges they control
Competing host	Ranking	Fake reviews, calendar blocking with requests never confirmed

Actor	Motivation	Typical attempts
Malicious or careless staff member (insider)	Curiosity, money	Viewing KYC files without need, manual top-up to a friend's wallet, mass export
Compromised dependency or CI	Wide access	Malicious package, stolen deployment secrets
Hosting or infrastructure provider incident	none (accident)	Disk loss, data centre outage, misrouted backups

Trusted components: the Next.js servers (BFF), the Django API, workers, PostgreSQL, Redis, object storage, all in the private network in Algeria (02-containers). Trust boundaries: browser to proxy (internet), proxy to Next.js, Next.js to Django (private network, service authentication), Django to external providers (SMS, email, gateway in R2).

3. Threats by STRIDE

STRIDE	Threat in BLADUS	Main controls	Rules / reqs
Spoofing	Login with stolen password or brute-forced OTP	Argon2 passwords, breached-password check, OTP 6 digits / 10 min / 5 tries, lockout, MFA for staff and payout hosts	BR-ACC-02, 04, 05
Spoofing	Fake webhook claiming a payment (R2)	Signature verified, event re-fetched from the gateway, stored raw, processed once	BR-SYS-02, BR-PAY-08
Spoofing	Request pretending to come from the BFF with a forged client IP	Django trusts `X-Forwarded-For` only from proxy/BFF addresses; BFF authenticates to Django with a service key on the private network	section 4.5
Tampering	Changing a price or commission in the request	Server computes every price (quote); writable fields allowlisted; `status`, `commission_rate_bp` never writable by hosts	BR-PRC-01, API3
Tampering	Editing ledger or audit rows directly in the database	No UPDATE/DELETE grant for the app role; deferred balance trigger; hash chain; nightly verification	BR-PAY-02, BR-STF-02
Tampering	Turning an approved listing into a fake one	Key-field edits go to a revision under review	BR-LST-02, ADR-009
Repudiation	Staff denies refunding or approving	Audit event with actor id, role, IP, request id, before/after; hash chain	BR-STF-02, 03
Repudiation	Host denies accepting a booking	`booking_event` with actor and time, insert-only	BR-BKG-06
Information disclosure	Reading another user's booking (`/bookings/{id}`)	Object-level checks in selectors; UUIDv7 ids; 404 for other people's objects	API1, NFR-12
Information disclosure	Exact address before confirmation	Public point fuzzed by up to 500 m; exact point only for host and confirmed guest	BR-SRC-03
Information disclosure	KYC file leaked by URL sharing	Private bucket; signed URLs valid 5 min; every view audited	BR-MED-02
Information disclosure	GPS position inside photos	Metadata stripped on re-encode	BR-MED-01
Information disclosure	Personal data in logs or error tracking	Log scrubbing of phone, email, tokens, document numbers; Sentry/GlitchTip `send_default_pii = False` and scrubbers	09-devops-hosting.md section 9
Denial of service	Booking requests that block calendars without intent	Request TTL 24 h; per-guest limit of open requests (5); velocity flags	BR-BKG-04, API6
Denial of service	SMS pumping draining budget	Per phone / IP / device limits, country restriction, every send recorded in `otp_send_log` (hashed phone, IP, cost), daily budget with automatic stop	BR-ACC-02, ACC-02, NFR-11
Denial of service	Large uploads, expensive search	Presigned size limits, decompression-bomb limit, page size max 50, statement timeouts	API4
Elevation of privilege	Host calling an admin endpoint	Separate `/api/v1/admin` surface, staff token + MFA, deny by default	STF-05, BR-STF-01
Elevation of privilege	Organisation agent accepting bookings or seeing money	Organisation role permissions via django-rules	BR-ORG-05, DEC-009
Elevation of privilege	Staff member giving themselves a role	`team.manage` only for super admin; role changes audited and notified to all super admins	BR-STF-01

4. Authentication and sessions

4.1 Building blocks

django-allauth in **headless** mode, "app" client (research B.3): the API returns a session token; the Next.js server keeps it. allauth modules used: `account` (email and phone), `mfa` (TOTP, WebAuthn passkeys, recovery codes), `usersessions` (list and revoke sessions). Passwords hashed with **Argon2id**.

4.2 Cookies held by the Next.js servers

Cookie	App	Content	Flags	Lifetime
`__Host-bladus_session`	Website	allauth session token, encrypted (AES-256-GCM) with `BLADUS_SESSION_COOKIE_KEY`, plus active host account id	`HttpOnly; Secure; SameSite=Lax; Path=/`; no `Domain` (the `__Host-` prefix forces this)	30 days sliding (BR-ACC-03)
`__Host-bladus_admin`	Admin	staff session token, encrypted with `ADMIN_SESSION_COOKIE_KEY`	`HttpOnly; Secure; SameSite=Strict; Path=/`	8 h absolute, 30 min idle (BR-ACC-03)
`bladus_locale`	Website	`en`, `fr` or `ar`	`Secure; SameSite=Lax`	1 year

- `SameSite=Lax` plus an **Origin check** on every mutating route handler and server action protects against CSRF on the website; the admin uses `Strict`.
- The Django API itself uses no cookies for the apps (token in header), so classic CSRF does not apply to it. The Django admin console uses Django's own session and CSRF cookies on its own host name.
- These replace the current `darna_session` (website, HMAC-signed, built-in development secret in test mode) and `darna_admin_session` (admin, JWT that cannot be revoked). With test mode OFF the website must refuse to start without its key.

4.3 Sign-in rules

Topic	Rule
Sign-up	Email + password, or Algerian phone + OTP. Verification code before the first booking (DEC-012, BR-ACC-01)
Passwords	Minimum 10 characters; refused if in a local list of the 100,000 most common breached passwords; no composition rules; change revokes all other sessions (ACC-04)
OTP	6 digits, valid 10 min, invalidated after 5 wrong tries, max 3 sends per phone per hour and 10 per IP per hour, same error for every failure (BR-ACC-02). Phone numbers outside Algeria need an extra check (CAPTCHA by a self-hosted provider, or email verification first) to limit SMS pumping
Lockout	5 failures in 15 min lock the account and the IP for 60 s, counters in Redis shared by all servers (BR-ACC-05). Repeated locks increase the wait (60 s, 5 min, 15 min)
Staff MFA	Mandatory before any admin endpoint works (`403 mfa_required`); TOTP or passkey; 10 single-use recovery codes; passkeys recommended for super admins (BR-ACC-04)
Host MFA	Mandatory before adding or changing payout or refund account details (BR-ACC-04, BR-PAY-09)
Step-up	Sensitive actions (change email or phone, payout details, ownership transfer, staff role change, export) need a fresh authentication within the last 10 minutes
Session revocation	Logout, password change, staff deactivation, ban: tokens deleted server-side immediately; the next API call returns `401` (ACC-03, STF-02)
New device	Email or SMS notice on sign-in from a new device for hosts with a wallet and for staff
Staff invitation	One-time link valid 48 h; staff sets password, then must enrol MFA before the first admin page (STF-02)

4.4 Account recovery and SIM swap

Phone-number recovery alone is not enough for hosts with payout details or for staff: they also need the email link or an MFA recovery code. After a phone or email change, payout changes are blocked for 72 h and the old contact is notified (BR-PAY-09, research D.2).

4.5 Service-to-service authentication

- Next.js to Django: private network only, plus a header `X-BFF-Key` with a per-app secret (`BFF_SERVICE_KEY_WEBSITE` , `BFF_SERVICE_KEY_ADMIN`) checked by a Django middleware. A request to `/api/v1/admin` with the website key is refused even with a staff token.
- Django to PostgreSQL and Redis: TLS on the private network, separate database roles (section 7.3).
- Workers to providers (SMS, email): API keys from the secret store; outbound traffic through an egress proxy allowlist (SSRF protection, API7).

---

5. Authorisation

5.1 Three surfaces, deny by default

1. **Public and guest** (`/public` , `/me`): object ownership checked in selectors (`bookings_for_guest(user)`); other people's objects return **404**, not 403, so ids cannot be probed.
2. **Host** (`/host`): every request names `X-Host-Account` ; a permission class checks the user holds it (individual owner or active membership) and the organisation role permission (table 5.3). Lists are always filtered by that host account.
3. **Staff** (`/admin`): staff token, MFA done, permission from the staff role (table 5.2). Field-level: personal data shown only to roles that need it (for example analysts see counts, not phone numbers).

Output fields per role are separate serializers (no "one serializer with hidden fields"). Writable fields are an explicit allowlist per endpoint (mass-assignment protection, API3).

5.2 Staff roles and permissions

34 permissions stored in `staff_permission` , linked to the 5 roles in `staff_role_permission` (seeded from today's `admin/lib/permissions.ts` , BR-STF-01). X = granted.

Permission	super_admin	moderator	support	finance	analyst
hosts.view	X	X	X	X	X
hosts.approve, hosts.reject, hosts.suspend	X	X			
hosts.delete	X				
hosts.commission_override	X			X	
listings.view	X	X	X		X
listings.approve, listings.reject, listings.suspend, listings.feature	X	X			
listings.delete	X				
users.view	X	X	X		X
users.suspend	X	X	X		
users.ban, users.delete	X				
bookings.view	X		X	X	X
bookings.cancel, bookings.dispute	X		X		
bookings.refund	X		X	X	
reviews.view	X	X	X		X
reviews.moderate	X	X			
reviews.delete	X				
wallets.view	X			X	X

Permission	super_admin	moderator	support	finance	analyst
wallets.recharge, commissions.manage, payouts.approve	X			X	
onboarding.edit, team.manage, settings.manage	X				
categories.manage, promotions.manage	X	X			
reports.export	X	X		X	X
audit.view	X	X	X	X	X

Additions for the backend (new permissions, added to the seed): `kyc.view_documents` (moderator, super_admin; every view audited), `privacy.handle_requests` (super_admin and a named privacy officer), `wallets.approve_large_topup` (finance, must be a different user from the creator, BR-PAY-07). Proposed new role later: `privacy_officer`.

Separation of duties: the person who creates a manual top-up cannot approve it; a super admin cannot remove the last other super admin; role changes are notified to every super admin.

5.3 Organisation roles

From BR-ORG-05, table 2a in 04-business-rules.md:

Permission	owner	admin	manager	agent	guide
org.edit, members.manage	X	X			
org.delete, ownership transfer	X				
listings.manage, trips.manage, calendar.manage	X	X	X		
bookings.view	X	X	X	X	own departures
bookings.respond (accept, decline)	X	X	X		
bookings.cancel_as_host	X	X			
messages.reply	X	X	X	X	X
wallet.view, invoices.view	X	X			
payout_details.edit (with MFA)	X				

Individual hosts hold every permission on their own host account.

6. OWASP API Top 10 (2023) mapped to BLADUS endpoints

Risk	Endpoints most at risk	Controls	How we test
API1 Broken object level authorisation	`GET/PATCH /me/bookings/{id}`, `/host/bookings/{id}`, `/me/conversations/{id}`, `/host/documents/{id}`, `/admin/hosts/{id}/documents/{doc}`	Scoped selectors; UUIDv7; 404 for foreign objects; tests per endpoint with a second user	`test_api1_*` per app: user B cannot read user A's object
API2 Broken authentication	`/_allauth/app/v1/auth/login`, `/auth/code/request`, `/auth/password/reset`	Section 4; lockout; same error message for all failures	Brute-force test hits lock at try 6
API3 Broken object property level authorisation	`PATCH /host/listings/{id}` (status, commission), `GET /public/listings/{id}` (exact address, host phone)	Explicit writable fields; output serializers per role; host phone hidden before confirmation	Schemathesis + tests that `status` in PATCH body is ignored or refused
API4 Unrestricted resource consumption	`/public/search`, `/public/search/map`, `/host/media/uploads`, `/auth/code/request`	Page size max 50, 500 map pins, upload size in presigned policy, OTP and SMS budget, statement timeout 5 s	k6 abuse scenarios; limits in unit tests
API5 Broken function level authorisation	`/admin/*` (refund, top-up, ban), `/host/organizations/{id}/members`	Separate surface; staff permission per view; org role per view; deny by default	Matrix test: every admin endpoint x every role
API6 Unrestricted access to sensitive business flows	`POST /me/bookings` (calendar blocking), `/public/quotes`, reviews, invitations, coupons (R2)	Max 5 open requests per guest; request TTL; reviews only after completed booking; coupon per-user limit; velocity rules	Business-flow tests; risk flags (R2)
API7 Server-side request forgery	iCal import feeds (`calendar_feed`, Later, AVL-06), image-by-URL (not offered), webhook callbacks	No URL fetch at MVP. For `calendar_feed` imports: URL stored encrypted (`url_encrypted`), fetched only by a worker through an egress proxy that blocks private and link-local IP ranges after DNS resolution, allows only http(s) on 80/443, no redirects to other hosts, 2 MB and 10 s limits; export feeds use a secret token stored only as a hash (`export_token_hash`)	SSRF test list (169.254.169.254, 10.0.0.0/8, 127.0.0.1, `[::1]`, DNS names resolving to private IPs, redirect to localhost)
API8 Security misconfiguration	Whole API, Django admin console, Redis, storage buckets	Hardening checklist (section 13); `DEBUG = False`; CORS closed (BFF only); security headers; Redis not public; buckets private by default	Automated config checks (`manage.py check --deploy`), external scan before launch
API9 Improper inventory management	Old `/api/v1` endpoints, staging with real data, forgotten debug routes	OpenAPI is the inventory; CI diff on every change; staging uses synthetic data only; old versions removed on a schedule	OpenAPI diff gate; route list compared with spec
API10 Unsafe consumption of APIs	Payment gateway (R2), SMS and email providers, Nominatim	Webhook signature + server-side status re-check; timeouts; validate provider responses; never trust redirect parameters	Contract tests with recorded provider responses

7. Secrets, keys and encryption

7.1 Secrets management

Rule	Detail
Where secrets live	Production and staging secrets in an encrypted file per environment (SOPS with age keys) in a private operations repository; decrypted only on the servers at deploy time into files readable by the service user. Move to HashiCorp Vault or OpenBao when the team grows
Never	In the application repository, Docker images, `NEXT_PUBLIC_` variables, logs, error reports, tickets or chat
CI	GitHub Actions environment secrets with required reviewers for `production`; CI never holds database or encryption keys, only deploy credentials
Access	Two named people hold production key access; each access to the master encryption key is logged
Rotation	See table 7.2. Rotation is rehearsed in staging first
Leak response	Rotate, revoke, search logs for use, record in the incident register

7.2 Key and secret inventory

Secret	Used for	Rotation
`DJANGO_SECRET_KEY`	Django signing (password reset links, Django admin console)	Yearly, or on leak (supports a fallback list)
`BLADUS_SESSION_COOKIE_KEY`, `ADMIN_SESSION_COOKIE_KEY`	Encrypt BFF cookies	Yearly (two keys accepted during rotation)
`BFF_SERVICE_KEY_WEBSITE`, `BFF_SERVICE_KEY_ADMIN`	Next.js to Django service authentication	Every 6 months
`FIELD_ENCRYPTION_KEYS` (master key, versioned)	Envelope encryption of payout numbers	Yearly; old versions kept to decrypt, data re-wrapped by a job
`ID_HASH_PEPPER`	HMAC of document numbers and recipient hashes	Never rotated casually (hashes would change); rotation needs a planned re-hash from the source documents, impossible after images are deleted, so guard it carefully
Database passwords (per role)	PostgreSQL	Every 6 months
Redis password	Redis AUTH	Every 6 months
Storage access keys	Object storage	Every 6 months
SMS, email, gateway API keys and webhook secrets	Providers	Yearly or per provider policy
Backup encryption key (age)	Encrypt dumps and archives	Yearly; private key stored offline by two people

7.3 Encryption

Layer	How
In transit, public	TLS 1.2 and 1.3 only, modern ciphers, HSTS `max-age=63072000; includeSubDomains; preload` after a trial period, certificates by ACME (Let's Encrypt) through Caddy
In transit, private	PostgreSQL `sslmode=verify-full`; Redis with TLS and AUTH; object storage over HTTPS even inside the network
At rest, disks	Full-disk encryption (LUKS) on database, storage and backup volumes where the provider allows it; otherwise encrypted volumes on top of provider disks (VERIFY with ICOSNET / Djezzy Cloud what is offered)
At rest, backups	Every dump and WAL archive encrypted with age before leaving the server (see 09-devops-hosting.md)
At rest, object storage	Server-side encryption on private and quarantine buckets
Field level: payout numbers	AES-256-GCM with a random data key per row; data key wrapped by the master key from `FIELD_ENCRYPTION_KEYS`; stored as `key_version + wrapped_key + nonce + ciphertext` in `payout_account.number_encrypted`; only `number_last4` in clear (ADR-014). The master key is never in the database or in backups
Field level: ID numbers	Never stored in clear. Normalised (digits only, upper case), then `HMAC-SHA256(ID_HASH_PEPPER, type + ":" + number)` in `host_document.number_hash`, used to detect the same ID on two accounts (research D.2)
Recipients in logs	`delivery_log.recipient_hash` is an HMAC of the phone or email, never the clear value
Database roles	`bladus_migrate` (owner, used only by migrations), `bladus_app` (no UPDATE/DELETE on ledger, audit, `booking_event`), `bladus_readonly` (reports, replicas), `bladus_backup`

8. File security

Control	Detail	Rule
Direct upload with presigned PUT	Valid 5 minutes, exact `Content-Type` and `Content-Length` in the signature, into the **quarantine** bucket only	MED-01
Real type check	Magic bytes, not file name or browser type; SVG, HTML and Office files refused	BR-MED-01, 02
Decompression bombs	Pillow `MAX_IMAGE_PIXELS = 50,000,000`; worker memory and time limits	API4
Metadata	Photos re-encoded; EXIF, XMP, IPTC (including GPS) removed	BR-MED-01
Virus scan	ClamAV daemon (signatures updated hourly) for every document and message attachment; infected files deleted and the event audited	BR-MED-02
PDF safety	Refuse PDFs with JavaScript, launch actions or embedded files	BR-MED-02
Private bucket	No public listing; downloads by signed GET valid 5 minutes, `Content-Disposition: attachment`; each view writes `audit_event` (`KYC_DOCUMENT_VIEW`)	BR-MED-02
Naming	Random UUID keys; original file name kept only as metadata, sanitised	
Retention	ID images deleted 90 days after the verification decision (VERIFY)	BR-PRV-05

9. Rate limits

Two layers: coarse limits per IP at the proxy (Caddy) to stop floods, fine limits per user / IP / phone in Django (DRF throttles and allauth limits) stored in Redis db 2. Responses use `429` with `Retry-After`.

Scope	Limit	Layer
Any request per IP	30 per second burst, 600 per minute	Proxy
Login attempts	5 failures in 15 min per account and per IP, then lock 60 s (growing)	allauth + Redis (BR-ACC-05)
OTP send	3 per phone per hour, 10 per IP per hour, 20 per device id per day; every attempt (sent, failed, `blocked_rate_limit`) written to `otp_send_log`	allauth + Redis (BR-ACC-02, ACC-02)
OTP verify	5 tries per code	allauth
Password reset request	3 per account per hour, 10 per IP per hour	allauth
Search and public reads	120 per minute per IP, 300 per minute per signed-in user	DRF
Quote	60 per minute per user or IP	DRF
Booking create	10 per hour per guest; max 5 open requests at once	DRF + service (API6)
Messages	30 per minute per user; 200 per day to new conversations	DRF
Message polling	1 per 3 seconds per user per conversation	DRF
Upload slots	60 per hour per host account	DRF
Reviews and reports	10 reports per day per user	DRF
Admin API	600 per minute per staff user; exports 10 per hour	DRF
SMS budget	daily and monthly caps in settings, spend summed from `otp_send_log.cost_minor` and `delivery_log.cost_minor`; alert at 80%, stop non-critical SMS at 100%; abnormal volume per phone prefix or IP range raises a pumping alert	job `notifications.sms_budget_check` (NFR-11)

10. Audit log with hash chain

- **What is logged** (STF-03): every staff write; every host money action (top-up, commission override, payout details change); team changes (invite, role change, removal, ownership transfer); logins, failed logins, MFA changes and session revocations; every KYC file view; exports (filters and row count); settings and onboarding publishes; privacy requests and their handling.
- **Row content**: `occurred_at`, `actor_id`, `actor_kind`, `actor_role`, `action`, `object_type`, `object_id`, `host_account_id`, `before`, `after`, `ip`, `user_agent`, `request_id`, `prev_hash`, `hash` (schema.dbml). `before` and `after` keep only changed fields and never contain secrets, full document numbers or message bodies.
- **Hash**: `hash = SHA-256(prev_hash || canonical_json(row without hash))`, where canonical JSON has sorted keys and UTC timestamps. Rows are appended through one service which takes a short advisory lock so the chain has a single order.
- **Protection**: insert-only for `bladus_app` (no UPDATE, DELETE, TRUNCATE); monthly partitions; the job `audit.verify_hash_chain` recomputes yesterday's chain each night and stores the day's final hash as an **anchor** in a separate place (the backup bucket in the second data centre and an email to two named people). A break raises a critical alert.
- **Access**: `audit.view` for reading; nobody can edit. Exports of the audit log are themselves audited.
- **Retention**: 5 years for staff and security events, 10 years where linked to accounting (VERIFY).

`pghistory` row history (bookings, listings, memberships, settings) complements the audit log for "what did this row look like before" questions (ADR-012).

11. Fraud and trust controls

Pattern (research D.2)	Signals	Controls	Rules
Off-platform payment push (CCP/RIP numbers, BaridiMob, WhatsApp links before confirmation)	Regex and number patterns in messages	Masked before confirmation; repeated attempts flag the thread; guest warning banner	BR-MSG-01, MSG-04
Fake listing with stolen photos	Perceptual hash match, reverse-image hits, new host with many listings	Review before publishing; proof of property for the first listing; trust case on duplicates	BR-LST-02, BR-MED-01, BR-HST-03
Duplicate accounts after a ban	Same phone, same device id, same ID number hash	Block sign-up or verification with a matching hash; case for staff	BR-ACC-01, ADR-014
Account takeover via SIM swap	Phone change then payout change; new device; new country	Re-verification; MFA; 72 h hold on payout changes; notices to old contacts	BR-PAY-09
Fake wallet top-up receipt	Same receipt reference twice; large amounts	Unique receipt per method; four-eyes above 500,000 DZD	BR-PAY-07
Calendar blocking by fake requests	Many open requests, no confirmations	24 h TTL; max open requests per guest; flags	BR-BKG-04, API6
Fake reviews and review rings	Reviews only from completed bookings; bursts; same IP/device	Review eligibility in the database; burst and ring detection (R2)	BR-REV-01, BR-MOD-04
SMS pumping	Many OTP requests to sequential numbers or foreign ranges, visible in `otp_send_log` (by phone hash and IP)	Limits per phone/IP/device; foreign numbers need extra checks; budget stop	BR-ACC-02, ACC-02, NFR-11
Staff insider abuse	Unusual KYC views, exports, top-ups	Least privilege; four-eyes; every view audited; weekly review of KYC views and exports by a super admin	BR-STF-01, 02, 04
Host with many cancellations	3 host cancellations in 12 months	Moderation case	BR-CAN-03

Principle: risk rules add friction (extra check, case for staff), they never silently block (BR-MOD-04, Airbnb "targeted friction", research A.1).

12. Privacy by design (Law 18-07 as amended by Law 25-11)

All of this section: **VERIFY WITH A PROFESSIONAL.** Status of the law text: research C.3 rates it [secondary].

12.1 Roles and formalities

Topic	Plan
Controller	BLADUS (the legal entity operating the platform)
Processors	Hosting provider (ICOSNET or Djezzy Cloud), backup site, SMS aggregator, email provider, later payment gateway. Each needs a written contract with data protection clauses
ANPDP formalities	File the prior declaration or request authorisation for each processing in the register (12.2) **before launch**; KYC and any transfer abroad likely need authorisation (L2)
Data protection contact	Name a person responsible for personal data (and a DPO if required, L7)
Privacy notice	In en, fr, ar; versioned; acceptance stored in `consent_record` (PRV-01)

12.2 Register of processing (first version)

Processing	Data	People	Purpose	Legal basis (to confirm)	Retention	Recipients	Location
Accounts and login	Name (ar, latin), email, phone, password hash, sessions, IP	Guests, hosts, staff	Provide the service, security	Contract	Account life + 3 years after closure (VERIFY)	BLADUS staff by role	Algeria
Host verification (KYC)	ID images, document numbers (hashed), RC, NIF, NIS, licences, proof of property	Hosts, organisation representatives	Fraud prevention, legal checks on hosts	Legal obligation / legitimate interest (VERIFY)	Images 90 days after decision; decision and hash for account life + 5 years	Moderators with `kyc.view_documents`	Algeria
Bookings	Guest identity, dates, guests count, amounts, address revealed	Guests, hosts	Contract execution, Law 18-05 transaction records	Contract, legal obligation	10 years (accounting, VERIFY)	Host of the booking, support, finance	Algeria
Commission wallet and invoices	Host account, amounts, receipts, payout account (encrypted)	Hosts	Accounting, commission	Contract, legal obligation	10 years	Finance	Algeria
Messages	Message text (masked view), attachments	Guests, hosts	Communication, dispute handling, fraud prevention	Contract, legitimate interest	3 years after last activity	Participants; support on a case	Algeria
Reviews	Ratings, comments, first name	Guests, hosts	Trust	Contract / legitimate interest	Account life; anonymised on deletion	Public	Algeria
Notifications	Phone, email, language	All users	Service messages	Contract	Delivery log 1 year (hashed recipient)	SMS and email processors	Algeria; foreign providers only after authorisation
Marketing	Email, phone, opt-in record	Opted-in users	Promotions	Consent (Law 18-05 art. 31)	Until withdrawal	Email provider	As above
Audit and security logs	Actor id, IP, user agent, actions	Staff, users	Security, accountability	Legitimate interest / legal obligation	5 to 10 years (see 10)	Super admins	Algeria
Foreign guest declaration support (R2)	Nationality, ID type	Foreign guests	Help hosts with Law 08-11 art. 29 duty	Legal obligation (VERIFY, L6)	As required by law	Host	Algeria

12.3 Consents

- Stored in `consent_record` with document (`terms`, `privacy`, `marketing`), version, time, IP (BR-PRV-01).
- Marketing is opt-in, never pre-ticked, withdrawable in one click from every message (NTF-03, BR-PRV-03). Critical service messages cannot be switched off.
- A new version of the terms or privacy notice asks for acceptance again at the next sign-in.

12.4 DPIA for KYC (outline)

1. Description: which documents per host type (table 3a in 04-business-rules.md), who sees them, how long.
2. Necessity: why each document is needed (L4); can a check be done without keeping the image (keep decision + hash)?
3. Risks: leak of ID images, insider viewing, identity theft, over-retention.
4. Measures: private bucket, scan, 5-minute signed views, every view audited, weekly review, 90-day deletion, HMAC instead of numbers, encryption at rest, data in Algeria.
5. Residual risk and sign-off by the responsible person; review yearly or on change.

12.5 Data subject requests

Request	How	Deadline (VERIFY)
Access	Self-service JSON export from the profile (`/me/privacy/export`), built by a job, link valid 24 h	Target 30 days, usually same day
Correction	Profile edit; name changes on verified hosts go through re-verification	30 days
Deletion	`data_subject_request` (kind `deletion`), identity re-confirmed, 14-day cooling-off, then anonymisation job: names, contacts, photos, ID files erased; bookings, ledger, invoices kept with a pseudonymous id for the legal period (BR-PRV-02)	30 days
Objection to marketing	One click, immediate	Immediate

Request	How	Deadline (VERIFY)
Requests by email or letter	Logged by staff with `privacy.handle_requests` in the same table, `due_at` tracked on the admin dashboard	as above

Blocking cases: open bookings or disputes, negative wallet balance; the user is told what remains and why.

12.6 Retention table (proposal, VERIFY WITH A PROFESSIONAL)

Data	Keep	Then	Job
ID document images	90 days after the verification decision	Delete file; keep decision, type, number hash	`accounts.retention_sweep`
Rejected applicant documents	90 days after rejection	Delete	same
Booking records	10 years after check-out	Anonymise guest identity	same
Ledger, top-ups, invoices	10 years after the financial year	Archive offline	manual yearly
Messages	3 years after last activity	Delete bodies, keep metadata 1 more year	same
Notifications	1 year	Delete	same
Delivery log	1 year	Delete	same
Quotes not booked	24 hours after expiry	Delete	`pricing.purge_expired_quotes`
Idempotency records	24 hours	Delete	`core.purge_idempotency_records`
OTP send log (`otp_send_log` , hashed phone, IP)	90 days	Delete	`accounts.retention_sweep`
Incident register (`incident_record`)	5 years after closure (VERIFY)	Archive offline	yearly
Application logs	90 days hot, 1 year archived	Delete	log rotation
Audit events	5 years (10 if money-related)	Archive offline	yearly
Backups	PITR 7 days, nightly 35 days, monthly 12 months	Delete	backup tool
Deleted accounts	Anonymised after 30 days	Pseudonymous id only	`accounts.process_deletion_requests`

12.7 Personal data breach runbook (5-day clock)

Law 25-11 requires notifying ANPDP within **5 days** (research C.3, VERIFY the exact start of the clock and the content of the notice).

Time	Action	Who
Hour 0	Anyone who suspects a breach reports it in the incident channel; the on-call engineer creates an `incident_record` row (`detected_at` = time of discovery = start of the clock; `personal_data_involved` ; the system sets `anpdp_deadline_at = detected_at + 5 days`)	Anyone, on-call
Hour 0 to 4	Contain: revoke keys or sessions, block accounts or IPs, take affected systems offline if needed; preserve logs and evidence (no wiping)	On-call + tech lead
Hour 4 to 24	Assess: which data, how many people, which categories (KYC, contacts, money), whether data left the system; severity (low / high)	Tech lead + data protection contact
Day 1 to 2	Decide on notification with the lawyer; draft the ANPDP notice (nature of the breach, categories and number of people and records, likely consequences, measures taken, contact person)	Data protection contact + lawyer
Day 2 to 4	If high risk to people: prepare messages to affected users in en, fr, ar (what happened, what to do, contact)	Data protection contact
Before day 5	Send the notice to ANPDP; set `anpdp_notified_at` and attach the sending proof	Data protection contact
Day 5+	Inform users if required (set `users_notified_at`); follow up; root cause analysis; fixes with deadlines; update this runbook; set `closed_at`	Team

The incident register is the `incident_record` table (staff-only, every change audited in `audit_event`). It keeps every incident, even those not notified to ANPDP, with the reasoning in `summary` and `actions_taken` . The hourly job `audit.incident_deadline_watch` reminds the owner and super admins at day 2, day 4 and 12 hours before `anpdp_deadline_at` while `anpdp_notified_at` is empty.

12.8 Data residency

Data	Location	Note
PostgreSQL, Redis, private and quarantine buckets, backups, logs, error tracking	**Algeria only** (primary and second data centre)	BR-PRV-04, PRV-03
Public listing photos and static JS/CSS	Algeria by default; a foreign CDN only for static files and photo renditions **without people's faces or personal data**, after legal advice	VERIFY (L1, L2)
Error tracking	Self-hosted Sentry-compatible tool (GlitchTip) in Algeria, PII scrubbing on; Sentry SaaS would send data abroad	L10
SMS and email providers	Prefer Algerian providers. A foreign provider (Twilio, Resend) receives phone numbers and email addresses: that is a **transfer abroad** that may need ANPDP authorisation	VERIFY (L2, L10)
CI (GitHub Actions)	Source code and synthetic test data only; never production data or secrets for data stores	
Staging	Synthetic data only, so it may run anywhere; we still keep it in Algeria for parity	

13. Pre-launch security checklist

- [ ] `NEXT_PUBLIC_BLADUS_TEST_MODE=0` in the website production build; the build fails otherwise; demo accounts absent; no development secrets anywhere.
- [ ] All `DARNA_SESSION_SECRET` / `ADMIN_AUTH_SECRET` mock auth code removed after the auth module migration.
- [ ] `manage.py check --deploy` clean; `DEBUG = False`; `ALLOWED_HOSTS` exact; `SECURE_PROXY_SSL_HEADER` set correctly; security headers (HSTS, CSP, `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`, `frame-ancestors 'none'`) on both Next.js apps and Django admin console.
- [ ] TLS A grade on an external test; HTTP redirects to HTTPS.
- [ ] Django API not reachable from the internet except the webhook path; Redis, PostgreSQL, storage admin ports closed from outside (external port scan).
- [ ] Database roles and grants applied; `bladus_app` cannot UPDATE or DELETE ledger, audit, `booking_event` (test).
- [ ] Every staff account has MFA; super admins use passkeys; recovery codes stored offline.
- [ ] Admin app reachable only through allowlisted IPs or a VPN, in addition to MFA.
- [ ] Rate limits active and tested (section 9); SMS budget caps configured.
- [ ] Secrets in the secret store only; repository scanned for secrets (gitleaks) including history.
- [ ] Dependency scans clean or accepted (pip-audit, npm audit, image scan with Trivy).
- [ ] Field encryption key and ID pepper generated, stored, backed up offline by two people; restore of an encrypted field tested.
- [ ] Backups encrypted, in the second Algerian data centre; restore drill done once (see 09-devops-hosting.md).
- [ ] Audit hash chain verification job running; anchor delivered.
- [ ] ClamAV running with fresh signatures; EXIF strip verified on a phone photo with GPS.
- [ ] Log and error scrubbing verified (no phone, email, token, document number in logs).
- [ ] OWASP API Top 10 tests green (section 6); external penetration test done and high findings fixed.
- [ ] Privacy notice and terms published in 3 languages; consent capture working.
- [ ] ANPDP declarations or authorisations filed (VERIFY); processor contracts signed.
- [ ] Breach runbook printed, contacts current, one tabletop exercise done (including creating a test `incident_record` and seeing the deadline reminders).
- [ ] Incident on-call rota defined.

14. Items needing a lawyer (VERIFY WITH A PROFESSIONAL)

#	Question	Linked
L1	Does Law 18-05 art. 8 (hosting in Algeria, `.com.dz`) apply to BLADUS, to the API, to static files and to a CDN?	12.8, 09
L2	Which ANPDP declarations or authorisations must be filed before launch?	12.1, 12.8
L3	Is the prepaid commission deposit (M1) outside payment-service licensing (Règlement 25-02)?	DEC-001
L4	Which KYC documents are necessary and proportionate per host type?	12.4
L6	Foreign guest declaration duty (Law 08-11 art. 29): what must BLADUS collect or remind?	12.2
L7	Retention periods (ID images, bookings, messages, invoices, logs, OTP send log, incident register); is a DPO mandatory?	12.6
L8	Terms of use and privacy notice texts in ar, fr, en, versioned for consent records	12.1, 12.3
L10	Do foreign providers touching personal data (SMS fallback, email, error tracking, uptime monitoring) count as a transfer abroad?	12.8
L11	Company formalities, including the `.com.dz` domain holder	09 section 4
L13 (new)	Exact content, addressee and start of the 5-day breach notification clock; duty to inform users	12.7
L14 (new)	Legal basis for each processing in the register (contract, consent, legal obligation, legitimate interest as recognised by Law 18-07)	12.2
L15 (new)	Rights deadlines (access, correction, deletion) and identity checks allowed before answering	12.5
L16 (new)	Is publishing a reviewer's first name and the host's name allowed without separate consent?	12.2

L1 to L12 are tracked in 13-decisions-needed.md. New items L13 to L16 should be added there at its next update.

Check yourself

- [ ] Name the three most sensitive assets and where each is stored.
- [ ] Why does the website cookie use the `__Host-` prefix, and what does `HttpOnly` prevent?
- [ ] A host with payout details changes their phone number. What happens to payout changes for the next 72 hours, and why?
- [ ] Which OWASP API risk covers "guest reads another guest's booking", and what does the API answer?
- [ ] How is an ID card number stored, and why can it still detect a banned person signing up again?
- [ ] What makes a deleted audit row visible?
- [ ] A laptop with a database dump is stolen on day 0. List what must happen before day 5.
- [ ] Why might a foreign SMS provider need ANPDP authorisation?

09. DevOps and hosting: where BLADUS runs and how we operate it

> **In plain words**
> BLADUS runs in three places: your laptop (dev), a test copy with fake data (staging), and the real site (production).
> Production and every copy of personal data live in Algerian data centres (ICOSNET or Djezzy Cloud), with backups in a second Algerian site. This follows our reading of Law 18-05 art. 8 and Law 18-07/25-11: **VERIFY WITH A PROFESSIONAL**.
> We start simple: a few servers running Docker Compose. Every code change is checked automatically on GitHub (tests, types, API contract) before it can be deployed.
> We can lose at most 15 minutes of data (RPO) and be back within 4 hours (RTO); a restore is practised every month.
> This page also gives a rough monthly cost, step-by-step runbooks (deploy, rollback, incident, restore), and the list of settings each program needs.

Terms used here (see also glossary.md):

- **Environment**: one complete copy of the system (dev, staging, production) with its own data and secrets.
- **Container / image**: an image is a packaged program with everything it needs; a container is a running copy of it. **Docker Compose** starts several containers from one file.
- **Reverse proxy**: the front door server (Caddy) that receives HTTPS traffic and passes it to the right program.
- **TLS certificate**: the file that makes `https://` work and proves the site's identity.
- **CDN**: content delivery network, servers in many places that keep copies of static files close to users.
- **CI/CD**: continuous integration (check every change automatically) and continuous delivery (deploy automatically after checks).
- **GitHub Actions**: GitHub's service that runs these checks and deployments.
- **Migration**: a versioned script that changes the database structure.
- **WAL / PITR**: PostgreSQL writes every change first into a log (write-ahead log). Keeping that log lets us restore the database to any minute in the past: point-in-time recovery.
- **Replica**: a second database server that continuously copies the main one.
- **RPO / RTO**: recovery point objective (how much recent data we accept to lose) / recovery time objective (how long we accept to be down).
- **Observability**: logs (what happened), metrics (numbers over time), traces and error reports, so we see problems before users report them.
- **Health check**: a small URL that says "I am alive" (liveness) or "I can serve requests" (readiness).
- **Runbook**: a written step-by-step procedure for an operation or an emergency.
- **Environment variable (env var)**: a named setting given to a program when it starts (for example `DATABASE_URL`).

Inputs: 02-research.md sections B.16, C.3, C.6; 03-requirements.md NFR-01 to NFR-13; 05-database-schema.md section 7; 07-architecture.md; 08-security-privacy.md; diagram 02-containers.

---

1. Environments

Environment	Purpose	Where	Data	Deploys	Access
dev	Developer laptops	Local Docker Compose	Factories and demo seed only	Manual	Developer
ci	Automatic checks on every pull request	GitHub Actions runners (temporary)	Synthetic test data created by tests	Every push	Automatic
staging	Final check before production; demos; shadow-read migration of the apps (07-architecture.md section 10.4)	One server in Algeria (same provider as production, for parity)	Synthetic data from the demo seed; **never a copy of production**	Automatic on merge to `main`	Team, testers, product owner
production	Real users	Algerian data centre A (primary) + data centre B (backups, replica)	Real personal data	On a release tag, after manual approval	Two named operators; staff use the admin app only

Rules: staging has its own secrets, own SMS sandbox (or a capped real account), and its email sends only to an allowlist of team addresses. Production data is never copied to dev, ci or staging (OWASP API9). For a bug that needs production data, a person with production access investigates inside production.

2. Hosting plan (Algeria first)

All legal points: **VERIFY WITH A PROFESSIONAL** (L1, L2, L10 in 13-decisions-needed.md).

Site	Provider (candidates)	What runs there	Why
Data centre A (primary)	ICOSNET Algiers (recommended in research C.6), or Djezzy Cloud	Everything in production: proxy, Next.js apps, Django API, workers, PostgreSQL primary, Redis, object storage, ClamAV, monitoring	Law 18-05 art. 8 points to hosting in Algeria; Law 18-07/25-11 restricts personal data transfers abroad
Data centre B (second site)	ICOSNET Oran if A is ICOSNET Algiers; or Djezzy Cloud Béjaïa; ideally a different provider from A	PostgreSQL streaming replica (warm standby), pgBackRest repository (PITR archive), nightly encrypted dumps, object storage replica of private and public buckets, audit anchors	Survive the loss of site A; backups must also stay in Algeria (BR-PRV-04)
Staging	Same provider as A, separate account or project	All-in-one server	Parity of network, OS and latency
Outside Algeria (optional, flagged)	Cloudflare or a European CDN; Hetzner for build caches	Only non-personal assets: Next.js static JS/CSS, fonts, maybe public photo renditions without people. **No API, no HTML pages with personal data, no backups**	Faster static files; only after the lawyer answers L1 and L2. **VERIFY**
GitHub	GitHub (USA)	Source code, CI on synthetic data, container images (no data, no secrets)	Standard tooling; no personal data involved

Questions to ask the Algerian providers before signing (record answers in the ops repository): available VPS sizes and prices; dedicated servers; private network between servers; disk encryption; snapshots; S3-compatible object storage offer (if none, we run MinIO or Garage ourselves); bandwidth limits and international transit quality; IPv6; DDoS protection; SLA; support hours; site B availability with the same provider; data processing contract with data protection clauses.

3. Container layout (Docker Compose to start)

One Docker image for the backend (`bladus-api`) used by the API, workers and beat with different commands; one image per Next.js app. Kubernetes is **not** needed at launch; revisit when we run more than about 6 servers or need automatic scaling.

Container	Image / program	Command or role	Server (launch)	Replicas
`caddy`	Caddy 2	TLS, HTTP/2 and HTTP/3, reverse proxy, coarse rate limits, security headers, access logs	app	1
`website`	Next.js (frontend)	`next start`; BFF for guests and hosts	app	2 (rolling deploys)
`admin`	Next.js (admin)	`next start`; staff BFF; reachable only from allowlisted IPs or VPN	app	1
`api`	`bladus-api`	`gunicorn config.wsgi` with sync workers (2 x CPU + 1), 30 s timeout	app	2
`worker-critical`	`bladus-api`	`celery worker -Q critical,default` (OTP, booking notifications, outbox)	worker	1 (concurrency 8)
`worker-media`	`bladus-api`	`celery worker -Q media` (images, documents), memory limit 2 GB	worker	1 (concurrency 2)
`worker-reports`	`bladus-api`	`celery worker -Q reports` (exports, invoices)	worker	1 (concurrency 1)
`beat`	`bladus-api`	`celery beat`; **exactly one** in the whole system	worker	1
`clamav`	ClamAV daemon	`clamd` + `freshclam` hourly	worker	1
`postgres`	PostgreSQL 17 + PostGIS 3.5	Primary database, `pg_stat_statements` on	db	1
`pgbouncer`	PgBouncer	Connection pooling, transaction mode	db	1
`pgbackrest`	pgBackRest	WAL archiving and backups to site B	db	1
`postgres-replica`	PostgreSQL 17 + PostGIS	Streaming replica, read-only reports	site B	1
`redis`	Redis 7	db 0 broker, db 1 cache, db 2 limits and OTP; AOF on; password; TLS	app (move to its own server when needed)	1
`minio` (or `garage`)	S3-compatible storage	Buckets `public`, `private`, `quarantine`, `exports`; only if the provider has no S3 offer in Algeria	storage	1 (+ replica at site B)
`prometheus`, `alertmanager`, `grafana`	Monitoring	Metrics, alerts, dashboards	monitoring	1 each
`node-exporter`, `postgres-exporter`, `redis-exporter`, `celery-exporter`, `caddy` metrics	Exporters	Expose metrics	every server	1 each
`glitchtip` (+ its own Postgres and Redis)	Sentry-compatible error tracking, self-hosted	Errors from Django, Celery, both Next.js apps	monitoring	1
`loki` + `promtail` (or `vector`)	Log storage and shipping	JSON logs searchable 90 days	monitoring	1

Servers at launch (production): **app**, **worker**, **db**, **storage**, **monitoring** at site A; **backup/replica** at site B. Staging runs all containers on one server with smaller limits. The dev `compose.yaml` runs `postgres`, `redis`, `minio`, `clamav`, `api`, one worker, `beat` and Mailpit (fake email inbox).

4. Domains and DNS

Name (proposal)	Points to	Purpose
`bladus.com.dz` and `www.bladus.com.dz`	Caddy at site A	Website (guests and hosts)
`admin.bladus.com.dz`	Caddy at site A, IP allowlist	Admin app
`hooks.bladus.com.dz`	Caddy, routes only `/api/v1/webhooks/*` to Django	Payment gateway webhooks (R2)
`media.bladus.com.dz`	Storage (public bucket) through Caddy or CDN	Public photo renditions
`static.bladus.com.dz`	Caddy or CDN	Next.js static files (optional; Next.js can serve them itself)
`console.bladus.com.dz`	Caddy, VPN only	Django admin console (staff, MFA, read-mostly)
`status.bladus.com.dz`	Outside the main infrastructure	Public status page

- `.com.dz` registration goes through the Algerian registry (NIC.dz, run by CERIST) and normally requires a registered company (RC). Law 18-05 art. 8 mentions `.com.dz` for e-commerce. **VERIFY WITH A PROFESSIONAL** (L1, L11 in 13-decisions-needed.md). Also register `bladus.dz` if available and `bladus.com` to protect the brand; redirect them to the main name.
- The Django API has **no public host name**: only the Next.js servers reach it on the private network (BFF, 07-architecture.md section 10).
- Email: send from `notify.bladus.com.dz` with SPF, DKIM and DMARC (`p=quarantine`, then `reject`).
- DNS with two providers or a provider with anycast; low TTL (300 s) on records we may switch during an incident.

5. TLS

- Caddy obtains and renews certificates automatically with ACME (Let's Encrypt; ZeroSSL as fallback). If the `.com.dz` registry or provider requires another certificate authority, use that one (VERIFY).
- TLS 1.2 and 1.3 only; HSTS on after two weeks of stable HTTPS, then `includeSubDomains` and `preload`.
- Internal traffic: TLS from Caddy to Next.js is optional on the same server; **required** between servers (Django to PostgreSQL with `verify-full`, Redis TLS, storage HTTPS) using a small internal certificate authority (step-ca) or provider private network plus TLS.

6. CDN

- **Launch without a foreign CDN.** Next.js serves its static files with long cache headers (`immutable`), Caddy compresses (zstd, gzip), public photos are served from the storage server through Caddy with `Cache-Control: public, max-age=31536000, immutable` (keys are random and never reused).

- If measurements show slow images or static files for users far from site A, options in order: (1) a second Caddy cache in another Algerian city; (2) Cloudflare or another CDN **only** for `static.` and `media.`, after the lawyer confirms public photos and static files are not personal data subject to residency (VERIFY). Never route the website HTML, the admin or the API through a foreign CDN, because the CDN would decrypt personal data.

7. CI/CD on GitHub Actions

7.1 Pipelines

Workflow	Trigger	Jobs (in order)	Blocks merge or deploy?
`backend-ci.yml`	Every pull request and push	1. **Lint**: `ruff check`, `ruff format --check`. 2. **Types**: `mypy` with django-stubs and djangorestframework-stubs. 3. **Architecture**: `lint-imports` (import-linter layer contracts). 4. **Migrations**: `makemigrations --check --dry-run`, migration linter, migrate from zero. 5. **Tests**: `pytest` with a **PostGIS service container** (`postgis/postgis:17-3.5`) and Redis service; coverage gates (10-testing-quality.md section 11). 6. **Traceability**: every BR rule has a test. 7. **Security**: `pip-audit`, `bandit`, `gitleaks`	Yes
`contract.yml`	Every pull request	1. Generate `openapi.yaml` with `spectacular --validate --fail-on-warn`. 2. **OpenAPI diff** against `main` with `oasdiff breaking`; a breaking change fails unless the PR has the `api-breaking` label and a `/v2` plan. 3. Start the API with seed data and run **schemathesis** (all endpoints, stateful links). 4. Generate TypeScript types and run `tsc --noEmit` in `frontend/` and `admin/` against them	Yes
`frontend-ci.yml` / `admin-ci.yml`	Changes under `frontend/` or `admin/`	`npm ci`, `next lint`, `tsc --noEmit`, unit tests, Playwright smoke tests (admin already has Playwright), `npm audit --omit=dev`	Yes
`build.yml`	Merge to `main`, tags	Build images (`bladus-api`, `bladus-website`, `bladus-admin`) with build args; **refuse the website production build unless** `NEXT_PUBLIC_BLADUS_TEST_MODE=0`; Trivy image scan (fail on critical); SBOM; push to GitHub Container Registry with the commit SHA tag	Yes for deploy
`deploy-staging.yml`	After `build.yml` on `main`	Deploy to staging (runbook 11.1), run smoke tests, run k6 smoke (1 minute)	Yes for production
`deploy-prod.yml`	Release tag `vYYYY.MM.DD.N`	Manual approval by one of two named approvers (GitHub environment protection), then runbook 11.1 on production, smoke tests, automatic rollback on failure	
`nightly.yml`	Every night	Concurrency test suite with repeats, full schemathesis run, dependency scan, k6 performance run against staging (weekly)	Opens an issue on failure

7.2 How deployment reaches Algeria

The deploy job runs on a **self-hosted GitHub Actions runner** inside the private network at site A (one for staging, one for production, separate machines). It only pulls signed images and runs Compose commands; it holds no database passwords (those are on the servers, decrypted from the secret store at deploy time). Alternative: a bastion host with SSH deploy keys restricted to one command.

8. Database operations

8.1 Migrations policy

1. **Expand, then contract.** A release may only add things (columns nullable or with a constant default, new tables, new indexes). Removing or renaming happens in a later release, after no running code uses the old name. This keeps the previous app version working, so rollback never needs a down-migration.
2. Large indexes use `AddIndexConcurrently` in a non-atomic migration; constraints on large tables are added `NOT VALID` then validated separately.
3. Every migration sets `lock_timeout = 5s`; if it cannot get the lock it fails instead of freezing the site.
4. Data migrations are separate from schema migrations and must be safe to run twice; big backfills run as Celery jobs in batches, not inside migrations.
5. Migrations run as a **one-off container** (`manage.py migrate`) before the new app containers start, using the `bladus_migrate` role. The app role cannot change the schema.
6. A pgBackRest incremental backup is taken right before any release that contains a migration flagged `risky` in review.
7. Squash migrations only once a year, never across a release still deployed somewhere.

8.2 Backups, PITR and replica

Mechanism	Tool	Frequency	Kept	Target (NFR-04)
WAL archive	pgBackRest, `archive_timeout = 60 s`, repository at site B, encrypted (`repo-cipher-type = aes-256-cbc`)	continuous	7 days of PITR	RPO 15 min (expected: about 1 min)
Full + differential backups	pgBackRest	full weekly (Sunday 02:00), differential daily	2 full cycles	base for PITR
Streaming replica	PostgreSQL physical replication to site B	continuous	live	fast failover, RTO 4 h
Logical dump	`pg_dump -Fc`, encrypted with **age** before leaving the server	nightly 01:00	35 daily, 12 monthly	protection against logical mistakes and tool bugs
Object storage	MinIO bucket replication (or provider replication) to site B; versioning on private buckets	continuous	30 days of versions	documents and photos
Secrets and keys	Offline copies of the backup decryption key and field encryption master key, held by two people	on change	always	without them backups are useless

All backup copies stay in Algeria (BR-PRV-04). Backup jobs report a "last success" metric; an alert fires if older than 26 hours (dumps) or 10 minutes (WAL archive).

8.3 Monthly restore drill

On the first Tuesday of each month: restore the latest full backup plus WAL to a scratch server at site B to a chosen time 1 hour earlier; run `manage.py check`, row counts, `ledger.reconcile_balances` and `audit.verify_hash_chain` on the copy; open 3 random private documents from the storage replica; record the time taken (must be under 4 hours) and the result in the drill log; destroy the scratch server. A failed drill is a high-priority incident.

8.4 Routine care

`pg_stat_statements` reviewed weekly (top 20 queries by total time, compared with the query plan targets in 05-database-schema.md section 7); autovacuum tuned for hot tables (`room_type_night`, `outbox_event`, `notification`); minor PostgreSQL updates monthly, major upgrades yearly with a rehearsal in staging; disk alerts at 70% and 85%.

---

9. Observability

9.1 Logs

- **structlog** writes one JSON object per line to stdout; Promtail or Vector ships them to Loki (in Algeria), searchable 90 days, archived 1 year.
- Fields on every API log line: `ts`, `level`, `event`, `request_id`, `user_id`, `host_account_id`, `staff` (boolean), `method`, `route` (pattern, not raw path), `status`, `duration_ms`, `db_queries`, `db_ms`, `ip_prefix` (IPv4 /24 for analytics; full IP only in the audit log), `app_version`. Celery lines add `task`, `task_id`, `queue`, `attempt`.
- **Never logged**: passwords, OTP codes, session tokens, `Authorization` or `X-Session-Token` headers, ID numbers, payout numbers, message bodies, full phone numbers or emails (masked as `+213 5** ** ** 12`). A scrubbing processor enforces this and has its own tests.

9.2 Errors

GlitchTip (Sentry-compatible, self-hosted in Algeria) with the Sentry SDKs in Django, Celery and both Next.js apps; `send_default_pii = False`; request bodies and headers scrubbed; release and environment tags; source maps uploaded for the Next.js apps but not served publicly.

9.3 Metrics

Prometheus scrapes every 15 s. Key metrics:

Metric	Source	Why
`http_request_duration_seconds{route,method,status}` (histogram)	django-prometheus	NFR-01, NFR-02 latency
`bladus_booking_create_total{result}` (`confirmed`, `pending`, `conflict`, `price_changed`, `error`)	app	Booking health, double-booking attempts
`bladus_outbox_pending`, `bladus_outbox_oldest_age_seconds`, `bladus_outbox_failed`	app (scraped from DB by a collector)	Notifications delayed
`celery_queue_length{queue}`, `celery_task_failed_total{task}`	celery-exporter	Background work
`bladus_beat_heartbeat_timestamp`	`core.heartbeat` task every minute	Beat is alive
`bladus_otp_sent_total{channel,provider}`, `bladus_sms_spend_dzd_today`	app	SMS pumping, budget
`bladus_login_failures_total`, `bladus_lockouts_total`	app	Attacks
`bladus_ledger_reconcile_mismatch`	`ledger.reconcile_balances`	Money correctness
`bladus_audit_chain_ok`	`audit.verify_hash_chain`	Tamper evidence
`bladus_backup_last_success_timestamp{kind}`	backup scripts	NFR-04
`pg_replication_lag_seconds`, `pg_stat_activity_count`, `pg_database_size_bytes`	postgres-exporter	Database health
`redis_memory_used_bytes`, evictions	redis-exporter	Cache and limits
Node CPU, memory, disk, network	node-exporter	Capacity

9.4 Health checks

Endpoint	Checks	Used by
`/internal/healthz` (Django)	Process answers	Container restart policy
`/internal/readyz` (Django)	`SELECT 1` on PostgreSQL through PgBouncer, Redis `PING`, storage bucket reachable, no unapplied migrations	Caddy upstream health, deploy smoke test
`/api/health` (each Next.js app)	Process answers and can reach Django `readyz`	Caddy, deploy
Celery	`celery inspect ping` + beat heartbeat age	Monitoring
External	Uptime probe from outside the provider (website home, login page, one search) every minute	Status page, alerts

9.5 Alerts

Alert	Condition	Severity	First action
Site down	External probe fails 3 times in a row	critical	Incident runbook 11.3
API slow	p95 read latency > 300 ms or booking create > 600 ms for 10 min (NFR-01)	high	Check DB, recent deploy
Search slow	p95 search > 500 ms for 10 min (NFR-02)	high	`pg_stat_statements`, indexes
Error rate	5xx > 1% of requests for 5 min	critical	GlitchTip, rollback if after a deploy
Outbox stuck	Oldest pending event > 5 min, or any `failed`	high	Worker logs, provider status

Alert	Condition	Severity	First action
Beat stopped	Heartbeat older than 3 min	critical	Restart beat (only one!)
Queue backlog	`critical` queue > 100 for 5 min	high	Scale worker, check provider
SMS spend	80% of daily budget	high	Check for pumping; at 100% non-critical SMS stop automatically
Login attack	Lockouts > 50 in 10 min	high	Block IP ranges at Caddy
Ledger mismatch	`bladus_ledger_reconcile_mismatch > 0`	critical	Freeze top-ups; finance and tech lead investigate
Audit chain broken	`bladus_audit_chain_ok == 0`	critical	Security incident; preserve evidence
Backup late	WAL archive > 10 min or nightly dump > 26 h	critical	Fix before anything else (RPO at risk)
Replica lag	> 60 s for 10 min	high	Network to site B, disk
Disk	> 85% on any server	high	Expand or clean
Certificate	Expires in < 14 days	high	Check Caddy renewal
Restore drill	Not recorded this month by day 10	medium	Schedule it

Alerts go to a team chat channel and, for critical ones, by SMS to the on-call person (through a different provider than the user SMS, so a provider outage does not hide its own alert).

---

10. Capacity and cost (rough, VERIFY WITH QUOTES)

Capacity basis: NFR-01 (50 requests per second on one app node, p95 300 ms reads), NFR-02 (search p95 500 ms with 20,000 listings), and the two-year growth estimate in 05-database-schema.md section 7 (fits one PostgreSQL primary with 8 vCPU / 32 GB). Expected launch traffic is far below these numbers; the sizes below leave headroom and can start smaller.

Prices are rough planning numbers, not quotes. Only "ICOSNET from 2,800 DZD/month per VPS" comes from research (C.6, [secondary]); other lines are estimates by analogy. Exchange rate assumed **1 EUR = 150 DZD** (official rate order of magnitude; VERIFY at the time of purchase).

Item	Size (launch)	DZD per month	EUR per month
App server (Caddy, 2 x website, admin, 2 x API, Redis)	8 vCPU, 16 GB RAM, 160 GB SSD	25,000 to 45,000	165 to 300
Worker server (Celery queues, beat, ClamAV)	4 vCPU, 8 GB, 80 GB	12,000 to 25,000	80 to 165
Database server (PostgreSQL primary, PgBouncer, pgBackRest)	8 vCPU, 32 GB, 500 GB SSD	40,000 to 80,000	265 to 535
Storage server (MinIO, if no provider S3)	2 vCPU, 4 GB, 2 TB disk	15,000 to 35,000	100 to 235
Monitoring server (Prometheus, Grafana, Loki, GlitchTip)	4 vCPU, 8 GB, 200 GB	10,000 to 20,000	65 to 135
Site B: replica + backup repository + storage replica	8 vCPU, 32 GB, 3 TB	35,000 to 70,000	235 to 465
Staging (all-in-one)	8 vCPU, 16 GB, 200 GB	15,000 to 30,000	100 to 200
SMS (assume 10,000 per month; local aggregator 3 to 8 DZD each, prices not published, VERIFY; Twilio about USD 0.27)		30,000 to 80,000	200 to 535
Email (transactional, provider in Algeria or self-hosted relay, VERIFY)		0 to 5,000	0 to 35
Domains (`.com.dz` , `.dz` , `.com`), yearly cost spread		500 to 1,500	3 to 10
GitHub (Team plan, about 5 users)		2,500 to 4,000	17 to 27
TLS certificates (ACME)		0	0
Total per month		**about 185,000 to 395,000**	**about 1,230 to 2,640**
One-off: external penetration test before launch		600,000 to 1,500,000	4,000 to 10,000
One-off: legal review (privacy, e-commerce, hosting)		quote needed	quote needed

Ways to start cheaper: merge worker and monitoring onto the app server for the first months (saves about 25,000 DZD); use email or WhatsApp codes before SMS where users accept it (SMS is the largest variable cost); keep staging off outside working hours.

When to grow: second API server when CPU > 60% at peak for a week; Redis on its own server when memory > 2 GB; read replica for search when DB CPU > 60% at peak; external search only per 07-architecture.md section 7.

---

11. Runbooks

11.1 Deploy

1. Check: CI green on the commit; release notes list migrations and flags; no open critical alert; not Thursday evening or Friday (Algerian weekend starts Friday; avoid deploying before it).
2. Announce in the team channel with the release tag.
3. Runner pulls images for the tag and verifies their digests.
4. If the release has a `risky` migration: trigger a pgBackRest incremental backup and wait for success.
5. Run the migration container (`manage.py migrate --noinput`) with the `bladus_migrate` role. On failure: stop here (nothing else changed); open an incident if the site is affected.
6. Rolling update of `api` : start the new container, wait for `/internal/readyz` , remove one old container from Caddy, stop it gracefully (30 s), repeat.
7. Restart workers with warm shutdown (`SIGTERM` , tasks finish), then start new ones. Restart `beat` last, making sure only one runs.
8. Rolling update of `website` and `admin` the same way.
9. Smoke tests: home page in 3 languages, search with dates, listing detail, login, host dashboard (staging account), admin login with MFA (staging only), `/internal/readyz` .
10. Watch dashboards 15 minutes (error rate, latency, outbox). Announce success.

11.2 Rollback

1. Decide fast: a new error rate above 1% or a broken key flow after a deploy means rollback first, analysis later.
2. Redeploy the previous image tag with steps 6 to 9 of 11.1. **Do not** run down-migrations: the expand-then-contract rule keeps the previous code compatible with the new schema.
3. If the bad release changed data (wrong postings, wrong statuses), stop the affected jobs (feature flag or scale the worker to 0), then write correcting transactions through services (ledger: reversing transactions). Never edit rows by hand.
4. If a migration itself destroyed data (should be impossible under the policy): restore to a scratch server with PITR to the minute before the migration and copy back the affected rows with a reviewed script (runbook 11.4).
5. Write a short incident note.

11.3 Incident

1. **Detect**: alert or user report. The on-call person acknowledges within 15 minutes (business hours) or 30 minutes (night).
2. **Declare**: open an entry in the incident register with severity (SEV1 site down or data at risk; SEV2 key feature broken; SEV3 degraded). Name one incident lead.
3. **Communicate**: status page updated within 30 minutes for SEV1/SEV2 in en, fr, ar; staff informed in the admin app banner.
4. **Stabilise**: rollback (11.2), switch off a feature flag, block abusive traffic at Caddy, scale workers, or fail over (step 5).
5. **Failover to site B** (site A lost or database primary lost for more than 30 minutes): promote the replica at site B; switch DNS (TTL 300 s) to site B's proxy; start app, worker and beat containers there from the same images; verify `readyz`, the ledger reconcile and the audit chain; keep site A isolated until repaired, then rebuild it as the new replica. Target total time under 4 hours (RTO).
6. **Personal data involved?** Start the breach runbook in 08-security-privacy.md section 12.7 at once: the 5-day ANPDP clock starts at discovery.
7. **Close**: when stable for 1 hour. Within 5 working days: blameless review (timeline, cause, what worked, actions with owners and dates).

11.4 Restore (data loss or corruption)

1. Freeze writes if corruption is spreading: put the platform in maintenance mode (`maintenance_mode` setting; the website shows a notice; staff read-only).
2. Identify the target time (just before the bad event) from logs, `booking_event`, `audit_event`.
3. Restore with pgBackRest to a **scratch** server at site B to that time (`--type=time --target=...`). Never restore over the live primary first.
4. Verify on the scratch copy: row counts, ledger reconcile, audit chain up to the target time, sample bookings.
5. Choose: (a) **partial repair**: export the affected rows from the scratch copy and re-apply them through a reviewed script or services (preferred, keeps later good data); (b) **full restore**: replace the primary with the restored copy (loses everything after the target time; users must be told; bookings after that time must be rebuilt from emails and logs).
6. For object storage: restore object versions from the site B replica.
7. Lift maintenance mode, monitor, record the restore in the drill/incident log. If personal data was lost or exposed, apply the breach runbook.

---

12. Environment variables (names and purpose only)

No values here. Real values live in the secret store (08-security-privacy.md section 7). Typed and validated at start by pydantic-settings (`config/env.py`); a missing required value stops the program.

12.1 Django API, workers and beat

Variable	Purpose
`BLADUS_ENV`	`dev`, `ci`, `staging` or `production`; selects defaults and safety checks
`DJANGO_SETTINGS_MODULE`	Settings module for the environment
`DJANGO_SECRET_KEY` / `DJANGO_SECRET_KEY_FALLBACKS`	Django signing key, and old keys accepted during rotation
`DJANGO_ALLOWED_HOSTS`	Host names Django answers to (private names only)
`TRUSTED_PROXY_IPS`	Addresses allowed to set `X-Forwarded-For` (Caddy, BFFs)
`BFF_SERVICE_KEY_WEBSITE` / `BFF_SERVICE_KEY_ADMIN`	Service keys checked on requests from each Next.js server
`DATABASE_URL`	PostgreSQL connection through PgBouncer, app role
`DATABASE_MIGRATE_URL`	Connection with the migration role (migration container only)
`DATABASE_REPLICA_URL`	Read-only replica for reports and exports
`DB_STATEMENT_TIMEOUT_MS` / `DB_LOCK_TIMEOUT_MS`	Database time limits (07-architecture.md section 3)
`REDIS_URL`	Redis for cache (db 1)
`CELERY_BROKER_URL`	Redis for Celery (db 0)
`RATELIMIT_REDIS_URL`	Redis for rate limits, OTP counters, locks (db 2)
`S3_ENDPOINT_URL`	Object storage address (in Algeria)
`S3_ACCESS_KEY_ID` / `S3_SECRET_ACCESS_KEY`	Object storage credentials
`S3_BUCKET_PUBLIC` / `S3_BUCKET_PRIVATE` / `S3_BUCKET_QUARANTINE` / `S3_BUCKET_EXPORTS`	Bucket names
`MEDIA_PUBLIC_BASE_URL`	Base URL for public photo renditions
`PRESIGNED_URL_TTL_SECONDS`	Lifetime of upload and private download links (300)
`CLAMAV_HOST` / `CLAMAV_PORT`	Antivirus daemon address
`FIELD_ENCRYPTION_KEYS`	Versioned master keys for envelope encryption of payout numbers
`ID_HASH_PEPPER`	Secret for HMAC of ID numbers and recipient hashes
`SMS_PROVIDER` / `SMS_API_KEY` / `SMS_SENDER_ID`	Main SMS provider (local aggregator)
`SMS_FALLBACK_PROVIDER` / `SMS_FALLBACK_API_KEY`	Fallback SMS provider (transfer abroad: VERIFY)
`SMS_DAILY_BUDGET_DZD` / `SMS_MONTHLY_BUDGET_DZD`	SMS spending caps (NFR-11)

Variable	Purpose
`EMAIL_BACKEND` / `EMAIL_HOST` / `EMAIL_PORT` / `EMAIL_HOST_USER` / `EMAIL_HOST_PASSWORD` / `EMAIL_USE_TLS`	Outgoing email
`DEFAULT_FROM_EMAIL`	Sender address (`no-reply@notify.bladus.com.dz`)
`STAGING_EMAIL_ALLOWLIST`	Staging only: addresses allowed to receive email
`SENTRY_DSN` / `SENTRY_ENVIRONMENT` / `SENTRY_RELEASE`	Error tracking (GlitchTip)
`LOG_LEVEL` / `LOG_JSON`	Logging level and format
`PROMETHEUS_MULTIPROC_DIR`	Shared folder for metrics from several Gunicorn workers
`GUNICORN_WORKERS` / `GUNICORN_TIMEOUT`	API process tuning
`CELERY_WORKER_CONCURRENCY`	Worker tuning per container
`NOMINATIM_URL` / `NOMINATIM_USER_AGENT`	Address lookup service (server-side, cached, 1 request per second)
`MONEY_MODEL`	`M1` at launch; other models stay off until legal approval (BR-PAY-11)
`CHARGILY_API_KEY` / `CHARGILY_WEBHOOK_SECRET`	Payment gateway (R2 only)
`AUDIT_ANCHOR_EMAILS`	Who receives the daily audit hash anchor
`TIME_ZONE_BUSINESS`	`Africa/Algiers` (business dates; storage stays UTC)

Business settings (commission rates, request TTL, credit limits, maintenance mode, feature flags) are **not** environment variables: they live in the versioned `platform_settings` table (BR-CFG-01).

12.2 Website (Next.js)

Variable	Purpose
`BLADUS_API_URL`	Private address of the Django API (server-only)
`BFF_SERVICE_KEY`	This app's service key for Django (server-only)
`BLADUS_SESSION_COOKIE_KEY` / `BLADUS_SESSION_COOKIE_KEY_PREVIOUS`	Encryption key for the session cookie, and the previous key during rotation (replaces `DARNA_SESSION_SECRET`)
`BLADUS_API_MODULES`	Server-only list of `lib/data` modules served by the API instead of mock data (migration flag)
`BLADUS_SHADOW_MODULES`	Staging: modules running in shadow-read mode
`NEXT_PUBLIC_SITE_URL`	Public origin for canonical URLs, sitemap, Open Graph
`NEXT_PUBLIC_BLADUS_TEST_MODE`	Demo test mode. **Must be `0` in production and any environment with real data**; the build refuses otherwise
`NEXT_PUBLIC_MEDIA_BASE_URL`	Base URL of public photos
`NEXT_PUBLIC_MAP_TILES_URL`	Map tiles (OpenFreeMap, with attribution)
`SENTRY_DSN` / `NEXT_PUBLIC_SENTRY_DSN`	Error tracking, server and browser (browser DSN is not a secret)

12.3 Admin (Next.js)

Variable	Purpose
`BLADUS_API_URL`	Private address of the Django API
`BFF_SERVICE_KEY`	Admin service key for Django
`ADMIN_SESSION_COOKIE_KEY` / `ADMIN_SESSION_COOKIE_KEY_PREVIOUS`	Encryption key for the staff cookie (replaces `ADMIN_AUTH_SECRET`)
`ADMIN_IDLE_TIMEOUT_MINUTES`	Fallback idle timeout if settings are unreachable (30)
`BLADUS_API_MODULES`	Migration flag, as for the website
`NEXT_PUBLIC_FRONTEND_URL`	Public website origin (for "view public page" links)
`SENTRY_DSN`	Error tracking

The current `ADMIN_SEED_*` variables (mock staff passwords) disappear when the admin `auth` module moves to the API; staff accounts are then created by invitation (STF-02).

12.4 Operations

Variable	Purpose
`PGBACKREST_REPO1_HOST` / `PGBACKREST_REPO1_CIPHER_PASS`	Backup repository at site B and its encryption passphrase
`BACKUP_AGE_RECIPIENT`	Public key used to encrypt nightly dumps (the private key is offline)
`REPLICATION_PASSWORD`	Password of the replication role
`MINIO_ROOT_USER` / `MINIO_ROOT_PASSWORD`	Storage administrator (if self-hosted)
`GRAFANA_ADMIN_PASSWORD`	Monitoring administrator
`ALERT_SMS_PROVIDER_KEY` / `ALERT_CHAT_WEBHOOK_URL`	Where alerts are sent

Check yourself

- [ ] Why is staging never filled with a copy of production data?
- [ ] Which data may live outside Algeria, under which condition, and who must confirm it?
- [ ] Which container must run exactly once, and what happens if two run?
- [ ] A release removes a column. Why must the removal wait for a later release?
- [ ] What gives us an RPO of 15 minutes, and what gives us an RTO of 4 hours?
- [ ] What do we check during the monthly restore drill?
- [ ] Which pipeline step refuses a website build with test mode on?
- [ ] Where do commission rates live: environment variables or the database? Why?

10. Testing and quality

> **In plain words**
> This page says how we prove the backend does what the rules say, before every deployment.
> Most tests are small and fast; fewer are big and slow (the "test pyramid"). Money, refunds and dates are tested with thousands of random examples, not only a few hand-picked ones.
> Double booking is tested by firing many requests at the same time against a real PostgreSQL database.
> The API contract is checked automatically so the website and admin never break silently; speed is measured against the targets in NFR-01 and NFR-02.
> Every business rule (`BR-...`) must have at least one test whose name contains the rule ID; the build fails if one is missing.

Terms used here (see also glossary.md):

- **Unit test**: tests one small function alone, in milliseconds, often without a database.
- **Service test / integration test**: tests a service function with a real database, inside a transaction that is rolled back afterwards.
- **API test**: sends an HTTP request to an endpoint and checks the response (status, body, permissions).
- **End-to-end (E2E) test**: drives a real browser through the website or admin (Playwright).
- **Test pyramid**: many unit tests, fewer service and API tests, very few E2E tests.
- **pytest / pytest-django**: the Python test runner and its Django plugin.
- **Factory (factory_boy)**: code that creates realistic test rows (a user, a listing, a booking) with one line.
- **Property-based test (hypothesis)**: instead of one example, you state a rule that must always hold ("refund never exceeds the total"), and the tool tries hundreds of random inputs, then shrinks any failure to the simplest example.
- **Concurrency test**: starts many requests at the same moment to check that locks and constraints hold.
- **Contract test (schemathesis)**: reads `openapi.yaml` and sends many generated requests to every endpoint, checking the answers match the contract and the server never crashes.
- **Load / performance test (k6)**: simulates many users to measure response times.
- **Coverage**: the share of code lines (or branches) executed by tests.
- **Mutation testing**: the tool makes small bugs in the code on purpose; good tests should fail. Measures test strength.
- **Quality gate**: a check in CI that must pass before code can be merged or deployed.
- **Definition of Done (DoD)**: the checklist a change must satisfy before it counts as finished.
- **Traceability**: being able to go from a rule to its test and back.

Inputs: 03-requirements.md (NFR-01, 02, 06, 08, 10, 13), 04-business-rules.md, 05-database-schema.md sections 6 and 7, 07-architecture.md, 08-security-privacy.md, 09-devops-hosting.md section 7, research B.14.

---

1. Test strategy (pyramid)

Level	Share of tests	Tools	Database	Runs
Unit (pure functions: money, policies, date ranges, masking, transitions)	about 45%	pytest, hypothesis	none	every push, seconds
Service (one service or selector with real PostgreSQL + PostGIS)	about 35%	pytest-django, factory_boy, time-machine (frozen clock)	real PostgreSQL in a rolled-back transaction	every push
API (endpoint, permissions, serializers, error codes)	about 15%	pytest-django + DRF `APIClient`	real PostgreSQL	every push
Concurrency (parallel requests)	small set, critical	pytest with `transaction=True`, threads, barrier	real PostgreSQL, committed	every push (short) + nightly (repeated)
Contract	all endpoints	schemathesis, drf-spectacular, oasdiff, `tsc` in both apps	seeded API	every pull request
E2E (key journeys through the Next.js apps)	about 5% (around 20 journeys)	Playwright (admin already uses it)	staging-like stack	before deploy to production
Performance	scenarios in section 7	k6	staging with 20,000 listings	weekly and before major releases

Principles:

- **Real PostgreSQL always.** No SQLite: exclusion constraints, `daterange`, PostGIS, partial indexes and triggers are part of the rules (BR-AVL-01, BR-PAY-02).
- **Test the rule where it is enforced.** A DB-enforced rule is tested by trying to break it with raw ORM writes (bypassing the service) and expecting the database to refuse.
- **Frozen time.** Services receive `now` from `core.clock`; tests freeze it in Africa/Algiers for date rules (BR-SYS-05).
- **No network.** SMS, email, storage and gateway calls go to fakes that record what was sent; one contract test per provider runs against recorded responses.

2. What to test per app

App	Must test	Key rules
`core`	`Money` arithmetic and rounding; idempotency (same key same body, different body 422, concurrent 409, expiry 24 h); error format and translation; UUIDv7 ordering	BR-PAY-01, BR-SYS-01, BR-SYS-04, NFR-12
`geo`	Seed completeness (69 wilayas, 1,541 communes, codes unique, `legacy_code_58` mapping); autocomplete with accents and Arabic	BR-SRC-02
`config`	Unknown settings key refused; new version per change; publish creates immutable version; core steps not removable; old version readable	BR-CFG-01 to 03
`audit`	Insert-only (UPDATE refused by DB); hash chain correct; chain break detected; export formula neutralisation; export audited with filters and count	BR-STF-02, BR-STF-04
`media`	Presign limits; magic bytes; EXIF/GPS removed (fixture photo with GPS); decompression bomb refused; virus test file (EICAR) rejected; signed URL expiry; KYC view audited	BR-MED-01, 02
`notifications`	Outbox written in the same transaction; rolled-back change sends nothing; retry and backoff; idempotent sending (one SMS per event); language of recipient; critical messages ignore opt-out	BR-SYS-03, BR-NTF-01, 02, BR-PRV-03

App	Must test	Key rules
`accounts`	Email unique case-insensitive; phone format; OTP limits and expiry; lockout shared via Redis; session revocation on logout and password change; staff MFA required; consent records; deletion anonymises and keeps legal records	BR-ACC-01 to 05, BR-PRV-01, 02
`hosts`	Host account is user XOR organisation; one owner; invitations (7 days, once, matching contact); role permissions matrix; legal field change resets verification; verification transitions; documents all approved before host approval; publish rights per host type; payout change needs MFA and sets 72 h hold	BR-ORG-01 to 06, BR-HST-01 to 05, BR-PAY-09
`catalog`	Listing state machine (every allowed and every forbidden move); revision under review keeps live version; hotel without room type cannot submit; reference rows cannot be hard-deleted; experience and trip types	BR-LST-01 to 05
`inventory`	Counter check constraint; atomic multi-night reservation all-or-nothing; holds expire; host block vs booking race; horizon job creates 18 months	BR-AVL-02 to 06
`trips`	Only approved agencies; seats never oversold; cancelling a departure cancels all bookings with refunds owed	BR-TRP-01 to 03
`pricing`	Quote lines and totals; weekly vs monthly discount; no guest fee; quote expiry and re-validation; tampered price ignored; tax rates by validity date (R2)	BR-PRC-01 to 04
`ledger`	Balanced transactions only (DB trigger); no UPDATE/DELETE; commission once per booking; reversal amounts; rate resolution; top-up limits, unique receipt, four-eyes; credit floor; balance cache equals sum; dormant models return 404	BR-PAY-01 to 12
`bookings`	Instant vs request resolution; snapshot frozen; state machine; expiry and completion jobs; cancellation by guest, host, staff with refunds; refunds never exceed total; disputes window (R2); references unique	BR-BKG-01 to 07, BR-CAN-01 to 05
`reviews`	Eligibility (completed, 14 days, one per booking); states and reasons; ratings recomputed; double-blind (R2)	BR-REV-01 to 04
`messaging`	Cursor sync returns only newer messages; unread counts; masking of phones, emails, CCP/RIP, WhatsApp links before confirmation and unmasked after; repeated attempts flag the thread	BR-MSG-01
`trust`	Queue priority by waiting time since last submission; change request items visible to host; suspension cascade pauses listings and flags future bookings; risk flags open cases, never block	BR-MOD-01 to 04
`search`	Only published listings of approved hosts; facet counts exclude own filter; public point only for anonymous users; partial availability sub-range; map pin cap; hotel and hostel under "Hotels"; search doc refreshed by outbox	BR-SRC-01 to 04, BR-AVL-05

Security tests (from 08-security-privacy.md section 6) live in each app as `test_api1_*` to `test_api10_*`, plus one matrix test that calls every `/api/v1/admin` endpoint with each of the 5 staff roles and checks allow or deny against the permission table.

---

3. Property-based tests (hypothesis)

Pure functions for money, refunds and dates are written so they can be tested without a database. Example properties (the tests will name the rule they protect):

3.1 Money

Property	Rule
For any list of price lines, `total == sum(rounded lines)`; never a float anywhere	BR-PAY-01
`commission(base, rate_bp)` is between `floor(base x rate / 10,000)` and that value + 1 centime, and `0 <= commission <= base`	BR-PAY-04
Commission is monotonic: a bigger base or rate never gives a smaller commission	BR-PAY-04
Applying the discount: only the larger of weekly and monthly applies; result `>= 0`; 30 nights never uses the weekly rate	BR-PRC-03
For any sequence of postings, every ledger transaction sums to zero per currency, and the wallet balance equals the sum of its entries	BR-PAY-02, 03
Displayed whole dinars: `display(amount)` rounds half up and `display(a) + display(b)` may differ from `display(a + b)` by at most 1 DZD, which is why totals are summed from lines	BR-PAY-01

Example (illustration only, no code files are created in this phase):

```
@given(base=st.integers(0, 10**12), rate_bp=st.integers(0, 10_000))
def test_br_pay_04_commission_bounds(base, rate_bp):
    c = commission_minor(base, rate_bp)
    exact_floor = base * rate_bp // 10_000
    assert exact_floor <= c <= exact_floor + 1
    assert 0 <= c <= base
```

3.2 Refunds and cancellations

Property	Rule
`0 <= refund <= total - already_refunded` for any policy, any day, any amounts	BR-CAN-04
Refund is non-decreasing with days before check-in (cancelling earlier never refunds less)	BR-CAN-01
Before check-in, the cleaning fee and its taxes are always fully refunded	BR-CAN-01, 02
Host cancellation: refund owed == total paid; commission reversal == commission posted	BR-CAN-03, BR-PAY-05
Guest cancellation: commission kept == `round(rate x non-refunded subtotal)`; kept + reversed == posted	BR-PAY-05
Several partial staff refunds in any order: their sum never exceeds the total; the last one that would exceed is refused	BR-CAN-04
Worked example 8a reproduces exactly: 31,750 DZD owed, 3,330 DZD back to the wallet	BR-CAN-02

```
@given(policy=st.sampled_from(POLICIES), b=bookings(), d1=st.integers(-2, 400), d2=st.integers(-2, 400))
def test_br_can_01_refund_never_decreases_when_cancelling_earlier(policy, b, d1, d2):
    early, late = max(d1, d2), min(d1, d2)
    assert refund_for(policy, b, days_before=early) >= refund_for(policy, b, days_before=late)
```

3.3 Date ranges

Property	Rule
For `[a, b)`, nights == `b - a` in days; `a < b` always; an empty range is refused	BR-SYS-05, BR-AVL-01
Overlap is symmetric: `overlaps(x, y) == overlaps(y, x)`; back-to-back ranges (`x.end == y.start`) never overlap	BR-AVL-01
The Python overlap function agrees with PostgreSQL `&&` on `daterange` for any pair (tested against the database)	BR-AVL-01
`largest_free_subrange(request, busy)` is inside the request, contains no busy night, and no longer free sub-range exists	BR-AVL-05
Day counting for policies uses Africa/Algiers calendar days; around midnight UTC (00:00 to 01:00 Algiers) the day count is still correct	BR-CAN-01, BR-SYS-05
Weekend detection: Friday and Saturday only; week starts on Sunday	BR-SYS-05
Pool reservation of `n` units over `k` nights changes exactly `k` counter rows or none	BR-AVL-02

Hypothesis settings: `max_examples = 200` on every push, `2,000` in the nightly run; the example database is kept in CI cache so a found failure is replayed until fixed.

4. Concurrency tests on real PostgreSQL

How: `@pytest.mark.django_db(transaction=True)` (real commits), a `ThreadPoolExecutor` where each thread opens its own database connection, and a `threading.Barrier` so all threads call the service at the same instant. Each scenario runs 5 times on every push and 200 times in the nightly run. CI uses the same PostgreSQL and PostGIS versions as production.

Scenario	Setup	Expected	Rule
Same nights, single unit	50 guests book the same home for the same nights	Exactly 1 booking created; 49 get `409 dates_unavailable`; no 500	BR-AVL-01
Back-to-back	2 guests book `[1, 4)` and `[4, 6)` at the same time	Both succeed	BR-AVL-01
Last hotel room	Room type with 1 unit left; 20 parallel requests for 1 room	Exactly 1 success; counters `held + booked <= total`	BR-AVL-02
Last seats on a departure	3 seats left; 10 requests for 2 seats each	Exactly 1 success; seats booked = 2	BR-TRP-02
Host blocks while guest books	Host blocks night N while a guest books a stay including N	One wins; never both	BR-AVL-04
Same idempotency key	10 identical requests with one key	1 booking; others get the same response or `409 request_in_progress`	BR-SYS-01
Double confirmation	Host clicks accept twice; job tries to expire at the same time	One transition wins; commission posted once	BR-PAY-04, BR-BKG-06
Credit floor race	Wallet just above the floor; 2 instant bookings at once whose commissions together cross the floor	Second one becomes `pending`, never both confirmed below the floor	BR-PAY-10
Duplicate top-up receipt	2 finance users record the same receipt at once	1 succeeds, 1 refused	BR-PAY-07
Outbox workers	3 dispatchers on the same pending events	Each event handled once (`SKIP LOCKED`); one SMS per event	BR-SYS-03
Expiry vs accept	Request accepted at the exact second the expiry job runs	Final status is either `confirmed` or `expired`, consistent with ledger and counters	BR-BKG-04
Ownership transfer	Two owners-to-be at once	Exactly one active owner	BR-ORG-04

A scenario passes only if the database ends in a consistent state: a final check runs the ledger reconciliation and counter sums after each run.

5. Contract tests

Check	How	Fails when
Schema is valid	`manage.py spectacular --validate --fail-on-warn`	An endpoint has no documented serializer, an enum is unnamed, a response is undocumented
No accidental breaking change	`oasdiff breaking main.yaml pr.yaml`	A field is removed or renamed, a type changes, a required parameter is added, an enum value disappears, without the `api-breaking` label and a version plan
API matches its contract	`schemathesis run openapi.yaml --checks all --stateful=links` against the API with seed data, authenticated as guest, host and each staff role	A response does not match the schema, a 500 occurs, an undocumented status code appears, a response is slower than 5 s
Error format	Custom schemathesis check	Any 4xx body without `code` and `message` (BR-SYS-04)
Frontend compile check	`openapi-typescript` generates `schema.d.ts`; `tsc --noEmit` in `frontend/` and `admin/`	A Next.js call uses a removed field or a wrong type
Adapter parity	For each migrated `lib/data` module, a test feeds the same fixture through the mock and the API adapter and compares DTO shapes	The website or admin would render different data after the switch (07-architecture.md section 10.4)
Provider contracts	Recorded responses (SMS, email, later Chargily) replayed against our client code	Our parser or signature check breaks

6. Migration tests

Check	When	Tool
No missing migrations	every push	`makemigrations --check --dry-run`
Migrate from zero	every push	fresh database, `migrate`, then load production seeds 1 to 8 (05-database-schema.md section 9)

Check	When	Tool
Dangerous operations	every push	migration linter: refuses column drops or renames in the same release as code changes, non-concurrent index creation on large tables, `NOT NULL` without default on existing tables
Data migrations	per data migration	`django-test-migrations`: build the old state, insert rows, migrate forward, assert the new state; run twice to prove it is safe to repeat
Old code on new schema	before release	run the previous release's API test suite against the new schema (expand-then-contract proof, 09-devops-hosting.md section 8.1)
Duration on realistic size	before release with a `risky` migration	apply on the performance dataset (section 7) and record the time and locks taken; above 10 s of locking needs a different plan
Constraints exist	every push	test queries `pg_constraint` and `pg_indexes` for the constraints in 05-database-schema.md section 6 (exclusion, partial unique, checks, triggers, grants)

7. Performance tests

Tool: **k6** (scripts in `tests/performance/`). Locust is an acceptable alternative if the team prefers Python. Dataset: generated by factories at the scale in 05-database-schema.md section 7: 20,000 listings across 69 wilayas, 2,000 room types with 18 months of nights, 1 million booking rows, 5 million messages. Run against staging (same server size as production app node), never production.

Scenario	Mix	Load	Pass criteria
S1 Browse and search	search with filters (60%), search with dates (25%), map box (15%)	ramp to 50 requests/s, hold 10 min	p95 < 500 ms, errors < 0.5% (NFR-02)
S2 Listing detail and calendar	detail + 90-day calendar	50 requests/s	p95 < 300 ms (NFR-01)
S3 Booking create	quote then book on random free dates (some collisions on purpose)	10 bookings/s for 5 min	p95 < 600 ms; zero double bookings; conflicts return 409 (NFR-01, BR-AVL-01)
S4 Message polling	2,000 open conversations polling every 10 s	about 200 requests/s	p95 < 100 ms (query Q7)
S5 Notifications badge	5,000 signed-in users polling every 30 s	about 170 requests/s	p95 < 50 ms (query Q8)
S6 Mixed day	70% S1/S2, 20% S4/S5, 5% S3, 5% host and admin lists	50 requests/s for 30 min	all targets above; no growth in outbox lag
S7 Abuse	OTP request flood, oversized page sizes, deep pagination	as fast as possible from 5 IPs	limits return 429; API latency for normal users unchanged

Also: `EXPLAIN (ANALYZE, BUFFERS)` snapshots for queries Q1 to Q16 on the dataset, stored with the results and compared release to release; `django-assert-num-queries` style assertions in API tests to stop N+1 queries (each list endpoint has a fixed maximum number of queries).

8. Accessibility and i18n checks for API messages

Check	How	Rule
Every error code has text in en, fr, ar	A test lists every `DomainError` code from a registry and checks a translation exists in each `.po` file	NFR-06, BR-SYS-04
No missing translations	CI runs `makemessages` and fails on a diff; `compilemessages` with errors as failures; no empty `msgstr`	NFR-06
Stable codes	Snapshot test of the error code list: removing or renaming a code is a breaking change (the apps map codes to text)	NFR-13
Language selection	API tests with `Accept-Language: ar`, `fr`, `en`, unknown, and none: correct language or `fr` fallback	NFR-06
Latin digits	Messages and notifications with numbers in Arabic contain digits 0 to 9, never Arabic-Indic digits	NFR-06, research C.7
Arabic plurals	Messages with counts use plural forms (Arabic has 6); test values 0, 1, 2, 3, 11, 100	NFR-06
Templates render	Every notification template renders in the 3 languages with sample data, no missing variables, no raw HTML in SMS	BR-NTF-02
SMS length	SMS texts measured: Arabic texts use UCS-2 (70 characters per segment); critical SMS must fit in 2 segments	cost, NFR-11
Bidirectional text	Mixed Arabic text with Latin values (booking reference, amounts) wrapped with Unicode isolation marks in SMS and email	NFR-06
Plain language	Host-facing reason texts (`reason_code.host_text_*`) reviewed by a native speaker per language before release	MOD-03
Apps	The website and admin keep their Playwright + axe checks for screen readers and right-to-left layout; API messages are shown inside those flows	NFR-13

9. Test data and factories

- One `tests/factories.py` per app with factory_boy factories: `UserFactory`, `HostAccountFactory` (traits `individual`, `agency`, `hotel`, `hostel`, `approved`), `ListingFactory` (traits per vertical, `published`), `RoomTypeFactory` (with nights), `TripDepartureFactory`, `BookingFactory` (traits `pending`, `confirmed`, `completed`), `WalletFactory` (with a top-up posted through the ledger service, never by inserting balance rows).
- Factories that involve money or inventory call **services**, so test data always satisfies the rules (no impossible states created by shortcuts).
- Algerian realism: custom Faker providers for `+213` mobiles (`^\+213[5-7]\d{8}$`), wilaya and commune ids from the geo seed, Arabic and Latin names, DZD amounts in centimes.
- Determinism: fixed random seed per test run (printed in the output); time frozen with time-machine; UUIDv7 generated from the frozen clock.
- Demo data for staging (website demo users Yacine, Amina, Karim; admin staff Nadia; 18 listings, 4 trips) is a separate `seed_demo` command built on the same factories; it refuses to run when `BLADUS_ENV=production` (05-database-schema.md section 9).
- Never production data in tests. Bugs found in production are reproduced with factories.

10. Definition of Done

A change is done when:

- [ ] The requirement ID (`BKG-03`) and rule IDs (`BR-BKG-04`) are in the pull request description.
- [ ] Logic is in services or selectors, not in views, serializers or signals; import-linter passes.
- [ ] Every new or changed business rule has a test named `test_br_<area>_<nn>_...` (section 13).
- [ ] Money, refund or date logic has property tests; booking, inventory or ledger changes have a concurrency test.
- [ ] Database rules are in constraints where possible, with a test that bypasses the service and expects the database to refuse.
- [ ] Permissions: tests for allowed and denied roles (guest, other guest, host, other host, each org role or staff role concerned).
- [ ] API: OpenAPI updated (generated), no unplanned breaking change, error codes translated in en, fr, ar.
- [ ] Migrations follow the expand-then-contract policy and pass the migration checks.
- [ ] Audit events and outbox events written where the rules require them.
- [ ] No personal data in logs; new personal data fields added to the register of processing and the retention table (08-security-privacy.md section 12).
- [ ] Coverage gates pass (section 11); CI fully green.
- [ ] Docs updated if behaviour changed (rules, requirements, this blueprint); no em dash characters in docs.
- [ ] Reviewed and approved by one other developer (two for ledger, security or migrations marked `risky`).
- [ ] Deployed to staging and checked there; flag state documented if behind a feature flag.

11. Coverage targets

Scope	Line coverage	Branch coverage	Note
`services.py` in all apps	**90%**	85%	NFR-10
`ledger` , `pricing` , `bookings` (money and availability)	95%	90%	Plus mutation testing (mutmut) on `core.money` , `pricing` , cancellation policy functions each quarter: target 80% of mutants killed
`selectors.py`	85%		
`apis.py` and permissions	85%		Every endpoint has at least one allow and one deny test
Whole backend	85%		Trend must not go down more than 0.5 point per pull request
Migrations, settings, admin.py	excluded		Covered by migration and smoke tests

Coverage is a floor, not a goal: a test without meaningful assertions does not count in review.

12. Quality gates in CI

Gate	Tool	Blocks merge	Blocks deploy
Lint and format	ruff	yes	yes
Types	mypy (strict on `core` , `ledger` , `pricing` , `bookings`)	yes	yes
App boundaries	import-linter	yes	yes
Migrations	`makemigrations --check` , migration linter, migrate from zero	yes	yes
Unit, service, API tests	pytest with PostGIS	yes	yes
Concurrency tests (short)	pytest	yes	yes
Coverage	coverage.py thresholds (section 11)	yes	yes
Rule traceability	`tests/traceability/check_rules.py` (section 13)	yes	yes
i18n completeness	makemessages diff, compilemessages, error code registry test	yes	yes
OpenAPI valid + no unplanned breaking change	spectacular, oasdiff	yes	yes
Contract fuzzing	schemathesis	yes	yes
Frontend compile against API types	`tsc --noEmit` in both apps	yes	yes
Dependency and secret scanning	pip-audit, npm audit, gitleaks, bandit	yes for high/critical	yes
Image scan	Trivy		yes for critical
Test mode off in production build	build script check		yes
Nightly: long concurrency, full hypothesis, full schemathesis	scheduled	opens an issue	blocks the next production deploy until fixed
Weekly / pre-release: k6 performance	scheduled on staging		yes before a major release if a target is missed

13. Traceability: every rule has a test

Naming convention: the test function name starts with the rule ID in lower case with underscores, then a short description:

```
BR-PAY-04  ->  test_br_pay_04_commission_posted_once_per_booking
BR-AVL-01  ->  test_br_avl_01_parallel_same_nights_only_one_succeeds
BR-CAN-01  ->  test_br_can_01_moderate_policy_half_refund_from_4_to_1_days
BR-ACC-02  ->  test_br_acc_02_sixth_attempt_with_right_code_fails
```

A test that covers several rules carries extra markers: `@pytest.mark.rules("BR-PAY-05", "BR-CAN-03")` . Requirements can be tagged the same way (`@pytest.mark.req("BKG-10")`) for the NFR and requirement report.

Checker (`tests/traceability/check_rules.py` , run in CI):

1. Reads every rule ID from 04-business-rules.md (pattern `BR-[A-Z]+-\d{2}`).
2. Collects test names and `rules` markers with `pytest --collect-only` .
3. Fails if a rule has no test, unless the rule is marked R2 or Later and listed in an allowlist file with a target release.
4. Writes `traceability.csv` (rule, tests, files) as a CI artifact, so the product owner can see which tests protect which rule.

Example of the report:

Rule	Tests	Kind
BR-AVL-01	`test_br_avl_01_exclusion_constraint_refuses_overlap_in_raw_sql` , `test_br_avl_01_parallel_same_nights_only_one_succeeds` , `test_br_avl_01_back_to_back_allowed`	DB, concurrency
BR-PAY-02	`test_br_pay_02_unbalanced_transaction_fails_at_commit` , `test_br_pay_02_app_role_cannot_update_entries` , `test_br_pay_02_any_posting_sequence_balances`	DB, property
BR-CAN-02	`test_br_can_02_example_8a_moderate_three_days_before`	service
BR-SYS-01	`test_br_sys_01_same_key_same_body_returns_same_booking` , `test_br_sys_01_same_key_other_body_422` , `test_br_sys_01_concurrent_same_key_409`	API, concurrency

Check yourself

- [ ] Why do we never use SQLite for backend tests?
- [ ] Give one property that must hold for every refund, whatever the policy and dates.
- [ ] In the "same nights" concurrency test, what exactly must happen to the 49 losing requests?
- [ ] What does schemathesis do that hand-written API tests do not?
- [ ] How does the frontend find out at build time that an API field was removed?
- [ ] What must a test for rule BR-REV-01 be called, at minimum?
- [ ] Which performance scenario checks NFR-02, and what is its pass criterion?
- [ ] Name three items from the Definition of Done that concern personal data or languages.

11. Roadmap: how we build the BLADUS backend

> **In plain words**
> This is the build plan, in eight steps (M0 to M7) up to the public launch, then Release 2 and "Later".
> Each step has a goal, the requirement IDs it delivers, a test to know it is finished, what it waits for, and a rough length in weeks.
> The two apps keep working on demo data the whole time: in each step, a few of their data files switch to the real API, one at a time, behind a switch we can turn back.
> Some steps cannot finish without an answer from a lawyer or an accountant. Those answers are shown as gates at the step that needs them.
> With 2 backend developers and 1 part-time frontend developer, the MVP takes about 29 weeks of work plus 2 weeks of buffer (about 7 months).

Terms used here (see glossary.md):

- **Milestone (M0 to M7)**: a block of work that ends with a check anyone can run ("exit criteria").
- **Exit criteria**: the yes/no checks that say a milestone is done.
- **Gate**: an outside answer (lawyer, accountant, bank, product owner) without which a milestone may not finish. Gate IDs (L1, F3, Q21...) come from 14-open-questions.md and are explained in 13-decisions-needed.md.
- `lib/data` **module**: a file in the website (`frontend/lib/data/`) or admin (`admin/lib/data/`) whose functions read and write demo data today. "Switching" it means its function bodies call the API instead.
- **Feature flag (data source flag)**: a setting per module, for example `DATA_SOURCE_LISTINGS=mock|api` , read on the Next.js server. It lets us switch one module at a time and switch back in seconds if something breaks.
- **Staging**: a copy of the system with demo data only, used for testing. **Production**: the real system with real people's data.
- **Closed beta**: real hosts and guests, invited by us, before the public can sign up.
- **Gantt chart**: a bar chart of tasks over time.

Inputs: 03-requirements.md (147 requirements), 04-business-rules.md, 05-database-schema.md, 01-system-inventory.md section 4 (the `lib/data` bridge), 02-research.md section E. The API contract is written in parallel in `06-api-design.md` and `openapi.yaml` ; operations, security and testing details in `07` to `10` . The full requirement-to-milestone list is in traceability.md.

---

1. Assumptions

#	Assumption	If it is wrong
A1	**Team: 2 backend developers full-time** (Django, PostgreSQL), **1 frontend developer half-time** (Next.js, switches the `lib/data` modules), a product owner who answers within 3 working days, and one of the backend developers acting as tech lead.	With 1 backend developer, multiply backend durations by about 1.8.
A2	Work starts **Monday 2026-11-16**. Dates in the chart are targets, not promises.	Shift every date by the delay.
A3	**Ramadan 2027** (about 8 February to 9 March, VERIFY the exact dates) and the two Eids lower capacity; M3 and M4 are planned a little longer for this.	Add 1 week if the team works reduced hours.
A4	The 147 requirements and DEC-001 to DEC-015 do not change much. New ideas go to R2 by default (risk K-21 in 12-risks.md).	Re-plan at each milestone review.
A5	Staging runs on any cloud with **demo data only**. **No real personal data exists anywhere until production is hosted in Algeria** (gates L1, L2).	Real data abroad may break Law 18-07 (VERIFY WITH A PROFESSIONAL).
A6	The money model stays **M1 (prepaid commission)** for the MVP (DEC-001). Collecting guest money (M2 to M4) is a separate project behind a licence gate.	A different model adds 6 to 10 weeks and a legal track.
A7	Each milestone ends with a 1-hour review: demo on staging, exit criteria checked, next milestone confirmed.	

2. How the apps move from demo data to the API

1. Every `lib/data` module gets one flag (`DATA_SOURCE_<MODULE>`), default `mock` . On staging we turn modules to `api` **one at a time**, in the order of section 4.
2. The API is seeded with the **same demo cast** (Yacine, Amina, Karim, Nadia...) and the same IDs mapped to UUIDs (decision T3), so a module still on demo data can point at records that already live in the API.
3. A module is "switched" when: its functions call the typed API client (NFR-08), every page that uses it passes the frontend smoke test, and the flag has been `api` on staging for 5 working days without a rollback.
4. Once all modules of an app are switched and the launch is done, the demo store and flags are deleted (in M7 for production builds).
5. The website's `listings.ts` switches as a whole in M2 against a simple search endpoint; M3 then upgrades that same endpoint (filters, facets, availability, map) **without** a frontend change. This keeps "one module, one switch".

---

3. Milestones at a glance

Milestone	Name	Weeks	Target dates	Requirements	Main gate
M0	Foundations	3	2026-11-16 to 2026-12-06	18	Q22, T1 to T4, T8, T9
M1	Accounts, staff and host verification	5	2026-12-07 to 2027-01-10	28	L4 (draft list), T6
M2	Catalogue, trips and moderation	4	2027-01-11 to 2027-02-07	23	Q16, Q19, Q24, Q28
M3	Search, availability and quotes	3	2027-02-08 to 2027-02-28	11	T7, Q17
M4	Bookings, cancellations and the commission wallet	5	2027-03-01 to 2027-04-04	27	Q14, Q15, Q18, Q20, Q25, Q26, F5, F6, F7, L3 (for Q18 only)
M5	Messaging and reviews	3	2027-04-05 to 2027-04-25	8	**L1, L2 answered** (to sign the Algerian hosting contract), T1 final
M6	Reports, privacy and hardening	3	2027-04-26 to 2027-05-16	8	L7, L8, L10, L12, Q23

Milestone	Name	Weeks	Target dates	Requirements	Main gate
M7	Production in Algeria, closed beta, launch	3	2027-05-17 to 2027-06-06	3	**L1, L2, L4, L5, L9, L11 cleared** before real data; Q21, Q27
Buffer	Contingency	2	2027-06-07 to 2027-06-20		
Launch	Public launch		target in the week of 2027-06-07, or early July after the baccalaureate exams (Q23)		
R2-A	Online money: gateway, invoices, taxes	6	about 2027-07 to 2027-08	4	**F1, F2, F3, F4**
R2-B	Trust and growth	8	about 2027-07 to 2027-09 (parallel to R2-A)	15	L6 (for PRV-06)
LATER	Behind a legal or business gate	open	not scheduled	2	**L3 + PSP licence** for PAY-10

Total MVP: **126 requirements** in M0 to M7 (109 Must, 17 Should). Counts per milestone and module: traceability.md sections 1 and 2.

---

4. Milestones in detail

M0. Foundations (3 weeks)

- **Goal**: an empty but production-shaped backend: repository, CI, Django 6.1 + DRF + PostgreSQL/PostGIS + Redis + Celery (R-1), sign-up and sign-in that the website can use, the 69 wilayas, platform settings, and the shared conventions (money, time, IDs, errors, languages).
- **Scope**: ACC-01, ACC-02, ACC-03, ACC-04, ACC-07, PAY-12, SRC-04, CFG-01, PRV-01, NFR-05, NFR-06, NFR-07, NFR-08, NFR-09, NFR-10, NFR-11, NFR-12, NFR-13.
- **Exit criteria**:
  - CI runs lint, type checks, tests with coverage gate, dependency scan and OpenAPI generation on every pull request; main branch deploys to staging automatically.
  - A visitor signs up on the website (staging) with email or `+213` phone, gets a code, verifies, signs in, signs out; the session token is revoked on logout (ACC-03).
  - 69 wilayas and 1,541 communes are seeded with `source` and `valid_from` (BR-SRC-02); a test counts them.
  - `platform_settings` v1 holds the default commission rates (1,200 / 1,500 / 1,800 / 1,000 bp) and request TTL 24 h.
  - Error envelope has stable codes and en/fr/ar messages; money helper and UUIDv7 helper have property tests.
- **Dependencies**: decisions T1 (staging host only), T2, T3, T4, T8, T9; Q22 (foreign phone numbers) for ACC-01.
- **`lib/data` switches**: website `auth.ts` (sign-up, sign-in, profile read). Website `lib/algeria.ts` static wilaya list replaced by the geography endpoint (same flag pattern).
- **Gates**: none blocking. Start the lawyer engagement now (L1, L2, L4, L5, L9, L11 take months).

M1. Accounts, staff and host verification (5 weeks)

- **Goal**: staff can sign in to the admin with 2FA and the right permissions; a user can become a host (individual or organisation), upload documents, and be verified by a moderator; organisations have teams.
- **Scope**: ACC-05, ACC-06, ACC-08, ORG-01 to ORG-07, HST-01, HST-02, HST-03, HST-04, HST-06, NTF-04, MED-02, MOD-01, MOD-02, MOD-03, MOD-08, STF-01 to STF-05, CFG-02, CFG-04.
- **Exit criteria**:
  - A staff member is invited, sets a password and TOTP, signs in; a guest session is refused on `/api/v1/admin/*` (STF-05).
  - Every staff mutation writes one `audit_event`; the app's database role cannot UPDATE or DELETE that table (test).
  - A user completes the 12-step individual flow and the 5-step agency flow served from the published config; the draft resumes on another browser.
  - Documents go to the private bucket; a moderator approves them one by one; the host is approved only when all required documents are approved (BR-HST-03).
  - Invitations can be accepted; an organisation always has exactly one owner (DB constraint test).
  - The outbox sends the invitation email through the worker with retries.
- **Dependencies**: M0; T6 (object storage); a **draft** KYC document list per host type from L4 (the list is data, so the final list can change later without code).
- **`lib/data` switches** (in this order): admin `credentials.ts`, `team.ts`, `audit.ts`, `settings.ts`, `users.ts`, `onboarding.ts`, `hosts.ts`, `approvals.ts`; website `organizations.ts`, then the onboarding part of `auth.ts` (`updateUserHostStatus`).
- **Gates**: L4 draft answer expected by the end of M1 (soft). KYC files on staging are fake files only (A5).

M2. Catalogue, trips and moderation (4 weeks)

- **Goal**: approved hosts create listings of every vertical and agencies create trips with departures; everything goes through review before publication; photos are processed safely.
- **Scope**: ACC-09, HST-07, LST-01 to LST-08, LST-10, LST-11, LST-12, TRP-01, TRP-02, TRP-03, TRP-04, TRP-07, AVL-03, MED-01, MED-03, MOD-06, CFG-05.
- **Exit criteria**:
  - An individual host can only create home and vehicle listings; an agency only experiences and trips; a hotel only its property (API and database trigger tests, DEC-014).
  - A submitted listing is `pending_review`; it appears publicly only after approval; editing a key field returns it to review via `listing_revision`.
  - Photos are uploaded directly to quarantine, re-encoded, stripped of GPS data; a home needs 5 photos (Q19).
  - Suspending a host pauses all its listings and trips (MOD-06).
  - The demo catalogue (18 listings, 4 trips) is re-created through the API on staging.
- **Dependencies**: M1 (host accounts, moderation queues); decisions Q16 (hostel vertical), Q19 (minimum photos), Q24 (moderation staffing), Q28 (verticals at launch).
- **`lib/data` switches**: admin `categories.ts`, `listings.ts`; website `favorites.ts`, `listings.ts` (as a whole, against a basic published-only search), `vehicles.ts`, `experiences.ts`, `trips.ts`, `calendar.ts`.
- **Gates**: L5 (whole-home rental by individuals) should be answered before M7; if the answer is negative, the home vertical for individuals is switched off by setting, not by code.

M3. Search, availability and quotes (3 weeks)

- **Goal**: guests find what is free on their dates, in Arabic, French or English, on a list or a map, and get a server-computed price.
- **Scope**: TRP-05, AVL-04, PRC-01, PRC-02, PRC-03, PRC-05, SRC-01, SRC-02, SRC-03, SRC-05, SRC-06.
- **Exit criteria**:
  - Search filters and facet counts match the mock's behaviour on the demo catalogue (the website's existing filter tests pass against the API).
  - Searching "تيزي وزو", "Tizi Ouzou" and "tizi-ouzou" returns the same top results (SRC-03, decision T7).
  - A quote lists nights, per-day prices, discount, cleaning fee, total in whole dinars, has an ID and expires after 30 minutes.
  - First performance run: search p95 measured on 20,000 generated listings (target 500 ms is enforced in M6).
- **Dependencies**: M2; T7 (Arabic search approach); Q17 (one room type per booking).
- **`lib/data` switches**: none new on the website (the `listings.ts` endpoint gets richer); admin `network.ts` and `command-palette.ts` (global search).
- **Gates**: none.

M4. Bookings, cancellations and the commission wallet (5 weeks)

- **Goal**: the heart of the product: instant and request bookings that can never double-book, cancellations by policy, and the prepaid commission wallet on a double-entry ledger (model M1).
- **Scope**: HST-05, AVL-01, AVL-02, AVL-05, BKG-01 to BKG-10, CAN-01 to CAN-04, PAY-01, PAY-02, PAY-03, PAY-04, PAY-05, PAY-07, PAY-08, NTF-01, NTF-02.
- **Exit criteria**:
  - Concurrency test on real PostgreSQL: 50 parallel requests for the same home and nights give exactly one booking; for a 3-room hotel room type, exactly 3 (AVL-01, AVL-02).
  - Same `Idempotency-Key` twice returns the same booking (BKG-10).
  - Expiry and completion jobs run on schedule, not on read (BKG-03, BKG-04).
  - Ledger property test: every transaction sums to zero; the worked examples of 04-business-rules.md section 9 give the exact centimes.
  - A finance staff member records a top-up with a receipt; the same receipt is refused a second time; above the threshold a second person must approve (Q15).
  - Booking confirmed, declined, expired and cancelled each produce an in-app notification and an email or SMS in the user's language.
- **Dependencies**: M3; decisions Q14 (credit floor), Q15 (four-eyes threshold), Q18 (deposit withdrawals), Q20 (host cancellation penalty), Q25 (minimum first deposit), Q26 (reply window for agencies); accountant answers F5, F6, F7 (how deposits and commissions are recorded and receipted); bank or CCP account opened for host top-ups.
- `lib/data` **switches** (in this order): website `wallet.ts`, `reservations.ts`, `notifications.ts`; admin `ledger.ts`, `finance.ts`, `bookings.ts`.
- **Gates**: if Q18 says hosts may withdraw unused deposit, **L3** must confirm this is not a payment service before the withdrawal flow is switched on (VERIFY WITH A PROFESSIONAL). The rest of M4 does not wait.

M5. Messaging and reviews (3 weeks)

- **Goal**: guests and hosts talk before and after booking, with contact details masked until confirmation; guests review completed stays; ratings come from real reviews.
- **Scope**: REV-01, REV-03, REV-04, REV-05, MSG-01, MSG-02, MSG-03, MSG-04.
- **Exit criteria**:
  - Messages sync by cursor; a phone number or CCP number typed before confirmation is masked for the other side (MSG-04).
  - A review is accepted only for a completed booking within 14 days, once (REV-01); listing rating changes after publication.
  - The admin can hide and restore a reported review with a reason.
- **Dependencies**: M4 (completed bookings).
- `lib/data` **switches**: website `conversations.ts`, `reviews.ts`; admin `reviews.ts`.
- **Gates**: **L1 and L2 must be answered by the end of M5** so the Algerian hosting contract (T1) can be signed and set up during M6. This is the most important date in the plan.

M6. Reports, privacy and hardening (3 weeks)

- **Goal**: the admin can run the business (KPIs, exports), users can exercise their data rights, and the system meets its speed and safety targets.
- **Scope**: STF-06, RPT-01, RPT-03, PRV-02, PRV-04, PRV-05, NFR-01, NFR-02.
- **Exit criteria**:
  - Load test: reads p95 under 300 ms and booking create under 600 ms at 50 requests per second on one app node; search p95 under 500 ms with 20,000 listings.
  - A user downloads their data as JSON and asks for deletion; the profile is anonymised while bookings and ledger stay (BR-PRV-02).
  - Retention job deletes ID images 90 days after a decision (period confirmed by L7).
  - Breach runbook written and rehearsed once (5-day ANPDP clock), incident register exists (T5).
  - External security review (OWASP API Top 10) with no open high findings.
- **Dependencies**: M5; L7 (retention periods), L8 (terms and privacy notice texts), L10 (foreign email/SMS providers), L12 (consumer-law checks on the booking flow); Q23 (launch date).
- `lib/data` **switches**: admin `analytics.ts`; the KPIs and exports read real data. All modules of both apps are now on `api` on staging.
- **Gates**: L7, L8, L10, L12 answered. Production hosting (T1) provisioned in Algeria during this milestone.

M7. Production in Algeria, closed beta, launch (3 weeks + 2 weeks buffer)

- **Goal**: run in production in Algeria with real data, first with invited hosts and guests, then open to the public.
- **Scope**: PRV-03, NFR-03, NFR-04 (plus the launch checklist below).
- **Exit criteria**:
  - Production and its backups run in Algerian data centres; a restore drill from the second location meets RPO 15 min and RTO 4 h.
  - Production builds of both apps have no mock store and no demo passwords (fixes the current leak).
  - Closed beta: at least 30 real hosts (mix of the four host types, Q21 wilayas) verified, at least 50 real bookings, no double booking, no ledger mismatch in the daily reconciliation.
  - Support channels and on-call rota ready (Q27); status page and uptime monitoring active.
  - Legal checklist signed by the lawyer: L1, L2, L4, L5, L9, L11 cleared (VERIFY WITH A PROFESSIONAL).
- **Dependencies**: M6; Q21 (launch wilayas), Q23 (launch date), Q27 (support).
- `lib/data` **switches**: none new; flags and demo stores are removed from production builds.
- **Gates**: **no real personal data before L1 and L2 are cleared and production is in Algeria.**

R2-A. Online money: gateway, invoices, taxes (about 6 weeks, after launch)

- **Goal**: hosts top up online by CIB or Edahabia through Chargily; BLADUS issues monthly commission invoices; quotes show tourist tax and VAT where they apply.
- **Scope**: PRC-04, PAY-06, PAY-09, PAY-11.
- **Exit criteria**: a test-mode top-up is credited only after a verified webhook, once per event; invoice numbers have no gaps per year; tax lines match the rates in settings.
- **Dependencies**: launch; Chargily contract signed (start the paperwork during M6, lead time is long).
- `lib/data` **switches**: website `wallet.ts` gets the top-up call (same flag); admin `finance.ts` invoices, `settings.ts` tax rates.
- **Gates**: **F1** (VAT rate on commission), **F2** (taxe de séjour), **F3** (gateway contract and amount units), **F4** (invoice numbering and retention). VERIFY WITH A PROFESSIONAL.

R2-B. Trust and growth (about 8 weeks, in parallel with R2-A)

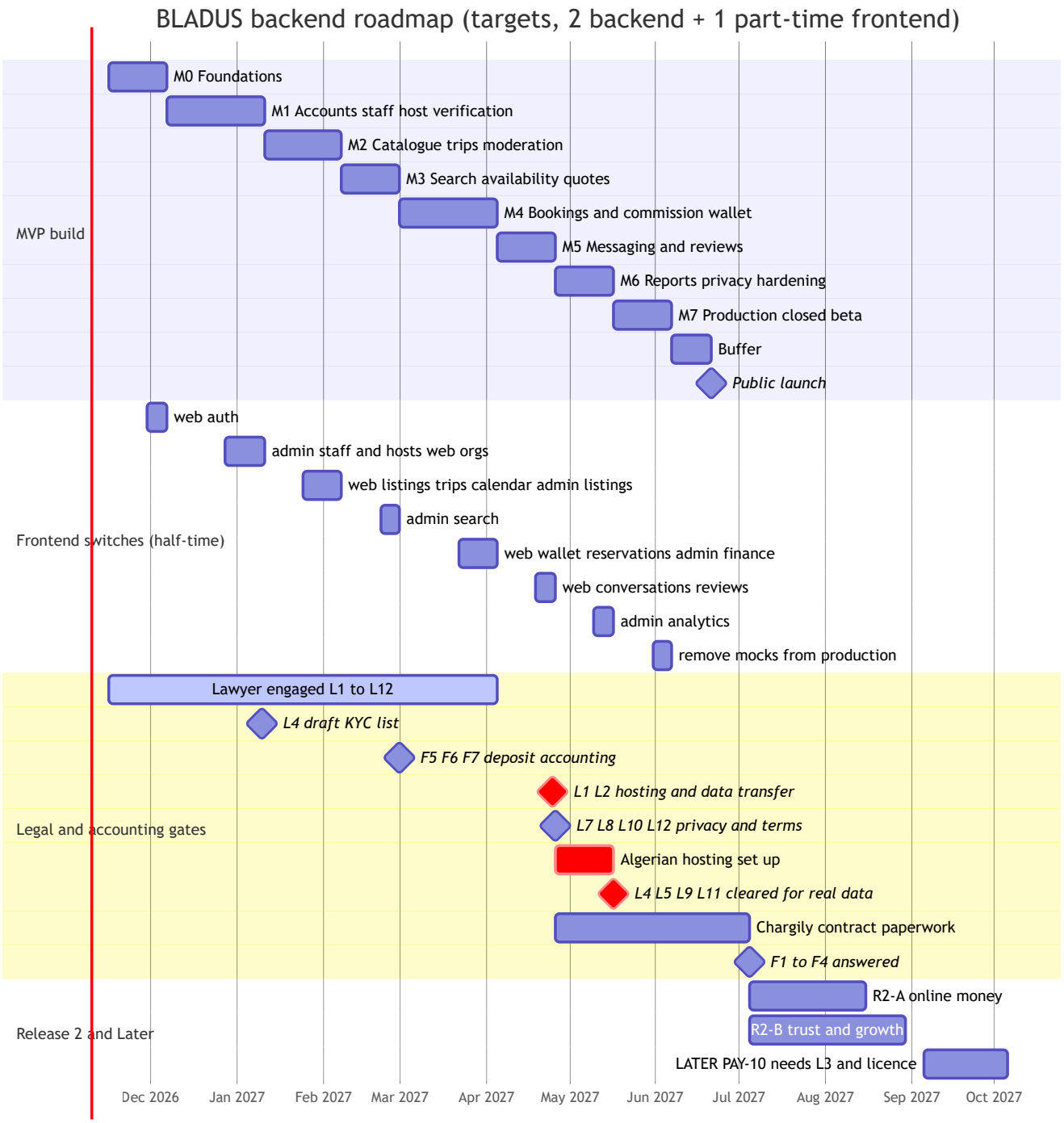
- **Goal**: disputes, host reviews of guests, risk flags, bulk moderation, coupons, featured listings, attachments, notification preferences, onboarding field editor, departure cancellation, foreign guest fields.
- **Scope**: LST-09, TRP-06, BKG-11, REV-02, MSG-05, MSG-06, NTF-03, MOD-04, MOD-05, MOD-07, CFG-03, PRM-01, PRM-02, RPT-02, PRV-06.
- **Exit criteria**: each requirement's acceptance sentence passes on staging; no MVP regression (full test suite green).
- **Dependencies**: launch; one backend developer each for R2-A and R2-B.
- `lib/data` **switches**: admin `promotions.ts` (last mocked admin module); admin messages pages get a new `lib/data` module (none exists today).
- **Gates**: L6 (foreign guest declaration) before PRV-06.

LATER. Behind a legal or business gate (not scheduled)

- **Scope**: PAY-10 (guest payments, escrow, host payouts: money models M2 to M4), AVL-06 (iCal sync).
- **Gate for PAY-10**: **L3** (is it a payment service under Règlement 25-02?) and either a PSP licence (160 million DA minimum capital, 02-research.md C.1) or a licensed partner bank. VERIFY WITH A PROFESSIONAL. The ledger accounts already exist (BR-PAY-11); the work is mostly integration, reconciliation and payouts (estimate 8 to 12 weeks once the gate opens).
- **AVL-06**: needs a new table and SSRF-safe fetching; schedule when hosts ask for channel managers.

---

5. Gantt chart



The LATER bar is a placeholder only: nothing starts until the L3 gate opens.

---

6. Gates by milestone (summary)

Gate	Question (short)	Blocks	Why there
Q22	Can foreign phone numbers verify?	M0	Sign-up rules

Gate	Question (short)	Blocks	Why there
L4 (draft)	KYC documents per host type	M1 (soft), M7 (hard)	Document types are data; final list before real hosts
Q16, Q19, Q24, Q28	Hostel vertical, minimum photos, moderation staffing, verticals at launch	M2	Catalogue rules
Q17	One room type per booking	M3	Quote shape
Q14, Q15, Q18, Q20, Q25, Q26	Credit floor, four-eyes threshold, deposit withdrawal, host cancel penalty, first deposit, agency reply window	M4	Wallet and booking rules
F5, F6, F7	Receipts or invoices for deposits, accounting of deposits, collection account	M4	Before real top-ups in M7
L3 (narrow)	Are deposit withdrawals a payment service?	M4 (withdrawal flow only)	Q18
L1, L2	Hosting in Algeria, data transfer abroad	**End of M5**	Hosting contract lead time
L7, L8, L10, L12	Retention, terms text, foreign providers, consumer law	M6	Privacy jobs, consent versions
L4, L5, L9, L11	KYC, whole-home rental, agency licence for BLADUS, company formalities	**M7**	Before real hosts
Q21, Q23, Q27	Launch wilayas, launch date, support	M7	Beta and launch
F1 to F4	VAT, tourist tax, gateway contract, invoices	**R2-A**	Online money
L6	Foreign guest declaration	R2-B	PRV-06
L3 + licence	Collecting guest money	**LATER**	Models M2 to M4

All legal and tax gates: VERIFY WITH A PROFESSIONAL.

7. What can make this plan slip

The top schedule risks (full register in 12-risks.md): late legal answers (K-23), the Algerian hosting set-up (K-18), the mock-to-API switch breaking pages (K-13), the half-time frontend developer becoming the bottleneck (K-20), scope creep from R2 into the MVP (K-21), and the baccalaureate exam period around the launch (K-17). The 2-week buffer covers one of these, not all.

Check yourself

- [ ] Which milestone is the first where a guest can actually book, and which two milestones must be done before it?
- [ ] Why is the end of M5 the most important legal date in the plan?
- [ ] What does "one module at a time behind a flag" mean, and how do we roll back a module that breaks?
- [ ] Why does the website's `listings.ts` switch in M2 even though full search arrives in M3?
- [ ] Which accountant answers block online top-ups, and in which milestone?
- [ ] Can staging contain real people's data? Why not?
- [ ] What has to be true before PAY-10 (guest payments and payouts) can even be scheduled?
- [ ] If you had only one backend developer, roughly how long would the MVP take?

12. Risks: what could go wrong, and what we do about it

> **In plain words**
> This is the list of things that could hurt BLADUS while we build and launch the backend: legal, money, fraud, technical, operations, team and scope.
> Each risk gets two marks from 1 to 5: how likely it is, and how bad it would be. Multiplied, they give a score from 1 to 25.
> Each risk has an owner (a role, not a name), what we do to reduce it, and the early sign that tells us it is happening.
> The heat map at the end shows the dangerous corner at a glance. We review this list at every milestone review (11-roadmap.md).
> Legal and tax points are engineering reading, not advice: VERIFY WITH A PROFESSIONAL.

Terms used here (see glossary.md):

- **Risk register**: a table of risks, kept up to date, with owners and actions.
- **Likelihood (L)**: 1 rare, 2 unlikely, 3 possible, 4 likely, 5 almost certain (during the MVP build and first 6 months after launch).
- **Impact (I)**: 1 small annoyance, 2 a few days lost, 3 a feature late or a few unhappy users, 4 launch delayed or money lost, 5 legal stop, data breach or loss of trust.
- **Score**: L x I. **15 to 25 high**, **8 to 14 medium**, **1 to 7 low**.
- **Mitigation**: what we do now to make the risk smaller or less harmful.
- **Trigger / early signal**: what we would see first if the risk starts to happen.
- **SMS pumping**: fraud where bots request many one-time codes to numbers that earn the attacker money.
- **SIM swap**: a criminal gets a victim's phone number moved to a new SIM card and receives their codes.
- **Exclusion constraint**: a PostgreSQL rule that refuses two rows whose ranges overlap (used against double booking).
- **PSP**: payment service provider, a licensed company allowed to hold and move other people's money.

Owner roles: **PO** product owner, **TL** tech lead, **BE** backend developers, **FE** frontend developer, **LAW** external lawyer, **ACC** external accountant, **FIN** BLADUS finance staff, **MOD** moderation lead, **SUP** support lead.

---

1. Risk register

ID	Risk	L	I	Score	Owner	Mitigation	Trigger / early signal	Linked
K-01	**Legal: hosting.** Law 18-05 art. 8 requires the e-commerce site to be hosted in Algeria with `.com.dz`; production abroad could be unlawful (VERIFY WITH A PROFESSIONAL)	3	5	**15**	PO + LAW	Plan production in Algeria from day one (R-5); staging with demo data only; ask L1 early; keep the deployment portable (containers, no cloud-only services)	Lawyer says the API server is covered too; Algerian provider cannot give a date	PRV-03, BR-PRV-04, R-5, R-6, L1, T1
K-02	**Legal: data transfer abroad** through foreign services (email, SMS fallback, error tracking, CI logs, backups) without ANPDP authorisation under Law 18-07 / 25-11 (VERIFY WITH A PROFESSIONAL)	4	4	**16**	TL + LAW	List every processor (T8); self-host error tracking; keep personal data out of logs; local SMS aggregator first; no backups abroad; ask L2 and L10	A new SaaS appears in the stack; personal data found in logs or error reports	PRV-03, NFR-09, BR-PRV-04, L2, L10, T8
K-03	**Legal: PSP licence.** Collecting guest money (M2 to M4), or even refunding host deposits, may be a payment service under Règlement 25-02 (VERIFY WITH A PROFESSIONAL)	2	5	10	PO + LAW	Launch with M1 only (DEC-001); M2 to M4 switched off by setting (BR-PAY-11); deposit withdrawals only after L3	Pressure to "just take card payments"; a host asks for a refund of deposit	PAY-10, BR-PAY-11, DEC-001, Q18, L3
K-04	**Legal: KYC.** Wrong document list per host type, or storing ID images longer than allowed, or without the required declaration (VERIFY WITH A PROFESSIONAL)	3	4	12	PO + LAW	Document types as data (change without code); private bucket, 5-minute signed URLs, every view audited; delete ID images after 90 days (BR-PRV-05) pending L7	Moderators ask for documents not on the list; L4 answer differs from the draft	HST-03, MED-02, MOD-08, BR-HST-03, BR-PRV-05, L4, L7
K-05	**Legal: tourism rules.** Agencies need a licence; "chez l'habitant" has room and guest caps; whole-home rental by individuals has no clear frame; BLADUS itself might need a tourism registration to sell trips (VERIFY WITH A PROFESSIONAL)	3	4	12	PO + LAW	Agency licence number required at verification (BR-TRP-01); verticals and host types switchable by setting; ask L5 and L9	Ministry or wilaya warning; a competitor shut down; lawyer flags L9	TRP-01, HST-07, LST-01, BR-TRP-01, BR-HST-05, L5, L9
K-06	**Tax and invoices.** Wrong VAT rate on commission (19% or 9%), no receipts for deposits at launch, missing tourist tax handling (VERIFY WITH A PROFESSIONAL)	3	3	9	ACC + FIN	Invoices in R2 behind F1 and F4; ask F5 now about receipts at launch; keep every ledger line with host NIF so invoices can be issued backwards	Accountant asks for documents we cannot produce; a host asks for an invoice	PAY-11, PRC-04, BR-PAY-12, BR-PRC-04, F1, F2, F4, F5
K-07	**Payments: manual top-ups.** Typing errors, the same receipt credited twice, fake bank or CCP receipts	3	3	9	FIN	Receipt reference unique per method; four-eyes above a threshold (Q15); daily reconciliation against the bank or CCP statement (F7); audit of every top-up	Reconciliation mismatch; a top-up reversed by the bank	PAY-05, BR-PAY-07, STF-03, Q15, F7
K-08	**Payments: gateway (R2).** Chargily contract slow; amount units (dinars or centimes) misread; webhooks missed or replayed	3	3	9	TL + FIN	Start paperwork in M6; contract test on amount units (F3); store raw webhook, verify signature, dedupe by event id (BR-SYS-02); second gateway (SlickPay) as fallback	Contract not signed 4 weeks before R2-A; sandbox amounts off by 100	PAY-06, PAY-09, BR-PAY-08, BR-SYS-02, R-7, F3
K-09	**Fraud: off-platform payment scams and fake listings.** Fake host asks for a CCP transfer before arrival; stolen photos	4	4	**16**	MOD + SUP	Review before publication (DEC-003); contact masking until confirmation (BR-MSG-01); perceptual photo hash for duplicates (BR-MED-01); risk flags in R2 (MOD-07); guest safety tips in the UI	Reports mentioning "paid by CCP, host vanished"; duplicate photo hashes across hosts	LST-02, MSG-04, MED-01, MOD-07, BR-MSG-01, BR-MOD-04
K-10	**Fraud: SMS pumping and OTP abuse** driving SMS cost up	3	3	9	TL	Limits per phone, IP and device (BR-ACC-02); daily SMS budget alarm (NFR-11); block premium and foreign ranges unless Q22 allows; email OTP where possible	SMS spend per day above budget; many codes to one prefix	ACC-02, NFR-11, BR-ACC-02, Q22
K-11	**Fraud: account takeover** of hosts or staff (SIM swap, password reuse)	2	4	8	TL	2FA for staff and for hosts before payout detail changes (BR-ACC-04); 72 h hold after a change (BR-PAY-09); notify old email and phone	Payout account changes followed by support complaints	ACC-05, HST-05, BR-ACC-04, BR-PAY-09
K-12	**Technical: double booking** if the database guarantees are built wrong (exclusion constraint, inventory rows, holds)	2	5	10	BE	Constraints in the database, not only in Python (ADR-002); concurrency tests on real PostgreSQL in CI (M4 exit); idempotency keys on create (BKG-10)	Any concurrency test flaky; support ticket "two guests same night"	AVL-01, AVL-02, AVL-05, BKG-10, BR-AVL-01, BR-AVL-02, BR-SYS-01
K-13	**Technical: mock-to-API switch** breaks pages: different IDs, enum names, field names (58 vs 69 wilayas, statuses), behaviour the mock did silently	4	3	12	FE + TL	One module at a time behind a flag (roadmap section 2); demo cast seeded with stable IDs (T3); enum mapping from 05-database-schema.md section 8; typed client (NFR-08); 5 days on staging before a module counts as switched	A module is rolled back twice; contract tests fail after an API change	NFR-08, NFR-13, T3, T9, schema section 8

ID	Risk	L	I	Score	Owner	Mitigation	Trigger / early signal	Linked
K-14	**Technical: performance.** Search with facets, availability and map is slow on small Algerian VPS sizes	3	3	9	BE	Search table `listing_search_doc` (ADR-011); indexes from schema section 7; load test from M3, enforced in M6; cache facet counts briefly	Search p95 above 500 ms on staging data	NFR-01, NFR-02, SRC-01, SRC-02
K-15	**Technical: Arabic search quality.** Spelling variants (alef forms, ta marbuta), Latin transliterations (Tizi Ouzou, Tizi-Ouzou), Darija names; PostgreSQL trigram works less well on Arabic	4	3	12	BE	Normalise Arabic text before indexing; alias table for place names in three scripts from the geography seed; test set of 50 real queries (T7)	Zero-result rate above 10% for Arabic queries	SRC-03, SRC-04, T7
K-16	**Operations: SMS deliverability and cost.** Codes not arriving on some operators (Mobilis, Djezzy, Ooredoo), sender ID registration delays, about USD 0.27 per SMS abroad	4	4	**16**	TL + PO	`SmsProvider` interface with a local aggregator first and a fallback (R2 C.2); email and WhatsApp codes where possible; measure delivery per operator from the beta; budget alarm	Code verification rate below 85%; complaints "no code received"	ACC-01, ACC-02, NTF-02, BR-NTF-02, DEC-012
K-17	**Operations: internet restrictions during the baccalaureate exams** (usually June; mobile data and social networks cut or slowed nationwide) during launch or high season	4	3	12	PO	Do not launch in the exam weeks (Q23); SMS (not internet) for critical alerts; job schedules tolerate delays (expiry at deadline even if nobody is online); communicate in advance	Official exam calendar published; cuts announced	NFR-03, BKG-03, Q23
K-18	**Operations: single hosting provider in Algeria.** Outage, slow support, few managed services (no managed PostgreSQL or object storage), contract lead time	3	4	12	TL	Choose provider by end of M5 (T1); infrastructure as code so we can move; self-managed PostgreSQL with streaming replica; MinIO for files (T6); status page	Provider cannot give a date; repeated incidents on staging trial	NFR-03, NFR-04, PRV-03, T1, T6, R-5
K-19	**Operations: backups.** No second Algerian location, or restores never tested	3	4	12	TL	Second Algerian data centre or provider for encrypted dumps (T10); monthly restore drill (NFR-04); point-in-time recovery 7 days	A restore drill fails or is skipped	NFR-04, PRV-03, T10
K-20	**Team and skills.** Two backend developers: bus factor of 1 on ledger and availability; PostGIS, Celery and allauth headless new to the team; half-time frontend developer is the bottleneck for switches	4	4	**16**	PO + TL	Pair on ledger and booking code; written ADRs; code review by the other developer always; spike weeks in M0 for allauth and PostGIS; frontend switch tasks planned in the chart; budget for a second frontend half-time in M4	Switch tasks slip twice; one developer is the only reviewer of a module	NFR-10, roadmap A1
K-21	**Scope creep.** R2 items (online payment, coupons, realtime chat, disputes) or new admin wishes pulled into the MVP	4	3	12	PO	New ideas go to R2 by default; any MVP addition must remove something of equal size; milestone reviews check the scope list	Requirements count in MVP grows above 126; milestone slips by more than a week	03-requirements.md section 5, DEC-001
K-22	**Moderation backlog at launch.** Listings stuck in `pending_review`, hosts give up	3	3	9	MOD	Queues with waiting time and priority (MOD-01); reason catalogue (MOD-03); staffing decision (Q24); fast track for good hosts later (DEC-003)	Oldest item in the listing queue older than 2 days	MOD-01, MOD-03, LST-02, BR-MOD-01, Q24
K-23	**Legal answers arrive late** (L1, L2, L4, L5, L9, L11) and block production	4	4	**16**	PO	Engage the lawyer in M0; give the lawyer this blueprint's questions in writing; dated gates in the roadmap; design so that a negative answer is a setting, not a rewrite	No written answer to L1 or L2 by mid-M5	L1 to L12, roadmap section 6
K-24	**Calendar slip** from Ramadan, Eids and year-end holidays	4	2	8	PO + TL	Longer M3 and M4; 2-week buffer; no launch in Ramadan	Velocity in M3 lower than 70% of M2	roadmap A3
K-25	**Business: hosts do not pay back negative balances** under model M1	3	2	6	FIN	Credit floor per host type (BR-PAY-10, Q14); below 0 instant booking switches to request; dashboard of negative wallets (PAY-07)	Total negative balance grows month over month	PAY-07, BR-PAY-10, Q14, Q25
K-26	**Security breach**, especially of KYC documents, with a 5-day ANPDP notification duty	2	5	10	TL	OWASP API Top 10 controls (NFR-05); private documents with short signed URLs (MED-02); breach runbook and drill (PRV-05); external review in M6	Unusual document view counts in the audit log; dependency alert	NFR-05, MED-02, PRV-05, STF-03, BR-MED-02
K-27	**Geography data wrong** for the new wilayas 59 to 69	3	2	6	BE	Seed with `source` and `valid_from` (BR-SRC-02); check against the official journal before M7; correction is a data migration	Hosts in new wilayas report the wrong name or commune	SRC-04, BR-SRC-02, DEC-013
K-28	**Diaspora and foreign guests cannot verify** because phone OTP accepts only Algerian mobiles (DEC-012)	4	3	12	PO	Decide Q22 in M0 (email or international number verification for non-Algerian numbers, with stricter limits)	Sign-up drop-off from abroad; support requests from France and Canada	ACC-01, ACC-02, BR-ACC-01, DEC-012, Q22
K-29	**Email deliverability** (spam folders) and low email use by some hosts	3	2	6	TL	SPF, DKIM, DMARC on the sending domain; SMS for critical events (BR-NTF-02); in-app notifications always	Bounce or complaint rate above 2%	NTF-02, ACC-04, BR-NTF-02
K-30	**Map and geocoding services** (OpenFreeMap tiles, Nominatim 1 request per second) limit or change terms	2	2	4	BE	Server-side geocoding with cache; commune plus pin, no autocomplete on public Nominatim; self-host tiles if needed	Tile errors or 429 responses	SRC-02, LST-06
K-31	**Money rounding errors** between centimes and whole dinars, or between quote and booking	2	4	8	BE	Centimes everywhere (BR-PAY-01); one rounding helper; property tests and the worked examples in 04-business-rules.md as tests	Ledger reconciliation off by a few centimes	PAY-12, PRC-05, PAY-03, BR-PAY-01, BR-PAY-04
K-32	**Business: hosts refuse to pre-pay** a commission deposit (model M1), so supply stays low	3	4	12	PO	Credit floor so hosts can start before paying (Q14, Q25); simple top-up by CCP or BaridiMob; online top-up in R2-A; clear commission rates	Few top-ups during the beta; hosts stay in request mode	PAY-02, PAY-05, PAY-07, DEC-001, Q25

32 risks: 6 high, 22 medium, 4 low.

2. Heat map (likelihood x impact)

Cells list risk IDs. Bold cells are high (score 15 or more).

Likelihood \ Impact	1 small	2 days lost	3 feature late	4 launch late or money lost	5 legal stop or breach
5 almost certain					
4 likely		K-24	K-13, K-15, K-17, K-21, K-28	**K-02, K-09, K-16, K-20, K-23**	
3 possible		K-25, K-27, K-29	K-06, K-07, K-08, K-10, K-14, K-22	K-04, K-05, K-18, K-19, K-32	**K-01**
2 unlikely		K-30		K-11, K-31	K-03, K-12, K-26
1 rare					

3. The six high risks, and what to watch first

Risk	First action	When
K-01 Hosting in Algeria	Send L1 to the lawyer; shortlist ICOSNET and Djezzy Cloud (T1)	M0
K-02 Data transfer abroad	Processor list (T8) reviewed by the lawyer (L10)	M0, then M6
K-09 Off-platform scams	Contact masking and review before publication ship in the MVP	M2, M5
K-16 SMS deliverability	Contract a local aggregator and test all three operators on staging	M0
K-20 Team and skills	Pairing rule and review rule written in the contributing guide	M0
K-23 Late legal answers	Written question list to the lawyer with the roadmap dates	M0

Medium risks with the highest impact on dates: K-13 (switch breaks), K-18 (hosting set-up), K-21 (scope creep), K-17 (exam period).

Check yourself

- [ ] How is the score calculated, and which band is "high"?
- [ ] Which two risks are about where data lives, and what single decision reduces both?
- [ ] Why is double booking (K-12) only "unlikely" even though its impact is 5?
- [ ] What is the early signal that SMS pumping (K-10) is happening?
- [ ] Which risk explains why the launch should not fall in the baccalaureate exam weeks?
- [ ] Name two ways we keep the mock-to-API switch (K-13) from breaking the apps.
- [ ] Who owns the scope creep risk, and what rule do they apply to new ideas?

13. Decisions needed: who must decide what, and by when

> **In plain words**
> Building the backend needs some answers we cannot invent: product choices, legal checks, tax rules and technical picks.
> This page lists every decision still open, grouped by who decides: you (product owner), a lawyer, an accountant or bank, and the tech lead.
> For each one: the options, our recommendation, the milestone it blocks, and what happens if nobody decides.
> Decisions already taken (DEC-001 to DEC-015) are repeated first so you can see the whole picture.
> Legal and tax items are questions, not advice: VERIFY WITH A PROFESSIONAL.

Terms used here (see glossary.md):

- **Decision**: a choice that changes what we build. Once taken it gets a DEC number in 03-requirements.md section 2.
- **Deadline (blocks)**: the roadmap milestone (11-roadmap.md) that cannot finish without the answer.
- **Default**: what we build if nobody answers by the deadline. Every default here can be changed later by a setting or a data change unless marked "costly to change".
- **Setting**: a value in `platform_settings` that staff can change without new code.
- **PSP**: payment service provider (a licensed company allowed to move other people's money).
- **ANPDP**: Algeria's personal data protection authority.

IDs: Q (product) and L (legal) and F (finance) come from 14-open-questions.md. T (tech lead) items are new on this page.

---

1. Decisions already made (for reference)

ID	Decision	Date
DEC-001	Money model M1: prepaid commission; guests pay hosts directly; ledger ready for M2 to M4	2026-11-04
DEC-002	Commission base = accommodation subtotal (nights x price after discount), no cleaning fee or taxes; trips and experiences: seats x price	2026-11-04
DEC-003	New listings and trips need staff review before publication	2026-11-04
DEC-004	One person can be an individual host and an organisation member, with an active-organisation switcher	2026-11-04
DEC-005	Hotel is its own search vertical, counted in the home family for analytics	2026-11-04
DEC-006	Trips are their own tables with departures	2026-11-04
DEC-007	Keep the 12-step individual onboarding	2026-11-04
DEC-008	Request-to-book reply window 24 h, as a setting	2026-11-04
DEC-009	In an organisation, owner, admin and manager may answer requests	2026-11-04
DEC-010	Host earnings count at completion	2026-11-04
DEC-011	Three cancellation policies (flexible, moderate, strict) stored as time windows	2026-11-04
DEC-012	Guests verify their phone before the first booking	2026-11-04
DEC-013	Show 69 wilayas (codes 59 to 69 to confirm)	2026-11-04
DEC-014	Individuals publish homes and vehicles; agencies experiences and trips; hotels and hostels their property	2026-11-04
DEC-015	8 experience types and 8 trip types, one shared list	2026-11-04

Technical choices already taken in research: R-1 to R-11 in 02-research.md section E (Django 6.1 + DRF + PostgreSQL/PostGIS + Celery/Redis + allauth headless; database-enforced availability; centimes and double-entry ledger; polling for messages; PostgreSQL search).

---

2. Summary

Who decides	Count	Earliest deadline	Most urgent
Product owner	15 (Q14 to Q28)	M0	Q22 foreign phone numbers, Q28 verticals at launch
Lawyer	16 (L1 to L16)	end of M5 (L1, L2)	**L1, L2** hosting and data transfer
Accountant or bank	7 (F1 to F7)	M4 (F5 to F7)	F5 to F7 how to record and receipt deposits
Tech lead	11 (T1 to T11, T5 already closed)	M0	T1 hosting, T3 mock-to-API switch
Total	**49 (48 still open)**		

---

3. Product owner

Q14 to Q20 are the proposals already built into the schema (Phase 5); the default is the proposal. Q21 to Q28 are new (found while planning).

ID	Decision	Options	Recommendation	Deadline (blocks)	If no decision
Q14	How far below zero may a host's commission deposit go before instant booking switches off?	(a) 0 DZD; (b) 20,000 DZD for every host type; (c) per host type (e.g. individual 10,000, agency/hotel/hostel 50,000)	(c), starting at 20,000 for all, per host type in settings (BR-PAY-10)	M4	20,000 DZD for all; changeable by setting

ID	Decision	Options	Recommendation	Deadline (blocks)	If no decision
Q15	Above which top-up amount must a second finance person approve?	(a) never; (b) 500,000 DZD; (c) every top-up	(b) 500,000 DZD (BR-PAY-07)	M4	500,000 DZD; changeable by setting
Q16	Hostels: own vertical shown under the "Hotels" search tab?	(a) own vertical under Hotels tab; (b) a hotel subtype; (c) own tab	(a) (ADR-005)	M2	(a); changing later needs a data migration (costly to change)
Q17	One room type per booking at launch (several rooms of that type allowed)?	(a) one type per booking; (b) mixed types in one booking	(a); mixed in a later release (ADR-003)	M3	(a)
Q18	Can a host withdraw unused deposit money, and how fast?	(a) no withdrawals; (b) on request, paid within 7 days after the 72 h hold; (c) only when the host leaves BLADUS	(c) at launch, (b) after L3 confirms it is not a payment service	M4 (withdrawal flow only)	(c): withdrawal only on closing the host account, handled manually by finance
Q19	Minimum number of photos	(a) 5 for homes, 3 per room type; (b) 3 for all; (c) none	(a) (BR-MED-01)	M2	(a); a setting
Q20	Penalty when a host cancels	(a) none; (b) moderation case after 3 cancellations in 12 months; (c) fee	(b), no fee (BR-CAN-03)	M4	(b)
Q21	**New.** Launch geography: all 69 wilayas or a few pilot wilayas?	(a) all 69; (b) pilot (e.g. Algiers, Oran, Constantine, Béjaïa, Tizi Ouzou, Ghardaïa, Tamanrasset); (c) all, but marketing in pilots only	(c): no code limit, beta hosts recruited in pilot wilayas	M7 (beta recruitment)	(a) with no focus; beta harder to staff
Q22	**New.** How do guests without an Algerian mobile verify (diaspora, foreigners)? DEC-012 and BR-ACC-01 only accept `+213[5-7]`	(a) Algerian mobile only; (b) any international mobile by SMS (higher cost and fraud); (c) email code for non-Algerian numbers plus stricter booking limits	(c)	M0 (sign-up)	(a): diaspora guests cannot book (risk K-28)
Q23	**New.** Public launch date relative to the baccalaureate exams (internet cuts, usually June) and the summer season	(a) first week of June 2027, before the exams; (b) early July 2027, after the exams; (c) September	(b): closed beta in late May and June, public launch early July at the start of the summer season	M6 (to plan beta and marketing)	Launch falls in exam weeks; poor first impression (risk K-17)
Q24	**New.** Who moderates at launch, and what review time do we promise hosts?	(a) 1 moderator, 2 working days; (b) 2 moderators, 1 working day; (c) outsourced	(b) during the beta and first 3 months	M2	Queues grow, hosts give up (risk K-22)
Q25	**New.** Must a host top up a first deposit before going live, or before instant booking?	(a) no, credit floor only; (b) first deposit before instant booking; (c) first deposit before publishing	(a) to grow supply, with the credit floor of Q14	M4	(a)
Q26	**New.** Is a short request reply window (24 h) right for agencies whose trips depart weeks later?	(a) 24 h for all; (b) per host type setting; (c) per listing	(b): keep 24 h default, allow 48 h for agencies	M4	(a) for all
Q27	**New.** Support channels and hours at launch	(a) email and in-app only; (b) plus phone line; (c) plus WhatsApp Business	(c) WhatsApp Business with templates, office hours 9:00 to 18:00 Sunday to Thursday, on-call for safety issues	M7	No clear channel; scams handled late
Q28	**New.** Which verticals open at launch?	(a) all six (homes, hotels, hostels, experiences, vehicles, trips); (b) stays and trips first, vehicles and experiences later; (c) homes only	(a) if L5 and the vehicle rules (L4) allow; otherwise switch the blocked vertical off by setting	M2	(a), with a legal risk for verticals whose rules are unclear

4. Lawyer (VERIFY WITH A PROFESSIONAL)

ID	Decision	Options	Recommendation (engineering view)	Deadline (blocks)	If no decision
L1	Does Law 18-05 art. 8 ("hosted in Algeria", `.com.dz`) apply to BLADUS as an intermediary, and to the API server too?	(a) yes, everything in Algeria; (b) only the public website; (c) does not apply	Plan for (a)	**End of M5** (hosting contract), hard gate M7	We host everything in Algeria anyway (safe default, higher ops cost)
L2	Does storing personal data or backups abroad need ANPDP authorisation (Law 18-07 / 25-11)? Which declarations before launch?	(a) all data and backups in Algeria; (b) abroad with authorisation	(a), and file the declarations before M7	**End of M5**, hard gate M7	No real personal data can be processed: launch blocked
L3	Is collecting guest money and paying hosts a payment service (Règlement 25-02)? Is the prepaid deposit model safe, including deposit withdrawals?	(a) M1 safe, withdrawals safe; (b) M1 safe, withdrawals only on closure; (c) partner bank or PSP needed	Confirm (a) or (b) for MVP; (c) for M2 to M4	M4 for withdrawals (Q18); LATER for PAY-10	Withdrawals only on account closure; PAY-10 stays off
L4	Which documents must each host type provide (individual home, chez l'habitant, agency, hotel, hostel, vehicle rental, guide)?	List per host type	Draft list from 02-research.md C3.5, confirmed by the lawyer	Draft by end of M1, final before M7	Draft list used for real hosts: compliance risk (K-04)
L5	Is whole-home short-term rental legal for individuals, and what must BLADUS check?	(a) legal with checks; (b) only "chez l'habitant"; (c) not allowed	Ask for the list of checks	M7	Individual home vertical may need to be switched off by setting
L6	Duty to declare foreign guests (Law 08-11 art. 29): whose duty, what must BLADUS store?	(a) host's duty, BLADUS reminds; (b) BLADUS must store and transmit	(a) with nationality captured only when needed	R2-B (PRV-06); earlier if (b)	Host reminder text only
L7	Retention periods for IDs, bookings, messages, invoices. Is a DPO required?	Periods per data type	ID images 90 days after decision; accounting 10 years; messages 2 years after last booking	M6 (retention jobs)	Proposed periods used (BR-PRV-05)
L8	**New.** Terms of use, host terms and privacy notice texts in ar/fr/en, versioned for consent records	Lawyer drafts or reviews our draft	Lawyer reviews our draft	M6 (consent versions live before beta)	No valid terms at beta: launch blocked
L9	**New.** Does BLADUS itself need a tourism registration or travel agency licence (Law 99-06, Décret 17-161) because it sells agency trips and experiences?	(a) no, pure intermediary; (b) licence needed; (c) only if BLADUS takes payment	Ask; keep BLADUS out of the money flow for trips (M1)	M7	Trips vertical may need to be switched off at launch
L10	**New.** Do foreign service providers that touch personal data (transactional email, international SMS fallback, error tracking, uptime monitoring) count as transfer abroad?	(a) yes, all need authorisation; (b) allowed with contract clauses; (c) allowed for some data types	Assume (a); use Algerian or self-hosted services where possible (T8)	M6	We self-host or use Algerian providers only (higher cost)
L11	**New.** Company formalities before launch: commerce register activity code for an online marketplace, CNRC e-commerce registration, `.com.dz` domain holder	Checklist	Lawyer provides the checklist in M0	M7	Launch blocked by missing registration

ID	Decision	Options	Recommendation (engineering view)	Deadline (blocks)	If no decision
L12	**New.** Does the booking flow meet Law 18-05 art. 10 to 13 (offer content, 3-step order: review, confirm, contract) for both instant and request bookings?	(a) current flow is enough; (b) changes needed	Ask with screenshots of the flow	M6	Risk of a non-compliant checkout; late UI changes
L13	**New (from 08).** Exact content, addressee and start of the 5-day breach notification clock; duty to inform affected users	(a) clock starts at detection; (b) at confirmation	Ask for the official ANPDP procedure and template	M6	Runbook timing may be wrong in a real breach
L14	**New (from 08).** Legal basis for each processing in the register (contract, consent, legal obligation, legitimate interest as recognised by Law 18-07)	per processing line in 08 section 12.2	Lawyer reviews the register table line by line	M6	Register incomplete at the ANPDP declaration
L15	**New (from 08).** Deadlines to answer access, correction and deletion requests, and which identity checks are allowed first	(a) 30 days; (b) shorter legal deadline	Use 30 days until confirmed	M6	Requests answered late
L16	**New (from 08).** May BLADUS publish a reviewer's first name and the host's name without separate consent?	(a) yes under the terms; (b) separate consent; (c) initials only	(c) initials until answered	M6	Possible complaint from a reviewer

5. Accountant or bank (VERIFY WITH A PROFESSIONAL)

ID	Decision	Options	Recommendation (engineering view)	Deadline (blocks)	If no decision
F1	VAT on BLADUS commission: 19% or 9% (tourism)? Who issues which invoice?	19%; 9% until end of 2027; depends on host type	Store the VAT rate per invoice line so either works	R2-A (PAY-11)	Invoices not issued; R2-A blocked
F2	Taxe de séjour: does BLADUS collect it or only show it?	(a) show only, host pays; (b) BLADUS collects	(a) for M1	R2-A (PRC-04)	Tax lines stay hidden
F3	Chargily or SlickPay: contract terms, settlement delays, amount units in their APIs	Chargily first, SlickPay second (R-7)	Sign Chargily; test amount units in sandbox	R2-A (PAY-06); paperwork starts M6	Online top-up delayed
F4	Invoice numbering and retention for commission invoices	Yearly sequence without gaps; 10 years	As BR-PAY-12	R2-A (PAY-11)	Invoices not issued
F5	**New.** Before PAY-11 (R2), what must BLADUS give a host at launch for each top-up and each commission deducted: a receipt, a monthly statement, or a full invoice?	(a) receipt per top-up and monthly statement; (b) invoice from day one	Ask; (b) would move PAY-11 into the MVP	M4	Hosts get only the wallet history (PAY-08); possible non-compliance
F6	**New.** Accounting treatment of the prepaid deposit: customer advance (liability) until commission is deducted, then revenue? When is VAT due?	(a) liability then revenue at deduction; (b) revenue at top-up	(a), which matches the ledger accounts (BR-PAY-03)	M4	Ledger accounts may need renaming; reports wrong
F7	**New.** Which bank or CCP account receives host top-ups, and in what format can we get statements for daily reconciliation?	Bank account; CCP account; both	Both, with CSV statements	M4 (account opened before M7)	Manual reconciliation only (risk K-07)

6. Tech lead

ID	Decision	Options	Recommendation	Deadline (blocks)	If no decision
T1	Production hosting provider in Algeria (staging can be anywhere with demo data)	ICOSNET; Djezzy Cloud; another Algerian data centre	Trial ICOSNET and Djezzy Cloud on staging in M1 to M4; sign by end of M5	Staging: M0. Production: end of M5 (blocks M7)	No production environment in time; launch slips
T2	Where the backend code lives	(a) `backend/` folder in this repository (monorepo); (b) separate repository	(a): one pull request can change API and both apps; shared CI	M0	Developers start in different places; merge pain
T3	Mock-to-API switch mechanism and demo data identity	(a) per-module env flag `DATA_SOURCE_<MODULE>` on the Next.js server + API seeded with the same demo cast and a map from old IDs to UUIDs; (b) big-bang switch; (c) per-function flags	(a)	M0	Big-bang switch late in the plan (risk K-13)
T4	How the two Next.js servers hold the Django session	(a) allauth headless app tokens stored in an httpOnly cookie on the Next.js server; (b) Django session cookie shared on a common domain; (c) JWT	(a): revocable server-side (ACC-03)	M0	Sign-in cannot ship
T5	Small schema gaps found while planning	**Closed on 2026-11-09**: `incident_record`, `calendar_feed` and `otp_send_log` were added to `schema.dbml`	Done	n/a	n/a
T6	Object storage and virus scanning in Algeria	(a) MinIO self-hosted + ClamAV; (b) provider S3-compatible storage if offered	(a) unless the chosen provider offers (b) in Algeria	M1 (documents)	KYC upload blocked
T7	Text search in Arabic, French and English	(a) PostgreSQL unaccent + trigram + Arabic normalisation + place-name alias table; (b) self-hosted Meilisearch	(a) (R-11), with a 50-query test set; revisit (b) if the zero-result rate stays high	M3	Weak Arabic search (risk K-15)
T8	Third-party processors that see personal data	Email: Algerian SMTP relay or self-hosted vs Resend/Brevo; SMS: local aggregator + fallback; errors: self-hosted GlitchTip/Sentry vs SaaS; uptime: self-hosted	Prefer Algerian or self-hosted for anything with personal data; final list reviewed by the lawyer (L10)	M0 (interfaces), M6 (final providers)	Data transfer risk (K-02)
T9	Typed API client for both apps	(a) openapi-typescript + openapi-fetch; (b) orval; (c) hand-written	(a): small, no runtime magic	M0	Drift between API and apps
T10	Second location for backups and restore drills	(a) second Algerian provider; (b) second data centre of the same provider; (c) abroad (needs L2)	(a) for provider independence	Decide in M6, running in M7	Backups in one place only (risk K-19)

ID	Decision	Options	Recommendation	Deadline (blocks)	If no decision
T11	Six new staff permissions proposed in 06 and 08: `kyc.view_documents`, `privacy.handle_requests`, `wallets.approve_large_topup`, `reports.view`, `ledger.adjust` (plus a later `privacy_officer` role)	(a) add to the permission seed now; (b) reuse existing permissions	(a): least privilege for ID documents, four-eyes top-ups, read-only reporting	M1 (staff sign-in)	Moderators keep broad access to ID documents

---

Check yourself

- [ ] Which two lawyer answers have the earliest hard deadline, and why that milestone?
- [ ] What happens by default if nobody answers Q22 (foreign phone numbers)?
- [ ] Which accountant question could move commission invoices (PAY-11) into the MVP, and with which answer?
- [ ] Why is Q18 (deposit withdrawals) linked to a legal question?
- [ ] Which decisions can be changed later by a setting, and which one is marked costly to change?
- [ ] Which tech decision makes "one module at a time behind a flag" possible?
- [ ] Find the decision that keeps BLADUS from needing a PSP licence at launch.

14. Open questions (running list)

> **In plain words**
> Questions the code, the spec and the research could not answer. Some need you, some need a lawyer or accountant.
> This list is kept up to date after each phase. The ones that block design are marked **blocking**.

Status (2026-11-09, Phase 8):

- Q1 to Q13: decided (DEC-001 to DEC-013); DEC-014 and DEC-015 were added later by the product owner.
- **Moved to 13-decisions-needed.md** with options, recommendation, deadline (roadmap milestone) and the default if nobody decides: **Q14 to Q20**, **L1 to L7**, **F1 to F4**, and the new items below (**Q21 to Q28**, **L8 to L12**, **F5 to F7**). The tables on this page stay as the running record; answers are written here and in 13 when they arrive.
- New technical decisions for the tech lead (**T1 to T10**: hosting provider, mock-to-API switch, session model, storage, search, processors, backups) are listed only in 13-decisions-needed.md section 6.
- Deadlines refer to the milestones in 11-roadmap.md. The most urgent outside answers are **L1 and L2, needed by the end of M5**.

Started 2026-11-04 (end of Phase 2). On 2026-11-04 the product owner accepted all defaults for Q1 to Q13; they are recorded as decisions DEC-001 to DEC-013 in `03-requirements.md` section 2.

For you (product owner)

	Question	Blocking?	Answer
Q1	Money model: does BLADUS only take a commission (host pre-pays a deposit, guest pays host), or collect guest payments and pay hosts out?	**blocking**	**Decided 2026-11-04 (default accepted):** M1 prepaid commission, ledger ready for later models
Q2	Commission base: accommodation subtotal (website today), base price (admin), or full total incl. cleaning fee?	**blocking**	**Decided 2026-11-04 (default accepted):** Subtotal (nights x price), excluding cleaning fee and taxes
Q3	Must new listings and trips be approved by staff before going live?	**blocking**	**Decided 2026-11-04 (default accepted):** Yes: `pending_review` first; hosts with a good history can be fast-tracked later
Q4	Can one person be an individual host and also a member of organisations at the same time?	**blocking**	**Decided 2026-11-04 (default accepted):** Yes, with an active-organisation switcher
Q5	Is "hotel" its own vertical or a kind of home?	**blocking**	**Decided 2026-11-04 (default accepted):** Own vertical in search, `home` family in analytics
Q6	Trips: own tables with departures (seats, price, status per date)?	**blocking**	**Decided 2026-11-04 (default accepted):** Yes
Q7	Is the admin's 2-step cap on the individual onboarding intended?	no	**Decided 2026-11-04 (default accepted):** No: keep the 12 steps
Q8	Request-to-book reply window: keep 24 hours? configurable?	no	**Decided 2026-11-04 (default accepted):** 24 h, platform setting
Q9	Who in an organisation may accept or decline a booking request?	no	**Decided 2026-11-04 (default accepted):** owner, admin, manager
Q10	When are host earnings counted: at confirmation, check-in or check-out?	no	**Decided 2026-11-04 (default accepted):** At check-out (completed)
Q11	Cancellation policies: keep flexible / moderate / strict, or add more?	no	**Decided 2026-11-04 (default accepted):** Keep three, stored as windows
Q12	Do guests need phone verification before their first booking?	no	**Decided 2026-11-04 (default accepted):** Yes, phone OTP
Q13	Which wilaya count do the apps show until the official codes 59 to 69 are confirmed?	no	**Decided 2026-11-04 (default accepted):** 69

Raised by the schema (Phase 5), open (moved to 13-decisions-needed.md on 2026-11-09)

These are proposals built into the schema and rules. Each one can be changed without a redesign.

#	Question	Blocking?	Proposal in the schema
Q14	How far below zero may a host's commission deposit go before instant booking is switched off?	no	20,000 DZD, per host type in settings (BR-PAY-10)
Q15	Above which top-up amount must a second finance person approve?	no	500,000 DZD (BR-PAY-07)
Q16	Hostels: their own vertical, shown under the "Hotels" search tab?	no	Yes (ADR-005)
Q17	One room type per booking at launch (several rooms of the same type allowed)?	no	Yes; mixed room types later (ADR-003)
Q18	Can a host withdraw unused deposit money, and how fast?	no	Yes, on request, paid by finance within 7 days, after the 72 h hold rule
Q19	Minimum number of photos	no	5 for homes, 3 per room type (BR-MED-01)
Q20	Host cancellation penalty	no	Moderation case after 3 host cancellations in 12 months; no fee (BR-CAN-03)

For a lawyer (VERIFY WITH A PROFESSIONAL)

L1 to L7 moved to 13-decisions-needed.md section 4 on 2026-11-09 (still open).

#	Question
L1	Does Law 18-05 art. 8 ("hosted in Algeria", `.com.dz`) apply to BLADUS as an intermediary, and to the API server as well as the website?
L2	Does storing personal data or backups abroad need ANPDP authorisation under Law 18-07 as amended by Law 25-11? Which declarations must BLADUS file before launch?
L3	Is collecting guest money and paying hosts a payment service under Règlement 25-02? Is the prepaid commission deposit model safe?
L4	Which documents must each host type provide (individual home, "chez l'habitant", agency, hotel, hostel, vehicle rental, guide)?

#	Question
L5	Is whole-home short-term rental legal for individuals, and what must BLADUS check?
L6	Duty to declare foreign guests (Law 08-11 art. 29): whose duty, and what must BLADUS store?
L7	Data retention periods for IDs, bookings, messages and invoices. Is a DPO required?

For an accountant or bank

F1 to F4 moved to 13-decisions-needed.md section 5 on 2026-11-09 (still open).

#	Question
F1	VAT on BLADUS commission: 19% or 9% (tourism)? Who issues which invoice?
F2	Taxe de séjour: does BLADUS collect it or only show it?
F3	Chargily or SlickPay contract terms, settlement delays, amount units in their APIs.
F4	Invoice numbering and retention rules for commission invoices to hosts.

Added 2026-11-09 (Phase 8: roadmap, risks, decisions)

Found while planning the build. All are open and listed with options and deadlines in 13-decisions-needed.md.

For you (product owner)

#	Question	Blocking?	Deadline	Proposal
Q21	Launch geography: all 69 wilayas or a few pilot wilayas?	no	M7	All open, beta hosts recruited in pilot wilayas
Q22	How do guests without an Algerian mobile (diaspora, foreigners) verify? DEC-012 and BR-ACC-01 only accept `+213[5-7]` numbers	**blocking** (sign-up)	M0	Email code for non-Algerian numbers, stricter booking limits
Q23	Public launch date relative to the baccalaureate exams (internet cuts, usually June) and the summer season	no	M6	Closed beta May to June, public launch early July 2027
Q24	Who moderates at launch, and what review time do we promise hosts?	no	M2	2 moderators, 1 working day
Q25	Must a host top up a first deposit before going live or before instant booking?	no	M4	No, the credit floor (Q14) is enough
Q26	Is the 24 h request reply window right for agencies whose trips depart weeks later?	no	M4	24 h default, 48 h for agencies (setting per host type)
Q27	Support channels and hours at launch	no	M7	WhatsApp Business + email + in-app, office hours Sunday to Thursday
Q28	Which verticals open at launch?	no	M2	All six, unless a legal answer (L4, L5, L9) closes one; closed by setting

For a lawyer (VERIFY WITH A PROFESSIONAL)

#	Question
L8	Terms of use, host terms and privacy notice texts in ar/fr/en, versioned for consent records (PRV-01). Who drafts, who signs off?
L9	Does BLADUS itself need a tourism registration or travel agency licence (Law 99-06, Décret 17-161) because it lists and books agency trips and experiences?
L10	Do foreign service providers that touch personal data (transactional email, international SMS fallback, error tracking, uptime monitoring) count as a transfer abroad needing ANPDP authorisation?
L11	Company formalities before launch: commerce register activity code for an online marketplace, CNRC e-commerce registration, holder of the `.com.dz` domain.
L12	Does the booking flow (instant and request) meet Law 18-05 art. 10 to 13 (offer content, 3-step order: review, confirm, contract)?

For an accountant or bank

#	Question
F5	Before commission invoices (PAY-11, R2), what must BLADUS give a host at launch for each top-up and each commission deducted: a receipt, a monthly statement, or a full invoice?
F6	Accounting treatment of the prepaid deposit: customer advance (liability) until commission is deducted, then revenue? When is VAT due?
F7	Which bank or CCP account receives host top-ups, and can we get statements in a format fit for daily reconciliation?

Check yourself

- [ ] Which product questions were decided on 2026-11-04, and where are they recorded?
- [ ] Where do the open questions go once they become decisions to make?
- [ ] Which questions need a lawyer, and which need an accountant or bank?
- [ ] What is the default if a product question is not answered in time?

Traceability matrix: requirement to rule, table, API, test and milestone

> **In plain words**
> One line per requirement (all 147 from requirements.csv).
> Each line says which business rules it follows, which database tables hold its data, which API calls serve it,
> where its tests will live, in which roadmap milestone it is built, and which app data module stops using demo data.
> Use it to answer "if we change X, what else moves?" and "is everything planned somewhere?".
> This page and traceability.csv are generated by `python _tools/build_traceability.py`. Edit the script, not the tables.

Terms used here (see glossary.md):

- **Traceability**: being able to follow one need from the request, through the rules and the database, to the test that proves it.
- **Milestone**: a block of build work with an exit test, defined in 11-roadmap.md.
- **API area**: the family of web addresses (endpoints) the apps call. Final paths are in `06-api-design.md` and `openapi.yaml` (tagged with `x-requirements`); names here are plain REST guesses.
- **job:** a scheduled background task (Celery). **internal:** a service called by other code, with no public address.
- `lib/data` **module**: the file in the website (`web:`) or admin (`admin:`) app whose functions switch from demo data to the API, behind a per-module flag.

---

1. Requirements per milestone

Milestone	Name	Release	Requirements	of which Must
M0	Foundations	MVP	18	17
M1	Accounts, staff and host verification	MVP	28	26
M2	Catalogue, trips and moderation	MVP	23	20
M3	Search, availability and quotes	MVP	11	8
M4	Bookings, cancellations and the commission wallet	MVP	27	25
M5	Messaging and reviews	MVP	8	6
M6	Reports, privacy and hardening	MVP	8	5
M7	Production in Algeria, closed beta, launch	MVP	3	2
R2-A	Release 2, online money (gateway, invoices, taxes)	R2	4	0
R2-B	Release 2, trust and growth	R2	15	0
LATER	Later, behind a legal or business gate	Later	2	0
Total			**147**	**109**

2. Requirements per module and milestone

Module	M0	M1	M2	M3	M4	M5	M6	M7	R2-A	R2-B	LATER	Total
ACC	5	3	1									9
ORG		7										7
HST		5	1		1							7
LST			11							1		12
TRP			5	1						1		7
AVL			1	1	3						1	6
PRC				4					1			5
BKG					10					1		11
CAN					4							4
PAY	1				7				3		1	12
REV						4				1		5
MSG						4				2		6
NTF		1			2					1		4
SRC	1			5								6
MED		1	2									3
MOD		4	1							3		8
STF		5					1					6

Module	M0	M1	M2	M3	M4	M5	M6	M7	R2-A	R2-B	LATER	Total
CFG	1	2	1							1		5
PRM										2		2
RPT							2			1		3
PRV	1						3	1		1		6
NFR	9						2	2				13
Total	18	28	23	11	27	8	8	3	4	15	2	**147**

3. Full matrix (sorted by milestone, then ID order of requirements.csv)

ID	Requirement	Pri.	Milestone	Rules	Tables	API area	Tests	`lib/data` switch
ACC-01	Sign up with email or Algerian phone number	Must	M0	BR-ACC-01	user; consent_record; allauth email tables	POST /api/v1/auth/signup; POST /api/v1/auth/phone/verify; POST /api/v1/auth/email/verify; PUT /api/v1/me/phone; POST /api/v1/me/bookings	`tests/accounts/test_signup.py`	web:auth.ts
ACC-02	Sign in with password or one-time phone code	Must	M0	BR-ACC-02	user; allauth tables (one-time codes in Redis)	POST /api/v1/auth/login; POST /api/v1/auth/code/request; POST /api/v1/auth/code/confirm	`tests/accounts/test_login_otp.py`	web:auth.ts
ACC-03	Session for both Next.js apps	Must	M0	BR-ACC-03	allauth session tables	POST /api/v1/auth/login; GET /api/v1/auth/session; DELETE /api/v1/auth/session; POST /api/v1/admin/auth/login; GET /api/v1/admin/auth/session; DELETE /api/v1/admin/auth/session; (+4 more)	`tests/accounts/test_sessions.py`	web:auth.ts
ACC-04	Password reset and change	Must	M0		user; allauth tables	POST /api/v1/auth/password/request; POST /api/v1/auth/password/reset; POST /api/v1/auth/password/change; POST /api/v1/admin/auth/password/change	`tests/accounts/test_password.py`	web:auth.ts
ACC-07	Login rate limiting shared across servers	Must	M0	BR-ACC-05	none (Redis counters)	POST /api/v1/auth/login; POST /api/v1/auth/code/request; POST /api/v1/admin/auth/login	`tests/accounts/test_login_lockout.py`	web:auth.ts
PAY-12	Money as centimes with currency	Must	M0	BR-PAY-01	all *_minor columns with currency	POST /api/v1/quotes	`tests/core/test_money.py`	none (shared convention)
SRC-04	69 wilayas and 1,541 communes	Must	M0	BR-SRC-02	wilaya; daira; commune	GET /api/v1/geo/wilayas; GET /api/v1/geo/wilayas/{wilaya_code}/communes; GET /api/v1/geo/communes	`tests/geo/test_seed_69_wilayas.py`	web:lib/algeria.ts (s list)
CFG-01	Platform settings	Must	M0	BR-CFG-01	platform_settings	GET /api/v1/admin/commission-rates; PUT /api/v1/admin/commission-rates; GET /api/v1/admin/settings; PATCH /api/v1/admin/settings; GET /api/v1/admin/settings/versions	`tests/config/test_settings.py`	admin:settings.ts (switches in M1, aft staff sign-in works)
PRV-01	Consent records	Must	M0	BR-PRV-01	consent_record	POST /api/v1/auth/signup; GET /api/v1/me/consents; POST /api/v1/me/consents	`tests/privacy/test_consents.py`	web:auth.ts
NFR-05	Security baseline	Must	M0		none	no endpoint: all /api/v1/*	`tests/security/test_baseline.py`	none
NFR-06	Three languages and right-to-left	Must	M0	BR-SYS-04	name_en/name_fr/name_ar columns on reference tables	no endpoint: all errors and notifications	`tests/core/test_i18n_errors.py`	none
NFR-07	Time and dates	Must	M0	BR-SYS-05	timestamptz and date columns	no endpoint: all /api/v1/*	`tests/core/test_time_rules.py`	none
NFR-08	API contract	Must	M0		none	no endpoint: GET /api/v1/schema (OpenAPI)	`tests/core/test_openapi_contract.py`	both apps: generate typed client
NFR-09	Observability	Must	M0		none	no endpoint: GET /api/v1/health; metrics endpoint	`tests/core/test_logging.py`	none
NFR-10	Test coverage	Must	M0		all	no endpoint: all	`CI coverage gate (pytest-cov) and tests/**/test_*_property.py`	none
NFR-11	Rate limits	Must	M0		none (Redis)	POST /api/v1/auth/login	`tests/core/test_rate_limits.py`	none
NFR-12	Unique identifiers	Must	M0		all primary keys (UUIDv7); booking.reference	GET /api/v1/me/bookings/{booking_id}	`tests/core/test_ids.py`	none
NFR-13	Accessibility of messages	Should	M0		none	no endpoint: error envelope on all /api/v1/*	`tests/core/test_error_codes.py`	both apps: ActionResult mappi

ID	Requirement	Pri.	Milestone	Rules	Tables	API area	Tests	lib/data switch
ACC-05	Two-factor login	Must	M1	BR-ACC-04	allauth MFA authenticator tables; staff_profile	POST /api/v1/auth/2fa/authenticate; GET /api/v1/auth/authenticators; GET /api/v1/auth/authenticators/totp; POST /api/v1/auth/authenticators/totp; POST /api/v1/host/payout-accounts; POST /api/v1/admin/auth/2fa/authenticate; (+2 more)	tests/accounts/test_mfa.py	admin:credentials.ts
ACC-06	Profile management	Must	M1		user	POST /api/v1/auth/phone/verify; POST /api/v1/auth/email/verify; GET /api/v1/users/{user_id}; GET /api/v1/me; PATCH /api/v1/me; PUT /api/v1/me/phone	tests/accounts/test_profile.py	web:auth.ts
ACC-08	One user, many roles	Must	M1	BR-ORG-01	user; host_account; membership	GET /api/v1/me; POST /api/v1/me/invitations/accept; GET /api/v1/me/workspaces; GET /api/v1/admin/users; GET /api/v1/admin/users/{user_id}	tests/hosts/test_one_user_many_roles.py	web:organizations.t
ORG-01	Create an organisation (agency, hotel, hostel)	Must	M1	BR-ORG-02	organization; host_account; membership	POST /api/v1/host/onboarding/drafts/{host_type}/submit; GET /api/v1/host/organization; PATCH /api/v1/host/organization	tests/hosts/test_org_create.py	web:organizations.t
ORG-02	Active-organisation switcher	Must	M1	BR-ORG-01	membership; host_account	GET /api/v1/me/workspaces; PUT /api/v1/me/workspaces/active; GET /api/v1/host/account	tests/hosts/test_active_host_account.py	web:organizations.t
ORG-03	Invite a team member	Must	M1	BR-ORG-03	invitation; membership; outbox_event	POST /api/v1/me/invitations/accept; GET /api/v1/host/organization/invitations; POST /api/v1/host/organization/invitations; DELETE /api/v1/host/organization/invitations/{invitation_id}	tests/hosts/test_invitations.py	web:organizations.t
ORG-04	Change role, remove member, transfer ownership	Must	M1	BR-ORG-04	membership; audit_event	DELETE /api/v1/host/organization; GET /api/v1/host/organization/members; PATCH /api/v1/host/organization/members/{membership_id}; DELETE /api/v1/host/organization/members/{membership_id}; POST /api/v1/host/organization/transfer-ownership	tests/hosts/test_members.py	web:organizations.t
ORG-05	Per-role permissions inside an organisation	Must	M1	BR-ORG-05	membership	GET /api/v1/host/organization/members; PATCH /api/v1/host/organization/members/{membership_id}; POST /api/v1/host/bookings/{booking_id}/accept; POST /api/v1/host/bookings/{booking_id}/decline	tests/hosts/test_org_permissions.py	web:organizations.t
ORG-06	Re-verification on legal field change	Should	M1	BR-ORG-06	organization; moderation_case	GET /api/v1/host/organization; PATCH /api/v1/host/organization	tests/hosts/test_org_reverification.py	web:organizations.t
ORG-07	Public organisation profile	Must	M1		organization; host_account; listing; trip	GET /api/v1/organizations; GET /api/v1/organizations/{org_slug}	tests/hosts/test_org_public_profile.py	web:organizations.t
HST-01	Guided onboarding per host type	Must	M1	BR-HST-01	onboarding_flow_version; onboarding_draft	GET /api/v1/onboarding-flows/{host_type}; POST /api/v1/host/onboarding/drafts/{host_type}/submit	tests/hosts/test_onboarding_flow.py	web:auth.ts (updateUserHostSta
HST-02	Save onboarding drafts on the server	Must	M1	BR-HST-02	onboarding_draft	GET /api/v1/host/onboarding/drafts; GET /api/v1/host/onboarding/drafts/{host_type}; PUT /api/v1/host/onboarding/drafts/{host_type}; DELETE /api/v1/host/onboarding/drafts/{host_type}; POST /api/v1/host/onboarding/drafts/{host_type}/submit	tests/hosts/test_onboarding_draft.py	web:auth.ts (updateUserHostSta
HST-03	Upload verification documents	Must	M1	BR-HST-03	host_document; media_asset	GET /api/v1/host/documents; POST /api/v1/host/documents; DELETE /api/v1/host/documents/{document_id}	tests/hosts/test_documents_upload.py	web:organizations.t
HST-04	Host verification lifecycle	Must	M1	BR-HST-04	host_account; moderation_case; moderation_decision	GET /api/v1/host/account; POST /api/v1/host/verification/submit; POST /api/v1/admin/moderation/cases/{case_id}/decisions; GET /api/v1/admin/hosts; GET /api/v1/admin/hosts/{host_account_id}; DELETE /api/v1/admin/hosts/{host_account_id}; (+7 more)	tests/hosts/test_verification_states.py	admin:hosts.ts
HST-06	Default approval mode per host or organisation	Must	M1	BR-BKG-02	host_account; listing; platform_settings	GET /api/v1/host/account; PATCH /api/v1/host/account	tests/hosts/test_approval_mode_default.py	web:organizations.t
NTF-04	Outbox pattern	Must	M1	BR-SYS-03	outbox_event; delivery_log	no endpoint: internal: core.outbox_dispatch	tests/core/test_outbox.py	none (workers)
MED-02	Private documents	Must	M1	BR-MED-02	media_asset; host_document; audit_event	POST /api/v1/media/uploads; POST /api/v1/media/{media_id}/complete; GET /api/v1/media/{media_id}/url; POST /api/v1/host/documents; GET /api/v1/admin/documents/{document_id}/url	tests/media/test_private_documents.py	admin:hosts.ts
MOD-01	Moderation queues with waiting time	Must	M1	BR-MOD-01	moderation_case	GET /api/v1/admin/moderation/counts; GET /api/v1/admin/dashboard/attention; GET /api/v1/admin/moderation/cases; GET /api/v1/admin/moderation/cases/{case_id}; POST /api/v1/admin/moderation/cases/{case_id}/assign; GET /api/v1/admin/hosts; (+1 more)	tests/trust/test_queues.py	admin:approvals.ts

ID	Requirement	Pri.	Milestone	Rules	Tables	API area	Tests	`lib/data` switch
MOD-02	Change requests to hosts	Must	M1	BR-MOD-02	moderation_case; change_request_item	POST /api/v1/host/verification/submit; GET /api/v1/host/change-requests; GET /api/v1/admin/moderation/cases/{case_id}; POST /api/v1/admin/moderation/cases/{case_id}/decisions; POST /api/v1/admin/hosts/{host_account_id}/request-changes; POST /api/v1/admin/listings/{listing_id}/request-changes	`tests/trust/test_change_requests.py`	admin:approvals.ts
MOD-03	Reason catalogue	Should	M1		reason_code	GET /api/v1/reference/reasons; POST /api/v1/admin/moderation/cases/{case_id}/decisions; GET /api/v1/admin/reason-codes	`tests/trust/test_reason_codes.py`	admin:approvals.ts
MOD-08	Document-level review	Must	M1	BR-HST-03	host_document; moderation_decision	GET /api/v1/host/documents; GET /api/v1/admin/moderation/cases; POST /api/v1/admin/moderation/cases/{case_id}/decisions; POST /api/v1/admin/hosts/{host_account_id}/approve; GET /api/v1/admin/documents/{document_id}/url; POST /api/v1/admin/documents/{document_id}/approve; (+1 more)	`tests/trust/test_document_review.py`	admin:hosts.ts
STF-01	Staff roles and permissions in the database	Must	M1	BR-STF-01	staff_profile; staff_permission; staff_role_permission	GET /api/v1/admin/auth/session; GET /api/v1/admin/staff; GET /api/v1/admin/staff/roles; PATCH /api/v1/admin/staff/{staff_id}	`tests/accounts/test_staff_permissions.py`	admin:team.ts
STF-02	Staff invitation and activation	Must	M1		staff_profile; user; allauth session tables	POST /api/v1/admin/auth/invitations/accept; POST /api/v1/admin/auth/authenticators/totp; GET /api/v1/admin/staff; POST /api/v1/admin/staff/invitations; POST /api/v1/admin/staff/{staff_id}/deactivate; POST /api/v1/admin/staff/{staff_id}/activate; (+1 more)	`tests/accounts/test_staff_invite.py`	admin:team.ts
STF-03	Append-only audit log	Must	M1	BR-STF-02	audit_event	GET /api/v1/admin/documents/{document_id}/url; POST /api/v1/admin/users/{user_id}/suspend; POST /api/v1/admin/users/{user_id}/unsuspend; POST /api/v1/admin/users/{user_id}/ban; POST /api/v1/admin/users/{user_id}/restore; GET /api/v1/admin/audit-events	`tests/audit/test_append_only.py`	admin:audit.ts
STF-04	Actors by id, not name	Must	M1	BR-STF-03	every actor foreign key (ledger_transaction, moderation_decision, onboarding_flow_version, refund)	GET /api/v1/admin/audit-events	`tests/audit/test_actor_ids.py`	admin:audit.ts
STF-05	Separate admin API surface	Must	M1		none	POST /api/v1/admin/auth/login; POST /api/v1/admin/auth/2fa/authenticate; GET /api/v1/admin/search	`tests/accounts/test_admin_surface.py`	admin:credentials.ts
CFG-02	Versioned onboarding config	Must	M1	BR-CFG-02	onboarding_flow_version	GET /api/v1/admin/onboarding-flows; GET /api/v1/admin/onboarding-flows/{host_type}/versions; GET /api/v1/admin/onboarding-flows/{host_type}/versions/{version}; PUT /api/v1/admin/onboarding-flows/{host_type}/draft; POST /api/v1/admin/onboarding-flows/{host_type}/publish; POST /api/v1/admin/onboarding-flows/{host_type}/versions/{version}/republish	`tests/config/test_flow_versions.py`	admin:onboarding.t
CFG-04	Serve published config to the website	Must	M1	BR-HST-02	onboarding_flow_version; onboarding_draft	GET /api/v1/onboarding-flows/{host_type}; PUT /api/v1/host/onboarding/drafts/{host_type}; POST /api/v1/host/onboarding/drafts/{host_type}/submit; POST /api/v1/admin/onboarding-flows/{host_type}/publish	`tests/config/test_serve_flow.py`	web:auth.ts (updateUserHostSta
ACC-09	Favourites	Should	M2		favorite	GET /api/v1/me/favorites; PUT /api/v1/me/favorites/{target_type}/{target_id}; DELETE /api/v1/me/favorites/{target_type}/{target_id}	`tests/accounts/test_favorites.py`	web:favorites.ts
HST-07	What each host type may publish	Must	M2	BR-HST-05	host_account; organization; listing; trip	GET /api/v1/reference; GET /api/v1/host/account; POST /api/v1/host/onboarding/drafts/{host_type}/submit; POST /api/v1/host/listings; PATCH /api/v1/host/listings/{listing_id}; POST /api/v1/host/trips	`tests/catalog/test_host_offer_rule.py`	web:listings.ts
LST-01	Create and edit a listing (home, hotel, hostel, experience, vehicle)	Must	M2	BR-LST-01	listing; vehicle_detail; experience_detail; listing_revision	POST /api/v1/host/onboarding/drafts/{host_type}/submit; GET /api/v1/host/listings; POST /api/v1/host/listings; GET /api/v1/host/listings/{listing_id}; PATCH /api/v1/host/listings/{listing_id}	`tests/catalog/test_listing_create.py`	web:listings.ts
LST-02	Listing review before publication	Must	M2	BR-LST-02	listing; listing_revision; moderation_case	PATCH /api/v1/host/listings/{listing_id}; POST /api/v1/host/listings/{listing_id}/status; POST /api/v1/admin/moderation/cases/{case_id}/decisions; GET /api/v1/admin/listings/{listing_id}; POST /api/v1/admin/listings/{listing_id}/approve; POST /api/v1/admin/listings/{listing_id}/reject; (+3 more)	`tests/catalog/test_listing_review.py`	admin:listings.ts
LST-03	Listing status lifecycle	Must	M2	BR-LST-03	listing	GET /api/v1/host/listings; POST /api/v1/host/listings/{listing_id}/status; POST /api/v1/host/trips/{trip_id}/status; GET /api/v1/admin/listings; POST /api/v1/admin/listings/{listing_id}/approve; POST /api/v1/admin/listings/{listing_id}/reject; (+4 more)	`tests/catalog/test_listing_states.py`	admin:listings.ts

ID	Requirement	Pri.	Milestone	Rules	Tables	API area	Tests	`lib/data` switch
LST-04	Room types for hotels and hostels	Must	M2	BR-AVL-03	room_type; room_type_amenity	POST /api/v1/host/listings; GET /api/v1/host/listings/{listing_id}/room-types; POST /api/v1/host/listings/{listing_id}/room-types; PATCH /api/v1/host/listings/{listing_id}/room-types/{room_type_id}; DELETE /api/v1/host/listings/{listing_id}/room-types/{room_type_id}	`tests/catalog/test_room_types.py`	web:listings.ts
LST-05	Amenities and categories from a shared list	Must	M2	BR-LST-04	amenity; category; property_type; listing_amenity	GET /api/v1/reference; POST /api/v1/host/listings; GET /api/v1/admin/categories; POST /api/v1/admin/categories; PATCH /api/v1/admin/categories/{category_id}; DELETE /api/v1/admin/categories/{category_id}; (+8 more)	`tests/catalog/test_reference_lists.py`	admin:categories.ts
LST-06	Location: wilaya, commune, exact and public point	Must	M2	BR-SRC-03	listing; commune; wilaya	GET /api/v1/listings/{listing_id}; POST /api/v1/host/listings	`tests/catalog/test_listing_location.py`	web:listings.ts
LST-07	House rules, stay rules and cancellation policy	Must	M2	BR-CAN-01	listing; cancellation_policy	POST /api/v1/host/listings; PATCH /api/v1/host/listings/{listing_id}	`tests/catalog/test_stay_rules.py`	web:listings.ts
LST-08	Pause and soft delete	Must	M2	BR-LST-03	listing	DELETE /api/v1/host/organization; DELETE /api/v1/host/listings/{listing_id}; POST /api/v1/host/listings/{listing_id}/status; DELETE /api/v1/host/trips/{trip_id}; DELETE /api/v1/admin/listings/{listing_id}	`tests/catalog/test_listing_soft_delete.py`	web:listings.ts
LST-10	Vehicle listing specifics	Should	M2		vehicle_detail	GET /api/v1/listings/{listing_id}; POST /api/v1/host/listings	`tests/catalog/test_vehicle_fields.py`	web:vehicles.ts
LST-12	Canonical experience types	Must	M2	BR-LST-05	listing (experience_type enum)	GET /api/v1/reference; GET /api/v1/listings; GET /api/v1/listings/facets; POST /api/v1/host/listings; PATCH /api/v1/host/listings/{listing_id}; GET /api/v1/admin/listings	`tests/catalog/test_experience_types.py`	web:experiences.ts
LST-11	Experience listing specifics	Should	M2		experience_detail; experience_slot	GET /api/v1/listings/{listing_id}; GET /api/v1/listings/{listing_id}/availability; POST /api/v1/host/listings; GET /api/v1/host/listings/{listing_id}/slots; POST /api/v1/host/listings/{listing_id}/slots; PATCH /api/v1/host/listings/{listing_id}/slots/{slot_id}	`tests/catalog/test_experience_fields.py`	web:experiences.ts
TRP-01	Agency-only trips	Must	M2	BR-TRP-01	trip; host_account; organization	GET /api/v1/host/trips; POST /api/v1/host/trips; DELETE /api/v1/host/trips/{trip_id}	`tests/trips/test_agency_only.py`	web:trips.ts
TRP-02	Trip itinerary and inclusions	Must	M2		trip; trip_itinerary_day	GET /api/v1/trips/{trip_id}; POST /api/v1/host/trips; GET /api/v1/host/trips/{trip_id}; PATCH /api/v1/host/trips/{trip_id}	`tests/trips/test_itinerary.py`	web:trips.ts
TRP-03	Departures with seats and price	Must	M2	BR-TRP-02	trip_departure	GET /api/v1/trips/{trip_id}; GET /api/v1/trips/{trip_id}/departures; GET /api/v1/host/trips/{trip_id}/departures; POST /api/v1/host/trips/{trip_id}/departures; PATCH /api/v1/host/trips/{trip_id}/departures/{departure_id}	`tests/trips/test_departures.py`	web:trips.ts
TRP-04	Trips go through review	Must	M2	BR-LST-02	trip; moderation_case	PATCH /api/v1/host/trips/{trip_id}; POST /api/v1/host/trips/{trip_id}/status; POST /api/v1/admin/moderation/cases/{case_id}/decisions; GET /api/v1/admin/trips; GET /api/v1/admin/trips/{trip_id}; POST /api/v1/admin/trips/{trip_id}/status	`tests/trips/test_trip_review.py`	admin:listings.ts
TRP-07	Canonical trip types	Must	M2	BR-LST-05	trip (trip_type enum)	GET /api/v1/reference; GET /api/v1/trips; POST /api/v1/host/trips; PATCH /api/v1/host/trips/{trip_id}; GET /api/v1/admin/trips	`tests/trips/test_trip_types.py`	web:trips.ts
AVL-03	Host calendar: block days and set prices	Must	M2	BR-AVL-04	listing_day; room_type_night	GET /api/v1/host/calendar; GET /api/v1/host/listings/{listing_id}/calendar; PUT /api/v1/host/listings/{listing_id}/calendar/blocks; PUT /api/v1/host/listings/{listing_id}/calendar/prices	`tests/inventory/test_host_calendar.py`	web:calendar.ts
MED-01	Photo upload with direct-to-storage	Must	M2	BR-MED-01	media_asset; listing_photo	POST /api/v1/media/uploads; POST /api/v1/media/{media_id}/complete; GET /api/v1/media/{media_id}; PUT /api/v1/host/listings/{listing_id}/photos	`tests/media/test_photo_pipeline.py`	web:listings.ts
MED-03	Photo ordering and cover	Must	M2		listing_photo	PUT /api/v1/host/listings/{listing_id}/photos	`tests/media/test_photo_order.py`	web:listings.ts
MOD-06	Cascade on host suspension	Must	M2	BR-MOD-03	host_account; listing; trip; booking; moderation_case	POST /api/v1/admin/hosts/{host_account_id}/suspend; POST /api/v1/admin/hosts/{host_account_id}/unsuspend; POST /api/v1/admin/hosts/{host_account_id}/ban	`tests/trust/test_suspension_cascade.py`	admin:hosts.ts
CFG-05	Taxonomy management	Must	M2	BR-LST-04	category; amenity; property_type	GET /api/v1/reference; GET /api/v1/admin/categories; POST /api/v1/admin/categories; PATCH /api/v1/admin/categories/{category_id}; DELETE /api/v1/admin/categories/{category_id}; GET /api/v1/admin/amenities; (+7 more)	`tests/catalog/test_taxonomy_admin.py`	admin:categories.ts
TRP-05	Search trips by date	Should	M3		trip_departure; listing_search_doc	GET /api/v1/trips	`tests/search/test_trip_dates.py`	web:trips.ts

ID	Requirement	Pri.	Milestone	Rules	Tables	API area	Tests	lib/data switch
AVL-04	Availability window and partial matches	Must	M3	BR-AVL-05	listing_day; room_type_night; listing_search_doc	GET /api/v1/listings; GET /api/v1/listings/{listing_id}/availability; POST /api/v1/quotes	tests/inventory/test_partial_availability.py	web:listings.ts
PRC-01	Server-side price quote	Must	M3	BR-PRC-01	quote; listing_day; room_type_night; tax_rate	POST /api/v1/quotes; GET /api/v1/quotes/{quote_id}; PUT /api/v1/host/listings/{listing_id}/calendar/prices	tests/pricing/test_quote.py	web:reservations.ts
PRC-02	No guest service fee	Must	M3	BR-PRC-02	quote	POST /api/v1/quotes	tests/pricing/test_no_guest_fee.py	web:reservations.ts
PRC-03	Length-of-stay discounts	Should	M3	BR-PRC-03	listing; quote	POST /api/v1/quotes; PATCH /api/v1/host/listings/{listing_id}	tests/pricing/test_length_of_stay_discount.py	web:reservations.ts
PRC-05	Whole dinars to guests, centimes inside	Must	M3	BR-PAY-01	quote; booking	POST /api/v1/quotes	tests/pricing/test_rounding.py	web:reservations.ts
SRC-01	Search with filters and facets	Must	M3	BR-SRC-01	listing_search_doc	GET /api/v1/listings; GET /api/v1/listings/facets	tests/search/test_filters_facets.py	web:listings.ts
SRC-02	Map search	Must	M3		listing_search_doc	GET /api/v1/listings; GET /api/v1/listings/map	tests/search/test_map_search.py	web:listings.ts
SRC-03	Text search in three languages	Should	M3		listing_search_doc; wilaya; commune	GET /api/v1/geo/communes; GET /api/v1/listings	tests/search/test_text_search_ar_fr_en.py	web:listings.ts
SRC-05	Sorting and pagination	Must	M3		listing_search_doc	GET /api/v1/listings; GET /api/v1/trips	tests/search/test_sort_pagination.py	web:listings.ts
SRC-06	Hotels as their own vertical	Must	M3	BR-SRC-04	listing; listing_search_doc	GET /api/v1/listings; GET /api/v1/listings/{listing_id}	tests/search/test_hotel_vertical.py	web:listings.ts
HST-05	Payout or refund account details	Should	M4	BR-PAY-09	payout_account; audit_event	GET /api/v1/host/payout-accounts; POST /api/v1/host/payout-accounts; DELETE /api/v1/host/payout-accounts/{payout_account_id}; POST /api/v1/host/payouts	tests/hosts/test_payout_account.py	web:wallet.ts
AVL-01	No double booking for single units	Must	M4	BR-AVL-01	booking (exclusion constraint on unit and stay range)	POST /api/v1/me/bookings	tests/bookings/test_no_double_booking.py	web:reservations.ts
AVL-02	Pooled inventory for rooms, beds, seats	Must	M4	BR-AVL-02	room_type_night; experience_slot; trip_departure	GET /api/v1/listings/{listing_id}/availability; POST /api/v1/me/bookings	tests/inventory/test_pooled_inventory.py	web:reservations.ts
AVL-05	Temporary holds during payment	Should	M4	BR-AVL-06	room_type_night; experience_slot; trip_departure; quote	no endpoint: POST /api/v1/bookings (hold step); job: inventory.release_holds	tests/inventory/test_holds.py	web:reservations.ts
BKG-01	Instant booking	Must	M4	BR-BKG-01	booking; booking_event; ledger_transaction; account_balance	POST /api/v1/me/bookings	tests/bookings/test_booking_create_instant.py	web:reservations.ts
BKG-02	Request to book	Must	M4	BR-BKG-03	booking; booking_event	GET /api/v1/reference/reasons; POST /api/v1/me/bookings; POST /api/v1/host/listings/{listing_id}/status; POST /api/v1/host/bookings/{booking_id}/accept; POST /api/v1/host/bookings/{booking_id}/decline	tests/bookings/test_request_to_book.py	web:reservations.ts
BKG-03	Automatic expiry of requests	Must	M4	BR-BKG-04	booking; booking_event; outbox_event	PATCH /api/v1/admin/settings	tests/bookings/test_request_expiry.py	web:reservations.ts
BKG-04	Automatic completion after check-out	Must	M4	BR-BKG-05	booking; booking_event	no endpoint: job: bookings.complete_stays	tests/bookings/test_auto_complete.py	web:reservations.ts
BKG-05	Unified booking state machine	Must	M4	BR-BKG-06	booking; booking_event	POST /api/v1/me/bookings; POST /api/v1/me/bookings/{booking_id}/cancel; GET /api/v1/admin/bookings; GET /api/v1/admin/bookings/{booking_id}	tests/bookings/test_booking_states.py	admin:bookings.ts
BKG-06	Booking snapshot	Must	M4	BR-BKG-07	booking; quote	POST /api/v1/me/bookings; GET /api/v1/admin/bookings/{booking_id}	tests/bookings/test_booking_snapshot.py	web:reservations.ts
BKG-07	Guest and host booking lists	Must	M4		booking	GET /api/v1/me/bookings; GET /api/v1/me/bookings/{booking_id}; GET /api/v1/host/listings; GET /api/v1/host/bookings; GET /api/v1/host/bookings/{booking_id}; GET /api/v1/admin/bookings	tests/bookings/test_booking_lists.py	web:reservations.ts
BKG-08	Booking reference	Must	M4		booking	GET /api/v1/me/bookings/{booking_id}; GET /api/v1/host/bookings/{booking_id}; GET /api/v1/admin/bookings	tests/bookings/test_booking_reference.py	admin:bookings.ts

ID	Requirement	Pri.	Milestone	Rules	Tables	API area	Tests	lib/data switch
BKG-09	Trip departure booking	Must	M4	BR-TRP-02	booking; trip_departure	POST /api/v1/quotes; GET /api/v1/trips/{trip_id}/departures; POST /api/v1/me/bookings	`tests/bookings/test_trip_booking.py`	web:reservations.ts
BKG-10	Idempotent booking creation	Must	M4	BR-SYS-01	idempotency_record	POST /api/v1/me/bookings	`tests/bookings/test_booking_idempotency.py`	web:reservations.ts
CAN-01	Guest cancellation by policy	Must	M4	BR-CAN-02	booking; refund; booking_event; cancellation_policy	GET /api/v1/me/bookings/{booking_id}/cancellation-preview; POST /api/v1/me/bookings/{booking_id}/cancel	`tests/bookings/test_guest_cancel.py`	web:reservations.ts
CAN-02	Host cancellation	Must	M4	BR-CAN-03	booking; refund; ledger_transaction	GET /api/v1/reference/reasons; GET /api/v1/host/bookings/{booking_id}/cancellation-preview; POST /api/v1/host/bookings/{booking_id}/cancel	`tests/bookings/test_host_cancel.py`	web:reservations.ts
CAN-03	Staff cancellation and refund	Must	M4	BR-CAN-04	refund; booking; audit_event	POST /api/v1/admin/bookings/{booking_id}/cancel; POST /api/v1/admin/bookings/{booking_id}/refunds; GET /api/v1/admin/refunds	`tests/bookings/test_staff_refund.py`	admin:bookings.ts
CAN-04	Commission reversal rules	Must	M4	BR-PAY-05	ledger_transaction; ledger_entry	POST /api/v1/me/bookings/{booking_id}/cancel; POST /api/v1/host/bookings/{booking_id}/decline; POST /api/v1/host/bookings/{booking_id}/cancel; POST /api/v1/admin/bookings/{booking_id}/cancel; POST /api/v1/admin/bookings/{booking_id}/refunds	`tests/ledger/test_commission_reversal.py`	web:wallet.ts
PAY-01	Double-entry ledger	Must	M4	BR-PAY-02	ledger_account; ledger_transaction; ledger_entry	GET /api/v1/host/wallet/entries; POST /api/v1/admin/bookings/{booking_id}/refunds; GET /api/v1/admin/ledger/transactions; POST /api/v1/admin/ledger/adjustments	`tests/ledger/test_double_entry.py`	admin:ledger.ts
PAY-02	Prepaid commission wallet (model M1)	Must	M4	BR-PAY-03	ledger_account; account_balance	GET /api/v1/host/wallet; GET /api/v1/admin/finance/summary; GET /api/v1/admin/wallets; GET /api/v1/admin/wallets/{host_account_id}; POST /api/v1/admin/wallets/{host_account_id}/topups	`tests/ledger/test_wallet_balance.py`	web:wallet.ts
PAY-03	Commission deduction on confirmation	Must	M4	BR-PAY-04	ledger_transaction; ledger_entry; booking	POST /api/v1/me/bookings; POST /api/v1/host/bookings/{booking_id}/accept	`tests/ledger/test_commission_on_confirm.py`	web:wallet.ts
PAY-04	Commission rate resolution	Must	M4	BR-PAY-06	host_account; platform_settings; booking	POST /api/v1/host/bookings/{booking_id}/accept; GET /api/v1/host/wallet; PUT /api/v1/admin/hosts/{host_account_id}/commission-override; DELETE /api/v1/admin/hosts/{host_account_id}/commission-override; GET /api/v1/admin/commission-rates; PUT /api/v1/admin/commission-rates	`tests/ledger/test_rate_resolution.py`	admin:hosts.ts
PAY-05	Manual wallet top-up by staff	Must	M4	BR-PAY-07	wallet_topup; ledger_transaction	POST /api/v1/admin/wallets/{host_account_id}/topups; GET /api/v1/admin/topups; POST /api/v1/admin/topups/{topup_id}/approve; POST /api/v1/admin/topups/{topup_id}/reject	`tests/ledger/test_manual_topup.py`	admin:finance.ts
PAY-07	Negative balance handling	Must	M4	BR-PAY-10	account_balance; platform_settings; notification	POST /api/v1/host/bookings/{booking_id}/accept; GET /api/v1/host/wallet; GET /api/v1/admin/dashboard/attention; GET /api/v1/admin/hosts; PUT /api/v1/admin/hosts/{host_account_id}/credit-limit-override; GET /api/v1/admin/finance/summary; (+1 more)	`tests/ledger/test_negative_balance.py`	admin:finance.ts
PAY-08	Wallet history for hosts	Must	M4		ledger_entry; account_balance	GET /api/v1/host/wallet; GET /api/v1/host/wallet/entries; GET /api/v1/admin/ledger/transactions	`tests/ledger/test_wallet_history.py`	web:wallet.ts
NTF-01	In-app notifications	Must	M4	BR-NTF-01	notification; outbox_event	GET /api/v1/me/notifications; POST /api/v1/me/notifications/{notification_id}/read; POST /api/v1/me/notifications/read-all	`tests/notifications/test_in_app.py`	web:notifications.ts
NTF-02	Email and SMS notifications	Must	M4	BR-NTF-02	outbox_event; delivery_log	no endpoint: internal: notifications.send_email_sms	`tests/notifications/test_email_sms.py`	none (workers)
REV-01	Write a review after a stay	Must	M5	BR-REV-01	review; booking	POST /api/v1/me/bookings/{booking_id}/review; GET /api/v1/me/reviews	`tests/reviews/test_write_review.py`	web:reviews.ts
REV-03	Host reply	Should	M5		review_reply	GET /api/v1/listings/{listing_id}/reviews; GET /api/v1/host/reviews; PUT /api/v1/host/reviews/{review_id}/reply	`tests/reviews/test_host_reply.py`	web:reviews.ts
REV-04	Report and moderate reviews	Must	M5	BR-REV-03	review; review_report; moderation_case	POST /api/v1/reviews/{review_id}/reports; POST /api/v1/admin/moderation/cases/{case_id}/decisions; GET /api/v1/admin/reviews; POST /api/v1/admin/reviews/{review_id}/hide; POST /api/v1/admin/reviews/{review_id}/restore; POST /api/v1/admin/reviews/{review_id}/dismiss-reports; (+1 more)	`tests/reviews/test_review_moderation.py`	admin:reviews.ts
REV-05	Ratings derived from reviews	Must	M5	BR-REV-04	review; listing; host_account; organization	GET /api/v1/listings/{listing_id}/reviews; GET /api/v1/trips/{trip_id}/reviews; GET /api/v1/users/{user_id}/reviews; GET /api/v1/host/reviews; POST /api/v1/admin/reviews/{review_id}/hide; POST /api/v1/admin/reviews/{review_id}/restore; (+1 more)	`tests/reviews/test_ratings.py`	web:reviews.ts

ID	Requirement	Pri.	Milestone	Rules	Tables	API area	Tests	`lib/data` switch
MSG-01	Conversations between guest and host	Must	M5		conversation; conversation_participant; message	GET /api/v1/conversations; POST /api/v1/conversations; GET /api/v1/conversations/{conversation_id}; GET /api/v1/conversations/{conversation_id}/messages; POST /api/v1/conversations/{conversation_id}/messages	`tests/messaging/test_conversations.py`	web:conversations.t
MSG-02	Incremental sync	Must	M5		message	GET /api/v1/conversations/sync; GET /api/v1/conversations/{conversation_id}/messages	`tests/messaging/test_sync_cursor.py`	web:conversations.t
MSG-03	Read state and unread counts	Must	M5		conversation_participant	GET /api/v1/conversations; GET /api/v1/conversations/sync; POST /api/v1/conversations/{conversation_id}/read	`tests/messaging/test_read_state.py`	web:conversations.t
MSG-04	Contact detail masking before confirmation	Should	M5	BR-MSG-01	message	GET /api/v1/conversations/{conversation_id}/messages; POST /api/v1/conversations/{conversation_id}/messages	`tests/messaging/test_contact_masking.py`	web:conversations.t
STF-06	Audited CSV exports	Must	M6	BR-STF-04	export_job; audit_event	POST /api/v1/admin/exports; GET /api/v1/admin/exports; GET /api/v1/admin/exports/{export_id}	`tests/audit/test_exports.py`	admin:analytics.ts
RPT-01	KPIs by period, host type, vertical, wilaya	Should	M6	BR-RPT-01	booking; ledger_entry; listing; host_account	GET /api/v1/admin/finance/summary; GET /api/v1/admin/analytics/kpis; GET /api/v1/admin/analytics/charts	`tests/reports/test_kpis.py`	admin:analytics.ts
RPT-03	Host earnings dashboard	Should	M6	BR-RPT-02	booking; ledger_entry	GET /api/v1/host/earnings	`tests/reports/test_host_earnings.py`	web:wallet.ts
PRV-02	Data subject requests	Must	M6	BR-PRV-02	data_subject_request	GET /api/v1/me/data-requests; POST /api/v1/me/data-requests; DELETE /api/v1/admin/users/{user_id}; GET /api/v1/admin/data-requests; POST /api/v1/admin/data-requests/{data_request_id}/complete	`tests/privacy/test_data_requests.py`	admin:users.ts (priv actions; the module itself switches in M
PRV-04	Retention schedule	Should	M6	BR-PRV-05	host_document; media_asset; data_subject_request	no endpoint: job: privacy.apply_retention	`tests/privacy/test_retention.py`	none (jobs)
PRV-05	Breach response	Must	M6		no table yet (incident register, decision T5 in 13-decisions-needed.md)	no endpoint: none (runbook)	`tests/ops/breach_drill.md (manual)`	none
NFR-01	API response time	Must	M6		all	no endpoint: all /api/v1/*	`tests/perf/locust_api.py`	none
NFR-02	Search response time	Must	M6		listing_search_doc	GET /api/v1/listings	`tests/perf/locust_search.py`	none
PRV-03	Data residency	Must	M7	BR-PRV-04	none (infrastructure and backups)	no endpoint: none	`tests/ops/test_residency_checklist.md (manual)`	none
NFR-03	Availability	Should	M7		none	no endpoint: GET /api/v1/health	`tests/ops/test_health.py`	none
NFR-04	Backups and restore	Must	M7		all	no endpoint: none	`tests/ops/restore_drill.md (manual)`	none
PRC-04	Tourist tax and VAT lines	Should	R2-A	BR-PRC-04	tax_rate; quote; platform_settings	POST /api/v1/quotes	`tests/pricing/test_tax_lines.py`	admin:settings.ts
PAY-06	Online wallet top-up via gateway	Should	R2-A	BR-PAY-08	wallet_topup; gateway_event; payment	POST /api/v1/host/wallet/topups; POST /api/v1/webhooks/gateway/{provider}	`tests/ledger/test_online_topup.py`	web:wallet.ts
PAY-09	Gateway webhooks	Should	R2-A	BR-SYS-02	gateway_event	POST /api/v1/webhooks/gateway/{provider}	`tests/ledger/test_gateway_webhooks.py`	none (server to serv
PAY-11	Commission invoices to hosts	Should	R2-A	BR-PAY-12	invoice	GET /api/v1/host/invoices; GET /api/v1/admin/invoices	`tests/ledger/test_commission_invoices.py`	admin:finance.ts
LST-09	Featured listings	Could	R2-B		featured_placement	PUT /api/v1/admin/listings/{listing_id}/featured	`tests/catalog/test_featured.py`	admin:listings.ts
TRP-06	Cancel a departure under minimum group	Should	R2-B	BR-TRP-03	trip_departure; booking; refund; ledger_transaction	POST /api/v1/host/trips/{trip_id}/departures/{departure_id}/cancel	`tests/trips/test_departure_cancel.py`	web:trips.ts
BKG-11	Disputes	Should	R2-B	BR-CAN-05	dispute; media_asset; moderation_case	POST /api/v1/me/bookings/{booking_id}/dispute; POST /api/v1/host/bookings/{booking_id}/dispute; POST /api/v1/admin/bookings/{booking_id}/dispute/resolve	`tests/bookings/test_disputes.py`	admin:bookings.ts
REV-02	Host reviews guest	Should	R2-B	BR-REV-02	review	POST /api/v1/host/bookings/{booking_id}/guest-review	`tests/reviews/test_double_blind.py`	web:reviews.ts
MSG-05	Attachments	Could	R2-B		message; media_asset	POST /api/v1/media/uploads	`tests/messaging/test_attachments.py`	web:conversations.t

ID	Requirement	Pri.	Milestone	Rules	Tables	API area	Tests	`lib/data` switch
MSG-06	Staff messages and broadcast	Should	R2-B		conversation; message; audit_event	no endpoint: POST /api/v1/admin/messages; POST /api/v1/admin/broadcasts	`tests/messaging/test_staff_broadcast.py`	admin (messages pages, no lib/data module yet)
NTF-03	Notification preferences	Should	R2-B	BR-PRV-03	notification_preference	POST /api/v1/me/consents; GET /api/v1/me/notification-preferences; PUT /api/v1/me/notification-preferences	`tests/notifications/test_preferences.py`	web:notifications.ts
MOD-04	Bulk actions with per-item results	Should	R2-B		moderation_decision; audit_event	POST /api/v1/admin/moderation/bulk-decisions	`tests/trust/test_bulk_actions.py`	admin:approvals.ts
MOD-05	Undo window	Could	R2-B		moderation_decision; audit_event	POST /api/v1/admin/moderation/cases/{case_id}/decisions	`tests/trust/test_undo.py`	admin:approvals.ts
MOD-07	Risk flags	Should	R2-B	BR-MOD-04	moderation_case	GET /api/v1/admin/users	`tests/trust/test_risk_flags.py`	admin:approvals.ts
CFG-03	Onboarding field editor	Should	R2-B	BR-CFG-03	onboarding_flow_version	PUT /api/v1/admin/onboarding-flows/{host_type}/draft	`tests/config/test_field_editor.py`	admin:onboarding.t
PRM-01	Coupons	Could	R2-B	BR-PRM-01	coupon; coupon_redemption; quote	GET /api/v1/admin/coupons; POST /api/v1/admin/coupons; PATCH /api/v1/admin/coupons/{coupon_id}; DELETE /api/v1/admin/coupons/{coupon_id}	`tests/promotions/test_coupons.py`	admin:promotions.t
PRM-02	Who pays the discount	Could	R2-B	BR-PRM-02	coupon; ledger_transaction	POST /api/v1/admin/coupons	`tests/promotions/test_coupon_funding.py`	admin:promotions.t
RPT-02	Host-type by vertical cross-tab	Could	R2-B		booking; listing; host_account	GET /api/v1/admin/analytics/charts	`tests/reports/test_crosstab.py`	admin:analytics.ts
PRV-06	Foreign guest declaration support	Could	R2-B		user (nationality_iso2); booking	no endpoint: PATCH /api/v1/me (nationality, ID type when required)	`tests/privacy/test_foreign_guest_fields.py`	web:auth.ts
AVL-06	External calendar sync (iCal)	Could	LATER		new table needed (ical_feed, not in schema.dbml yet)	no endpoint: GET /api/v1/host/listings/{id}/ical; POST /api/v1/host/listings/{id}/ical-imports	`tests/inventory/test_ical_sync.py`	web:calendar.ts
PAY-10	Payouts and guest payments (model M4)	Could	LATER	BR-PAY-11	payment; payout; payout_account; ledger_account	POST /api/v1/me/bookings/{booking_id}/payments; GET /api/v1/host/payouts; POST /api/v1/host/payouts; POST /api/v1/webhooks/gateway/{provider}; GET /api/v1/admin/payouts; POST /api/v1/admin/payouts/{payout_id}/approve; (+2 more)	`tests/ledger/test_payout_models.py`	admin:finance.ts

4. Known gaps found while building this matrix

Requirement	Gap	Where it is tracked
AVL-06	No table for external calendar feeds in `schema.dbml` (Later release, so not urgent)	T5 in 13-decisions-needed.md
PRV-05	No incident register table; the breach runbook needs one to record the 5-day ANPDP clock	T5 in 13-decisions-needed.md
ACC-02	One-time codes are kept in Redis, not in a table; the audit of code sends relies on `delivery_log`	T5 in 13-decisions-needed.md
MSG-06	The admin messages pages have no `lib/data` module today (screens are mocked inline)	11-roadmap.md R2-B

Check yourself

- [ ] In which milestone is BKG-01 (instant booking) built, and why can it not come before the wallet?
- [ ] Which tables hold the data for AVL-01, and which database feature stops double bookings?
- [ ] Which `lib/data` modules switch to the API in M4?
- [ ] How many requirements are planned for R2-A, and which outside answers (F1 to F4) do they wait for?
- [ ] Find a requirement with no business rule. Is that a problem? Why or why not?
- [ ] If you change BR-PAY-04, which requirements and test files must you look at?
- [ ] Where do you change a mapping so the CSV and this page stay the same?

15. Readiness verdict and first build prompt

> **In plain words**
> This page answers one question: can the team start building the Django backend now?
> Short answer: **yes, start the first milestone (M0, foundations) now**, with three conditions.
> Real customer data must wait for the lawyer's answers on hosting and data transfer (before the end of M5).
> Collecting guests' money must wait for a payment licence decision (not in the first release).
> At the end you will find the exact prompt to give a developer or an AI coding assistant to start M0.

Terms used here (see glossary.md):

- **Milestone (M0 to M7)**: a block of work of 3 to 5 weeks with clear "done" criteria (11-roadmap.md).
- **Gate**: a decision or legal answer that must exist before a milestone can finish.
- **Readiness verdict**: our judgement of whether the plan is complete enough to start coding.

Date: 2026-11-09.

---

1. Verdict

Area	Status	Evidence
What exists today is understood	**Ready**	01-system-inventory.md: every feature, entity and `lib/data` function of both apps, 22 spec disagreements resolved by decisions
Product scope	**Ready**	147 requirements, 126 in the first release (03-requirements.md); 15 product decisions taken (DEC-001 to DEC-015)
Business rules	**Ready**	Testable rules with state machines and worked money examples (04-business-rules.md)
Database design	**Ready**	77 tables, 35 enumerations, 156 foreign keys, database-level guarantees against double booking and unbalanced money (05-database-schema.md, schema.dbml, schema.sql)
API contract	**Ready**	266 operations, valid OpenAPI 3.1 (lint: 0 errors, 0 warnings), every website and admin data function mapped (06-api-design.md, openapi.yaml)
Architecture, security, hosting, testing	**Ready**	07, 08, 09, 10
Plan, risks, decisions	**Ready**	11 milestones, 32 risks, 49 decisions listed with owners and deadlines (11, 12, 13); traceability for all 147 requirements (traceability.md)
Legal and tax answers	**Not ready (not needed to start)**	L1 to L16 and F1 to F7 open; they gate production data (end of M5), payments (R2) and the licence question (later)

Verdict: GO for M0 now. GO for production launch only after the gates in section 3.

2. Conditions to start M0 (answer in the first week)

#	Decision	Recommended answer	Who
1	T2: where the backend code lives	`backend/` folder in this repository	Tech lead
2	T3 and T9: how the apps switch from demo data to the API, and the typed client	One flag per `lib/data` module, API seeded with the same demo cast; `openapi-typescript` + `openapi-fetch`	Tech lead
3	T4: how the Next.js servers hold the session	allauth headless session token in an httpOnly cookie on the Next.js server	Tech lead
4	Q22: guests without an Algerian mobile number	Email code for non-Algerian numbers, with stricter booking limits	Product owner
5	T1 (staging only) and T8 (third-party processors)	Staging anywhere with demo data only; prefer Algerian or self-hosted services for anything with personal data	Tech lead, then lawyer (L10)

And one action that cannot wait: **engage a lawyer now** for L1, L2, L4, L5, L9 and L11. Their answers take months and block production (M5 to M7).

3. Gates before going live

Gate	Blocks	Source
L1, L2: hosting in Algeria and personal data transfer abroad	Signing production hosting (end of M5); any real personal data	02-research.md C.3, risks K-01 and K-02
L4, L5, L9, L11: host documents, short-term rental rules, platform licence, company formalities	Real hosts joining (M7)	13-decisions-needed.md
F1 to F4: VAT on commission, tourist tax, gateway contract, invoices	Online wallet top-up and invoices (R2-A)	13-decisions-needed.md
L3 and a payment licence or bank partner	Collecting guests' money (models M2 to M4)	DEC-001, BR-PAY-11
`NEXT_PUBLIC_BLADUS_TEST_MODE=0` in production builds	Public launch	09-devops-hosting.md

4. Known gaps carried into the build

These are small and tracked; none blocks M0.

- Two R2 features have no endpoint yet: staff broadcast messages (MSG-06) and the foreign-guest declaration (PRV-06). Design them when R2-B starts.
- 18 requirements are jobs or quality targets and have no endpoint by nature (for example BKG-04 automatic completion, NFR-01 response times); they are covered by jobs (07-architecture.md) and tests (10-testing-quality.md). See the "no endpoint" rows in traceability.csv.
- Someone who is both a solo host and an organisation member cannot yet reach their solo workspace from the website's workspace switcher (the API supports it: `GET /api/v1/me/workspaces` lists every host account, including the solo one).
- Dates, costs and SMS prices are estimates; legal and tax points are research, not advice (VERIFY WITH A PROFESSIONAL).

---

5. First build prompt (M0)

Give this prompt, unchanged, to the developer or AI coding assistant who starts the backend. It is self-contained but expects the `docs/backend` folder to be in the repository.

```
You are starting the BLADUS backend (Algerian travel marketplace: homes, hotels,
hostels, experiences, vehicles, agency trips). The full blueprint is in
docs/backend/ of this repository. Read, in this order, before writing code:
00-START-HERE.md, 03-requirements.md, 04-business-rules.md,
05-database-schema.md, 06-api-design.md, 07-architecture.md,
08-security-privacy.md, 09-devops-hosting.md, 10-testing-quality.md,
11-roadmap.md (section M0), 13-decisions-needed.md (T1 to T4, T8, T9, Q22).

Goal: deliver milestone M0 "Foundations" exactly as defined in 11-roadmap.md:
requirements ACC-01, ACC-02, ACC-03, ACC-04, ACC-07, PAY-12, SRC-04, CFG-01,
PRV-01, NFR-05 to NFR-13. Nothing from later milestones.

Build:
1. A Django 6.1 project in backend/ (Python 3.13) with the folder layout of
   07-architecture.md. Create only the apps M0 needs: core, accounts, geo,
   config, audit, notifications (outbox only). Use the services/selectors
   pattern and import-linter layers from 07-architecture.md.
2. PostgreSQL 17 + PostGIS, Redis, Celery worker and beat, all in Docker
   Compose for local development (09-devops-hosting.md). Settings with
   pydantic-settings; no secrets in code.
3. Tables for M0 only, matching docs/backend/schema.dbml exactly (names,
   types, enums, constraints): user, consent_record, staff_profile,
   staff_permission, staff_role_permission, wilaya, daira, commune,
   platform_settings, audit_event, outbox_event, delivery_log, otp_send_log,
   idempotency_record. Money is BIGINT centimes + currency; IDs are UUIDv7.
4. Accounts with django-allauth headless (session tokens, email and phone
   one-time codes, rate limits of BR-ACC-01, BR-ACC-02, BR-ACC-05) and the
   /api/v1/auth/* and /api/v1/me endpoints exactly as in
   docs/backend/openapi.yaml (same paths, schemas, error codes).
5. Geography seed: 69 wilayas and 1,541 communes from
   el-amin-dev/algeria-wilayas-communes (CC0) with coordinates from
   ihahachi/Algeria-Cities (MIT), keeping source and valid_from columns
   (BR-SRC-02), plus the read endpoints in openapi.yaml.
6. platform_settings v1 with the defaults of 05-database-schema.md section 9
   (commission bp 1200/1500/1800/1000, request TTL 24 h) and the error
   envelope of 06-api-design.md (stable codes, en/fr/ar messages).
7. drf-spectacular generates the schema in CI; CI fails if the generated
   schema differs from docs/backend/openapi.yaml for the M0 paths.
8. Tests (10-testing-quality.md): pytest-django, factory_boy, hypothesis for
   the Money and UUIDv7 helpers, one test per business rule named
   test_br_<area>_<nn>_..., a test that counts 69 wilayas and 1,541 communes,
   and the auth flow end to end. Coverage gate as in 10-testing-quality.md.
9. GitHub Actions: ruff, mypy with django-stubs, pytest with a PostGIS
   service, dependency scan, OpenAPI check.
10. Website integration for M0 only: behind the flag DATA_SOURCE_AUTH
    (default mock), make frontend/lib/data/auth.ts call the API through a
    typed client generated with openapi-typescript + openapi-fetch, using the
    BFF pattern (allauth session token in an httpOnly cookie on the Next.js
    server). Do not change any other frontend module.

Rules: follow the decisions DEC-001 to DEC-015; do not invent endpoints or
columns that are not in openapi.yaml or schema.dbml (if something is missing,
stop and write it in docs/backend/14-open-questions.md); no personal data
outside demo fixtures; never use the em dash character in code comments or
docs; keep internal "darna" identifiers that already exist. Finish with: the
M0 exit criteria of 11-roadmap.md checked one by one, test results, and a
CHANGELOG entry in docs/backend/CHANGELOG.md.
```

Check yourself

- [ ] Can the team start coding today? Which milestone, and what must be decided in the first week?
- [ ] Which two legal answers must arrive before production hosting can be signed, and by when?
- [ ] Why is collecting guests' money not part of the first release?
- [ ] Which two R2 features still have no API endpoint?
- [ ] In the first build prompt, what must the developer do if a column or endpoint they need is missing from the blueprint?

BLADUS frontend inventory (raw analysis notes for the backend blueprint)

Scope: `frontend/` only (guest and host website, Next.js 14 App Router, next-intl en/fr/ar). Admin app not covered. All paths are relative to `frontend/` unless stated. `file:line` refers to the state of the tree on 2026-11-04 (branch `main`, commit `03fc50c`). Persistence today: `lib/data/*` over in-memory `globalThis` arrays (`lib/data/_store.ts:14`, `lib/data/_tables.ts:31-62`) seeded from `lib/mock-data/*`; some tables are seeded inline inside `lib/data` (`calendar.ts:13`, `wallet.ts:16`, `reviews.ts:6`, credentials in `auth.ts:39`). Every `lib/data` file carries `// TODO: replace with Supabase query`: the comments predate the Django + PostgreSQL decision.

Legend for status: **built** = UI + server action/data function round trip works end to end on the mock store; **partial** = works but limited or inconsistent; **mocked-only** = UI exists, action is a toast, local state, hardcoded data or `localStorage`.

---

1. Feature inventory per actor

1.1 Visitor (not logged in)

Feature	Route(s) / component(s)	Status	Notes
Home editorial feed (homes, experiences, agency trips, hotels/hostels, vehicles, "weekend" featured, wilaya tiles, trust band, host CTA)	`app/[locale]/page.tsx`, `components/home/*`	built	Uses `getListings`, `getVehicles`, `getExperiences`, `getFeaturedListings`, `getTrips`. Card DTO slimming in `lib/server/card-dto.ts`. Category chip filter (`?category=`) is hardcoded string matching on `type`, `amenities`, `city` (`app/[locale]/page.tsx:77-94`). Wilaya tile counts by normalized city (`page.tsx:99-103`).
Global search bar (where / when / who)	`components/navbar/Search.tsx`, `navbar/search/{Where,When,Who}Popover.tsx`, `models/SearchModal.tsx`	built	Destinations list is static (`components/search/destinations.ts:14-25`, 10 cities).
Search results with side filters, facet counts, map, "show total price" toggle	`app/[locale]/search/page.tsx`, `components/search/SearchResultsClient.tsx`, `SearchFilterSidebar.tsx`, `SearchMap.tsx`, `searchFacets.ts`, `models/FilterModal.tsx`, `hook/useShowTotalPrice.ts`	built	Base search server side (`getListings`), side filters applied server side with the same matchers as the client (`search/page.tsx:62-65`). Map is OpenFreeMap (`components/map/OpenFreeMapLayer.tsx`). Total price toggle stored in `localStorage` `darna_show_total_price`. No sort, no pagination.
Vertical tabs (all, home, hotel, vehicle, experience, trip) with counts	same	built	`categoryCounts` computed in `search/page.tsx:67-71`.
Listing detail (home, vehicle, experience): gallery, lightbox, info, amenities, house rules, policies, reviews summary, map, similar listings, booking widget preview	`app/[locale]/listings/[listingId]/page.tsx`, `components/listing/ListingClient.tsx`, `ListingHead.tsx`, `ListingInfo.tsx`, `ListingReservation.tsx`, `PhotoLightbox.tsx`, `detail/*`	built	Availability and price overrides built server side and passed as props (`listings/[listingId]/page.tsx:89-90`). "Similar listings" = first 4 of same vertical, not geo based (`page.tsx:93-96`).
Hotel / hostel detail with room-type cards and per-room or per-bed booking	`components/listing/HotelHostelDetailClient.tsx`, `listing/hotel/RoomCard.tsx`	built	Chosen when listing has `roomTypes`.
Report listing	`ListingReservation.tsx:439`, `HotelHostelDetailClient.tsx:767`	mocked-only	`toast.info(reportSuccess)`; no entity, no action.
Share listing / trip	`ListingHead.tsx:69`, `TripDetailClient.tsx:136`	built (client)	Web Share / clipboard only.
Trip (agency circuit) detail with departures and seats left	`app/[locale]/trips/[tripId]/page.tsx`, `components/trips/TripDetailClient.tsx`	built	Seats sold per departure server side (`seatsSoldByDeparture`). "Save trip" heart is a toast (`TripDetailClient.tsx:336`): trips cannot be favorited. No "message agency" button.
Public host profile (individual)	`app/[locale]/users/[userId]/page.tsx`, `components/profile/UserProfileClient.tsx`, `ProfileHero.tsx`	built	Public projection via `toPublicUser` (`lib/server/dto.ts:12`). Lists the user's individual listings and verified businesses they own.
Public organization profile (agency / hotel / hostel)	`app/[locale]/{agencies,hotels,hostels}/[organizationId]/page.tsx`, `components/agency/orgProfilePage.tsx`, `AgencyProfileClient.tsx`	partial	Shows org facts, active team members, active listings, agency trips. Shows unverified orgs too (only a "pending verification" label, `messages/en/agencyPublic.json`). Category mismatch redirects 308 to the right route (`orgProfilePage.tsx:59-61`). No contact form.
Help center (FAQ search, categories, helpful vote, contact form)	`app/[locale]/help/page.tsx`, `components/help/HelpClient.tsx`	mocked-only	6 static articles in messages; feedback vote toast (`HelpClient.tsx:105`); contact form is a `setTimeout` + toast (`HelpClient.tsx:119-125`).
Register (email + password)	`components/models/LoginModal.tsx` (`AuthPanel`), `app/api/auth/register/route.ts`	built	Min password 8 (`lib/auth/password-rules.ts:2`). No email verification.
Login	`app/[locale]/login/page.tsx`, `LoginForm.tsx`, `LoginModal.tsx`, `app/api/auth/login/route.ts`	built	Demo quick-login buttons with hardcoded password `darna-demo-2026` (`LoginModal.tsx:46-51`).
Forgot password, social login, phone login	`LoginModal.tsx:330`, `:360`, `:374`	mocked-only	Toasts (`forgotPasswordSoon`, `socialSimulated`, `phoneSimulated`).
Language switch (en / fr / ar, RTL)	`components/navbar/LanguageSwitcher.tsx`, `middleware.ts`, `i18n.ts`	built	36 namespaces, 3417 keys per locale, full parity en/fr/ar (checked by script).
Currency display (DZD + approximate EUR)	`hook/useCurrency.ts`, `lib/currency.ts:1`, `components/common/PriceDisplay.tsx`	partial	Fixed rate `DEFAULT_EUR_TO_DZD = 152.47` (`lib/currency.ts:1`); no live rate source.
SEO: sitemap, robots, OG metadata, hreflang	`app/sitemap.ts`, `app/robots.ts`, `lib/site.ts`	built	Sitemap lists active listings, active trips, verified orgs.

1.2 Guest (logged in traveller)

Feature	Route(s) / component(s)	Status	Notes
Book a home / vehicle / experience (instant or request-to-book, optional message to host)	`ListingReservation.tsx -> createBookingAction`	built	Server recomputes quote and validates rules (`app/actions/bookings.ts:78-161`). No payment step: reservation is immediately `paymentStatus: "paid"` for instant bookings with `paymentMethod` defaulted to `"CIB / Edahabia Card"` (`lib/data/reservations.ts:291-293`).
Book hotel rooms / hostel beds	`HotelHostelDetailClient.tsx -> createBookingAction`	built	Beds: `unitCount` forced to `guestCount` (`bookings.ts:115-118`).
Book an agency trip seat on a departure	`TripDetailClient.tsx -> createBookingAction({kind:"trip"})`	built	Approval mode from the organization (`bookings.ts:190-193`).
Booking success dialog	`components/listing/detail/BookingSuccessDialog.tsx`	built	
My trips (upcoming / past / cancelled tabs, receipt dialog, cancel with refund preview)	`app/[locale]/trips/page.tsx` , `components/trips/TripsClient.tsx` , `TripCard.tsx`	built	Refund preview computed client side with the same pure function (`TripsClient.tsx:70-75`), recomputed server side. Receipt "download" is `window.print` of a print area (`TripsClient.tsx:397-508`). "Message host" links to the generic inbox (`TripsClient.tsx:311`), not to a booking thread.
Favorites / wishlist (listings only)	`components/HeartButton.tsx` , `hook/useFavorite.ts` , `app/[locale]/favorites/page.tsx` , `components/favorites/FavoritesClient.tsx`	built	Stored as `User.favoriteIds: string[]` (`types.ts:15`). No named wishlists. Favoriting a draft listing id is allowed (`app/actions/favorites.ts:13` uses raw `getListingById`).
Messaging (inbox, thread, send, mark read, start thread from listing)	`app/[locale]/messages/page.tsx` , `messages/[conversationId]/page.tsx` , `components/messages/MessagesClient.tsx` , `ListingInfo.tsx:92`	partial	No realtime, no polling. Attachments (`MessagesClient.tsx:762`), conversation settings (`:395`) and options (`:638`) are toasts. Start thread exists only from listing pages (not trips, not bookings).
Notifications (dropdown + page, mark one / all read)	`components/navbar/NotificationsDropdown.tsx` , `app/[locale]/notifications/page.tsx` , `components/notifications/NotificationsClient.tsx`	partial	Read and mark-read are real; nothing in the code ever creates a notification (only 4 seeded rows for `usr_yacine`).
Own profile view + edit (name, bio, phone, address)	`app/[locale]/profile/page.tsx` , `UserProfileClient.tsx -> updateProfileAction`	built	Photo change is a toast (`UserProfileClient.tsx:186`). `profile.json` has unused keys for trips/favorites/security tabs, emergency contact, passport (copy implies features not built).
Account settings: personal info	`app/[locale]/settings/page.tsx` , `components/settings/SettingsClient.tsx` , `PersonalSection.tsx`	partial	Legal name, phone, address saved via `updateProfileAction` . Preferred name and mailing address are toasts only (`PersonalSection.tsx:199` , `:277`). Email change is a toast (`:215-218`).
Account settings: change password	`SecuritySection.tsx -> changePasswordAction`	built	Sessions list / other devices is "coming soon" (`SecuritySection.tsx:140`).
Account settings: deactivate account	`SettingsClient.tsx:212`	mocked-only	Toast `deactivateSoon` .
Account settings: notifications matrix, payments (saved cards, payout account), languages and currency, privacy (search indexing, data export), taxes (NIF), booking permissions, travel for work, pro hosting	`NotificationsSection.tsx` , `PaymentsSection.tsx` , `LanguagesSection.tsx` , `HostingSections.tsx`	mocked-only	All local `useState` + toasts. Saved cards are hardcoded (CIB `4242` , Edahabia `8890` , BNA RIP) in `PaymentsSection.tsx:21-102` . Currency preference not persisted (`LanguagesSection.tsx:24`).
Accessibility (reduced motion)	`AccessibilitySection.tsx` , `components/motion/MotionProvider.tsx`	built (device)	`localStorage` only.
Leave a review	none	missing	No UI, no action, no data function writes reviews.
Report a user, block a user	none	missing	
Become a host entry	`components/navbar/UserMenu.tsx` , `/become-a-host`	built	See 1.3 / 1.4.

1.3 Host, individual

Feature	Route(s) / component(s)	Status	Notes
Become a host: choose host type	`app/[locale]/become-a-host/page.tsx` , `components/host/BecomeHostFlow.tsx` , `onboarding/HostTypeStep.tsx`	built	4 host types (`HostTypeStep.tsx`).
Individual listing wizard (vertical, category, placement, location, capacity, amenities, photos, title/description, approval mode, price, cancellation policy, review, publish)	`/become-a-host/[step]` , `BecomeHostFlow.tsx` , `DynamicStepFields.tsx` , `lib/onboarding-config.ts`	partial	Draft saved in `localStorage` keyed by user (`lib/client/user-storage.ts:7-41`). Photos are URLs typed by the host; fewer than 4 photos are silently replaced by 4 Unsplash stock photos (`BecomeHostFlow.tsx:628-637`). `city` sent as free text `"<city>, <wilaya>"` and no `commune` (`:659-660`). `placement` , `bedsCount` and admin-configured `customFieldsData` are collected but never sent (`:113-116` , `:175` , payload `:651-672`). Cleaning fee, min/max nights, advance notice, check-in/out times are not asked. Listing goes straight to `status: "active"` (no moderation, `lib/data/listings.ts:309`).
Host workspace guard	`app/[locale]/host/layout.tsx` , `lib/server/host-context.ts:47-60` , `middleware.ts:73-85`	built	Individual hosts with 0 listings are sent back to `/become-a-host/1` (`host/layout.tsx:25-30`). Individual hosts never see `/host` "Today": redirected to `/host/listings` (`host/page.tsx:20-22`).
My listings (grid / table, stats, search, filter, pause/activate, edit title/price/instant book, manage room types, delete)	`app/[locale]/host/listings/page.tsx` , `components/host/HostListingsClient.tsx`	partial	Edit modal only sends `title` , `price` , `instantBook` (`HostListingsClient.tsx:188-192`) although `updateListing` accepts ~45 fields. No editing of description, photos, location, fees, stay rules, cancellation policy. Delete is a hard delete.
Bookings (tabs all/pending/upcoming/current/past/cancelled, search, filter by listing, request timer, accept / decline with reason / cancel, net payout breakdown, mini calendar block/unblock)	`app/[locale]/host/bookings/page.tsx` , `components/host/HostBookingsClient.tsx`	built	Payout via `computeHostPayout` (`HostBookingsClient.tsx:589` , `:842`). "Message guest" links to `/host/messages` generic inbox (`:734`).
Calendar (month / week / day / year views, per listing, block / reopen dates and ranges, per-date custom price)	`app/[locale]/host/calendar/page.tsx` , `components/host/HostCalendarClient.tsx`	built	Price overrides refused for room-type listings (`app/actions/calendar.ts:74-76`). Blocking is per listing, not per room type.
Messages (host side)	`app/[locale]/host/messages/page.tsx` , `MessagesClient.tsx`	partial	Same component as guest inbox.

Feature	Route(s) / component(s)	Status	Notes
Prepaid wallet widget (balance, commission rate, ledger, negative balance alert, offline recharge instructions)	`components/host/HostWalletWidget.tsx` (in `HostListingsClient.tsx:378`, `HostTodayClient.tsx:515`, `AgencySettingsClient.tsx:185`) -> `getMyWalletAction`	built (read-only)	Recharge is offline, credited by admin (`messages/en/hostWallet.json`).
Host settings: payout methods (CCP / BNA / BEA), payout history	`app/[locale]/host/settings/page.tsx`, `components/host/HostSettingsClient.tsx`	mocked-only	Hardcoded payout methods (`HostSettingsClient.tsx:79-95`) and `PAYOUT_HISTORY` (`:63-66`); add / set default are toasts (`:141`, `:154`).
Host settings: identity / upgrade to agency	`HostSettingsClient.tsx:354`	mocked-only	Toast `agencyUpgradeRequested`.
Host settings: booking defaults (approval mode, advance notice, prep time, check-in/out, cancellation policy)	`HostSettingsClient.tsx:98-163`	partial	Only approval mode is saved (`setBookingApprovalModeAction`), onto `User.bookingApprovalMode`, which the booking engine never reads. Other defaults are local state.
Host settings: communication (SMS, email, auto welcome message)	`HostSettingsClient.tsx:109-111`, `:476`	mocked-only	Toast.
Host reservations page (account area variant)	`app/[locale]/reservations/page.tsx`, `components/reservations/ReservationsClient.tsx`	built	Duplicate of host bookings with KPIs (revenue, occupancy). Guests are redirected to `/trips`.
Legacy "properties" page	`app/[locale]/properties/page.tsx`	redirect	Redirects to `/host/listings`.

1.4 Host, agency / hotel / hostel team member

Feature	Route(s) / component(s)	Status	Notes
Agency registration (5 steps: basics, details, team invites, approval mode, review)	`/become-a-host/agency`, `BecomeHostFlow.tsx:676-708`	partial	Agency approval mode step exists in `onboarding-config.ts` (`agencyBookingMode`) but the payload omits `bookingApprovalMode` (`BecomeHostFlow.tsx:692-703`), so agencies always start `instant`.
Hotel registration (7 steps incl. star rating, total rooms, check-in/out, amenities, team, approval mode, rooms)	`/become-a-host/hotel`, `BecomeHostFlow.tsx:710-819`	partial	Creates org + one property listing. Room types are hardcoded ("Standard Deluxe Room" 24000 x20, "Executive Sea View Suite" 42000 x8) regardless of input (`:777-813`); property price hardcoded 24000. `hotelTaxId` collected, never sent.
Hostel registration (7 steps incl. beds, private rooms, facilities, curfew, age policy)	`/become-a-host/hostel`, `BecomeHostFlow.tsx:821-900`	partial	Same pattern: hardcoded dorm/private room types and price 3200.
Org pending verification banner	`components/host/HostLayoutShell.tsx:40-64`	built	Workspace usable while `pending`; `rejected` shows a red banner, nothing is blocked.
Today dashboard (KPIs: gross earnings, occupancy, active bookings, rating; pending requests; check-ins today; in-house; recent reviews; upcoming)	`app/[locale]/host/page.tsx`, `components/host/HostTodayClient.tsx`, `lib/host-kpis.ts`	partial	Reviews come from `getReviewsByUserId(ctx.user.id)` which matches `listing.hostUserId === userId` (`lib/data/reviews.ts:112`): seeded org listings have no `hostUserId`, so org reviews are always empty.
Listings / bookings / calendar / messages / wallet	same routes as 1.3	built / partial	Scoped by `organizationId` (`lib/server/host-context.ts:63-67`). Org hosts add listings through the individual wizard; it publishes under the org (`app/actions/host.ts:78-89`).
Agency trips list (KPIs, search, filter, publish/pause, delete)	`app/[locale]/host/agency/trips/page.tsx`, `components/host/agency/AgencyTripsClient.tsx`	partial	No edit of an existing trip (no `updateTripAction` for content).
Create trip (basics, itinerary, logistics, inclusions, review; save draft or publish)	`app/[locale]/host/agency/trips/new/page.tsx`, `components/host/agency/TripCreationClient.tsx`	built	No `commune`, no coordinates input (derived from city text, `lib/data/trips.ts:85`).
Team (roster, invite by email with role, change role, remove, transfer ownership, resend invite, permission matrix)	`app/[locale]/host/agency/team/page.tsx`, `components/host/AgencyTeamClient.tsx`	partial	Invites create `status: "invited"` rows with `userId: null`; there is no accept-invite flow and no email sent, so invited members can never become active. Resend is a toast (`AgencyTeamClient.tsx:179-182`).
Organization settings (profile, legal fields, approval mode, delete org)	`app/[locale]/host/agency/settings/page.tsx`, `components/host/AgencySettingsClient.tsx`	built	Legal field change resets verification to `pending` (`lib/data/organizations.ts:269`). Settings form does not expose hotel/hostel operational fields (star rating, amenities, facilities) even though the patch sanitizer accepts check-in/out, curfew, age policy (`app/actions/organizations.ts:52-55`). Delete org is a hard delete plus listings/trips set to draft.
Switch between organizations	none	missing	A user can own several orgs (seed: Karim owns 3, Amina owns 2) but there is no switcher; active org = session org or first membership (`lib/server/active-org.ts:22-23`).

---

2. Data inventory

Row counts are seed counts. "Store" = globalThis key.

2.1 Enums (all values)

Enum	Values	Defined
UserRole	`guest`, `host`, `admin`	`types.ts:1`
HostType	`individual`, `agency`, `hotel`, `hostel`	`types.ts:2`
CancellationPolicy	`flexible`, `moderate`, `strict`	`types.ts:45`
ListingVertical (Listing.category)	`home`, `hotel`, `experience`, `vehicle`, `trip`	`types.ts:47-52`
ListingOwnerType	`individual`, `organization`	`types.ts:53`
ListingStatus	`active`, `pending`, `suspended`, `draft` (owner may set only `active`, `draft`: `lib/data/_sanitize.ts:124`)	`types.ts:54`
Amenity.category	`essentials`, `features`, `location`, `safety`	`types.ts:60`
BookingUnit	`room`, `bed`	`types.ts:90`

Enum	Values	Defined
businessCategory (Listing, Organization)	`agency`, `hotel`, `hostel`	`types.ts:147`, `:300`
Listing.transmission	`automatic`, `manual`	`types.ts:182`
Listing.fuelType	`petrol`, `diesel`, `electric`, `hybrid`	`types.ts:184`
approval mode (User, Listing, Organization, Reservation)	`instant`, `request`	`types.ts:12`, `:150`, `:325`, `:240`
ReservationStatus	`confirmed`, `pending`, `cancelled`, `completed`, `declined`, `expired`	`types.ts:198-204`
PaymentStatus	`paid`, `pending`, `refunded`, `partially_refunded`, `voided`	`types.ts:206-211`
Reservation.cancelledBy	`guest`, `host`, `system` (`system` never set in code)	`types.ts:236`
Trip.status	`active`, `draft`	`types.ts:283`
OrganizationVerificationStatus	`pending`, `verified`, `rejected`	`types.ts:292-295`
Organization.teamSizeEstimate	`1`, `2-10`, `11-50`, `50+`	`types.ts:312`
MembershipRole	`owner`, `admin`, `manager`	`types.ts:361`
MembershipStatus	`active`, `invited`, `removed` (`removed` never set: removal is a hard delete)	`types.ts:362`
NotificationType	`booking`, `message`, `price`, `system`, `review`	`types.ts:488-493`
TransactionType	`recharge`, `commission_deduction`, `payout`, `adjustment` (`payout` never written)	`types.ts:574-578`
Wallet.ownerType	`user`, `organization`	`types.ts:594`
Wallet / Transaction currency	`DZD` only	`types.ts:585`, `:598`
OnboardingFieldType	`text`, `number`, `textarea`, `select`, `multi-select`, `image-upload`, `toggle`	`types.ts:507-514`
PricingModel (not in types.ts)	`per_night`, `per_room_night`, `per_bed_night`, `per_person`, `per_day`, `per_traveller`	`lib/booking/pricing.ts:27-33`
Coupon.discountType (admin type, unused here)	`percentage`, `fixed`	`types.ts:477`
AuditLogEntry.targetType (admin type, unused here)	`listing`, `booking`, `user`, `review`, `setting`, `organization`	`types.ts:462-468`
KpiPeriod	`month`, `30days`, `year`	`lib/host-kpis.ts:10`
Action error codes	66 codes (generic, auth, booking, calendar, messages, orgs, trips)	`lib/action-result.ts:8-83`

2.2 Entities

User (`types.ts:4-24`), store `__darna_users__`, seed `lib/mock-data/users.ts` (6 rows)

Field	Type	Notes
id	string	`usr_<uuid>` for new users (`lib/data/auth.ts:104`)
name	string	max 80
email	string	normalized lower case; unique (checked in `registerUser`)
image	string | null	only local paths `/...` accepted on update (`auth.ts:177`); no upload
role	UserRole	`guest -> host` when onboarding (`auth.ts:152`)
hostType	HostType | null	one host type per user (cannot be individual and org at once)
organizationId	string | null	"preferred" org; real link is Membership
bookingApprovalMode	instant | request	written by host settings; unused by booking engine
createdAt	ISO string	
emailVerified	string | null	never written
favoriteIds	string[]	listing ids (wishlist)
bio, phone, address	string	max 1000 / 40 / 200
languages	string[]	free text, max 10
isIdentityVerified	boolean	no verification flow
yearsHosting, responseRate, responseTime	number, number, string	stored static, should be derived

Seed rows: `usr_amina` (host individual, also owns 2 hostel orgs), `usr_karim` (host agency, owns agency + 2 hotels, also has 3 individual listings), `usr_yacine` (guest), `usr_admin` (admin), `usr_sofiane` (guest), `usr_leila` (guest). Credentials table (`auth.ts:39-44`, store `__darna_credentials__`): `userId -> "scrypt$salt$hash"`, all seeded users share `DEMO_PASSWORD = "darna-demo-2026"` (`auth.ts:14`).

PublicUser (`types.ts:30-43`) = projection without email, phone, address, favoriteIds, organizationId, bookingApprovalMode.

Listing (`types.ts:110-196`), store `__darna_listings__`, seed `lib/mock-data/listings.ts` (18 rows)

Group	Fields (type)
Identity / ownership	id (string), ownerType (ListingOwnerType), hostUserId (string|null), organizationId (string|null), businessCategory (agency|hotel|hostel), status (ListingStatus), createdAt, updatedAt
Content	title, description, subtitle (string), category (ListingVertical), type (free text subtype, e.g. "Villa", "SUV / 4x4", "Boutique Hotel"), imageSrc (string[] URLs)
Location	locationValue (string, "City or country code"), city (free text), commune (free text matched to `lib/algeria.ts`), country (always "Algeria" on create), latlng ([lat,lng])
Merchandising	featured, isGuestFavorite (boolean, manual flags), rating, reviewCount, bookingsCount (numbers stored, not derived)

Group	Fields (type)
Availability window	availableFrom, availableTo (date keys; `availableTo` = last checkout date)
Search-only computed	matchedAvailability (MatchedAvailability), quotedTotal (number)
Booking	instantBook (boolean), bookingApprovalMode (instant|request), cancellationPolicy
Pricing	price (DZD int; per night / per day / per person by vertical), originalPrice, cleaningFee, serviceFeeRate (ignored), taxRate
Stay rules	minNights, maxNights, advanceNoticeDays, checkInTime, checkOutTime ("HH:mm")
Home	guestCount, roomCount, bathroomCount, bedroomCount, amenities (string[] snake_case keys), houseRules (string[] free text)
Hotel / hostel	hotelStarRating, roomTypes (RoomType[] embedded)
Vehicle	make, model, year, transmission, seats, fuelType, pickupLocation, mileagePolicy, insuranceNotes
Experience	activityName, durationHours, groupSize, perPersonPrice (unused by pricing), includedItems, experienceLanguages

Seed composition: 7 homes (`lst_tipaza_villa` , `lst_algiers_casbah` , `lst_casbah_loft` , `lst_oran_seafront` , `lst_constantine_cliff` (request), `lst_tlemcen_andalusian` , `lst_ghardaia_ksar`), 4 vehicles (`lst_toyota_landcruiser` , `lst_nissan_patrol` , `lst_dacia_duster` , `lst_renault_clio`), 3 experiences (`lst_djanet_tassili` , `lst_taghit_dunes` , `lst_casbah_culinary`), 2 hotels (`lst_hotel_el_djazair` , `lst_hotel_royal_oran`), 2 hostels (`lst_casbah_hostel` , `lst_hostel_taghit_desert`). Hotels/hostels use `category: "home"` + `businessCategory` . All `status: "active"` . 10 owned by orgs (`org_atlas_escapes` 6: algiers_casbah, ghardaia_ksar, toyota_landcruiser, nissan_patrol, djanet_tassili, taghit_dunes; one each for the 2 hotels and 2 hostels), 8 by individuals (`usr_amina` 5: tipaza_villa, oran_seafront, tlemcen_andalusian, renault_clio, casbah_culinary; `usr_karim` 3: casbah_loft, constantine_cliff, dacia_duster).

RoomType (`types.ts:92-108`), embedded in `Listing.roomTypes` (10 rows across 4 listings)

Field	Type	Notes
id	string	kept if `/^[\w-]{1,80}$/` else `rt_<uuid>` (`_sanitize.ts:100`)
propertyListingId	string	parent listing
name, description, bedsDescription	string	
price	number	nightly per room or per bed
basePrice	number	mirrors price; pricing reads `basePrice ?? price` (`pricing.ts:72-76`)
bookingUnit	room | bed	
totalInventory	number	1 to 10000
inventoryCount	number	used first by `roomTypeInventory` (`pricing.ts:78-82`)
availableInventory	number	never written
capacity	number	max guests per unit
amenities	string[]	free English text ("King bed", "Privacy curtain"), different vocabulary than listing amenities
imageSrc	string[]	
isAvailable	boolean	never read by booking engine

Seed: El Djazair hotel (deluxe x12 @18000, executive x4 @32000, family x3 @40000), Casbah hostel (6-bed mixed bed x18 @3500, 4-bed female bed x8 @4200, private double x4 @9500), Royal Oran (superior x18 @24000, presidential x2 @48000), Taghit hostel (8-bed dorm bed x24 @3200, adobe private x6 @7800).

Reservation (`types.ts:213-250`), store `__darna_reservations__` , seed `lib/mock-data/reservations.ts` (9 rows)

Field	Type	Notes
id	string	`res_<uuid>`
userId	string	guest
listingId	string	for trips equals the trip id (`reservations.ts:349`)
tripId	string?	set for trip seat bookings
startDate, endDate	date key `yyyy-MM-dd`	half-open; experiences start = end; trips end = departure + duration - 1
totalPrice, subtotal, nightlyPrice (average unit price), cleaningFee, serviceFee, taxes	number DZD	snapshot of the server quote
guestsCount	number	
unitCount	number?	rooms or beds
selectedRoomTypeId	string?	
status	ReservationStatus	
paymentStatus	PaymentStatus	
paymentMethod	string	free text, default "CIB / Edahabia Card"
approvalMode	instant | request	
expiresAt	ISO	request + 24h
guestMessage, declineReason	string	max 2000 / 500
respondedAt, cancelledAt	ISO	
cancelledBy	guest | host | system	
cancellationPolicy	snapshot	

Field	Type	Notes
refundAmount	number	
commissionAmount, commissionRate	number	written after wallet deduction
listing, trip, user	populated relations (not stored)	`user` = guest contact projection incl. email + phone (`reservations.ts:17-41`)

Seed: 4 for `usr_yacine` + others; statuses: confirmed 3, completed 2, cancelled 1, pending 2, declined 1. No trip reservations seeded (wallet ledger references non-existent trip bookings).

Trip (`types.ts:261-290` , itinerary `:252-259`), store `__darna_trips__` , seed `lib/mock-data/trips.ts` (4 rows, all `org_atlas_escapes` , all active)

Fields: id, title, slug (unique, unicode-aware `lib/ids.ts:20-40`), description, coverPhoto, gallery[], itinerary[] {day, title, description, activities[], meals, accommodation}, durationDays, maxGroupSize, minGroupSize (not enforced), pricePerPerson, inclusions[], exclusions[], meetingPoint, city (free text, e.g. "Tipaza & Cherchell"), commune (never set by UI), locationValue, latlng (derived from city text), cancellationPolicy, organizationId (agency), status, rating, reviewsCount (stored static), departureDates (date keys[]), createdAt, organization (PublicOrganization, populated).

Organization (`types.ts:297-326`), store `__darna_orgs__` , seed `lib/mock-data/organizations.ts` (5 rows, all `verified`)

Fields: id, name, businessCategory, logo, description, businessRegistrationNumber (RC), businessType (free text), businessEmail, businessPhone, address, country, verificationStatus, createdAt, ownerUserId (denormalized from owner membership), teamSizeEstimate, rating, reviewCount (static), hotelStarRating, totalRoomCount, checkInTime, checkOutTime, hotelAmenities[], hostelFacilities[], curfewPolicy, agePolicy, bookingApprovalMode.

Seed: `org_atlas_escapes` (agency, owner karim), `org_el_djazair_hotel` (hotel, karim), `org_royal_oran` (hotel, karim), `org_casbah_hostel` (hostel, amina), `org_taghit_hostel` (hostel, amina). PublicOrganization (`types.ts:329-350`) drops RC, email, phone, address, ownerUserId; adds `isVerified` .

Membership (`types.ts:364-376`), store `__darna_memberships__` , seed (6 rows)

Fields: id, organizationId, userId (null while invited), name, email, role, status, invitedBy (userId or `"system"`), createdAt, joinedAt, avatar. Seed: 5 active owners + `mem_sarah_manager` (invited, Atlas). PublicMember (`types.ts:353-359`) drops email.

Conversation (`types.ts:378-393`) + Message (`types.ts:395-403`), stores `__darna_conversations__` (3 rows), `__darna_messages__` (6 rows), seed `lib/mock-data/conversations.ts`

Conversation: id, participants (string[2]), listingId (listing or trip id, used as "subject"), lastMessage, lastMessageAt (denormalized), unreadCount (Record<userId, number>), createdAt, populated `listing` , `otherUser` . Message: id, conversationId, senderId, receiverId, text (max 4000), createdAt, read.

Notification (`types.ts:495-505`), store `__darna_notifications__` , seed `lib/mock-data/notifications.ts` (4 rows, all `usr_yacine`)

Fields: id, userId, type, title, message (English text stored, not i18n keys), link (relative path), read, createdAt, timestamp (unused duplicate).

Review (`types.ts:71-88`), NOT in `_tables` : module const `mockReviews` in `lib/data/reviews.ts:6` (5 rows, read-only)

Fields: id, listingId, userId, rating, cleanliness, accuracy, communication, location, value (numbers), comment, createdAt, embedded `user {id,name,image}` snapshot. No reservationId, no host reply, no trip reviews.

DateAvailabilityOverride (`types.ts:405-412`), store `__darna_calendar_overrides__` , seed inline `lib/data/calendar.ts:13-17` (3 rows on `lst_tipaza_villa`)

Fields: id, listingId, date (date key), isBlocked, customPrice, note. Unique per (listingId, date) by convention (`calendar.ts:45-52`); rows with neither block nor price are pruned.

Wallet + Transaction (`types.ts:580-603`), store `__darna_wallets__` , seed inline `lib/data/wallet.ts:16-232` (5 wallets, 15 transactions)

Wallet: id (`wal_<ownerId>` for new), ownerType, ownerId, ownerName, hostType, currency, balance (stored, recomputed as sum), transactions (embedded array, newest first), createdAt, updatedAt. Transaction: id (`tx_rec_` , `tx_com_` , `tx_rev_` prefixes), walletId, type, amount (signed), currency, relatedBookingId, adminUserId, note (free text), timestamp. Seed wallets: `wal_amina` (32000), `wal_atlas_escapes` (145000), `wal_el_djazair` (310000), `wal_casbah_hostel` (-4200, intentionally negative), `wal_karim` (40000). No wallet for `org_royal_oran` / `org_taghit_hostel` (created lazily on first commission, `wallet.ts:331-367`).

Commission rates (`HostTypeCommissionRates` , `types.ts:605-610`), store `__darna_commission_rates__` (`wallet.ts:239-244`)

`{individual: 12, agency: 15, hotel: 18, hostel: 10}` percent, defaults in `lib/booking/config.ts:15-20` .

Session (not an entity in types.ts): `SessionPayload` (`lib/auth/session-token.ts:19-27`)

`{sid, userId, hostType, organizationId, exp}` HMAC-SHA256 signed cookie `darna_session` , 30 days. Revocation list store `__darna_revoked_sessions__` (`lib/server/session.ts:22-25`). Extra non-identity cookie `darna_mode` = `host` | `traveler` .

Onboarding config (`types.ts:507-572`), `lib/onboarding-config.ts`

Per host type step/field definitions (trilingual labels, required, options, order, isCore). Stored in browser `localStorage` (`darna_onboarding_config_<hostType>` , `onboarding-config.ts:1064-1092`), editable from the admin app on the same browser only.

Static reference data

Data	File	Count
Wilayas (code, name, nameEn, nameAr)	`lib/algeria.ts:14-73`	58
Communes (name, nameAr, wilaya)	`lib/algeria.ts:81-95`	13
City -> wilaya map	`lib/algeria.ts:98-111`	12
Popular destinations (key, name, wilaya, latlng)	`components/search/destinations.ts:14-25`	10
City coordinates for trips	`lib/trip-coordinates.ts:2-24`	23 keys
Amenity filter groups	`components/search/searchFacets.ts:127-148`	20
Amenity labels	`messages/en/amenities.json`	23
Amenity icons	`components/listing/detail/amenityIcons.ts`	~35 keys
Cancellation policy tiers	`lib/booking/cancellation.ts:45-72`	3
Help articles	`messages/*/help.json`	6

Admin-only types present but unused in this app: `AdminKPI` (`types.ts:442`), `AuditLogEntry` (`:457`), `Coupon` (`:474`), `Amenity` and `Category` interfaces (`:56`, `:63`; type names collide only with local identifiers).

2.3 Relations as the UI uses them

From	To	How	Where
Listing	User (host)	`hostUserId`	listing page host card (`listings/[listingId]/page.tsx:76-78`)
Listing	Organization	`organizationId`	host profile link (`:80-85`), ownership
Listing	RoomType	embedded array	hotel/hostel detail, booking
Listing	DateAvailabilityOverride	`listingId`	calendar, availability, pricing
Listing	Review	`listingId`	listing page
Reservation	User (guest)	`userId`	host booking views (with email + phone)
Reservation	Listing or Trip	`listingId` / `tripId`	both directions; trip bookings put trip id in `listingId` too
Reservation	RoomType	`selectedRoomTypeId`	availability per room type
Reservation	Wallet Transaction	`Transaction.relatedBookingId`	commission ledger
Trip	Organization (agency)	`organizationId`	trip page, org profile
Organization	User	`ownerUserId` + Membership rows	owner resolution (messages go to owner)
Membership	User	`userId` (null while invited), `email`	team page
Conversation	User x2	`participants`	inbox
Conversation	Listing / Trip	`listingId`	thread header
User	Listing	`favoriteIds[]`	favorites
Wallet	User or Organization	`ownerType` + `ownerId`	wallet widget

---

3. lib/data and server action function inventory

Conventions: "R" read, "W" write. "Auth" column describes checks performed inside the function itself; most `lib/data` functions perform none and rely on the caller. Proposed endpoints are REST under `/api/v1`.

3.1 `lib/data/_store.ts`, `_tables.ts`, `_sanitize.ts` (infrastructure)

Function	Inputs	Output	R/W	Callers	Auth	Future backend
`globalStore<T>` (`_store.ts:14`)	key: string, init: () => T	T	W (init)	all stores, `lib/server/session.ts:22`	none	Django ORM / PostgreSQL tables
`removeWhere<T>` (`_store.ts:23`)	items: T[], predicate	number	W	calendar, listings, organizations, trips	none	ORM `delete()`
`clone<T>` (`_store.ts:38`)	value	T	R	all data modules	none	serializer
`delay` (`_store.ts:44`)	ms	Promise	none	data modules (fake latency)	none	drop
table consts (`_tables.ts:31-62`)	n/a	arrays	n/a	lib/data only	none	models: User, Listing, Organization, Membership, Reservation, Trip, Conversation, Message, Notification
`ListingInputError` (`_sanitize.ts:9`)	code, params	Error	n/a	`actions/host.ts`, `actions/listings.ts`	n/a	DRF `ValidationError` with error code
`isAllowedImageUrl` (`_sanitize.ts:47`)	unknown	boolean	R	`_sanitize`, `actions/trips.ts`	allowlist of 6 hosts + local paths	media upload + storage URL validation
`OWNER_SETTABLE_LISTING_STATUSES` (`_sanitize.ts:124`)	n/a	`["active","draft"]`	n/a	`_sanitize` only	n/a	serializer choice restriction
`sanitizeListingInput` (`_sanitize.ts:130`)	input: unknown, listingId: string	Partial	R	`createListing`, `updateListing`, `actions/host.ts:128` (pre-validation)	whitelist, price range, max>=min nights	DRF `ListingWriteSerializer`

3.2 `lib/data/auth.ts`

Function	Inputs	Output	R/W	Callers	Auth	Future backend
`DEMO_PASSWORD` (`:14`)	const	string	n/a	(duplicated literal in `components/models/LoginModal.tsx:47`)	n/a	remove; seed fixtures only
`getUserById` (`:53`)	userId: string	User | null	R	`lib/server/session.ts:49`, `listings/[listingId]/page.tsx:76`, `users/[userId]/page.tsx:22`	none (full record incl. email)	`GET /api/v1/users/{id}` (public projection), `GET /api/v1/me` (private)
`getUsersByIds` (`:59`)	userIds: string[]	User[]	R	none	none	internal
`getUserByEmail` (`:63`)	email: string	User | null	R	none	none	internal (invite linking)

Function	Inputs	Output	R/W	Callers	Auth	Future backend
`verifyCredentials (:70)`	email, password	User | null	R	`app/api/auth/login/route.ts:23`	scrypt timing-safe, dummy hash for unknown emails	`POST /api/v1/auth/login`
`registerUser (:85)`	{name, email, password}	RegisterResult	W	`app/api/auth/register/route.ts:21`	email regex, min 8 chars, unique email	`POST /api/v1/auth/register`
`changePassword (:124)`	userId, currentPassword, newPassword	ChangePasswordResult	W	`actions/account.ts:28`	verifies current password	`POST /api/v1/me/password`
`updateUserHostStatus (:143)`	userId, {hostType?, organizationId?, bookingApprovalMode?}	User | null	W	`actions/host.ts:92,164,182`, `actions/organizations.ts:84`	none	internal service (onboarding); `PATCH /api/v1/me/host-preferences`
`updateUserProfile (:162)`	userId, {name?, bio?, phone?, address?, languages?, image?}	User | null	W	`actions/account.ts:54`	field whitelist and length caps	`PATCH /api/v1/me`
`setUserFavorite (:182)`	userId, listingId, favorite: boolean	string[] | null	W	`actions/favorites.ts:15`	none	`PUT/DELETE /api/v1/me/favorites/{listingId}`

3.3 `lib/data/listings.ts`

Function	Inputs	Output	R/W	Callers	Auth	Future backend
`isPublicListing (:14)`	Pick<Listing,"status">	boolean	R	`getListings`, `getFeaturedListings`, `getPublicListingById`, `actions/bookings.ts:97`, `actions/messages.ts:65`	n/a	queryset filter `status='active'`
`getListings (:18)`	SearchParams? (destination, checkIn/startDate, checkOut/endDate, guests, vertical, category, minPrice, maxPrice, propertyType, amenities, instantBook (ignored))	Listing[] (with `matchedAvailability`, `quotedTotal` when dates)	R (also triggers reservation settle via availability)	`app/[locale]/page.tsx:64`, `search/page.tsx:57-58`, `listings/[listingId]/page.tsx:93`, `users/[userId]/page.tsx:90`, `app/sitemap.ts:24`, `experiences.ts`, `vehicles.ts`	active only	`GET /api/v1/listings?vertical=&destination=&check_in=&check_out=&guests=&min_price=&` (+ facets endpoint)
`getListingById (:221)`	id	Listing | null	R	`actions/bookings.ts:94`, `actions/calendar.ts:34`, `actions/favorites.ts:13`, `actions/listings.ts:27,51`, `actions/messages.ts:64`, `getPublicListingById`	none (any status)	internal; `GET /api/v1/host/listings/{id}` for owners
`getPublicListingById (:231)`	id, canSeeUnpublished? (listing)	Listing | null	R	`listings/[listingId]/layout.tsx:19`, `listings/[listingId]/page.tsx:35,69`	active, or owner via callback	`GET /api/v1/listings/{id}`
`getFeaturedListings (:241)`	vertical?	Listing[]	R	`app/[locale]/page.tsx:67`	active + `featured`	`GET /api/v1/listings?featured=true`
`getHostListings (:256)`	{userId?, organizationId?}	Listing[] (all statuses)	R	`host/layout.tsx:26`, `host/listings/page.tsx:14`, `host/bookings/page.tsx:20`, `host/calendar/page.tsx:19`, `host/page.tsx:28`, `reservations/page.tsx:40`, `profile/page.tsx:29`, `components/agency/orgProfilePage.tsx:65`	none (caller scopes)	`GET /api/v1/host/listings` (owner scoped); public org listings `GET`
`createListing (:282)`	owner: ListingOwner {hostUserId, organizationId?, businessCategory?}, input: unknown	Listing	W	`actions/host.ts:79,91,157`	whitelist; server sets id/owner/stats; defaults `status:"active"`	`POST /api/v1/host/listings`
`updateListing (:332)`	id, input: unknown	Listing | null	W	`actions/listings.ts:33`	whitelist only (no ownership)	`PATCH /api/v1/host/listings/{id}`
`deleteListing (:346)`	id	boolean	W (hard delete)	`actions/listings.ts:55`	none	`DELETE /api/v1/host/listings/{id}` (soft delete / archive)

3.4 `lib/data/reservations.ts`

Function	Inputs	Output	R/W	Callers	Auth	Future backend
`getReservations (:94)`	{userId?, listingId?, tripId?, hostId?, organizationId?, status?}	Reservation[] populated (guest email/phone included)	R + W (settles every row: expired / completed, `:105`)	`trips/page.tsx:29` (userId), `host/page.tsx:27`, `host/bookings/page.tsx:19`, `host/calendar/page.tsx:20`, `reservations/page.tsx:39` (hostId/orgId)	none	`GET /api/v1/me/reservations`, `GET /api/v1/host/reservations?status=&listing=`
`getReservationById (:125)`	id	Reservation | null	R + W (settle)	`actions/bookings.ts:228,265`	none	`GET /api/v1/reservations/{id}` (guest or host scoped)

Function	Inputs	Output	R/W	Callers	Auth	Future backend
`getReservationOwner (:132)`	reservationId	{hostUserId, organizationId, guestUserId} | null	R	`actions/bookings.ts:224`, `lib/server/authz.ts:28`	none	internal permission helper
`ReservationConflictError (:150)`	message	Error	n/a	`actions/bookings.ts:203,301`	n/a	HTTP 409 with code
`unavailableNights (:202)`	listing, roomTypeId?, now?	Set	R	`actions/bookings.ts:132`, `getListingAvailability`, `isListingFreeForRange`	none	internal availability service
`getListingAvailability (:213)`	listing	ListingAvailability {today, unavailable[], unavailableByRoomType}	R	`listings/[listingId]/page.tsx:89`	none	`GET /api/v1/listings/{id}/availability?from=&to=`
`isListingFreeForRange (:227)`	listing, start, end	boolean	R	`getListings (listings.ts:191)`	none	SQL availability filter in search
`seatsSoldByDeparture (:244)`	tripId, now?	Record<DateKey, number>	R	`actions/bookings.ts:177`, `trips/[tripId]/page.tsx:83`	none	`GET /api/v1/trips/{id}/departures` (seats left)
`createListingReservation (:310)`	listingId, CreateReservationInput & {selectedRoomTypeId?, unitCount?}	Reservation	W (+ wallet commission when confirmed)	`actions/bookings.ts:147`	capacity check atomic in-process	`POST /api/v1/reservations` (transaction + row locks / exclusion constraint)
`createTripReservation (:341)`	tripId, CreateReservationInput	Reservation	W (+ commission)	`actions/bookings.ts:184`	seat check	`POST /api/v1/reservations` (kind=trip)
`cancelReservation (:402)`	id, by: guest|host, refund: RefundBreakdown	Reservation | null	W (+ commission reversal)	`actions/bookings.ts:241,288`	none (trusts caller refund)	`POST /api/v1/reservations/{id}/cancel`
`respondToRequest (:428)`	id, decision: confirmed|declined, {declineReason?}	Reservation | null	W (+ commission charge or reversal)	`actions/bookings.ts:274`	must be pending after settle	`POST /api/v1/host/reservations/{id}/accept` / `/decline`
internal: `guestForHost (:17)`, `tripFor (:43)`, `settle (:51)`, `populate (:64)`, `ownedInventory (:76)`, `activeListingReservations (:157)`, `reservationRoomTypeId (:162)`, `usageByNight (:171)`, `capacityOf (:192)`, `buildReservation (:269)`, `commissionOwner (:357)`, `commissionBase (:377)`, `chargeCommission (:381)`						domain services

3.5 `lib/data/calendar.ts`

Function	Inputs	Output	R/W	Callers	Auth	Future backend
`getAvailabilityOverrides (:23)`	listingId	DateAvailabilityOverride[]	R	`setDatesBlocked`, `setDatePrice`	none	`GET /api/v1/host/listings/{id}/calendar`
`getOverridesForListings (:27)`	listingIds[]	DateAvailabilityOverride[]	R	`host/bookings/page.tsx:24`, `host/calendar/page.tsx:23`	none	`GET /api/v1/host/calendar?listing_ids=&from=&to=`
`blockedDateKeys (:33)`	listingId	Set	R (sync)	`reservations.ts:204,233,320`	none	internal
`priceOverridesFor (:37)`	listingId	PriceOverrides	R (sync)	`actions/bookings.ts:143`, `listings/[listingId]/page.tsx:90`, `listings.ts:211`	none	internal pricing service
`setDatesBlocked (:58)`	listingId, dates[], blocked	overrides[]	W	`actions/calendar.ts:54`	none	`PUT /api/v1/host/listings/{id}/calendar/blocks`
`setDatePrice (:68)`	listingId, dates[], price | null	overrides[]	W	`actions/calendar.ts:78`	none	`PUT /api/v1/host/listings/{id}/calendar/prices`

3.6 `lib/data/conversations.ts`

Function	Inputs	Output	R/W	Callers	Auth	Future backend
`getConversations (:27)`	userId	Conversation[] (with listing + otherUser public fields)	R	`messages/page.tsx:27`, `messages/[conversationId]/page.tsx:38`, `host/messages/page.tsx:14`	participant filter	`GET /api/v1/conversations`
`getConversationForParticipant (:37)`	id, viewerId	Conversation | null	R	`messages/[conversationId]/page.tsx:32`	participant	`GET /api/v1/conversations/{id}`
`isConversationParticipant (:47)`	conversationId, userId	boolean	R	`actions/messages.ts:21,43`	n/a	permission class
`getMessages (:56)`	conversationId	Message[]	R	`actions/messages.ts:24`, `messages/page.tsx:31`, `messages/[conversationId]/page.tsx:39`, `host/messages/page.tsx:18`	none (caller checks)	`GET /api/v1/conversations/{id}/messages?cursor=`
`sendMessage (:63)`	{conversationId, senderId, text}	Message | null	W	`actions/messages.ts:35`	sender must be participant	`POST /api/v1/conversations/{id}/messages`

Function	Inputs	Output	R/W	Callers	Auth	Future backend
`markConversationAsRead (:95)`	conversationId, userId	void	W	`actions/messages.ts:46`	none	`POST /api/v1/conversations/{id}/read`
`findOrCreateConversation (:112)`	{userId, otherUserId, listingId?}	Conversation	W	`actions/messages.ts:82`	none	`POST /api/v1/conversations`

3.7 `lib/data/notifications.ts`, `favorites.ts`, `experiences.ts`, `vehicles.ts`, `reviews.ts`

Function	Inputs	Output	R/W	Callers	Auth	Future backend
`getNotifications (notifications.ts:8)`	userId	Notification[]	R	`actions/notifications.ts:16`, `notifications/page.tsx:22`	user scoped	`GET /api/v1/me/notifications`
`markNotificationAsRead (notifications.ts:16)`	id, userId	boolean	W	`actions/notifications.ts:23`	owner	`POST /api/v1/me/notifications/{id}/read`
`markAllNotificationsAsRead (notifications.ts:29)`	userId	void	W	`actions/notifications.ts:30`	user scoped	`POST /api/v1/me/notifications/read-all`
`getFavoriteListings (favorites.ts:7)`	userId	Listing[] (any status)	R	`favorites/page.tsx:29`	none	`GET /api/v1/me/favorites`
`getExperiences (experiences.ts:6)`	none	Listing[]	R	`app/[locale]/page.tsx:66`	active	`GET /api/v1/listings?vertical=experience`
`getVehicles (vehicles.ts:6)`	none	Listing[]	R	`app/[locale]/page.tsx:65`	active	`GET /api/v1/listings?vertical=vehicle`
`mockReviews (reviews.ts:6)`	const	Review[]	n/a	module only	n/a	Review table
`getReviewsByListingId (reviews.ts:104)`	listingId	Review[]	R	`listings/[listingId]/page.tsx:86`	none	`GET /api/v1/listings/{id}/reviews`
`getReviewsByUserId (reviews.ts:109)`	userId	Review[] (written by the user + on listings they host individually)	R	`host/page.tsx:29`, `profile/page.tsx:30`, `users/[userId]/page.tsx:90`	none	`GET /api/v1/users/{id}/reviews?role=author|host`, `GET /api/v1/host/reviews`

3.8 `lib/data/organizations.ts`

Function	Inputs	Output	R/W	Callers	Auth	Future backend
`ORGANIZATION_LEGAL_FIELDS (:14)`	const	name, businessRegistrationNumber, businessType, businessCategory, address, country	n/a	`updateOrganization`	n/a	serializer: re-verification trigger fields
`ORGANIZATION_EDITABLE_FIELDS (:24)`	const	legal + logo, description, businessEmail, businessPhone, teamSizeEstimate, hotelStarRating, totalRoomCount, checkInTime, checkOutTime, hotelAmenities, hostelFacilities, curfewPolicy, agePolicy, bookingApprovalMode	n/a	`updateOrganization`	n/a	serializer fields
`getOrganizationById (:44)`	id	Organization | null (full, private)	R	`lib/server/active-org.ts:24`, `actions/messages.ts:68,74`, `listings/[listingId]/page.tsx:80`, `users/[userId]/page.tsx:94`, `orgProfilePage.tsx:26`, `trips.ts:55,65`	none	`GET /api/v1/organizations/{id}` (pu `/api/v1/host/organization` (private)
`getOrganizations (:50)`	none	Organization[]	R	`app/sitemap.ts:24`	none	`GET /api/v1/organizations?verified`
`getOrganizationMembers (:55)`	organizationId	Membership[] (with emails)	R	`actions/organizations.ts:109,178`, `host/agency/team/page.tsx:22`, `orgProfilePage.tsx:64`	none	`GET /api/v1/host/organization/memb` `GET /api/v1/organizations/{id}/mem`
`getMembershipsForUser (:61)`	userId	Membership[] active	R	`lib/server/active-org.ts:19`, `users/[userId]/page.tsx:90`	n/a	`GET /api/v1/me/memberships`
`getMembership (:65)`	organizationId, userId	Membership | null	R	none	n/a	permission helper
`getMemberById (:72)`	memberId	Membership | null	R	`actions/organizations.ts:135,152,169`	none	internal
`createOrganization (:77)`	{name, businessCategory, ..., owner {userId,name,email}, invitedMembers[]}	Organization	W (org + owner membership + invites)	`actions/host.ts:131`	forces `verificationStatus:"pending"`	`POST /api/v1/organizations`
`updateOrganizationVerification (:173)`	id, status	Organization | null	W	none (admin)	none	admin: `POST /api/v1/admin/organizations/{id}/ve`
`addOrganizationMember (:184)`	{organizationId, name, email, role: admin|manager, invitedBy}	Membership	W	`actions/organizations.ts:114`	none	`POST /api/v1/host/organization/inv`

Function	Inputs	Output	R/W	Callers	Auth	Future backend
`updateOrganizationMemberRole (:208)`	memberId, role	Membership | null	W	`actions/organizations.ts:141`	none	`PATCH /api/v1/host/organization/me`
`transferOrganizationOwnership (:220)`	organizationId, newOwnerMemberId	boolean	W (demotes all owners to admin)	`actions/organizations.ts:175`	target active with userId	`POST /api/v1/host/organization/tra ownership`
`removeOrganizationMember (:238)`	memberId	boolean	W (hard delete)	`actions/organizations.ts:158`	none	`DELETE /api/v1/host/organization/members/`
`updateOrganization (:247)`	id, Partial	Organization | null	W (legal change -> `pending`)	`actions/organizations.ts:65`	field whitelist	`PATCH /api/v1/host/organization`
`deleteOrganization (:274)`	id	boolean	W (hard delete org + memberships, listings/trips -> draft)	`actions/organizations.ts:81`	none	`DELETE /api/v1/host/organization (`

3.9 `lib/data/trips.ts`

Function	Inputs	Output	R/W	Callers	Auth	Future backend
`getTrips (:20)`	TripSearchParams {organizationId?, destination?, city?, status? active|draft|all, maxPrice?, duration? (ignored)}	Trip[] + public org	R	`app/[locale]/page.tsx:68`, `search/page.tsx:59`, `host/agency/trips/page.tsx:22` (status all), `orgProfilePage.tsx:66`, `app/sitemap.ts:24`	default active only	`GET /api/v1/trips? destination=&max_price=`; `GET /api/v1/host/trips`
`getTripById (:61)`	idOrSlug	Trip | null (any status)	R	`actions/bookings.ts:166`, `actions/messages.ts:71`, `actions/trips.ts:98,111`, `getPublicTripById`	none	internal
`getPublicTripById (:70)`	idOrSlug, canSeeUnpublished?	Trip | null	R	`trips/[tripId]/layout.tsx:19`, `trips/[tripId]/page.tsx:27`	active or owner	`GET /api/v1/trips/{id_or_slug}`
`createTrip (:82)`	TripInput	Trip	W	`actions/trips.ts:90`	none (slug + coords derived)	`POST /api/v1/host/trips`
`updateTrip (:95)`	id, Partial	Trip | null	W (`Object.assign` raw)	`actions/trips.ts:102`	none	`PATCH /api/v1/host/trips/{id}`
`deleteTrip (:102)`	id	boolean	W (hard delete)	`actions/trips.ts:114`	none	`DELETE /api/v1/host/trips/{id}`

3.10 `lib/data/wallet.ts`

Function	Inputs	Output	R/W	Callers	Auth	Future backend
`DEFAULT_COMMISSION_RATES` re-export (`:14`)	const	rates	n/a	n/a	n/a	PlatformSettings
`getCommissionRates (:250)`	none	HostTypeCommissionRates	R	none (admin)	none	`GET /api/v1/admin/settings/commission-rates`
`saveCommissionRates (:255)`	Partial	rates	W	none (admin)	none	`PATCH /api/v1/admin/settings/commission-rates`
`getRateForHostType (:262)`	hostType	number (fallback 12.0)	R	`actions/wallet.ts:21`, internal	n/a	internal
`getWallets (:268)`	none	Wallet[]	R	none (admin)	none	`GET /api/v1/admin/wallets`
`getWalletById (:272)`	id	Wallet | null	R	none	none	admin
`getWalletByOwner (:277)`	ownerId, ownerType?	Wallet | null	R	`actions/wallet.ts:18,20`	none (ownerType effectively ignored, `:281-285`)	`GET /api/v1/host/wallet`
`rechargeWallet (:290)`	{walletId, amount, note?, adminUserId?}	{wallet, transaction}	W	none (admin)	none; default adminUserId `"usr_admin"`	`POST /api/v1/admin/wallets/{id}/recharges`
`deductBookingCommission (:405)`	{bookingId, commissionBase, hostType, hostUserId?, organizationId?, ownerName?}	{transaction, rate, amount}	W (idempotent)	`reservations.ts:383`	none	domain service on booking confirmation
`reverseBookingCommission (:446)`	bookingId, keepShare (0..1)	Transaction | null	W (idempotent)	`reservations.ts:419,447`	none	domain service on cancel / decline
`getBookingCommission (:478)`	bookingId	number | null	R	`reservations.ts:68,420`	none	internal
internal: `computeBalance (:246)`, `findOrCreateWallet (:331)`, `appendTransaction (:369)`, `netCommissionFor (:382)`						ledger service

3.11 Server actions (`app/actions/*`)

All return `ActionResult<T>` (`lib/action-result.ts:93-95`) with stable error codes.

File:line	Action	Inputs	Output	R/W	Callers	Auth / ownership checks	Future backend
`account.ts:10`	`changePasswordAction`	{currentPassword, newPassword, confirmPassword}	null	W	`components/settings/SecuritySection.tsx:48`	session; confirm match; new != current	`POST /api/v1/me/pas`
`account.ts:40`	`updateProfileAction`	{name?, bio?, phone?, address?, languages?}	User	W	`UserProfileClient.tsx:91`, `settings/PersonalSection.tsx:145`	session; non-empty name	`PATCH /api/v1/me`
`bookings.ts:78`	`createBookingAction`	CreateBookingInput: {kind:"listing", listingId, startDate, endDate, guestCount, unitCount?, selectedRoomTypeId?, guestMessage?} | {kind:"trip", tripId, departureDate, guestCount, guestMessage?}	Reservation	W	`ListingReservation.tsx`, `HotelHostelDetailClient.tsx`, `TripDetailClient.tsx`	session; listing active; not own individual listing (`:99-101`, org members can book own org listings); trip active; full rule validation; server quote	`POST /api/v1/reserv`
`bookings.ts:216`	`cancelMyReservationAction`	reservationId	Reservation	W	`components/trips/TripsClient.tsx`	session; guest owns reservation; policy refund; must be allowed	`POST /api/v1/reserv`
`bookings.ts:253`	`respondToReservationAction`	reservationId, decision confirm|decline|cancel, declineReason?	Reservation	W	`HostBookingsClient.tsx:218,245,280`, `HostTodayClient.tsx`, `ReservationsClient.tsx`	host context owns inventory (any org role); confirm/decline only if pending and not expired; cancel only if confirmed	`POST /api/v1/host/reserv`
`calendar.ts:40`	`setListingDatesBlockedAction`	{listingId, dates? | from+to (inclusive), blocked}	DateAvailabilityOverride[]	W	`HostCalendarClient.tsx`, `HostBookingsClient.tsx:127`	host owns listing; max 366 dates; no past dates	`PUT /api/v1/host/li`
`calendar.ts:60`	`setListingDatePriceAction`	{listingId, dates? | from+to, price | null}	DateAvailabilityOverride[]	W	`HostCalendarClient.tsx`	host owns listing; price 100..10,000,000 int; not room-type listings (past dates allowed)	`PUT /api/v1/host/li`
`favorites.ts:9`	`setFavoriteAction`	listingId, favorite: boolean	string[]	W	`hook/useFavorite.ts` (HeartButton, cards)	session; listing exists (any status)	`PUT/DELETE /api/v1/`
`host.ts:65`	`completeHostOnboardingAction`	{hostType:"individual", listing} | {hostType: agency|hotel|hostel, organization: OrganizationOnboardingInput, property?}	{hostType, organizationId, listingId}	W (listing and/or org, user host status, session refresh)	`BecomeHostFlow.tsx:566`	session; org fields required; email regex; invites capped 20 and self excluded; price validation before creating anything	`POST /api/v1/host/c` `/api/v1/organizatio`
`host.ts:177`	`setBookingApprovalModeAction`	mode instant|request	null	W	`HostSettingsClient.tsx:158`	session; user has hostType	`PATCH /api/v1/me/ho`
`listings.ts:19`	`updateMyListingAction`	listingId, patch: unknown	Listing	W	`HostListingsClient.tsx:154,188,216`	host owns listing (any org role)	`PATCH /api/v1/host/`
`listings.ts:44`	`deleteMyListingAction`	listingId	{id}	W	`HostListingsClient.tsx:235`	host owns listing (any org role)	`DELETE /api/v1/host`
`messages.ts:18`	`getConversationMessagesAction`	conversationId	Message[]	R	`MessagesClient.tsx:154`	participant	`GET /api/v1/convers`
`messages.ts:27`	`sendMessageAction`	conversationId, text	Message	W	`MessagesClient.tsx:227`	session; participant (in data layer); 1..4000 chars	`POST /api/v1/conver`
`messages.ts:40`	`markConversationReadAction`	conversationId	null	W	`MessagesClient.tsx:162`	participant	`POST /api/v1/conver`
`messages.ts:54`	`startConversationAction`	{listingId?} | {tripId?}	{conversationId}	W	`ListingInfo.tsx:92` (tripId path has no caller)	session; listing/trip public; recipient = listing host or org owner; not self	`POST /api/v1/conver`
`notifications.ts:13`	`getMyNotificationsAction`	none	Notification[]	R	`NotificationsDropdown.tsx:48`	session	`GET /api/v1/me/noti`

File:line	Action	Inputs	Output	R/W	Callers	Auth / ownership checks	Future backend
notifications.ts:19	markNotificationReadAction	notificationId	null	W	NotificationsDropdown.tsx:89, NotificationsClient.tsx	owner	POST /api/v1/me/not
notifications.ts:27	markAllNotificationsReadAction	none	null	W	NotificationsDropdown.tsx:103, NotificationsClient.tsx:78	session	POST /api/v1/me/not
organizations.ts:60	updateOrganizationAction	patch: unknown	Organization	W	AgencySettingsClient.tsx	owner or admin	PATCH /api/v1/host/
organizations.ts:72	deleteOrganizationAction	confirmName	{id}	W	AgencySettingsClient.tsx:114	owner; exact name match	DELETE /api/v1/host
organizations.ts:90	inviteMemberAction	{name?, email, role admin|manager}	Membership	W	AgencyTeamClient.tsx:113	owner or admin; only owner grants admin; no duplicate email	POST /api/v1/host/c
organizations.ts:126	changeMemberRoleAction	memberId, role admin|manager	Membership	W	AgencyTeamClient.tsx:139	owner; same org; owner role locked	PATCH /api/v1/host/
organizations.ts:147	removeMemberAction	memberId	{id}	W	AgencyTeamClient.tsx:164	owner removes non-owner; admin removes manager	DELETE /api/v1/host
organizations.ts:164	transferOwnershipAction	memberId	Membership[]	W	AgencyTeamClient.tsx:188	owner; target active with userId	POST /api/v1/host/c
trips.ts:76	createTripAction	input: unknown (trip fields)	Trip	W	TripCreationClient.tsx:256	org context with hostType agency (any role); min<=max group; price 100..10M	POST /api/v1/host/t
trips.ts:95	setTripStatusAction	tripId, status active|draft	Trip	W	AgencyTripsClient.tsx	agency org owns trip (any role)	PATCH /api/v1/host/
trips.ts:108	deleteTripAction	tripId	{id}	W	AgencyTripsClient.tsx	agency org owns trip (any role)	DELETE /api/v1/host
wallet.ts:12	getMyWalletAction	none	{wallet, commissionRate}	R	HostWalletWidget.tsx	host context; org wallet for org hosts	GET /api/v1/host/wa

3.12 Route handlers (app/api/*)

Route	Method	Body	Response	Checks	Future backend
app/api/auth/login/route.ts:9	POST	{email, password} JSON	{success, user{id,name,hostType}} + cookies darna_session, darna_mode	JSON content type + same Origin (lib/server/request-guard.ts:9); 401 on bad credentials	POST /api/v1/auth/login (session or JWT)
app/api/auth/register/route.ts:10	POST	{name, email, password}	same, 409 EMAIL_TAKEN	same	POST /api/v1/auth/register
app/api/auth/logout/route.ts:7	POST	none	{success}; clears cookies incl. legacy darna_user_id, darna_host_type, darna_agency_id	Origin match only	POST /api/v1/auth/logout

3.13 Server helpers relevant to authorization (lib/server/*)

Function	File:line	Purpose
getSession	session.ts:35	verify cookie + revocation list
getSessionUser	session.ts:46	load User from store by session userId
startSession / refreshSession / endSession	session.ts:67, :84, :109	issue, re-issue (same sid), revoke
resolveActiveOrganization	active-org.ts:15	org via active memberships only; preferred org honoured if member
getHostContext	host-context.ts:24	{user, hostType, isOrgHost, organization, membership}
requireHostContext	host-context.ts:47	redirects: login, /become-a-host, /become-a-host/<type>
hostOwnerKey	host-context.ts:63	{organizationId} for org hosts, {userId} for individuals
hostOwnsListing / hostOwnsTrip	authz.ts:13, :20	ownership predicates
getHostContextForReservation	authz.ts:25	host may act on reservation
getOrgContext(check)	authz.ts:38	org host with role predicate
toPublicUser / toPublicOrganization / toPublicMembers	dto.ts:12, :29, :55	projections

Function	File:line	Purpose
`toCardListing / toCardTrip`	`card-dto.ts:12, :30`	slim DTOs for cards
`isTrustedJsonRequest`	`request-guard.ts:9`	CSRF guard for JSON routes
`orgPermissions`	`lib/host-types.ts:18-27`	role matrix (see 4.13)

---

4. Business rules found in code

4.1 Money and pricing (`lib/booking/pricing.ts` , `lib/booking/config.ts`)

Rule	Value / formula	Ref
Currency	integer DZD; single rounding rule `Math.round ( roundDzd )`	`pricing.ts:56-58`
Price bounds (listing, room type, trip, date override)	100 to 10,000,000 DZD, integer	`config.ts:58-68` ; `_sanitize.ts:20-25` ; `actions/calendar.ts:71` ; `actions/trips.ts:84`
Pricing model selection	room type -> `per_bed_night` if bookingUnit bed else `per_room_night` ; experience -> `per_person` ; vehicle -> `per_day` ; trip -> `per_traveller` ; else `per_night`	`pricing.ts:95-105`
Home (per night)	subtotal = sum over each night of (override[night] ?? listing.price)	`pricing.ts:233-253`
Vehicle (per day)	same loop as home, nights = days between pickup and return	`pricing.ts:233-253`
Experience (per person)	subtotal = (override[startDate] ?? listing.price) x guests; nights = 1	`pricing.ts:193-208`
Hotel room	subtotal = roomTypePrice x rooms x nights; no per-date overrides	`pricing.ts:212-231`
Hostel bed	subtotal = bedPrice x beds x nights; beds default to guests	`pricing.ts:214-217` ; server forces beds = guests (`actions/bookings.ts:115-118`)
Trip	subtotal = pricePerPerson x travellers; cleaning 0; tax 0	`pricing.ts:256-274`
Room type price	`basePrice ?? price`	`pricing.ts:72-76`
Room type default selection	first room type when none or unknown id	`pricing.ts:85-93`
Cleaning fee	once per booking, `listing.cleaningFee ?? DEFAULT_CLEANING_FEES[vertical]` (all 0), 0 valid	`pricing.ts:183-186` ; `config.ts:40-46`
Guest service fee	0 for every vertical; per-listing `serviceFeeRate` override ignored on purpose	`config.ts:28-34` ; `pricing.ts:125-130`
Taxes	`round((subtotal + cleaning) x taxRate)` , `taxRate = clamp(listing.taxRate ?? PLATFORM_TAX_RATE)` , platform rate 0	`pricing.ts:148,188-191` ; `config.ts:52`
Total	subtotal + cleaning + serviceFee + taxes	`pricing.ts:159`
Average unit price stored as `nightlyPrice`	round(subtotal / (nights x quantity))	`pricing.ts:158` ; `reservations.ts:284`
Search card quote	quoted on overlap range, cheapest room type, 1 unit, max(1, guests)	`lib/data/listings.ts:194-214`
Approval mode for listing bookings	request if `listing.bookingApprovalMode === "request"` OR `listing.instantBook === false`	`pricing.ts:276-282`
Approval mode for trips	`organization.bookingApprovalMode === "request"`	`actions/bookings.ts:190-193`
Client totals are display only; server recomputes		`pricing.ts:1-5` ; `actions/bookings.ts:136-145`
Zero total rejected	`quote.total <= 0` -> INVALID_DATES	`actions/bookings.ts:145`

4.2 Commission by host type

Rule	Value	Ref
Default rates (% of accommodation subtotal)	individual 12, agency 15, hotel 18, hostel 10	`lib/booking/config.ts:15-20` ; also documented in `types.ts:605-610`
Live rates	server store, admin editable, fallback to defaults then 12.0	`lib/data/wallet.ts:239-244` , `:262-266`
Host type used	listing org's `businessCategory` ; org without category -> agency; no org -> individual	`lib/data/reservations.ts:357-374`
Rate precedence	existing wallet's stored `hostType` wins over the booking's host type	`wallet.ts:423`
Commission base	`res.subtotal ?? max(0, total - cleaning - serviceFee - taxes)`	`reservations.ts:377-379`
Amount	`round(base x rate / 100)`	`wallet.ts:424`
When charged	on creation if instant (confirmed), on host accept for requests	`reservations.ts:336,353,443`
Idempotency	no second charge if net commission for booking > 0	`wallet.ts:413-420`
Reversal	credit `net - round(original x keepShare)` ; guest cancel keepShare = 1 - refundPercent/100; host cancel and decline keepShare = 0	`reservations.ts:417-420,447` ; `wallet.ts:446-475`
Where charged	host prepaid wallet (user wallet for individuals, org wallet for orgs), created lazily `wal_<ownerId>` with balance 0	`wallet.ts:331-367`

4.3 Host payout (display formula, `lib/booking/payout.ts:17-34`)

- subtotal = `res.subtotal ?? max(0, total - cleaning - serviceFee - taxes)`.
- declined / expired -> net 0, commission 0.
- refunded = `refund >= total ? subtotal + cleaning : min(subtotal + cleaning, refund)`.
- net = `max(0, subtotal + cleaning - refunded - commissionAmount)`.
- Taxes never go to the host in the formula (they are excluded from subtotal and not added back). Service fee never goes to the host.
- Used by `HostBookingsClient.tsx:589,842`. KPIs instead use gross `totalPrice` (`lib/host-kpis.ts:36-40`).

4.4 Dates and availability

Rule	Detail	Ref
Date keys	`yyyy-MM-dd` strings, never timestamps; compared as strings	`lib/booking/dates.ts:1-9,24-30`
Business timezone	`Africa/Algiers` for "today" and legacy timestamps	`dates.ts:22,56-69`
Half-open stays	nights = [checkIn, checkOut); checkout day free for next guest (back-to-back allowed)	`dates.ts:80-96`
Night enumeration cap	3660 nights	`dates.ts:84`
Inventory-holding statuses	`confirmed`, and `pending` not past `expiresAt`	`lib/booking/lifecycle.ts:40-43`
Whole listing capacity	1 per night (homes, vehicles)	`reservations.ts:192-196`
Room type capacity	`inventoryCount ?? totalInventory` units per night; usage = `unitCount ?? (bed ? guests : 1)`	`reservations.ts:179-183`; `pricing.ts:78-82`
Experience capacity	`groupSize ?? guestCount ?? 10` guests per date	`reservations.ts:176-178,193`; `availability.ts:37`
Trip capacity	`maxGroupSize` seats per departure (sum of guestsCount)	`reservations.ts:244-251,347-348`
Host blocks	blocked override dates are unavailable for every unit of the listing (incl. all room types)	`reservations.ts:202-210`, `:233-234`
Legacy rows without room type	treated as first room type	`reservations.ts:162-165`
Atomic capacity check	check + insert in one synchronous block (single process only)	`reservations.ts:305-334`
Availability window	`availableFrom` = first check-in; `availableTo` = last checkout (homes, vehicles) or last date (experiences)	`availability.ts:88-106`; `listings.ts:164`
Partial availability in search	listing kept if window overlaps the searched range; `matchedAvailability.isPartial` with overlapStart/End; free check and quote use the overlap	`lib/data/listings.ts:147-215`
Property search with room types	listed if property not blocked and at least one room type is free every night	`reservations.ts:231-239`
Calendar shown to guests	from today forward only	`reservations.ts:216`

4.5 Stay rules (`lib/booking/config.ts:71-117`, `lib/booking/availability.ts:57-123`)

Rule	Default	Validation
minNights	1	`nights < min` -> MIN_NIGHTS / MIN_DAYS (vehicles); clamp >= 1
maxNights	90	`nights > max` -> MAX_NIGHTS / MAX_DAYS; max >= min; owner input 1..365 and MAX_NIGHTS_BELOW_MIN (`_sanitize.ts:149-156`)
advanceNoticeDays	0	start < today + N -> ADVANCE_NOTICE; owner input 0..365
checkInTime	"15:00"	used for cancellation cut-off
checkOutTime	"11:00"	used for auto-complete
Past start	n/a	start < today -> PAST_DATE (same-day booking allowed)
Min one night	n/a	nights < 1 -> MIN_ONE_NIGHT / MIN_ONE_DAY
Units	n/a	rooms/beds > inventory -> MAX_ROOMS / MAX_BEDS
Guests	n/a	integer >= 1; max = beds count (hostel), capacity x rooms (hotel), `seats ?? guestCount ?? 5` (vehicle), `groupSize ?? guestCount ?? 10` (experience), `guestCount ?? 16` (home) (`availability.ts:27-39`); server caps raw guests at 500 (`actions/bookings.ts:61-64`)
Trips	n/a	departure must be in `departureDates` and strictly after today; seats left; `minGroupSize` NOT enforced (`availability.ts:126-144`)
Trip end date	departure + max(1, durationDays) - 1	`availability.ts:147-149`

4.6 Approval modes and request expiry

- Request TTL: `REQUEST_TO_BOOK_TTL_HOURS = 24` (`config.ts:55`); `expiresAt = createdAt + 24h` (`reservations.ts:296-299`).
- Pending requests hold inventory until expiry (`lifecycle.ts:40-43`), payment status `pending`.
- Expiry is lazy: applied whenever reservations are read (`reservations.ts:51-62`, `:105`, `:128`); sets `status=expired`, `paymentStatus=voided`, `respondedAt=expiresAt`. No background job, no notification.
- Approval mode exists at three levels: `Listing.bookingApprovalMode` + `instantBook` (used for listings), `Organization.bookingApprovalMode` (used for trips only), `User.bookingApprovalMode` (written, never read by the engine).

4.7 Reservation status transitions

From	To	Trigger	Who	Side effects	Ref
(new)	confirmed	create on instant listing / trip	guest	paymentStatus paid; commission charged	`reservations.ts:273,291,336`

From	To	Trigger	Who	Side effects	Ref
(new)	pending	create on request listing / trip	guest	paymentStatus pending; expiresAt +24h	`reservations.ts:273,291,296`
pending	confirmed	accept	host or any org member	paymentStatus paid; respondedAt; commission charged	`reservations.ts:439-443` ; `actions/bookings.ts:270-283`
pending	declined	decline (optional reason <= 500)	host or any org member	paymentStatus voided; respondedAt; commission reversal (none charged)	`reservations.ts:444-447`
pending	expired	clock passes expiresAt (lazy)	system	paymentStatus voided	`lifecycle.ts:12-16` ; `reservations.ts:55-58`
pending	cancelled	guest cancels	guest	refund 0, paymentStatus voided, cancelledBy guest	`cancellation.ts:159-169` ; `actions/bookings.ts:216-245`
confirmed	cancelled	guest cancels before check-in time on arrival day	guest	refund per policy; commission partly reversed	`cancellation.ts:181-213` ; `reservations.ts:402-425`
confirmed	cancelled	host cancels	host or any org member	full refund `totalPrice` , paymentStatus refunded, full commission reversal	`cancellation.ts:217-239` ; `actions/bookings.ts:284-292`
confirmed	completed	clock passes checkout time on endDate (trips: 23:59 on last day) (lazy)	system	none	`lifecycle.ts:19-37`
any	cancelled by system	never implemented	n/a	`cancelledBy: "system"` unused	`types.ts:236`

Not allowed: host cancel of pending (RESERVATION_NOT_CANCELLABLE), accept/decline of non-pending (REQUEST_NOT_PENDING / REQUEST_EXPIRED), guest cancel of completed/declined/expired/cancelled (RESERVATION_NOT_CANCELLABLE), guest cancel after check-in time (CHECK_IN_PASSED; for trips the cut-off is `00:00` on departure day, `actions/bookings.ts:237`).

4.8 Cancellation policies and refunds (`lib/booking/cancellation.ts`)

Policy	Tiers (days before check-in -> refund % of subtotal)	Ref
flexible	>= 1 day: 100%; arrival day before check-in time: 50%	`:49-55`
moderate	>= 5 days: 100%; 1 to 4 days: 50%; arrival day: 0%	`:56-63`
strict	>= 14 days: 100%; 7 to 13 days: 50%; 0 to 6 days: 0%	`:64-71`

- Days before = calendar days between today in Algeria and startDate (`:154-157`).
- Guest refund = `min(total, round(subtotal x pct/100) + cleaningFee + round(taxes x pct/100) + (pct == 100 ? serviceFee : 0))` (`:199-205`). Cleaning fee always refunded in full. (The header comment at `:14-15` says the cleaning share of taxes is always refunded; code prorates all taxes: minor mismatch, irrelevant while tax rate is 0.)
- Payment status after refund: refund >= total -> refunded; > 0 -> partially_refunded; else paid (`:139-143`).
- Policy snapshot: `reservation.cancellationPolicy ?? listing ?? trip` (`actions/bookings.ts:233-236`); default `flexible` (`:74-81`).
- Host cancel / decline: always full refund (`:217-239`).
- Refund is only a number on the reservation: there is no refund transaction, no payment provider call.

4.9 Wallet / ledger behaviour (`lib/data/wallet.ts`)

- Prepaid commission deposit model: hosts top up offline (CCP, BaridiMob, bank wire); admin credits via `rechargeWallet` with a note and `adminUserId` (`:290-329`); messages `hostWallet.json` (`howItWorksBody` , `creditedManually`).
- Commission deductions are negative transactions, reversals are positive `adjustment` transactions; balance = sum of amounts, recomputed on each append (`:246-248` , `:369-379`).
- Balance may go negative (seed `wal_casbah_hostel` -4200, `:173`). UI warns that listings could be paused; no code pauses anything.
- Append-only in practice (no edit or delete functions).
- `payout` transaction type defined but never produced.

4.10 Ratings and reviews

- Reviews are read-only seed data (`lib/data/reviews.ts:6-102`), 5 rows.
- Summary: overall = mean of `rating` ; categories = mean of positive values of cleanliness, accuracy, communication, location, value; null when no reviews (show "New") (`lib/reviews.ts:15-26`).
- Listing `rating` / `reviewCount` , org `rating` / `reviewCount` , trip `rating` / `reviewsCount` are stored numbers unrelated to the review rows (e.g. `lst_tipaza_villa` reviewCount 48 vs 2 rows).
- Hotel UI relabels categories (communication = staff, value = value for money) (`HotelHostelDetailClient.tsx:72`).
- No rule on who may review (no reservation link), no review window, no host response.
- Search `minRating` filter compares stored `rating` (`searchFacets.ts:374`).

4.11 Favorites

- Listing ids on `User.favoriteIds` ; set semantics (`lib/data/auth.ts:182-194`).
- Adding requires the listing to exist (any status), removing does not (`actions/favorites.ts:13`).
- Optimistic UI with shared in-memory cache keyed by user (`hook/useFavorite.ts:16-27`).
- Trips cannot be favorited (toast only).

4.12 Messaging

- One conversation per (2 participants, subject) where subject = listing id or trip id or null (`conversations.ts:112-137`).
- Recipient for org listings and trips = organization `ownerUserId` (not the team, not the listing creator) (`actions/messages.ts:67-74`).
- Cannot message yourself (`OWN_LISTING_MESSAGE`); listing/trip must be public.
- Text trimmed, 1 to 4000 chars (`actions/messages.ts:30`).
- `unreadCount[receiver] += 1` on send (only if the map exists), reset to 0 on read; messages `read=true` for receiver (`conversations.ts:88-106`).
- `lastMessage` / `lastMessageAt` denormalized on the conversation.
- No link between a reservation and a conversation; booking `guestMessage` is stored on the reservation only, not posted to the inbox.

4.13 Notifications

- Per user, mark one (ownership checked) or all read (`lib/data/notifications.ts`).
- No producer: booking created/confirmed/declined/expired/cancelled, new message, org verification do not create notifications. Types `price` and `review` imply price-drop alerts and review prompts that do not exist.
- Stored English `title` / `message` text (not i18n keys).

4.14 Onboarding / become a host (4 types)

Host type	Creates	Rules	Ref
individual	1 listing (active), user role host + hostType individual + bookingApprovalMode from listing; session refreshed	if the user is already an org host with an org, the listing is created under the org instead	`actions/host.ts:74-100`
agency	Organization (pending verification) + owner Membership + invited Memberships	name, RC, phone, address required; valid business email; invites: valid emails, not self, max 20, role admin or manager (default manager)	`actions/host.ts:103-168` ; `lib/data/organizations.ts:77-171`
hotel	Organization + owner membership + invites + optional property listing with room types	defaults: businessType "Commercial Luxury Hotel", check-in 14:00, check-out 12:00	`organizations.ts:112-131` ; `actions/host.ts:156-162`
hostel	same as hotel	defaults: businessType "Youth & Backpacker Hostel", check-in 13:00, check-out 11:00	same

- Org defaults: logo `/assets/logo-vertical.png` , country Algeria, teamSizeEstimate `2-10` , bookingApprovalMode `instant` , rating 0 (`organizations.ts:105-139`).
- Property is validated (prices) before anything is created (`actions/host.ts:128`), but org creation and listing creation are not transactional.
- Middleware / layout gates: no session -> login; no hostType -> `/become-a-host` ; org host without org -> `/become-a-host/<type>` ; individual host without listing -> `/become-a-host/1` (`middleware.ts:65-85` ; `host/layout.tsx:25-30`).
- Verification: new orgs `pending` ; workspace usable; public listing immediately active and bookable regardless of verification. Legal-field edits reset to `pending` (`organizations.ts:247-272`).

4.15 Organizations, memberships, roles (`lib/host-types.ts:12-27`)

Permission	owner	admin	manager
canManageOperations (listings, bookings, calendar, trips)	yes	yes	yes
canEditSettings	yes	yes	no
canInviteMembers	yes	yes (manager role only)	no
canRemoveMember	any non-owner	managers only	no
canChangeRoles	yes	no	no
canTransferOwnership	yes	no	no
canDeleteOrganization	yes	no	no

- Only the owner can grant admin on invite (`actions/organizations.ts:105-108`).
- Owner role cannot be changed through role change (`OWNER_ROLE_LOCKED`), owner cannot be removed.
- Transfer: target must be active with userId; all current owners demoted to admin; `org.ownerUserId` updated (`organizations.ts:220-236`).
- Duplicate invite check by email among non-removed members (`actions/organizations.ts:109-112`).
- Delete org: owner only, typed name must equal org name; memberships deleted; listings and trips set to draft; deleting user's `organizationId` cleared and session refreshed (`actions/organizations.ts:72-88` ; `organizations.ts:274-282`). Other members keep `hostType` of an org type with no org and will be sent back to onboarding.
- Ops actions (`updateMyListingAction` , `deleteMyListingAction` , calendar, `respondToReservationAction` , trip status/delete) check ownership only, not role, consistent with `canManageOperations` .
- Trips are agency-only (`AGENCY_ONLY` , `actions/trips.ts:73-78`).

4.16 Search filters semantics

Base search (`lib/data/listings.ts:18-218`): active only; `vertical=hotel` matches businessCategory hotel/hostel, category hotel, or type containing "hotel"/"hostel"; `vertical=home` excludes hotels/hostels; `category` substring on type/amenities; `destination` substring on city, locationValue, country; `minPrice` / `maxPrice` on base `price` (not quoted total); guests filter only when > 1 and only on `guestCount` (listings without it pass); dates as in 4.4. `instantBook` in `SearchParams` is ignored by the data layer. Trips: destination substring on title/city/locationValue (`trips.ts:34-42`).

Side filters (`components/search/searchFacets.ts`), applied in-process on the base pool, same code client and server:

Group	URL param	Semantics	Ref
wilayas	`wilayas` (codes 1..58, CSV)	any of; wilaya from commune, else first known city part	`:278` , `:355-359` ; `lib/algeria.ts` `locateP1ace`
communes	`communes`	any of; accent-insensitive equality	`:279` , `:357`
types	`propertyType`	any of; special values hotel, hostel, entire/entire home (no hotel/hostel, no bed units), private room (type contains room or has room units), riad (riad/ksar/heritage); else exact type or businessCategory	`:339-347`
subtypes	`subtype`	any of; equals host-typed `type`	`:366`
price	`minPrice` , `maxPrice`	inclusive on base price (per-night/day/person/traveller)	`:367-370`
amenities	`amenities`	ALL of; each key maps to alias substrings over listing + room-type amenities	`:127-165` , `:371`
bedrooms	`roomCount`	min, applied only when > 1; `bedroomCount ?? roomCount`	`:77` , `:372`
bathrooms	`bathroomCount`	min, only when > 1	`:373`
rating	`minRating`	min stored rating	`:374` , `:409`
instant	`instantBook=true`	`instantBook` OR `bookingApprovalMode === "instant"` (differs from engine rule 4.1)	`:80` , `:375`
favorite	`isGuestFavorite=true`	manual flag	`:376`

Group	URL param	Semantics	Ref
freeCancel	`freeCancellation=true`	policy === flexible	`:377`
hosts	`host`	ownerType individual / organization	`:378`
stars	`stars`	min hotelStarRating	`:379`
transmissions	`transmission`	any of	`:380`
fuels	`fuel`	any of	`:381`
seats	`seats`	min seats	`:382`
expLengths	`expLength`	any of buckets upTo4h [0,4], halfDay (4,12], multiDay (12,inf) hours	`:174-178`, `:383-389`
languages	`lang`	any of experienceLanguages	`:390`
durations (trips)	`duration`	buckets 1-3, 4-7, 8-14, 15+ days	`:167-172`, `:401-407`
months (trips)	`months` (1..12)	any departure month	`:408`
group (trips)	`groupSize`	min maxGroupSize	`:410`
agencies (trips)	`agency`	org name equality	`:411`

Facet counts use the same matchers with the group skipped (`matchesListing(f, filters, skip)`, `:361-392`). Category counts from the unfiltered pool across all verticals (`search/page.tsx:67-71`).

4.17 Algeria geography (`lib/algeria.ts`)

- 58 wilayas with official codes incl. 2019 reform (49 to 58), French/Latin name, optional English (`Alger -> Algiers`), Arabic name (`:14-73`).
- 13 communes with wilaya code (`:81-95`), "extend as hosts add places".
- `locatePlace` : explicit commune first, else split `city` on `&`, `,`, `/`, " et ", " and " and match commune or city->wilaya map (`:97-155`).
- Wilaya filter accepts 1..58 (`searchFacets.ts:278`).

4.18 Auth and session

- scrypt N default, 16-byte salt, 64-byte key, format `scrypt$salt$hash` (`lib/data/auth.ts:18-36`); timing-equalized unknown email (`:46-47,76`).
- Password min length 8, no other complexity (`lib/auth/password-rules.ts:2`).
- Cookie `darna_session` : HMAC-SHA256, httpOnly, SameSite lax, secure in production, 30 days, secret `DARNA_SESSION_SECRET` >= 32 chars (dev fallback secret) (`lib/auth/session-token.ts:15-49,159-168`).
- Logout revokes `sid` in an in-memory map (`lib/server/session.ts:22-32,109-116`).
- Open-redirect guard for `callbackUrl` (`lib/auth/callback-url.ts:11-23`).
- CSRF: JSON content type + Origin host match for auth routes; server actions rely on Next's origin check (`lib/server/request-guard.ts`).
- No rate limiting, no lockout, no email verification, no password reset.

---

5. Gaps and inconsistencies

5.1 Missing entities / flows the UI implies

Gap	Evidence	Backend implication
Payments: no payment intent, charge, provider, capture, refund transaction, invoice	Reservation goes straight to `paid` with free-text `paymentMethod` default `"CIB / Edahabia Card"` (`lib/data/reservations.ts:291-293`); receipt is a print dialog (`TripsClient.tsx:397-508`); settings show fake saved cards (`components/settings/PaymentsSection.tsx`)	Need Payment / PaymentIntent (CIB / Edahabia via SATIM or similar), Refund, Invoice / Receipt numbering; `paymentStatus` must be driven by provider callbacks
Money flow contradiction	`lib/booking/payout.ts:1-5` and `NoGuestFees.tsx` say BLADUS deducts commission from the host payout (implies BLADUS collects guest money) while the wallet copy says commission is taken from an offline prepaid deposit "rather than from automated guest payouts" (`messages/en/hostWallet.json howItWorksBody`); host settings show payout methods and a payout history (`HostSettingsClient.tsx:63-95`)	Decide collection model (platform collects then pays out vs host collects and prepays commission). Need Payout, PayoutMethod (CCP / RIP with key), PayoutBatch if platform collects
Payout methods and history	hardcoded `PAYOUT_HISTORY` and `payoutMethods` (`HostSettingsClient.tsx:63-95`)	PayoutMethod table (CCP number + key, RIB), Payout table
Reviews write path	no action, no UI; reviews are a module constant (`lib/data/reviews.ts:6`); seed users `usr_meriem`, `usr_tarek` do not exist; names differ from users table (Sofiane Boudiaf vs Mebarki, Leila Zerrouki vs Hadj)	Review table linked to reservation, eligibility window, host reply, aggregate denormalization job
Notification producers	nothing creates notifications (`lib/data/notifications.ts` has no create)	Event -> notification service (booking lifecycle, messages, verification, wallet low balance), plus email / SMS channels; preferences matrix (`NotificationsSection.tsx`) needs a table
Invitation acceptance	invited memberships have `userId: null`, no token, no email, no accept route (`organizations.ts:155-168,184-206` ; resend toast `AgencyTeamClient.tsx:179-182`)	Invitation table with token + expiry, email delivery, accept endpoint that links `userId` and sets user hostType
Organization switcher / multi-org	users own several orgs (Karim 3, Amina 2) but active org = session or first membership (`lib/server/active-org.ts:22-23`); `ctx.hostType` comes from `User.hostType`, not from the active org's category (`host-context.ts:28`)	Active org must be explicit (header or session) and host type derived from org `businessCategory` ; User.hostType should not be a single value
Individual user also owning orgs	Amina is `hostType: individual` but owns 2 hostels she can never manage (`getHostContext` treats her as individual)	Separate "personal hosting profile" from org memberships
Agency owner with individual listings	Karim (`hostType: agency`) is `hostUserId` of 3 individual listings (`lst_casbah_loft`, `lst_constantine_cliff`, `lst_dacia_duster`) that `hostOwnsListing` will never let him manage (`authz.ts:13-18`)	Same as above
Listing moderation	`ListingStatus` has `pending`, `suspended` but new listings go `active` immediately (`lib/data/listings.ts:309`); org verification does not gate publishing	Listing review workflow, gating on org verification

Gap	Evidence	Backend implication
Identity verification	`isIdentityVerified` static; notification seed mentions national ID verification; settings identity section	KYC / document upload entity
Media upload	images are URLs typed by hosts, allowlist of 6 hosts (`_sanitize.ts:38-45`); stock photos substituted (`BecomeHostFlow.tsx:628-637`); avatar change toast (`UserProfileClient.tsx:186`)	Media storage (S3-compatible), image entity with ordering, upload endpoints
Reports / trust and safety	report listing toast (`ListingReservation.tsx:439`)	Report entity + admin queue
Help / support tickets	help form is `setTimeout` (`HelpClient.tsx:119-125`)	SupportTicket entity, email
Password reset, email verification, email change, social and phone login, account deactivation, data export, sessions list	toasts (`LoginModal.tsx:330,360,374` ; `PersonalSection.tsx:218` ; `SettingsClient.tsx:212` ; `HostingSections.tsx:103` ; `SecuritySection.tsx:140`)	Auth endpoints and token tables; `emailVerified` is never set
User preferences	currency, notifications matrix, preferred name, mailing address, booking permissions, travel for work, tax NIF are local state	UserPreferences / TaxProfile tables
Host booking defaults	advance notice, prep time, check-in/out, cancellation policy defaults are local state (`HostSettingsClient.tsx:102-106`)	HostDefaults table applied to new listings
Trip content edit	only status and delete actions exist (`app/actions/trips.ts`)	`PATCH /host/trips/{id}`
Listing full edit	edit modal sends only title, price, instantBook (`HostListingsClient.tsx:188-192`)	full listing editor endpoints
Trip <-> conversation and booking <-> conversation	`startConversationAction({tripId})` has no caller; "message host/guest" buttons open the generic inbox (`TripsClient.tsx:311` , `HostBookingsClient.tsx:734`)	Conversation `reservation_id` FK, auto thread on booking with `guestMessage` as first message
Realtime messaging	no polling or websocket	Django Channels / SSE or polling endpoint
Coupons	`Coupon` type exists (`types.ts:474-486`), no UI	Coupon + redemption tables if admin feature ships
Wallet enforcement	negative balance has no effect besides a banner	Rule: pause listings or block confirmations under threshold (needs product decision)
Commission on trips	agency rate applies via org; seeded ledger references trip bookings `res_trip_*` and `res_tipaza_02` , `res_hotel_*` , `res_hostel_*` that do not exist in reservations seed (`wallet.ts:44-206`)	Seed consistency; FK from transaction to reservation
Price drop alerts	notification type `price` seeded	Saved-search / price watch feature not built

5.2 Mock shortcuts that will not survive a real backend

Shortcut	Where	Fix
globalThis arrays as DB; data lost on restart; not shared across instances	`lib/data/_store.ts:14-20` , `_tables.ts`	PostgreSQL
Reviews, calendar overrides, wallets, credentials seeded inline in data modules	`reviews.ts:6` , `calendar.ts:13` , `wallet.ts:16` , `auth.ts:39`	move to fixtures
Demo password shared by all seed users and duplicated in client bundle	`lib/data/auth.ts:14` ; `components/models/LoginModal.tsx:46-51`	remove from client; fixtures only in dev
Dev fallback session secret	`lib/auth/session-token.ts:29`	env-only secret
In-memory session revocation list (lost on restart, per instance)	`lib/server/session.ts:22-32`	server-side session table or JWT denylist
No rate limiting anywhere (login, register, messages, bookings)	grep found none	DRF throttling / reverse-proxy limits
Capacity check "atomic" only within one Node process	`lib/data/reservations.ts:305-334`	DB transaction with `SELECT ... FOR UPDATE` per listing/room type, or PostgreSQL exclusion constraint on daterange for whole-listing stays
Write-on-read lifecycle (`settle` mutates every reservation on each `getReservations`)	`reservations.ts:51-62,105`	scheduled job (Celery beat) for expiry / completion + read-time computation
Commission charged in a try/catch that only logs failures	`reservations.ts:381-396,418-423`	same DB transaction as status change, outbox
Org creation + listing creation + user update not transactional	`actions/host.ts:131-166`	one transaction
Derived values stored as truth: `Listing.rating` , `reviewCount` , `bookingsCount` , `isGuestFavorite` , `featured` ; `Organization.rating` , `reviewCount` ; `Trip.rating` , `reviewsCount` ; `User.yearsHosting` , `responseRate` , `responseTime` ; `Wallet.balance` ; `Conversation.lastMessage` , `unreadCount`	`types.ts`	compute or maintain via signals, mark read-only in API
Mock reservation totals inconsistent with the pricing engine and stored `taxes` while tax rate is 0 (e.g. `res_tipaza_01` 5 nights x 18500 + 4000 + 2500 = 99000 but `totalPrice` 86000; no `subtotal` stored)	`lib/mock-data/reservations.ts:5-21`	regenerate fixtures through the pricing service
Mock statuses contradicting rules: declined row with `paymentStatus: "refunded"` (code uses `voided`); request rows on `1st_tipaza_villa` which is instant; `res_constantine_pending` is Karim booking his own listing (blocked by OWN_LISTING)	`lib/mock-data/reservations.ts:75-127`	fix fixtures
Conversation seed: `unreadCount` keyed by non-participants (`usr_amina` in conv_02/03); `usr_sofiane` acts as a host but is a guest with no listings; `otherUser` snapshot names differ from users	`lib/mock-data/conversations.ts:20-49`	fixtures
Free text where references are needed: `Listing.city` ("Hydra, Algiers" from onboarding, `BecomeHostFlow.tsx:659`), `locationValue` , `commune` (string, not FK), `Trip.city` ("Tipaza & Cherchell"), `Listing.type` (free subtype), `Organization.businessType` , `paymentMethod` , `RoomType.amenities` (English sentences), `houseRules` , `languages` , `experienceLanguages` , `Notification.title/message` (English)	`types.ts` , onboarding	Wilaya / Commune tables (FK), ListingType / Subtype lookup, Amenity table (one vocabulary), Language codes (ISO), PaymentMethod enum, notification template keys
Three amenity vocabularies: listing snake_case keys (`sea_view`), filter keys camelCase (`seaView` , `messages/en/amenities.json`), room-type free text; search bridges them with substring aliases	`searchFacets.ts:127-165` ; `amenityIcons.ts` ; mock listings	single Amenity table with i18n labels and scope (listing / room / hotel / hostel facility)
`Listing.category` vs `businessCategory` vs `type` all used to infer "hotel" (`listings.ts:28-36` , `searchFacets.ts:55-61`); hotels stored as `category: "home"`		explicit `vertical` enum + property kind
Trip bookings put the trip id in `Reservation.listingId` (`reservations.ts:349`) and `getReservations({listingId})` matches either (`:110-112`)		separate nullable FKs `listing_id` , `trip_id` , `departure_id` with a check constraint
Trip departures are a string array; seats computed by summing reservations	`types.ts:286` ; `reservations.ts:244-251`	TripDeparture table (date, capacity, status, price override)

Shortcut	Where	Fix
Room types embedded in listing JSON; ids client-suppliable when they match `/^[\w-]{1,80}$/` (`_sanitize.ts:100`)		RoomType table, server ids only
Calendar overrides per listing only, not per room type; price overrides refused for room types	`actions/calendar.ts:74-76`	per room-type inventory / rate calendar
`inventoryCount` vs `totalInventory` vs `availableInventory` vs `isAvailable` overlap	`types.ts:100-107`; `pricing.ts:78-82`	single inventory field + calendar
`basePrice` mirrors `price` "for older readers"	`_sanitize.ts:108-109`	drop
`perPersonPrice` on experiences never used (pricing uses `price`)	`types.ts:193`	drop or use
`serviceFeeRate` and `taxRate` per listing labelled "admin-only override" but `serviceFeeRate` is ignored and neither is admin editable here	`types.ts:163-166`; `pricing.ts:125-130`	platform settings + tax rules (TVA / taxe de séjour)
Approval mode in three places with different readers (User, Listing + instantBook, Organization) and search facet "instant" disagrees with engine (`searchFacets.ts:80` vs `pricing.ts:276-282`)		one source per inventory item; drop `instantBook` or derive it
Agency onboarding never sends `bookingApprovalMode`; hotel/hostel onboarding hardcodes room types and prices; `hotelTaxId`, `placement`, `bedsCount`, admin custom fields collected but not sent	`BecomeHostFlow.tsx:692-703,777-813,175`; `:113-116`	onboarding API must accept real room types, tax id, custom fields
Onboarding step config in browser `localStorage` (admin edits only affect that browser)	`lib/onboarding-config.ts:1064-1092`	OnboardingConfig table served by API
Host drafts in `localStorage`	`lib/client/user-storage.ts:7-41`	optional server-side ListingDraft
`getListingById`, `getTripById` return any status; favorites can include drafts; `getFavoriteListings` returns unpublished listings to the guest	`actions/favorites.ts:13`; `lib/data/favorites.ts:7-11`	filter public status
Org profile pages render unverified and rejected orgs and their listings	`components/agency/orgProfilePage.tsx:56-80`	decide visibility rules
Guest contact (email + phone) is included on every populated reservation, including in guest-side `getReservations({userId})` responses and host lists for all statuses	`reservations.ts:17-31,64-73`	host-only serializer, possibly reveal contact only after confirmation
`ownerType` arg effectively ignored in `getWalletByOwner` (`wallet.ts:281-285`)		query by (owner_type, owner_id)
Wallet `hostType` stored on wallet and preferred over booking host type (`wallet.ts:423`)		derive from owner
Hard deletes: listings (`listings.ts:346-349`), trips, members, organizations; reservations keep dangling `listingId`		soft delete / archive, PROTECT FKs
`MembershipStatus.removed` never used; `transferOrganizationOwnership` demotes every owner	`organizations.ts:230-232`	unique owner constraint
Deleting an org leaves other members with an org `hostType` and no org	`actions/organizations.ts:72-88`	cascade membership + user role logic
`ListingOwnerType` duplicated by `hostUserId` / `organizationId` presence; org listings also store creator `hostUserId` (`actions/host.ts:79-85`) which then makes `getReviewsByUserId` and public user profiles inconsistent		`created_by` separate from owner
KPIs computed client side from full reservation lists (`lib/host-kpis.ts`); revenue uses gross `totalPrice` not net payout	`HostTodayClient.tsx:98-104`; `ReservationsClient.tsx:95`	server KPI endpoint
Exchange rate constant 152.47	`lib/currency.ts:1`	ExchangeRate table or provider
Search is in-memory full scan with substring matching; no pagination; price filter on base price only	`lib/data/listings.ts:18-218`	SQL / PostGIS (latlng), trigram search on city, pagination, sort
`SearchParams.instantBook` ignored by data layer (handled only by facets); `TripSearchParams.duration` ignored	`listings.ts`, `trips.ts:17`	clean API params
Fake latency `delay()` in data layer	`_store.ts:44`	drop
Legacy cookies `darna_user_id`, `darna_host_type`, `darna_agency_id` cleared on logout	`app/api/auth/logout/route.ts:25-27`	drop after migration
`TODO: replace with Supabase` comments everywhere	every `lib/data/*.ts`	update to Django
`ListingVertical` includes `trip` and `hotel` but trips are a separate entity and hotels are stored as `home`	`types.ts:47-52`	clarify enum
`minGroupSize` on trips not enforced; trip departure dates can be in the past at creation; `durationDays` not reconciled with itinerary length	`availability.ts:126-144`; `actions/trips.ts:44-57`	validation in API
Reservation `subtotal` missing on legacy rows, so commission base / refunds fall back to `total - fees - taxes`	`reservations.ts:377-379`; `cancellation.ts:129-137`; `payout.ts:25`	make `subtotal` NOT NULL

---

6. Page routes (34 `page.tsx`)

33 pages under `app/[locale]/` + root `app/page.tsx`. Auth column: "MW" = middleware redirect when no session (`middleware.ts:14-25,62-71`), "page" = page-level check.

#	Route	File	Purpose	Data functions used	Actor	Auth required
1	`/`	`app/page.tsx`	redirect to `/en`	none	any	no
2	`/[locale]`	`app/[locale]/page.tsx`	home feed, category grid, trips grid	`getSessionUser`, `getListings`, `getVehicles`, `getExperiences`, `getFeaturedListings`, `getTrips`, `toCardListing`, `toCardTrip`	visitor, guest	no
3	`/[locale]/[...rest]`	`app/[locale]/[...rest]/page.tsx`	localized 404 catch-all	none	any	no
4	`/[locale]/search`	`app/[locale]/search/page.tsx`	search results, facets, map, trips	`getSessionUser`, `getListings` (x2), `getTrips`, `parseSideFilters`, `toFacet`, `toTripFacet`, `matchesListing`, `matchesTrip`, `verticalOf`	visitor, guest	no
5	`/[locale]/listings/[listingId]`	`app/[locale]/listings/[listingId]/page.tsx` (+ `layout.tsx` existence check)	listing detail + booking	`getHostContext`, `hostOwnsListing`, `getPublicListingById`, `getSessionUser`, `getUserById`, `toPublicUser`, `getOrganizationById`, `getReviewsByListingId`, `getListingAvailability`, `priceOverridesFor`, `getListings`	visitor, guest; owner sees drafts	no (booking needs login)

#	Route	File	Purpose	Data functions used	Actor	Auth required
6	`/[locale]/trips/[tripId]`	`app/[locale]/trips/[tripId]/page.tsx` (+ `layout.tsx`)	agency trip detail + seat booking	`getHostContext`, `hostOwnsTrip`, `getPublicTripById`, `getSessionUser`, `seatsSoldByDeparture`, `todayKey`	visitor, guest; agency sees drafts	no (booking needs login)
7	`/[locale]/users/[userId]`	`app/[locale]/users/[userId]/page.tsx`	public user / host profile (`me` / `current` redirect to `/profile`)	`getUserById`, `getSessionUser`, `toPublicUser`, `getListings`, `getReviewsByUserId`, `getMembershipsForUser`, `getOrganizationById`	visitor, guest	no
8	`/[locale]/agencies/[organizationId]`	`app/[locale]/agencies/[organizationId]/page.tsx -> components/agency/orgProfilePage.tsx`	public agency profile	`getOrganizationById`, `getOrganizationMembers`, `getHostListings`, `getTrips`, `toPublicOrganization`, `toPublicMembers`	visitor	no
9	`/[locale]/hotels/[organizationId]`	`app/[locale]/hotels/[organizationId]/page.tsx`	public hotel profile	same as 8 (no trips)	visitor	no
10	`/[locale]/hostels/[organizationId]`	`app/[locale]/hostels/[organizationId]/page.tsx`	public hostel profile	same as 8 (no trips)	visitor	no
11	`/[locale]/help`	`app/[locale]/help/page.tsx`	help center + contact form (mock)	none	any	no
12	`/[locale]/login`	`app/[locale]/login/page.tsx`	login / register page	none (client posts to `/api/auth/*`)	visitor	no
13	`/[locale]/design-system`	`app/[locale]/design-system/page.tsx`	internal UI gallery, 404 in production	none	developer	no
14	`/[locale]/trips`	`app/[locale]/trips/page.tsx`	guest bookings (upcoming / past / cancelled), cancel, receipt	`getSessionUser`, `getReservations({userId})`	guest	MW + page
15	`/[locale]/favorites`	`app/[locale]/favorites/page.tsx`	wishlist	`getSessionUser`, `getFavoriteListings`	guest	MW + page
16	`/[locale]/messages`	`app/[locale]/messages/page.tsx`	inbox (first thread opened)	`getSessionUser`, `getConversations`, `getMessages`	guest, host	MW + page
17	`/[locale]/messages/[conversationId]`	`app/[locale]/messages/[conversationId]/page.tsx`	specific thread	`getSessionUser`, `getConversationForParticipant`, `getConversations`, `getMessages`	participant	MW + page (404 for non participants)
18	`/[locale]/notifications`	`app/[locale]/notifications/page.tsx`	notifications list	`getSessionUser`, `getNotifications`	guest, host	MW + page
19	`/[locale]/profile`	`app/[locale]/profile/page.tsx`	own profile view / edit	`getSessionUser`, `getHostListings({userId})`, `getReviewsByUserId`	guest, host	MW + page
20	`/[locale]/settings`	`app/[locale]/settings/page.tsx`	account settings (mostly mocked sections)	`getSessionUser`	guest, host	MW + page
21	`/[locale]/reservations`	`app/[locale]/reservations/page.tsx`	host reservations with KPIs (guests redirected to `/trips`)	`getSessionUser`, `getHostContext`, `hostOwnerKey`, `getReservations({hostId|organizationId})`, `getHostListings`	host (any type)	MW + page
22	`/[locale]/properties`	`app/[locale]/properties/page.tsx`	legacy, redirects to `/host/listings`	none	host	MW
23	`/[locale]/become-a-host`	`app/[locale]/become-a-host/page.tsx`	host type selection / wizard start	`getSessionUser`	guest becoming host	MW + page
24	`/[locale]/become-a-host/[step]`	`app/[locale]/become-a-host/[step]/page.tsx`	wizard step (`1..n`, `agency`, `hotel`, `hostel`)	`getSessionUser`	guest becoming host, existing host adding listing	MW + page
25	`/[locale]/host`	`app/[locale]/host/page.tsx`	Today dashboard (org hosts only; individuals redirected to listings)	`requireHostContext`, `hostOwnerKey`, `getReservations`, `getHostListings`, `getReviewsByUserId`	org host	MW + layout (`requireHostContext`)
26	`/[locale]/host/listings`	`app/[locale]/host/listings/page.tsx`	manage listings + wallet compact	`requireHostContext`, `getHostListings` (+ client `getMyWalletAction`)	host	MW + layout
27	`/[locale]/host/bookings`	`app/[locale]/host/bookings/page.tsx`	manage bookings, accept / decline / cancel, mini calendar	`requireHostContext`, `hostOwnerKey`, `getReservations`, `getHostListings`, `getOverridesForListings`	host	MW + layout
28	`/[locale]/host/calendar`	`app/[locale]/host/calendar/page.tsx`	multi-view calendar, block / price	`requireHostContext`, `getHostListings`, `getReservations`, `getOverridesForListings`	host	MW + layout
29	`/[locale]/host/messages`	`app/[locale]/host/messages/page.tsx`	host inbox (full screen)	`requireHostContext`, `getConversations`, `getMessages`	host	MW + layout
30	`/[locale]/host/settings`	`app/[locale]/host/settings/page.tsx`	payouts (mock), identity, booking defaults, communication	`requireHostContext`	host	MW + layout

#	Route	File	Purpose	Data functions used	Actor	Auth required
31	`/[locale]/host/agency/team`	`app/[locale]/host/agency/team/page.tsx`	org team management	`requireHostContext`, `getOrganizationMembers`	org host (any role; actions role-gated)	MW + layout; individuals redirected
32	`/[locale]/host/agency/settings`	`app/[locale]/host/agency/settings/page.tsx`	org settings, wallet, delete org	`requireHostContext` (+ client `getMyWalletAction`)	org host (owner / admin to save)	MW + layout; individuals redirected
33	`/[locale]/host/agency/trips`	`app/[locale]/host/agency/trips/page.tsx`	agency trips list	`requireHostContext`, `getTrips({organizationId, status:"all"})`	agency member	MW + layout; non agency redirected
34	`/[locale]/host/agency/trips/new`	`app/[locale]/host/agency/trips/new/page.tsx`	create trip wizard	`requireHostContext`	agency member	MW + layout; non agency redirected

Non-page server files that also read data: `app/[locale]/layout.tsx` (`getSessionUser` for every page), `app/[locale]/host/layout.tsx` (`requireHostContext`, `getHostListings`), `app/[locale]/listings/[listingId]/layout.tsx`, `app/[locale]/trips/[tripId]/layout.tsx`, `app/sitemap.ts` (`getListings`, `getTrips`, `getOrganizations`), `app/robots.ts`.

BLADUS admin back-office: raw inventory for the backend blueprint

Scope: `admin/` only (Next.js 14, next-intl, mock store). Read-only analysis, 2026-11-04. All paths are relative to `admin/` unless prefixed with `frontend/`. `L` = line number. Status legend: **built** = UI + server action + store mutation + audit; **partial** = some parts real, some hardcoded; **mocked-only** = UI state + toast, nothing persisted.

Key sources: `lib/types/index.ts`, `lib/permissions.ts`, `lib/route-permissions.ts`, `lib/domain/transitions.ts`, `lib/pricing.ts`, `lib/data/*`, `lib/actions/*`, `lib/auth/*`, `lib/mock-data/{store,generator,admins}.ts`, `middleware.ts`, `docs/DATA-MODEL.md`, `README.md`.

Note: `docs/DATA-MODEL.md` section 6 is stale: it still says `GUEST_SERVICE_FEE_PERCENT = 7`; code says `0` (`lib/pricing.ts:16`).

---

1. Feature inventory per role

1.1 Permission matrix (source: `lib/permissions.ts:3-93`)

34 permissions (`lib/types/index.ts:165-199`). `X` = granted.

Permission	super_admin	moderator	support	finance	analyst
hosts.view	X	X	X	X	X
hosts.approve	X	X			
hosts.reject	X	X			
hosts.suspend	X	X			
hosts.delete (also ban, restore)	X				
hosts.commission_override	X			X	
listings.view	X	X	X		X
listings.approve (also request changes, unsuspend, create wizards, change requests)	X	X			
listings.reject	X	X			
listings.suspend (also archive)	X	X			
listings.delete (also restore)	X				
listings.feature	X	X			
users.view	X	X	X		X
users.suspend (also unsuspend, mark active/pending)	X	X	X		
users.ban	X				
users.delete (also restore)	X				
bookings.view	X		X	X	X
bookings.cancel	X		X		
bookings.refund	X		X	X	
bookings.dispute	X		X		
reviews.view	X	X	X		X
reviews.moderate	X	X			
reviews.delete	X				
wallets.view	X			X	X
wallets.recharge	X			X	
commissions.manage	X			X	
payouts.approve	X			X	
onboarding.edit	X				
categories.manage	X	X			
promotions.manage	X	X			
reports.export	X	X		X	X
audit.view	X	X	X	X	X
team.manage	X				
settings.manage	X				
count	34	17	10	10	8

Notes:

- Finance has `hosts.commission_override` but NOT `listings.view` or `users.view` (`lib/permissions.ts:71-82`).
- Support can refund and resolve disputes but cannot see wallets, so it cannot see the ledger effect of its own refunds.

- `resolveDispute` with a refund outcome needs `bookings.dispute` AND `bookings.refund` (`lib/actions/bookings.ts:36-38` , `lib/data/bookings.ts:291-292`): only super_admin and support qualify.
- Role labels: `lib/permissions.ts:107-113` .
- Permissions are resolved from the CURRENT role in the store on every request (`lib/auth/current-admin.ts:44-59`), never from JWT claims. `AdminUser.permissions?` (`lib/types/index.ts:137`) exists but is unused (no per-user overrides).

1.2 Route -> permission map (source: `lib/route-permissions.ts:12-63` , longest prefix wins, `null` = any signed-in admin)

Route prefix	Permission	Route prefix	Permission
`/` (exact)	null	`/categories`	categories.manage
`/help`	null	`/stays/categories` , `/experiences/categories` , `/trips/categories` , `/cars/categories`	categories.manage
`/messages`	null	`/stays/amenities` , `/experiences/amenities`	categories.manage
`/where-did-it-go`	null	`/bookings`	bookings.view
`/users`	users.view	`/trips/booking-requests`	bookings.view
`/hosts`	hosts.view	`/cars/rentals`	bookings.view
`/moderation`	hosts.view	`/cars/owners`	hosts.view
`/approvals`	hosts.view	`/payments`	wallets.view
`/listings` , `/network` , `/search`	listings.view	`/reviews`	reviews.view
`/stays` , `/experiences` , `/trips` , `/cars`	listings.view	`/promotions`	promotions.manage
`/stays/new` , `/experiences/new` , `/trips/new` , `/cars/new`	listings.approve	`/stats` , `/reports`	reports.export
`/stays/change-requests` , `/experiences/change-requests`	listings.approve	`/settings`	settings.manage
		`/settings/onboarding`	onboarding.edit
		`/settings/commissions`	commissions.manage
		`/settings/team`	team.manage
		`/settings/audit-log`	audit.view

This map is client-side only (sidebar, bottom nav, palette). The real gate is `guardPage(perm)` in each server page (`lib/auth/page-guard.tsx:19-28`) plus `runAction(perm)` in each server action (`lib/auth/run-action.ts:24-45`). Page guards mirror the map except: `/cars/insurance` and `/cars/pricing` pages use `listings.view` (map says `/cars` = listings.view, consistent), and `/payments/commissions` is `wallets.view` while `/settings/commissions` is `commissions.manage` (same component).

1.3 Feature table (role access derived from 1.1 + 1.2)

SA = super_admin, MO = moderator, SU = support, FI = finance, AN = analyst. "view" = can open, "act" = can mutate.

#	Feature	Route(s)	Component(s)	Status	SA	MO	SU	FI
1	Sign-in / sign-out / session	`/login`	`(auth)/login/LoginForm.tsx` , `lib/actions/auth.ts` , `components/layout/Header.tsx`	built	act	act	act	act
2	Dashboard KPIs, charts, attention list, recent audit	`/`	`(panel)/page.tsx` , `DashboardClient.tsx`	partial	view	view	view	view
3	Approvals inbox (hosts, listings, documents, reviews, disputes)	`/approvals?tab=`	`approvals/ApprovalsClient.tsx`	partial	act	act (no disputes refund)	view	view
4	Provider validation (host KYC queue)	`/moderation/providers` , `/moderation` (redirect), `/providers` (legacy), `/cars/owners` (redirect)	`moderation/providers/ProvidersValidationClient.tsx`	partial	act	act	view	view
5	Hosts directory + detail drawer	`/hosts` , `/hosts/[id]`	`hosts/HostsClient.tsx` , `hosts/[id]/page.tsx`	built	act	approve/suspend	view	override
6	Host commission override	`/hosts` drawer	`HostsClient.tsx:377`	built	act			act
7	Host wallet recharge (from host drawer)	`/hosts`	`HostsClient.tsx:397-426`	built	act			act
8	Guests/users directory	`/users` , `/users/guests` , `/users?status=suspended,banned`	`users/UsersClient.tsx`	partial	act	suspend	suspend	

#	Feature	Route(s)	Component(s)	Status	SA	MO	SU	FI
9	All listings (cross-vertical)	`/listings`	`listings/ListingsClient.tsx`, `components/features/listings/useListingStatusActions.tsx`	built	act	act (no delete)	view	
10	Vertical lists	`/stays`, `/experiences`, `/trips`, `/cars`	`StaysListClient`, `ExperiencesListClient`, `TripsListClient`, `CarsListClient`	partial	act	act	view	
11	Listing detail + decision	`/stays/[id]`, `/experiences/[id]`, `/trips/[id]`, `/cars/[id]`	`stays/[id]/StayDetailClient.tsx` (shared)	built	act	act	view	
12	Listing review wizard (checklist)	`/x/[id]/review`	`stays/[id]/review/StayReviewClient.tsx` (shared)	partial	act	act	view	
13	Create listing wizards	`/stays/new`, `/experiences/new`, `/trips/new`, `/cars/new`	`StayWizardClient.tsx`, `ExperienceWizardClient.tsx` (reused for trips and cars)	mocked-only	act	act		
14	Listing change requests	`/stays/change-requests`, `/experiences/change-requests`	`components/management/ChangeRequestsClient.tsx`	mocked-only	act	act		
15	Trip booking requests	`/trips/booking-requests`	same `ChangeRequestsClient`	mocked-only	view		view	view
16	Categories (global)	`/categories`	`categories/CategoriesClient.tsx`	built	act	act		
17	Per-vertical categories	`/stays/categories`, `/experiences/categories`, `/trips/categories`, `/cars/categories`	`components/management/CategoriesManagerClient.tsx`	partial	act	act		
18	Car pricing tiers, car insurance	`/cars/pricing`, `/cars/insurance`	`CategoriesManagerClient`	mocked (misuse)	act	act	view	
19	Amenities	`/stays/amenities`, `/experiences/amenities`	redirect to categories	missing				
20	Bookings	`/bookings`, `/cars/rentals` (redirect `?vertical=cars`)	`bookings/BookingsClient.tsx`, `components/features/bookings/BookingMoneyDialogs.tsx`	built	act		act	refund
21	Finance overview + wallets + ledger + payouts + refunds	`/payments`, `/payments/{wallets,transactions,commissions,payouts,refunds}`, `/finance` (legacy)	`payments/PaymentsClient.tsx`	built	act			act
22	Default commission rates	`/payments/commissions`, `/settings/commissions`	`PaymentsClient` initialTab=commissions	built	act			act
23	Reviews moderation	`/reviews`, `/reviews?status=flagged`	`reviews/ReviewsClient.tsx`	built	act	act (no delete)	view	
24	Review "reports" and "disputes" tabs	`/reviews/reports`, `/reviews/disputes`	`ReviewsClient`	mocked-only	view	view	view	
25	Coupons	`/promotions`	`promotions/PromotionsClient.tsx`	built	act	act		
26	Featured placements	`/promotions/featured`	`PromotionsClient`	mocked-only	act	act		
27	Messages, sent notifications, templates, broadcast	`/messages`, `/messages/notifications`, `/messages/templates`	`messages/MessagesClient.tsx`	mocked-only	act	act	act	act
28	Stats / reports	`/stats`, `/stats/{providers,services,wilayas,revenue,reports}`, `/reports`	`stats/StatsClient.tsx`	partial	view	view		view
29	Network explorer, search	`/network`, `/search?q=`	`NetworkClient.tsx`, `SearchClient.tsx`	built	act	act	view	
30	Platform settings (general, languages, notifications, security)	`/settings`, `/settings/{languages,notifications,security}`	`settings/SettingsClient.tsx`	partial	act			
31	Team & roles	`/settings/team`, `/team` (legacy)	`settings/team/TeamClient.tsx`	built	act			
32	Audit log	`/settings/audit-log`, `/audit-log` (legacy)	`settings/audit-log/AuditLogClient.tsx`	built	view+export	view+export	view	view+exp
33	Onboarding configurator	`/settings/onboarding`, `/onboarding` (legacy)	`settings/onboarding/OnboardingClient.tsx`	partial	act			

#	Feature	Route(s)	Component(s)	Status	SA	MO	SU	FI
34	Command palette	global (Ctrl/Cmd+K)	`components/layout/CommandPalette.tsx`	built	act	act	act	act
35	Help, where-did-it-go, product tour, training mode, what-changed banner	`/help`, `/where-did-it-go`	`HelpClient`, `WhereDidItGoClient`, `ProductTour`, `TrainingModeBanner`, `WhatChangedBanner`	UI only	view	view	view	view

1.4 Per-role summary

Role	Can do (real, persisted)	Sees but cannot act	Notably cannot
super_admin	everything in 1.3	n/a	nothing gated; only role with onboarding, team, settings, listings.delete, users.ban/delete, reviews.delete, hosts.delete/ban
moderator	approve/reject/suspend hosts and listings, feature listings, request changes, review hide/restore/dismiss, categories, coupons, suspend guests, CSV exports (hosts, listings, audit, KPI)	users, audit log, dashboard revenue	bookings (no `bookings.view`), wallets, delete anything, ban
support	cancel/refund bookings, resolve disputes (incl. refunds), suspend/unsuspend guests, mark guest active/pending	hosts, listings, reviews, audit	wallets/ledger, any export (no `reports.export`), stats
finance	wallet recharge, payout approve/reject, default commission rates, host commission override, booking refunds (not disputes), ledger/wallet/booking/host exports	hosts, bookings, audit	listings, users, reviews (no `.view`), cancel bookings, resolve disputes
analyst	CSV exports of hosts, listings, bookings, wallets, ledger, audit, KPI	hosts, listings, users, bookings, reviews, wallets, audit, stats	any mutation

---

2. Data inventory

2.1 Types in `lib/types/index.ts`

Type	Lines	Fields (abridged where noted)	Enums / relations	Used by store?
`UserRole`	1	`"guest" | "host" | "admin"`	copied from frontend	unused
`HostType`	2	`individual | agency | hotel | hostel`	key of commission rates, onboarding flows	yes
`ProviderCategory`	4-9	`host | hotel_hostel | travel_agency | experience_company | rental_agency`	derived by `getProviderCategory` (`lib/data/hosts.ts:38-45`)	derived only
`Document`	11-23	id, type, title, fileName, fileSize, fileUrl, mimeType?, status `valid|to_check|missing|rejected`, reviewer?, reviewedAt?, rejectionReason?		unused
`ModerationChecklistItem` / `ModerationChecklist`	25-46	items, internalNotes, decision `approve_publish|request_changes|reject`, decisionReason, decisionAt, decidedBy; entityType `stay|experience|trip|car|provider`; item status `valid|to_check|missing|rejected|conforme|verifiee|complete`		unused
`ChangeRequest`	48-60	id, entityType, entityId, entityTitle, requestedBy, requestedAt, status `pending|approved|rejected`, beforeState, afterState, reason?, adminResponse?		unused (UI has its own local type)
`WizardDraft`	62-68	id, entityType, step, data, updatedAt		unused
`PointOfInterest`, `PickupPoint`, `ProgrammeItem`, `PricingTier`, `Insurance`, `HouseRules`	70-115	listing sub-structures	`Listing.pointsOfInterest`, `.pickupPoints`, `.programme`, `.pricingTiers`, `.houseRules`	never seeded
`Amenity`	117-124	id, name{en,fr,ar}, icon, category, order?, isFeatured?		unused
`AdminRole`	126	`super_admin | moderator | support | finance | analyst`		yes
`AdminUser`	128-142	id, name, email, role, avatar?, isActive, createdAt, lastLoginAt?, permissions?, status? `active|invited|deactivated`, invitedAt?, invitedBy?	`store.team`	yes
`AdminActor`	149-155	id, name, role, ip?	server-derived	runtime
`AuditActor`	158-163	id, name, role `AdminRole|"anonymous"`, ip?		runtime
`Permission`	165-199	34 values, see 1.1		
`HostStatus`	201-208	`applied | under_review | approved | rejected | suspended | banned | deleted`		yes
`VerificationStatus`	210	`verified | pending | missing | expired`		yes

Type	Lines	Fields (abridged where noted)	Enums / relations	Used by store?
`HostDocument`	212-221	id, type `id_card|commercial_register|tax_card|tourism_licence|property_deed`, title, fileUrl, status `pending|approved|rejected`, rejectionReason?, uploadedAt, reviewedAt?	embedded in `Host.documents`	yes
`OrganizationMember`	223-230	userId, name, email, role `owner|admin|manager|guide|agent`, phone?, avatar?		never seeded
`Organization`	232-250	id, name, hostType, ownerUserId, description?, logo?, coverImage?, address?, wilayaCode, phone?, email?, tradeRegisterNumber?, taxNumber?, tourismLicenseNumber?, verificationStatus **boolean**, members[], createdAt	`Host.organization?`, `Host.organizationId?`	never seeded
`Host`	252-298	id, ownerName, businessName?, email, phone, hostType, status, statusBeforeDelete?, verificationStatus, wilayaCode, city, address?, rating, reviewsCount, listingsCount, bookingsCount, walletBalance (derived), commissionRate (%), isCommissionOverridden?, joinedAt, lastActiveAt, bookingApprovalMode `instant|request`, verticalsOffered[], organizationId?, organization?, documents[], adminNotes[], providerCategory?, formattedId?, agencyMetadata{teamSize,hasTrips,licenseNumber}?, hotelMetadata{starRating,roomCount,amenities[]}?, hostelMetadata{bedCount,dormTypes[]}?	no link to a `GuestUser` /user account	yes
`ListingVertical`	300	`home | hotel | experience | vehicle | trip`		yes
`ListingStatus`	301-312	canonical `draft|pending_review|approved|changes_requested|rejected|suspended|archived|deleted` + legacy aliases `pending|published|refused`	`normalizeListingStatus` (`lib/domain/transitions.ts:73-78`)	yes
`BookingUnit`	314	`room | bed | property | person | day`		partially
`RoomType`	316-329	id, propertyListingId, name, description?, price, bookingUnit `room|bed`, totalInventory, availableInventory?, capacity, bedsDescription?, amenities[], imageSrc[]	`Listing.rooms` (hotel/hostel hosts only)	yes
`Listing`	331-386	id, formattedId?, title, description, category (vertical), vertical? (dup), propertyType, hostId, hostName, hostType (denormalised), status, statusBeforeDelete?, price, priceDzd? (dup), originalPrice?, imageSrc[], images? (dup), createdAt, updatedAt, wilayaCode, city, address?, coordinates{lat,lng}, guestCapacity, bedroomCount?, bathroomCount?, surfaceM2?, constructionYear?, rating, reviewsCount, bookingsCount, amenities[] (free strings), bookingApprovalMode, isFeatured, isReported, adminNotes[], moderationChecklist{photosOk,locationAccurate,priceReasonable,policiesPresent,descriptionAppropriate}?, rooms?, houseRules?, pointsOfInterest?, pickupPoints?, programme?, pricingTiers?, specs?	`hostId -> Host.id`	yes
`GuestStatus`	388	`active | pending | suspended | banned | deleted`		yes
`GuestUser`	390-411	id, name, email, phone?, avatar?, status, role `guest|host|both`, emailVerified (bool), idVerified, wilayaCode?, preferredLanguage `en|fr|ar`, bookingsCount, totalSpentDzd, walletBalanceDzd, reviewsWrittenCount, favoritesCount, joinedAt, lastActiveAt, isFlagged, adminNotes[]	counters are random, not derived	yes
`BookingStatus`	413-419	`pending | confirmed | cancelled | completed | disputed | refunded`		yes
`Booking`	421-466	id, reference (`BLD-2026-NNNN`), listingId, listingTitle, listingImage, vertical, hostId, hostName, hostType, guestId, guestName, guestEmail, guestPhone, wilayaCode, status, approvalMode, checkInDate, checkOutDate (YYYY-MM-DD), nightsCount, guestsCount, basePriceDzd, serviceFeeDzd (always 0), totalAmountDzd, commissionRate, commissionAmountDzd, hostPayoutDzd, paymentStatus, refundedAmountDzd?, refundedAt?, paymentMethod `cib_edahabia|ccp_transfer|baridimob|cash_on_arrival`, createdAt, cancelledAt?, disputeReason?, disputeResolution{outcome,refundAmountDzd,note,resolvedAt,resolvedBy}?, adminNotes[]	`listingId`, `hostId`, `guestId` (heavy denormalisation)	yes
`BookingPaymentStatus`	468	`paid | partially_refunded | refunded | pending | failed`		yes
`DisputeOutcome`	470	`refund_full | refund_partial | declined`		yes
`ReviewStatus`	472	`published | flagged | hidden | deleted`		yes
`ReviewItem`	474-497	id, listingId, listingTitle, vertical, guestId, guestName, guestAvatar?, hostId, hostName, hostType, wilayaCode, rating 1-5, comment, status, flagReason?, flaggedBy?, createdAt, hostReply{comment,createdAt}?, adminNotes[]	no `bookingId`	yes
`TransactionType`	499-505	`booking_payment | commission_deduction | host_payout | wallet_recharge | booking_refund | adjustment`		yes
`WalletTransaction`	507-523	id, walletId (`wlt_<hostId>`), hostId, hostName, hostType, type, amount (signed int), amountDzd? (dup), balanceAfter, reference, description, bookingRef?, receiptReference?, adminActor? (name, not id), createdAt		yes
`HostWallet`	525-542	id, hostId, ownerName, businessName?, hostName?, hostType, wilayaCode, balance (derived), balanceDzd?, totalEarned, totalCommissionsPaid, totalCommissionsPaidDzd?, totalPayouts, lastTransactionAt, isNegative, commissionRate?	1:1 with Host	yes
`PayoutRequest`	544-565	id, hostId, hostName, hostType, amountDzd, payoutMethod `ccp|bank_wire|baridimob`, accountDetails{ccpRip?,rib?,phone?}, status, requestedAt, processedAt?, processedBy? (name), rejectionReason?, transactionId?, adminNotes?	`transactionId -> WalletTransaction.id`	yes
`PayoutStatus`	567	`pending | approved | processed | rejected`	`processed` never produced	yes
`AuditEntry`	569-584	id, timestamp, actorId, actorName, actorRole, action (free string), entityType `host|listing|user|booking|review|wallet|payout|setting|onboarding|promotion|auth|export|team`, entityId, entityName? (never set), beforeState?, afterState?, details, ipAddress		yes
`ModerationCase`	586-600	id, type `host|listing|document|review|dispute`, entityId, title, subtitle, submittedAt, waitingDays, priority `high|normal|low`, hostType?, vertical?, wilayaCode?, status, details (any)	computed view	computed
`OnboardingFieldType`	603-613	`text | textarea | number | select | multiselect | radio | file | location | amenities | boolean`		yes
`OnboardingFieldConfig`	615-626	id?, key?, label{en,fr,ar}, type, required?, order?, placeholder?, helpText?, options?[{value,label}], isCore?		yes
`OnboardingStepConfig`	628-639	id, stepNumber?, order?, isCore?, isActive?, isRequired?, title?, name? (dup of title), description, fields[]		yes
`OnboardingConfig`	641-649	version (int, global), updatedAt, publishedBy (name), flows: Record<HostType,{maxPagesAllowed?, steps[]}>		yes
`CategoryItem`	651-661	id, name{en,fr,ar}, vertical, slug?, iconName?, icon? (dup), listingCount, isActive, order	not linked to listings (`Listing.propertyType` is free text)	yes
`PromotionCoupon`	663-680	id, code, discountType `percentage|fixed_dzd|fixed`, discountValue, minSpendDzd, maxUsage?/maxUses? (dup), usedCount/currentUses? (dup), startDate?/endDate?/expiresAt? (dup), status `active|expired|disabled`, scope? `all|ListingVertical`, targetWilayas?, targetHostTypes?		yes

Type	Lines	Fields (abridged where noted)	Enums / relations	Used by store?
`PlatformSettings`	682-701	platformName, supportEmail, supportPhone, defaultLocale, enabledLocales[], primaryCurrency `DZD`, supportedCurrencies[], guestServiceFeePercent, serviceFeePercent? (dup), defaultCommissionRates, instantBookingPlatformWide, maintenanceMode, sessionTimeoutMinutes, twoFactorAuthEnabled, security{sessionTimeoutMinutes?,twoFactorRequired?}? (dup)		yes
`PaginatedResult<T>`	703-710	rows, total, page, pageSize, totalPages, facets?		contract
`QueryParams`	712-719	page, pageSize, search, sortBy, sortOrder, filters		unused
`NetworkSearchParams`	721-730	vertical?, destination?, wilayaCode?, guests?, minPrice?, maxPrice?, hostType?, propertyType?		yes
`CommandPaletteResult`	732-739	id, type `host|listing|user|booking|review|action`, title, subtitle, href, badge?		yes

Other types outside `lib/types`: `CurrentAdmin` (`lib/auth/current-admin.ts:28-35`), `SessionPayload` (`lib/auth/session.ts:20-26`), `ActionResult` / `ActionErrorCode` (`lib/actions/result.ts:8-12`), `FinanceSummary` (`lib/data/finance.ts:389-403`), `WalletAggregate` (`lib/data/ledger.ts:12-18`), filter param interfaces in each `lib/data/*.ts`, `Wilaya` (`lib/utils/wilayas.ts`, code string `"01".."68"`, region `north|high_plateaus|south`).

2.2 Mock collections (`lib/mock-data/store.ts:30-211`, seeded by `lib/mock-data/generator.ts`)

Seed RNG: LCG seed `20261020` (`generator.ts:25-34`), but dates use `Date.now()` so timestamps shift every boot.

Collection	Rows	Id format	Seed distribution (generator lines)
`users` (GuestUser)	320	`usr_001`	status: active 85%, suspended 9%, banned 4%, deleted 2% (`:84`); role `both` for i<=80 else `guest` (`:95`); never `pending`; bookingsCount 0-18 random, totalSpent = count x 12k-65k, walletBalanceDzd 0-45k (`:85-102`)
`hosts` (Host)	128 (32 per host type, round robin `:117`)	`hst_001`	status: approved 65%, under_review 13%, applied 10%, suspended 6%, rejected 4%, banned 2% (`:121-127`); verification verified 70%/pending 15%/missing 10%/expired 5% (`:129-131`); 15% commission overridden at default +2.5 or -2.0 (`:146-149`); docs: id_card always approved, non-individual also RC, tax card, tourism licence (approved if verified else pending) (`:157-194`); listingsCount/bookingsCount/rating random (`:208-211`); walletBalance overwritten from ledger (`:544-546`)
`listings` (Listing)	430	`lst_001`	random host; vertical by host type: agency trip/experience/vehicle, hotel+hostel `hotel`, individual home/experience (`:278-282`); status approved 65%, pending_review 13%, changes_requested 8%, rejected 6%, suspended 4%, draft 2%, archived 2% (`:284-291`); price ranges per vertical (`:293-298`); 2 RoomTypes for hotel/hostel (`:300-327`); isFeatured 12%, isReported 4%; coordinates random box around 28.03,1.66 +/-4deg (`:347-350`)
`bookings` (Booking)	840	`bkg_0001`, ref `BLD-2026-8001..8840`	status completed 50%, confirmed 25%, pending 10%, cancelled 7%, disputed 5%, refunded 3% (`:382-387`); nights 1-7, base = listing.price x nights, amounts via `computeBookingAmounts` (`:389-396`); payment refunded/pending/paid by status (`:429`); paymentMethod never `cash_on_arrival` (`:431`); check-in -120..+60 days
`reviews` (ReviewItem)	330	`rev_001`	from a random booking; published 85%, flagged 9%, hidden 4%, deleted 2% (`:452-456`); flagged rating 1-2 else 4-5
`transactions` (WalletTransaction)	1550	`tx_0001`, ref `TXN-DZ-10001..`	booking_payment 50% (+15k..85k), commission_deduction 25% (-2k..15k), host_payout 15% (-20k..100k), wallet_recharge 10% (+20k..150k) (`:492-512`); NOT linked to real bookings (random `BLD-2026-xxxx` refs `:496,522`); no `booking_refund` or `adjustment` seeded; sorted chronologically and `balanceAfter` computed (`:527-541`)
`wallets` (HostWallet)	128	`wlt_<hostId>`	derived from transactions (`:549-570`)
`audit` (AuditEntry)	215 seeded (frozen)	`aud_001`; runtime `aud_<ms>_<rand>`	all actor adm_01; 7 template actions incl. `COMMISSION_CHANGE`, `DISPUTE_RESOLVE` which no code path emits; entityId like `host_42` (not real ids) (`:574-598`)
`payouts` (PayoutRequest)	14	`pay_001`	first 14 approved hosts; idx<9 pending, 9-11 approved, 12-13 rejected (`:615`); pending idx%4==3 deliberately exceeds balance (`:616-620`); seeded approved ones have no `transactionId` and no matching ledger debit
`team` (AdminUser)	5	`adm_01..adm_05`; invite `adm_inv_<base36>`	one per role (`lib/mock-data/admins.ts:14-65`); passwords from env vars with dev defaults (`:67-82`)
`categories` (CategoryItem)	3	`cat_villas`, `cat_sahara`, `cat_4x4`; runtime `cat_<ms>`	listingCount hardcoded 48/42/28 (`store.ts:44-48`)
`promotions` (PromotionCoupon)	3	`cpn_sahara2026`, `cpn_marhaban`, `cpn_oran_sea`; runtime `cpn_<ms>`	SAHARA2026 15% experience; MARHABAN 5000 DZD fixed all; ORANSEA 10% home expired (`store.ts:50-90`)
`platformSettings`	1	n/a	`store.ts:92-106`: defaultLocale `fr`, currencies DZD/EUR/USD, guestServiceFeePercent 0, default rates, sessionTimeoutMinutes 480, twoFactorAuthEnabled false
`onboardingConfig`	1	n/a	version 1; individual 2 steps (5 fields), agency 1 step (2 fields), hotel 1 step (2), hostel 1 step (1) (`store.ts:108-191`)
`credentials`	Map adminId -> `scrypt$salt$hash`	n/a	lazily hashed on first login (`lib/data/credentials.ts:9-18`)

Generator file comments say ">= 125 hosts", ">= 420 listings" etc. Actual loop bounds: 320 / 128 / 430 / 840 / 330 / 1550 / 215 / 14.

---

3. Function inventory: `lib/data/*` and `lib/actions/*`

Conventions:

- Every `lib/data` function is `server-only`; every `lib/actions` function is `"use server"` and wraps the data call in `runAction(permission, fn)` (`lib/auth/run-action.ts:24-45`) which returns `{ok,data}|{ok:false,code}` with codes `unauthenticated|forbidden|invalid|conflict|failed`.
- Read functions in `lib/data` do NOT call `assertCan`; protection relies on the action or on `guardPage` in the calling server page. Mutations call `assertCan(actor.role, perm)` again (defense in depth).
- All mock functions `await delay(100..200ms)`.

- "Audit" column = calls `logAudit` / `logExport`.

3.1 analytics

File:L	Function	Inputs	Output	R/W	Callers	Perm	Audit	Future backend
data/analytics.ts:11	getDashboardKPIs	period `7d|30d|90d|1y`	net revenue, GMV, commission, counts, fixed occupancy 78.4/cancel 4.8/response 24, fake trends	R	`(panel)/page.tsx` (direct), StatsClient via action	page: none; action: reports.export	no	`GET /api/v1/admin/analytics/kpis?period=` (real time windows)
data/analytics.ts:55	getDashboardCharts	period (ignored)	monthlyRevenue (hardcoded), revenueByHostType, hostCategoryPerformance, revenueByVertical (listing counts), bookingsByWilaya top 10	R	dashboard page, StatsClient	same	no	`GET /api/v1/admin/analytics/charts?period=`
data/analytics.ts:161	getAttentionItems	none	counts of pending hosts/listings, flagged reviews, disputes + top items with hrefs	R	dashboard page	none	no	`GET /api/v1/admin/attention` (or reuse moderation queue counts)
data/analytics.ts:206	exportKpiReport	reportName, period, actor	CsvExport	R	StatsClient	reports.export	yes DATA_EXPORT `kpi_report`	`POST /api/v1/admin/reports/kpi/export` (async job)
actions/analytics.ts:6,10,14	getDashboardKPIs / getDashboardCharts / exportKpiReport	as above	ActionResult		StatsClient	reports.export	as data	

3.2 approvals

File:L	Function	Inputs	Output	R/W	Callers	Perm	Audit	Future backend
data/approvals.ts:8	getApprovalsInbox	tab `hosts|listings|documents|reviews|disputes`	`{items: ModerationCase[], counts}` oldest first	R	`(panel)/layout.tsx` (count badge, if hosts.view), ApprovalsClient	action: hosts.view	no	`GET /api/v1/admin/moderation/queue?type=&page=` + `GET /api/v1/admin/moderation/counts`
actions/approvals.ts:6	getApprovalsInbox	tab			ApprovalsClient	hosts.view		

3.3 audit

File:L	Function	Inputs	Output	R/W	Callers	Perm	Audit	Future backend
data/audit.ts:20	logAudit	actor, action, entityType, entityId, details, before?, after?	AuditEntry	W (append)	every mutating data fn, auth actions	n/a	is the audit	DB insert in same transaction as the mutation (Django signal or service layer)
data/audit.ts:48	logExport	actor, dataset, rowCount, filters	void	W	all export fns	n/a	`DATA_EXPORT`	same
data/audit.ts:102	listAuditLogs	page, pageSize(20), actorRole, entityType, search	Paginated	R	dashboard page (pageSize 6, if audit.view), AuditLogClient	audit.view	no	`GET /api/v1/admin/audit-logs?actor_role=&entity_type=&q=&page=`
data/audit.ts:127	exportAuditLogs	filters, actor	CsvExport	R	AuditLogClient	audit.view + reports.export	yes	`POST /api/v1/admin/audit-logs/export`
actions/audit.ts:8,13	listAuditLogs / exportAuditLogs				AuditLogClient	audit.view (+reports.export)		

3.4 auth / credentials / team-auth

File:L	Function	Inputs	Output	R/W	Callers	Perm	Audit	Future backend
actions/auth.ts:33	loginAction	FormData email, password, locale, returnTo	redirect or `{error}`	W (cookie, lastLoginAt, rate-limit map)	LoginForm	public	LOGIN_BLOCKED / LOGIN_FAILURE / LOGIN_SUCCESS	`POST /api/v1/admin/auth/log` (set httpOnly cookie or return token)
actions/auth.ts:116	logoutAction	FormData locale	redirect	W (cookie delete)	Header	signed-in	LOGOUT	`POST /api/v1/admin/auth/log` (server-side session revocation)
actions/auth.ts:137	getDemoCredentials	email	`{email,password}` or null (dev only)	R	LoginForm	public, NODE_ENV=development	no	drop
data/credentials.ts:9	getAdminPasswordHash	adminId	hash or null	R/W (lazy hash)	lib/auth/credentials.ts	n/a	no	Django auth `password` column
data/credentials.ts:21	listDevDemoAccounts	none	SA, MO, FI emails (dev)	R	login page	public dev	no	drop
data/credentials.ts:30	getDevSeedPassword	adminId	string (dev)	R	getDemoCredentials	dev	no	drop
auth/credentials.ts:17	verifyAdminCredentials	email, password	ok/unknown_account/bad_password/inactive	R	loginAction	n/a	no	Django `authenticate()`

File:L	Function	Inputs	Output	R/W	Callers	Perm	Audit	Future backend
auth/current-admin.ts:44	getCurrentAdmin (React cache)	cookie	CurrentAdmin or null	R	layout, guardPage, login page, dashboard, logout	n/a	no	`GET /api/v1/admin/auth/me`
auth/current-admin.ts:66,73	requireAdmin / requirePermission	perm	AdminActor	R	runAction			DRF permission classes

3.5 bookings

File:L	Function	Inputs	Output	R/W	Callers	Perm	Audit	Future backend
data/bookings.ts:93	listBookings	status, vertical, hostType, wilayaCode, paymentStatus, approvalMode, search, pendingOver24h, sortBy `created|checkIn|amount`, sortOrder, page, pageSize(12)	Paginated + facets status, vertical (global counts)	R	BookingsClient	bookings.view	no	`GET /api/v1/admin/bookings status=&vertical=&host_type`
data/bookings.ts:138	getBookingById	id	Booking or null	R	none (action unused)	bookings.view	no	`GET /api/v1/admin/bookings/`
data/bookings.ts:144	exportBookings	filters, actor	CsvExport (19 cols)	R	BookingsClient	reports.export + bookings.view	yes DATA_EXPORT `bookings`	`POST /api/v1/admin/booking`
data/bookings.ts:165	cancelBooking	id, reason (>=3), actor	Booking	W	BookingsClient	bookings.cancel	BOOKING_CANCEL	`POST /api/v1/admin/booking`
data/bookings.ts:205	applyRefund (private)	idx, amount, note, actor	{prev,next}	W + ledger	refundBooking, resolveDispute		via caller	service function inside a DB tr
data/bookings.ts:251	refundBooking	id, amount (int DZD), reason, actor	Booking	W + ledger `booking_refund`	BookingsClient (RefundDialog)	bookings.refund	BOOKING_REFUND / BOOKING_PARTIAL_REFUND	`POST /api/v1/admin/booking`
data/bookings.ts:284	resolveDispute	id, outcome, note, actor, amount?	Booking	W + ledger	BookingsClient (DisputeDialog), ApprovalsClient	bookings.dispute (+bookings.refund unless declined)	BOOKING_DISPUTE_RESOLVED / _DECLINED	`POST /api/v1/admin/booking`
actions/bookings.ts:11,15,19,24,30,50	listBookings, getBookingById, cancelBooking, refundBooking, resolveDispute, exportBookings	validated with `id`, `reason`, `dzd`, `oneOf`			as above	as above		

3.6 categories

File:L	Function	Inputs	Output	R/W	Callers	Perm	Audit	Future backend
data/categories.ts:10	listCategories	none	CategoryItem[] sorted by order	R	CategoriesClient, CategoriesManagerClient	action: **null** (any admin)	no	`GET /api/v1/admin/categories?vertical=`
data/categories.ts:15	createCategory	item minus id/listingCount, actor	CategoryItem	W	CategoriesClient, CategoriesManagerClient	categories.manage	CATEGORY_CREATE (entityType `setting`, no before/after)	`POST /api/v1/admin/categories`
data/categories.ts:33	toggleCategoryStatus	id, actor	CategoryItem	W	both	categories.manage	CATEGORY_TOGGLE	`PATCH /api/v1/admin/categories/{id} {is_active}`
data/categories.ts:59	deleteCategory	id, actor	true (hard delete, silent if unknown)	W	CategoriesClient	categories.manage	CATEGORY_DELETE	`DELETE /api/v1/admin/categories/{id}` (block if listings attached)
actions/categories.ts:7,11,15,19	same 4	`obj(item)` only, no field validation						

3.7 command palette

File:L	Function	Inputs	Output	R/W	Callers	Perm	Audit	Future backend
data/command-palette.ts:8	searchGlobalCommandPalette	query	<=12 results across hosts, listings, bookings, users (substring), or 5 default actions	R	CommandPalette	action: null, then filters by per-type perm + `canAccessPath` (`actions/command-palette.ts:18-26`)	no	`GET /api/v1/admin/search?q=` (Postgres trigram / full-text)

3.8 finance + ledger

File:L	Function	Inputs	Output	R/W	Callers	Perm	Audit	Future backend
data/ledger.ts:21	computeWalletAggregates	none	Map hostId -> {balance, totalEarned, totalCommissionsPaid, totalPayouts, lastTransactionAt}	R	listWallets, getFinanceSummary, listHosts	n/a	no	SQL `GROUP BY ho` balance with ledg

File:L	Function	Inputs	Output	R/W	Callers	Perm	Audit	Future backend
data/ledger.ts:38	getHostBalance	hostId	number	R	recharge, payouts, hosts	n/a	no	`SUM(amount)` unc
data/ledger.ts:58	appendLedgerEntry	host, type, amount (non-zero int), description, bookingRef?, receiptReference?, adminActor?	WalletTransaction (prefix ref `TXN-BKG/COM/PAY/REC/RFD/ADJ-<base36>`)	W	rechargeWallet, approvePayout, applyRefund	n/a	caller	`ledger_entries`
data/finance.ts:82	listWallets	hostType, wilayaCode, onlyNegative, minBalance, maxBalance, search, page, pageSize(12)	Paginated sorted by balance desc, facets hostType, solvency	R	PaymentsClient	wallets.view	no	`GET /api/v1/adm`
data/finance.ts:145	listTransactions	hostId, type, search, page, pageSize(15)	Paginated newest first	R	PaymentsClient	wallets.view	no	`GET /api/v1/adm` host=&type=&q=&p
data/finance.ts:160	exportTransactions	type, search, hostId, actor	CsvExport	R	PaymentsClient	reports.export + wallets.view	DATA_EXPORT `wallet_ledger`	`POST /api/v1/ad`
data/finance.ts:183	exportWallets	filters, actor	CsvExport	R	PaymentsClient	reports.export + wallets.view	DATA_EXPORT `wallets`	`POST /api/v1/ad`
data/finance.ts:205	rechargeWallet	hostId, amount 1..10,000,000, method (free string), reason (free, not min length), receiptReference (required), actor	{transaction, newBalance}	W ledger `wallet_recharge`	PaymentsClient, HostsClient	wallets.recharge	WALLET_RECHARGE (before/after balance, txId)	`POST /api/v1/admin/wa` {amount_dzd, met receipt_referenc receipt_reference
data/finance.ts:247	getCommissionRates	none	Record<HostType, number>	R	PaymentsClient	wallets.view	no	`GET /api/v1/adm`
data/finance.ts:252	updateCommissionRates	rates (each 0..100, 2 dp), actor	rates	W settings	PaymentsClient	commissions.manage	COMMISSION_RATES_UPDATE	`PUT /api/v1/adm` (versioned, effect
data/finance.ts:283	listPayouts	status	PayoutRequest[] pending first, no paging	R	PaymentsClient	wallets.view	no	`GET /api/v1/adm` status=&page=
data/finance.ts:311	approvePayout	id, actor	PayoutRequest	W ledger `host_payout` + payout	PaymentsClient	payouts.approve	PAYOUT_APPROVE	`POST /api/v1/admin/pa` later `/mark-proce`
data/finance.ts:359	rejectPayout	id, reason (>=3), actor	PayoutRequest	W	PaymentsClient	payouts.approve	PAYOUT_REJECT	`POST /api/v1/ad` {reason}
data/finance.ts:406	getFinanceSummary	none	FinanceSummary (wallet total, negatives, pending payouts, month commissions/payouts, refunds, escrow)	R	PaymentsClient	wallets.view	no	`GET /api/v1/adm`
data/finance.ts:468	listRefunds	page, pageSize(10)	refunded bookings projection	R	PaymentsClient	wallets.view	no	`GET /api/v1/adm` a `refunds` table, field)
actions/finance.ts:22-128	12 wrappers	validated with dzd/rate/reason/str				as above		

3.9 hosts

File:L	Function	Inputs	Output	R/W	Callers	Perm	Audit	Future bacl
data/hosts.ts:38	getProviderCategory	Host	ProviderCategory	R	listHosts, exportHosts			DB column
data/hosts.ts:47	listHosts	category, hostType, status (comma list), verificationStatus, wilayaCode (str or array), search, approvalMode, vertical, minRating, minBookings, minListings, onlyNegativeBalance, minStars, hasTrips, sortBy `joined|rating|listings|bookings|balance`, sortOrder, page, pageSize(12) (minPrice/maxPrice declared but ignored)	Paginated + facets category (filtered), hostType, status (global) + formattedId `#P00001`	R	`hosts/page.tsx` (direct SSR), HostsClient, ProvidersValidationClient	hosts.view	no	`GET /api/v`
data/hosts.ts:199	getHostById	id	Host with ledger balance	R	`hosts/[id]/page.tsx` (direct); action unused	hosts.view	no	`GET /api/v`
data/hosts.ts:206	exportHosts	filters, actor	CsvExport (17 cols)	R	HostsClient	reports.export + hosts.view	DATA_EXPORT `hosts`	`POST /api/`
data/hosts.ts:249	transitionHost (private)	id, action, actor, reason?	Host	W	7 wrappers below	per action (`:221-229`)	yes (`:231-239`)	service + sta

File:L	Function	Inputs	Output	R/W	Callers	Perm	Audit	Future back
data/hosts.ts:304	approveHost	id, actor	Host (status approved, verification verified)	W	ApprovalsClient, HostsClient, ProvidersValidationClient	hosts.approve	HOST_APPROVE	`POST /api/v1/adm`
data/hosts.ts:308	rejectHost	id, reason, actor	Host	W	ApprovalsClient, ProvidersValidationClient	hosts.reject	HOST_REJECT	`POST /api/v1/adm {reason}`
data/hosts.ts:312	suspendHost	id, reason, actor	Host	W	HostsClient	hosts.suspend	HOST_SUSPEND	`POST /api/v1/adm {reason}`
data/hosts.ts:316	unsuspendHost	id, actor	Host	W	HostsClient, ProvidersValidationClient	hosts.suspend	HOST_UNSUSPEND	`POST /api/v1/adm`
data/hosts.ts:320	banHost	id, reason, actor	Host	W	none (no UI)	hosts.delete	HOST_BAN	`POST /api/ {reason}`
data/hosts.ts:324	deleteHost	id, actor	Host (soft)	W	HostsClient	hosts.delete	HOST_DELETE	`DELETE /ap` (soft)
data/hosts.ts:328	restoreHost	id, actor	Host	W	HostsClient	hosts.delete	HOST_RESTORE	`POST /api/v1/adm`
data/hosts.ts:332	overrideCommission	id, rate, actor	Host	W	HostsClient	hosts.commission_override	HOST_COMMISSION_OVERRIDE	`PUT /api/v1/adm {rate} + D`
actions/hosts.ts:8-57	11 wrappers							

3.10 listings + network

File:L	Function	Inputs	Output	R/W	Callers
data/listings.ts:39	matchListingStatus	listingStatus, filterStatus (comma list, alias aware)	bool	R	listListings
data/listings.ts:68	listListings	status, vertical, hostType, wilayaCode, search, minPrice, maxPrice, approvalMode, isFeatured, isReported, minCapacity, minBedrooms, amenity, sortBy `created|price|rating|bookings`, sortOrder, page, pageSize(12)	Paginated `+ facets status (scoped to vertical, incl. alias keys), vertical; formattedId #H/#EXP/#T/#V`	R	`listings/page.t` ListingsClient, Stays/Experience
data/listings.ts:212	getListingById	id	Listing	R	`stays
data/listings.ts:219	exportListings	filters, actor	CsvExport (17 cols)	R	ListingsClient, useListingStatusA
data/listings.ts:252	transitionListing (private)	id, target, actor, {reason, auditAction, notePrefix}	Listing	W	wrappers
data/listings.ts:296	approveListing	id, actor	Listing	W	Approvals, Listing Experiences, Trips StayReview, Netw
data/listings.ts:300	rejectListing	id, reason, actor	Listing	W	Approvals, StayD
data/listings.ts:304	requestChangesListing	id, notes, actor	Listing	W	Listings, StayDeta
data/listings.ts:312	suspendListing	id, reason, actor	Listing	W	NetworkClient
data/listings.ts:316	deleteListing	id, actor	Listing (soft)	W	ListingsClient, useListingStatusA
data/listings.ts:321	restoreListing	id, actor	Listing	W	StayDetail, useLis
data/listings.ts:348	featureListing	id, isFeatured, actor	Listing	W	ListingsClient
data/listings.ts:379	updateListingStatus	id, status, actor, reason?	Listing	W	Stays/Experience StayDetail, useLis
data/network.ts:11	searchNetwork	NetworkSearchParams (vertical default `home`)	{results (all statuses, unpaginated), total, vertical}	R	NetworkClient, S
actions/listings.ts:39-95, actions/network.ts:7	wrappers				

3.11 onboarding

File:L	Function	Inputs	Output	R/W	Callers	Perm	Audit	Future backend
data/onboarding.ts:10	getOnboardingConfig	none	deep copy OnboardingConfig	R	OnboardingClient	onboarding.edit	no	admin: `GET /api/v1/admin/onboarding-configs` ; public: `GET /api/v1/onboarding-config/{hostType}` (published only)
data/onboarding.ts:15	updateOnboardingFlow	hostType, steps (unvalidated array), actor	OnboardingConfig (version+1)	W	OnboardingClient	onboarding.edit	ONBOARDING_CONFIG_PUBLISH (before/after flow)	`POST /api/v1/admin/onboarding-configs/{hostType}/versions` (draft) + `POST .../versions/{v}/publish`
actions/onboarding.ts:9,13	wrappers	`oneOf(hostType)` , `Array.isArray(steps)` only						

3.12 promotions

File:L	Function	Inputs	Output	R/W	Callers	Perm	Audit	Future backend
data/promotions.ts:10	listPromotions	none	PromotionCoupon[]	R	PromotionsClient	promotions.manage	no	`GET /api/v1/admin/coupons`
data/promotions.ts:15	createPromotion	coupon minus id/usedCount (no validation, no code uniqueness)	coupon	W	PromotionsClient	promotions.manage	PROMOTION_CREATE	`POST /api/v1/admin/coupons`
data/promotions.ts:33	togglePromotionStatus	id	coupon (`active` <-> `disabled`)	W	PromotionsClient	promotions.manage	PROMOTION_TOGGLE	`PATCH /api/v1/admin/coupons/{id}`
data/promotions.ts:60	deletePromotion	id	true (hard delete)	W	PromotionsClient	promotions.manage	PROMOTION_DELETE	`DELETE /api/v1/admin/coupons/{id}` (soft if used)

3.13 reviews

File:L	Function	Inputs	Output	R/W	Callers	Perm	Audit	Future backend
data/reviews.ts:20	listReviews	status, vertical, rating (exact), wilayaCode, search, page, pageSize(12)	Paginated + facets status	R	ReviewsClient	reviews.view	no	`GET /api/v1/admin/reviews?status=&vertical=&rating=&wilaya=&q=&page=`
data/reviews.ts:82	hideReview	id	ReviewItem	W	ReviewsClient, ApprovalsClient (reject)	reviews.moderate	REVIEW_HIDE (full object before/after)	`POST /api/v1/admin/reviews/{id}/hide {reason}`
data/reviews.ts:111	restoreReview	id	ReviewItem (published)	W	ReviewsClient	reviews.moderate	REVIEW_RESTORE	`POST /api/v1/admin/reviews/{id}/restore`
data/reviews.ts:140	deleteReview	id	ReviewItem (soft)	W	ReviewsClient	reviews.delete	REVIEW_DELETE	`DELETE /api/v1/admin/reviews/{id}`
data/reviews.ts:169	dismissFlag	id	ReviewItem (published, flagReason cleared)	W	ReviewsClient, ApprovalsClient (approve)	reviews.moderate	REVIEW_FLAG_DISMISS	`POST /api/v1/admin/reviews/{id}/flags/dismiss`

3.14 settings

File:L	Function	Inputs	Output	R/W	Callers	Perm	Audit	Future backend
data/settings.ts:10	getPlatformSettings	none	PlatformSettings	R	SettingsClient	settings.manage	no	`GET /api/v1/admin/settings`
data/settings.ts:15	updatePlatformSettings	Partial (no field validation, shallow merge)	PlatformSettings	W	SettingsClient	settings.manage	PLATFORM_SETTINGS_UPDATE	`PATCH /api/v1/admin/settings` with serializer whitelist

3.15 team

File:L	Function	Inputs	Output	R/W	Callers	Perm	Audit	Future backend
data/team.ts:19	listTeamMembers	none	AdminUser[]	R	TeamClient	team.manage	no	`GET /api/v1/admin/staff`
data/team.ts:28	getTeamMemberById	id	AdminUser	R	getCurrentAdmin	internal	no	session lookup
data/team.ts:33	findTeamMemberByEmail	email	AdminUser	R	verifyAdminCredentials, getDemoCredentials	internal	no	
data/team.ts:42	recordTeamMemberLogin	id	void	W lastLoginAt	loginAction	internal	no (LOGIN_SUCCESS covers it)	`last_login`
data/team.ts:51	updateMemberRole	userId, role, actor	AdminUser	W	TeamClient	team.manage	TEAM_ROLE_UPDATE (entityType `user`)	`PATCH /api/v1/admin/staff/{id} {role}`
data/team.ts:96	toggleMemberActive	userId, actor	AdminUser	W	TeamClient	team.manage	TEAM_STATUS_TOGGLE (no before/after)	`POST /api/v1/admin/staff/{id}/activate` / `/deactivate`

File:L	Function	Inputs	Output	R/W	Callers	Perm	Audit	Future backend
data/team.ts:143	inviteMember	{name, email, role}, actor	AdminUser (invited, inactive)	W	TeamClient	team.manage	TEAM_MEMBER_INVITE (entityType `team`)	`POST /api/v1/admin/staff/invitations` + `POST /api/v1/admin/auth/accept-invite {token, password}`

3.16 users (guests)

File:L	Function	Inputs	Output	R/W	Callers	Perm	Audit	Future backend
data/users.ts:26	listUsers	status (comma list), role, emailVerified, idVerified, wilayaCode, search, minBookings, minSpend, isFlagged, sortBy `joined|bookings|spend|activity`, sortOrder, page, pageSize(12)	Paginated + facets status, role	R	`users/page.tsx`, `users/guests/page.tsx` (direct), UsersClient	users.view	no	`GET /api/v1/admin/users?...`
data/users.ts:127	getUserById	id	GuestUser	R	none	users.view	no	`GET /api/v1/admin/users/{id}`
data/users.ts:133	suspendUser	id, reason (any string, may be empty), actor	GuestUser	W	UsersClient	users.suspend	USER_SUSPEND (full user objects incl. PII in before/after)	`POST /api/v1/admin/users/{id}/suspend {reason}`
data/users.ts:164	unsuspendUser	id	GuestUser	W	none	users.suspend	USER_UNSUSPEND	`POST /api/v1/admin/users/{id}/unsuspend`
data/users.ts:193	banUser	id, reason	GuestUser	W	UsersClient	users.ban	USER_BAN	`POST /api/v1/admin/users/{id}/ban {reason}`
data/users.ts:224	deleteUser	id	GuestUser (soft)	W	UsersClient	users.delete	USER_DELETE	`DELETE /api/v1/admin/users/{id}` (soft + anonymise)
data/users.ts:253	restoreUser	id	GuestUser (active)	W	none	users.delete	USER_RESTORE	`POST /api/v1/admin/users/{id}/restore`
data/users.ts:282	updateUserStatus	id, status (any GuestStatus), actor	GuestUser	W	UsersClient (mark active / mark pending)	NO assertCan in data; action maps status -> perm (`actions/users.ts:12-16,46-51`)	USER_STATUS_CHANGE	`POST /api/v1/admin/users/{id}/status {status, reason}` with state machine

3.17 Unused exports (no UI caller)

`actions/bookings.getBookingById`, `actions/hosts.getHostById`, `actions/hosts.banHost`, `actions/listings.getListingById`, `actions/users.getUserById`, `actions/users.unsuspendUser`, `actions/users.restoreUser`. Missing entirely: document approve/reject, host "changes requested", amenity CRUD, category edit/reorder, coupon edit, review reply/report handling, payout `processed`, ledger `adjustment`, change-request CRUD, listing create/edit, messaging, notifications, templates, staff password reset/MFA.

---

4. Business rules in code

4.1 Money and commissions

Rule	Value	Source
All money integer DZD	`isValidDzdAmount` int in [min,max]	`lib/pricing.ts:54-66`, `lib/auth/run-action.ts:81-86`
Guest service fee	**0%** ("BLADUS does not charge travellers")	`lib/pricing.ts:11-16`; frontend agrees (`frontend/lib/booking/config.ts:21-34`); `docs/DATA-MODEL.md` s6 still says 7 (stale)
Default commission per host type	individual 12, agency 15, hotel 18, hostel 10 (% of base)	`lib/pricing.ts:19-25`; copied into settings `store.ts:101`; hardcoded AGAIN in `lib/data/analytics.ts:69-74` and `StatsClient.tsx:61-66`
Commission rate validation	0..100, max 2 decimals	`lib/pricing.ts:32-34,69-74`, `run-action.ts:89-94`
Change defaults	`updateCommissionRates` replaces `platformSettings.defaultCommissionRates`; does NOT update `Host.commissionRate` of non-overridden hosts and does NOT reprice existing bookings	`lib/data/finance.ts:252-279`
Per-host override	sets `commissionRate` + `isCommissionOverridden=true`; no way to clear the override	`lib/data/hosts.ts:332-364`
Seed overrides	15% of hosts, default +2.5 or -2.0	`generator.ts:146-149`
Booking amounts	`serviceFee = round(base*0/100)=0`, `total = base + serviceFee`, `commission = round(base*rate/100)`, `hostPayout = base - commission`	`lib/pricing.ts:77-92`; base = listing.price x nights (`generator.ts:390`)
Commission rate frozen on booking	`Booking.commissionRate` stored at creation	`generator.ts:426`
Display discounts	weekly 10, biweekly 15, monthly 25, group 10; `applyDiscount`	`lib/pricing.ts:37-51`; display only (ListingsClient, CarsListClient)
Max manual recharge / generic max amount	10,000,000 DZD	`lib/pricing.ts:28-30`

4.2 Wallet ledger

Rule	Detail	Source
Balance = sum of ledger entries; `Host.walletBalance` and `HostWallet.balance` re-derived on every read		`lib/data/ledger.ts:1-42`, `finance.ts:34-49`, `hosts.ts:62-63,202`
Entry types and signs	booking_payment +, commission_deduction -, host_payout -, wallet_recharge +, booking_refund -, adjustment +/-	`lib/types/index.ts:499-505`, `docs/DATA-MODEL.md` s4
Entry must be non-zero integer	DomainError otherwise	`ledger.ts:67-69`
`balanceAfter` = current sum + amount at write time		`ledger.ts:71`
Reference prefixes	TXN-BKG, TXN-COM, TXN-PAY, TXN-REC, TXN-RFD, TXN-ADJ + base36 time + random	`ledger.ts:44-51,72`
Atomicity	"no await between check and write" (single-threaded mock)	`ledger.ts:53-57`; real DB needs transaction + `SELECT ... FOR UPDATE`
Who writes entries at runtime	only recharge (+), payout approval (-), refund host share (-). Nothing writes `booking_payment` or `commission_deduction` when a booking is confirmed/completed.	`finance.ts:225`, `finance.ts:328`, `bookings.ts:229`
Negative balance allowed	yes (seed has negatives; `isNegative` flag; refund clawback can go below 0)	`generator.ts:568`, `bookings.ts:226-237`

4.3 Wallet recharge (`lib/data/finance.ts:205-245`, `lib/actions/finance.ts:46-66`)

- Perm `wallets.recharge`; amount int 1..10,000,000; `receiptReference` required (trimmed, <=200 chars); `method` free string <=100 (UI sends values from PaymentsClient select or hardcoded `ccp_transfer` in HostsClient `:411`); `reason` free string, no minimum.
- Writes `wallet_recharge` entry with description `Admin manual recharge (<method>): <reason>`, `adminActor` = actor NAME.
- **No idempotency**: same receipt reference can be credited twice; no duplicate check, no four-eyes approval.
- Audit `WALLET_RECHARGE` with balance before/after and txId.

4.4 Payouts (`lib/data/finance.ts:281-385`)

Step	Rule
List	pending first then newest (`:287-291`), no paging
Approve	perm `payouts.approve`; only from `pending` (`findPendingPayout :295-303`); host must exist; amount positive int; refuse if `getHostBalance < amount` with "Insufficient wallet balance" DomainError (`:322-327`); writes `host_payout` -amount; sets status `approved`, processedAt, processedBy (name), transactionId (`:335-342`)
Double-debit protection	status check + balance check + ledger write + status write in one synchronous block (`:315-343`); second call fails because status is no longer `pending`. No idempotency key, no DB lock (mock only).
Confirmation UI	confirm dialog `payoutToApprove`, busy flag prevents double click (`PaymentsClient.tsx:356-376`)
Reject	from `pending` only, reason >=3 chars, no ledger movement (`:359-385`)
`processed`	type value exists; no code transitions approved -> processed (no bank-confirmation step)
Seed	14 requests; 2 of the 9 pending deliberately exceed balance (`generator.ts:613-620`)
Account display	masked last 4 of ccpRip ?? rib ?? phone (DATA-MODEL s4)

4.5 Bookings, cancellations, refunds, disputes (`lib/domain/transitions.ts:13-51`, `lib/data/bookings.ts`)

- `BOOKING_TRANSITIONS`: pending -> confirmed|cancelled; confirmed -> completed|cancelled|disputed|refunded; completed -> disputed|refunded; disputed -> confirmed|completed|refunded; cancelled -> refunded; refunded -> none (`transitions.ts:13-20`). Only cancel, refund and dispute-resolution transitions are reachable from the admin; confirm/complete/dispute-open are not.
- Cancel: allowed from pending/confirmed, reason >=3; sets cancelledAt; **no automatic refund, no ledger reversal** (`bookings.ts:165-197`).
- Refundable statuses: confirmed, completed, cancelled, disputed; payment must be `paid` or `partially_refunded`; remaining = total - refundedSoFar (`transitions.ts:32-45`).
- Refund: int DZD, `0 < amount <= remaining`; cumulative `refundedAmountDzd`; full -> status `refunded` + payment `refunded`, else payment `partially_refunded` (`bookings.ts:205-249`).
- Host clawback: `hostShare = round(amount * hostPayoutDzd / totalAmountDzd)` written as negative `booking_refund` (`bookings.ts:225-237`). Platform keeps the commission share of a refund implicitly (commission never reversed).
- Dispute resolve: only from `disputed`; note >=3 required; `refund_full` refunds remaining -> `refunded`; `refund_partial` requires amount < remaining -> back to `completed` if checkout passed else `confirmed`; `declined` no money -> lifecycle status; stores `disputeResolution` (`bookings.ts:284-346`).
- Saved view "pending > 24h": `status=pending && createdAt < now-24h` (`bookings.ts:72-78`). Matches frontend `REQUEST_TO_BOOK_TTL_HOURS = 24` (`frontend/lib/booking/config.ts:55`) but admin never expires anything.
- Escrow (finance KPI): sum of total - refunded for confirmed/disputed paid bookings (`finance.ts:443-448`).
- No idempotency on refunds: two concurrent identical refund calls in a real DB would both pass without a lock.

4.6 Host state machine (`lib/domain/transitions.ts:55-71`, `lib/data/hosts.ts:221-302`)

Action	From	To	Reason	Perm	Audit
approve	applied, under_review, rejected	approved (+verification `verified`)	no	hosts.approve	HOST_APPROVE
reject	applied, under_review	rejected	yes	hosts.reject	HOST_REJECT
suspend	approved	suspended	yes	hosts.suspend	HOST_SUSPEND
unsuspend	suspended	approved	no	hosts.suspend	HOST_UNSUSPEND
ban	approved, suspended, rejected, under_review, applied	banned	yes	hosts.delete	HOST_BAN
delete	any except deleted	deleted, stores statusBeforeDelete	no	hosts.delete	HOST_DELETE
restore	deleted	statusBeforeDelete or under_review	no	hosts.delete	HOST_RESTORE

- No transition applied -> under_review (no "start review" action). Banned is terminal except via delete/restore.

- Reasons appended to `Host.adminNotes` as text (`hosts.ts:269-278`).
- Approve sets verification to `verified` without touching `documents[].status` .
- Suspending/banning a host does NOT cascade to its listings or open bookings.
- A second, contradictory transition table exists in `lib/status.ts:58,68` (`HOST_TRANSITIONS` , `LISTING_TRANSITIONS`) that differs from `lib/domain/transitions.ts` (e.g. allows `deleted -> approved|suspended` , `banned -> suspended`). Only `lib/domain/transitions.ts` is enforced.

4.7 Listing state machine (`lib/domain/transitions.ts:73-109` , `lib/data/listings.ts:233-387`)

- Transitions: draft -> pending_review|deleted; pending_review -> approved|changes_requested|rejected|deleted; changes_requested -> pending_review|approved|rejected|deleted; approved -> suspended|archived|changes_requested|pending_review|deleted; rejected -> pending_review|deleted; suspended -> approved|pending_review|deleted; archived -> approved|deleted; deleted -> only `restoreListing` (`:83-93`).
- Reason (>=3 chars, `MIN_REASON_LENGTH`) required for rejected, suspended, changes_requested (`:102-112`); appended to `adminNotes` .
- Permission by target: rejected -> listings.reject; suspended/archived -> listings.suspend; deleted -> listings.delete; anything else (approved, pending_review, changes_requested, draft) -> listings.approve (`listings.ts:233-245`).
- Legacy aliases accepted on read and filter (`matchListingStatus :39-66`), folded on write.
- Feature: only `approved` listings can be featured; unfeature always allowed (`listings.ts:360-362`). Featuring is a boolean only (no placement, no period, no ordering, no price).
- Approval queue only picks literal `pending_review` (`approvals.ts:33`), not legacy `pending` .

4.8 Reviews (`lib/data/reviews.ts`)

- No state machine: hide/restore/delete/dismiss can be applied from any status (e.g. restore a deleted review to published). No reason captured. Flags are a single `flagReason` string on the review (no flag entity, no reporter list). `hostReply` is display only.

4.9 Guests (`lib/data/users.ts`)

- No state machine; `updateUserStatus` can move any status to any status; reason optional/empty for suspend and ban (action uses `str` , not `reason` , `actions/users.ts:26-36`). `pending` status exists but nothing explains its meaning.

4.10 Moderation SLA / waiting time (`lib/data/approvals.ts`)

Queue	Source rows	waitingDays from	Priority rule	Line
hosts	status applied or under_review	`joinedAt`	high if >3 days	12-30
listings	status `pending_review`	`createdAt` (not submission time)	high if >4 days	32-50
documents	`Host.documents[].status == pending`	`uploadedAt`	high if >3 days	52-72
reviews	status flagged	`createdAt` of review	always high	74-90
disputes	booking status disputed	booking `createdAt` (not dispute time)	always high	92-108

`waitingDays = max(1, floor((now - t)/1 day))` ; sorted oldest first (`:126-127`). No `low` priority produced. No assignment/claiming, no SLA breach timestamps.

4.11 Promotions

- Coupon toggle `active <-> disabled` (`promotions.ts:43`); `expired` is never computed from dates. No validation of code uniqueness, ranges, or dates. Create from UI sends `maxUses` + `expiresAt` (`PromotionsClient.tsx:120-129`) while seeds use `maxUsage` + `startDate/endDate` ; `scope` not set on create. `usedCount` never incremented (no redemption path).
- Featured placements tab is static (`PromotionsClient.tsx:45-49`).

4.12 Categories taxonomy

- Flat list: `{id, name{en,fr,ar}, vertical, slug?, iconName?/icon?, listingCount, isActive, order}` . 3 seeds (home, experience, vehicle). `listingCount` hardcoded, not computed. No parent/child, no link from `Listing` (listing uses free-text `propertyType` , seeds per vertical in `generator.ts:267-273`). Hard delete. Car "pricing" and "insurance" pages also write into this list.

4.13 Onboarding config versioning (`lib/data/onboarding.ts`)

- Single global `version` integer incremented on every publish of ANY host type (`:31`); `updatedAt` , `publishedBy` (name). No draft vs published, no history kept (previous flow only in the audit `beforeState`), no rollback.
- Only server rule: `individual` max 2 steps (`:24-26` , also UI `OnboardingClient.tsx:118,158`). `maxPagesAllowed` is preserved but not checked. Steps are not schema-validated.
- Audit `ONBOARDING_CONFIG_PUBLISH` , entityType `onboarding` , entityId = hostType.

4.14 Audit log (`lib/data/audit.ts` , `lib/mock-data/store.ts:240-254`)

- Fields: id, timestamp, actorId, actorName, actorRole, action, entityType, entityId, details, beforeState?, afterState?, ipAddress (`types:569-584`). `entityName` never filled.
- Append-only: store has no setter; entries frozen (`store.ts:245-254`). Actor and IP from server session / headers (`client-ip.ts` : first X-Forwarded-For, then X-Real-IP, else `unknown` , max 64 chars).
- Logged actions (53 code paths, see list): BOOKING_CANCEL, BOOKING_REFUND, BOOKING_PARTIAL_REFUND, BOOKING_DISPUTE_RESOLVED, BOOKING_DISPUTE_DECLINED, CATEGORY_CREATE, CATEGORY_TOGGLE, CATEGORY_DELETE, COMMISSION_RATES_UPDATE, DATA_EXPORT, HOST_APPROVE, HOST_REJECT, HOST_SUSPEND, HOST_UNSUSPEND, HOST_BAN, HOST_DELETE, HOST_RESTORE, HOST_COMMISSION_OVERRIDE, LISTING_APPROVE, LISTING_REJECT, LISTING_REQUEST_CHANGES, LISTING_SUSPEND, LISTING_DELETE, LISTING_RESTORE, LISTING_FEATURE, LISTING_UNFEATURE, LISTING_STATUS_CHANGE, LOGIN_BLOCKED, LOGIN_FAILURE, LOGIN_SUCCESS, LOGOUT, ONBOARDING_CONFIG_PUBLISH, PAYOUT_APPROVE, PAYOUT_REJECT, PLATFORM_SETTINGS_UPDATE, PROMOTION_CREATE, PROMOTION_TOGGLE, PROMOTION_DELETE, REVIEW_HIDE, REVIEW_RESTORE, REVIEW_DELETE, REVIEW_FLAG_DISMISS, TEAM_ROLE_UPDATE, TEAM_STATUS_TOGGLE, TEAM_MEMBER_INVITE, USER_SUSPEND, USER_UNSUSPEND, USER_BAN, USER_DELETE, USER_RESTORE, USER_STATUS_CHANGE, WALLET_RECHARGE. Seed-only: COMMISSION_CHANGE, DISPUTE_RESOLVE.
- NOT logged: reads/views of PII, failed permission checks (ForbiddenError), approvals inbox views, mocked-only UI actions, login with empty fields (`auth.ts:56-59` records rate-limit failure but no audit).
- Inconsistent entityType: team role/toggle use `user` (`team.ts:86,130`) while invite uses `team` ; categories use `setting` .
- before/after sizes vary: users/reviews/team store FULL objects (PII, `users.ts:157-158`), hosts/listings store only `{status}` .

- Export audit records active filters and row count (`audit.ts:48-68`).

4.15 Auth, session, rate limit, MFA

Item	Value	Source
Cookie	`darna_admin_session` , httpOnly, secure in prod, sameSite strict, path `/` , maxAge = TTL	`lib/auth/config.ts:8` , `lib/actions/auth.ts:96-102`
JWT	HS256, iss `urn:darna:admin` , aud `urn:darna:admin-panel` , sub = userId, claims userId/email/name/role/avatar	`config.ts:11-12` , `session.ts:28-51`
Secret	`ADMIN_AUTH_SECRET` >= 32 chars required in prod; dev fallback constant	`config.ts:14-35`
Session TTL	**8 h** absolute (no sliding, no refresh, no server-side revocation list)	`config.ts:9`
`platformSettings.sessionTimeoutMinutes`	480 in store, UI writes `security.sessionTimeoutMinutes` (default 120 in UI); neither is read by auth	`store.ts:104` , `SettingsClient.tsx:125,147,166`
Revocation	effective immediately for deactivation / role change because every request re-reads the store (`current-admin.ts:44-59`); logout only deletes the cookie (token still valid until exp)	
Middleware	edge check of JWT only; redirects to `/{locale}/login?returnTo=` ; `/login` exact match exempt	`middleware.ts:19-56`
returnTo	same-origin relative path only, no `//` , `\` , control chars, not the login page, <=2048 chars	`lib/auth/return-to.ts:17-37`
Password hashing	scrypt, 16-byte salt, 64-byte key, `scrypt$salt$hash` ; constant-time compare; dummy hash for unknown emails	`lib/auth/password.ts:12-34` , `credentials.ts:17-27`
Login rate limit	5 failures within 15 min window -> 60 s lock, keyed separately by `email:<lower>` and `ip:<ip>` ; success clears only the email key	`lib/auth/rate-limit.ts:9-77`
Generic error	unknown email, bad password, inactive -> same `invalidCredentials`	`auth.ts:69-82`
Input caps	email 254, password 256	`auth.ts:25-26`
MFA	**none**. Only boolean flags `twoFactorAuthEnabled` / `security.twoFactorRequired` in settings, never enforced	`types:696-700` , `store.ts:105`
Password reset / change for staff	none	
Invite acceptance	none (invited accounts can never sign in)	`team.ts:138-186`

4.16 Exports

- CSV built server-side by `toCsv` (`lib/utils/csv.ts`): all cells quoted, `"` doubled, cells starting with `= + - @ TAB CR` prefixed with `'` . Exports cover the full filtered dataset, require `reports.export` plus the dataset view permission (`requireAlso`), and are audited as `DATA_EXPORT` . Filenames `darna_<dataset>_<date>.csv` (still "darna").

---

5. Onboarding configurator contract

5.1 Admin shape (what `/settings/onboarding` edits and publishes)

```
OnboardingConfig {                      // lib/types/index.ts:641-649
  version: number                       // global counter, +1 per publish of any flow
  updatedAt: ISO string
  publishedBy: string                   // admin NAME
  flows: {
    individual | agency | hotel | hostel: {
      maxPagesAllowed?: number          // 2 for individual
      steps: OnboardingStepConfig[]
    }
  }
}
OnboardingStepConfig {                  // :628-639
  id: string
  stepNumber?: number   order?: number     // seeds use stepNumber, UI-created steps use order
  title?: {en,fr,ar}    name?: {en,fr,ar}  // seeds use title, UI-created steps use name
  description: {en,fr,ar}
  isCore?, isActive?, isRequired?: boolean // seeds: isCore+isActive; UI: isRequired
  fields: OnboardingFieldConfig[]
}
OnboardingFieldConfig {                 // :615-626
  id?: string  key?: string             // seeds use id, UI-created field uses key "custom_notes"
  label: {en,fr,ar}
  type: text|textarea|number|select|multiselect|radio|file|location|amenities|boolean
  required?, order?, isCore?
  placeholder?, helpText?: {en?,fr?,ar?}
  options?: [{value, label{en,fr,ar}}]
}
```

5.2 Seeded flows (`lib/mock-data/store.ts:108-191`)

Host type	Step id	Title (en)	Fields (id: type, required)
individual (max 2)	step_ind_1	Property Basics & Space	vertical: select (core, NO options), propertyType: text, wilaya: location (core)
individual	step_ind_2	Pricing & Photos	priceDzd: number (core), photos: file (core)
agency	step_ag_1	Agency Registration	tradeRegister: text, tourismLicense: text
hotel	step_htl_1	Hotel Classification	starRating: number, roomCount: number

Host type	Step id	Title (en)	Fields (id: type, required)
hostel	step_hst_1	Hostel & Dormitory Details	bedCount: number

5.3 Editor capabilities (`app/[locale]/(panel)/settings/onboarding/OnboardingClient.tsx`)

- Switch host type; reorder steps (renumbers `order` , `:89-108`); toggle `isRequired` (`:110-114`); add step with fixed template `id: step_<Date.now()>` , `name` , one `custom_notes` text field (`:116-148`); remove step (no `isCore` guard, `:150-154`); publish whole step array for the active host type (`:156-176`); view/copy JSON of the full config (`:386-418`).
- Cannot: edit step titles/descriptions, add/edit/remove fields, set field types/options/placeholders, toggle `isActive` , preview, save a draft, see history, roll back.

5.4 Guest-site counterpart (`frontend/lib/onboarding-config.ts` , `frontend/types.ts:507-572`)

Aspect	Admin	Frontend
Container	`OnboardingConfig.flows[hostType].steps` + global version	`Record<HostType, OnboardingStepConfig[]>` , optional `HostTypeOnboardingConfig {hostType, steps, updatedAt?}`
Storage	server mock store	hardcoded defaults + browser `localStorage` key `darna_onboarding_config_<hostType>` (`onboarding-config.ts:1064-1091`)
Step count	individual 2, agency 1, hotel 1, hostel 1	individual **12** (`:5-430`), agency 5 (`:433-605`), hotel 7 (`:607-840`), hostel 7 (`:842-1062`)
Step ids	`step_ind_1` , `step_ag_1` , … , UI `step_<ms>`	`category_vertical` , `placement` , `location` , …, `agency_basics` , `hotel_basics` , `hostel_basics` ; custom steps must start with `custom_step_`
Step required props	`stepNumber?` , `title?` , `isActive?` optional	`stepNumber` , `title` , `isActive` required
Field id	`id?` or `key?`	`id` required, `order` and `required` required
Field types	text, textarea, number, select, multiselect, radio, file, location, amenities, boolean	text, number, textarea, select, multi-select, image-upload, toggle
Field `defaultValue`	absent	present
Consumption	none	`BecomeHostFlow.tsx:189` calls `getOnboardingSteps(hostType)` ; renders extra non-core fields of a step matched by step id (`:192-200`) and fully renders only custom steps whose id starts with `custom_step_` (`:2387-2389`); progress bar counts `isActive` steps (`:1035-1038`)

Consequences for a shared backend:

1. Admin-published steps would not match any frontend step id, so the guest wizard would ignore them except for the progress count; admin-created steps (`step_<ms>`) are never rendered because the prefix is not `custom_step_` .
2. The admin "individual max 2 pages" rule contradicts the frontend 12-step individual flow.
3. Field types need a canonical enum + mapping (multiselect/multi-select, file/image-upload, boolean/toggle; radio, location, amenities have no frontend renderer). `docs/DATA-MODEL.md` s5 documents the mapping.
4. Core vs custom fields: frontend skips `isCore` fields and `BUILT_IN_FIELD_IDS` , i.e. the config can only ADD custom fields to built-in steps; it cannot reorder/remove built-in steps. Admin UI implies it can.
5. Recommended contract: `GET /api/v1/onboarding-config/{hostType}` returns `{hostType, version, publishedAt, steps:[{id, stepNumber, title{en,fr,ar}, description, isActive, isCore, fields:[{id, type, label, required, order, options?, placeholder?, helpText?, defaultValue?}]}]}` with a per-host-type version; custom answers stored as JSON on the host application (`custom_fields`), validated server-side against the published version id the host started with.

---

6. Gaps and inconsistencies

6.1 Mock shortcuts that will not survive a real backend

#	Issue	Where
1	Atomicity relies on "no await" in single-threaded JS; needs DB transactions + row locks on wallet/booking/payout	`ledger.ts:53-57` , `finance.ts:315-343` , `bookings.ts:199-204`
2	No idempotency keys on recharge, refund, payout approval, dispute resolution	`finance.ts:205` , `bookings.ts:251,284`
3	Ids generated with `Date.now()` / `Math.random()` (collisions possible)	`audit.ts:30` , `ledger.ts:72-74` , `categories.ts:24` , `promotions.ts:24` , `team.ts:163`
4	Rate limiter in process memory (`globalThis`), lost on restart, not shared across instances	`rate-limit.ts:19-22`
5	Session not revocable server-side (logout only deletes cookie)	`auth.ts:116-130`
6	Audit actor/admin recorded by NAME in ledger (`adminActor`), payouts (`processedBy`), onboarding (`publishedBy`), disputes (`resolvedBy`), team (`invitedBy`); needs FK to staff id	`ledger.ts:65` , `finance.ts:339` , `onboarding.ts:33` , `bookings.ts:330` , `team.ts:171`
7	Denormalised names copied into bookings/reviews/transactions (hostName, guestName, listingTitle, hostType); will drift on rename	`types:421-466,474-497,507-523`
8	Counters stored, not derived: Host.listingsCount/bookingsCount/rating/reviewsCount, Listing.rating/reviewsCount/bookingsCount, GuestUser.bookingsCount/totalSpentDzd/walletBalanceDzd, CategoryItem.listingCount	`generator.ts:85-107,208-211,354-356` , `store.ts:45-47`
9	Seed ledger not linked to bookings (random refs); no booking lifecycle writes `booking_payment` / `commission_deduction`	`generator.ts:492-524`
10	Hardcoded figures in UI: sidebar badges 186/46/24/42/8 (`Sidebar.tsx:100,116,132,149,186` + reviews), bookings tab counts 1248/642/… (`BookingsClient.tsx:189-194`), dashboard fallbacks `	
11	Read functions without `assertCan` ; server pages call data directly: dashboard shows revenue KPIs and attention items to every role (`(panel)/page.tsx:15-21`)	
12	`updatePlatformSettings` accepts any partial object, shallow merge, no whitelist	`settings.ts:15-39`
13	`updateOnboardingFlow` accepts any steps payload	`onboarding.ts:15-56` , `actions/onboarding.ts:13-17`

#	Issue	Where
14	`createCategory`, `createPromotion` accept unvalidated objects	`actions/categories.ts:11`, `actions/promotions.ts:11`
15	Hard deletes for categories and coupons	`categories.ts:59-73`, `promotions.ts:60-74`
16	Files: every document/photo is `/assets/template.jpeg` or `/assets/house.png`; no upload, storage, virus scan, signed URLs	`generator.ts:162,312`
17	`Date.now()`-based seed timestamps: data changes on every boot	generator throughout
18	Errors thrown as plain `Error` (not DomainError) in users/reviews/team/categories/promotions/onboarding become `code: "failed"` instead of `conflict` / `invalid`	`users.ts:142`, `reviews.ts:90`, `team.ts:57,61,73,117`, `onboarding.ts:25`
19	Commission default change not propagated to non-overridden hosts; no "revert override"	`finance.ts:252-279`, `hosts.ts:332-364`
20	Payout `processed` state and bank/CCP confirmation missing; debit happens at approval	`finance.ts:311-356`
21	Cancel does not trigger refund or ledger reversal; refund never reverses commission	`bookings.ts:165-197,225-237`

6.2 Features with no persistence (mocked-only)

Messages/conversations/broadcast/templates (`MessagesClient.tsx`), change requests and trip booking requests (`ChangeRequestsClient.tsx`), create wizards for all 4 verticals (`StayWizardClient.tsx`, `ExperienceWizardClient.tsx`), featured placements (`PromotionsClient.tsx:45-49,347`), review reports/disputes tabs (`ReviewsClient.tsx:90`), review CSV export (`:225`), guest password reset and guest export (`UsersClient.tsx:287,567`), document approve/reject (`ApprovalsClient.tsx:281-284`), approvals "undo" (`:289`), provider "request changes" (`ProvidersValidationClient.tsx:383-388`), listing review checklist + internal notes (`StayReviewClient.tsx:84-105`), settings notification toggles and languages tab (`SettingsClient.tsx:129-133,283-330`), maintenance mode / 2FA / session timeout (stored, not enforced), training mode (localStorage only), car pricing tiers and insurance (stored as categories), amenities (redirect, no model), team invitations (no email, no acceptance).

6.3 Admin vs guest-site model mismatches (`frontend/types.ts`)

Concept	Admin	Frontend	Note
Users	`GuestUser` (`usr_001`) and `Host` (`hst_001`) are separate tables with no link	single `User` with `role guest|host|admin`, `hostType`, `organizationId` (`frontend/types.ts:4-24`)	backend needs one `users` table + host profile; admin `Host.organizationId` / `Organization.ownerUserId` never seeded
Organization	`Organization.verificationStatus: boolean`, `hostType`	`verificationStatus: pending|verified|rejected`, `businessCategory agency|hotel|hostel`, `businessRegistrationNumber`, `businessEmail/Phone`, hotel/hostel policy fields (`:292-326`)	admin flattens org into Host + `agencyMetadata/hotelMetadata/hostelMetadata`
Listing status	8 canonical + 3 aliases	`active|pending|suspended|draft` (`:54`)	admin `approved` = frontend `active`; `changes_requested`, `rejected`, `archived`, `deleted` invisible on frontend
Listing fields	`propertyType`, `hostId/hostName/hostType`, `wilayaCode`, `coordinates{lat,lng}`, `guestCapacity`, `isFeatured`, `reviewsCount`, `rooms`, `houseRules` object, `bookingApprovalMode`, duplicates `vertical/priceDzd/images`	`type`, `ownerType` + `hostUserId` / `organizationId`, `locationValue` / `commune` / `country`, `latlng [lat,lng]`, `guestCount`, `featured`, `reviewCount`, `roomTypes`, `houseRules string[]`, `instantBook` + `bookingApprovalMode`, plus `cleaningFee`, `cancellationPolicy`, `minNights/maxNights`, `serviceFeeRate`, `taxRate`, vehicle and experience fields (`:110-196`)	admin cannot see or moderate cleaning fee, cancellation policy, stay rules, vehicle/experience specifics
Trips	listing with `category: "trip"`	separate `Trip` entity (`trip_*` ids) with itinerary, departures, `organizationId` (`:261-290`)	admin has no trip itinerary model
Wilaya	string code `"01".."68"`, **68** entries (`lib/utils/wilayas.ts`, `{ code:` count = 68); README and UI strings say **69** (`messages/en.json:291,747,894,895,1417`, `HostsClient.tsx:905`)	numeric code 1..58, **58** entries (`frontend/lib/algeria.ts`, header comment), footer "58 wilayas" (`frontend/messages/en/footer.json:32`); wilaya derived from `commune`	three different counts (58 / 68 / 69); codes 59-68 in admin (Aflou, Barika, ...) do not exist on the frontend; string vs number type
Booking	`Booking`, ref `BLD-2026-NNNN`, `checkInDate/checkOutDate`, `basePriceDzd`, `totalAmountDzd`, `refundedAmountDzd`, statuses incl. `disputed`, `refunded`	`Reservation` (`res_*`), `startDate/endDate`, `subtotal`, `totalPrice`, `cleaningFee`, `taxes`, `refundAmount`, `cancelledBy`, `expiresAt`, `declineReason`, `selectedRoomTypeId`, `unitCount`, statuses incl. `declined`, `expired` (`:198-250`)	admin ignores cleaning fee in payout math; frontend payout = subtotal + cleaning - commission - refund (`frontend/lib/booking/payout.ts:17-34`), admin payout = base - commission
Payment status	`paid|partially_refunded|refunded|pending|failed`	`paid|pending|refunded|partially_refunded|voided`	
Payment method	`cib_edahabia|ccp_transfer|baridimob|cash_on_arrival`	free string	
Wallet model	per host (`wlt_<hostId>`), mixes guest-money settlement (`booking_payment` credits, `host_payout` withdrawals, refunds)	per owner `ownerType user|organization`, PREPAID COMMISSION wallet: `recharge`, `commission_deduction`, `payout`, `adjustment` (`:574-603`, `frontend/lib/data/wallet.ts`)	fundamentally different money flow; must be decided before modelling the ledger
Transaction	`amount`, `balanceAfter`, `reference`, `bookingRef`, `adminActor` name	`amount`, `currency`, `relatedBookingId`, `adminUserId`, `note`, `timestamp`	
Commission	on base price (`lib/pricing.ts:84`)	on accommodation subtotal (excl. cleaning) (`frontend/lib/booking/config.ts:10-19`)	same rates; frontend also stores live rates in its own server store (`frontend/lib/data/wallet.ts:250-266`), a second source of truth
Review	`ReviewItem` with status/flags/hostReply, single rating	`Review` with 5 sub-ratings (cleanliness, accuracy, communication, location, value), no status (`:71-88`)	
Coupon	`PromotionCoupon` (`discountType percentage|fixed_dzd|fixed`, `maxUsage/maxUses`, `usedCount/currentUses`, `startDate/endDate/expiresAt`, `status`, `scope`)	`Coupon` (`percentage|fixed`, `validFrom/validTo`, `usageLimit/usageCount`, `verticalScope`, `isActive`) (`:474-486`)	
Category	`name{en,fr,ar}`, `iconName`, `listingCount`, `isActive`, `order`	`labelKey`, `icon`, `descriptionKey` (i18n keys) (`:63-69`)	
Amenity	`name{en,fr,ar}`, free `category`	`nameKey`, `category essentials|features|location|safety`	admin listings store amenities as free strings ("Fast WiFi")

Concept	Admin	Frontend	Note
Audit	`actorId/actorName/actorRole/entityType/entityId/before/after/ipAddress`	`AuditLogEntry adminId/adminName/targetType/targetId` (`:457-472`)	frontend has its own admin-ish audit type
Ids	sequential `usr_001`, `hst_001`, `lst_001`, `bkg_0001`, `rev_001`, `tx_0001`, `pay_001`	slug-like seeds (`lst_tipaza_villa`, `res_tipaza_01`, `usr_amina...`) and `prefix_<uuid>` at runtime (`frontend/lib/ids.ts:6-8`)	pick UUIDs (or ULIDs) + human references
Locale default	`i18n.ts` default `en`, root redirects `/en`; platformSettings.defaultLocale `fr`		
Brand in identifiers	cookie `darna_admin_session`, JWT iss `urn:darna:admin`, CSV `darna_*`, localStorage `darna_admin_*`	`darna_onboarding_config_*`	intentional per project memory, but worth listing for the backend

6.4 Other inconsistencies inside admin

#	Issue	Where
1	Settings field mismatch: UI writes `serviceFeePercent` and `security.{sessionTimeoutMinutes,twoFactorRequired}`, store/type primary fields are `guestServiceFeePercent`, `sessionTimeoutMinutes`, `twoFactorAuthEnabled`; UI always shows fee 0 and timeout default 120	`SettingsClient.tsx:141-172`, `store.ts:92-106`, `types:682-701`
2	Two transition tables (`lib/status.ts` vs `lib/domain/transitions.ts`)	`lib/status.ts:58-80`
3	Command palette and dashboard attention links to non-existent routes `/listings/{id}`, `/bookings/{id}`, `/users/{id}` (404 via `[...rest]`)	`command-palette.ts:56,76,92`, `analytics.ts:190,198`
4	Sidebar `/bookings?type=stay` ignored (client reads `vertical`); `/cars/rentals?status=in_progress` not a BookingStatus	`Sidebar.tsx:150-153,182-185`, `BookingsClient.tsx:73-74`
5	`/x/[id]` detail pages do not check vertical	`cars/[id]/page.tsx:18` etc.
6	Trips and cars "new" pages reuse the experience wizard	`trips/new/page.tsx:2`, `cars/new/page.tsx:2`
7	Document approval approves the whole host	`ApprovalsClient.tsx:281-284`
8	Category manager per vertical shows all verticals	`CategoriesManagerClient.tsx:58-67`
9	StatsClient provider type `carRental` is not a HostType	`StatsClient.tsx:61-66,73-77`
10	`listHosts` declares `minPrice/maxPrice` filters but ignores them	`hosts.ts:22-23`
11	Approvals queue ignores legacy `pending` listings; listing waiting time uses `createdAt`	`approvals.ts:33,35`
12	Many unused types (Document, ModerationChecklist, ChangeRequest, WizardDraft, Amenity, Insurance, PricingTier, Organization, QueryParams, UserRole)	`lib/types/index.ts`
13	`docs/DATA-MODEL.md` says 7% service fee; README says "Supabase Ready" while target is Django	`docs/DATA-MODEL.md` §6, `README.md` §4
14	README "Every mutation executes `assertCan(session, permission)`": false for `updateUserStatus`	`users.ts:282-308`
15	Messages area (incl. broadcast to all users) needs no permission	`messages/*/page.tsx` `guardPage(null)`

---

7. Page routes (80 `page.tsx` under `app/[locale]` + `app/page.tsx`)

All panel pages also pass through `middleware.ts` (JWT) and `(panel)/layout.tsx` (store-backed admin check + approvals count via `getApprovalsInbox` when `hosts.view`). "Data" lists server-side data calls and client actions.

Route	Purpose	Data / actions	guardPage
`/` (app/page.tsx)	redirect to `/en`	none	n/a
`/[locale]`	Dashboard	data: getDashboardKPIs, getDashboardCharts, getAttentionItems, listAuditLogs (if audit.view)	null
`/login`	Sign-in	data: listDevDemoAccounts, getCurrentAdmin; actions: loginAction, getDemoCredentials	public (redirects if signed in)
`/approvals`	Moderation inbox, 5 queues	actions: getApprovalsInbox, approveHost, rejectHost, approveListing, rejectListing, hideReview, dismissFlag, resolveDispute	hosts.view
`/audit-log`	legacy redirect -> `/settings/audit-log`	none	none (target guarded)
`/bookings`	Bookings list, cancel, refund, dispute, export	actions: listBookings, cancelBooking, refundBooking, resolveDispute, exportBookings	bookings.view
`/cars`	Vehicle listings list	actions: listListings + useListingStatusActions (updateListingStatus, deleteListing, restoreListing, exportListings)	listings.view
`/cars/[id]`	Listing detail (shared StayDetailClient)	data: getListingById; actions: approve/reject/requestChanges/restore/updateListingStatus	listings.view
`/cars/[id]/review`	Review wizard (shared StayReviewClient)	data: getListingById; actions: approveListing, rejectListing, requestChangesListing	listings.view
`/cars/categories`	Vehicle categories	actions: listCategories, createCategory, toggleCategoryStatus	categories.manage
`/cars/insurance`	"Insurance" (category manager reuse)	same as above	listings.view
`/cars/new`	Add car (experience wizard reuse), mocked	none	listings.approve
`/cars/owners`	redirect `/moderation/providers?category=rental_agency&status=all`	none	hosts.view
`/cars/pricing`	"Pricing tiers" (category manager reuse)	categories actions	listings.view
`/cars/rentals`	redirect `/bookings?vertical=cars[&status=]`	none	bookings.view
`/categories`	Global category management	actions: listCategories, createCategory, toggleCategoryStatus, deleteCategory	categories.manage

Route	Purpose	Data / actions	guardPage
`/experiences`	Experience listings	listListings, approveListing, updateListingStatus + useListingStatusActions	listings.view
`/experiences/[id]`	Detail	getListingById + listing actions	listings.view
`/experiences/[id]/review`	Review wizard	getListingById + decisions	listings.view
`/experiences/amenities`	redirect `/experiences/categories`	none	categories.manage
`/experiences/categories`	Experience categories	categories actions	categories.manage
`/experiences/change-requests`	Change requests, mocked	none	listings.approve
`/experiences/new`	Add experience wizard, mocked	none	listings.approve
`/finance`	legacy redirect -> `/payments`	none	none
`/help`	Help center (static)	none	null
`/hosts`	Hosts directory	data: listHosts (SSR); actions: listHosts, approveHost, suspendHost, unsuspendHost, deleteHost, restoreHost, overrideCommission, rechargeWallet, exportHosts	hosts.view
`/hosts/[id]`	Host profile (server rendered)	data: getHostById	hosts.view
`/listings`	All listings	data: listListings (SSR); actions: listListings, approveListing, requestChangesListing, featureListing, deleteListing, exportListings + useListingStatusActions	listings.view
`/messages`	Conversations, mocked	none	null
`/messages/notifications`	Sent notifications, mocked	none	null
`/messages/templates`	Templates, mocked	none	null
`/moderation`	redirect `/moderation/providers`	none	hosts.view
`/moderation/providers`	Provider (host) validation	actions: listHosts, approveHost, rejectHost, unsuspendHost	hosts.view
`/network`	Network explorer	actions: searchNetwork, approveListing, suspendListing	listings.view
`/onboarding`	legacy redirect -> `/settings/onboarding`	none	none
`/payments`	Finance overview tab	actions: getFinanceSummary, listWallets, listTransactions, listPayouts, listRefunds, getCommissionRates, rechargeWallet, approvePayout, rejectPayout, updateCommissionRates, exportTransactions, exportWallets	wallets.view
`/payments/commissions`	Commission defaults tab	same	wallets.view (save needs commissions.manage)
`/payments/payouts`	Payout requests tab	same	wallets.view
`/payments/refunds`	Refunds tab	same	wallets.view
`/payments/transactions`	Ledger tab	same	wallets.view
`/payments/wallets`	Wallets tab	same	wallets.view
`/promotions`	Coupons	actions: listPromotions, createPromotion, togglePromotionStatus, deletePromotion	promotions.manage
`/promotions/featured`	Featured placements, mocked	same client, static data	promotions.manage
`/providers`	legacy redirect -> `/moderation/providers`	none	none
`/reports`	Reports tab of stats	actions: getDashboardKPIs, getDashboardCharts, exportKpiReport	reports.export
`/reviews`	Reviews moderation	actions: listReviews, hideReview, restoreReview, dismissFlag, deleteReview	reviews.view
`/reviews/disputes`	"Disputes" tab (unfiltered reviews)	same	reviews.view
`/reviews/reports`	"User reports" tab (unfiltered reviews)	same	reviews.view
`/search`	Search results (`?q=`)	actions: searchNetwork	listings.view
`/settings`	General settings	actions: getPlatformSettings, updatePlatformSettings	settings.manage
`/settings/audit-log`	Audit trail	actions: listAuditLogs, exportAuditLogs	audit.view
`/settings/commissions`	Commission defaults (PaymentsClient)	finance actions	commissions.manage
`/settings/languages`	Languages tab (static)	settings actions	settings.manage
`/settings/notifications`	Notification prefs (not saved)	settings actions	settings.manage
`/settings/onboarding`	Onboarding configurator	actions: getOnboardingConfig, updateOnboardingFlow	onboarding.edit
`/settings/security`	Session timeout, 2FA flag (not enforced)	settings actions	settings.manage
`/settings/team`	Staff and roles	actions: listTeamMembers, updateMemberRole, toggleMemberActive, inviteMember	team.manage
`/stats`	Stats overview (mostly hardcoded)	analytics actions	reports.export
`/stats/providers`	By provider type	analytics actions	reports.export
`/stats/reports`	Report exports	analytics actions + exportKpiReport	reports.export
`/stats/revenue`	Revenue	analytics actions	reports.export
`/stats/services`	By vertical	analytics actions	reports.export
`/stats/wilayas`	By wilaya (hardcoded 4 rows)	analytics actions	reports.export
`/stays`	Stay listings	listListings, approveListing, updateListingStatus + useListingStatusActions	listings.view

Route	Purpose	Data / actions	guardPage
`/stays/[id]`	Detail	getListingById + listing actions	listings.view
`/stays/[id]/review`	Review wizard	getListingById + decisions	listings.view
`/stays/amenities`	redirect `/stays/categories`	none	categories.manage
`/stays/categories`	Stay categories	categories actions	categories.manage
`/stays/change-requests`	Change requests, mocked	none	listings.approve
`/stays/new`	Add stay wizard, mocked	none	listings.approve
`/team`	legacy redirect -> `/settings/team`	none	none
`/trips`	Trip listings	listListings, approveListing, updateListingStatus + useListingStatusActions	listings.view
`/trips/[id]`	Detail	getListingById + listing actions	listings.view
`/trips/[id]/review`	Review wizard	getListingById + decisions	listings.view
`/trips/booking-requests`	"Booking requests" (change-requests mock)	none	bookings.view
`/trips/categories`	Trip categories	categories actions	categories.manage
`/trips/new`	Add trip (experience wizard reuse), mocked	none	listings.approve
`/users`	Users (guests + both)	data: listUsers (SSR); actions: listUsers, suspendUser, banUser, deleteUser, updateUserStatus	users.view
`/users/guests`	Users filtered `role=guest`	same	users.view
`/where-did-it-go`	Old-to-new navigation map (static)	none	none (layout auth only)
`/[...rest]`	Localised 404	none	n/a

Global (every panel page): `CommandPalette` -> `searchGlobalCommandPalette`; `Header` -> `logoutAction`; `AdminSessionProvider.useCan` hides buttons by permission.

BLADUS: spec vs code (raw analysis for the backend blueprint)

Working note, 2026-11-04. Read-only comparison of the recovered specs with the current code in `frontend/` (guest + host app) and `admin/` (back-office). Paths are relative to the repo root `C:\Users\OASIS\Documents\darna\`. Line numbers were checked against the working tree on this date.

Status legend: **implemented** (behaviour present end to end on mock data), **partial** (present but incomplete or only in one app), **missing**, **different** (built, but with a rule that contradicts the spec).

0. Source notes (read first)

Source	What it actually contains	Caveat
`docs/backend/_sources/darna_spec.md`	The real spec: sections 0 to 10 plus Rules (lines 1-179).	Lines 180-692 are not spec: they are a pasted IDE context dump (an unrelated `senior-frontend` SKILL.md). Ignore them.
`docs/backend/_sources/clean_user_spec.md`	Same text as `darna_spec.md` lines 1-179, prefixed by "using this skill @[...]".	Not an independent source. Cited below only as "same as darna_spec".
`docs/backend/_sources/usability-report.md` and `admin-usability-report.md`	Byte-identical (`cmp` returns no difference). Both are the **admin** usability report; each file also repeats its own content twice (lines 1-127 and 128-254).	No guest/host usability report was recovered.
`frontend/DARNA-AUDIT-REPORT.md` (2026-10-28)	Security/correctness audit.	Many findings are now fixed (server actions, signed session, availability engine). Treat its line numbers as stale.
`frontend/AUDIT.md` (2026-10-16)	Older audit.	Mostly superseded; refers to an in-frontend `/admin` that no longer exists.
`admin/docs/DATA-MODEL.md`	Admin canonical enums, transitions, money rules.	Section 6 still says guest service fee 7 % and frontend 10 %/8 %; both apps now use 0 (see section 3 below).
`frontend/README.md`, `admin/README.md`	Current architecture notes.	`admin/README.md` section 6 describes an individual onboarding flow capped at 2 steps, which contradicts the spec (see D-14).

Internal numbering bug in the spec: section 1 cross-references are off by one or two ("Section 8" = wallet, really 9; "Section 7" = analytics, really 8; "Section 6's step-editor", really 7; "Section 4 in my numbering" = search fix, really 6). This document uses the real headings.

---

1. Requirements extracted from the specs, with code status

Spec file is `darna_spec.md` (abbreviated **DS**) unless stated. `clean_user_spec.md` carries the same requirements at the same line numbers +0.

1.1 Host types and organizations (DS section 2.1, 2.2)

#	Spec ref	Requirement	Status	Evidence in code
R1	DS 2 (l.28)	Host-type selection offers 4 single-select cards: Individual, Agency, Hotel, Hostel (icon, label, one-line description).	implemented	`frontend/components/host/onboarding/HostTypeStep.tsx:18,40-47`
R2	DS 2.1 (l.31)	`hostType = "individual" | "agency" | "hotel" | "hostel"`.	implemented	`frontend/types.ts:2`; `admin/lib/types/index.ts:235,258`
R3	DS 2.1 (l.32)	Hotel and Hostel reuse `Organization` / `Membership`; `Organization.businessCategory = "agency" | "hotel" | "hostel"`.	implemented (frontend), different (admin)	`frontend/types.ts:297-326,361-376`; `frontend/lib/host-types.ts:5`. Admin models a single `Host` row (with optional `organizationId`) instead: `admin/lib/types/index.ts:255-280`
R4	DS 2.1 (l.33)	`RoomType` child of a property `Listing`, with own price, capacity, amenities, photos, inventory count and availability; search shows one card per property.	implemented	`frontend/types.ts:92-108,149`; per-room inventory in `frontend/lib/data/reservations.ts:167-210`; search card `frontend/components/listing/ListingCard.tsx:165-194`
R5	DS 2.1 (l.34)	`RoomType.bookingUnit = "room" | "bed"` for per-bed hostel dorms.	implemented	`frontend/types.ts:90,99`; bed = one per guest `frontend/app/actions/bookings.ts:115-119`; pricing `frontend/lib/booking/pricing.ts:212-231`
R6	DS 2.2 (l.37)	Individual goes straight into the listing-creation flow.	implemented	`frontend/app/actions/host.ts:74-100`
R7	DS 2.2 (l.38)	Agency flow: basics, details, team size, invites, review (5 steps).	implemented	`frontend/components/host/BecomeHostFlow.tsx:103,1706-1862`
R8	DS 2.2 (l.39)	Hotel flow: property name, optional star rating, room count, check-in/out policy, hotel amenities, business details, team invites; creates Org `businessCategory: "hotel"`, redirects to workspace.	implemented	`BecomeHostFlow.tsx:101,709-790,1864-2090`; server `frontend/app/actions/host.ts:102-168`; redirect `BecomeHostFlow.tsx:553,583`
R9	DS 2.2 (l.40)	Hostel flow: dorm/private mix, shared facilities, optional age and curfew policy; Org `businessCategory: "hostel"`.	implemented	`BecomeHostFlow.tsx:102,854-872,2320-2340`; `frontend/types.ts:322-324`
R10	DS 2.2 (implied)	New organizations go through verification before trust.	partial	Org starts `pending` (`frontend/app/actions/host.ts:60-64`), but its listings are created `status: "active"` immediately (`frontend/lib/data/listings.ts:309`). Admin has a full host lifecycle (`admin/docs/DATA-MODEL.md` section 2) that the frontend never reads.
R11	DS 1 (l.17)	Remove every place that reads `hostType` as binary individual/agency.	partial	Fixed for commission owner (`frontend/lib/data/reservations.ts:370-371`) and host layout. Remaining binaries: trips only for `hostType === "agency"` (correct by spec, `frontend/app/actions/trips.ts:73`); org pages still live under `/host/agency/*` for hotels and hostels (`frontend/components/host/HostNavbar.tsx:84-85`).
R12	(not in spec, but forced by data)	A user can hold several roles (Karim is individual host, agency owner and hotel owner per `frontend/README.md`).	different	`User.hostType` is a single value (`frontend/types.ts:10`); active org chosen per session (`frontend/lib/server/session.ts:68-75`, `frontend/lib/server/active-org.ts:15`). Karim also has an individual wallet `frontend/lib/data/wallet.ts` (`wal_karim`). See D-1.

1.2 Hotel/Hostel workspace (DS 2.3)

#	Spec ref	Requirement	Status	Evidence
R13	DS 2.3 (I.43)	Hotel/Hostel get the full Host Mode (Today, Bookings, Calendar, Listings, Messages, Team, Settings) with org identity.	implemented	`frontend/components/host/HostNavbar.tsx:59-62,82-97` ; layout guard `frontend/app/[locale]/host/layout.tsx:25-28`
R14	DS 2.3 (I.44)	Listings page is a property + room-type manager with per-room status/price/availability quick-edit.	partial	Room-type manager modal `frontend/components/host/HostListingsClient.tsx:103-118,207-209` . No per-room status field on `RoomType` (`frontend/types.ts:92-108` , only `isAvailable?`).
R15	DS 2.3 (I.45)	Calendar drills into per-room-type (and per-bed) availability.	partial	Room-type selector `frontend/components/host/HostCalendarClient.tsx:139,1108-1120` . Blocks and price overrides are per listing, not per room type (`frontend/app/actions/calendar.ts:40,60` ; `frontend/lib/data/reservations.ts:232-234`).
R16	DS 2.3 (I.46)	Public card shows "multiple room types, from lowest price"; detail page booking widget is a room-type picker.	implemented	`ListingCard.tsx:165-166,188-194` ; `frontend/components/listing/HotelHostelDetailClient.tsx:113-116`

1.3 Booking approval mode (DS 3)

#	Spec ref	Requirement	Status	Evidence
R17	DS 3 (I.52)	Explicit "How do you want to handle booking requests?" step in **all four** onboarding flows.	partial	Individual: `BecomeHostFlow.tsx:1600-1624` (toggle + `approval_mode` config step). Hotel: step 5 `BecomeHostFlow.tsx:2072-2080` . Hostel: step 5 `:2320-2328` . **Agency: missing in the UI**; the config defines `agency_approval` (`frontend/lib/onboarding-config.ts:559-575`) but the hardcoded agency flow never renders it and `handleRegisterAgency` sends no mode (`BecomeHostFlow.tsx:690-703`), so the server defaults to `"instant"` (`frontend/app/actions/host.ts:151`).
R18	DS 3 (I.55)	Instant Book = auto-approved "the moment a guest completes payment".	different	No payment step exists; instant bookings are written `confirmed` + `paymentStatus: "paid"` directly (`frontend/lib/data/reservations.ts:273,291`).
R19	DS 3 (I.56)	Request to Book: host, or for orgs "any team member with permission", approves/declines "within a reasonable window".	partial	Any active member of any role can approve (`frontend/lib/server/authz.ts:25-35` , `frontend/lib/host-types.ts:19` `canManageOperations` = any role). Window = 24 h (`frontend/lib/booking/config.ts:55`), enforced lazily on read (`frontend/lib/booking/lifecycle.ts:12-16,34` ; `reservations.ts:51-62`).
R20	DS 3 (I.57)	`bookingApprovalMode` stored on `User` (individual) or `Organization` (org) as the **default for new listings**.	different	Fields exist (`frontend/types.ts:12,325`). But the individual's User value is **derived from** the first listing (`frontend/app/actions/host.ts:92-95`), and neither User nor Org value is applied to listings created later (`frontend/lib/data/listings.ts:280-322` defaults `instantBook: true`). At booking time only the listing is read (`frontend/lib/booking/pricing.ts:276-282` , `frontend/app/actions/bookings.ts:155`). Trips read the org value (`bookings.ts:190-193`).
R21	DS 3 (I.57)	Optional per-listing override ("state whether you did").	implemented (de facto)	Per-listing `instantBook` toggle `frontend/components/host/HostListingsClient.tsx:171,191,670` ; `Listing.bookingApprovalMode` + `instantBook` both exist (`frontend/types.ts:150` and `instantBook?`). Two fields encode one fact (see D-6).
R22	DS 3 (I.58)	Request mode creates `pending` and surfaces on Today and Bookings with Accept/Decline.	implemented	`reservations.ts:273` ; action `frontend/app/actions/bookings.ts:253-306` ; UI `frontend/components/host/HostTodayClient.tsx` , `HostBookingsClient.tsx`

1.4 Solo (individual) host interface (DS 4)

#	Spec ref	Requirement	Status	Evidence
R23	DS 4.1 (I.67)	Page 1 "My listings" = landing page; thumbnail, title, Live/Paused, price, plain-language upcoming count; tap opens the creation form for edit; "+ Add a new place".	partial	Landing redirect `frontend/app/[locale]/host/page.tsx:20-21` . Edit opens a **modal**, not the creation step form (`HostListingsClient.tsx:171-191,670`).
R24	DS 4.1 (I.68)	Page 2 "Bookings": chronological list, Accept/Decline on card; no Today, Calendar, KPI or analytics pages for solo.	partial	Solo nav = My listings + Bookings (`HostNavbar.tsx:54-56`). `/host/calendar` is not guarded for individuals (`frontend/app/[locale]/host/calendar/page.tsx:14-31`). Mobile bottom nav has 3 tabs incl. Messages (`HostBottomNav.tsx:48-51`).
R25	DS 4.1 (I.69)	Optional read-only month mini-calendar at the top of Bookings.	implemented	`HostBookingsClient.tsx:98,110-111,413-429`
R26	DS 4.1 (I.70)	Messages through the shared Messages system, not a third page.	implemented	`frontend/app/[locale]/host/messages/page.tsx` (shared component)
R27	DS 4.2 (I.74)	Solo settings (payout method, approval toggle) in one simple section reachable from My listings.	partial	Wallet widget on listings page (`HostListingsClient.tsx:28`); approval toggle lives on a separate `/host/settings` page (`HostSettingsClient.tsx:99-100,158,373`) and does not change existing listings (see R20).
R28	DS 4.3 (I.78)	"Switch to hosting" routes solo to 2-page UI, orgs to full UI, based on hostType/businessCategory.	implemented	`frontend/lib/server/host-context.ts:26-36,57` ; `frontend/middleware.ts:79`

1.5 Trips (agency-only multi-day product) and departures (DS 5)

#	Spec ref	Requirement	Status	Evidence
R29	DS 5.1 (I.87)	`Trip` : title, cover + gallery, day-by-day itinerary, duration, group size limits, per-person price, inclusions/exclusions, meeting point, cancellation policy, `organizationId` of an Agency.	implemented	`frontend/types.ts:252-293`
R30	DS 5.1 (I.87)	Enforce agency-only at creation; hotels/hostels do not see the option.	implemented	`frontend/app/actions/trips.ts:73-78,97,110` ; nav `HostNavbar.tsx:82,94,117`
R31	DS 5.2 (I.90)	6-step creation: basics, itinerary, logistics, inclusions, **cancellation policy**, review & publish.	partial	5 steps (`frontend/components/host/agency/TripCreationClient.tsx:56-58`); cancellation policy not a step of its own.
R32	DS 5.2 (I.91)	Trips list with view/edit/pause/delete; publish = live immediately.	implemented	`frontend/components/host/agency/AgencyTripsClient.tsx:8,90,108` . "Pause" = `status: "draft"` (`frontend/types.ts:288`). No moderation (admin moderates trips as listings, see D-9).
R33	DS 5.3 (I.94)	"Trips" as the **4th** segment of the hero switcher (Homes / Experiences / Vehicles / Trips).	different	5 segments: Homes, Hotels, Experiences, Vehicles, Trips (`frontend/components/home/HomeHero.tsx:33-41`).

#	Spec ref	Requirement	Status	Evidence
R34	DS 5.3 (l.95-96)	Trips feed and detail page (itinerary, agency card, group size + per-person widget).	implemented	`frontend/app/[locale]/page.tsx:60-68,96` ; `frontend/components/trips/TripDetailClient.tsx`
R35	DS 5.3 (l.97)	Booking a Trip creates a reservation against the Trip, respects the agency approval mode, shows in guest reservations and agency Bookings.	implemented	`frontend/app/actions/bookings.ts:163-199` ; `frontend/lib/data/reservations.ts:340-355` ; owner scoping `:75-92`
R36	(implied by "bookable" + group size)	Departures with seat capacity.	partial	`Trip.departureDates: string[]` only (`frontend/types.ts:286`); capacity = `maxGroupSize` per date (`reservations.ts:243-251,346-348`). No departure entity (no per-departure price, status, guide, min-group go/no-go).
R37	(naming)	Guest "Trips" page.	different	`/trips` = guest's own reservations (`frontend/app/[locale]/trips/page.tsx:6,29`) while `/trips/[tripId]` = agency Trip product. Same word, two resources (D-11).

1.6 Search and date overlap (DS 6)

#	Spec ref	Requirement	Status	Evidence
R38	DS 6 (l.103)	Listing appears when its availability window **intersects** the searched window (not containment).	implemented	`frontend/lib/data/listings.ts:147-183`
R39	DS 6 (l.105)	Partial matches show "Available X-Y within your dates" on card and detail.	implemented	Card `ListingCard.tsx:354-361` ; detail `frontend/components/listing/ListingReservation.tsx:78-101`
R40	DS 6 (l.105)	Booking widget only allows the true sub-range; server must not allow booking outside availability.	implemented	Client clamp `ListingReservation.tsx:71-106` ; server `frontend/lib/booking/availability.ts:88-120` (`OUTSIDE_WINDOW` , `DATES_UNAVAILABLE`)
R41	DS 6 (l.103)	Same logic for hotel/hostel room types.	partial	Property kept if at least one room type is free on every night of the overlap (`reservations.ts:226-241`); no per-room-type partial badge.
R42	DS 6 (l.106)	Same logic for Vehicles and Experiences.	implemented	They are `Listing` rows and go through the same filter (`listings.ts:147-213`). Experiences are single-date; quote uses start only (`:200`).
R43	DS 6 (l.106)	Same logic for **Trips**.	missing	`getTrips` has no date parameter (`frontend/lib/data/trips.ts:11-59`).
R44	(audit)	Exclude booked/blocked nights, back-to-back allowed, timezone-safe date keys.	implemented	`listings.ts:186-192` ; `frontend/lib/booking/dates.ts`

1.7 Admin onboarding-step editor (DS 7)

#	Spec ref	Requirement	Status	Evidence
R45	DS 7.1 (l.115)	`OnboardingStepConfig` per hostType: ordered steps, id, title, fields (id, label per locale, type text/number/select/multi-select/image-upload/toggle, required, options), active flag.	different	Two incompatible schemas: frontend `frontend/types.ts:507-572` (spec field names) vs admin `admin/lib/types/index.ts:602-650` (`multiselect` , `file` , `boolean` , `radio` , `location` , `amenities` ; `name` / `title` , `order` / `stepNumber` , `key` / `id`).
R46	DS 7.1 (l.116)	Seed from real flows: Individual 12 steps, Agency 5, Hotel and Hostel equivalents.	partial	Frontend seed matches (individual 12 ids, agency 5, hotel 7, hostel 7: `frontend/lib/onboarding-config.ts:7-1045`). Admin seed has **2** individual steps and a hard cap (`admin/lib/mock-data/store.ts:108-140`).
R47	DS 7.2 (l.119)	Admin UI: tab per host type.	implemented	`admin/app/[locale]/(panel)/settings/onboarding/OnboardingClient.tsx` (canonical route; `/onboarding` redirects, `admin/app/[locale]/(panel)/onboarding/page.tsx`)
R48	DS 7.2 (l.120)	Reorder steps; add/remove/reorder/edit **fields** (label, type, required); add/remove steps.	partial	Steps: move up/down, add, remove, toggle required (`OnboardingClient.tsx:95-154`). Fields: count only, no field editor (`:329`).
R49	DS 7.2 (l.121)	Live onboarding flows **read** the admin config (config-driven renderer).	missing	Frontend reads its own defaults or `localStorage` key `darna_onboarding_config_*` (`frontend/lib/onboarding-config.ts:1064-1091`); admin publishes to its own in-memory store (`admin/lib/data/onboarding.ts:15-56`). Only non-core extra fields of the individual flow are rendered dynamically (`BecomeHostFlow.tsx:192-205`) and their values are **never submitted** (`customFieldsData` only goes to the local draft, `:175,423,506`).
R50	DS 7.2 (l.122)	Guard removal of core steps/fields with confirmation; warn on zero steps or removing "price".	missing	`handleRemoveStep` removes with no dialog (`OnboardingClient.tsx:150-154`); server only checks the solo cap (`admin/lib/data/onboarding.ts:23-26`). `isCore` flags exist but are not enforced.
R51	(admin)	Versioned publish with audit.	implemented	`admin/lib/data/onboarding.ts:28-53`

1.8 Admin analytics (DS 8)

#	Spec ref	Requirement	Status	Evidence
R52	DS 8 (l.130)	Every major KPI (revenue, bookings, active listings, avg booking value, growth) filterable by host type, plus 4-way comparison.	partial	Per-host-type table and revenue series `admin/lib/data/analytics.ts:69-120` . Growth rates are hardcoded strings (`:43-50`); headline KPIs are not filterable by type.
R53	DS 8 (l.131)	Same KPIs by vertical (Homes / Vehicles / Experiences / Trips).	partial	"revenueByVertical" counts **listings**, not revenue (`analytics.ts:123-130`); 5 verticals incl. Hotels.
R54	DS 8 (l.132)	Cross-tab host type x vertical, impossible cells shown as a placeholder, not 0.	missing	No matrix/heatmap in `admin/app` or `admin/lib` (grep).
R55	DS 8 (l.134)	New KPIs: total commission, avg commission rate by type, host growth by type, Trips metrics (bookings, revenue, avg group size), Hotel/Hostel occupancy and avg nightly rate.	partial	Commission total and effective rate per type present (`analytics.ts:20-22,88-95`). Occupancy is a constant 78.4 (`:41`); no group size or nightly-rate metrics; period filter is a multiplier, not a date window (`:14`).
R56	DS 8 (l.135)	Organised as Overview / By Host Type / By Vertical / Financial tabs.	different	Tabs are overview / providers / services / wilayas / revenue / reports (`admin/app/[locale]/(panel)/stats/StatsClient.tsx:36,130-160`).

1.9 Wallet, commission, payouts, ledger (DS 9)

#	Spec ref	Requirement	Status	Evidence
R57	DS 9.1 (l.144)	`Wallet` per User (individual) or Organization, DZD.	implemented (frontend), different (admin)	`frontend/types.ts:592-603`; admin wallets keyed by `hostId` (`admin/lib/types/index.ts:526-543`).
R58	DS 9.1 (l.145)	`Transaction`: type recharge / commission_deduction / payout (+ others), signed amount, `relatedBookingId`, `adminUserId`, note, timestamp.	different	Frontend: `recharge`, `commission_deduction`, `payout`, `adjustment` (`frontend/types.ts:574-590`). Admin: `wallet_recharge`, `booking_payment`, `commission_deduction`, `host_payout`, `booking_refund`, `adjustment`, with `balanceAfter`, `reference`, `receiptReference`, `adminActor` name (`admin/lib/types/index.ts:499-520`).
R59	DS 9.1 (l.145)	Append-only ledger; balance derived from the sum.	partial	Frontend recomputes on each append (`frontend/lib/data/wallet.ts:243-245,370-380`) but seed rows carry a literal `balance` (consistent today: e.g. `wal_amina` 50000-8400-9600 = 32000). Admin derives on every read (`admin/lib/data/ledger.ts:21-56`). Neither is a real append-only store.
R60	DS 9.2 (l.148)	Admin "Recharge balance" on host/org detail: amount + optional note, confirm dialog, creates recharge.	different	`admin/lib/data/finance.ts:205-245`: receipt reference is **required**, amount integer 1..10,000,000 DZD, audited. UI `admin/app/[locale]/(panel)/hosts/HostsClient.tsx:160-163,397-410`.
R61	DS 9.2 (l.149)	Balance and transaction history on the admin detail view.	implemented	`admin/lib/data/finance.ts:82-160`; Hosts drawer Money tab.
R62	DS 9.3 (l.152)	Platform Settings: 4 commission rates, one per host type, editable.	implemented	`admin/lib/data/finance.ts:247-280` (validated, audited); defaults 12/15/18/10 in `admin/lib/pricing.ts:19` and `frontend/lib/booking/config.ts:15-20`.
R63	DS 9.3 (l.153)	On every confirmed booking, auto `commission_deduction` = **booking total** x host-type rate.	different	Frontend charges on **subtotal** (no cleaning, fee, taxes) (`frontend/lib/data/reservations.ts:376-396`; `wallet.ts:405-439`), idempotent, reversed on decline/cancel (`reservations.ts:417-423,447`; `wallet.ts:446-476`). Admin never deducts on confirm; seed data only (`admin/lib/mock-data/generator.ts:494-501`).
R64	DS 9.3 (l.154)	Host sees balance and recent transactions read-only.	implemented	`frontend/components/host/HostWalletWidget.tsx`; `frontend/app/actions/wallet.ts:12`
R65	DS 9.3 (l.155)	Negative balance: banner for host, flag for admin incl. dashboard "action needed".	partial	Host banner `HostWalletWidget.tsx:94,127-129`. Admin: negative filter/count (`admin/lib/data/finance.ts:42,61,103,417`; `HostsClient.tsx:249-268`) but **not** in dashboard attention items (`admin/lib/data/analytics.ts:161-203`).
R66	DS 9.1 (l.145)	`payout` transaction type.	partial	Admin payout requests: approve debits wallet, reject needs reason (`admin/lib/data/finance.ts:283-390`). Frontend has the type but no payout request flow (`frontend/app/actions/wallet.ts` has only `getMyWalletAction`).
R67	(admin, beyond spec)	Per-host commission override.	different (extra)	`admin/lib/data/hosts.ts:332-364`; frontend ignores it (rate by type only, `frontend/lib/data/wallet.ts:262-266`).

1.10 Cross-cutting rules (DS Rules)

#	Spec ref	Requirement	Status	Evidence
R68	DS Rules (l.177)	Every string in en/fr/ar, RTL-correct.	partial	Audit: frontend largely unwired (`frontend/DARNA-AUDIT-REPORT.md` section 5). Backend impact: localized fields (onboarding labels) must be stored as `{en,fr,ar}`.
R69	DS Rules (l.178)	Money formatted with DZD convention.	partial	Integer DZD rule `frontend/lib/booking/pricing.ts:55-58`; admin `formatDzd`. Seed notes still use `toLocaleString("fr-DZ")` (`frontend/lib/data/wallet.ts:434`).

---

2. Disagreements (spec vs frontend vs admin)

No winner is picked. "Impact" = what the backend has to decide.

ID	Topic	Spec says	Frontend does	Admin does	Impact on backend design
D-1	Who is a host	`hostType` on User; org types use Organization (DS 2.1)	Single `User.hostType` + active org in session; same person can own an individual wallet and orgs (`frontend/types.ts:10`, `frontend/lib/server/session.ts:68-75`)	One `Host` row per provider with `hostType`, optional `organizationId` (`admin/lib/types/index.ts:255-280`)	Need a canonical "host account" (user-as-host vs organization) that both apps key wallets, listings, commission and moderation on. Decide whether one user can be individual host and org member at once.
D-2	Commission default rates	Per type, values not given (DS 9.3)	12 / 15 / 18 / 10 (`frontend/lib/booking/config.ts:15-20`)	Same values, duplicated (`admin/lib/pricing.ts:19`, plus hardcoded `defaultRate` in `admin/lib/data/analytics.ts:69-74`)	One `platform_settings` row; analytics must read it, not constants.
D-3	Commission base	"booking total x rate" (DS 9.3 l.153)	Accommodation subtotal only, excl. cleaning and taxes (`frontend/lib/data/reservations.ts:376-379`)	`base x rate` where base = price before fees (`admin/lib/pricing.ts:76-90`); no cleaning fee concept	Pick one base; store `commissionBase`, `commissionRate`, `commissionAmount` on the booking at confirmation time.
D-4	Commission timing and reversal	On every confirmed booking (DS 9.3)	On confirm; reversed fully on decline, pro-rata on guest cancel, fully on host cancel (`reservations.ts:417-447`, `wallet.ts:446-476`)	No automatic deduction; refunds write a proportional negative `booking_refund` (`admin/docs/DATA-MODEL.md` section 3)	Ledger posting rules per booking event (confirm, cancel, refund, dispute) must be specified once.
D-5	Per-host commission override	Not specified (per type only)	Not supported	Supported, audited (`admin/lib/data/hosts.ts:332-364`)	Rate resolution order needed: host override, then type rate. Snapshot the rate on the booking.
D-6	Approval-mode storage	Default on User/Org, optional per-listing override (DS 3)	Listing decides (`bookingApprovalMode` and legacy `instantBook`, `pricing.ts:276-282`); User value derived from first listing; Org value used only for Trips (`bookings.ts:190-193`)	`Host.bookingApprovalMode` (`admin/lib/types/index.ts:274`) plus a platform switch `instantBookingPlatformWide` (`admin/lib/mock-data/store.ts:102`) the frontend never reads	Define resolution: platform switch, then listing override, then owner default. Drop one of `instantBook` / `bookingApprovalMode`.

ID	Topic	Spec says	Frontend does	Admin does	Impact on backend design
D-7	Who can approve a request	Host, or org member "with permission" (DS 3)	Any active member, any role (`frontend/lib/host-types.ts:19`, `frontend/lib/server/authz.ts:25-35`)	Admin can move `pending -> confirmed` (`admin/lib/domain/transitions.ts:11`)	Need an explicit permission (e.g. `bookings.respond`) per org role, and whether staff may confirm on behalf of a host.
D-8	Request expiry	"within a reasonable window" (DS 3)	24 h TTL, lazy expiry on read, payment `voided` (`frontend/lib/booking/config.ts:55`, `lifecycle.ts:12-16`)	No `expired` status at all (`admin/lib/types/index.ts:413-419`)	Need a scheduled job (not lazy-on-read) so expiries also fire notifications and appear in admin. TTL should be a setting.
D-9	Booking statuses	`pending`, `confirmed` (DS 3)	`pending, confirmed, cancelled, completed, declined, expired` (`frontend/types.ts:198-204`)	`pending, confirmed, cancelled, completed, disputed, refunded` (`admin/lib/types/index.ts:413-419`)	Union state machine needed: e.g. keep `declined` / `expired`, add `disputed`; model `refunded` as payment status, not booking status (the frontend already does).
D-10	Listing statuses and moderation	Not specified; trips "live immediately" (DS 5.2)	`active, pending, suspended, draft`; new listings and trips go live at once (`frontend/lib/data/listings.ts:309`, `frontend/app/actions/trips.ts:67`)	8 canonical statuses with review queue (`admin/docs/DATA-MODEL.md` section 1)	Decide if publish requires approval. If yes, frontend create must write `pending_review`.
D-11	Trips model	Separate `Trip` type owned by an agency (DS 5.1)	Separate `Trip` table; reservation has `tripId` and reuses `listingId = trip.id` (`reservations.ts:349`); departures are a string array	Trips are `Listing` rows with `category: "trip"` moderated like listings (`admin/app/[locale]/(panel)/trips/TripsListClient.tsx:82,155`)	Choose: dedicated `trips` + `trip_departures` tables, or listings with a trip subtype. Reservation needs a proper polymorphic target (listing / room type / trip departure). Also rename the guest `/trips` page or the API resource to avoid the "Trips" collision.
D-12	Verticals	4: Homes, Experiences, Vehicles, Trips; hotels are "Homes-like" (DS 5.3, 8)	5 including `hotel` in the type, but hotels are created as `category: "home"` with `businessCategory` (`frontend/types.ts:47-52`; `BecomeHostFlow.tsx:759,860`)	5 with `hotel` as a real vertical (`admin/lib/types/index.ts:300`)	Analytics "by vertical" will double-count or miss hotels unless one rule is fixed.
D-13	Onboarding field types and schema	`text, number, select, multi-select, image-upload, toggle` (DS 7.1)	Same as spec + `textarea` (`frontend/types.ts:507-514`)	`multiselect, file, boolean, radio, location, amenities` + others; different key names (`admin/lib/types/index.ts:603-650`)	One JSON schema for `onboarding_flows` with versioning; mapping table already in `admin/docs/DATA-MODEL.md` section 5.
D-14	Individual onboarding length	12 steps; "2 pages" applies to the **host interface**, not onboarding (DS 4.1, 7.1)	12 steps (`frontend/lib/onboarding-config.ts`)	Capped at 2 steps, enforced on save (`admin/lib/data/onboarding.ts:23-26`, `admin/README.md` section 6)	Admin rule likely misreads DS 4. If kept, the frontend flow cannot be config-driven from admin without losing 10 steps.
D-15	Onboarding config source of truth	Admin config drives live flows (DS 7.2)	Defaults + browser `localStorage` (`frontend/lib/onboarding-config.ts:1064-1091`)	In-memory store, versioned (`admin/lib/data/onboarding.ts`)	Backend must serve the published flow per host type; custom field answers need storage (e.g. JSONB on listing/org application).
D-16	Wallet model	Prepaid: host pays platform offline, admin recharges, commission deducted (DS 9.2)	Prepaid commission wallet only (`frontend/lib/data/wallet.ts`)	Mixed: also `booking_payment` credits (host earnings) and `host_payout` debits (`admin/lib/mock-data/generator.ts:492-501`), i.e. an escrow/payout wallet	Fundamental: does BLADUS collect guest money (escrow, payouts) or not (prepaid commission)? Determines payment integration, ledger accounts and payout flow. The spec itself is mixed: DS 3 "guest completes payment" vs DS 9.2 offline recharge.
D-17	Recharge input	Amount + optional note (DS 9.2)	`rechargeWallet` takes amount + note, any amount (`frontend/lib/data/wallet.ts:290-327`, admin-only, no caller)	Receipt reference required; integer 1..10,000,000; method field (`admin/lib/data/finance.ts:205-219`)	Adopt admin validation; keep `adminUserId` as an id, not a display name (admin stores `adminActor` name).
D-18	Transaction type names	`recharge` / `commission_deduction` / `payout` (DS 9.1)	Same + `adjustment`	`wallet_recharge` / `host_payout` / `booking_payment` / `booking_refund` + `commission_deduction`, `adjustment`	Single enum; commission reversal is `adjustment` in frontend vs `booking_refund` share in admin.
D-19	Room type fields	"inventory count and availability" (DS 2.1)	`totalInventory`, `inventoryCount`, `availableInventory`, `basePrice`, `price` (`frontend/types.ts:92-108`)	`totalInventory`, `availableInventory` (`admin/lib/types/index.ts:316-329`)	Keep one inventory column; availability is computed from bookings and blocks, not stored.
D-20	Hero switcher	4 segments (DS 5.3)	5 (Hotels added)	n/a	Search API must support a `hotel` vertical filter or a `businessCategory` filter consistently.
D-21	Trip creation steps	6 incl. cancellation step (DS 5.2)	5 (`TripCreationClient.tsx:56-58`)	Admin trip wizard separate	Minor; affects only validation order.
D-22	Analytics tabs	Overview / By Host Type / By Vertical / Financial (DS 8)	n/a	overview / providers / services / wilayas / revenue / reports (`StatsClient.tsx:36`)	Reporting API should expose dimensions (host type, vertical, wilaya, period) rather than fixed pages.

3. Policy change: no guest service fee, brand BLADUS

Current policy (confirmed in code): guests pay **no** platform fee; BLADUS earns only the host commission. Evidence: `frontend/lib/booking/config.ts:22-34` (`SERVICE_FEE_RATES` all 0), `frontend/lib/booking/pricing.ts:125-130` (per-listing override ignored), `admin/lib/pricing.ts:14-16` (`GUEST_SERVICE_FEE_PERCENT = 0`). Brand is BLADUS; internal `darna` identifiers are intentionally kept (cookies `darna_session`, env `DARNA_SESSION_SECRET`, storage key `darna_onboarding_config_`, global store keys `__darna_wallets__`).

3.1 Spec and doc items made obsolete or changed

Item	Where	Why obsolete / what changes
"Commission = booking total x rate"	DS 9.3 l.153	With no guest fee, "total" = subtotal + cleaning + taxes. Code uses subtotal only. The spec wording no longer identifies a unique base (D-3).
Guest service fee 7 % (admin) and 10 % / 8 % (frontend)	`admin/docs/DATA-MODEL.md` section 6; `frontend/DARNA-AUDIT-REPORT.md` section 9 table	Both apps are 0 now; the doc is stale.
Audit "serviceFee: 0 becomes 10 %" and "commission charged on total incl. BLADUS service fee"	`frontend/DARNA-AUDIT-REPORT.md` section 4	No longer applicable.
`platformSettings.guestServiceFeePercent`	`admin/lib/mock-data/store.ts:100` ; `admin/lib/types/index.ts:690`	Dead setting; do not carry into the schema (or keep locked at 0).
"Net platform revenue = commission + service fees"	`admin/lib/data/analytics.ts:23-26`	Service-fee term is always 0; platform revenue = commission only.
`Listing.serviceFeeRate` admin override	`frontend/types.ts:164`	Ignored on purpose (`pricing.ts:125`). Drop from schema.
Reservation `serviceFee` / admin `serviceFeeDzd` columns and their UI lines	`frontend/types.ts:222` ; `frontend/components/trips/TripsClient.tsx:458-472` ; `admin/app/[locale]/(panel)/bookings/BookingsClient.tsx:581-584` ; CSV export `admin/lib/data/bookings.ts:150`	Always 0; keep only if historic bookings need it.
Usability KPI "Money paid by guests" vs "DARNA earnings"	`usability-report.md` section 3 and 4.2	Label must read "BLADUS earnings"; with no guest fee, earnings = commission only, and the "paid by guests" figure equals what hosts are owed before commission.
Audit "Total price incl. fees on cards" gap	`frontend/DARNA-AUDIT-REPORT.md` section 10	Reduced to price + cleaning + taxes; no BLADUS fee line.
Brand strings "DARNA" in all spec documents and the usability report	all sources	Read as BLADUS. The usability report's language audit term "DARNA earnings" is now wrong UI copy.
Wallet model ambiguity	DS 3 vs DS 9.2 (D-16)	With no guest fee, a prepaid-commission wallet is coherent only if hosts collect guest money themselves; if BLADUS collects payments, commission can be netted from payouts instead and recharges become optional. Needs a product decision.

4. Usability-report findings that imply backend requirements

Source: `docs/backend/_sources/usability-report.md` (identical to `admin-usability-report.md`). The report describes the admin console as tested; several claims are UI-only in the code, so each one becomes a backend requirement.

#	Report claim (section)	Current code	Backend requirement
U1	"Needs your attention" cards with counts and "oldest item has been waiting 2 days" (section 3)	Counts are real (`admin/lib/data/analytics.ts:161-203`); oldest-waiting values are hardcoded 3/2/1 days (`admin/app/[locale]/(panel)/DashboardClient.tsx:177-216`)	Queue endpoint returning count + `oldestCreatedAt` per queue (hosts, listings, flagged reviews, disputes, negative wallets, expiring requests). SLA timers need indexed `submittedAt` on each moderated entity.
U2	8-second Undo toast after approve / hide review (sections 2.1 task 7, 4.3)	Undo only shows a toast "undo requested" (`admin/app/[locale]/(panel)/approvals/ApprovalsClient.tsx:286-290`)	Reversible actions: either a delayed commit window or an inverse transition endpoint, both audited. Transition tables must allow the reverse move.
U3	Bulk approvals with progress "Approving 4 of 12" (section 4.3)	Sequential client loop, no progress, partial failure leaves mixed state (`ApprovalsClient.tsx:342-358`)	Bulk endpoint with per-item results, idempotency keys, and one audit entry per item.
U4	Live result counts "14 listings match in Oran"; hotels by wilaya (tasks 11, 12)	Admin list functions return `{rows,total,facets}` from in-memory filtering (`admin/lib/data/listings.ts:68-210` , `admin/README.md` section 4)	Server-side filtering, pagination and facet counts (wilaya, host type, status, price range) on indexed columns. Exports must cover the full filtered set (already the rule, `admin/docs/DATA-MODEL.md` section 8).
U5	Reject host with pre-set reasons; host "notified politely" (task 2)	Reason required (>= 3 chars) per DATA-MODEL section 1-2; no notification is sent from admin to frontend (`admin/lib/data/hosts.ts` has no notify call)	Reason catalogue table, and a notification/outbox service shared by both apps (email/SMS/in-app).
U6	Request changes on a listing with checkbox templates (task 4)	`changes_requested` exists only in admin; frontend shows it as `pending` (`admin/docs/DATA-MODEL.md` section 1)	Host-facing change-request object (items + notes) readable by the frontend, plus resubmission transition.
U7	Restore a deleted item (task 6)	Soft delete with `statusBeforeDelete` in admin; frontend hard-deletes listings (`frontend/lib/data/listings.ts:343-347`)	Soft delete everywhere (`deletedAt` , `statusBeforeDelete`), since admin restore cannot work on hard-deleted rows.
U8	Booking dispute with guest claim and host response side by side (task 8)	Admin `disputed` state and outcomes; frontend has no dispute state or way to file one (D-9)	Dispute entity with both parties' statements and attachments, created from the guest/host app.
U9	Recharge modal shows current and new balance (task 9)	Admin computes from ledger (`admin/lib/data/finance.ts:221-244`)	Balance read must be consistent with the write (transactional ledger insert with `balanceAfter`), and the frontend wallet must be the same ledger.
U10	Change commission rate in "Money > Commission tiers" (task 10)	Admin store only; frontend has its own rates store (`frontend/lib/data/wallet.ts:239-266`)	One shared settings table; rate snapshot on each booking.
U11	Edit onboarding steps with toggles and drag handles (task 13)	Admin-only effect (R49)	Published onboarding flow served to the frontend; versioned (see D-15).
U12	4 KPI cards with % change vs last month (section 3)	Trends are hardcoded strings (`admin/lib/data/analytics.ts:43-50`)	Period-over-period aggregates over real date ranges (materialized views or nightly rollups).
U13	"History of actions" (audit log) trusted (section 4.2)	Server-side actor and IP, frozen entries (`admin/docs/DATA-MODEL.md` section 8; `admin/lib/data/audit.ts`)	Persist the audit log append-only in the database (no update/delete grants); include host-side sensitive actions (org role changes, payouts) too.
U14	Filter chip "Suspended/Deleted" (task 6)	Facets by status exist in admin	Status filters must include soft-deleted rows only for permitted roles.
U15	Arabic with Latin digits; Sunday first day; Algerian weekend Fri-Sat (section 5.2)	Formatting rule in `admin/lib/utils/format.ts`	Store dates as date keys / UTC timestamps with Africa/Algiers business timezone; week-based reports start Sunday.
U16	Novice moderator vs power users (Nadia, Karim, Samir)	RBAC roles in `admin/lib/permissions.ts`	Role/permission model in the database with server enforcement (already intended in `admin/README.md` section 3).

5. Quick list of open questions for the blueprint

1. Wallet model: prepaid commission (DS 9) vs escrow with payouts (admin seed). Drives payments, ledger accounts and payout flow (D-16).
2. Commission base and rate resolution order, including per-host override (D-3, D-5).
3. Approval mode resolution and who may respond to requests; request TTL as a setting with a real scheduler (D-6, D-7, D-8).
4. Unified booking, listing and host status machines (D-9, D-10, admin `docs/DATA-MODEL.md`).
5. Trips as their own table with departures vs listings subtype, and the `/trips` naming collision (D-11).
6. Vertical taxonomy: is `hotel` a vertical or a business category of `home` (D-12, D-20)?
7. Onboarding config schema and whether the 2-step individual cap is intended (D-13, D-14, D-15).
8. Moderation before publish for listings and trips (D-10).

Click-through observations (website, dev server, 2026-11-04)

Signed in with the documented demo accounts (password from `frontend/lib/data/auth.ts` DEMO_PASSWORD).

#	Flow	What happened	Backend implication
O1	Guest login (yacine.traveler@bladus.dz)	Works, redirects home.	Email + password login exists; no OTP, no email verification step observed.
O2	Instant book, Tipaza villa, Dec 10 to 13	Confirmed immediately. Card: 18,500 DZD/night x 3 = 55,500 + cleaning 4,000 = 59,500 DZD, "No BLADUS fees".	No payment step exists at all: a booking becomes "confirmed" with no money moving. Real flow needs payment intent before confirmation.
O3	Same link carried `guestCount=2`	Booking card showed "1 guest".	URL param name mismatch (`guestCount` vs `guests`); API must define one name.
O4	Overlap: Dec 12 to 14 on same villa	Blocked: "Some of the selected dates are no longer available". Reserve disabled.	Overlap check exists in app code; must move to a database-level guarantee.
O5	Back-to-back: Dec 13 to 15	Allowed.	Confirms half-open date ranges [check-in, check-out).
O6	Request to book, Constantine residence, Dec 20 to 23	"Request sent, host usually replies within 24 hours, you are not charged until then", status Awaiting approval, total 36,000 DZD.	Request mode with pending status; expiry job needed (24 h).
O7	Trips page	Lists both new bookings. Every card says "Payment: CIB / Edahabia Card, Total paid", including the pending request where nothing was paid.	Payment method and paid status are not modelled; UI fakes them. Need payments table and booking payment status.
O8	Messages	Threads per listing with read receipts and unread count.	Conversations, messages, read state needed.
O9	Become a host	4 host types: Individual, Agency/Fleet, Hotel & Resort, Hostel & Community; each starts a guided flow with draft saving and live preview.	Onboarding drafts must be persisted per user; flow content per host type.
O10	Host mode as Karim (agency owner, also owns 2 hotels)	Today page shows "Agency command centre", rating 4.96 across 159 reviews but "No guest reviews yet", prepaid wallet 145,000 DZD, "Prepaid commission deposit ledger (platform take rate 15%)", "Deducted per confirmed booking (agency rate)".	Wallet model = PREPAID COMMISSION DEPOSIT: host tops up, BLADUS deducts commission per confirmed booking. Different from Airbnb's payout model; must be a ledger. Rating aggregates inconsistent with review rows. A user can own several organizations: active-organization context needed.
O11	Gross host earnings show 0 DZD while there is an active confirmed booking	Earnings only count completed stays?	Define earnings recognition rule (at check-in, check-out, or confirmation).

BLADUS backend research: comparable platforms and Django ecosystem

Status: working notes for the backend blueprint. Research date for every source: **2026-11-04** (all URLs accessed on that date). Scope: Airbnb-style marketplace for Algeria (homes, hotels with room types, hostels with per-bed booking, experiences, vehicle rentals, multi-day agency trips with departures). Two Next.js frontends (customer app, admin/partner app), backend Django + PostgreSQL.

Conventions in this file:

- "Verified" means the fact was read from the cited page or from the package registry (PyPI, npm, GitHub API) on 2026-11-04.
- "Unverified" or "secondary" means the fact comes from a third-party summary, not a primary source. Treat those as hints.
- Version numbers come from the PyPI JSON API (`https://pypi.org/pypi/<name>/json`), the npm registry, or GitHub releases, queried on 2026-11-04.

Part A. How comparable platforms are built

A.1 Airbnb engineering

Topic	What Airbnb published	Source
Monolith to SOA	Airbnb moved from a Rails monolith ("Monorail") to services starting around 2016. Services were defined in a Thrift-based IDL with generated clients; request context and deadlines are propagated; load shedding via request deadlines; schema-based mocks for tests to avoid a "hidden monolith". InfoQ coverage notes a design rule: services own both reads and writes of their data, and the migration compared outputs between monolith and services by replaying traffic.	Building Services at Airbnb, parts 1 to 4: https://medium.com/airbnb-engineering/building-services-at-airbnb-part-1-c4c1d8fa811b , https://medium.com/airbnb-engineering/building-services-at-airbnb-part-2-142be1c5d506 , https://medium.com/airbnb-engineering/building-services-at-airbnb-part-3-ac6d4972fc2d , https://medium.com/airbnb-engineering/building-services-at-airbnb-part-4-23c95e428064 ; InfoQ: https://www.infoq.com/news/2019/02/airbnb-monolith-migration-soa/
Search and ranking	Ranking started with gradient boosted decision trees, then moved to neural networks when gains plateaued; later papers cover diversity in ranking and embedding-based retrieval (2026).	arXiv 1810.09591 https://arxiv.org/abs/1810.09591 ; blog https://medium.com/airbnb-engineering/applying-deep-learning-to-airbnb-search-7ebd7230891f ; follow-ups https://arxiv.org/pdf/2002.05515 , https://arxiv.org/pdf/2210.07774 , https://arxiv.org/pdf/2601.06873
Pricing	Smart Pricing suggests a price for every check-in date from demand, supply and listing features, and models booking lead time per date and location.	https://medium.com/airbnb-engineering/learning-market-dynamics-for-optimal-pricing-97cffbcc53e3
Availability / calendar	No primary Airbnb engineering post describing the calendar/availability service architecture was found. Unverified, so not used.	(none)
Payments idempotency	"Orpheus" idempotency library: client sends an idempotency key per new request and reuses it on retries; request split into pre-RPC, RPC, post-RPC phases with a row-level lock on the key; errors must be classified retryable vs non-retryable; idempotency data read and written on the primary DB (never replicas) to avoid duplicate payments.	https://medium.com/airbnb-engineering/avoiding-double-payments-in-a-distributed-payments-system-2981f6b070bb
Ledger / financial reporting	Financial data is modeled as accounting events producing a double-entry subledger (accounts such as receivables, payables/host liabilities, revenue, taxes). The old approach (reverse-engineering finance from audit trails and DB triggers) was error-prone.	https://medium.com/airbnb-engineering/tracking-the-money-scaling-financial-reporting-at-airbnb-6d742b80f040 ; https://medium.com/airbnb-engineering/scaling-airbnbs-payment-platform-43ebfc99b324
Messaging	"Messaging sync": clients fetch only changed threads/messages since last sync instead of full refetch; messaging is tightly coupled to the booking flow (status changes appear in threads).	https://medium.com/airbnb-engineering/messaging-sync-scaling-mobile-messaging-at-airbnb-659142036f06
Trust and safety	ML risk scores (e.g. chargeback model) drive *targeted friction*: only risky users get extra verification steps, so good users are not penalized; effect measured via experiments.	https://medium.com/airbnb-engineering/fighting-financial-fraud-with-targeted-friction-82d950d8900e
Reviews	Homes: both guest and host have 14 days after checkout; reviews are published when both submitted or when the 14 days end, whichever comes first (double-blind).	https://www.airbnb.com/help/article/13 ; https://airbnb.com/help/article/995
Cancellation policies	Third-party (secondary) sources report Airbnb restructured standard policies on 2025-10-01 to Flexible / Moderate / Limited / Firm with a 24 h grace period and optional non-refundable rate. Not verified on an Airbnb primary page.	secondary: https://www.hostfully.com/blog/airbnb-cancellation-policy/

A.2 Booking.com, Expedia, Vrbo, GetYourGuide

Platform	Pattern	Source
Booking.com	Inventory model = **room type** x **rate plan** ("roomrate"). Rooms-to-sell is set per room type per date and shared across all rate plans of that room type; prices and most restrictions (min stay, closed to arrival, etc.) are per room-type + rate-plan + date. Child rate plans can derive from a parent. They recommend loading 12 months ahead.	https://developers.booking.com/connectivity/docs/ari.md ; https://developers.booking.com/connectivity/docs/b_xml-availability ; https://developers.booking.com/connectivity/docs/room-type-and-rate-plan-management/managing-roomrates ; https://developers.booking.com/connectivity/docs/room-type-and-rate-plan-management/managing-rate-plans
Booking.com reviews	Only people who booked through the platform can review (includes no-shows and bookers for others); written reviews are moderated; reviews remain visible 36 months; removal conditions are enumerated.	https://partner.booking.com/en-us/help/guest-reviews/general/guest-review-process-and-conditions ; https://www.booking.com/reviews_guidelines.html
Expedia Rapid	Cancellation policy is data, not text: each rate has `refundable` + `cancel_penalties[]` (time windows with amount/percent/nights) + `nonrefundable_date_ranges` ; since Feb 2025 also `current_refundability` (refundable / non_refundable / partially_refundable). Booking page must show the policy before confirmation.	https://developers.expediagroup.com/rapid/lodging/shopping/constructing-cancellation-policies ; https://developers.expediagroup.com/rapid/setup/launch-requirements/lodging-launch-reqs
GetYourGuide	Activities inventory = time-slotted vacancies per product option; flow is availability then reserve (hold) then confirm; supplier systems push availability updates; they rebuilt inventory on Kafka Streams when DB tables and stale availability caused booking errors.	https://www.getyourguide.careers/posts/scaling-the-inventory-parts-of-our-system-using-kafka-streams ; https://github.com/getyourguide/partner-api-spec ; https://www.getyourguide.supply/connectivity/partners-faqs
Vrbo	No primary engineering or API documentation could be retrieved (developer portal did not respond). Unverified; not used.	(none)

A.3 Open-source projects and reference models worth studying

Project	Stack	What is useful for BLADUS	Link
pretix	Django	Best Django reference for **quota-based inventory**: an item can belong to several quotas, all must be available; availability = limit minus paid orders, pending orders within payment period, unexpired cart holds, blocking vouchers, waiting list. Directly maps to experiences, trip departures and hostel beds. Active (pushed 2026-11-04).	https://github.com/pretix/pretix ; https://docs.pretix.eu/dev/development/concepts.html ; https://docs.pretix.eu/dev/development/implementation/models.html
Saleor	Django + GraphQL	Order/payment state machines, checkout, channels, multi-currency, webhooks, permission groups. BSD-3.	https://github.com/saleor/saleor
django-oscar	Django	Mature domain-driven e-commerce: baskets, offers, order/payment events, app overriding pattern.	https://github.com/django-oscar/django-oscar
QloApps	PHP (PrestaShop-based)	Open-source hotel booking + channel manager concepts: hotel, room type, room, booking per date, demand-based pricing. Data model reference only. OSL-3.0.	https://github.com/Qloapps/QloApps
django-hordak	Django	Double-entry accounting models (accounts, transactions, legs) enforced with DB constraints; good reference for the payouts ledger. Last push 2026-04-02, small project.	https://github.com/adamcharnock/django-hordak
TigerBeetle	Zig DB	Purpose-built double-entry ledger DB; worth reading its design docs on debits/credits and pending (two-phase) transfers even if we do not adopt it.	https://github.com/tigerbeetle/tigerbeetle
Modern Treasury "Accounting for developers"	Article series	Clear explanation of double-entry for engineers.	https://www.moderntreasury.com/journal/accounting-for-developers-part-i
Chargily Pay v2	Algerian gateway	EDAHABIA (Algerie Poste) and CIB (SATIM) checkouts; HMAC `signature` header on webhooks; test and live modes; Python SDK exists. Likely the first local payment integration.	https://dev.chargily.com/pay-v2/introduction ; https://dev.chargily.com/pay-v2/webhooks

A.4 Domain model summary for BLADUS (derived)

Unifying idea: every bookable product reduces to **an inventory unit + a time shape + a capacity**.

Vertical	Listing	Sellable unit	Time shape	Capacity per slot	Comparable model
Home	Property	the whole property	nightly date range	1	Airbnb
Hotel	Property	RoomType (pool of N physical Rooms) + RatePlan	nightly date range	N per date (rooms-to-sell)	Booking.com roomrate
Hostel	Property	BedType / Dorm (pool of beds), optionally specific Bed	nightly date range	beds per date	Booking.com room type with count
Experience	Activity	Option	time slot (start datetime)	seats per slot	GetYourGuide, pretix quota
Vehicle	Vehicle (or vehicle class pool)	Vehicle	datetime range (pickup/return)	1 per vehicle, N per class	car rental
Agency trip	Trip	Departure (fixed dates)	fixed multi-day departure	seats per departure	pretix quota per departure

Core entities: `Organization` (host or agency, with members and roles), `Listing` (polymorphic by vertical), `InventoryUnit`, `RatePlan`, `DailyRate` / `PriceRule`, `AvailabilityBlock`, `CancellationPolicy` (structured), `Booking` + `BookingLine` + state machine, `Hold` (short TTL), `Payment`, `Refund`, `Payout`, `LedgerAccount` + `LedgerEntry`, `Review` (bound to a completed booking), `Thread` + `Message`, `Report` / `ModerationCase`, `AuditEvent`.

A.5 Hard problems and what we decide

Hard problem	What the platforms show	BLADUS decision
Double booking	Airbnb publishes nothing primary on calendars; Django/PostgreSQL offer native exclusion constraints on ranges (`ExclusionConstraint` + `RangeOperators.OVERLAPS` , requires `btree_gist`).	For capacity-1 units (home, specific vehicle, specific room or bed when assigned): store `stay = DateRangeField` (half-open `[checkin, checkout)`) and add an `ExclusionConstraint(unit WITH =, stay WITH &&) WHERE status IN active states` . The DB is the last line of defense; the service layer catches `IntegrityError` and returns a 409. For pooled capacity (hotel room types, hostel beds, experience slots, trip departures): an `InventoryDay` / `SlotInventory` row per unit per date with `total` , `held` , `booked` and a `CHECK (held + booked <= total)` ; reserve with `UPDATE ... SET held = held + n WHERE ... AND held + booked + n <= total` (atomic, row-locked) inside a transaction. Holds expire via a scheduled job (pretix cart-hold idea).
Availability search at scale	Booking.com precomputes ARI per date; GetYourGuide moved to streaming when DB-backed inventory became stale and slow.	Phase 1: query PostgreSQL directly (PostGIS bounding box + filters + `NOT EXISTS` overlapping bookings / `InventoryDay` min availability over the range), with GiST and B-tree indexes. Keep a denormalized `ListingSearchDoc` row per listing updated by signals/tasks. Only add an external engine when measured latency demands it (Algeria-scale catalog is small). Never trust search results for booking: re-check inside the booking transaction.
Pricing	Booking.com: price per room-type x rate-plan x date; Airbnb: per-date suggestions.	Store explicit nightly prices per unit/date (`DailyRate`) materialized from rules (base, weekend, season, length-of-stay discount). Price quote is computed server side, returned with a signed `quote_id` and expiry, and re-validated at booking.
Payments, escrow, payouts	Airbnb: idempotency keys with row lock on primary DB; double-entry subledger from accounting events.	Every money-moving endpoint requires an `Idempotency-Key` header (IETF draft https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/ , Stripe pattern https://stripe.com/docs/api/idempotent_requests). Immutable double-entry ledger in PostgreSQL: `LedgerTransaction` with `LedgerEntry` legs whose sum is zero per currency (enforced by a deferred constraint trigger or by service + check). Accounts: guest_receivable, platform_escrow (funds held until check-in + N hours), host_payable, platform_revenue (fees), tax_payable, refunds. Balances derived from entries, never edited. Payouts are batch jobs that move host_payable to paid. Gateway webhooks (Chargily HMAC) are stored raw, verified, deduplicated by event id, then processed idempotently.
Cancellation policies	Expedia models policies as penalty windows; Airbnb has named tiers.	Store `CancellationPolicy` as named templates with structured `windows[] (hours_before_start, refund_percent, fee_nights)` ; snapshot the policy onto the booking at confirmation (policy changes never affect existing bookings). Refund calculation is a pure function tested with hypothesis.
Fraud and fake listings	Airbnb: risk scores plus targeted friction; a single decision model applies to users, listings and reservations (secondary source https://dmitry.pt/2023/detecting-risk-in-real-time-technology).	Listing goes `draft -> pending_review -> published` ; admin moderation queue in the back-office. KYC for hosts and agencies (ID + registre de commerce for agencies) before payout enablement. Rule-based risk flags (new account + high value, many failed payments, duplicate photo hashes via perceptual hash, off-platform contact info in messages) that trigger friction (phone OTP, manual review) instead of blanket blocks. All decisions recorded as `ModerationCase` with audit trail.
Review integrity	Booking.com: only bookers can review, moderation, fixed visibility window. Airbnb: 14-day window, double-blind reveal.	Review requires a `Booking` in `completed` state, one review per booking per direction, window of 14 days after end, double-blind reveal (publish when both submitted or window closes), no edits after reveal, host may reply once, moderation for abuse with reasons logged. Aggregate ratings stored denormalized on the listing and recomputed by task.

Hard problem	What the platforms show	BLADUS decision
Messaging	Airbnb sync model: only fetch changes.	Thread per booking/inquiry; cursor-based "changes since" endpoint so polling is cheap; realtime push is an optimization layered on top (see B.6). Mask phone numbers/emails before booking confirmation.
Architecture	Airbnb SOA was a response to hundreds of engineers on one codebase.	Start as a **modular monolith** (one Django project, bounded-context apps, services/selectors, no cross-app model writes). Services are a later option, not a starting point.

Part B. Django ecosystem decisions

Baseline versions (PyPI, 2026-11-04): **Django 6.1.3** (latest), 6.0.10, **5.2.19 LTS** (extended support until April 2028); next LTS 6.2 scheduled April 2027 (https://www.djangoproject.com/download/). Django 6.0 (released 2025-12-03) added built-in Tasks framework and CSP support (https://www.djangoproject.com/weblog/2025/dec/03/django-60-released/ , https://docs.djangoproject.com/en/6.0/releases/6.0/). psycopg 3.3.7.

Recommendation for baseline: Django 6.1 on Python 3.13, plan upgrade to 6.2 LTS in 2027. Rationale: 5.2 LTS lacks `django.tasks` and CSP; 6.1 is supported until December 2027, overlapping 6.2 LTS. Alternative if the team prefers fewer upgrades: 5.2 LTS + `django-tasks` backport. Note the user brief says the frontends are Next.js 14; npm `next` latest is 16.4.5, which matters for SSR auth patterns below.

B.1 PostgreSQL + PostGIS, range types and exclusion constraints

Option	Pros	Cons
PostGIS + GeoDjango (`django.contrib.gis`)	Real geography types, `ST_DWithin` distance queries with GiST index, bounding-box map search, polygon areas (wilayas, communes). First-class in Django ORM.	Needs GDAL/GEOS/PROJ system libs in containers; managed PG must offer PostGIS.
Plain lat/lng floats + Haversine or `earthdistance` / `cube`	No system libs.	No polygon support, weaker indexing, reinventing GIS.

Range types + exclusion: `django.contrib.postgres.fields.DateRangeField` / `DateTimeRangeField` , `ExclusionConstraint` with `RangeOperators.OVERLAPS` and `RangeOperators.EQUAL` , optional `condition=Q(...)` (partial), `BtreeGistExtension` migration op required for the equality part (verified in docs: https://docs.djangoproject.com/en/6.0/ref/contrib/postgres/constraints/ , https://docs.djangoproject.com/en/6.0/ref/contrib/postgres/fields/ , https://www.postgresql.org/docs/current/rangetypes.html , https://www.postgresql.org/docs/current/btree-gist.html).

Recommendation: PostGIS + GeoDjango (https://docs.djangoproject.com/en/6.0/ref/contrib/gis/ , https://postgis.net/). Store `location = PointField(geography=True, srid=4326)` and public "fuzzed" point for homes (exact address revealed after booking). Use `DateRangeField` + `ExclusionConstraint` for capacity-1 units and `CHECK` -constrained inventory counters for pooled units (see A.5). Note: violation of an exclusion constraint surfaces as `IntegrityError` on save; `violation_error_code` only applies to model validation.

B.2 API layer: DRF vs Django Ninja

	Django REST Framework	Django Ninja
Latest	3.18.3 (2026-10-06)	1.7.3 (2026-10-29)
Activity	Very active releases in 2026; maintainers state it is "considered feature-complete" (https://www.django-rest-framework.org/community/release-notes/)	Active (pushed 2026-11-04), 9.2k stars
Pros	Largest ecosystem (drf-spectacular, guardian/rules integration, throttling, filters, SimpleJWT, standardized errors); every Django dev knows it; browsable API.	Pydantic schemas, type hints drive validation and OpenAPI automatically, fast, async-friendly, less boilerplate.
Cons	Serializer verbosity, OpenAPI via add-on, sync-first.	Smaller ecosystem: permissions, throttling, filtering patterns are thinner; fewer battle-tested auth/permission libs.

Recommendation: DRF, used in the HackSoft style (thin APIView + input/output serializers + services). Reason: the permission, throttling, audit and OpenAPI tooling we need is mature around DRF, and the hiring pool in Algeria will know DRF. Ninja is a fine choice if the team values Pydantic typing more; allauth headless supports both.

B.3 Auth

	django-allauth (headless)	dj-rest-auth + SimpleJWT
Latest	65.21.1 (2026-10-30), very active	dj-rest-auth 7.2.0 (2026-03-15); SimpleJWT 5.5.1 (2025-07-21)
Features	Headless API for SPA/mobile with session tokens or JWT strategies, OpenAPI spec, email + **phone login and SMS verification** (adapter implements `send_verification_code_sms`), social login, built-in **MFA** (TOTP, recovery codes, WebAuthn/passkeys), rate limits.	REST endpoints for login/registration/password reset, JWT or tokens; social via allauth underneath.
Cons	Opinionated flow model; headless docs are spread across pages.	Two libraries glued together, slower release cadence, no built-in MFA or phone OTP.

Sources: https://docs.allauth.org/en/latest/headless/index.html , https://docs.allauth.org/en/latest/headless/token-strategies/index.html , https://docs.allauth.org/en/latest/account/phone.html , https://docs.allauth.org/en/latest/mfa/index.html , https://github.com/iMerica/dj-rest-auth , https://github.com/jazzband/djangorestframework-simplejwt

Phone OTP: allauth phone support needs us to provide phone storage and an SMS sender. SMS provider choice for Algeria (local aggregators vs Twilio/Vonage) was not researched here; unverified. Store numbers with `django-phonenumber-field` 9.0.0 (E.164).

Sessions vs JWT for Next.js SSR: server components run on the Next.js server, so they cannot read browser `localStorage` ; they can forward cookies. Next.js docs describe cookie-based session management for the App Router (https://nextjs.org/docs/app/guides/authentication). Two workable patterns:

1. Same-site deployment (`app.bladus.dz` + `api.bladus.dz`): Django session cookie with `Domain=.bladus.dz` , `HttpOnly` , `Secure` , `SameSite=Lax` , CSRF token for mutations. Server components forward the incoming `cookie` header to the API.
2. Next.js as BFF: Next.js stores an encrypted httpOnly cookie containing the allauth session token (app client flow, `X-Session-Token`) and attaches it on server-side fetches.

Recommendation: django-allauth headless with session-token strategy, Next.js BFF pattern (2) for the customer app, short sessions plus mandatory MFA (allauth MFA TOTP/WebAuthn) for staff in the admin app. Avoid long-lived JWTs in the browser (revocation is hard). django-otp 1.7.5 is the alternative if we keep Django admin login separate; prefer one MFA system (allauth MFA) unless Django admin is exposed, in which case add `django-otp` on the admin site only.

B.4 Permissions (object-level, org roles owner/admin/manager)

	DRF permissions (custom classes)	django-guardian	django-rules
Latest	built into DRF 3.18.3	3.5.2 (2026-10-20), active	`rules` 3.5 (2024-09-02); repo pushed 2025-10
Model	`has_permission` / `has_object_permission` per view	per-object permission rows in DB (user/group x perm x object)	predicates (Python functions) composed with `&`, `
Pros	Simple, no deps	Fine-grained grants, admin integration	Expressive, testable, zero storage, integrates with Django's perm API and DRF
Cons	Logic scattered per view	Row explosion (every listing x member x perm), sync burden when memberships change	Release cadence slow (stable, small codebase); queryset filtering must be written separately

Sources: https://www.django-rest-framework.org/api-guide/permissions/ , https://django-guardian.readthedocs.io/ , https://github.com/dfunckt/django-rules

Recommendation: Org-scoped RBAC in our own tables (`Membership(user, organization, role)`), predicates in **django-rules** (`is_org_owner` , `is_org_manager_of(listing)`), exposed through thin DRF permission classes; plus selectors that always filter querysets by the caller's organizations (permissions alone do not protect list endpoints). Skip guardian: our grants derive from org membership, not per-object ACLs.

B.5 Background jobs

Option	Latest	Notes
Celery + Redis/RabbitMQ	celery 5.6.3 (2026-03-26)	De facto standard; beat scheduler, retries, chords, Flower monitoring. Heavier config, many knobs. https://docs.celeryq.dev/en/stable/
Dramatiq	2.2.1 (2026-09-02)	Simpler, reliable defaults (acks late), LGPL. https://dramatiq.io/
Django-Q2	1.11.1 (2026-08-26)	Can use the DB as broker, admin integration; smaller community. https://django-q2.readthedocs.io/
Huey	3.5.4 (2026-11-01)	Lightweight, Redis or SQLite, periodic tasks. https://huey.readthedocs.io/
Django `django.tasks` (Django 6.0+)	built-in	DEP 14 "Background workers" status **Accepted**, implemented in 6.0. Django provides definition, queuing and results only; built-in backends (`ImmediateBackend` , `DummyBackend`) are for dev/test; production needs a third-party backend with a worker, e.g. `django-tasks-db` 0.13.0 (2026-08-28) or `django-tasks-rq` 0.12.0. Arguments must be JSON-serializable; enqueue inside `transaction.on_commit` . https://docs.djangoproject.com/en/6.0/topics/tasks/ , https://raw.githubusercontent.com/django/deps/main/final/0014-background-workers.rst , https://github.com/RealOrangeOne/django-tasks

Recommendation: Celery 5.6 + Redis for now (needs: hold expiry, payout batches, scheduled reminders, image processing, search doc refresh, webhooks retry with backoff). Write tasks as thin wrappers around services and enqueue via `transaction.on_commit` , so moving to `django.tasks` later is mechanical once its worker backends mature. Use `celery beat` with `django-celery-beat` (not version-checked here) or a code-defined schedule.

B.6 Realtime messaging

Option	Pros	Cons
Django Channels + channels-redis	Same codebase and auth; channels 4.3.2 (2025-11-20), channels-redis 4.3.0 (2025-07-22). https://channels.readthedocs.io/	Needs ASGI servers and sticky long-lived connections managed by us; release cadence slowed in 2025.
Centrifugo (self-hosted, Go)	v6.10.2 (2026-10-23). Handles WebSocket/SSE/HTTP-streaming connections; Django only publishes over HTTP API and issues expiring JWT connection tokens; positions itself as a self-hosted alternative to Pusher/Ably. https://centrifugal.dev/docs/getting-started/comparisons	Another service to run. Some features only in paid PRO.
Pusher / Ably (managed)	Zero ops	Data leaves to foreign SaaS, per-message cost, not self-hostable. Pricing not researched here.
Polling / SSE	Simplest; works with "changes since cursor" endpoint (Airbnb messaging sync idea).	Latency and wasted requests at scale.

Recommendation: Launch with **cursor-based polling** (every 5 to 15 s while a thread is open, slower in inbox) over a sync endpoint, plus push notifications/email. Add **Centrifugo** when realtime matters (keeps Django WSGI-only, simpler than running Channels). Channels is the fallback if we want everything in Python.

B.7 Search

Option	Latest	Pros	Cons
PostgreSQL FTS + pg_trgm + PostGIS	built in	One datastore, transactional consistency with availability, Django `SearchVector` , `TrigramSimilarity` , GiST/GIN indexes. https://docs.djangoproject.com/en/6.0/ref/contrib/postgres/search/ , https://www.postgresql.org/docs/current/pgtrgm.html	Arabic stemming support in PostgreSQL FTS is limited (no built-in Arabic snowball config verified here); relevance tuning is manual.
Meilisearch	server v1.56.1 (2026-11-02), py client 0.43.0	Typo tolerance, facets, geosearch (`_geoRadius` , `_geoBoundingBox`), easy ops. https://www.meilisearch.com/docs/learn/filtering_and_sorting/geosearch	Separate index to keep in sync; availability must still be checked in PG.
Typesense	v30.2 (2026-04-19), py client 2.1.1	Similar feature set, geo search, in-memory speed. https://typesense.org/docs/guide/	Same sync burden; RAM-bound.
OpenSearch	3.9.0 (2026-09-29)	Most powerful (analyzers incl. Arabic, aggregations).	Heavy to operate for our scale.

Recommendation: PostgreSQL first (PostGIS distance/bbox + filters + `unaccent` + pg_trgm for place/title autocomplete, FTS for descriptions). Keep a `ListingSearchDoc` table that is engine-agnostic. Add **Meilisearch** later if multilingual typo-tolerant text search (especially Arabic and Latin transliterations like "Tizi Ouzou" vs "تيزي وزو") becomes a product need. Availability filtering always stays in PostgreSQL.

B.8 Files and images

	django-storages + S3-compatible	Cloudinary
Latest	django-storages 1.14.6 (2025-04-02; no release since, repo should be watched)	(SaaS; Python SDK not version-checked)
Pros	Vendor-neutral; Cloudflare R2 has no egress fees for S3 API / public access (https://developers.cloudflare.com/r2/pricing/), supports presigned URLs (https://developers.cloudflare.com/r2/api/s3/presigned-urls/); B2, Scaleway, OVH also S3-compatible.	On-the-fly transforms, CDN, moderation add-ons in one product.
Cons	We build the resize pipeline.	Cost scales with transformations and bandwidth; lock-in.

Pipeline options: Pillow 13.0.0 in a Celery task (generate fixed renditions, strip EXIF/GPS), or **imgproxy** v4.0.21 (2026-11-02) / thumbor (active) as on-the-fly resizer behind the CDN with signed URLs (https://docs.imgproxy.net/ , https://github.com/thumbor/thumbor). Virus scanning: ClamAV 1.5.4 (2026-08-07) via clamd in a worker for documents (KYC PDFs); images are re-encoded by Pillow, which neutralizes most payloads (https://docs.clamav.net/).

Recommendation: django-storages (S3 backend) + Cloudflare R2, private bucket for KYC documents, public bucket behind CDN for listing photos. Uploads via **presigned PUT** from the browser to a `quarantine/` prefix, then a task validates MIME + size, re-encodes with Pillow to WebP/AVIF + JPEG fallback renditions, computes a perceptual hash (duplicate/fake listing detection), and moves to `public/`. Consider imgproxy later. Data residency for Algerian law on personal data (Law 18-07) was not researched here and must be checked before storing KYC abroad.

B.9 Content translations fr/en/ar

	django-modeltranslation	django-parler
Latest	0.20.6 (2026-09-07), active	2.4 (2026-05-14), active
Storage	Extra columns per language on the same table (`title_fr` , `title_en` , `title_ar`)	Separate translation table per model
Pros	Simple queries, easy indexing/FTS per language, no joins, works with admin.	Unlimited languages without migrations, clean schema.
Cons	Adding a language = migration; wide tables.	Joins/prefetch on every read, harder filtering and ordering.

Sources: https://django-modeltranslation.readthedocs.io/ , https://django-parler.readthedocs.io/

Recommendation: django-modeltranslation for platform-owned taxonomy (amenities, categories, wilayas, policy names): fixed 3 languages, fast reads, per-language search columns. For user-generated text (listing titles/descriptions) store the host's original text + `source_language` , plus optional translated fields filled by humans or machine translation later; do not force hosts to write three versions.

B.10 Back-office

	Custom Next.js admin (existing)	Django admin + Unfold
Latest	existing app	django-unfold 0.110.1 (2026-11-01), very active (https://unfoldadmin.com/)
Pros	Product-quality UX for partners and ops, same design system, same API (dogfooding).	Free CRUD on every model, filters, actions, inline history; minutes to add a screen.
Cons	Every internal tool must be built through the API.	Not for partners/hosts; risk of bypassing service-layer rules if admins edit models directly.

Recommendation: Both, with clear roles. The **Next.js admin** is the product (partner dashboards, moderation queue, payouts review, support). **Django admin + Unfold** is a staff-only, MFA-protected, IP-restricted engineering/support console, mostly read-only for financial models (ledger entries never editable), with admin actions calling services.

B.11 Audit logging

	django-simple-history	django-auditlog	django-pghistory
Latest	3.13.0 (2026-07-22), django-commons	3.4.1 (2025-12-18), jazzband	3.9.2 (2026-02-17), AmbitionEng
Mechanism	Python signals, full historical table per model	Python signals, single LogEntry table with JSON diffs	**PostgreSQL triggers** (django-pgtrigger), event tables, context (user, request) attached
Pros	Point-in-time snapshots, admin integration	Simple, one table	Captures bulk updates, raw SQL and `queryset.update()` that signals miss; reliable
Cons	Misses `update()` /bulk ops	Same; diffs only	PostgreSQL only (fine for us), triggers in migrations

Sources: https://django-simple-history.readthedocs.io/en/stable/ , https://django-auditlog.readthedocs.io/ , https://django-pghistory.readthedocs.io/

Recommendation: django-pghistory for bookings, payments, payouts, listings, memberships, policies (bulk updates in inventory and payouts would escape signal-based tools). Plus an explicit domain `AuditEvent` /activity log written by services for human-readable "who did what" in the back-office. The ledger is itself append-only and needs no history.

B.12 Rate limiting, caching, idempotency

- **DRF throttling** (built in; scoped throttles per endpoint, Redis cache backend) https://www.django-rest-framework.org/api-guide/throttling/ . **django-ratelimit** 4.1.0 is from 2023-07-24 with repo last pushed 2024-07: stable but inactive. allauth has its own rate limits for auth endpoints. Recommendation: DRF scoped throttles for API, allauth limits for auth, plus edge limits at the reverse proxy/CDN.
- **Caching:** Redis via Django's built-in `RedisCache` . **django-cachalot** 2.9.1 (2026-08-10) auto-caches ORM queries and invalidates per table; on write-heavy tables (inventory, bookings) invalidation is constant, so value is low. Recommendation: explicit caching of computed results (search facets, listing detail, quote) with versioned keys; no cachalot.
- **Idempotency keys:** `django-idempotency-key` 1.3.0 last released 2023-05-02 (inactive). Recommendation: small in-house implementation following Airbnb Orpheus and the IETF draft: `IdempotencyRecord(key, user, endpoint, request_hash, status, response, locked_at, expires_at)` on the primary DB, `SELECT ... FOR UPDATE` , replay the stored response for repeats, 409 for concurrent in-flight, 422 for same key different body.

B.13 API docs and typed TS client

- **drf-spectacular** 0.30.0 (2026-07-06), OpenAPI 3 generation for DRF, active (https://drf-spectacular.readthedocs.io/). Add **drf-standardized-errors** 0.16.0 for a uniform error schema.
- TS generators (npm, 2026-11-04):

Tool	Latest	Output	Notes
openapi-typescript + openapi-fetch	7.13.0 / 0.17.0 (2026-02-11)	Types only + tiny typed fetch	Minimal runtime, works in server components
orval	8.46.0 (2026-10-30)	Clients + TanStack Query hooks + MSW mocks + Zod	Very active, heavier
@hey-api/openapi-ts	0.99.0 (2026-06-22)	SDK, types, Zod, TanStack plugins	Pre-1.0 (breaking changes possible)

Recommendation: drf-spectacular -> `openapi.json` committed in CI -> **openapi-typescript + openapi-fetch** shared package used by both Next.js apps (fits server components and route handlers; no hooks runtime required). Consider orval only if the apps standardize on TanStack Query in client components.

B.14 Testing

Tool	Latest	Role
pytest-django	4.14.0 (2026-08-10)	test runner
factory_boy	3.3.3 (2025-02-03)	explicit factories, traits, SubFactory
model_bakery	1.24.2 (2026-10-06)	quick auto-filled instances
hypothesis	6.170.14 (2026-10-30)	property tests: pricing, refunds, ledger balancing, overlap logic
schemathesis	4.33.0 (2026-11-01)	contract/fuzz tests from the OpenAPI schema

Recommendation: pytest-django + **factory_boy** (explicit, readable domain factories) + hypothesis for money and date-range logic + schemathesis in CI against the generated schema. Concurrency tests for double booking using real PostgreSQL (`transaction=True` tests with threads), never SQLite.

B.15 Project structure

- **HackSoft Django Styleguide**: business logic in `services` (writes) and `selectors` (reads), thin APIs, models without heavy logic, explicit input/output serializers (https://github.com/HackSoftware/Django-Styleguide). Last pushed 2025-09.
- **cookiecutter-django**: production template (Docker, Celery, allauth, django-environ, Sentry, pytest), release 2026.11.3 on 2026-11-04, very active (https://github.com/cookiecutter/cookiecutter-django).
- Two Scoops of Django: book; no current edition for Django 6 was verified, so treat as general guidance only.

Recommendation: Bootstrap from cookiecutter-django ideas (not necessarily the generator), organize as a **modular monolith** with apps per bounded context: `accounts`, `organizations`, `catalog` (listings, units, media, amenities), `inventory` (availability, holds), `pricing`, `bookings`, `payments` (gateway, ledger, payouts), `reviews`, `messaging`, `trust` (moderation, risk, KYC), `notifications`, `geo`, `search`, `audit`. Follow HackSoft services/selectors; cross-app calls go through services, never through another app's models directly. Enforce boundaries with `import-linter` (not version-checked here).

B.16 Settings, linting, CI/CD, observability

Topic	Option A	Option B	Recommendation
Settings	django-environ 0.14.0 (2026-06-18)	pydantic-settings 2.16.2 (2026-10-28)	**pydantic-settings**: typed, validated at boot, fails fast. django-environ is fine and is what cookiecutter uses.
Lint/format	ruff 0.17.3 (2026-10-27)	(black + isort + flake8)	**ruff** for lint + format.
Typing	mypy 2.4.1 + django-stubs 6.1.2 + djangorestframework-stubs 3.18.2	pyright	**mypy + django-stubs** (plugin understands the ORM).
CI/CD	GitHub Actions: ruff, mypy, pytest with PostGIS service container, schema diff, schemathesis, build image		GitHub Actions.
Errors	sentry-sdk 2.73.0		Sentry (Django + Celery integrations) https://docs.sentry.io/platforms/python/integrations/django/
Logs	django-structlog 11.0.0 / structlog 26.1.0	std logging JSON formatter	**structlog** JSON logs with request_id, user_id, org_id.
Tracing	opentelemetry-instrumentation-django 0.66b2 (still beta versioning)		Add OTel when we have >1 service; Sentry performance first.
Metrics	django-prometheus 2.5.0 (2026-05-26)		Prometheus metrics endpoint; plus django-health-check 4.11.4 for probes.

B.17 Money handling

- ISO 4217 List One gives **DZD numeric 012, minor unit 2** (verified via copies of List One: SAP documentation https://help.sap.com/doc/saphelp_pos22/2.2/en-US/4f/efa63a9bb04d94a0ae4c32fd6cd827/content.htm?no_cache=true , Adyen currency table https://docs.adyen.com/development-resources/currency-codes ; ISO's own page returned HTTP 403 to automated fetch; the maintenance agency is SIX https://www.six-group.com/en/products-services/financial-information/data-standards.html). Wikipedia notes centimes rarely circulate (https://en.wikipedia.org/wiki/Algerian_dinar).
- Options:

	Integer minor units (`BigIntegerField` , centimes)	**`DecimalField(max_digits=14, decimal_places=2)` / django-money**
Pros	Exact, fast, matches gateway APIs that use minor units, trivial sums in ledger, no rounding surprises.	Human-readable in DB; django-money 3.6.2 adds a `Money` type and currency field.
Cons	Must format at the edges.	Decimal context/rounding discipline required; **django-money's GitHub repo was archived and the project is migrating to Codeberg (notice dated 2026-06-07)**, https://codeberg.org/django-money/django-money , so its maintenance path is in transition.

Recommendation: Store **integer minor units (centimes) in** `BigIntegerField` + `currency CHAR(3)` on every amount column, wrapped in a small in-house `Money` value object (amount_minor, currency) with explicit rounding rules. Business rule: prices shown and charged in whole dinars (amounts are multiples of 100 centimes), but fees, commissions and tax splits are computed in centimes and rounded with a documented rule (round half up at the line level, remainder assigned to platform), so the ledger always balances exactly. This keeps us ISO-correct (minor unit 2) while matching practical whole-dinar pricing. Chargily's amount unit (dinars vs centimes) must be confirmed against its API reference before integration; not verified here.

Summary table of recommendations

#	Topic	Choice
0	Framework	Django 6.1 (Python 3.13), upgrade to 6.2 LTS in 2027
1	DB	PostgreSQL + PostGIS; DateRange + ExclusionConstraint for capacity-1, CHECKed counters for pools
2	API	DRF 3.18 + HackSoft style
3	Auth	allauth headless (session tokens, phone OTP, MFA), Next.js BFF cookie
4	Permissions	Own org RBAC + django-rules + queryset scoping
5	Jobs	Celery 5.6 + Redis, tasks as thin service wrappers
6	Realtime	Polling sync endpoint first, Centrifugo later
7	Search	PostgreSQL FTS + pg_trgm + PostGIS; Meilisearch only if needed
8	Files	django-storages + R2, presigned uploads, Pillow renditions, ClamAV for docs
9	i18n	modeltranslation for taxonomy; original + translations for UGC
10	Back-office	Next.js admin for product; Django admin + Unfold for staff console
11	Audit	django-pghistory + domain AuditEvent
12	Limits/cache/idempotency	DRF throttles, Redis explicit caching, in-house idempotency
13	API docs	drf-spectacular + openapi-typescript/openapi-fetch
14	Tests	pytest-django, factory_boy, hypothesis, schemathesis
15	Structure	Modular monolith, bounded-context apps
16	Ops	pydantic-settings, ruff, mypy+django-stubs, GitHub Actions, Sentry, structlog, Prometheus
17	Money	BigInteger centimes + currency code, in-house Money type, double-entry ledger

Not verified / open questions

- Airbnb calendar/availability architecture: no primary source found.
- Vrbo: no primary docs retrieved.
- Airbnb current cancellation tiers: only third-party sources.
- SMS OTP providers serving Algeria, pricing for Pusher/Ably, Cloudinary SDK version, django-celery-beat and import-linter versions: not checked.
- Algerian personal data law (Law 18-07) implications for hosting KYC files outside Algeria: needs legal review.
- Chargily amount units and payout (transfer to host) capabilities: confirm in their API reference.
- PostgreSQL FTS Arabic configuration availability: not verified.

BLADUS backend research: Algeria context, payments, compliance, infra, security

Research date / access date for every source below: **2026-11-04**.

Conventions used in this file:

- **[verified]** = read on an official or primary source page during this session.
- **[secondary]** = from a press article, agency blog or vendor page (not official text).
- **[unverified]** = could not confirm, conflicting, or inferred. Treat as a lead only.
- Every legal, tax and financial point is flagged **VERIFY WITH A PROFESSIONAL (not legal advice)**. This file is engineering research, not legal, tax or financial advice.

Executive summary of risks and surprises

1. **Law 18-05 art. 8 requires an e-commerce site to be "hosted in Algeria" with a `.com.dz` extension**, plus RC registration and CNRC e-supplier filing. Text read directly in JO n° 28 of 16 May 2018 [verified]. This directly conflicts with an "EU VPS only" plan. VERIFY WITH A PROFESSIONAL.
2. **Law 18-07 was amended by Law 25-11 of 24 July 2025** (JO n° 48). Transfers of personal data abroad need ANPDP authorization; illicit transfer abroad carries 1 to 5 years prison and 500,000 to 1,000,000 DA fine [secondary]. Hosting user data in the EU is a transfer abroad. VERIFY WITH A PROFESSIONAL.
3. **Marketplace payouts to hosts are a regulated activity risk.** Collecting guest money and paying it out to third parties looks like a payment service; Banque d'Algérie Règlement 25-02 (PSP licence, minimum capital 160 million DA) now frames this [secondary]. Practical route: each host gets paid through their own gateway account, or BLADUS invoices only its commission. VERIFY WITH A PROFESSIONAL.
4. **Online card acceptance is DZD only and domestic (CIB / Edahabia) for most merchants.** Since a Banque d'Algérie update of 14 May 2026, banks may let e-merchants accept foreign Visa/Mastercard/Amex [secondary]; BDL already offers it at 5% per transaction + 65,000 DA/month + 70,000 DA setup, legal entities only [verified on bdl.dz].
5. **The country now has 69 wilayas and 1,541 communes** (Loi 26-06, JO n° 25 of 5 April 2026, per datasets quoting the JO) [secondary, JO itself not opened]. Codes 59 to 69 differ between sources.
6. **Public Nominatim forbids autocomplete** and caps at 1 req/s for the whole app [verified].
7. **SMS OTP via Twilio costs about USD 0.27 per SMS to Algeria** and needs sender ID registration [verified on twilio.com]. Local routes are much cheaper but less documented.
8. **Hetzner raised cloud prices twice in 2026** (April and June); old cheap plans are not orderable [secondary].

C1. Payments in Algeria

C1.1 The rails

Rail	Operator	What it is	Notes
CIB card	SATIM (interbank switch) + banks	Domestic interbank debit card	Online acceptance through SATIM e-payment gateway. Merchant needs GIE Monétique authorization + bank contract [verified, cibweb.dz / satim.dz].
Edahabia card	Algérie Poste (CCP accounts)	Postal debit card	Also accepted via SATIM gateway on e-commerce sites [verified, cibweb.dz].
BaridiMob / CCP-to-CCP transfer (RIP)	Algérie Poste	App transfer between CCP accounts	MPT FAQ gives 1,000 DZD min and 200,000 DZD/day max [secondary, older]; Algerie360 says min 500 DZD [secondary]. Fees conflicting (baridimob.com says fee by amount; others say free) [unverified].
BaridiPay (QR)	Algérie Poste	Merchant QR payment in BaridiMob, since 14 June 2025, min 100 DA	Needs a commercial CCP account [secondary]. No public API found [unverified].
Bank transfer (RIB)	Banks via ACI / interbank	Virement	Manual reconciliation; no public instant-payment API found [unverified].
Cash on arrival / COD	n/a	Very common in Algerian e-commerce	Yalidine COD capped at 150,000 DZD per parcel [secondary].
Foreign Visa/Mastercard	Acquiring banks (e.g. BDL) via SATIM	Since 14 May 2026 update of BdA list of authorized products, banks may offer it after a declaration + 15 days [secondary]	BDL "E-commerce international": legal entities domiciled at BDL; 5% per transaction, pack 65,000 DA HT/month, onboarding 70,000 DA HT; tourism, travel and vehicle rental listed as priority sectors [verified, bdl.dz].

Legal anchor (Law 18-05, JO n° 28, 2018) [verified, text read]:

- Art. 27: payment by authorized means, remote or on delivery; electronic payment goes through platforms "mises en place et exploitées exclusivement par les banques agréées par la Banque d'Algérie et Algérie Poste".
- Art. 28: connection to payment platform secured by electronic certification.
- Art. 29: payment platforms are under Banque d'Algérie control.
- Art. 7: proceeds of cross-border sales must land in the e-supplier's account domiciled in Algeria. VERIFY WITH A PROFESSIONAL.

C1.2 How a merchant gets direct CIB/Edahabia acceptance (GIE Monétique / SATIM)

From the official CIBWeb portal and bank pages [verified + secondary]:

1. Prerequisites: inscription au **registre du commerce** (or registre de l'artisanat et des métiers) and in the **CNRC national file of e-suppliers** (e-commerce activity code) [verified, cibweb.dz FAQ as quoted in search].
2. Apply on cibweb.dz with RC attached; GIE Monétique decides acceptability "en moins de 24h" [verified, cibweb.dz].
3. Two paths: with a **GIE-referenced payment module** (4 steps) or a **non-referenced module** (8 steps incl. API retrieval, test slot, certification appointment, PV of certification) [verified, cibweb.dz pages p21/p22].
4. Sign adhesion contract with your bank; SATIM issues the web-merchant ID [secondary, Al Baraka Bank].
5. Adhesion is for one year, renewable [secondary, BNA].
6. Fees: no official grid found. Agencies cite 1% to 2.5% per transaction, sometimes about 50 DA fixed + 1.2%; settlement typically D+1/D+2; agency-reported ceilings 100,000 DA/transaction and 500,000 DA/month [secondary, conflicting, unverified]. Negotiated with the bank.

Referenced modules on cibweb.dz (page 1 of 3): Guiddini (NC/04/000013/2021), EBLINK, ATA CORP, ADEX Technology (ADEXEPAY), CONEX (ALPAY), Smart Solution and Innovation (Mpay), LRD Algérie (MYBUSI), shipili store, Web Rocket (WooCommerce), TROIS S PAY (3SPAY) [verified]. These are certified **integration modules**: the merchant still needs its own RC + bank contract.

C1.3 Aggregators / gateways comparison

Provider	Exists?	Model	Fees	Onboarding	Methods	Payout / settlement	Limits	Refunds	Webhooks	Docs
Chargily Pay V2	Yes [verified]	Aggregator (merchant uses Chargily's acceptance)	Starter 0% (cap 300 tx or 300,000 DZD); Comfort 1.5% (min 15 / max 1,500 DZD); Supreme 3% (30/3,000); Elite 4.5% (45/4,500). First month 0%. Fees can be on merchant, customer or split [verified, chargily.com pricing + dev docs]	Two-step verification (user account + application) by Chargily team; required documents not published [verified]; whether RC/auto-entrepreneur is required: **unverified**	`edahabia`, `cib`, `chargily_app` [verified]; Visa/MC "coming soon" on marketing page [secondary]	Balance hold 72h / 48h / 24h by plan; 1, 2 or 4 free payouts/month; extra payout 1.5% (min 150 DZD) [verified]	Starter capped as above; others unlimited [verified]	Yes, 2% commission (min 20, max 2,000 DZD) [verified]	Yes, "Webhooks" page in docs [verified exists; signature details not read]	https://dev.chargily.com/pay-v2 ; SDKs JS, PHP, Go, C#, WordPress, Shopify, Odoo on GitHub [verified]
SlickPay	Yes [verified]	Aggregator / fintech, Algiers, founded 2022 [secondary]	Auto 2% (instant after SATIM confirm), Daily 1.9%, Weekly 1.6%, Bimonthly 1.5%, Monthly 1.4%, Enterprise custom [verified, slick-pay.com]	Search summary says merchant needs RC, business bank account, SATIM / GIE authorization [secondary, unverified]	CIB, Edahabia; payment links; app [verified]	Per schedule above [verified]	Not published	Not found [unverified]	Not confirmed (docs site did not render) [unverified]	https://developers.slick-pay.com/ ; prod base `https://prodapi.slick-pay.com/api/v2` [secondary] ; WordPress plugin [verified]
Guiddini Pay	Yes [verified, cibweb registry]	Certified SATIM module + API ("Api paiement en ligne Satim-gateway")	Not published [unverified]	Merchant still needs its own RC + bank/SATIM contract (module path) [inferred from cibweb process]	CIB, Edahabia	Bank settlement	n/a	n/a	n/a	https://guiddini.dz/docs/api [secondary]
Yalidine Pay	**No evidence it exists** [unverified]	Yalidine is a courier with COD ("recouvrement")	n/a	n/a	Cash on delivery	n/a	150,000 DZD/parcel COD [secondary]	n/a	n/a	n/a
BDL e-commerce international	Yes [verified]	Direct bank acquiring for foreign cards	5% per tx, 65,000 DA HT/month, 70,000 DA HT setup [verified]	Legal entity domiciled at BDL, business plan, web-merchant contract, NDA [verified]	Visa, Mastercard	Likely DZD (fees "converties en DZD") [unverified]	n/a	n/a	n/a	https://www.bdl.dz/ecommerce-international/
Other modules (ADEXEPAY, ALPAY, Mpay, 3SPAY...)	Listed as certified [verified]	Modules	Not researched							https://cibweb.dz/fr/modules-autorise

C1.4 Payout, escrow, refunds: realities for a marketplace

- **No public split-payment / connected-account product found** for any Algerian gateway (no Stripe Connect equivalent) [unverified: absence of evidence].
- Collecting funds for third parties and redistributing them is close to the definition of a payment service. **Règlement 25-02 of 14 April 2025** (JO n° 28) sets PSP licensing with **160 million DA minimum capital**; Instruction 06-2025 requires segregated ("cantonnement") customer-fund accounts [secondary, algerie-eco / El Watan; official list at bank-of-algeria.dz]. VERIFY WITH A PROFESSIONAL.
- **Realistic startup patterns** (engineering options, each needs legal review):
    1. **Commission-only + pay at property**: guest pays host directly (cash on arrival / BaridiMob to host's CCP); BLADUS invoices hosts a monthly commission. Lowest regulatory risk, weakest trust.
    2. **Deposit via BLADUS gateway = BLADUS service fee only** (Chargily/SlickPay), remainder paid to host on arrival.
    3. **Host-owned gateway accounts**: each pro host (hotel, agency) has its own Chargily/SlickPay/SATIM account; BLADUS creates checkouts with the host's keys. Not possible for individuals without verified accounts.
    4. **Full collection + payout (Airbnb model)**: needs a licensed PSP or bank partner. Treat as phase 2.
- **Escrow**: no consumer escrow product found [unverified]. "Hold until check-in + 24h" can only be emulated with gateway balance holds or BLADUS's own account (see risk above).
- **Refunds**: Chargily supports refunds (2% fee) [verified]. Plan refunds as partial amounts stored in a ledger; COD/cash refunds are manual.
- **Ledger**: build a double-entry ledger regardless (booking, fee, payout, refund, chargeback-like reversal), DZD integer amounts (centimes are not used in practice; store minor units anyway).

C1.5 Diaspora / tourists with foreign cards

- Options found: bank acquiring for foreign cards via BDL (legal entity, 5%) [verified]; other banks may follow the 14 May 2026 BdA update [secondary]. International gateways (Stripe, PayPal merchant) for an Algerian entity: **not supported / unverified**; using a foreign entity to collect for Algerian stays raises exchange-control questions (Law 18-05 art. 7 requires proceeds domiciled in Algeria) [verified text]. VERIFY WITH A PROFESSIONAL.

C2. Messaging and verification

SMS / OTP

Provider	Algeria price per SMS	Notes	Source
Twilio	USD 0.273 outbound; alphanumeric sender ID "Registration Required"	Unregistered traffic may be filtered by carriers	twilio.com/en-us/sms/pricing/dz ; support.twilio.com article 223133767 [verified]
Plivo	from USD 0.2077 (by network 0.21 to 0.32)	Network names outdated	plivo.com/sms/pricing/dz/ [secondary]
BulkGate	Djezzy EUR 0.076, Mobilis EUR 0.2412, Ooredoo EUR 0.1685	OTP API	bulkgate.com/en/pricing/sms/dz/algeria/ [secondary]
Messaggio	Djezzy EUR 0.12, Mobilis EUR 0.30, Ooredoo EUR 0.29	Also WhatsApp	messaggio.com [secondary]
Unimatrix	USD 0.16 flat		unimtx.com/sms/dz [secondary]
Infobip, Vonage	Quote only	Not published	[unverified]
Sobersys (local)	Not published; 30 free SMS trial	Claims direct links to Djezzy, Ooredoo, Mobilis; REST API	sobersys.com [secondary]
Direct operator bulk SMS (Djezzy/Mobilis/Ooredoo business)	Not found; one blog cites 5 to 8 DZD domestic	Usually requires company contract	[unverified]

Recommendation: abstract an `SmsProvider` interface, start with a local aggregator for cost, keep an international fallback (Twilio/BulkGate) for reliability, and prefer **WhatsApp or email OTP** where possible. Rate-limit OTP sends per phone, per IP and per device (SMS pumping fraud is real).

WhatsApp Business Platform

- Since **1 July 2025 Meta charges per delivered template message** (marketing, utility, authentication), priced by recipient country; 1,000 free service messages per business number per month noted by secondary sources; new rate cards 1 July 2026 and 1 October 2026 [secondary]. Algeria's exact authentication rate: **not found** (likely in a "Rest of Africa" style region bucket) [unverified]. Official source to check: https://developers.facebook.com/docs/whatsapp/pricing . BSPs (e.g. Twilio) add a markup [secondary].

Email

Provider	Free tier	Paid	Source
Resend	3,000/month, max 100/day, 1 domain	USD 20 for 50k; USD 35 for 100k	secondary (wpmailsmtp, dreamlit)
Amazon SES	3,000 msgs/month for 12 months (new accounts, changed recently)	USD 0.10 per 1,000	secondary (saaspricepulse, courier)
Brevo	300/day (marketing + transactional shared)	Starter from USD 9	brevo.com/bulk-email-service/ [verified claim of 300/day]
Use SPF, DKIM, DMARC on the BLADUS domain; if the main site must be `.com.dz`, send mail from that domain.			

Push

- Web Push (VAPID) and Firebase Cloud Messaging are free to send (general knowledge, **not re-verified in this session**). Note Algeria has periodic national internet restrictions (e.g. exam periods) [secondary, algeriatech.news], so do not rely on push for critical flows.

C3. Legal and compliance (ALL items: VERIFY WITH A PROFESSIONAL, not legal advice)

C3.1 Personal data: Law 18-07 as amended by Law 25-11

Point	Finding	Status
Base law	Loi n° 18-07 du 10 juin 2018; in force 10 August 2023	secondary (cna.dz, Gide)
Amendment	**Loi n° 25-11 du 24 juillet 2025**, JO n° 48 (2025); amends, does not replace 18-07	secondary (ACF, upgrowth, douane PDF title)
Authority	**ANPDP**, 15 rue Yahia Cherif Omar, Hydra, Alger; portal https://portail.anpdp.dz/	secondary (multiple)
Prior formality	Art. 12: prior **declaration** or **authorization** for any processing. Authorization regime if: transfer abroad, sensitive data, or file interconnection. Online filing then signed paper deposit at ANPDP by appointment. Receipt immediately or within 48h; review within 10 days	secondary (anpdp.dz pages as summarized in search)
Transfer abroad	Original art. 44: prior ANPDP authorization. 25-11 adds chapter 6 / art. 45 bis 13: adequacy assessment of destination, derogations if "necessary"	secondary
Penalty for illicit transfer abroad	1 to 5 years prison, 500,000 to 1,000,000 DA	secondary
DPO	Mandatory per one source; another says mainly public sector	**conflicting, unverified**
Breach notification	To ANPDP **within 5 days**; inform data subjects if high risk	secondary (two vendor blogs)
Foreign controller	Must appoint a representative in Algeria to deal with ANPDP	secondary

Point	Finding	Status
Data localization mandate in 18-07/25-11	Not found as an explicit rule; but transfer abroad needs authorization	unverified
Admin fine	up to 500,000 DA	secondary

Engineering consequences: processing register, DPIA for ID-document KYC (biometric photos, NIN), consent capture (Law 18-05 art. 26 also requires prior consent and data minimization) [verified text], breach runbook with a 5-day clock, data-subject request workflow.

C3.2 E-commerce Law 18-05 (JO n° 28, 16 May 2018) [verified, text read]

- **Art. 2** applies if any party is Algerian, resides in Algeria, is an Algerian legal person, or contract concluded/performed in Algeria.
- **Art. 3** bans online transactions on gambling, alcohol/tobacco, pharma, IP-infringing goods, prohibited goods, and **any good or service requiring an "acte authentique"** (notarial deed). Long-term leases needing a notarial act are therefore out of scope for online contracting; short stays: **verify**.
- **Art. 8** e-commerce requires RC (or craft register) registration **and a website hosted in Algeria with a ".com.dz" extension**.
- **Art. 9** national file of e-suppliers at CNRC.
- **Art. 10 to 13** mandatory electronic offer content and three-step order (review, confirm, contract).
- **Art. 25** e-supplier must keep transaction records and transmit them electronically to CNRC (modalities by regulation). Fine 20,000 to 200,000 DA [secondary].
- **Art. 31 to 33** opt-in required for marketing messages; free unsubscribe; burden of proof on the e-supplier.
- No general consumer "droit de rétractation" for services found in 18-05; art. 22/23 cover delivery delay and non-conformity of goods. Cancellation terms must be in the offer/contract (art. 11/13). Consumer protection Law 09-03 also applies [not researched in detail, unverified].

Hosting implication: if BLADUS is an "e-fournisseur" (or the platform is treated as one), the site must be hosted in Algeria on `.com.dz`. Whether an intermediary marketplace or an API backend abroad behind an Algerian-hosted front satisfies "hébergé en Algérie" is **not defined in sources found** [unverified]. A web agency claims the rule is specific to e-commerce, not general websites [secondary].

C3.3 Tourism, accommodation, agencies, vehicles

Topic	Finding	Status
Hotel classification	Décret exécutif n° 19-158 du 30 avril 2019 (definition, operation, classification of hotel establishments)	secondary
Rooms at the owner's home ("logement chez l'habitant")	Interministerial circular of **16 June 2012**: max 15 guests, max 5 rooms, insurance fire/theft/civil liability, owner greets guests, furnished + hygiene/safety; **declaration at the APC** with copy of property title; APC may issue written authorization	secondary (Maghreb Emergent, 8 June 2026, citing FNAI)
Foreign guests	Must be declared to security services, art. 29 of Law 08-11 of 25 June 2008	secondary (search summary)
Whole-apartment short-term rental (Airbnb style)	**No specific framework found**; general contract law; announced ministry project (Minister Didouche, ~2024) not found published	secondary, unverified
Rental income tax	IRG 7% on rental income cited (G n°51 monthly, G n°1 annual)	secondary, unverified
Travel agencies	Loi 99-06 du 4 avril 1999; Décret exécutif 17-161 du 15 mai 2017; licence signed by the Minister of Tourism after national commission opinion and security inquiry; categories A (domestic/inbound) and B (outbound)	verified exists on mta.gov.dz "Devenir agent de tourisme" + secondary
Usurping travel-agent title	50,000 to 200,000 DA + 3 months to 3 years	secondary
Vehicle rental (without driver)	**No decree found** in this session; agencies' own T&Cs: civil liability insurance mandatory and included, driver usually 25+ with licence 2+ years	**unverified** (needs JORADP search; likely a regulated activity in the RC nomenclature)
Motor insurance	Civil liability motor insurance mandatory (general knowledge + agency T&Cs)	secondary

C3.4 Tax identifiers and rates

Item	Finding	Status
RC	Registre du commerce (CNRC, Sidjilcom portal)	secondary
NIF	Tax ID, required on invoices	secondary
NIS	Statistical ID (ONS), required on invoices	secondary
Article d'imposition	On invoices	secondary
Carte d'artisan	Registre de l'artisanat et des métiers; accepted alternative to RC by art. 8 of 18-05	verified (text)
Auto-entrepreneur	Loi 22-23 du 18 décembre 2022; card from ANAE (anae.dz); turnover cap 5,000,000 DA/year (LF 2023 art. 51); IFU 0.5% of turnover (LF 2024 art. 18); minimum 10,000 DA/year; outside VAT; liberal, regulated and artisanal activities excluded; activity list exists but not retrieved	secondary, VERIFY
Can an auto-entrepreneur sell online / be a host?	RC/craft register is required by 18-05 art. 8; whether the ANAE card satisfies this is **unverified**	unverified
NIN	18-digit national identification number (Décret 10-210 of 2010; new decree in JO n° 60, 2023); different from the CNI card number	secondary
VAT standard	19%	PwC tax summaries [verified]
VAT reduced	9% (art. 23 CTCA list)	PwC [verified]
VAT tourism	**9% for tourism, hotel, thermal, classified restaurants, travel and tourist vehicle rental until 31 Dec 2027** (art. 42 LFC 2009 as amended by art. 215 LF 2025)	secondary (ACF, Deloitte)
Taxe de séjour	Per person per day (CIDTA art. 266 bis to 266 octies since LF 2022): 5* 600 DA, 4* 500, 3* 300, 2* 200, 1* 100. PLF 2027 proposes 50 DA for unclassified hotels and other accommodation (draft only)	secondary (visa-algerie.com, CNCC circular)
Invoices	Décret exécutif 05-468 du 10 décembre 2005 (under Law 04-02): seller identity, legal form, address, RC, NIF, NIS, article d'imposition, sequential number, date, description, HT, VAT rate and amount, TTC; cancelled invoices kept and marked	secondary
Retention of accounting docs	10 years commonly cited (Code de commerce; article number conflicting 12 vs 18)	secondary, unverified article

C3.5 Documents to collect per host type (proposal, VERIFY WITH A PROFESSIONAL)

Host type	Identity	Business / legal	Property / asset	Payout	Suggested retention
Individual (rooms / home)	CNI biométrique or passport (front/back), selfie liveness; NIN only if legally needed	Optional: auto-entrepreneur card (ANAE) if applicable; APC declaration receipt for "chez l'habitant"	Proof of ownership (acte) or lease + owner authorization; insurance certificate (fire/theft/civil liability)	CCP (RIP) or bank RIB in host's own name	ID images: delete or redact after verification + short window (e.g. 90 days) unless law requires; verification result + audit log: duration of account + 10 years if tied to invoices
Travel agency	Legal representative ID	RC, NIF, NIS, **Ministry of Tourism licence** (category A/B)	Office address	Company bank RIB	Contract + licence copies: relationship + 10 years
Hotel	Manager ID	RC, NIF, NIS, classification decision (Décret 19-158 regime), operating authorization	Address, star rating (drives taxe de séjour)	Company RIB	Relationship + 10 years
Hostel / auberge	Manager ID	RC or association/public status, operating authorization (regime **unverified**)	Address	RIB	Relationship + 10 years
Vehicle owner / rental agency	ID + driving licence of operator	RC with rental activity code (regime **unverified**), NIF, NIS	Carte grise, valid insurance (civil liability mandatory), contrôle technique	RIB	Relationship + 10 years; rental contracts per agency obligations
Experience guide	ID	Auto-entrepreneur card or RC; guide accreditation if any (**unverified**)	Activity insurance (civil liability) recommended	CCP/RIB	Relationship + 10 years

Retention rules of thumb: invoices/accounting 10 years [secondary]; transaction records to CNRC (18-05 art. 25, no duration in law) [verified text]; personal data no longer than needed for purpose (18-07 principle) [secondary]. Store KYC files encrypted, separate bucket, short-lived signed URLs, access logged.

C4. Geography data

Official count

- **69 wilayas, 1,541 communes**: Loi n° 26-06 (signed 4 April 2026, JO n° 25 of 5 April 2026) supplementing Loi 84-09 with art. 52 bis 10 to 52 bis 20; announced at the Council of Ministers of 16 November 2025 [secondary: GitHub datasets quoting JO text; Morocco World News; DzairTube]. JO not opened directly: **check https://www.joradp.dz/**.
- 11 new wilayas and commune counts quoted from JO text: Aflou 12, Barika 8, El Kantara 5, Bir El Ater 4, El Aricha 4, Ksar Chellala 6, Aïn Oussera 10, Messaad 8, Ksar El Boukhari 21, Bou Saada 23, El Abiodh Sidi Cheikh 7 (total 108) [secondary].
- **Codes 59 to 69: inconsistent across sources** [unverified]; use ONS codes when published.
- **Dairas**: 546 (58-wilaya era, othmanus) vs 591 (el-amin dataset, 2026) [conflicting, unverified].

Datasets

Dataset	Coverage	Fields	License	Source
el-amin-dev/algeria-wilayas-communes	69 / 591 dairas / 1,541	ar/fr/en names, postal codes (27 null)	CC0 1.0	https://github.com/el-amin-dev/algeria-wilayas-communes [verified]
ihahachi/Algeria-Cities	69 / 1,541	ar/fr, coordinates; CSV/JSON/GeoJSON/SQL	MIT	https://github.com/ihahachi/Algeria-Cities [secondary]
S450R1/algeria-cities-2025	69 / 1,541	ar/latin; SQL/JSON/XML	Public domain	https://github.com/S450R1/algeria-cities-2025 [secondary]
othmanus/algeria-cities	58 / 546 / 1,541 / 3,940 postcodes	ar/fr, dairas codes	MIT (per mirror)	https://github.com/othmanus/algeria-cities [secondary, outdated]
kossa/algerian-cities	58 + lat/lng	Laravel	MIT	https://github.com/kossa/algerian-cities [secondary]
OpenStreetMap	boundaries, POIs		ODbL (share-alike)	https://www.openstreetmap.org
GeoNames	places + admin codes		CC BY 4.0 (general knowledge, not re-verified)	https://www.geonames.org
data.gov.dz / ONS	official codes		not checked	[unverified]

Recommendation: seed from el-amin (CC0) for names, merge coordinates from ihahachi (MIT, keep notice), keep a `source` and `valid_from` column so the next reform is a migration, and store both legacy 58-wilaya codes and new codes.

Geocoding / maps

Service	Terms	Fit
Nominatim (public)	Max 1 req/s for the whole app, valid User-Agent/Referer, cache results, **autocomplete forbidden**, ODbL attribution	Backend reverse-geocode only, cached [verified, operations.osmfoundation.org/policies/nominatim/]
Photon (photon.komoot.io)	"please be fair, extensive usage will be throttled", no SLA; Apache 2.0 code, OSM data	Autocomplete prototype; self-host for production [secondary]
OpenFreeMap	Free, no limits, no keys, commercial use allowed, OSM attribution required	Vector tiles for MapLibre [verified, openfreemap.org]
Mapbox / Google Maps	Paid per request; Algeria address quality not assessed	[unverified]
Algerian addresses are often informal; geocode to commune + manual pin on map, not street-level text.		

C5. Hosting and infrastructure

Connectivity

- Submarine cables: Medex (Annaba to Marseille branch), SeaMeWe-4 (Annaba to Marseille), ORVAL/ALVAL (Algiers/Oran to Valencia, 40 Tbps, live 31 Dec 2020), plus Medusa landing in Algiers and Collo (2026) [secondary].
- **Measured latency Algeria to Paris/Frankfurt/Marseille/Milan: not found** [unverified]. Secondary article context suggests tens of ms (40 to 80 ms discussed) [secondary]. Measure with RIPE Atlas probes in AS36947 (Algérie Télécom) before deciding.
- Government internet restrictions during exam periods disabled Cloudflare Algerian PoPs in June 2025 and rerouted via Europe (+30 to 70 ms) [secondary].

Algerian hosting

Provider	Offer	Price	Source
ICOSNET	DCs in Algiers and Oran; VPS, public cloud instances, VDC	Cloud instance from 2,800 DZD/month or 4.5 DZD/hour; VPS from 2,800 DZD HT/month	icosnet.com.dz [secondary/vendor]
Djezzy Cloud	Sovereign cloud IaaS/PaaS, ARPCE accredited, DC in Béjaïa	Not published	datacenters.com [secondary]
Algérie Télécom	Data centers	Not found	[unverified]
Others listed	Hostarts (Birtouta), AYRADE (Zeralda)	n/a	datacenters.com [secondary]

CDN

- Cloudflare has PoPs listed for Algiers (ALG), Oran (ORN), Constantine (CZL), Annaba (AAE); status varies between "operational" and "re-routed"; one tracker says ALG does not serve traffic [conflicting, check https://www.cloudflarestatus.com/locations].

Recommended setups (costs rough, VERIFY current prices)

Because of 18-05 art. 8 and 18-07 transfers, the default should be **production data in Algeria**, with EU only for non-personal assets or encrypted offsite backups subject to ANPDP authorization.

Stage	Setup	Rough monthly cost
Starter (compliant-leaning)	1 to 2 ICOSNET (or Djezzy Cloud) VMs: Docker Compose (API, Postgres, Redis, worker, Caddy), daily `pg_dump` + WAL archiving to a second Algerian VM/DC (Oran vs Algiers), Cloudflare in front for static assets only	~6,000 to 20,000 DZD (from 2,800 DZD per VPS; sizes unverified)
Starter (EU, legal risk)	Hetzner CX23 / CX33 class: CX23 EUR 5.49 after June 2026, CX33 EUR 8.49 [secondary]; CPX22 jumped to EUR 19.49 for new orders	EUR 10 to 40
Scale-up	2+ app nodes behind LB, managed or self-run Postgres primary + replica in a second DC, object storage for photos, separate KYC bucket, observability stack	Algeria: quote-based; EU: EUR 100 to 400 (OVH/Scaleway/DigitalOcean managed Postgres prices **not verified** this session)
Backups: 3-2-1 rule, encrypted, restore test monthly, PITR for Postgres; ledger tables append-only.		

C6. Languages and culture

Topic	Finding	Status
Languages	Arabic (official), Tamazight (official), French widely used in business, English growing	general knowledge, not re-verified
RTL	Arabic UI is RTL; use logical CSS properties and `dir` per locale	n/a
Digits	Maghreb (incl. Algeria) uses Western digits 0-9 in everyday Arabic text; Eastern Arabic-Indic mostly Mashreq	secondary (Wikipedia + language sites). For `Intl`, use `ar-DZ` with `numberingSystem: "latn"` to force Latin digits
Weekend	Friday and Saturday since 2009 (work week Sunday to Thursday)	secondary (The National, OfficeHolidays, Lloyds)
Phone	+213; mobiles 9 digits after country code starting 5, 6, 7 (since ARPCE plan of 22 Feb 2008); historical mapping 5 Ooredoo, 6 Mobilis, 7 Djezzy but number portability means prefix does not prove carrier; landlines 8 digits	verified (arpce.dz PDF title), secondary (Wikipedia). Safe regex `^\+213[5-7]\d{8}$`
National ID	CNIBE (biometric electronic ID card, chip, photo + fingerprints); NIN 18 digits (Décret 10-210), distinct from card number	secondary (interieur.gov.dz, mfa.gov.dz pages exist)
Passport	Biometric passport, used to request CNIBE online	secondary
Names	Arabic and Latin transliterations differ across documents; store `name_ar`, `name_latin` separately, family name often written first on official docs	[unverified]
Time zone	Africa/Algiers, UTC+1, no DST	general knowledge (IANA tzdb), not re-verified
Currency	DZD, no minor unit used in practice	general knowledge

D. Security and trust

D1. OWASP API Security Top 10 (2023) mapped to BLADUS

Official list: https://owasp.org/API-Security/editions/2023/en/0x11-t10/ [verified].

OWASP	BLADUS example	Control
API1 BOLA	`GET /bookings/{id}`, `/wallets/{id}`, `/payouts/{id}`, `/kyc/{fileId}`, `/conversations/{id}`	Authorize on every object: guest owns booking OR host owns listing OR admin; use non-guessable IDs (UUIDv7/ULID) but never rely on them
API2 Broken Authentication	OTP brute force, SMS pumping, session fixation	OTP 6 digits, 5 attempts, 10 min TTL, per-phone/IP/device rate limits, refresh-token rotation, MFA for hosts and admins
API3 Broken Object Property Level Authorization	Guest sees host phone/address before confirmation; mass-assignment of `status`, `price`, `commission_rate`, `is_verified`	Response DTOs per role; allowlist writable fields
API4 Unrestricted Resource Consumption	Search scraping, image upload size, SMS cost	Pagination caps, upload limits, per-user quotas, SMS budget alarms
API5 Broken Function Level Authorization	Host calling admin `/approve-listing` or `/refund`	RBAC middleware + deny-by-default routes, separate admin API
API6 Sensitive Business Flows	Calendar hoarding with unpaid holds, coupon abuse, review farming, mass account creation	Hold TTL, deposit before hold, device fingerprint, velocity limits
API7 SSRF	iCal import URL, image-by-URL upload, webhook URLs	Allowlist schemes, block private IP ranges, fetch via egress proxy
API8 Security Misconfiguration	Open Postgres/Redis, debug endpoints, CORS `*`	Hardening checklist, CIS baseline, secrets in env/secret store
API9 Improper Inventory Management	Old `/v1` endpoints, staging with prod data	OpenAPI inventory, version sunset, no prod data in staging
API10 Unsafe Consumption of APIs	Trusting gateway redirects instead of webhooks; unsigned webhooks	Verify webhook signatures (Chargily provides webhooks), re-fetch checkout status server-side, idempotency keys

D2. Fraud and fake-listing patterns

- **Off-platform payment pushes**: fake host asks for transfer outside the platform for a "discount"; scammers prefer wire, gift cards, crypto [verified, consumer.ftc.gov/articles/rental-listing-scams; Airbnb off-platform policy via secondary]. In Algeria the equivalent is BaridiMob/CCP transfers and "versement" deposits: detect RIP/CCP numbers, phone numbers and WhatsApp links in messages before booking confirmation.
- **Hijacked / fake listings** with stolen or AI-generated photos [verified FTC]: perceptual-hash photos against other listings, reverse-image checks, require property proof for first listing, first-payout delay.
- **Too-low prices, new profiles** [secondary]: risk scoring.
- **Triangulation / stolen cards**: less relevant while domestic cards require OTP (3-D Secure style SMS), but relevant once foreign cards are accepted.
- **Host-side fraud**: duplicate accounts after bans (device + phone + NIN hash), fake cancellations to rebook off-platform.

D3. Review manipulation

- Only allow reviews tied to a completed booking; double-blind publishing (both sides or after 14 days); detect review rings (same device/IP/payment instrument), sudden review bursts; keep edit history.

D4. Account takeover

- OTP-based login is SIM-swap exposed: require re-verification for payout detail changes, notify old channels, hold payouts 48 to 72 h after payout account change; admin accounts with TOTP/WebAuthn.

D5. Data subject rights (18-07)

- Access, rectification, opposition, deletion workflow with identity check; export as JSON/PDF; deletion = anonymize users but retain booking/invoice records for legal retention (10 years for accounting [secondary]). Track each request in an auditable table with deadlines. Exact legal response deadlines: **unverified**.

D6. Backup, restore, audit trail

- Postgres: nightly logical dump + continuous WAL archiving (PITR), encrypted (age/GPG), stored in a second location (second Algerian DC preferred; offsite abroad only after ANPDP transfer analysis).
- Monthly restore drill, documented RPO/RTO.
- Append-only `audit_log` (actor, action, object, before/after hash, IP, user agent, request id), hash-chained for tamper evidence; admin actions, payouts, refunds, KYC views always logged.
- Incident register + 5-day ANPDP breach notification runbook [secondary].

Sources (all accessed 2026-11-04)

Payments

- https://www.cibweb.dz/fr/home ; https://cibweb.dz/fr/modules-autorise ; https://cibweb.dz/fr/page/processus-dautorisation-avec-un-module-de-paiement-reference-gie-monetique-p21 ; https://cibweb.dz/fr/page/processus-dautorisation-avec-un-module-de-paiement-non-reference-gie-monetique-p22
- https://www.satim.dz/index.php/fr/services-cib/paiement-en-ligne
- https://www.bna.dz/professionnel-integrer-le-paiement-en-ligne/
- https://www.albaraka-bank.dz/web-marchand-integrez-a-votre-site-de-commerce-le-paiement-en-ligne-rapide-et-sur-via-les-cartes-cib-ou-dhahabia-et-suivez-notre-guide-pratique/
- https://web-rocket.dz/activer-paiement-en-ligne-cibweb-procedure/
- https://dev.chargily.com/pay-v2/fees ; https://chargily.com/business/pay/pricing ; https://dev.chargily.com/pay-v2/account-verification ; https://dev.chargily.com/pay-v2/api-reference/checkouts/create ; https://chargily.com/dz/business/pay/help?lang=fr ; https://github.com/chargily
- https://www.slick-pay.com/ ; https://developers.slick-pay.com/ ; https://wordpress.org/plugins/slickpay-payment-gateway/ ; https://amarneche.me/blog/accept-online-payments-algeria
- https://guiddini.dz/docs/api/satim ; https://guiddini.com.dz/e-paiement-en-algerie/
- https://www.bdl.dz/ecommerce-international/

- https://maghrebemergent.news/fr/algerie-les-e-commercants-pourront-encaisser-des-cartes-bancaires-etrangeres/
- https://algerie-eco.com/2026/10/28/la-satim-certifiee-pour-personnaliser-les-cartes-visa-et-mastercard-en-algerie/
- https://www.algerie360.com/algerie-poste-quelles-sont-conditions-pour-transferer-de-largent-via-lapplication-baridimob/ ; https://algerie-eco.com/2025/08/25/algerie-poste-fixe-le-minimum-de-paiement-via-baridpay-a-100-da/ ; https://www.mpt.gov.dz/faq-electronic-banking-services/ ; https://www.baridimob.com/
- https://dz-ecom.com/en/blog/integrer-paiement-cib-satim-site-ecommerce-algerie ; https://webminds.dz/blog/paiement-en-ligne-algerie-cib-dahabia.html ; https://symloop.com/blog/ecommerce-cib-edahabia-algerie/
- https://www.storeino.com/fr/articles/yalidine-express-storeino
- https://algerie-eco.com/2025/05/14/banque-dalgerie-nouveau-cadre-reglementaire-pour-les-prestataires-de-services-de-paiement/ ; https://elwatan.dz/prestataires-de-services-de-paiement-la-banque-dalgerie-complete-le-dispositif-reglementaire/ ; https://www.bank-of-algeria.dz/agrement-instruction/

Messaging

- https://www.twilio.com/en-us/sms/pricing/dz ; https://support.twilio.com/hc/en-us/articles/223133767-International-Support-for-Alphanumeric-Sender-ID
- https://www.plivo.com/sms/pricing/dz/ ; https://www.bulkgate.com/en/pricing/sms/dz/algeria/ ; https://messaggio.com/bulk-sms/algeria/ ; https://www.unimtx.com/sms/dz ; https://sobersys.com/en/
- https://developers.facebook.com/docs/whatsapp/pricing ; https://sleekflow.io/blog/whatsapp-business-price
- https://wpmailsmtp.com/resend-review/ ; https://www.saaspricepulse.com/tools/amazon-ses ; https://www.brevo.com/bulk-email-service/ ; https://dreamlit.ai/blog/brevo-review

Legal / tax

- https://www.joradp.dz/FTP/JO-FRANCAIS/2018/F2018028.pdf (Law 18-05, text read)
- https://www.commerce.gov.dz/fr/actualites/la-loi-relative-au-commerce-electronique-fixant-les-regles-generales-du-commerce-electronique-des-biens-et-des-services ; https://legal-doctrine.com/en/edition/ecommerce-en-algerie-les-conditions-dexercice-des-activites-commerciales-en-ligne ; https://www.reporters.dz/ce-qu-il-faut-savoir-sur-la-loi-sur-le-commerce-electronique-hebergement-obligatoire-en-algerie/
- https://www.douane.gov.dz/IMG/pdf/loi_18-07_du_24_07_2025_fr_dz.pdf (title only; fetch failed) ; https://www.acf-dz.com/modification-de-la-loi-relative-a-la-protection-des-personnes-physiques-dans-le-traitement-des-donnees-a-caractere-personnel/ ; https://itsolutions.dz/loi-25-11-vs-18-07-algerie/ ; https://intervalle-technologies.com/blog/loi-25-11-donnees-personnelles-algerie-guide/ ; https://www.upgrowth.dz/journal-officiel/loi-18-07-protection-donnees-personnelles
- https://anpdp.dz/fr/quelles-sont-vos-obligations/ ; https://portail.anpdp.dz/ ; https://cna.dz/entree-en-vigueur-de-la-loi-sur-la-protection-des-donnees-a-caractere-personnel-en-algerie/14/08/2023/ ; https://www.gide.com/en/news-insights/setting-up-of-the-national-authority-for-the-protection-of-personal-data-anpdp/
- https://maghrebemergent.news/fr/location-chez-lhabitant-la-fnai-rappelle-lexistence-dun-cadre-reglementaire/ ; https://www.layyali.com/legislation-algerienne-location-saisonniere ; https://www.visa-algerie.com/chambres-dhotes-pour-touristes-en-algerie-lactivite-sera-reglementee/ ; https://zoomalgerie.com/location-airbnb-algerie-rentabilite-investissement-locatif-diaspora/
- https://www.mta.gov.dz/devenir-agent-de-tourisme/?lang=fr ; https://www.mta.gov.dz/wp-content/uploads/2022/06/recueil-texte-tourism-fr2022-.pdf ; https://www.algerie360.com/omra-2026-2027-agences-agreees-arnaques/
- https://algeria-rental.com/conditions-generales/ ; https://algerlocation.com/conditions-generales-de-location/
- https://www.upgrowth.dz/en/auto-entrepreneur-algeria ; https://www.algerie360.com/carte-auto-entrepreneur-anae-algerie/ ; https://www.village-justice.com/articles/les-avantages-statut-auto-entrepreneur-focus-sur-recente-loi-algerienne,49162.html
- https://taxsummaries.pwc.com/algeria ; https://www.pwc.co.za/en/publications/vat-in-africa/algeria-overview.html ; https://www.acf-dz.com/wp-content/uploads/2025/01/Mesure-phase-loi-de-finances-2025-ACF.pdf ; https://blog.avocats.deloitte.fr/algerie-les-principales-mesures-importantes-de-la-loi-de-finances-pour-2025/ ; https://lesenjeuxeco.dz/loi-de-finances-2026-les-principales-mesures-fiscales/
- https://www.visa-algerie.com/hotels-lalgerie-veut-instaurer-une-nouvelle-taxe-de-sejour/ ; https://www.cn-cncc.dz/admin/Upload/site/revue/revue-5bfc01759dece5d6c71e6bd2ee96e7b3.pdf
- https://www.dcwalger.dz/fr/index.php/component/content/category/9-haut-p ; https://lamacta.com/en/blog/decret-executif-05-468-facture-legale-algerie/
- https://algerie-eco.com/2023/09/22/les-dispositions-applicables-au-numero-didentification-national-definies-par-decret/ ; https://passeport.interieur.gov.dz/ ; https://clr.africanchildforum.org/Legislation%20Per%20Country/algeria/algeria_idnumber_2010_fr.pdf

Geography / maps

- https://github.com/islam-re/Algeria-wilayas/blob/main/JO%20Article%202026.md ; https://github.com/el-amin-dev/algeria-wilayas-communes ; https://github.com/ihahachi/Algeria-Cities ; https://github.com/S450R1/algeria-cities-2025 ; https://github.com/othmanus/algeria-cities ; https://github.com/kossa/algerian-cities
- https://www.moroccoworldnews.com/2025/11/268183/algeria-creates-11-new-wilayas-as-critics-question-political-motives/ ; https://www.dzair-tube.dz/en/algeria-sets-stage-for-new-territorial-order-69-provinces-over-1500-communes-enter-final-phase-of-realization/ ; https://en.wikipedia.org/wiki/Provinces_of_Algeria
- https://operations.osmfoundation.org/policies/nominatim/ ; https://github.com/komoot/photon/discussions/730 ; https://openfreemap.org/

Infra

- https://icosnet.com.dz/en/web-hosting/ ; https://icosnet.com.dz/cloud-link/ ; https://www.datacenters.com/providers/djezzy-cloud ; https://www.datacenters.com/locations/algeria
- https://www.cloudflarestatus.com/locations ; https://mining.cloudflare.manfredi.io/pops/alg/ ; https://algeriatech.news/dns-cdn-web-performance-algeria/ ; https://algeriatech.news/algeria-medusa-cable-2026-bandwidth-enterprise-cloud-latency-guide-2026/ ; https://www.samenacouncil.org/samena_daily_news?news=76897
- https://northflank.com/blog/hetzner-cloud-server-price-increases ; https://www.bitdoze.com/hetzner-cloud-cost-optimized-plans/

Culture

- https://www.arpce.dz/fr/file/u1h4y0 ; https://en.wikipedia.org/wiki/Telephone_numbers_in_Algeria
- https://www.thenationalnews.com/mena/2021/12/07/when-is-the-weekend-in-the-arab-world/ ; https://www.officeholidays.com/countries/algeria/2026
- https://en.wikipedia.org/wiki/Arabic_numerals ; https://en.wikipedia.org/wiki/Modern_Arabic_mathematical_notation

Security

- https://owasp.org/API-Security/editions/2023/en/0x11-t10/ ; https://api-security.owasp.org/editions/2023/en/0x00-toc/
- https://consumer.ftc.gov/articles/rental-listing-scams ; https://guard.io/blog/airbnb-vacation-rental-scams-fake-listing

Backend blueprint changelog

All notable changes to the backend documentation package. Newest first.

2026-11-09

- Product rules added since the schema pause: DEC-014 (what each host type may publish) and DEC-015 (8 experience types, 8 trip types). Requirements now 147 (LST-12, TRP-07, HST-07), rules BR-LST-05 and BR-HST-05, enums `experience_type` and `trip_type` in the schema.
- Schema now 77 tables: `incident_record`, `calendar_feed`, `otp_send_log` (closes decision T5); profile fields on `user` (bio, spoken languages, home commune, private address) and `trip.slug` (gaps found while writing the API). `schema.sql` added (PostgreSQL rendering of `schema.dbml`).
- OpenAPI generator copied into `_tools/openapi_gen/`; permission name aligned to `privacy.handle_requests`; trip slug and profile fields added; lint still 0 errors, 0 warnings.
- `traceability.csv` now takes its API column from `openapi.yaml` (`x-requirements`); 127 requirements map to operations, 20 are jobs, quality targets or R2 items without endpoints.
- Decisions: L13 to L16 (from 08) and T11 (new staff permissions) added to `13-decisions-needed.md`; total 49, 48 open.
- `diagrams/03-django-apps` redrawn in layers L0 to L6 with `search` and `core`, matching 07.
- Plain-language layer: `00-START-HERE.md`, `explain-like-im-new.md`, `README.md`, glossary extended to 63 terms.
- `15-readiness-and-first-build-prompt.md`: verdict GO for M0, gates before live data, and the first build prompt.
- `dist/`: browsable HTML site and one PDF with everything, built by `_tools/build_dist.mjs`.
- Phase 8 done: `11-roadmap.md` (assumptions, mock-to-API switch method, milestones M0 to M7 plus buffer to launch, R2-A, R2-B and LATER, each with scope by requirement ID, exit criteria, dependencies, weeks, `lib/data` switches and legal/accounting gates; Mermaid gantt chart).
- Phase 8: `12-risks.md` (32 risks K-01 to K-32 with likelihood, impact, score, owner role, mitigation, early signal and links; heat map).
- Phase 8: `13-decisions-needed.md` (DEC-001 to DEC-015 for reference; 44 open decisions: product owner Q14 to Q28, lawyer L1 to L12, accountant/bank F1 to F7, tech lead T1 to T10, each with options, recommendation, blocking milestone and default).
- Phase 8: `14-open-questions.md` status note; Q14 to Q20, L1 to L7, F1 to F4 marked as moved to `13-decisions-needed.md`; new questions Q21 to Q28, L8 to L12, F5 to F7.
- Phase 8: `traceability.csv` and `traceability.md` (all 147 requirements linked to rules, tables, API area, test file area, milestone and `lib/data` module), generated by `_tools/build_traceability.py`, which also checks that every cited rule exists and that each milestone matches the requirement's release.
- Phase 7: `07-architecture.md` (modular monolith with 17 apps + `core` kernel owning all 77 tables, layered dependencies checked by import-linter, services/selectors, transaction and lock order rules, outbox, 31 Celery jobs with schedules and rule IDs, caching, search, upload pipeline, i18n, BFF and typed client, module-by-module migration from mock `lib/data` behind server-only flags, future repo tree, ADR-015 to ADR-026).
- Phase 7: `08-security-privacy.md` (assets, actors, STRIDE table, OWASP API Top 10 mapped to endpoints, sessions and cookie flags, OTP and lockout, staff MFA, staff and organisation permission matrices, secrets and key inventory, field encryption and ID hashing, file security, rate limits incl. `otp_send_log`, hash-chained audit, fraud controls, Law 18-07/25-11 register of processing, DPIA outline, retention table, 5-day breach runbook on `incident_record`, data residency, pre-launch checklist, lawyer items incl. new L13 to L16).
- Phase 7: `09-devops-hosting.md` (dev/ci/staging/production, Algeria-first hosting with a second Algerian site, Docker Compose container layout, `.com.dz` domains, TLS, CDN policy, GitHub Actions pipelines incl. test-mode build guard, migrations policy, pgBackRest PITR, RPO 15 min / RTO 4 h, monthly restore drill, observability and alerts, rough cost in DZD and EUR, deploy/rollback/incident/restore runbooks, env var list).
- Phase 7: `10-testing-quality.md` (test pyramid, tests per app, hypothesis properties for money, refunds and date ranges, concurrency tests on real PostgreSQL, schemathesis and OpenAPI diff contract tests, migration tests, k6 scenarios against NFR-01/02, i18n checks for API messages, factories, Definition of Done, coverage targets, CI quality gates, BR rule traceability with `test_br_<area>_<nn>_...` naming).
- Phase 6: `openapi.yaml` (OpenAPI 3.1 contract: 266 operations on 220 paths under `/api/v1`, tags for auth, public, me, host, admin and webhooks; Money, Problem, cursor and page schemas; enums copied from `schema.dbml`; every operation has `x-requirements`, `x-rules`, `x-release`; 8 dormant M2 to M4 operations with `x-dormant`; Redocly lint: 0 errors, 0 warnings).
- Phase 6: `06-api-design.md` (conventions, BFF and allauth session model, staff MFA, active host account and permissions, error code catalogue, rate limits, idempotency and booking race, webhooks, versioning, Next.js consumption with openapi-typescript and openapi-fetch, every website and admin `lib/data` function and server action mapped to an endpoint or marked removed, five worked examples).

2026-11-04

- Started the BLADUS (formerly DARNA) backend discovery and blueprint. Target stack: Django + PostgreSQL. Documentation only, no backend code.
- Recovered the original specification documents from git history into `_sources/`: `darna_spec.md`, `clean_user_spec.md`, `usability-report.md`, `admin-usability-report.md` (they were deleted from the working tree in earlier commits).
- Phase 1 started: inventory of the website and admin apps, spec-versus-code comparison, live click-through of the main guest and host flows (`_work/click-through.md`).
- Phase 2 started: research on comparable platforms, the Django ecosystem, Algeria (payments, messaging, legal, geography, hosting, culture) and security.
- Phase 1 done: `01-system-inventory.md` (features per actor, data, `lib/data` bridge, 25 business rules, click-through, gaps), with raw appendices A to E.
- Phase 2 done: `02-research.md` (sections A to E, decisions R-1 to R-11, verification flags).
- Added `glossary.md` and `14-open-questions.md` (13 product questions, 7 legal, 4 finance).
- Paused for product-owner feedback before Phase 3, as agreed.
- Product owner accepted all defaults (Q1 to Q13), recorded as DEC-001 to DEC-013.
- Phase 3 done: `03-requirements.md` + `requirements.csv` (144 requirements: 123 MVP, 19 R2, 2 Later; generated from `_tools/requirements_data.py`), `04-business-rules.md` (rules by area, 4 state machines, worked money examples, ledger postings). Every rule ID cited by a requirement exists.
- Phase 4 done: 29 diagrams in `diagrams/` (context, containers, Django apps, 4 state machines, 5 sequences, money flow, document map, 15 ERDs), each with caption and legend in `diagrams/README.md`; rendered to SVG and PNG in `diagrams/rendered/`.
- Phase 5 done: `schema.dbml` (74 tables, 33 enums, 152 foreign keys; validated with the official DBML CLI), `05-database-schema.md` (14 ADRs, SQL constraints, query plan, mapping, seed plan), `05b-data-dictionary.md` (generated). Added questions Q14 to Q20.
- Paused for product-owner feedback on the schema, as agreed.